

ChArUco Board を使ったカメラキャリブレーション 0.0.2

2026年09月18日(金) 08時33分06秒作成 - ChArUco Board を使ったカメラキャリブレーション / 構成: Doxygen 1.15.0

2026年09月18日(金) 08時33分06秒

1	ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版.	1
2	作業指示および開発履歴	3
2.1	概要	3
2.2	作業履歴	3
2.2.1	1. OpenCV依存の排除とMedia Foundation化	3
2.2.2	2. MFTによるH.264手動デコード実装	3
2.2.3	3. 初期化遅延（フリーズ）の回避（Lazy Initialization）	3
2.2.4	4. MFT バッファ管理と低遅延（Low Latency）化	4
2.2.5	5. レイテンシ優先モード（Drop old frames）の導入と例外修正	4
2.3	現在のステータスと課題	4
2.3.1	6. コードのクリーンアップと堅牢性 (Robustness) の向上	4
2.3.2	7. 依存ライブラリ管理とビルド環境のアップデート	5
2.3.3	8. キャプチャ開始時に投影方式固有のパラメータが失われる問題の修正	5
2.3.4	9. Menuを中心とした状態管理とクラス境界の整理	5
2.3.5	10. GStreamer対応の廃止	6
2.3.6	11. 入力画像の表示領域への自動フィット	6
2.3.7	12. 入力オープン時の初期画角計算の復元	6
2.3.8	13. クラスメンバ変数の初期化位置の最適化	7
2.3.9	14. 共通処理における命名規約・コメントの統一とドキュメント同期	7
2.3.10	15. mfcapture における GStreamer 関連コードの削除	7
2.3.11	16. ChArUco Board のマス目数（縦横）設定の追加と calib-wom からの rebase	7
3	名前空間索引	9
3.1	名前空間一覧	9
4	階層索引	11
4.1	クラス階層	11
5	クラス索引	13
5.1	クラス一覧	13
6	ファイル索引	15
6.1	ファイル一覧	15
7	名前空間詳解	17
7.1	gg 名前空間	17
7.1.1	詳解	20
7.1.2	列挙型詳解	20
7.1.2.1	BindingPoints	20
7.1.3	関数詳解	20
7.1.3.1	_ggError()	20
7.1.3.2	_ggFBOError()	21
7.1.3.3	ggArraysObj()	21

7.1.3.4 ggConjugate()	22
7.1.3.5 ggCreateComputeShader()	22
7.1.3.6 ggCreateNormalMap()	23
7.1.3.7 ggCreateShader()	24
7.1.3.8 ggCross() [1/2]	25
7.1.3.9 ggCross() [2/2]	26
7.1.3.10 ggDistance3() [1/2]	27
7.1.3.11 ggDistance3() [2/2]	27
7.1.3.12 ggDistance4() [1/2]	28
7.1.3.13 ggDistance4() [2/2]	29
7.1.3.14 ggDot3() [1/2]	30
7.1.3.15 ggDot3() [2/2]	30
7.1.3.16 ggDot4() [1/2]	31
7.1.3.17 ggDot4() [2/2]	32
7.1.3.18 ggElementsMesh()	32
7.1.3.19 ggElementsObj()	33
7.1.3.20 ggElementsSphere()	34
7.1.3.21 ggEllipse()	35
7.1.3.22 ggEulerQuaternion() [1/3]	35
7.1.3.23 ggEulerQuaternion() [2/3]	36
7.1.3.24 ggEulerQuaternion() [3/3]	37
7.1.3.25 ggFrustum()	38
7.1.3.26 ggIdentity()	39
7.1.3.27 ggIdentityQuaternion()	39
7.1.3.28 ggInit()	40
7.1.3.29 ggInvert() [1/2]	40
7.1.3.30 ggInvert() [2/2]	41
7.1.3.31 ggLength3() [1/2]	41
7.1.3.32 ggLength3() [2/2]	42
7.1.3.33 ggLength4() [1/2]	43
7.1.3.34 ggLength4() [2/2]	43
7.1.3.35 ggLoadComputeShader()	44
7.1.3.36 ggLoadHeight()	45
7.1.3.37 ggLoadImage()	46
7.1.3.38 ggLoadShader() [1/2]	47
7.1.3.39 ggLoadShader() [2/2]	47
7.1.3.40 ggLoadSimpleObj() [1/2]	48
7.1.3.41 ggLoadSimpleObj() [2/2]	49
7.1.3.42 ggLoadTexture()	50
7.1.3.43 ggLookat() [1/3]	51
7.1.3.44 ggLookat() [2/3]	52
7.1.3.45 ggLookat() [3/3]	52

7.1.3.46 ggMatrixQuaternion() [1/2]	53
7.1.3.47 ggMatrixQuaternion() [2/2]	54
7.1.3.48 ggNorm()	55
7.1.3.49 ggNormal()	55
7.1.3.50 ggNormalize()	56
7.1.3.51 ggNormalize3() [1/4]	56
7.1.3.52 ggNormalize3() [2/4]	57
7.1.3.53 ggNormalize3() [3/4]	57
7.1.3.54 ggNormalize3() [4/4]	58
7.1.3.55 ggNormalize4() [1/4]	58
7.1.3.56 ggNormalize4() [2/4]	59
7.1.3.57 ggNormalize4() [3/4]	60
7.1.3.58 ggNormalize4() [4/4]	60
7.1.3.59 ggOrthogonal()	61
7.1.3.60 ggPerspective()	62
7.1.3.61 ggPointsCube()	62
7.1.3.62 ggPointsSphere()	63
7.1.3.63 ggQuaternionMatrix()	63
7.1.3.64 ggQuaternionTransposeMatrix()	64
7.1.3.65 ggReadImage()	65
7.1.3.66 ggRectangle()	66
7.1.3.67 ggRotate() [1/5]	66
7.1.3.68 ggRotate() [2/5]	67
7.1.3.69 ggRotate() [3/5]	68
7.1.3.70 ggRotate() [4/5]	68
7.1.3.71 ggRotate() [5/5]	69
7.1.3.72 ggRotateQuaternion() [1/3]	70
7.1.3.73 ggRotateQuaternion() [2/3]	70
7.1.3.74 ggRotateQuaternion() [3/3]	71
7.1.3.75 ggRotateX()	71
7.1.3.76 ggRotateY()	72
7.1.3.77 ggRotateZ()	73
7.1.3.78 ggSaveColor()	74
7.1.3.79 ggSaveDepth()	75
7.1.3.80 ggSaveTga()	76
7.1.3.81 ggScale() [1/3]	77
7.1.3.82 ggScale() [2/3]	78
7.1.3.83 ggScale() [3/3]	78
7.1.3.84 ggSlerp() [1/4]	79
7.1.3.85 ggSlerp() [2/4]	80
7.1.3.86 ggSlerp() [3/4]	81
7.1.3.87 ggSlerp() [4/4]	82

7.1.3.88 ggTranslate() [1/3]	83
7.1.3.89 ggTranslate() [2/3]	84
7.1.3.90 ggTranslate() [3/3]	85
7.1.3.91 ggTranspose()	85
7.1.3.92 operator*()	86
7.1.3.93 operator+() [1/2]	86
7.1.3.94 operator+() [2/2]	87
7.1.3.95 operator-() [1/2]	87
7.1.3.96 operator-() [2/2]	87
7.1.3.97 operator/()	88
7.1.4 変数詳解	88
7.1.4.1 ggBufferAlignment	88
8 クラス詳解	89
8.1 Buffer クラス	89
8.1.1 詳解	90
8.1.2 構築子と解体子	90
8.1.2.1 Buffer() [1/4]	90
8.1.2.2 Buffer() [2/4]	91
8.1.2.3 Buffer() [3/4]	92
8.1.2.4 Buffer() [4/4]	92
8.1.2.5 ~Buffer()	93
8.1.3 関数詳解	93
8.1.3.1 bindBuffer()	93
8.1.3.2 channelsToFormat()	94
8.1.3.3 copy()	94
8.1.3.4 copyBuffer()	95
8.1.3.5 create()	96
8.1.3.6 discard()	97
8.1.3.7 formatToChannels()	98
8.1.3.8 getAspect()	99
8.1.3.9 getBufferName()	100
8.1.3.10 getChannels()	100
8.1.3.11 getFormat()	101
8.1.3.12 getHeight()	101
8.1.3.13 getSize()	102
8.1.3.14 getWidth()	103
8.1.3.15 map()	104
8.1.3.16 operator=() [1/2]	104
8.1.3.17 operator=() [2/2]	105
8.1.3.18 resize() [1/2]	105
8.1.3.19 resize() [2/2]	106

8.1.3.20 unbindBuffer()	107
8.1.3.21 unmap()	107
8.2 Calibration クラス	108
8.2.1 詳解	109
8.2.2 構築子と解体子	109
8.2.2.1 Calibration() [1/2]	109
8.2.2.2 Calibration() [2/2]	110
8.2.2.3 ~Calibration()	110
8.2.3 関数詳解	110
8.2.3.1 calibrate()	110
8.2.3.2 createBoard()	111
8.2.3.3 detectBoard()	111
8.2.3.4 detectMarkers()	112
8.2.3.5 discardCorners()	112
8.2.3.6 drawBoard()	113
8.2.3.7 finished()	113
8.2.3.8 getAllMarkerPoses()	113
8.2.3.9 getCameraMatrix()	114
8.2.3.10 getCornersCount()	114
8.2.3.11 getDistortionCoefficients()	115
8.2.3.12 getReprojectionError()	115
8.2.3.13 getSampleCount()	115
8.2.3.14 getTotalCount()	115
8.2.3.15 loadParameters()	115
8.2.3.16 operator=()	116
8.2.3.17 recordCorners()	116
8.2.3.18 RvecTvecToPose()	117
8.2.3.19 saveParameters()	117
8.2.3.20 setDictionary()	118
8.2.4 メンバ詳解	119
8.2.4.1 dictionaryList	119
8.3 CamCv クラス	120
8.3.1 詳解	122
8.3.2 構築子と解体子	122
8.3.2.1 CamCv()	122
8.3.2.2 ~CamCv()	122
8.3.3 関数詳解	122
8.3.3.1 close()	122
8.3.3.2 decreaseExposure()	123
8.3.3.3 decreaseGain()	123
8.3.3.4 getCodec() [1/2]	124
8.3.3.5 getCodec() [2/2]	124

8.3.3.6	getFps()	125
8.3.3.7	getPosition()	125
8.3.3.8	increaseExposure()	125
8.3.3.9	increaseGain()	126
8.3.3.10	open() [1/2]	126
8.3.3.11	open() [2/2]	127
8.3.3.12	setExposure()	127
8.3.3.13	setGain()	128
8.3.3.14	setPosition()	128
8.4	Camera クラス	129
8.4.1	詳解	130
8.4.2	構築子と解体子	131
8.4.2.1	Camera() [1/2]	131
8.4.2.2	Camera() [2/2]	131
8.4.2.3	~Camera()	132
8.4.3	関数詳解	132
8.4.3.1	capture()	132
8.4.3.2	close()	133
8.4.3.3	decreaseExposure()	133
8.4.3.4	decreaseGain()	133
8.4.3.5	getChannels()	133
8.4.3.6	getFps()	134
8.4.3.7	getFrames()	134
8.4.3.8	getHeight()	134
8.4.3.9	getPrioritizeLatency()	134
8.4.3.10	getSize()	135
8.4.3.11	getWidth()	135
8.4.3.12	increaseExposure()	135
8.4.3.13	increaseGain()	135
8.4.3.14	isRunning()	135
8.4.3.15	operator=()	135
8.4.3.16	setPrioritizeLatency()	136
8.4.3.17	start()	136
8.4.3.18	stop()	137
8.4.3.19	transmit() [1/3]	137
8.4.3.20	transmit() [2/3]	137
8.4.3.21	transmit() [3/3]	138
8.4.4	メンバ詳解	138
8.4.4.1	captured	138
8.4.4.2	channels	138
8.4.4.3	frame	138
8.4.4.4	height	139

8.4.4.5 image	139
8.4.4.6 in	139
8.4.4.7 interval	139
8.4.4.8 mtx	139
8.4.4.9 out	139
8.4.4.10 prioritizeLatency	140
8.4.4.11 reusableFrame	140
8.4.4.12 running	140
8.4.4.13 thr	140
8.4.4.14 total	140
8.4.4.15 width	140
8.5 CamImage クラス	141
8.5.1 詳解	143
8.5.2 構築子と解体子	143
8.5.2.1 CamImage() [1/2]	143
8.5.2.2 CamImage() [2/2]	143
8.5.2.3 ~CamImage()	144
8.5.3 関数詳解	144
8.5.3.1 isOpened()	144
8.5.3.2 load()	144
8.5.3.3 open()	145
8.5.3.4 save()	146
8.6 CamMf クラス	147
8.6.1 詳解	149
8.6.2 構築子と解体子	149
8.6.2.1 CamMf()	149
8.6.2.2 ~CamMf()	149
8.6.3 関数詳解	150
8.6.3.1 capture()	150
8.6.3.2 close()	150
8.6.3.3 getDeviceList()	151
8.6.3.4 getFormatList()	151
8.6.3.5 open()	151
8.6.3.6 select()	152
8.6.3.7 stop()	153
8.7 Capture クラス	153
8.7.1 詳解	154
8.7.2 構築子と解体子	154
8.7.2.1 Capture() [1/2]	154
8.7.2.2 Capture() [2/2]	154
8.7.2.3 ~Capture()	154
8.7.3 関数詳解	155

8.7.3.1 close()	155
8.7.3.2 getFps()	155
8.7.3.3 getPrioritizeLatency()	155
8.7.3.4 getSize()	156
8.7.3.5 isImage()	156
8.7.3.6 isOpened()	156
8.7.3.7 openDevice()	156
8.7.3.8 openImage()	157
8.7.3.9 openMovie()	157
8.7.3.10 operator bool()	158
8.7.3.11 retrieve() [1/2]	158
8.7.3.12 retrieve() [2/2]	159
8.7.3.13 setPrioritizeLatency()	159
8.7.3.14 start()	160
8.7.3.15 stop()	160
8.8 CaptureFormat 構造体	160
8.8.1 詳解	160
8.8.2 構築子と解体子	161
8.8.2.1 CaptureFormat()	161
8.8.3 メンバ詳解	161
8.8.3.1 codec	161
8.8.3.2 fps	161
8.8.3.3 index	161
8.8.3.4 resolution	162
8.9 Compute クラス	162
8.9.1 詳解	162
8.9.2 構築子と解体子	162
8.9.2.1 Compute()	162
8.9.2.2 ~Compute()	163
8.9.3 関数詳解	163
8.9.3.1 execute()	163
8.9.3.2 get()	163
8.9.3.3 use()	163
8.10 Config クラス	164
8.10.1 詳解	164
8.10.2 構築子と解体子	164
8.10.2.1 Config()	164
8.10.2.2 ~Config()	165
8.10.3 関数詳解	165
8.10.3.1 getBackground()	165
8.10.3.2 getCheckerLength()	165
8.10.3.3 getCheckerSize()	166

8.10.3.4	getDictionaryName()	166
8.10.3.5	getHeight()	167
8.10.3.6	getInitialImage()	167
8.10.3.7	getMarkerLength()	168
8.10.3.8	getMenuFont()	168
8.10.3.9	getMenuFontSize()	168
8.10.3.10	getPreferences()	168
8.10.3.11	getSettings()	169
8.10.3.12	getTitle()	169
8.10.3.13	getWidth()	169
8.10.3.14	initialize()	170
8.10.3.15	load()	170
8.10.3.16	save()	171
8.10.3.17	setSettings()	171
8.11	Expand クラス	172
8.11.1	詳解	172
8.11.2	構築子と解体子	172
8.11.2.1	Expand() [1/2]	172
8.11.2.2	Expand() [2/2]	173
8.11.2.3	～Expand()	173
8.11.3	関数詳解	173
8.11.3.1	getProgram()	173
8.11.3.2	operator=()	173
8.11.3.3	setup()	174
8.12	Framebuffer クラス	175
8.12.1	詳解	177
8.12.2	構築子と解体子	178
8.12.2.1	Framebuffer() [1/4]	178
8.12.2.2	Framebuffer() [2/4]	178
8.12.2.3	Framebuffer() [3/4]	179
8.12.2.4	Framebuffer() [4/4]	179
8.12.2.5	～Framebuffer()	180
8.12.3	関数詳解	180
8.12.3.1	bindFramebuffer()	180
8.12.3.2	copy()	181
8.12.3.3	create()	182
8.12.3.4	discard()	183
8.12.3.5	getChannels()	183
8.12.3.6	getFramebufferName()	184
8.12.3.7	getSize()	184
8.12.3.8	operator=() [1/2]	184
8.12.3.9	operator=() [2/2]	185

8.12.3.10 show()	186
8.12.3.11 unbindFramebuffer()	187
8.12.3.12 update() [1/2]	187
8.12.3.13 update() [2/2]	188
8.13 GgApp クラス	189
8.13.1 詳解	189
8.13.2 構築子と解体子	189
8.13.2.1 GgApp() [1/3]	189
8.13.2.2 GgApp() [2/3]	190
8.13.2.3 GgApp() [3/3]	190
8.13.2.4 ~GgApp()	191
8.13.3 関数詳解	191
8.13.3.1 main()	191
8.13.3.2 operator=() [1/2]	193
8.13.3.3 operator=() [2/2]	193
8.14 gg::GgBuffer< T > クラステンプレート	194
8.14.1 詳解	195
8.14.2 構築子と解体子	195
8.14.2.1 GgBuffer() [1/3]	195
8.14.2.2 GgBuffer() [2/3]	196
8.14.2.3 GgBuffer() [3/3]	196
8.14.2.4 ~GgBuffer()	196
8.14.3 関数詳解	196
8.14.3.1 bind()	196
8.14.3.2 copy()	196
8.14.3.3 getBuffer()	197
8.14.3.4 getCount()	197
8.14.3.5 getStride()	198
8.14.3.6 getTarget()	199
8.14.3.7 map() [1/2]	199
8.14.3.8 map() [2/2]	200
8.14.3.9 operator=() [1/2]	201
8.14.3.10 operator=() [2/2]	201
8.14.3.11 read()	202
8.14.3.12 send()	202
8.14.3.13 unbind()	203
8.14.3.14 unmap()	203
8.15 gg::GgColorTexture クラス	203
8.15.1 詳解	204
8.15.2 構築子と解体子	204
8.15.2.1 GgColorTexture() [1/3]	204
8.15.2.2 GgColorTexture() [2/3]	204

8.15.2.3 GgColorTexture() [3/3]	205
8.15.2.4 ~GgColorTexture()	205
8.15.3 関数詳解	206
8.15.3.1 load() [1/2]	206
8.15.3.2 load() [2/2]	207
8.16 gg::GgElements クラス	208
8.16.1 詳解	210
8.16.2 構築子と解体子	210
8.16.2.1 GgElements() [1/2]	210
8.16.2.2 GgElements() [2/2]	210
8.16.2.3 ~GgElements()	211
8.16.3 関数詳解	211
8.16.3.1 draw()	211
8.16.3.2 getIndexBuffer()	212
8.16.3.3 getIndexCount()	212
8.16.3.4 load()	213
8.16.3.5 send()	213
8.17 gg::GgMatrix クラス	215
8.17.1 詳解	218
8.17.2 構築子と解体子	218
8.17.2.1 GgMatrix() [1/6]	218
8.17.2.2 GgMatrix() [2/6]	218
8.17.2.3 GgMatrix() [3/6]	219
8.17.2.4 GgMatrix() [4/6]	219
8.17.2.5 GgMatrix() [5/6]	220
8.17.2.6 GgMatrix() [6/6]	220
8.17.2.7 ~GgMatrix()	221
8.17.3 関数詳解	221
8.17.3.1 frustum()	221
8.17.3.2 get() [1/2]	222
8.17.3.3 get() [2/2]	224
8.17.3.4 invert()	224
8.17.3.5 loadFrustum()	225
8.17.3.6 loadIdentity()	225
8.17.3.7 loadInvert() [1/2]	226
8.17.3.8 loadInvert() [2/2]	226
8.17.3.9 loadLookat() [1/3]	227
8.17.3.10 loadLookat() [2/3]	228
8.17.3.11 loadLookat() [3/3]	228
8.17.3.12 loadNormal() [1/2]	229
8.17.3.13 loadNormal() [2/2]	230
8.17.3.14 loadOrthogonal()	231

8.17.3.15 loadPerspective()	231
8.17.3.16 loadRotate() [1/5]	232
8.17.3.17 loadRotate() [2/5]	233
8.17.3.18 loadRotate() [3/5]	234
8.17.3.19 loadRotate() [4/5]	235
8.17.3.20 loadRotate() [5/5]	236
8.17.3.21 loadRotateX()	237
8.17.3.22 loadRotateY()	238
8.17.3.23 loadRotateZ()	238
8.17.3.24 loadScale() [1/3]	239
8.17.3.25 loadScale() [2/3]	240
8.17.3.26 loadScale() [3/3]	240
8.17.3.27 loadTranslate() [1/3]	241
8.17.3.28 loadTranslate() [2/3]	242
8.17.3.29 loadTranslate() [3/3]	242
8.17.3.30 loadTranspose() [1/2]	243
8.17.3.31 loadTranspose() [2/2]	244
8.17.3.32 lookat() [1/3]	244
8.17.3.33 lookat() [2/3]	245
8.17.3.34 lookat() [3/3]	246
8.17.3.35 normal()	247
8.17.3.36 operator*() [1/3]	248
8.17.3.37 operator*() [2/3]	249
8.17.3.38 operator*() [3/3]	250
8.17.3.39 operator*=() [1/2]	251
8.17.3.40 operator*=() [2/2]	252
8.17.3.41 operator+() [1/2]	252
8.17.3.42 operator+() [2/2]	253
8.17.3.43 operator+=() [1/2]	254
8.17.3.44 operator+=() [2/2]	254
8.17.3.45 operator-() [1/2]	255
8.17.3.46 operator-() [2/2]	256
8.17.3.47 operator-=() [1/2]	257
8.17.3.48 operator-=() [2/2]	257
8.17.3.49 operator/() [1/2]	258
8.17.3.50 operator/() [2/2]	259
8.17.3.51 operator/=() [1/2]	259
8.17.3.52 operator/=() [2/2]	260
8.17.3.53 operator=() [1/3]	261
8.17.3.54 operator=() [2/3]	261
8.17.3.55 operator=() [3/3]	262
8.17.3.56 orthogonal()	262

8.17.3.57 perspective()	263
8.17.3.58 projection() [1/4]	264
8.17.3.59 projection() [2/4]	264
8.17.3.60 projection() [3/4]	264
8.17.3.61 projection() [4/4]	264
8.17.3.62 rotate() [1/5]	265
8.17.3.63 rotate() [2/5]	265
8.17.3.64 rotate() [3/5]	266
8.17.3.65 rotate() [4/5]	266
8.17.3.66 rotate() [5/5]	267
8.17.3.67 rotateX()	268
8.17.3.68 rotateY()	269
8.17.3.69 rotateZ()	269
8.17.3.70 scale() [1/3]	270
8.17.3.71 scale() [2/3]	271
8.17.3.72 scale() [3/3]	272
8.17.3.73 translate() [1/3]	273
8.17.3.74 translate() [2/3]	274
8.17.3.75 translate() [3/3]	274
8.17.3.76 transpose()	275
8.18 gg::GgNormalTexture クラス	276
8.18.1 詳解	276
8.18.2 構築子と解体子	276
8.18.2.1 GgNormalTexture() [1/3]	276
8.18.2.2 GgNormalTexture() [2/3]	277
8.18.2.3 GgNormalTexture() [3/3]	277
8.18.2.4 ~GgNormalTexture()	278
8.18.3 関数詳解	278
8.18.3.1 load() [1/2]	278
8.18.3.2 load() [2/2]	279
8.19 gg::GgPoints クラス	280
8.19.1 詳解	281
8.19.2 構築子と解体子	281
8.19.2.1 GgPoints() [1/2]	281
8.19.2.2 GgPoints() [2/2]	282
8.19.2.3 ~GgPoints()	283
8.19.3 関数詳解	283
8.19.3.1 draw()	283
8.19.3.2 getBuffer()	284
8.19.3.3 getCount()	284
8.19.3.4 load()	285
8.19.3.5 operator bool()	285

8.19.3.6 operator"()	285
8.19.3.7 send()	286
8.20 gg::GgPointShader クラス	286
8.20.1 詳解	287
8.20.2 構築子と解体子	287
8.20.2.1 GgPointShader() [1/3]	287
8.20.2.2 GgPointShader() [2/3]	288
8.20.2.3 GgPointShader() [3/3]	289
8.20.2.4 ~GgPointShader()	289
8.20.3 関数詳解	290
8.20.3.1 get()	290
8.20.3.2 load() [1/2]	290
8.20.3.3 load() [2/2]	291
8.20.3.4 loadMatrix() [1/2]	292
8.20.3.5 loadMatrix() [2/2]	293
8.20.3.6 loadModelviewMatrix() [1/2]	294
8.20.3.7 loadModelviewMatrix() [2/2]	295
8.20.3.8 loadProjectionMatrix() [1/2]	296
8.20.3.9 loadProjectionMatrix() [2/2]	296
8.20.3.10 unuse()	297
8.20.3.11 use() [1/5]	297
8.20.3.12 use() [2/5]	298
8.20.3.13 use() [3/5]	299
8.20.3.14 use() [4/5]	299
8.20.3.15 use() [5/5]	300
8.21 gg::GgQuaternion クラス	301
8.21.1 詳解	304
8.21.2 構築子と解体子	304
8.21.2.1 GgQuaternion() [1/7]	304
8.21.2.2 GgQuaternion() [2/7]	305
8.21.2.3 GgQuaternion() [3/7]	306
8.21.2.4 GgQuaternion() [4/7]	306
8.21.2.5 GgQuaternion() [5/7]	307
8.21.2.6 GgQuaternion() [6/7]	307
8.21.2.7 GgQuaternion() [7/7]	308
8.21.2.8 ~GgQuaternion()	308
8.21.3 関数詳解	308
8.21.3.1 add() [1/4]	308
8.21.3.2 add() [2/4]	309
8.21.3.3 add() [3/4]	309
8.21.3.4 add() [4/4]	310
8.21.3.5 conjugate()	311

8.21.3.6 divide() [1/4]	312
8.21.3.7 divide() [2/4]	312
8.21.3.8 divide() [3/4]	313
8.21.3.9 divide() [4/4]	314
8.21.3.10 euler() [1/3]	315
8.21.3.11 euler() [2/3]	315
8.21.3.12 euler() [3/3]	316
8.21.3.13 get()	317
8.21.3.14 getConjugateMatrix() [1/3]	317
8.21.3.15 getConjugateMatrix() [2/3]	318
8.21.3.16 getConjugateMatrix() [3/3]	318
8.21.3.17 getMatrix() [1/3]	319
8.21.3.18 getMatrix() [2/3]	320
8.21.3.19 getMatrix() [3/3]	321
8.21.3.20 invert()	322
8.21.3.21 loadAdd() [1/4]	322
8.21.3.22 loadAdd() [2/4]	323
8.21.3.23 loadAdd() [3/4]	323
8.21.3.24 loadAdd() [4/4]	324
8.21.3.25 loadConjugate() [1/2]	325
8.21.3.26 loadConjugate() [2/2]	326
8.21.3.27 loadDivide() [1/4]	326
8.21.3.28 loadDivide() [2/4]	327
8.21.3.29 loadDivide() [3/4]	327
8.21.3.30 loadDivide() [4/4]	328
8.21.3.31 loadEuler() [1/2]	329
8.21.3.32 loadEuler() [2/2]	329
8.21.3.33 loadIdentity()	330
8.21.3.34 loadInvert() [1/2]	331
8.21.3.35 loadInvert() [2/2]	332
8.21.3.36 loadMatrix() [1/2]	332
8.21.3.37 loadMatrix() [2/2]	333
8.21.3.38 loadMultiply() [1/4]	334
8.21.3.39 loadMultiply() [2/4]	334
8.21.3.40 loadMultiply() [3/4]	335
8.21.3.41 loadMultiply() [4/4]	336
8.21.3.42 loadNormalize() [1/2]	337
8.21.3.43 loadNormalize() [2/2]	338
8.21.3.44 loadRotate() [1/3]	339
8.21.3.45 loadRotate() [2/3]	339
8.21.3.46 loadRotate() [3/3]	340
8.21.3.47 loadRotateX()	341

8.21.3.48 loadRotateY()	341
8.21.3.49 loadRotateZ()	342
8.21.3.50 loadSlerp() [1/4]	343
8.21.3.51 loadSlerp() [2/4]	344
8.21.3.52 loadSlerp() [3/4]	344
8.21.3.53 loadSlerp() [4/4]	345
8.21.3.54 loadSubtract() [1/4]	346
8.21.3.55 loadSubtract() [2/4]	346
8.21.3.56 loadSubtract() [3/4]	347
8.21.3.57 loadSubtract() [4/4]	348
8.21.3.58 multiply() [1/4]	349
8.21.3.59 multiply() [2/4]	350
8.21.3.60 multiply() [3/4]	351
8.21.3.61 multiply() [4/4]	352
8.21.3.62 norm()	353
8.21.3.63 normalize()	353
8.21.3.64 operator*() [1/3]	354
8.21.3.65 operator*() [2/3]	355
8.21.3.66 operator*() [3/3]	355
8.21.3.67 operator*=() [1/3]	356
8.21.3.68 operator*=() [2/3]	356
8.21.3.69 operator*=() [3/3]	356
8.21.3.70 operator+() [1/3]	357
8.21.3.71 operator+() [2/3]	357
8.21.3.72 operator+() [3/3]	358
8.21.3.73 operator+=() [1/3]	358
8.21.3.74 operator+=() [2/3]	359
8.21.3.75 operator+=() [3/3]	359
8.21.3.76 operator-() [1/3]	359
8.21.3.77 operator-() [2/3]	360
8.21.3.78 operator-() [3/3]	360
8.21.3.79 operator-=() [1/3]	361
8.21.3.80 operator-=() [2/3]	361
8.21.3.81 operator-=() [3/3]	361
8.21.3.82 operator/() [1/3]	362
8.21.3.83 operator/() [2/3]	362
8.21.3.84 operator/() [3/3]	363
8.21.3.85 operator/=() [1/3]	363
8.21.3.86 operator/=() [2/3]	364
8.21.3.87 operator/=() [3/3]	364
8.21.3.88 operator=() [1/4]	364
8.21.3.89 operator=() [2/4]	365

8.21.3.90 operator=()	[3 / 4]	366
8.21.3.91 operator=()	[4 / 4]	366
8.21.3.92 rotate()	[1 / 3]	366
8.21.3.93 rotate()	[2 / 3]	367
8.21.3.94 rotate()	[3 / 3]	368
8.21.3.95 rotateX()		369
8.21.3.96 rotateY()		369
8.21.3.97 rotateZ()		370
8.21.3.98 slerp()	[1 / 2]	370
8.21.3.99 slerp()	[2 / 2]	371
8.21.3.100 subtract()	[1 / 4]	371
8.21.3.101 subtract()	[2 / 4]	372
8.21.3.102 subtract()	[3 / 4]	372
8.21.3.103 subtract()	[4 / 4]	373
8.22 gg::GgShader クラス		374
8.22.1 詳解		374
8.22.2 構築子と解体子		374
8.22.2.1 GgShader()	[1 / 4]	374
8.22.2.2 GgShader()	[2 / 4]	375
8.22.2.3 GgShader()	[3 / 4]	376
8.22.2.4 GgShader()	[4 / 4]	376
8.22.2.5 ~GgShader()		377
8.22.3 関数詳解		377
8.22.3.1 get()		377
8.22.3.2 operator=()	[1 / 2]	377
8.22.3.3 operator=()	[2 / 2]	377
8.22.3.4 unuse()		378
8.22.3.5 use()		378
8.23 gg::GgShape クラス		379
8.23.1 詳解		379
8.23.2 構築子と解体子		379
8.23.2.1 GgShape()		379
8.23.2.2 ~GgShape()		380
8.23.3 関数詳解		380
8.23.3.1 draw()		380
8.23.3.2 get()		381
8.23.3.3 getMode()		381
8.23.3.4 operator bool()		382
8.23.3.5 operator"!()		382
8.23.3.6 setMode()		382
8.24 gg::GgSimpleObj クラス		383
8.24.1 詳解		383

8.24.2 構築子と解体子	383
8.24.2.1 GgSimpleObj()	383
8.24.2.2 ~GgSimpleObj()	384
8.24.3 関数詳解	384
8.24.3.1 draw()	384
8.24.3.2 get()	384
8.24.3.3 operator bool()	385
8.24.3.4 operator"!()	385
8.25 gg::GgSimpleShader クラス	386
8.25.1 詳解	387
8.25.2 構築子と解体子	388
8.25.2.1 GgSimpleShader() [1/4]	388
8.25.2.2 GgSimpleShader() [2/4]	388
8.25.2.3 GgSimpleShader() [3/4]	389
8.25.2.4 GgSimpleShader() [4/4]	390
8.25.2.5 ~GgSimpleShader()	390
8.25.3 関数詳解	390
8.25.3.1 load() [1/2]	390
8.25.3.2 load() [2/2]	391
8.25.3.3 loadMatrix() [1/4]	392
8.25.3.4 loadMatrix() [2/4]	393
8.25.3.5 loadMatrix() [3/4]	393
8.25.3.6 loadMatrix() [4/4]	394
8.25.3.7 loadModelviewMatrix() [1/4]	395
8.25.3.8 loadModelviewMatrix() [2/4]	396
8.25.3.9 loadModelviewMatrix() [3/4]	396
8.25.3.10 loadModelviewMatrix() [4/4]	397
8.25.3.11 operator=()	398
8.25.3.12 use() [1/13]	399
8.25.3.13 use() [2/13]	400
8.25.3.14 use() [3/13]	401
8.25.3.15 use() [4/13]	402
8.25.3.16 use() [5/13]	402
8.25.3.17 use() [6/13]	403
8.25.3.18 use() [7/13]	404
8.25.3.19 use() [8/13]	405
8.25.3.20 use() [9/13]	406
8.25.3.21 use() [10/13]	406
8.25.3.22 use() [11/13]	407
8.25.3.23 use() [12/13]	407
8.25.3.24 use() [13/13]	408
8.26 gg::GgTexture クラス	409

8.26.1 詳解	409
8.26.2 構築子と解体子	409
8.26.2.1 GgTexture() [1/3]	409
8.26.2.2 GgTexture() [2/3]	410
8.26.2.3 GgTexture() [3/3]	411
8.26.2.4 ~GgTexture()	411
8.26.3 関数詳解	411
8.26.3.1 bind()	411
8.26.3.2 getHeight()	412
8.26.3.3 getSize() [1/2]	412
8.26.3.4 getSize() [2/2]	412
8.26.3.5 getTexture()	413
8.26.3.6 getWidth()	413
8.26.3.7 operator=() [1/2]	413
8.26.3.8 operator=() [2/2]	414
8.26.3.9 swapRandB()	414
8.26.3.10 unbind()	415
8.27 gg::GgTrackball クラス	415
8.27.1 詳解	419
8.27.2 構築子と解体子	419
8.27.2.1 GgTrackball()	419
8.27.2.2 ~GgTrackball()	420
8.27.3 関数詳解	420
8.27.3.1 begin()	420
8.27.3.2 end()	421
8.27.3.3 get()	421
8.27.3.4 getMatrix()	421
8.27.3.5 getQuaternion()	422
8.27.3.6 getScale() [1/3]	422
8.27.3.7 getScale() [2/3]	422
8.27.3.8 getScale() [3/3]	422
8.27.3.9 getStart() [1/3]	423
8.27.3.10 getStart() [2/3]	423
8.27.3.11 getStart() [3/3]	423
8.27.3.12 motion()	423
8.27.3.13 operator=()	424
8.27.3.14 region() [1/2]	425
8.27.3.15 region() [2/2]	425
8.27.3.16 reset()	426
8.27.3.17 rotate()	427
8.28 gg::GgTriangles クラス	427
8.28.1 詳解	428

8.28.2 構築子と解体子	428
8.28.2.1 GgTriangles() [1/2]	428
8.28.2.2 GgTriangles() [2/2]	429
8.28.2.3 ~GgTriangles()	430
8.28.3 関数詳解	430
8.28.3.1 draw()	430
8.28.3.2 getBuffer()	431
8.28.3.3 getCount()	431
8.28.3.4 load()	431
8.28.3.5 send()	432
8.29 gg::GgUniformBuffer< T > クラステンプレート	433
8.29.1 詳解	433
8.29.2 構築子と解体子	434
8.29.2.1 GgUniformBuffer() [1/3]	434
8.29.2.2 GgUniformBuffer() [2/3]	434
8.29.2.3 GgUniformBuffer() [3/3]	435
8.29.2.4 ~GgUniformBuffer()	435
8.29.3 関数詳解	435
8.29.3.1 bind()	435
8.29.3.2 copy()	436
8.29.3.3 fill()	437
8.29.3.4 getBuffer()	438
8.29.3.5 getCount()	439
8.29.3.6 getStride()	440
8.29.3.7 getTarget()	440
8.29.3.8 load() [1/2]	441
8.29.3.9 load() [2/2]	442
8.29.3.10 map() [1/2]	442
8.29.3.11 map() [2/2]	443
8.29.3.12 read()	443
8.29.3.13 send()	444
8.29.3.14 unbind()	445
8.29.3.15 unmap()	445
8.30 gg::GgVector クラス	446
8.30.1 詳解	447
8.30.2 構築子と解体子	447
8.30.2.1 GgVector() [1/6]	447
8.30.2.2 GgVector() [2/6]	448
8.30.2.3 GgVector() [3/6]	449
8.30.2.4 GgVector() [4/6]	449
8.30.2.5 GgVector() [5/6]	450
8.30.2.6 GgVector() [6/6]	450

8.30.2.7 ~GgVector()	451
8.30.3 関数詳解	451
8.30.3.1 distance3()	451
8.30.3.2 distance4()	452
8.30.3.3 dot3()	452
8.30.3.4 dot4()	453
8.30.3.5 length3()	454
8.30.3.6 length4()	454
8.30.3.7 normalize3()	455
8.30.3.8 normalize4()	455
8.30.3.9 operator*() [1/2]	455
8.30.3.10 operator*() [2/2]	456
8.30.3.11 operator*=() [1/2]	457
8.30.3.12 operator*=() [2/2]	458
8.30.3.13 operator+() [1/2]	459
8.30.3.14 operator+() [2/2]	460
8.30.3.15 operator+=() [1/2]	461
8.30.3.16 operator+=() [2/2]	462
8.30.3.17 operator-() [1/2]	463
8.30.3.18 operator-() [2/2]	464
8.30.3.19 operator-=() [1/2]	465
8.30.3.20 operator-=() [2/2]	466
8.30.3.21 operator/() [1/2]	467
8.30.3.22 operator/() [2/2]	468
8.30.3.23 operator/=() [1/2]	469
8.30.3.24 operator/=() [2/2]	470
8.30.3.25 operator=() [1/2]	471
8.30.3.26 operator=() [2/2]	471
8.31 gg::GgVertex 構造体	471
8.31.1 詳解	472
8.31.2 構築子と解体子	473
8.31.2.1 GgVertex() [1/4]	473
8.31.2.2 GgVertex() [2/4]	473
8.31.2.3 GgVertex() [3/4]	473
8.31.2.4 GgVertex() [4/4]	474
8.31.3 メンバ詳解	474
8.31.3.1 normal	474
8.31.3.2 position	475
8.32 gg::GgVertexArray クラス	475
8.32.1 詳解	475
8.32.2 構築子と解体子	475
8.32.2.1 GgVertexArray() [1/3]	475

8.32.2.2 GgVertexArray() [2/3]	476
8.32.2.3 GgVertexArray() [3/3]	477
8.32.2.4 ~GgVertexArray()	478
8.32.3 関数詳解	478
8.32.3.1 bind()	478
8.32.3.2 get()	478
8.32.3.3 operator=() [1/2]	478
8.32.3.4 operator=() [2/2]	479
8.33 Intrinsics 構造体	480
8.33.1 詳解	480
8.33.2 構築子と解体子	480
8.33.2.1 Intrinsics() [1/3]	480
8.33.2.2 Intrinsics() [2/3]	481
8.33.2.3 Intrinsics() [3/3]	481
8.33.3 関数詳解	481
8.33.3.1 setCenter()	481
8.33.3.2 setFov() [1/2]	482
8.33.3.3 setFov() [2/2]	482
8.33.3.4 setFps()	483
8.33.3.5 setSize()	483
8.33.4 メンバ詳解	483
8.33.4.1 center	483
8.33.4.2 fov	484
8.33.4.3 fps	484
8.33.4.4 sensorSize	484
8.33.4.5 size	484
8.34 gg::GgSimpleShader::Light 構造体	485
8.34.1 詳解	485
8.34.2 メンバ詳解	486
8.34.2.1 ambient	486
8.34.2.2 diffuse	486
8.34.2.3 position	486
8.34.2.4 specular	486
8.35 gg::GgSimpleShader::LightBuffer クラス	487
8.35.1 詳解	488
8.35.2 構築子と解体子	488
8.35.2.1 LightBuffer() [1/3]	488
8.35.2.2 LightBuffer() [2/3]	489
8.35.2.3 LightBuffer() [3/3]	490
8.35.2.4 ~LightBuffer()	490
8.35.3 関数詳解	490
8.35.3.1 load() [1/2]	490

8.35.3.2 load() [2/2]	491
8.35.3.3 loadAmbient() [1/3]	492
8.35.3.4 loadAmbient() [2/3]	493
8.35.3.5 loadAmbient() [3/3]	493
8.35.3.6 loadColor()	494
8.35.3.7 loadDiffuse() [1/3]	495
8.35.3.8 loadDiffuse() [2/3]	496
8.35.3.9 loadDiffuse() [3/3]	497
8.35.3.10 loadPosition() [1/4]	497
8.35.3.11 loadPosition() [2/4]	498
8.35.3.12 loadPosition() [3/4]	499
8.35.3.13 loadPosition() [4/4]	500
8.35.3.14 loadSpecular() [1/3]	501
8.35.3.15 loadSpecular() [2/3]	501
8.35.3.16 loadSpecular() [3/3]	502
8.35.3.17 select()	503
8.36 gg::GgSimpleShader::Material 構造体	504
8.36.1 詳解	505
8.36.2 メンバ詳解	505
8.36.2.1 ambient	505
8.36.2.2 diffuse	505
8.36.2.3 shininess	505
8.36.2.4 specular	506
8.37 gg::GgSimpleShader::MaterialBuffer クラス	506
8.37.1 詳解	507
8.37.2 構築子と解体子	508
8.37.2.1 MaterialBuffer() [1/3]	508
8.37.2.2 MaterialBuffer() [2/3]	508
8.37.2.3 MaterialBuffer() [3/3]	509
8.37.2.4 ~MaterialBuffer()	510
8.37.3 関数詳解	510
8.37.3.1 load() [1/2]	510
8.37.3.2 load() [2/2]	510
8.37.3.3 loadAmbient() [1/3]	511
8.37.3.4 loadAmbient() [2/3]	512
8.37.3.5 loadAmbient() [3/3]	513
8.37.3.6 loadAmbientAndDiffuse() [1/3]	513
8.37.3.7 loadAmbientAndDiffuse() [2/3]	514
8.37.3.8 loadAmbientAndDiffuse() [3/3]	515
8.37.3.9 loadDiffuse() [1/3]	516
8.37.3.10 loadDiffuse() [2/3]	517
8.37.3.11 loadDiffuse() [3/3]	518

8.37.3.12 loadShininess() [1/2]	518
8.37.3.13 loadShininess() [2/2]	519
8.37.3.14 loadSpecular() [1/3]	520
8.37.3.15 loadSpecular() [2/3]	521
8.37.3.16 loadSpecular() [3/3]	522
8.37.3.17 select()	522
8.38 Menu クラス	523
8.38.1 詳解	524
8.38.2 構築子と解体子	524
8.38.2.1 Menu() [1/2]	524
8.38.2.2 Menu() [2/2]	525
8.38.2.3 ~Menu()	525
8.38.3 関数詳解	526
8.38.3.1 draw()	526
8.38.3.2 getCheckerLength()	526
8.38.3.3 getCheckerSize()	526
8.38.3.4 getMarkerLength()	527
8.38.3.5 getMenuBarHeight()	527
8.38.3.6 getPose()	528
8.38.3.7 initializeInputIntrinsics()	528
8.38.3.8 operator bool()	528
8.38.3.9 operator=()	528
8.38.3.10 saveImage()	529
8.38.3.11 setup()	529
8.38.4 メンバ詳解	530
8.38.4.1 detectBoard	530
8.38.4.2 detectMarker	530
8.38.4.3 prioritizeLatency	531
8.39 Mesh クラス	531
8.39.1 詳解	531
8.39.2 構築子と解体子	531
8.39.2.1 Mesh() [1/2]	531
8.39.2.2 Mesh() [2/2]	532
8.39.2.3 ~Mesh()	532
8.39.3 関数詳解	532
8.39.3.1 draw()	532
8.39.3.2 drawMesh()	532
8.39.3.3 operator=()	533
8.40 Preference クラス	533
8.40.1 詳解	534
8.40.2 構築子と解体子	534
8.40.2.1 Preference() [1/3]	534

8.40.2.2 Preference() [2/3]	535
8.40.2.3 Preference() [3/3]	535
8.40.2.4 ~Preference()	536
8.40.3 関数詳解	536
8.40.3.1 buildShader()	536
8.40.3.2 getDescription()	536
8.40.3.3 getIntrinsics()	536
8.40.3.4 getShader()	536
8.40.3.5 setPreference()	536
8.41 Settings 構造体	537
8.41.1 詳解	538
8.41.2 関数詳解	538
8.41.2.1 getFocal()	538
8.41.3 メンバ詳解	538
8.41.3.1 checkerLength	538
8.41.3.2 checkerSize	538
8.41.3.3 defaultCheckerLength	539
8.41.3.4 defaultCheckerSize	539
8.41.3.5 defaultEuler	539
8.41.3.6 defaultFocal	539
8.41.3.7 defaultFocalRange	539
8.41.3.8 defaultMarkerLength	539
8.41.3.9 dictionaryName	540
8.41.3.10 euler	540
8.41.3.11 focal	540
8.41.3.12 focalRange	540
8.41.3.13 markerLength	540
8.41.3.14 samples	540
8.42 Texture クラス	541
8.42.1 詳解	543
8.42.2 構築子と解体子	543
8.42.2.1 Texture() [1/4]	543
8.42.2.2 Texture() [2/4]	544
8.42.2.3 Texture() [3/4]	544
8.42.2.4 Texture() [4/4]	545
8.42.2.5 ~Texture()	545
8.42.3 関数詳解	546
8.42.3.1 bindTexture()	546
8.42.3.2 copy()	547
8.42.3.3 create()	548
8.42.3.4 discard()	549
8.42.3.5 draw()	550

8.42.3.6 drawPixels() [1/5]	551
8.42.3.7 drawPixels() [2/5]	551
8.42.3.8 drawPixels() [3/5]	552
8.42.3.9 drawPixels() [4/5]	553
8.42.3.10 drawPixels() [5/5]	553
8.42.3.11 getChannels()	554
8.42.3.12 getSize()	554
8.42.3.13 getTextureName()	555
8.42.3.14 operator=() [1/2]	555
8.42.3.15 operator=() [2/2]	556
8.42.3.16 readPixels() [1/3]	557
8.42.3.17 readPixels() [2/3]	558
8.42.3.18 readPixels() [3/3]	558
8.42.3.19 unbindTexture()	559
8.42.4 メンバ詳解	560
8.42.4.1 mesh	560
8.43 GgApp::Window クラス	560
8.43.1 詳解	561
8.43.2 構築子と解体子	562
8.43.2.1 Window() [1/3]	562
8.43.2.2 Window() [2/3]	564
8.43.2.3 Window() [3/3]	564
8.43.2.4 ~Window()	564
8.43.3 関数詳解	565
8.43.3.1 get()	565
8.43.3.2 getAltArrowX()	565
8.43.3.3 getAltArrowY()	566
8.43.3.4 getAltIArrow()	566
8.43.3.5 getArrow() [1/2]	567
8.43.3.6 getArrow() [2/2]	567
8.43.3.7 getArrowX()	568
8.43.3.8 getArrowY()	569
8.43.3.9 getAspect()	570
8.43.3.10 getControlArrow()	570
8.43.3.11 getControlArrowX()	570
8.43.3.12 getControlArrowY()	571
8.43.3.13 getFboHeight()	572
8.43.3.14 getFboSize() [1/2]	572
8.43.3.15 getFboSize() [2/2]	572
8.43.3.16 getFboWidth()	573
8.43.3.17 getHeight()	573
8.43.3.18 getKey()	574

8.43.3.19	getLastKey()	574
8.43.3.20	getMouse() [1/3]	574
8.43.3.21	getMouse() [2/3]	575
8.43.3.22	getMouse() [3/3]	576
8.43.3.23	getMouseX()	576
8.43.3.24	getMouseY()	576
8.43.3.25	getRotation()	576
8.43.3.26	getRotationMatrix()	577
8.43.3.27	getScrollMatrix()	577
8.43.3.28	getShiftArrow()	577
8.43.3.29	getShiftArrowX()	578
8.43.3.30	getShiftArrowY()	579
8.43.3.31	getSize() [1/2]	579
8.43.3.32	getSize() [2/2]	579
8.43.3.33	getTranslation()	580
8.43.3.34	getTranslationMatrix()	580
8.43.3.35	getUserPointer()	581
8.43.3.36	getWheel() [1/3]	581
8.43.3.37	getWheel() [2/3]	581
8.43.3.38	getWheel() [3/3]	581
8.43.3.39	getWheelX()	582
8.43.3.40	getWheelY()	582
8.43.3.41	getWidth()	582
8.43.3.42	operator bool()	583
8.43.3.43	operator=() [1/2]	583
8.43.3.44	operator=() [2/2]	583
8.43.3.45	reset()	584
8.43.3.46	resetRotation()	584
8.43.3.47	resetTranslation()	585
8.43.3.48	restoreViewport()	585
8.43.3.49	selectInterface()	585
8.43.3.50	setClose()	586
8.43.3.51	setKeyboardFunc()	587
8.43.3.52	setMouseFunc()	587
8.43.3.53	setResizeFunc()	588
8.43.3.54	setUserPointer()	588
8.43.3.55	setVelocity()	588
8.43.3.56	setWheelFunc()	589
8.43.3.57	shouldClose()	589
8.43.3.58	swapBuffers()	590
8.43.3.59	updateViewport()	590

9 ファイル詳解	591
9.1 Buffer.cpp ファイル	591
9.1.1 詳解	591
9.2 Buffer.cpp	592
9.3 Buffer.h ファイル	594
9.3.1 詳解	595
9.3.2 マクロ定義詳解	595
9.3.2.1 USE_PIXEL_BUFFER_OBJECT	595
9.4 Buffer.h	596
9.5 calib.cpp ファイル	597
9.5.1 詳解	598
9.5.2 マクロ定義詳解	598
9.5.2.1 CONFIG_FILE	598
9.6 calib.cpp	599
9.7 Calibration.cpp ファイル	600
9.7.1 詳解	601
9.7.2 マクロ定義詳解	601
9.7.2.1 USE_RODRIGUES	601
9.8 Calibration.cpp	601
9.9 Calibration.h ファイル	607
9.9.1 詳解	608
9.10 Calibration.h	608
9.11 CamCv.h ファイル	610
9.11.1 詳解	611
9.12 CamCv.h	612
9.13 Camera.h ファイル	615
9.13.1 詳解	616
9.14 Camera.h	617
9.15 CamImage.h ファイル	619
9.15.1 詳解	620
9.16 CamImage.h	621
9.17 CamMf.cpp ファイル	622
9.17.1 詳解	623
9.17.2 関数詳解	623
9.17.2.1 SafeRelease()	623
9.17.2.2 SubTypeToName()	624
9.18 CamMf.cpp	624
9.19 CamMf.h ファイル	638
9.19.1 詳解	639
9.20 CamMf.h	639
9.21 Capture.cpp ファイル	641
9.21.1 詳解	641

9.22 Capture.cpp	642
9.23 Capture.h ファイル	644
9.23.1 詳解	645
9.24 Capture.h	646
9.25 Compute.h ファイル	647
9.25.1 詳解	647
9.26 Compute.h	648
9.27 Config.cpp ファイル	648
9.27.1 詳解	649
9.28 Config.cpp	649
9.29 Config.h ファイル	653
9.29.1 詳解	654
9.30 Config.h	654
9.31 Expand.cpp ファイル	656
9.32 Expand.cpp	656
9.33 Expand.h ファイル	658
9.33.1 詳解	659
9.34 Expand.h	659
9.35 Framebuffer.cpp ファイル	660
9.35.1 詳解	660
9.36 Framebuffer.cpp	661
9.37 Framebuffer.h ファイル	664
9.37.1 詳解	665
9.38 Framebuffer.h	665
9.39 gg.cpp ファイル	666
9.39.1 詳解	666
9.40 gg.cpp	667
9.41 gg.h ファイル	743
9.41.1 詳解	747
9.41.2 マクロ定義詳解	747
9.41.2.1 ggError	747
9.41.2.2 ggFBOError	747
9.41.3 型定義詳解	747
9.41.3.1 pathChar	747
9.41.3.2 pathString	748
9.41.4 関数詳解	748
9.41.4.1 TCharToUtf8()	748
9.41.4.2 Utf8ToTChar()	748
9.42 gg.h	749
9.43 GgApp.cpp ファイル	803
9.43.1 詳解	803
9.44 GgApp.cpp	804

9.45 GgApp.h ファイル	817
9.45.1 詳解	818
9.45.2 マクロ定義詳解	818
9.45.2.1 GG.BUTTON_COUNT	818
9.45.2.2 GG.INTERFACE_COUNT	818
9.45.2.3 GG.USE_IMGUI	818
9.46 GgApp.h	819
9.47 Intrinsics.cpp ファイル	827
9.47.1 詳解	827
9.48 Intrinsics.cpp	827
9.49 Intrinsics.h ファイル	828
9.49.1 詳解	829
9.50 Intrinsics.h	830
9.51 main.cpp ファイル	830
9.51.1 詳解	831
9.51.2 マクロ定義詳解	831
9.51.2.1 HEADER_STR	831
9.51.3 関数詳解	832
9.51.3.1 main()	832
9.52 main.cpp	833
9.53 Menu.cpp ファイル	833
9.53.1 詳解	834
9.53.2 関数詳解	834
9.53.2.1 getAnyList()	834
9.54 Menu.cpp	835
9.55 Menu.h ファイル	849
9.55.1 詳解	850
9.56 Menu.h	850
9.57 Mesh.h ファイル	852
9.57.1 詳解	853
9.58 Mesh.h	854
9.59 opencv_link.h ファイル	855
9.59.1 詳解	855
9.60 opencv_link.h	856
9.61 parseconfig.h ファイル	856
9.61.1 詳解	858
9.61.2 関数詳解	858
9.61.2.1 getString() [1/3]	858
9.61.2.2 getString() [2/3]	858
9.61.2.3 getString() [3/3]	859
9.61.2.4 getValue() [1/2]	859
9.61.2.5 getValue() [2/2]	860

9.61.2.6 setString() [1/3]	861
9.61.2.7 setString() [2/3]	861
9.61.2.8 setString() [3/3]	862
9.61.2.9 setValue() [1/2]	862
9.61.2.10 setValue() [2/2]	862
9.62 parseconfig.h	863
9.63 Preference.cpp ファイル	866
9.63.1 詳解	866
9.64 Preference.cpp	866
9.65 Preference.h ファイル	867
9.65.1 詳解	868
9.66 Preference.h	869
9.67 REQUESTS.md ファイル	870
9.68 Texture.cpp ファイル	870
9.68.1 詳解	870
9.69 Texture.cpp	871
9.70 Texture.h ファイル	874
9.70.1 詳解	875
9.71 Texture.h	875
Index	879

Chapter 1

ゲームグラフィックス特論の宿題用補助 プログラム **GLFW3** 版.

著作権所有

Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies or substantial portions of the Software.

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

作業指示および開発履歴

2.1 概要

本プロジェクトでは、RICOH THETA V などの H.264 出力Webカメラにおけるキャプチャおよびデコードのパフォーマンス問題（低フレームレート、数秒の遅延）を解消するため、Microsoft Media Foundation を直接利用したキャプチャモジュール (CamMf) の開発と最適化を行っている。

2.2 作業履歴

2.2.1 1. OpenCV依存の排除とMedia Foundation化

- 指示: `cv::VideoCapture` によるキャプチャ・デコードが遅いため、Media Foundation を使って自前でキャプチャし、OpenCV に依存しない `CamMf` クラスを実装する。
- 対応: `Camera.h` から OpenCV の依存を排除し、バッファとして `std::vector<GLubyte>` を利用する設計に変更。transmit 処理をテンプレート化。

2.2.2 2. MFTによるH.264手動デコード実装

- 指示: H.264 などの圧縮フォーマットから RGB32 への変換を高速に行うため、MFT (Media Foundation Transform) のデコードとカラーコンバータを手動で構築して処理する方式への変更。
- 対応: `CamMf` クラスに H.264 デコードとカラーコンバータを組み込み、CPU メモリへ直接出力するように実装。

2.2.3 3. 初期化遅延（フリーズ）の回避（Lazy Initialization）

- 指示: カメラ認識時やフォーマット一覧取得時 (`updateFormatList`、`openDevice`) にデコードが不要に初期化され、数秒のフリーズやエラーが発生する問題を解決。
- 対応: リスト取得時には `setFormat` を行わず、ユーザーが「開始」ボタンを押した際 (`capture.select`) にのみフォーマット適用・デコード初期化を実行する遅延評価 (Lazy Initialization) を導入。UIについても動的生成に最適化や3段階のドロップダウン化を実施。

2.2.4 4. MFT バッファ管理と低遅延 (Low Latency) 化

- ・ 指示: キャプチャ画像に数秒 (1~2秒) の遅延 (レイテンシ) が発生している問題を解決。
- ・ 対応: MFT_OUTPUT_DATA_BUFFER の CurrentLength を 0 にして渡すことで E_FAIL を回避。さらに H.264 デコーダの ICodecAPI を通じて CODECAPI_AVLowLatencyMode を設定。

2.2.5 5. レイテンシ優先モード (Drop old frames) の導入と例外修正

- ・ 指示: H.264 デコードで複数フレームがバッファリングされる環境において、従来の「全フレーム処理」では遅延が大きくなるため、「古いフレームを捨てて常に最新フレームを使う」レイテンシ最優先モードを追加し、実行時に切り替えられるようにする。
- ・ 対応:
 - Camera 基底クラスおよび CamMf に prioritizeLatency フラグと UI を追加し、有効時にはメインスレッドを待たずにキャプチャループを最高速で回し続ける設計とした。
 - Windows 8 以降の Media Foundation の仕様に合わせ、CODECAPI_AVLowLatencyMode を正しい型 (VT_UI4) で再設定し、デコードの遅延を劇的に短縮させた。
 - 同時に発生していた 0x8007007A (バッファ長上書きバグによる表示停止) および 0x80070057 (属性取得エラーによる例外スロー・パフォーマンス低下) を修正した。

2.3 現在のステータスと課題

- ・ フレームレートとレイテンシ: 大幅に改善され、実用上極めて低遅延で滑らかな動作となっている。MPC-HC (ハードウェアデコード + Zero-copy レンダリング) と比べると、CPU メモリ展開による数ミリ秒の遅延の差はあるものの、現状のアーキテクチャでは最速のパフォーマンスを達成し、課題完了とした。

2.3.1 6. コードのクリーンアップと堅牢性 (Robustness) の向上

- ・ 指示: コード内に存在する冗長な記述の整理、および潜在的なスレッド競合、メモリリーク、バッファオーバーフローの危険性を調査・解消する。
- ・ 対応:
 - CamMf.cpp 内に重複して記述されていたバッファ生成処理を createDecoderBuffer() および createConverterBuffer() へ集約し、安全に SafeRelease を行うようにリファクタリング。
 - Camera.h の共有フラグ (prioritizeLatency, captured, running) を std::atomic<bool> に変更し、スレッド間の競合 (データレース) を防止。
 - CamMf::enumerateFormats() のループ内で continue 時に発生していた pMediaType の COM オブジェクトのメモリリークを RAII (スコープベースの解放構造体) を導入して修正。
 - Camera::transmit() メソッドにおいて、OpenGL (PBO) や cv::Mat のバッファサイズを超えて memcpy 等が行われる可能性 (バッファオーバーフローの危険性) を考慮し、コピーするデータ長を std::min で安全なサイズにクランプ (制限) するように修正。

2.3.2 7. 依存ライブラリ管理とビルド環境のアップデート

- ・ 指示: libs ディレクトリの取り扱いを既存のディレクトリへのジャンクションから CMake による自動ダウンロード方式に変更し、Visual Studio のバージョンを 2022 以降とする。
- ・ 対応: CMakeLists.txt の構成に合わせて GEMINI.md の依存ライブラリ（インクルード／ライブラリパスの参照）およびデバッグ環境設定（DLL コピー、PATH 自動構成）の記述を更新した。あわせて、IDEの要件を Visual Studio 2022 以降に変更した。

2.3.3 8. キャプチャ開始時に投影方式固有のパラメータが失われる問題の修正

- ・ 指示: カメラを選択してキャプチャを開始した際、選択済みの投影方式が正しく反映されず、投影方式を選び直す必要がある問題を調査・修正する。
- ・ 原因: Menu::setSize() が入力解像度だけでなく、選択中の投影方式が持つ画角と中心位置まで汎用値で上書きしていた。投影方式の再選択時にはPreferenceのIntrinsicsが再コピーされるため、選び直すと正常に見えていた。
- ・ 対応:
 - Menu::setSize() は実際の入力解像度だけを更新するよう変更した。
 - 投影方式固有の画角と中心位置は、キャプチャ開始、静止画像読込、動画読込の際にも保持するようにした。
 - 関連する実装コメントとDoxygenを現在の挙動に合わせて更新した。

この時点の画角に関する方針は、起動時・入力オープン時の見かけを考慮して後述の第12項で変更した。

2.3.4 9. Menuを中心とした状態管理とクラス境界の整理

- ・ 指示: 度重なる修正で複雑化したMenu.cppと、クラスのスコープ制限によって不自然になっていた構造を点検し、問題を順に改善する。
- ・ 対応:
 - Config::load() を、一時領域へ読み込んで検証後に置換するトランザクション型の処理へ変更した。再読込時にpreferenceListへ追記し続ける問題も解消した。
 - 実行時に再読込したPreferenceのシェーダーを構築してから利用するようにした。
 - 各Preferenceのデストラクターが共有shaderListを消去し、他の投影方式のシェーダーポインターを無効化する問題を修正した。
 - 投影方式とIntrinsicsの同期をMenu::selectPreference()へ集約した。
 - デバイスのオープン、フォーマット適用、レイテンシ設定、実解像度反映、スレッド開始をMenu::startCapture()へ集約した。
 - Windowsのフォーマット情報をCaptureFormatとして構造化し、Menuが表示文字列を再解析する処理を廃止した。
 - Menu::draw() を、メニューバー、入力パネル、校正パネル、エラーダイアログの描画関数へ分割した。
 - Menuがconst Config&をconst_castして変更する構造を廃止した。
 - friend class Menuを廃止し、Configに用途を明示した読み取り・更新APIを追加した。
- ・ 検証: Debug構成とRelease構成の両方でビルドが成功した。

2.3.5 10. GStreamer対応の廃止

- 指示: 今後GStreamerを使用しないため、関連する処理、構成、文書を削除する。
- 対応:
 - 非Windows環境の入力バックエンド一覧から`cv::CAP_GSTREAMER`を削除した。
 - `Menu::openDevice()`にあったGStreamerパイプライン専用の入力処理を削除した。
 - `Config`からGStreamerパイプラインの保持、JSON読込・保存、公開getterを削除した。
 - `calib_config.json`から`gststreamer`項目と既定パイプラインを削除した。
 - `CamCv`、`README.md`、`GEMINI.md`の入力方式に関する説明を、GStreamerを含まない現行仕様へ更新した。

2.3.6 11. 入力画像の表示領域への自動フィット

- 指示: 画像、動画、カメラを開いた際の初期表示を、縦横比を維持しながら表示領域からはみ出さない最大サイズにする。
- 対応:
 - 投影方式固有の画角・中心位置と、最終画面への配置計算を分離した。
 - `Texture::draw()`で表示領域と画像の縦横比を比較し、表示矩形の頂点スケールを計算するようにした。
 - `draw.vert`ではテクスチャ座標を画像全体に固定し、頂点位置だけを縮小して`contain`配置するようにした。
 - `draw.frag`を通すことで、CPU/OpenCV側のBGRA配列を画面表示用のRGBAへ変換する従来の処理を維持した。
 - `GL_TEXTURE_SWIZZLE_RGBA`が反映されない`glBlitFramebuffer()`は最終表示に使用しない方針とした。
 - HiDPI環境でも正しいサイズになるよう、論理ウィンドウサイズではなく実Framebufferサイズを使用するようにした。

2.3.7 12. 入力オープン時の初期画角計算の復元

- 指示: 起動時および画像、動画、カメラを開いた際の見かけを維持するため、投影方式にかかわらず入力画像のサイズとアスペクト比に合わせて初期画角を設定する。一方、投影方式を選び直した場合は、その方式の設定画角へ戻す。
- 対応:
 - 入力オープン後の処理を`Menu::initializeInputIntrinsics()`へ集約した。
 - 実際の入力解像度を反映した後、現在の焦点距離を用いて`Intrinsics::setFov()`で初期画角を計算するようにした。
 - 入力初期化では中心位置を変更せず、選択中の投影方式の設定値を維持するようにした。
 - `Menu::selectPreference()`は実解像度だけを維持し、画角と中心位置を選択した投影方式の設定値へ戻す処理として明確化した。
 - 無効な解像度または焦点距離では画角計算を行わず、ゼロ除算を避けるようにした。

2.3.8 13. クラスメンバ変数の初期化位置の最適化

- 指示: クラスメンバ変数の初期化を、コンストラクタからクラス定義（ヘッダ内）へ移行する。
- 対応:
 - [Buffer](#), [Camera](#), [CamCv](#), [CamImage](#), [CamMf](#), [Capture](#), [Config](#), [Expand](#), [Framebuffer](#), [Intrinsics](#), [Menu](#), [Preference](#), [Texture](#) 等の全クラスで、初期値をヘッダ内（インクラス初期化構文 `int x{ 0 };`, `Framebuffer() = default;` 等）へ集約した。
 - コンストラクタ初期化子リストをシンプル化し、メンバーの初期化漏れを防ぐ構造ヘリファクタリングした。

2.3.9 14. 共通処理における命名規約・コメントの統一とドキュメント同期

- 指示:
 - `calib-wom-msmf` と `mfcapture` で共通する変数名・関数名は `mfcapture` のものに合わせる。
 - コメント表現は `calib-wom-msmf` に合わせる。
 - 修正内容を両プロジェクトのドキュメント (Markdown, HTML) に反映する。
- 対応:
 - `calib-wom-msmf` と `mfcapture` 間で共通する変数名・関数名を `mfcapture` の命名規則へ統一し、[Doxygen](#) および実装コメント記述を `calib-wom-msmf` の解説表現へ統一した。
 - C++ ソースは UTF-8 with BOM、GLSL ソースは UTF-8 without BOM の保存形式を再検証し、`Debug / Release` 両構成での正常ビルドを確認した。
 - `presentation.html`, `presentation.md`, `workshop_handbook.html`, `workshop_handbook.md`, `images/` 内のプレゼンテーション・ハンドブック教材資産に C++ クラス設計・カプセル化方針を追記し、`calib-wom-msmf` および `mfcapture` の両ワークツリーへ反映・同期した。

2.3.10 15. `mfcapture` における `GStreamer` 関連コードの削除

- 指示: `GStreamer` は使用しないため、`mfcapture` から関連コードを削除し、設計・履歴ドキュメントを同期更新する。
- 対応:
 - `mfcapture` の [Menu.cpp](#) から `cv::CAP_GSTREAMER` バックエンドリスト項目の削除と `GStreamer` パイプライン処理分岐の完全撤去を行った。
 - `GEMINI.md` に `GStreamer` サポート対象外の規定を追加・同期した。

2.3.11 16. `ChArUco Board` のマス目数（縦横）設定の追加と `calib-wom` からの `rebase`

- 指示: `calib-wom` では `ChArUco Board` の検出設定にマス目のサイズしか指定できなかったが、マス目の縦横の数も設定できるようにする。また、本修正を `calib-wom` に適用した上で、`calib-wom-msmf` を `calib-wom` から `rebase` する。
- 対応:
 - `calib-wom` 側の [Settings](#)、[Calibration](#)、[Menu](#)、[calib.cpp](#)、および [Config](#) にマス目の縦横の数 (`checkerSize`) の設定・UI・シリアライズ処理を追加し、コミットした。

- `calib-wom-msmf` を最新の `calib-wom` から `rebase` し、インクラス初期化構文や `Media Foundation` 固有処理との競合を安全に解消した。
- `Settings` (`Config.h`) にマス目の横・縦の数を保持する `checkerSize` (デフォルト { 10, 7 }) を追加し、getter `getCheckerSize()` を `Config` および `Menu` に実装した。
- `Calibration` クラスのコンストラクタ、`createBoard()`、および `setDictionary()` に `checkerSize` 引数を追加し、ハードコードされていた `cv::Size{ 10, 7 }` を動的に指定できるようにした。
- `Menu` の校正パネルに `ImGui::InputInt2(u8"升目数", settings.checkerSize, &data())` を追加し、マス目数の変更時に `calibration.createBoard()` を呼び出してボード検出器を再生成できるようにした (最小値 2 のクランプ処理を含む)。
- `Config::load()` および `Config::save()` において、マス目数 (キー "squares" / "checkerSize") の保存と読み込みに対応した。
- C++ ソースファイルに UTF-8 BOM を付与し、`Debug / Release` ビルドおよび `Doxygen` が正常に通ることを確認した。

Chapter 3

名前空間索引

3.1 名前空間一覧

全名前空間の一覧です。

99 17

Chapter 4

階層索引

4.1 クラス階層

クラス階層一覧です。大雑把に文字符号順で並べられています。

std::array	
gg::GgMatrix	215
gg::GgVector	446
gg::GgQuaternion	301
gg::GgTrackball	415
Buffer	89
Texture	541
Framebuffer	175
Calibration	108
Camera	129
CamCv	120
CamImage	141
CamMf	147
Capture	153
CaptureFormat	160
Compute	162
Config	164
Expand	172
GgApp	189
gg::GgBuffer< T >	194
gg::GgColorTexture	203
gg::GgNormalTexture	276
gg::GgPointShader	286
gg::GgSimpleShader	386
gg::GgShader	374
gg::GgShape	379
gg::GgPoints	280
gg::GgTriangles	427
gg::GgElements	208
gg::GgSimpleObj	383
gg::GgTexture	409
gg::GgUniformBuffer< T >	433
gg::GgUniformBuffer< Light >	433
gg::GgSimpleShader::LightBuffer	487

gg::GgUniformBuffer< Material >	433
gg::GgSimpleShader::MaterialBuffer	506
gg::GgVertex	471
gg::GgVertexArray	475
Intrinsics	480
gg::GgSimpleShader::Light	485
gg::GgSimpleShader::Material	504
Menu	523
Mesh	531
Preference	533
Settings	537
GgApp::Window	560

Chapter 5

クラス索引

5.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

Buffer	89
Calibration	108
CamCv	120
Camera	129
CamImage	141
CamMf	147
Capture	153
CaptureFormat	160
Compute	162
Config	164
Expand	172
Framebuffer	175
GgApp	189
gg::GgBuffer< T >	194
gg::GgColorTexture	203
gg::GgElements	208
gg::GgMatrix	215
gg::GgNormalTexture	276
gg::GgPoints	280
gg::GgPointShader	286
gg::GgQuaternion	301
gg::GgShader	374
gg::GgShape	379
gg::GgSimpleObj	383
gg::GgSimpleShader	386
gg::GgTexture	409
gg::GgTrackball	415
gg::GgTriangles	427
gg::GgUniformBuffer< T >	433
gg::GgVector	446
gg::GgVertex	471
gg::GgVertexArray	475
Intrinsics	480
gg::GgSimpleShader::Light	485
gg::GgSimpleShader::LightBuffer	487

gg::GgSimpleShader::Material	504
gg::GgSimpleShader::MaterialBuffer	506
Menu	523
Mesh	531
Preference	533
Settings	537
Texture	541
GgApp::Window	560

Chapter 6

ファイル索引

6.1 ファイル一覧

ファイル一覧です。

Buffer.cpp	591
Buffer.h	594
calib.cpp	597
Calibration.cpp	600
Calibration.h	607
CamCv.h	610
Camera.h	615
CamImage.h	619
CamMf.cpp	622
CamMf.h	638
Capture.cpp	641
Capture.h	644
Compute.h	647
Config.cpp	648
Config.h	653
Expand.cpp	656
Expand.h	658
Framebuffer.cpp	660
Framebuffer.h	664
gg.cpp	666
gg.h	743
GgApp.cpp	803
GgApp.h	817
Intrinsics.cpp	827
Intrinsics.h	828
main.cpp	830
Menu.cpp	833
Menu.h	849
Mesh.h	852
opencv_link.h	855
parseconfig.h	856
Preference.cpp	866
Preference.h	867
Texture.cpp	870
Texture.h	874

Chapter 7

名前空間詳解

7.1 gg 名前空間

クラス

- class GgVector
- class GgMatrix
- class GgQuaternion
- class GgTrackball
- class GgTexture
- class GgColorTexture
- class GgNormalTexture
- class GgBuffer
- class GgUniformBuffer
- class GgVertexArray
- class GgShape
- class GgPoints
- struct GgVertex
- class GgTriangles
- class GgElements
- class GgShader
- class GgPointShader
- class GgSimpleShader
- class GgSimpleObj

列挙型

- enum BindingPoints { LightBindingPoint = 0 , MaterialBindingPoint }

関数

- void `ggInit` ()
- void `_ggError` (const std::string &name="", unsigned int line=0)
- void `_ggFBOError` (const std::string &name="", unsigned int line=0)
- void `ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength3` (const GLfloat *a)
- GLfloat `ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `ggNormalize3` (GLfloat *a)
- GLfloat `ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength4` (const GLfloat *a)
- GLfloat `ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `ggNormalize4` (GLfloat *a)
- const `GgVector` & `operator+` (const `GgVector` &v)
- `GgVector` `operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `operator-` (const `GgVector` &v)
- `GgVector` `operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength3` (const `GgVector` &a)
- GLfloat `ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize3` (const `GgVector` &a)
- void `ggNormalize3` (`GgVector` *a)
- GLfloat `ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength4` (const `GgVector` &a)
- GLfloat `ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize4` (const `GgVector` &a)
- void `ggNormalize4` (`GgVector` *a)
- `GgMatrix` `ggIdentity` ()
- `GgMatrix` `ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggTranslate` (const GLfloat *t)
- `GgMatrix` `ggTranslate` (const `GgVector` &t)
- `GgMatrix` `ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggScale` (const GLfloat *s)
- `GgMatrix` `ggScale` (const `GgVector` &s)
- `GgMatrix` `ggRotateX` (GLfloat a)
- `GgMatrix` `ggRotateY` (GLfloat a)
- `GgMatrix` `ggRotateZ` (GLfloat a)
- `GgMatrix` `ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix` `ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r)
- `GgMatrix` `ggRotate` (const `GgVector` &r)
- `GgMatrix` `ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix` `ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix` `ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix` `ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)

- `GgMatrix ggTranspose` (const `GgMatrix` &m)
- `GgMatrix ggInvert` (const `GgMatrix` &m)
- `GgMatrix ggNormal` (const `GgMatrix` &m)
- `GgQuaternion ggIdentityQuaternion` ()
- `GgQuaternion ggMatrixQuaternion` (const `GLfloat` *a)
- `GgQuaternion ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion ggRotateQuaternion` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` a)
- `GgQuaternion ggRotateQuaternion` (const `GLfloat` *v, `GLfloat` a)
- `GgQuaternion ggRotateQuaternion` (const `GLfloat` *v)
- `GgQuaternion ggEulerQuaternion` (`GLfloat` heading, `GLfloat` pitch, `GLfloat` roll)
- `GgQuaternion ggEulerQuaternion` (const `GLfloat` *e)
- `GgQuaternion ggEulerQuaternion` (const `GgVector` &e)
- `GgQuaternion ggSlerp` (const `GLfloat` *a, const `GLfloat` *b, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GgQuaternion` &q, const `GLfloat` *a, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GLfloat` *a, const `GgQuaternion` &q, `GLfloat` t)
- `GLfloat ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion ggInvert` (const `GgQuaternion` &q)
- bool `ggSaveTga` (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool `ggSaveColor` (const std::string &name)
- bool `ggSaveDepth` (const std::string &name)
- bool `ggReadImage` (const std::string &name, std::vector< `GLubyte` > &image, `GLsizei` *pWidth, `GLsizei` *pHeight, `GLenum` *pFormat)
- `GLuint ggLoadTexture` (const `GLvoid` *image, `GLsizei` width, `GLsizei` height, `GLenum` format=`GL_RGB`, `GLenum` type=`GL_UNSIGNED_BYTE`, `GLenum` internal=`GL_RGB`, `GLenum` wrap=`GL_CLAMP_TO_EDGE`, bool swizzle=false)
- `GLuint ggLoadImage` (const std::string &name, `GLsizei` *pWidth=nullptr, `GLsizei` *pHeight=nullptr, `GLenum` internal=`GL_RGBA`, `GLenum` wrap=`GL_CLAMP_TO_EDGE`)
- void `ggCreateNormalMap` (const `GLubyte` *hmap, `GLsizei` width, `GLsizei` height, `GLenum` format, `GLfloat` nz, `GLenum` internal, std::vector< `GgVector` > &nmap)
- `GLuint ggLoadHeight` (const std::string &name, `GLfloat` nz, `GLsizei` *pWidth=nullptr, `GLsizei` *pHeight=nullptr, `GLenum` internal=`GL_RGBA`)
- `GLuint ggCreateShader` (const std::string &vsrsrc, const std::string &fsrc="", const std::string &gsrsrc="", `GLint` nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std::string &ftext="fragment shader", const std::string >ext="geometry shader")
- `GLuint ggLoadShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint ggLoadShader` (const std::array< std::string, 3 > &files, `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint ggCreateComputeShader` (const std::string &csrsrc, const std::string &ctext="compute shader")
- `GLuint ggLoadComputeShader` (const std::string &comp)
- std::shared_ptr< `GgPoints` > `ggPointsCube` (`GLsizei` countv, `GLfloat` length=1.0f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- std::shared_ptr< `GgPoints` > `ggPointsSphere` (`GLsizei` countv, `GLfloat` radius=0.5f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- std::shared_ptr< `GgTriangles` > `ggRectangle` (`GLfloat` width=1.0f, `GLfloat` height=1.0f)
- std::shared_ptr< `GgTriangles` > `ggEllipse` (`GLfloat` width=1.0f, `GLfloat` height=1.0f, `GLuint` slices=16)
- std::shared_ptr< `GgTriangles` > `ggArraysObj` (const std::string &name, bool normalize=false)
- std::shared_ptr< `GgElements` > `ggElementsObj` (const std::string &name, bool normalize=false)
- std::shared_ptr< `GgElements` > `ggElementsMesh` (`GLuint` slices, `GLuint` stacks, const `GLfloat`(*pos)[3], const `GLfloat`(*norm)[3]=nullptr)

- `std::shared_ptr< GgElements > ggElementsSphere` (GLfloat radius=1.0f, int slices=16, int stacks=8)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, bool normalize=false)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint `ggBufferAlignment`
使用している GPU のバッファアライメント。

7.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラムの名前空間

7.1.2 列挙型詳解

7.1.2.1 BindingPoints

```
enum gg::BindingPoints
```

光源と材質の uniform buffer object の結合ポイント。

列挙値

LightBindingPoint	光源の uniform buffer object の結合ポイント。
MaterialBindingPoint	材質の uniform buffer object の結合ポイント。

`gg.h` の 1349 行目に定義があります。

7.1.3 関数詳解

7.1.3.1 _ggError()

```
void gg::_ggError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

OpenGL のエラーをチェックする。

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない。
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない。

覚え書き

OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する。

`gg.cpp` の 2615 行目に定義があります。

7.1.3.2 ggFBOError()

```
void gg::ggFBOError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

FBO のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

gg.cpp の 2659 行目に定義があります。

7.1.3.3 ggArraysObj()

```
std::shared_ptr< gg::GgTriangles > gg::ggArraysObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

GgTriangles 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイルを読み込んで GgArrays 形式の三角形データを生成する.

gg.cpp の 5281 行目に定義があります。

呼び出し関係図:



7.1.3.4 ggConjugate()

```
GgQuaternion gg::ggConjugate (
    const GgQuaternion & q) [inline]
```

共役四元数を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* の共役四元数.

gg.h の 4756 行目に定義があります.

呼び出し関係図:



7.1.3.5 ggCreateComputeShader()

```
GLuint gg::ggCreateComputeShader (
    const std::string & csrc,
    const std::string & ctext = "compute shader") [extern]
```

コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する.

引数

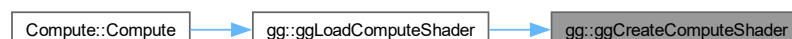
<i>csrc</i>	コンピュートシェーダのソースプログラムの文字列.
<i>ctext</i>	コンピュートシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5054 行目に定義があります.

被呼び出し関係図:



7.1.3.6 ggCreateNormalMap()

```
void gg::ggCreateNormalMap (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLfloat nz,
    GLenum internal,
    std::vector< GgVector > & nmap) [extern]
```

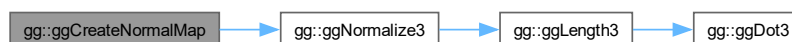
グレースケール画像 (8bit) から法線マップのデータを作成する。

引数

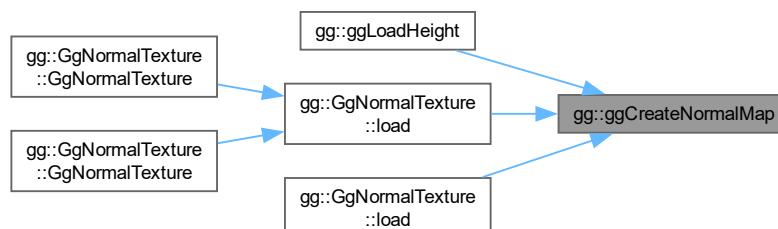
<i>hmap</i>	グレースケール画像のデータ.
<i>width</i>	高さマップのグレースケール画像 hmap の横の画素数.
<i>height</i>	高さマップのグレースケール画像 hmap の縦の画素数.
<i>format</i>	データの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線の z 成分の割合.
<i>internal</i>	法線マップを格納するテクスチャの内部フォーマット.
<i>nmap</i>	法線マップを格納する vector.

gg.cpp の 3881 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.7 ggCreateShader()

```
GLuint gg::ggCreateShader (
    const std::string & vsrc,
    const std::string & fsrc = "",
    const std::string & gsrc = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr,
    const std::string & vtext = "vertex shader",
    const std::string & ftext = "fragment shader",
    const std::string & gtext = "geometry shader") [extern]
```

シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

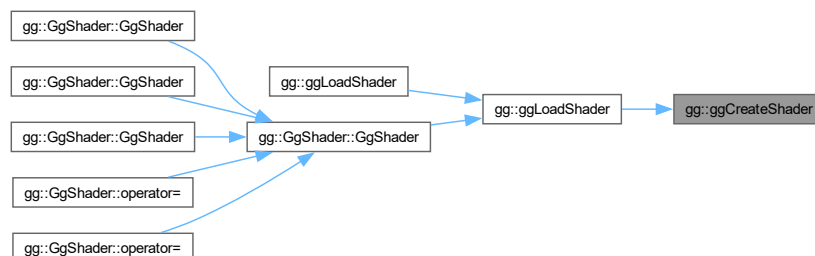
<i>vsrc</i>	バーテックスシェーダのソースプログラムの文字列.
<i>fsrc</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>gsrc</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).
<i>vtext</i>	バーテックスシェーダのコンパイル時のメッセージに追加する文字列.
<i>ftext</i>	フラグメントシェーダのコンパイル時のメッセージに追加する文字列.
<i>gtext</i>	ジオメトリシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の [4878](#) 行目に定義があります。

被呼び出し関係図:



7.1.3.8 ggCross() [1/2]

```
GgVector gg::ggCross (  
    const GgVector & a,  
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の外積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の外積.

覚え書き

戻り値の *w* (第4) 要素は 0.

gg.h の 1994 行目に定義があります。

呼び出し関係図:



7.1.3.9 ggCross() [2/2]

```
void gg::ggCross (
    GLfloat * c,
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の外積.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.
<i>c</i>	結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1429 行目に定義があります。

被呼び出し関係図:



7.1.3.10 ggDistance3() [1/2]

```
GLfloat gg::ggDistance3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の距離.

引数

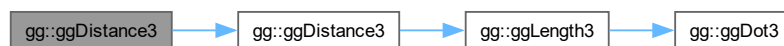
<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の距離.

gg.h の 2031 行目に定義があります。

呼び出し関係図:



7.1.3.11 ggDistance3() [2/2]

```
GLfloat gg::ggDistance3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の距離.

引数

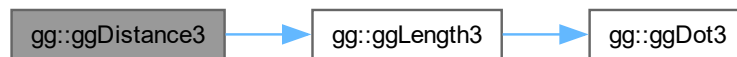
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

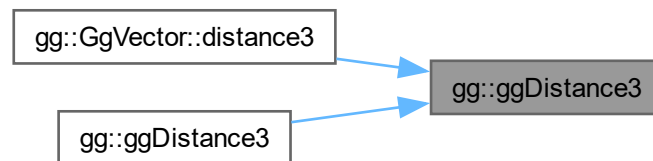
`a` と `b` の距離.

`gg.h` の 1466 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.12 `ggDistance4()` [1/2]

```

GLfloat gg::ggDistance4 (
    const GgVector & a,
    const GgVector & b) [inline]
  
```

`GgVector` 型の 4 要素の距離.

引数

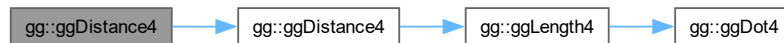
<code>a</code>	<code>GgVector</code> 型の変数.
<code>b</code>	<code>GgVector</code> 型の変数.

戻り値

2 つの [GgVector](#) a, b の距離.

[gg.h](#) の 2095 行目に定義があります。

呼び出し関係図:



7.1.3.13 ggDistance4() [2/2]

```
GLfloat gg::ggDistance4 (  
    const GLfloat * a,  
    const GLfloat * b) [inline]
```

GLfloat 型の 4 要素の距離.

引数

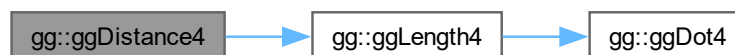
a	GLfloat 型の 4 要素の配列変数.
b	GLfloat 型の 4 要素の配列変数.

戻り値

a と b のそれぞれの 4 要素の距離.

[gg.h](#) の 1531 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.14 ggDot3() [1/2]

```
GLfloat gg::ggDot3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の内積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* のそれぞれの 3 要素の内積.

[gg.h](#) の 2008 行目に定義があります。

呼び出し関係図:



7.1.3.15 ggDot3() [2/2]

```
GLfloat gg::ggDot3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の内積.

引数

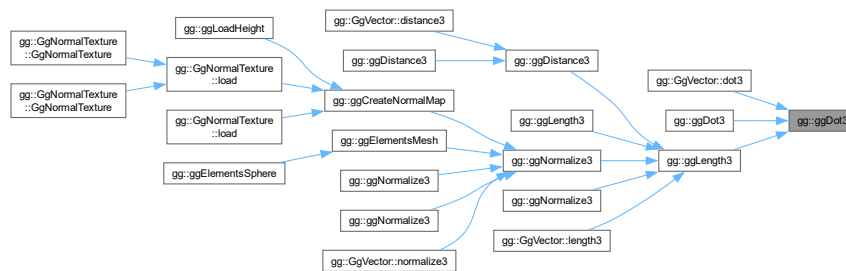
a	GLfloat 型の 3 要素の配列変数.
b	GLfloat 型の 3 要素の配列変数.

戻り値

a と b のそれぞれの 3 要素の内積.

gg.h の 1443 行目に定義があります。

被呼び出し関係図:



7.1.3.16 ggDot4() [1/2]

```
GLfloat gg::ggDot4 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 4 要素の内積.

引数

a	GgVector 型の変数.
b	GgVector 型の変数.

戻り値

a と b のそれぞれの 4 要素の内積.

gg.h の 2072 行目に定義があります。

呼び出し関係図:



<i>pos</i>	メッシュの頂点の位置.
<i>norm</i>	メッシュの頂点の法線, nullptr なら頂点の位置から算出する.

戻り値

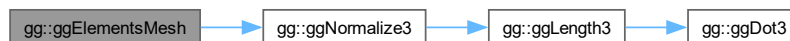
[GgElements](#) 型のポインタ.

覚え書き

メッシュ状に [GgElements](#) 形式の三角形データを生成する.

[gg.cpp](#) の 5315 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.19 ggElementsObj()

```
std::shared_ptr< gg::GgElements > gg::ggElementsObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイル を読み込む (Elements 形式).

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

`GgElements` 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイル を読み込んで `GgElements` 形式の三角形データを生成する.

`gg.cpp` の 5297 行目に定義があります.

呼び出し関係図:



7.1.3.20 ggElementsSphere()

```

std::shared_ptr< gg::GgElements > gg::ggElementsSphere (
    GLfloat radius = 1.0f,
    int slices = 16,
    int stacks = 8) [extern]
  
```

球状に三角形データを生成する (Elements 形式).

引数

<i>radius</i>	球の半径.
<i>slices</i>	球の経度方向の分割数.
<i>stacks</i>	球の緯度方向の分割数.

戻り値

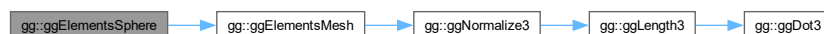
`GgElements` 型のポインタ.

覚え書き

球状に `GgElements` 形式の三角形データを生成する.

`gg.cpp` の 5410 行目に定義があります.

呼び出し関係図:



7.1.3.21 ggEllipse()

```
std::shared_ptr< gg::GgTriangles > gg::ggEllipse (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f,
    GLuint slices = 16) [extern]
```

楕円状に三角形を生成する。

引数

<i>width</i>	楕円の横幅.
<i>height</i>	楕円の高さ.
<i>slices</i>	楕円の分割数.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5256 行目に定義があります。

7.1.3.22 ggEulerQuaternion() [1/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GgVector & e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す。

引数

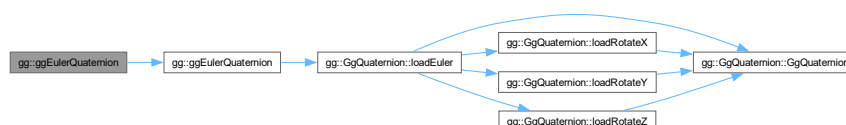
<i>e</i>	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
----------	--

戻り値

回転を表す四元数.

[gg.h](#) の 4670 行目に定義があります。

呼び出し関係図:



7.1.3.23 ggEulerQuaternion() [2/3]

```
GgQuaternion gg::ggEulerQuaternion (  
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

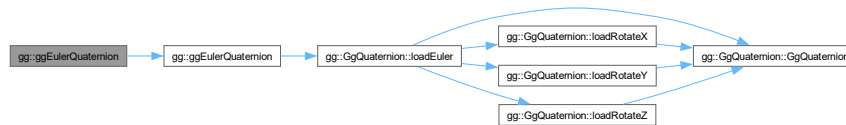
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

回転を表す四元数.

gg.h の 4659 行目に定義があります。

呼び出し関係図:



7.1.3.24 ggEulerQuaternion() [3/3]

```

GgQuaternion gg::ggEulerQuaternion (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) [inline]
  
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を返す.

引数

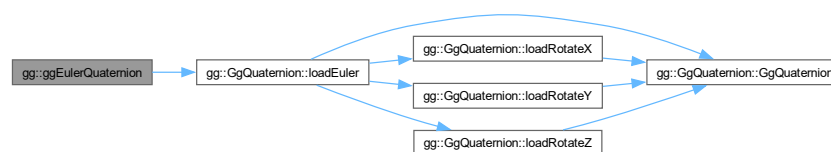
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

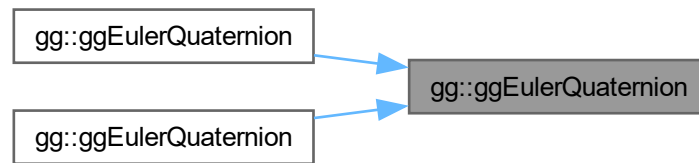
回転を表す四元数.

gg.h の 4647 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.25 ggFrustum()

```
GgMatrix gg::ggFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

透視透視投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3388 行目に定義があります。

呼び出し関係図:



7.1.3.26 ggIdentity()

```
GgMatrix gg::ggIdentity () [inline]
```

単位行列を返す.

戻り値

単位行列.

gg.h の 3113 行目に定義があります。

呼び出し関係図:



7.1.3.27 ggIdentityQuaternion()

```
GgQuaternion gg::ggIdentityQuaternion () [inline]
```

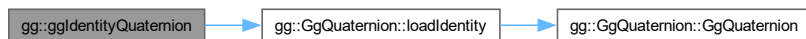
単位四元数を返す.

戻り値

単位四元数.

gg.h の 4545 行目に定義があります。

呼び出し関係図:



7.1.3.28 ggInit()

```
void gg::ggInit () [extern]
```

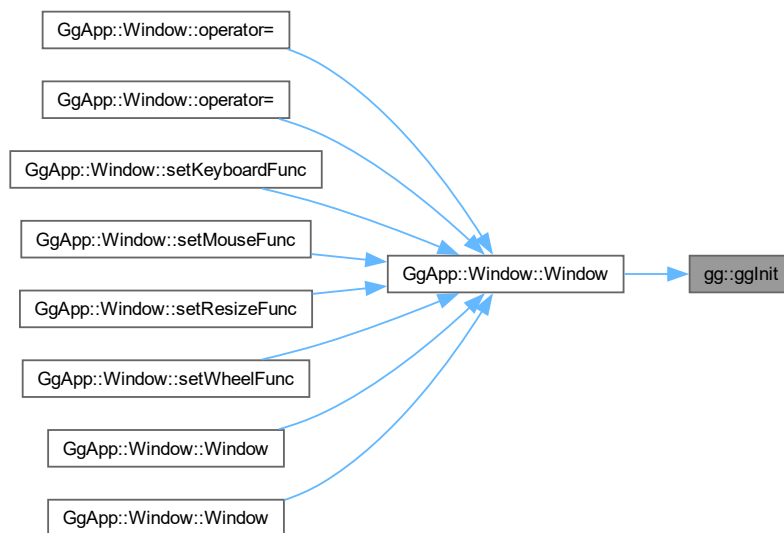
ゲームグラフィックス特論の都合にもとづく初期化を行う。

覚え書き

Windows において OpenGL 1.2 以降の API を有効化する。

gg.cpp の 1360 行目に定義があります。

被呼び出し関係図:



7.1.3.29 ggInvert() [1/2]

```
GgMatrix gg::ggInvert (
    const GgMatrix & m) [inline]
```

逆行列を返す。

引数

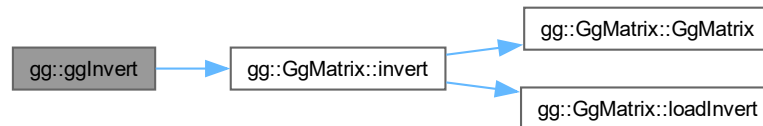
<i>m</i>	元の変換行列.
----------	---------

戻り値

m の逆行列.

gg.h の 3433 行目に定義があります。

呼び出し関係図:



7.1.3.30 ggInvert() [2/2]

```
GgQuaternion gg::ggInvert (
    const GgQuaternion & q) [inline]
```

四元数の逆元を求める.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 q の逆元.

gg.h の 4767 行目に定義があります。

呼び出し関係図:



7.1.3.31 ggLength3() [1/2]

```
GLfloat gg::ggLength3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の長さ.

引数

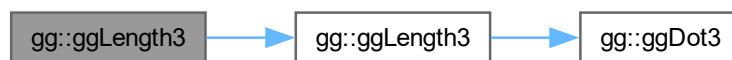
<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

a の 3 要素の長さ.

gg.h の 2019 行目に定義があります。

呼び出し関係図:



7.1.3.32 ggLength3() [2/2]

```
GLfloat gg::ggLength3 (  
    const GLfloat * a) [inline]
```

3 要素の長さ.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
----------	-----------------------

戻り値

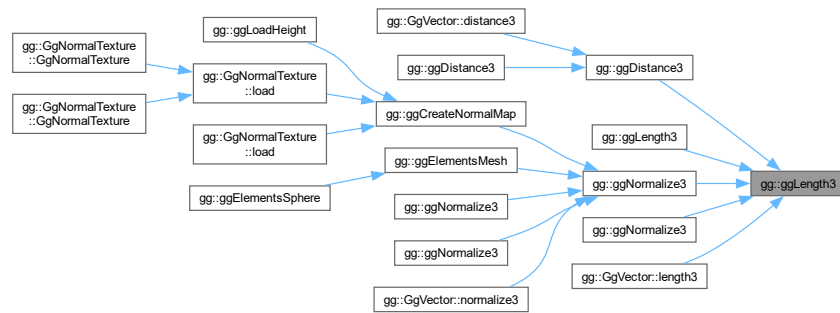
a の 3 要素の長さ.

gg.h の 1454 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.33 ggLength4() [1/2]

```
GLfloat gg::ggLength4 (
    const GgVector & a) [inline]
```

GgVector 型の 4 要素の長さ.

引数

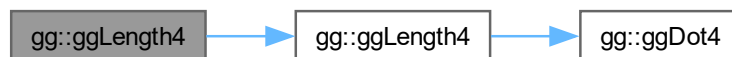
<i>a</i>	GgVector 型の変数.
----------	-----------------------

戻り値

a の 4 要素の長さ.

gg.h の 2083 行目に定義があります.

呼び出し関係図:



7.1.3.34 ggLength4() [2/2]

```
GLfloat gg::ggLength4 (
    const GLfloat * a) [inline]
```

GLfloat 型の 4 要素の長さ.

引数

<code>a</code>	GLfloat 型の 4 要素の配列変数.
----------------	-----------------------

戻り値

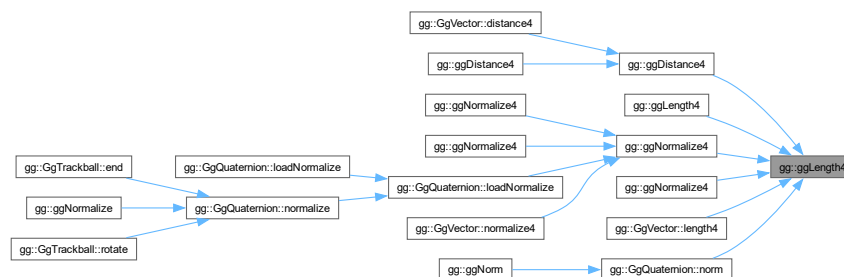
`a` の 4 要素の長さ.

`gg.h` の 1519 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.35 ggLoadComputeShader()

```

GLuint gg::ggLoadComputeShader (
    const std::string & comp) [extern]
  
```

コンピュータシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<code>comp</code>	コンピュータシェーダのソースファイル名.
-------------------	----------------------

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5095 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.36 ggLoadHeight()

```

GLuint gg::ggLoadHeight (
    const std::string & name,
    GLfloat nz,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA) [extern]
  
```

TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する。

引数

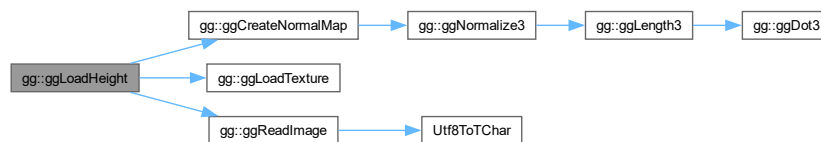
<i>name</i>	読み込むファイル名.
<i>nz</i>	法線の z 成分の割合.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3966 行目に定義があります。

呼び出し関係図:



7.1.3.37 ggLoadImage()

```

GLuint gg::ggLoadImage (
    const std::string & name,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE) [extern]
  
```

テクスチャを作成して TGA フォーマットの画像ファイルを読み込む。

引数

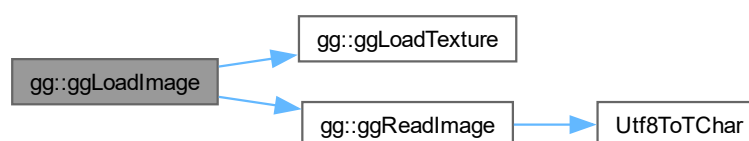
<i>name</i>	読み込むファイル名.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	テクスチャの内部フォーマット, デフォルトは GL_RGBA. 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3832 行目に定義があります。

呼び出し関係図:



7.1.3.38 ggLoadShader() [1/2]

```
GLuint gg::ggLoadShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.h](#) の 5156 行目に定義があります.

呼び出し関係図:



7.1.3.39 ggLoadShader() [2/2]

```
GLuint gg::ggLoadShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [extern]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

<code>varyings</code>	フィードバックする <code>varying</code> 変数のリスト (<code>nullptr</code> なら不使用).
-----------------------	--

戻り値

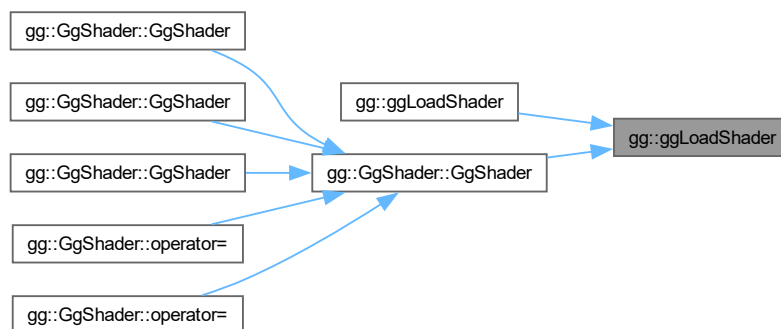
プログラムオブジェクトのプログラム名 (作成できなければ 0).

`gg.cpp` の 5021 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.40 ggLoadSimpleObj() [1/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルと MTL ファイルを読み込む (Arrays 形式)

引数

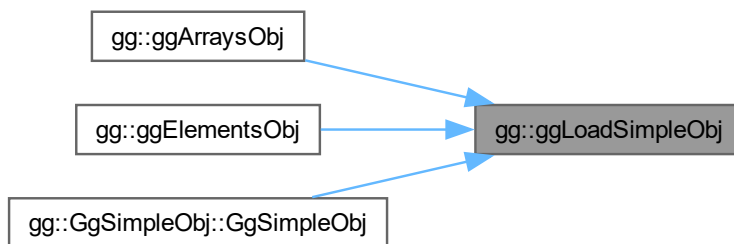
<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの <code>GgSimpleShader::Material</code> 型の材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

`gg.cpp` の 4630 行目に定義があります。

被呼び出し関係図:



7.1.3.41 ggLoadSimpleObj() [2/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    std::vector< GLuint > & face,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルを読み込む (Elements 形式).

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの材質.

<i>vert</i>	読み込んだデータの頂点属性.
<i>face</i>	読み込んだデータの三角形の頂点インデックス.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

[gg.cpp](#) の 4719 行目に定義があります。

7.1.3.42 ggLoadTexture()

```
GLuint gg::ggLoadTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [extern]
```

テクスチャを作成して確保して画像データをテクスチャとして読み込む。

引数

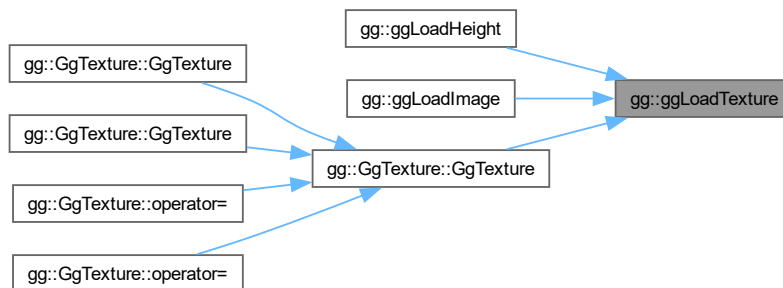
<i>image</i>	テクスチャとして読み込むデータ, nullptr ならテクスチャの作成のみを行う.
<i>width</i>	テクスチャとして読み込むデータ <i>image</i> の横の画素数.
<i>height</i>	テクスチャとして読み込むデータ <i>image</i> の縦の画素数.
<i>format</i>	<i>image</i> のフォーマット.
<i>type</i>	<i>image</i> のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

[gg.cpp](#) の 3781 行目に定義があります。

被呼び出し関係図:



7.1.3.43 ggLookat() [1/3]

```
GgMatrix gg::ggLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した <code>GgVector</code> 型の変数.
<i>t</i>	目標点の位置を格納した <code>GgVector</code> 型の変数.
<i>u</i>	上方向のベクトルを格納した <code>GgVector</code> 型の変数.

戻り値

求めたビュー変換行列.

`gg.h` の 3348 行目に定義があります。

呼び出し関係図:



7.1.3.44 ggLookat() [2/3]

```
GgMatrix gg::ggLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

求めたビュー変換行列.

gg.h の 3330 行目に定義があります。

呼び出し関係図:



7.1.3.45 ggLookat() [3/3]

```
GgMatrix gg::ggLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) [inline]
```

ビュー変換行列を返す.

引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

求めたビュー変換行列.

`gg.h` の 3312 行目に定義があります。

呼び出し関係図:

`gg::ggLookat`



`gg::GgMatrix::loadLookat`

7.1.3.46 `ggMatrixQuaternion()` [1/2]

```
GgQuaternion gg::ggMatrixQuaternion (  
    const GgMatrix & m) [inline]
```

回転の変換行列 `m` を表す四元数を返す.

引数

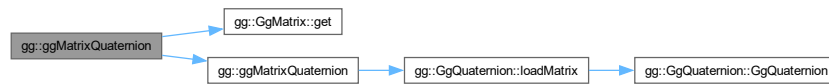
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

`m` による回転の変換に相当する四元数.

`gg.h` の 4569 行目に定義があります。

呼び出し関係図:



7.1.3.47 ggMatrixQuaternion() [2/2]

```
GgQuaternion gg::ggMatrixQuaternion (
    const GLfloat * a) [inline]
```

回転の変換行列 `m` を表す四元数を返す.

引数

<code>a</code>	GLfloat 型の 16 要素の配列変数.
----------------	------------------------

戻り値

`a` による回転の変換に相当する四元数.

`gg.h` の 4557 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.48 ggNorm()

```
GLfloat gg::ggNorm (
    const GgQuaternion & q) [inline]
```

四元数のノルムを返す。

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* のノルム.

gg.h の 4734 行目に定義があります。

呼び出し関係図:



7.1.3.49 ggNormal()

```
GgMatrix gg::ggNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を返す。

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の法線変換行列.

gg.h の 3444 行目に定義があります。

呼び出し関係図:



7.1.3.50 ggNormalize()

```
GgQuaternion gg::ggNormalize (
    const GgQuaternion & q) [inline]
```

正規化した四元数を返す.

引数

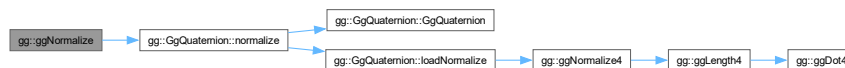
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* を正規化した四元数.

gg.h の 4745 行目に定義があります.

呼び出し関係図:



7.1.3.51 ggNormalize3() [1/4]

```
GgVector gg::ggNormalize3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

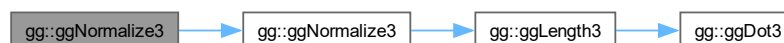
a の 3 要素を正規化したもの.

覚え書き

戻り値の *w* (第4) 要素は操作しない.

gg.h の 2045 行目に定義があります.

呼び出し関係図:



7.1.3.52 ggNormalize3() [2/4]

```
void gg::ggNormalize3 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

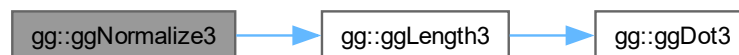
3 要素の正規化.

引数

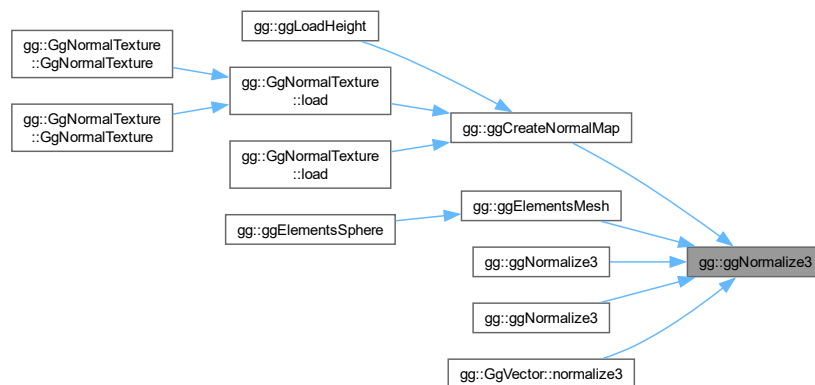
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	<i>a</i> の 3 要素を正規化した結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1478 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



7.1.3.53 ggNormalize3() [3/4]

```
void gg::ggNormalize3 (
    GgVector * a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

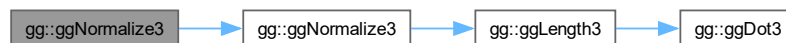
<i>a</i>	<code>GgVector</code> 型の変数のポインタ.
----------	----------------------------------

覚え書き

a の *w* (第4) 要素は操作しない.

`gg.h` の 2060 行目に定義があります。

呼び出し関係図:



7.1.3.54 `ggNormalize3()` [4/4]

```
void gg::ggNormalize3 (
    GLfloat * a) [inline]
```

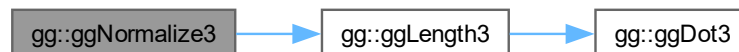
3 要素の正規化.

引数

<i>a</i>	<code>GLfloat</code> 型の 3 要素の配列変数.
----------	------------------------------------

`gg.h` の 1492 行目に定義があります。

呼び出し関係図:



7.1.3.55 `ggNormalize4()` [1/4]

```
GgVector gg::ggNormalize4 (
    const GgVector & a) [inline]
```

`GgVector` 型の 4 要素の正規化.

引数

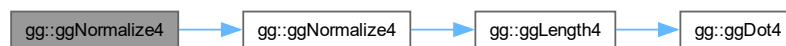
<i>a</i>	GgVector 型の変数.
----------	----------------

戻り値

a の 4 要素を正規化した結果.

gg.h の 2106 行目に定義があります。

呼び出し関係図:



7.1.3.56 ggNormalize4() [2/4]

```
void gg::ggNormalize4 (  
    const GLfloat * a,  
    GLfloat * b) [inline]
```

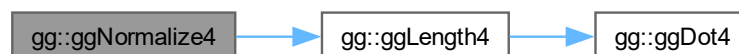
GLfloat 型の 4 要素の正規化.

引数

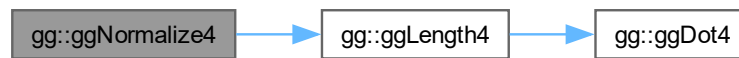
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	<i>a</i> を正規化した結果を格納する GLfloat 型の 4 要素の配列変数.

gg.h の 1543 行目に定義があります。

呼び出し関係図:



呼び出し関係図:



7.1.3.59 ggOrthogonal()

```
GgMatrix gg::ggOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

直交投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた直交投影変換行列.

[gg.h](#) の [3369](#) 行目に定義があります。

呼び出し関係図:



7.1.3.60 ggPerspective()

```
GgMatrix gg::ggPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

画角を指定して透視投影変換行列を返す.

引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3407 行目に定義があります。

呼び出し関係図:



7.1.3.61 ggPointsCube()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsCube (
    GLsizei countv,
    GLfloat length = 1.0f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を立方体状に生成する.

引数

<i>countv</i>	生成する点の数.
---------------	----------

<i>length</i>	点群を生成する立方体の一辺の長さ.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5182 行目に定義があります。

7.1.3.62 ggPointsSphere()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsSphere (
    GLsizei countv,
    GLfloat radius = 0.5f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を球状に生成する.

引数

<i>countv</i>	生成する点の数.
<i>radius</i>	点群を生成する半径.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5208 行目に定義があります。

7.1.3.63 ggQuaternionMatrix()

```
GgMatrix gg::ggQuaternionMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の変換行列を返す.

引数

q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する **GgMatrix** 型の変換行列.

gg.h の 4580 行目に定義があります。

呼び出し関係図:



7.1.3.64 ggQuaternionTransposeMatrix()

```
GgMatrix gg::ggQuaternionTransposeMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の転置した変換行列を返す.

引数

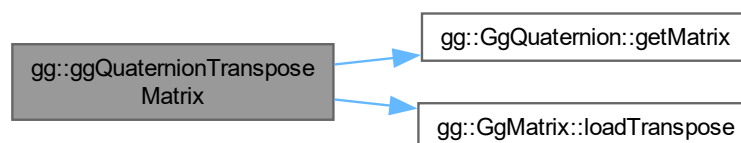
q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する転置した **GgMatrix** 型の変換行列.

gg.h の 4593 行目に定義があります。

呼び出し関係図:



7.1.3.65 ggReadImage()

```
bool gg::ggReadImage (
    const std::string & name,
    std::vector< GLubyte > & image,
    GLsizei * pWidth,
    GLsizei * pHeight,
    GLenum * pFormat) [extern]
```

TGA ファイル (8/16/24/32bit) をメモリに読み込む。

引数

<i>name</i>	読み込むファイル名.
<i>image</i>	読み込んだデータを格納する vector.
<i>pWidth</i>	読み込んだ画像の横の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pHeight</i>	読み込んだ画像の縦の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pFormat</i>	読み込んだファイルの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA) の格納先のポインタ, nullptr なら格納しない.

戻り値

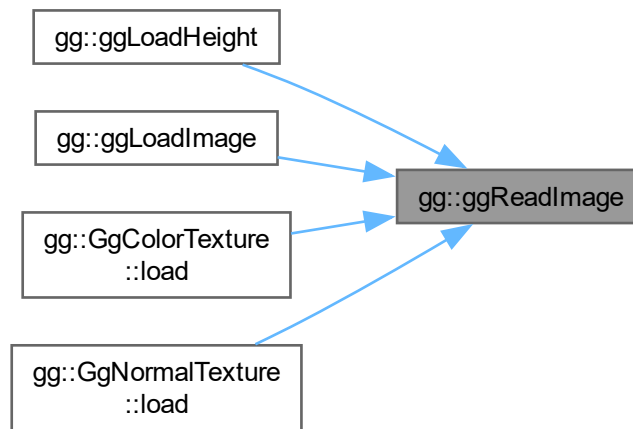
読み込みに成功すれば true, 失敗すれば false.

gg.cpp の 3673 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.66 ggRectangle()

```
std::shared_ptr< gg::GgTriangles > gg::ggRectangle (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f) [extern]
```

矩形状に 2 枚の三角形を生成する.

引数

<i>width</i>	矩形の横幅.
<i>height</i>	矩形の高さ.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5235 行目に定義があります。

7.1.3.67 ggRotate() [1/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<code>r</code>	回転軸のベクトルと回転角を表す <code>GgVector</code> 型の変数.
----------------	---

戻り値

(`r[0]`, `r[1]`, `r[2]`) を軸に `r[3]` だけ回転する変換行列.

`gg.h` の 3292 行目に定義があります.

呼び出し関係図:



7.1.3.68 ggRotate() [2/5]

```

GgMatrix gg::ggRotate (
    const GgVector & r,
    GLfloat a) [inline]
  
```

`r` 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<code>r</code>	回転軸のベクトルを表す <code>GgVector</code> 型の変数.
<code>a</code>	回転角.

戻り値

`r` を軸に `a` だけ回転する変換行列.

`gg.h` の 3268 行目に定義があります.

呼び出し関係図:



7.1.3.69 ggRotate() [3/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルと回転角を表す GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸に $r[3]$ だけ回転する変換行列.

gg.h の 3280 行目に定義があります。

呼び出し関係図:



7.1.3.70 ggRotate() [4/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

r を軸に a だけ回転する変換行列.

gg.h の 3255 行目に定義があります。

呼び出し関係図:



7.1.3.71 ggRotate() [5/5]

```
GgMatrix gg::ggRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

x	回転軸の x 成分.
y	回転軸の y 成分.
z	回転軸の z 成分.
a	回転角.

戻り値

(x, y, z) を軸にさらに a 回転する変換行列.

gg.h の 3242 行目に定義があります。

呼び出し関係図:



7.1.3.72 ggRotateQuaternion() [1/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を返す.

引数

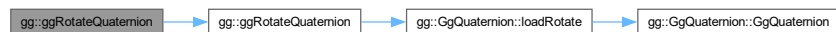
v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

回転を表す四元数.

gg.h の 4634 行目に定義があります.

呼び出し関係図:



7.1.3.73 ggRotateQuaternion() [2/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を返す.

引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転を表す四元数.

gg.h の 4623 行目に定義があります.

呼び出し関係図:



7.1.3.74 ggRotateQuaternion() [3/3]

```
GgQuaternion gg::ggRotateQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) を軸として角度 a 回転する四元数を返す.

引数

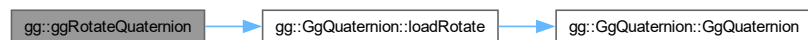
x	軸ベクトルの x 成分.
y	軸ベクトルの y 成分.
z	軸ベクトルの z 成分.
a	回転角.

戻り値

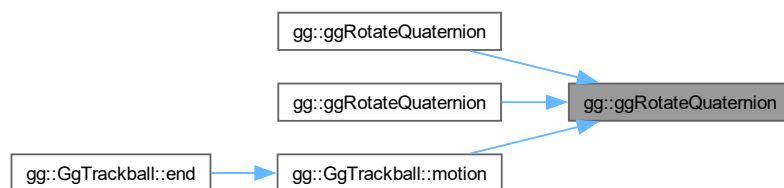
回転を表す四元数.

gg.h の 4610 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.75 ggRotateX()

```
GgMatrix gg::ggRotateX (
    GLfloat a) [inline]
```

x 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

x 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3203 行目に定義があります。

呼び出し関係図:



7.1.3.76 ggRotateY()

```
GgMatrix gg::ggRotateY (
    GLfloat a) [inline]
```

y 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

y 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3215 行目に定義があります。

呼び出し関係図:



7.1.3.77 ggRotateZ()

```
GgMatrix gg::ggRotateZ (  
    GLfloat a) [inline]
```

z 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3227 行目に定義があります。

呼び出し関係図:



7.1.3.78 ggSaveColor()

```
bool gg::ggSaveColor (
    const std::string & name) [extern]
```

カラーバッファの内容を TGA ファイルに保存する.

引数

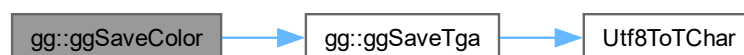
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

[gg.cpp](#) の 3617 行目に定義があります。

呼び出し関係図:



7.1.3.79 ggSaveDepth()

```
bool gg::ggSaveDepth (  
    const std::string & name) [extern]
```

デプスバッファの内容を TGA ファイルに保存する.

引数

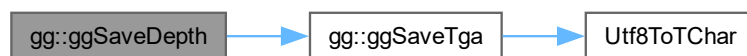
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3643 行目に定義があります。

呼び出し関係図:



7.1.3.80 ggSaveTga()

```
bool gg::ggSaveTga (  
    const std::string & name,  
    const void * buffer,  
    unsigned int width,  
    unsigned int height,  
    unsigned int depth) [extern]
```

配列の内容を TGA ファイルに保存する。

引数

<i>name</i>	保存するファイル名.
<i>buffer</i>	画像データを格納した配列.
<i>width</i>	画像の横の画素数.
<i>height</i>	画像の縦の画素数.
<i>depth</i>	1画素のバイト数.

戻り値

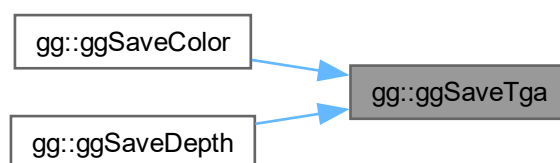
保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3541 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.81 ggScale() [1/3]

```
GgMatrix gg::ggScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GgVector 型の変数.
---	-------------------------------------

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3191 行目に定義があります.

呼び出し関係図:



7.1.3.82 ggScale() [2/3]

```
GgMatrix gg::ggScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小の変換行列.

gg.h の 3179 行目に定義があります。

呼び出し関係図:



7.1.3.83 ggScale() [3/3]

```
GgMatrix gg::ggScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

拡大縮小の変換行列を返す.

引数

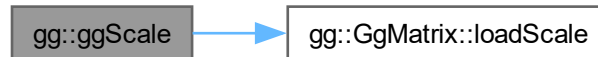
x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	拡大率のスケールファクタ (= 1.0f).

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3167 行目に定義があります。

呼び出し関係図:



7.1.3.84 ggSlerp() [1/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

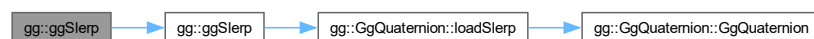
<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

q, *r* を *t* で内分した四元数.

[gg.h](#) の 4697 行目に定義があります。

呼び出し関係図:



7.1.3.85 ggSlerp() [2/4]

```
GgQuaternion gg::ggSlerp (  
    const GgQuaternion & q,  
    const GLfloat * a,  
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

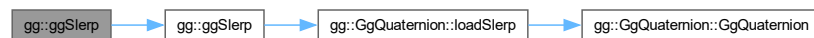
<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

q, *a* を *t* で内分した四元数.

gg.h の 4710 行目に定義があります。

呼び出し関係図:



7.1.3.86 ggSlerp() [3/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

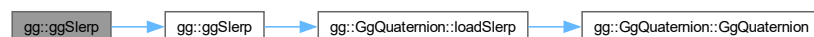
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

a, *q* を *t* で内分した四元数.

gg.h の 4723 行目に定義があります。

呼び出し関係図:



7.1.3.87 ggSlerp() [4/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

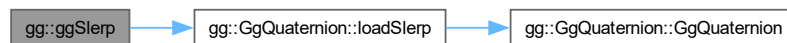
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

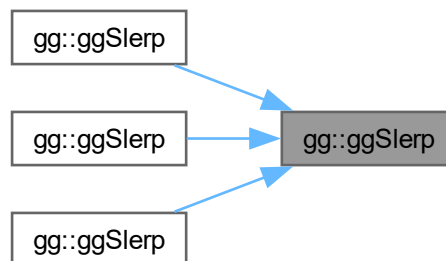
a, *b* を *t* で内分した四元数.

gg.h の 4683 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.88 ggTranslate() [1/3]

```
GgMatrix gg::ggTranslate (  
    const GgVector & t) [inline]
```

平行移動の変換行列を返す.

引数

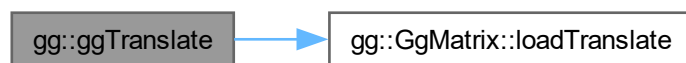
<code>t</code>	移動量の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

平行移動の変換行列.

`gg.h` の 3152 行目に定義があります.

呼び出し関係図:



7.1.3.89 ggTranslate() [2/3]

```
GgMatrix gg::ggTranslate (  
    const GLfloat * t) [inline]
```

平行移動の変換行列を返す.

引数

<code>t</code>	移動量の <code>GLfloat</code> 型の 3 要素の配列変数 (x, y, z).
----------------	---

戻り値

平行移動の変換行列.

`gg.h` の 3140 行目に定義があります.

呼び出し関係図:



7.1.3.90 ggTranslate() [3/3]

```
GgMatrix gg::ggTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

平行移動の変換行列を返す。

引数

<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	移動量のスケールファクタ (= 1.0f).

戻り値

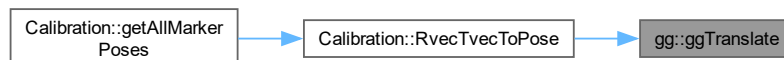
平行移動の変換行列.

gg.h の 3128 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.91 ggTranspose()

```
GgMatrix gg::ggTranspose (
    const GgMatrix & m) [inline]
```

転置行列を返す。

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の転置行列.

[gg.h](#) の 3422 行目に定義があります。

呼び出し関係図:



7.1.3.92 operator*()

```
GgVector gg::operator* (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに [GgVector](#) 型の各要素を乗じた積を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a に *b* の各要素を乗じた積.

[gg.h](#) の 1967 行目に定義があります。

7.1.3.93 operator+() [1/2]

```
const GgVector & gg::operator+ (
    const GgVector & v) [inline]
```

単項の + 演算子なので何もしない.

引数

v	<code>GgVector</code> 型のベクトル.
-----	-------------------------------

戻り値

v の値.

`gg.h` の 1920 行目に定義があります。

7.1.3.94 `operator+()` [2/2]

```
GgVector gg::operator+ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに `GgVector` 型の各要素を足した和を返す.

引数

a	GLfloat 型の値.
b	<code>GgVector</code> 型のベクトル.

戻り値

a に b の各要素を足した和.

`gg.h` の 1932 行目に定義があります。

7.1.3.95 `operator-()` [1/2]

```
const GgVector gg::operator- (
    const GgVector & v) [inline]
```

符号の反転.

引数

v	<code>GgVector</code> 型のベクトル.
-----	-------------------------------

戻り値

v の値の符号を反転した結果.

`gg.h` の 1943 行目に定義があります。

7.1.3.96 `operator-()` [2/2]

```
GgVector gg::operator- (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーから `GgVector` 型の各要素を引いた差を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a から *b* の各要素を引いた差.

gg.h の 1955 行目に定義があります。

7.1.3.97 operator/()

```
GgVector gg::operator/ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーを GgVector 型の各要素で割った商を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a を *b* の各要素で割った商.

gg.h の 1979 行目に定義があります。

7.1.4 変数詳解

7.1.4.1 ggBufferAlignment

```
GLint gg::ggBufferAlignment [extern]
```

使用している GPU のバッファアライメント.

使用している GPU のバッファオブジェクトのアライメント, 初期化に取得される.

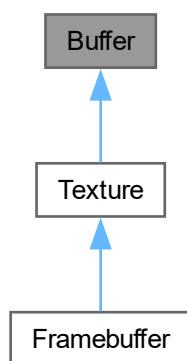
Chapter 8

クラス詳解

8.1 Buffer クラス

```
#include <Buffer.h>
```

Buffer の継承関係図



公開メンバ関数

- `Buffer ()=default`
- `Buffer (GLsizei width, GLsizei height, int channels)`
- `Buffer (const Buffer &buffer)`
- `Buffer (Buffer &&buffer) noexcept`
- `virtual ~Buffer ()`
- `Buffer & operator= (const Buffer &buffer)`
- `Buffer & operator= (Buffer &&buffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `void resize (GLsizei width, GLsizei height, int channels)`
- `void resize (const Buffer &buffer)`

- virtual void `copy` (const `Buffer` &buffer) noexcept
- virtual void `discard` ()
- auto `getBufferName` () const
- virtual const std::array< GLsizei, 2 > & `getSize` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- virtual int `getChannels` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

限定公開 メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const `Buffer` &buffer) noexcept

8.1.1 詳解

バッファクラス

フレームを格納するピクセルバッファオブジェクト

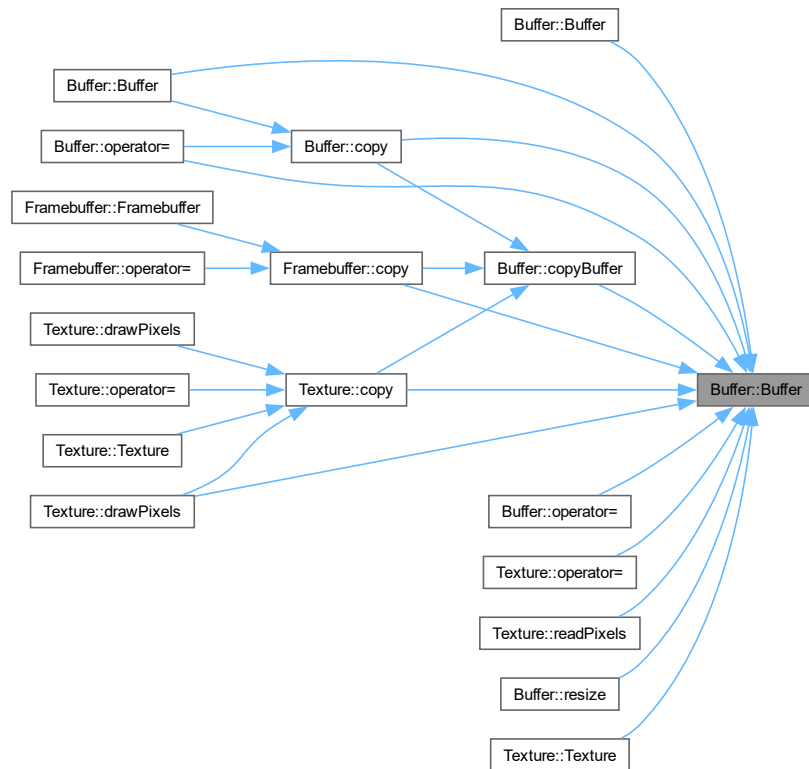
`Buffer.h` の 24 行目に定義があります。

8.1.2 構築子と解体子

8.1.2.1 `Buffer()` [1/4]

```
Buffer::Buffer () [default]
```


バッファのデフォルトコンストラクタ 被呼び出し関係図:



8.1.2.2 Buffer() [2/4]

```

Buffer::Buffer (
    GLsizei width,
    GLsizei height,
    int channels) [inline]

```

バッファを作成するコンストラクタ

引数

<i>width</i>	格納するフレームの横の画素数
<i>height</i>	格納するフレームの縦の画素数
<i>channels</i>	格納するフレームのチャンネル数

覚え書き

ピクセルバッファオブジェクトを作成して、そこにフレームのデータを格納する。

[Buffer.h](#) の 94 行目に定義があります。

呼び出し関係図:



8.1.2.3 Buffer() [3/4]

```
Buffer::Buffer (
    const Buffer & buffer) [inline]
```

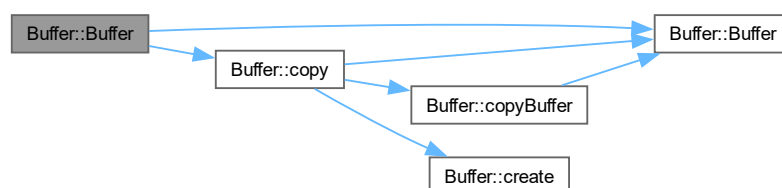
コピーコンストラクタ

引数

<i>buffer</i>	コピー元のバッファ
---------------	-----------

[Buffer.h](#) の 104 行目に定義があります。

呼び出し関係図:



8.1.2.4 Buffer() [4/4]

```
Buffer::Buffer (
    Buffer && buffer) [inline], [noexcept]
```

ムーブコンストラクタ

引数

<i>buffer</i>	ムーブ元のバッファ
---------------	-----------

[Buffer.h](#) の 114 行目に定義があります。

呼び出し関係図:



8.1.2.5 ~Buffer()

```
virtual Buffer::~Buffer () [inline], [virtual]
```

デストラクタ

[Buffer.h](#) の 135 行目に定義があります。

呼び出し関係図:



8.1.3 関数詳解

8.1.3.1 bindBuffer()

```
void Buffer::bindBuffer (  
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトを結合する

引数

<i>target</i>	結合するターゲット
---------------	-----------

[Buffer.h](#) の [294](#) 行目に定義があります。

8.1.3.2 channelsToFormat()

```
auto Buffer::channelsToFormat (
    int channels) const [inline], [protected]
```

チャンネル数からフォーマットを求める

引数

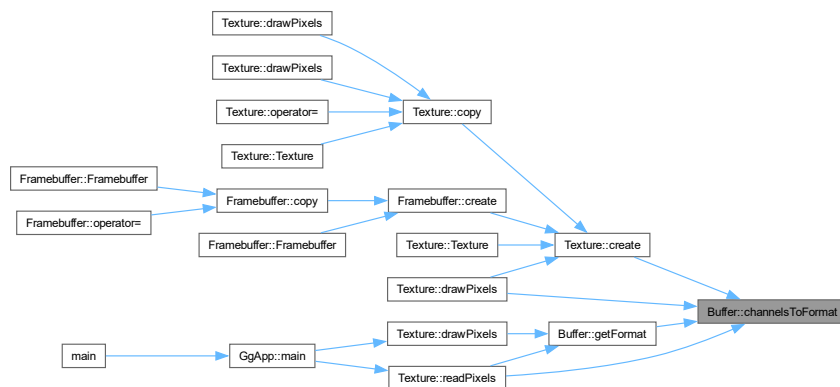
<i>channels</i>	フレームのチャンネル数
-----------------	-------------

戻り値

テクスチャのフォーマット

[Buffer.h](#) の [52](#) 行目に定義があります。

被呼び出し関係図:



8.1.3.3 copy()

```
void Buffer::copy (
    const Buffer & buffer) [virtual], [noexcept]
```

バッファをコピーする

引数

<code>buffer</code>	コピー元のバッファ
---------------------	-----------

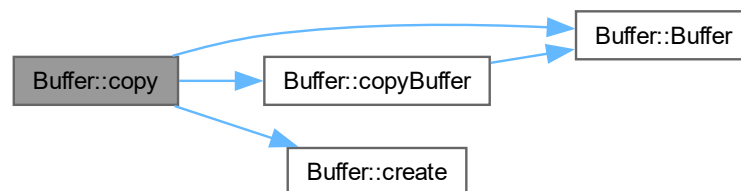
覚え書き

このバッファのサイズがコピー元のバッファのサイズと異なれば、このバッファを削除して新しいバッファを作り直してコピーする。

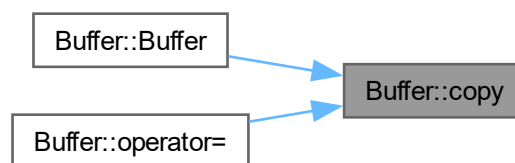
[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.cpp](#) の 93 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.4 copyBuffer()

```
void Buffer::copyBuffer (
    const Buffer & buffer) [protected], [noexcept]
```

既存のバッファにコピーする

引数

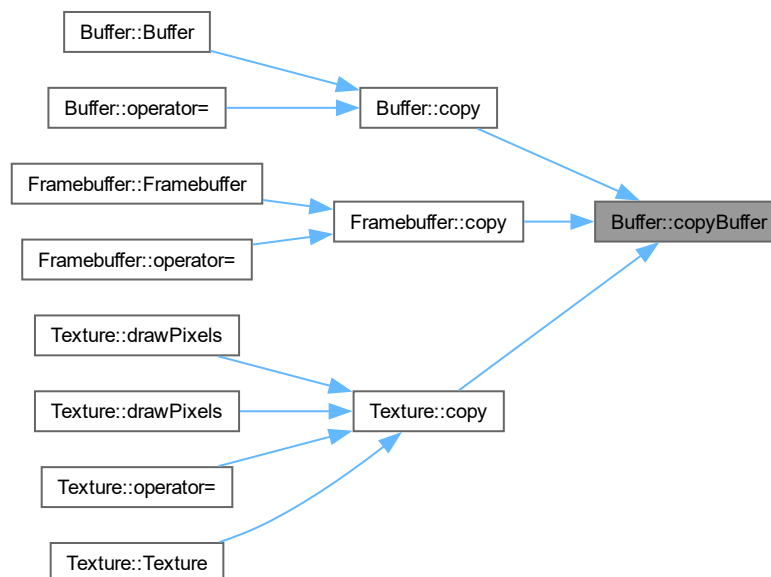
<i>buffer</i>	コピー元のバッファ
---------------	-----------

[Buffer.cpp](#) の 39 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.5 create()

```

void Buffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

バッファを作成する

引数

<i>width</i>	バッファに格納するフレームの横の画素数
<i>height</i>	バッファに格納するフレームの縦の画素数
<i>channels</i>	バッファに格納するフレームのチャンネル数

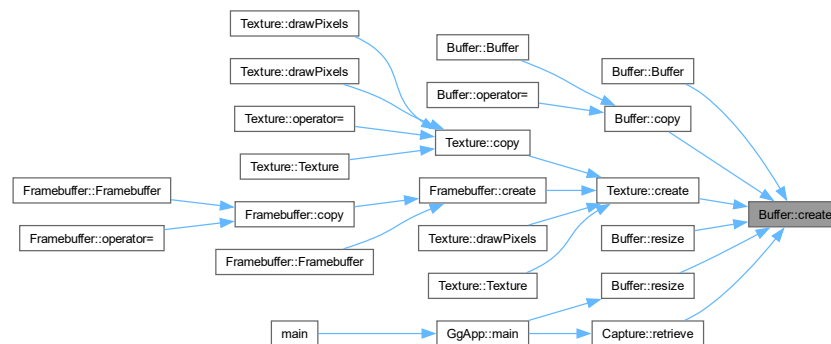
覚え書き

このバッファのサイズが引数で指定したサイズと異なれば、このバッファを削除して新しいバッファを作り直す。

Framebuffer, Textureで再実装されています。

Buffer.cpp の 60 行目に定義があります。

被呼び出し関係図:



8.1.3.6 discard()

```
void Buffer::discard () [virtual]
```

バッファを破棄する

Framebuffer, Textureで再実装されています。

Buffer.cpp の 187 行目に定義があります。

被呼び出し関係図:



8.1.3.7 formatToChannels()

```
int Buffer::formatToChannels (
    GLenum format) const [protected]
```

フォーマットからチャンネル数を求める

引数

<i>format</i>	テクスチャのフォーマット
---------------	--------------

戻り値

フレームのチャンネル数

[Buffer.cpp](#) の 16 行目に定義があります。

8.1.3.8 getAspect()

```
auto Buffer::getAspect () const [inline]
```

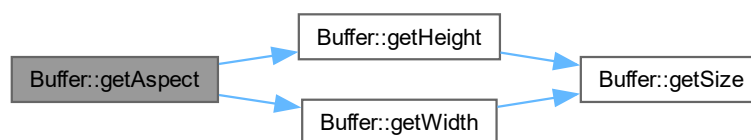
格納されているフレームの縦横比を得る

戻り値

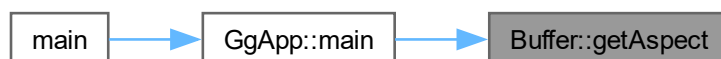
格納されているフレームの縦横比

[Buffer.h](#) の 284 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.9 getBufferName()

```
auto Buffer::getBufferName () const [inline]
```

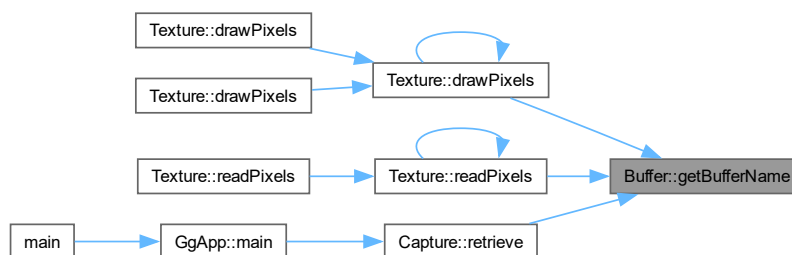
バッファのピクセルバッファオブジェクト名を得る

戻り値

ピクセルバッファオブジェクト名

[Buffer.h](#) の 221 行目に定義があります。

被呼び出し関係図:



8.1.3.10 getChannels()

```
virtual int Buffer::getChannels () const [inline], [virtual]
```

格納されているフレームのチャンネル数を得る

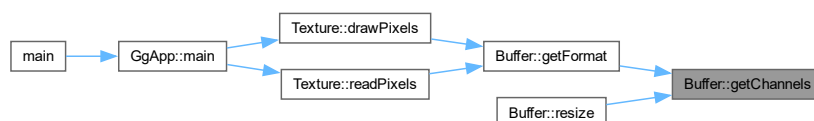
戻り値

格納されているフレームのチャンネル数

[Framebuffer](#), [Texture](#) で再実装されています。

[Buffer.h](#) の 264 行目に定義があります。

被呼び出し関係図:



8.1.3.11 getFormat()

```
auto Buffer::getFormat () const [inline]
```

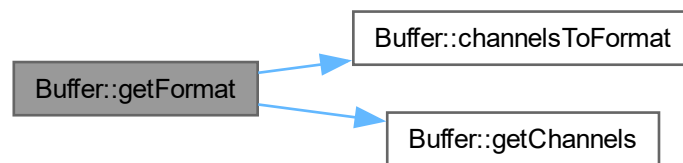
格納されているフレームのフォーマットを得る

戻り値

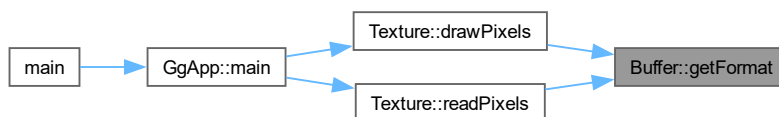
格納されているフレームのフォーマット

[Buffer.h](#) の 274 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.12 getHeight()

```
GLsizei Buffer::getHeight () const [inline]
```

格納されているフレームの縦の画素数を得る

戻り値

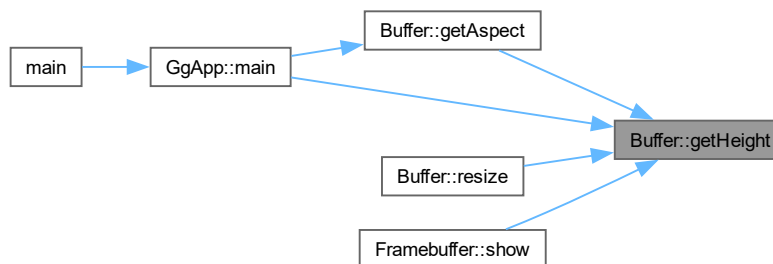
格納されているフレームの縦の画素数

[Buffer.h](#) の 254 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.13 getSize()

```
virtual const std::array< GLsizei, 2 > & Buffer::getSize () const [inline], [virtual]
```

格納されているフレームのサイズを得る

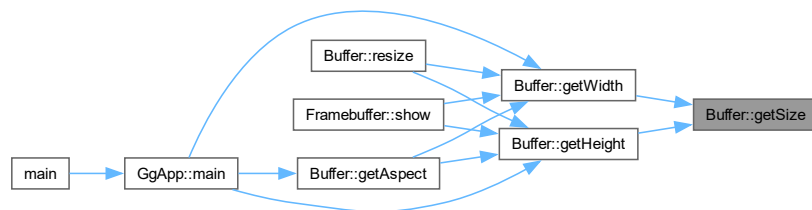
戻り値

格納されているフレームのサイズ

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.h](#) の 234 行目に定義があります。

被呼び出し関係図:



8.1.3.14 getWidth()

```
GLsizei Buffer::getWidth () const [inline]
```

格納されているフレームの横の画素数を得る

戻り値

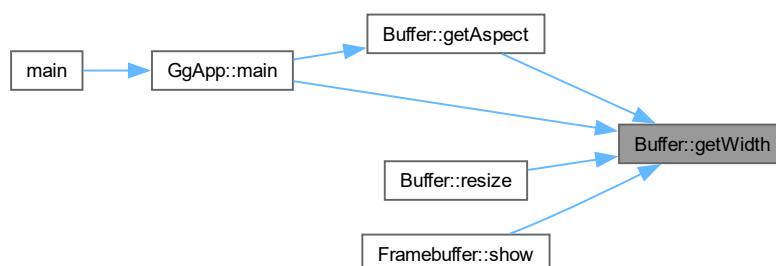
格納されているフレームの横の画素数

[Buffer.h](#) の 244 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.15 map()

```
GLvoid * Buffer::map () const
```

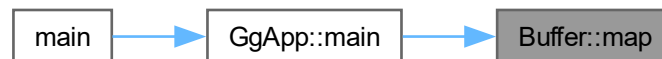
バッファのピクセルバッファオブジェクトをマップする

戻り値

ピクセルバッファオブジェクトをマップしたメモリ

[Buffer.cpp](#) の [152](#) 行目に定義があります。

被呼び出し関係図:



8.1.3.16 operator=() [1/2]

```
Buffer & Buffer::operator= (
    Buffer && buffer) [noexcept]
```

ムーブ代入演算子

引数

<i>buffer</i>	ムーブ代入元のバッファ
---------------	-------------

戻り値

ムーブ代入結果のバッファの参照

[Buffer.cpp](#) の [121](#) 行目に定義があります。

呼び出し関係図:



8.1.3.17 operator=() [2/2]

```
Buffer & Buffer::operator= (  
    const Buffer & buffer)
```

代入演算子

引数

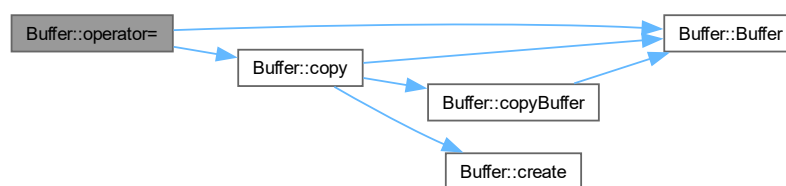
<i>buffer</i>	代入元のバッファ
---------------	----------

戻り値

代入結果のバッファの参照

[Buffer.cpp](#) の 106 行目に定義があります。

呼び出し関係図:



8.1.3.18 resize() [1/2]

```
void Buffer::resize (  
    const Buffer & buffer) [inline]
```

オブジェクトが保持するフレームのサイズを引数に指定したオブジェクトと同じにする

引数

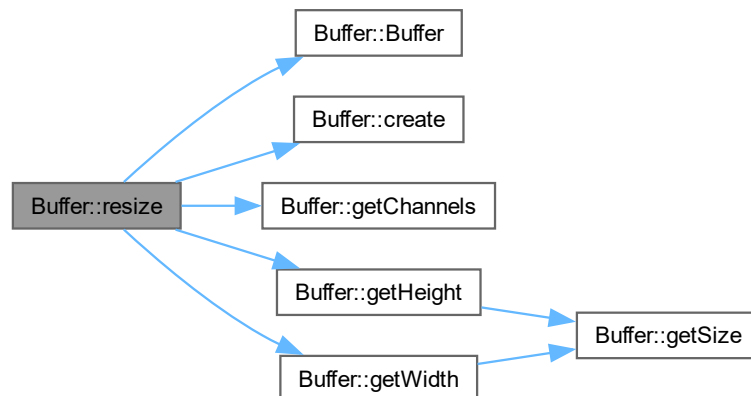
<i>buffer</i>	サイズの基準に用いるオブジェクト
---------------	------------------

覚え書き

このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

Buffer.h の 194 行目に定義があります。

呼び出し関係図:



8.1.3.19 `resize()` [2/2]

```
void Buffer::resize (
    GLsizei width,
    GLsizei height,
    int channels) [inline]
```

オブジェクトが保持するフレームのサイズを変更する

引数

<i>width</i>	フレームの横の画素数
<i>height</i>	フレームの縦の画素数
<i>channels</i>	フレームのチャンネル数

覚え書き

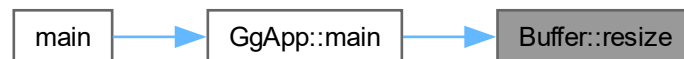
このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

[Buffer.h](#) の 180 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.20 unbindBuffer()

```
void Buffer::unbindBuffer (
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトの結合を解除する

引数

<i>target</i>	結合を解除するターゲット
---------------	--------------

[Buffer.h](#) の 306 行目に定義があります。

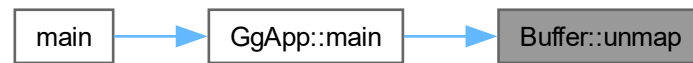
8.1.3.21 unmap()

```
void Buffer::unmap () const
```

バッファのピクセルバッファオブジェクトをアンマップする

[Buffer.cpp](#) の 173 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Buffer.h](#)
- [Buffer.cpp](#)

8.2 Calibration クラス

```
#include <Calibration.h>
```

公開メンバ関数

- [Calibration](#) (const std::string &dictionaryName, const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- [Calibration](#) (const [Calibration](#) &calibration)=delete
- virtual ~[Calibration](#) ()
- [Calibration](#) & operator= (const [Calibration](#) &calibration)=delete
- void [createBoard](#) (const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- void [setDictionary](#) (const std::string &dictionaryName, const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- void [drawBoard](#) (cv::Mat &boardImage, int width, int height)
- void [detectBoard](#) (cv::Mat &image)
- void [detectMarkers](#) (cv::Mat &image, float markerLength)
- void [recordCorners](#) ()
- void [discardCorners](#) ()
- bool [calibrate](#) ()
- auto [getCornersCount](#) () const
- auto [getSampleCount](#) () const
- auto [getTotalCount](#) () const
- const auto & [getCameraMatrix](#) () const
- const auto & [getDistortionCoefficients](#) () const
- auto [getReprojectionError](#) () const
- auto [finished](#) ()
- [GgMatrix RvecTvecToPose](#) (const cv::Vec3d &rvec, const cv::Vec3d &tvec)
- void [getAllMarkerPoses](#) (float markerLength, std::map< int, [GgMatrix](#) > &poses)
- bool [loadParameters](#) (const std::string &filename)
- bool [saveParameters](#) (const std::string &filename) const

静的公開変数類

- static const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > [dictionaryList](#)
ArUco Marker 辞書のリスト

8.2.1 詳解

較正クラス

[Calibration.h](#) の 24 行目に定義があります。

8.2.2 構築子と解体子

8.2.2.1 Calibration() [1/2]

```
Calibration::Calibration (
    const std::string & dictionaryName,
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
```

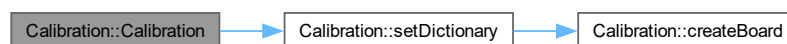
較正オブジェクトのコンストラクタ

引数

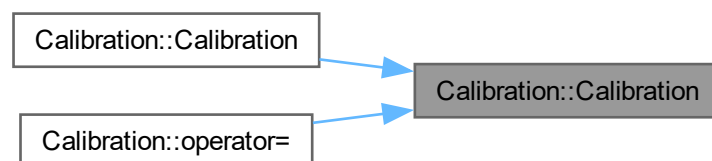
<i>dictionaryName</i>	ArUco Marker の辞書名
<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 29 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.2.2 Calibration() [2/2]

```
Calibration::Calibration (
    const Calibration & calibration) [delete]
```

コピーコンストラクタは使用しない

引数

<i>calibration</i>	コピー元
--------------------	------

呼び出し関係図:



8.2.2.3 ~Calibration()

```
Calibration::~~Calibration () [virtual]
```

デストラクタ

[Calibration.cpp](#) の 39 行目に定義があります。

8.2.3 関数詳解

8.2.3.1 calibrate()

```
bool Calibration::calibrate ()
```

校正する

戻り値

校正に成功したら true

[Calibration.cpp](#) の 242 行目に定義があります。

呼び出し関係図:



8.2.3.2 createBoard()

```
void Calibration::createBoard (
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
```

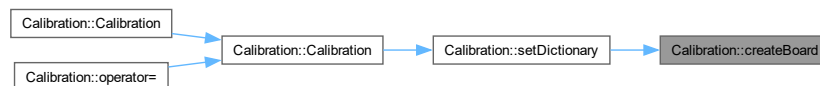
ChArUco Board を作成する

引数

<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 46 行目に定義があります。

被呼び出し関係図:



8.2.3.3 detectBoard()

```
void Calibration::detectBoard (
    cv::Mat & image)
```

ChArUco Board を検出する

引数

<i>image</i>	ChArUco Board を検出する画像
--------------	-----------------------

[Calibration.cpp](#) の 97 行目に定義があります。

被呼び出し関係図:



8.2.3.4 detectMarkers()

```
void Calibration::detectMarkers (
    cv::Mat & image,
    float markerLength)
```

ArUco Marker を検出する

引数

<i>image</i>	ArUco Marker を検出する画像
<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 133 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.5 discardCorners()

```
void Calibration::discardCorners ()
```

取得した標本と校正結果を破棄する

[Calibration.cpp](#) の 222 行目に定義があります。

被呼び出し関係図:



8.2.3.6 drawBoard()

```
void Calibration::drawBoard (
    cv::Mat & boardImage,
    int width,
    int height)
```

ChArUco Board を描く

引数

<i>boardImage</i>	ChArUco Board を描き込む画像
<i>width</i>	ChArUco Board の横の画素数
<i>height</i>	ChArUco Board の縦の画素数

[Calibration.cpp](#) の 89 行目に定義があります。

8.2.3.7 finished()

```
auto Calibration::finished () [inline]
```

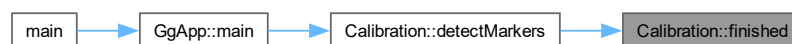
較正が完了したかどうかを調べる

戻り値

較正が完了していたら true

[Calibration.h](#) の 241 行目に定義があります。

被呼び出し関係図:



8.2.3.8 getAllMarkerPoses()

```
void Calibration::getAllMarkerPoses (
    float markerLength,
    std::map< int, GgMatrix > & poses)
```

ArUco Marker の 3 次元姿勢の変換行列を求める

引数

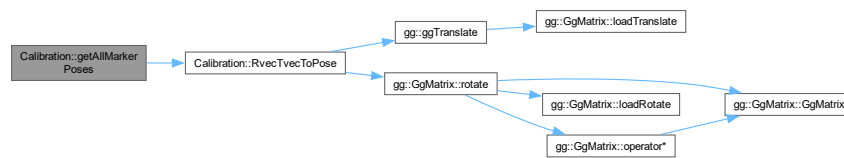
<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)
<i>poses</i>	推定した ArUco Marker の 3 次元姿勢

覚え書き

これはキャリブレーション終了後に単独マーカの位置推定に用いる

[Calibration.cpp](#) の 335 行目に定義があります。

呼び出し関係図:



8.2.3.9 getCameraMatrix()

```
const auto & Calibration::getCameraMatrix () const [inline]
```

カメラ行列を取り出す

戻り値

カメラ行列

[Calibration.h](#) の 211 行目に定義があります。

8.2.3.10 getCornersCount()

```
auto Calibration::getCornersCount () const [inline]
```

検出数を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 181 行目に定義があります。

8.2.3.11 getDistortionCoefficients()

```
const auto & Calibration::getDistortionCoefficients () const [inline]
```

歪パラメータを取り出す

戻り値

歪パラメータ

[Calibration.h](#) の 221 行目に定義があります。

8.2.3.12 getReprojectionError()

```
auto Calibration::getReprojectionError () const [inline]
```

再投影誤差を得る

戻り値

再投影誤差

[Calibration.h](#) の 231 行目に定義があります。

8.2.3.13 getSampleCount()

```
auto Calibration::getSampleCount () const [inline]
```

標本数を取得する

戻り値

保存された標本の数

[Calibration.h](#) の 191 行目に定義があります。

8.2.3.14 getTotalCount()

```
auto Calibration::getTotalCount () const [inline]
```

検出数の合計を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 201 行目に定義があります。

8.2.3.15 loadParameters()

```
bool Calibration::loadParameters (  
    const std::string & filename)
```

ファイルからキャリブレーションパラメータを読み込む

引数

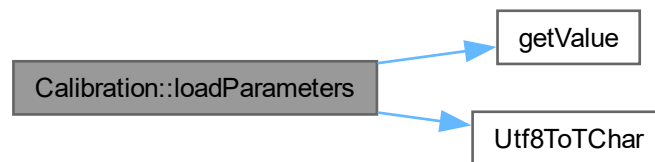
<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの読み込みに成功したら true

[Calibration.cpp](#) の 419 行目に定義があります。

呼び出し関係図:



8.2.3.16 operator=()

```
Calibration & Calibration::operator= (
    const Calibration & calibration) [delete]
```

代入演算子は使用しない

引数

<i>calibration</i>	代入元
--------------------	-----

呼び出し関係図:



8.2.3.17 recordCorners()

```
void Calibration::recordCorners ()
```

標本を取得する

[Calibration.cpp](#) の 191 行目に定義があります。

8.2.3.18 RvecTvecToPose()

```
GgMatrix Calibration::RvecTvecToPose (
    const cv::Vec3d & rvec,
    const cv::Vec3d & tvec)
```

回転ベクトルと並進ベクトルから姿勢の変換行列を求める

引数

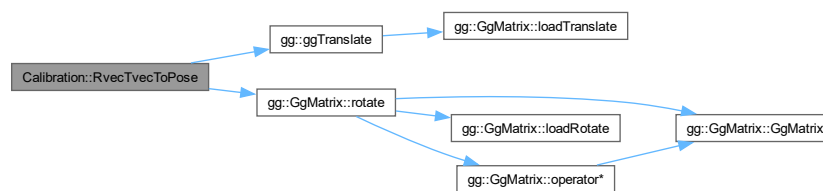
<i>rvec</i>	回転ベクトル
<i>tvec</i>	並進ベクトル

戻り値

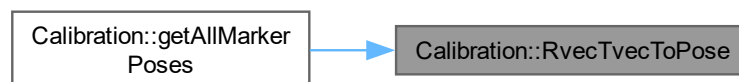
姿勢の変換行列

[Calibration.cpp](#) の 286 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.19 saveParameters()

```
bool Calibration::saveParameters (
    const std::string & filename) const
```

キャリブレーションパラメータをファイルに保存する

引数

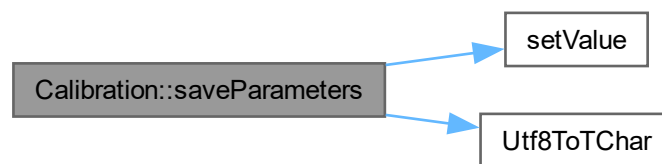
<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの書き込みに成功したら true

Calibration.cpp の 455 行目に定義があります。

呼び出し関係図:



8.2.3.20 setDictionary()

```

void Calibration::setDictionary (
    const std::string & dictionaryName,
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
  
```

ArUco Marker の辞書と検出器を設定する

引数

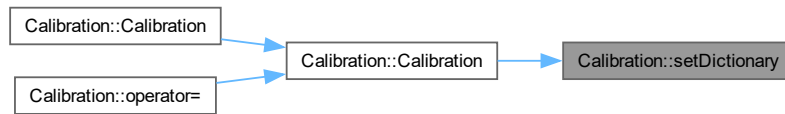
<i>dictionaryName</i>	ArUco Marker の辞書名
<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

Calibration.cpp の 66 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.4 メンバ詳解

8.2.4.1 dictionaryList

```
const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > Calibration←
::dictionaryList [static]
```

初期値:

```
{
  { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
  { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
  { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
  { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
  { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
  { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
  { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
  { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
  { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
  { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
  { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
  { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
  { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
  { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
  { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
  { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
  { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
  { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
  { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
  { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
  { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 }
}
```

ArUco Marker 辞書のリスト

[Calibration.h](#) の 483 行目に定義があります。

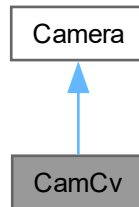
このクラス詳解は次のファイルから抽出されました:

- [Calibration.h](#)
- [Calibration.cpp](#)

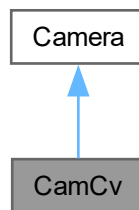
8.3 CamCv クラス

```
#include <CamCv.h>
```

CamCv の継承関係図



CamCv 連携図



公開メンバ関数

- `CamCv ()`=default
- `virtual ~CamCv ()`
- `auto open (int device, int width=0, int height=0, double fps=0.0, const char *fourcc="", int pref=cv::CAP_ANY)`
- `void close ()`
- `auto open (const std::string &file, int width=0, int height=0, double fps=0.0, const char *fourcc="", int pref=cv::CAP_ANY)`
- `virtual double getFps () const`
- `auto getCodec () const`
- `void getCodec (char *fourcc) const`
- `auto getPosition () const`
- `void setPosition (double frame)`
- `void setExposure (double exposure)`
- `void increaseExposure ()`
- `void decreaseExposure ()`
- `void setGain (double gain)`
- `void increaseGain ()`
- `void decreaseGain ()`

基底クラス **Camera** に属する継承公開メンバ関数

- **Camera** ()=default
- **Camera** (const **Camera** &camera)=delete
- virtual ~**Camera** ()
- **Camera** & operator= (const **Camera** &camera)=delete
- void **start** ()
- virtual void **stop** ()
- bool **transmit** (GLuint buffer)
- void **transmit** (std::vector< GLubyte > &buffer)
- template<typename MatType>
bool **transmit** (MatType &buffer)
- bool **isRunning** () const
- void **setPrioritizeLatency** (bool mode)
- bool **getPrioritizeLatency** () const
- std::array< int, 2 > **getSize** () const
- auto **getWidth** () const
- auto **getHeight** () const
- auto **getChannels** () const
- auto **getFrames** () const

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- double **in** { -1.0 }
ムービーファイルのインポイント
- double **out** { -1.0 }
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開変数類

- double **total** { -1.0 }
ムービーファイルの総フレーム数
- double **interval** { 10.0 }
キャプチャした画像のフレーム間隔
- int **width** { 0 }
解像度 (幅)
- int **height** { 0 }
解像度 (高さ)
- int **channels** { 0 }
チャンネル数
- std::vector< GLubyte > **frame**
キャプチャデバイスから取得したフレーム
- std::vector< GLubyte > **image**
キャプチャしたフレームを GPU に送るために用いる一時メモリ
- std::atomic< bool > **prioritizeLatency** { false }
レイテンシを優先するなら true
- std::atomic< bool > **captured** { false }
新しいフレームが取得されたら true
- bool **reusableFrame** { false }

- 再転送可能なフレーム（静止画像等）なら *true*
- `std::thread thr`
キャプチャを非同期に行うためのスレッド
- `std::mutex mtx`
キャプチャを非同期に行うためのミューテックス
- `std::atomic< bool > running { false }`
キャプチャスレッドが実行中なら *true*

8.3.1 詳解

OpenCV を使ってビデオをキャプチャするクラス

[CamCv.h](#) の 20 行目に定義があります。

8.3.2 構築子と解体子

8.3.2.1 CamCv()

```
CamCv::CamCv () [default]
```

コンストラクタ

8.3.2.2 ~CamCv()

```
virtual CamCv::~~CamCv () [inline], [virtual]
```

デストラクタ

[CamCv.h](#) の 198 行目に定義があります。

8.3.3 関数詳解

8.3.3.1 close()

```
void CamCv::close () [inline], [virtual]
```

キャプチャデバイスの使用を終了する

[Camera](#) を再実装しています。

[CamCv.h](#) の 222 行目に定義があります。

呼び出し関係図:



8.3.3.2 decreaseExposure()

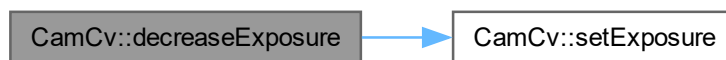
```
void CamCv::decreaseExposure () [inline], [virtual]
```

露出を一段階下げる

Cameraを再実装しています。

CamCv.h の 325 行目に定義があります。

呼び出し関係図:



8.3.3.3 decreaseGain()

```
void CamCv::decreaseGain () [inline], [virtual]
```

利得を一段階下げる

Cameraを再実装しています。

CamCv.h の 351 行目に定義があります。

呼び出し関係図:



8.3.3.4 getCodec() [1/2]

```
auto CamCv::getCodec () const [inline]
```

コーデックを調べる

戻り値

使用しているコーデックを表す 4 バイト

[CamCv.h](#) の 263 行目に定義があります。

被呼び出し関係図:

CamCv::getCodec



CamCv::getCodec

8.3.3.5 getCodec() [2/2]

```
void CamCv::getCodec (  
    char * fourcc) const [inline]
```

コーデックを調べる

引数

<i>fourcc</i>	使用しているコーデックを表す 4 文字の格納先
---------------	-------------------------

[CamCv.h](#) の 273 行目に定義があります。

呼び出し関係図:

CamCv::getCodec



CamCv::getCodec

8.3.3.6 getFps()

```
virtual double CamCv::getFps () const [inline], [virtual]
```

キャプチャしたフレームのフレームレートを得る

戻り値

キャプチャしたフレームのフレームレート

[Camera](#)を再実装しています。

[CamCv.h](#) の 253 行目に定義があります。

8.3.3.7 getPosition()

```
auto CamCv::getPosition () const [inline]
```

ファイルから入力しているとき現在のフレーム番号を得る

戻り値

現在入力しているフレーム番号

[CamCv.h](#) の 289 行目に定義があります。

8.3.3.8 increaseExposure()

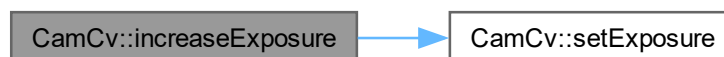
```
void CamCv::increaseExposure () [inline], [virtual]
```

露出を一段階上げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 317 行目に定義があります。

呼び出し関係図:



8.3.3.9 increaseGain()

```
void CamCv::increaseGain () [inline], [virtual]
```

利得を一段階上げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 343 行目に定義があります。

呼び出し関係図:



8.3.3.10 open() [1/2]

```
auto CamCv::open (
    const std::string & file,
    int width = 0,
    int height = 0,
    double fps = 0.0,
    const char * fourcc = "",
    int pref = cv::CAP_ANY) [inline]
```

ファイルまたはネットワークから入力する

引数

<i>file</i>	入力するファイルまたはネットワーク URL
<i>width</i>	入力するファイルを開く際に期待するフレームの横の画素数, 0 ならお任せ
<i>height</i>	入力するファイルを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	入力するファイルを開く際に期待するフレームフレームレート, 0 ならお任せ
<i>fourcc</i>	入力するファイルを開く際に期待するコーデックの 4 文字, "" ならお任せ
<i>pref</i>	OpenCV が使用する VideoCapture バックエンド

戻り値

入力するファイルが使用可能なら true

[CamCv.h](#) の 242 行目に定義があります。

8.3.3.11 open() [2/2]

```
auto CamCv::open (
    int device,
    int width = 0,
    int height = 0,
    double fps = 0.0,
    const char * fourcc = "",
    int pref = cv::CAP_ANY) [inline]
```

キャプチャデバイスを開く

引数

<i>device</i>	キャプチャデバイスの番号
<i>width</i>	キャプチャデバイスを開く際に期待するフレームの横の画素数, 0 ならお任せ
<i>height</i>	キャプチャデバイスを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	キャプチャデバイスを開く際に期待するフレームフレームレート, 0 ならお任せ
<i>fourcc</i>	キャプチャデバイスを開く際に期待するコーデックの 4 文字, "" ならお任せ
<i>pref</i>	OpenCV が使用する VideoCapture バックエンド

戻り値

キャプチャデバイスが使用可能なら true

[CamCv.h](#) の 213 行目に定義があります。

8.3.3.12 setExposure()

```
void CamCv::setExposure (
    double exposure) [inline]
```

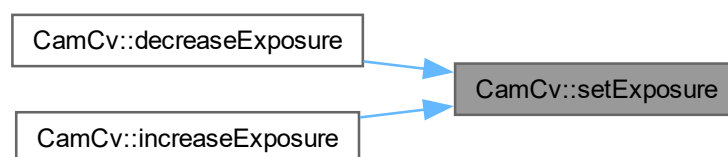
露出を設定する

引数

<i>exposure</i>	設定する露出
-----------------	--------

[CamCv.h](#) の 309 行目に定義があります。

被呼び出し関係図:



8.3.3.13 setGain()

```
void CamCv::setGain (
    double gain) [inline]
```

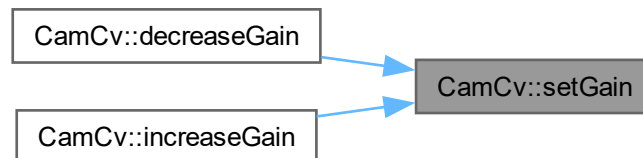
利得を設定する

引数

<i>gain</i>	設定する利得
-------------	--------

[CamCv.h](#) の 335 行目に定義があります。

被呼び出し関係図:



8.3.3.14 setPosition()

```
void CamCv::setPosition (
    double frame) [inline]
```

ファイルから入力しているとき再生位置を指定する

引数

<i>frame</i>	再生位置
--------------	------

[CamCv.h](#) の 299 行目に定義があります。

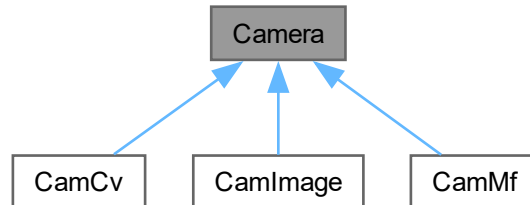
このクラス詳解は次のファイルから抽出されました:

- [CamCv.h](#)

8.4 Camera クラス

```
#include <Camera.h>
```

Camera の継承関係図



公開メンバ関数

- `Camera ()=default`
- `Camera (const Camera &camera)=delete`
- `virtual ~Camera ()`
- `Camera & operator= (const Camera &camera)=delete`
- `void start ()`
- `virtual void stop ()`
- `bool transmit (GLuint buffer)`
- `void transmit (std::vector< GLubyte > &buffer)`
- `template<typename MatType>`
`bool transmit (MatType &buffer)`
- `virtual void close ()`
- `bool isRunning () const`
- `void setPrioritizeLatency (bool mode)`
- `bool getPrioritizeLatency () const`
- `std::array< int, 2 > getSize () const`
- `auto getWidth () const`
- `auto getHeight () const`
- `auto getChannels () const`
- `auto getFrames () const`
- `virtual double getFps () const`
- `virtual void increaseExposure ()`
- `virtual void decreaseExposure ()`
- `virtual void increaseGain ()`
- `virtual void decreaseGain ()`

公開変数類

- `double in { -1.0 }`
ムービーファイルのインポイント
- `double out { -1.0 }`
ムービーファイルのアウトポイント

限定公開 メンバ関数

- virtual void `capture` ()

限定公開変数類

- double `total` { -1.0 }
ムービーファイルの総フレーム数
- double `interval` { 10.0 }
キャプチャした画像のフレーム間隔
- int `width` { 0 }
解像度 (幅)
- int `height` { 0 }
解像度 (高さ)
- int `channels` { 0 }
チャンネル数
- std::vector< GLubyte > `frame`
キャプチャデバイスから取得したフレーム
- std::vector< GLubyte > `image`
キャプチャしたフレームを GPU に送るために用いる一時メモリ
- std::atomic< bool > `prioritizeLatency` { false }
レイテンシを優先するなら `true`
- std::atomic< bool > `captured` { false }
新しいフレームが取得されたら `true`
- bool `reusableFrame` { false }
再転送可能なフレーム (静止画像等) なら `true`
- std::thread `thr`
キャプチャを非同期に行うためのスレッド
- std::mutex `mtx`
キャプチャを非同期に行うためのミューテックス
- std::atomic< bool > `running` { false }
キャプチャスレッドが実行中なら `true`

8.4.1 詳解

キャプチャデバイス関連の基底クラス

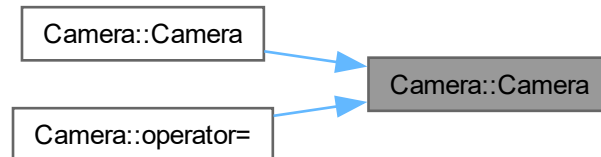
`Camera.h` の 57 行目に定義があります。

8.4.2 構築子と解体子

8.4.2.1 Camera() [1/2]

```
Camera::Camera () [default]
```

コンストラクタ 被呼び出し関係図:



8.4.2.2 Camera() [2/2]

```
Camera::Camera (  
    const Camera & camera) [delete]
```

コピーコンストラクタは使用しない

引数

<code>camera</code>	コピー元
---------------------	------

呼び出し関係図:



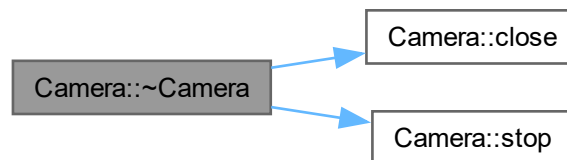
8.4.2.3 ~Camera()

```
virtual Camera::~Camera () [inline], [virtual]
```

デストラクタ

[Camera.h](#) の 135 行目に定義があります。

呼び出し関係図:



8.4.3 関数詳解

8.4.3.1 capture()

```
virtual void Camera::capture () [inline], [protected], [virtual]
```

フレームをキャプチャする

スレッドを起動するための仮想関数。

[CamMf](#)で再実装されています。

[Camera.h](#) の 106 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.4.3.2 close()

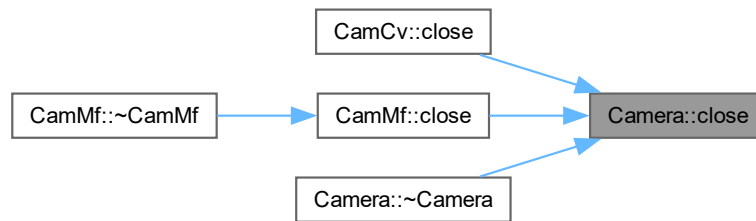
```
virtual void Camera::close () [inline], [virtual]
```

キャプチャデバイスの使用を終了する

[CamCv](#), [CamMf](#)で再実装されています。

[Camera.h](#) の 270 行目に定義があります。

被呼び出し関係図:



8.4.3.3 decreaseExposure()

```
virtual void Camera::decreaseExposure () [inline], [virtual]
```

露出を下げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 376 行目に定義があります。

8.4.3.4 decreaseGain()

```
virtual void Camera::decreaseGain () [inline], [virtual]
```

利得を下げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 390 行目に定義があります。

8.4.3.5 getChannels()

```
auto Camera::getChannels () const [inline]
```

キャプチャしたフレームのチャンネル数を調べる

戻り値

キャプチャしたフレームのチャンネル数

[Camera.h](#) の 341 行目に定義があります。

8.4.3.6 getFps()

```
virtual double Camera::getFps () const [inline], [virtual]
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[CamCv](#)で再実装されています。

[Camera.h](#) の 361 行目に定義があります。

8.4.3.7 getFrames()

```
auto Camera::getFrames () const [inline]
```

ムービーファイルの総フレーム数を得る

戻り値

ムービーファイルの総フレーム数

[Camera.h](#) の 351 行目に定義があります。

8.4.3.8 getHeight()

```
auto Camera::getHeight () const [inline]
```

キャプチャしたフレームの縦の画素数を得る

戻り値

キャプチャ中のフレームの縦の画素数

[Camera.h](#) の 331 行目に定義があります。

8.4.3.9 getPrioritizeLatency()

```
bool Camera::getPrioritizeLatency () const [inline]
```

レイテンシ優先モードかどうか調べる

戻り値

レイテンシを優先する場合は true

[Camera.h](#) の 301 行目に定義があります。

8.4.3.10 getSize()

```
std::array< int, 2 > Camera::getSize () const [inline]
```

キャプチャしたフレームのサイズを得る

戻り値

キャプチャしたフレームのサイズ

[Camera.h](#) の 311 行目に定義があります。

8.4.3.11 getWidth()

```
auto Camera::getWidth () const [inline]
```

キャプチャしたフレームの横の画素数を得る

戻り値

キャプチャ中のフレームの横の画素数

[Camera.h](#) の 321 行目に定義があります。

8.4.3.12 increaseExposure()

```
virtual void Camera::increaseExposure () [inline], [virtual]
```

露出を上げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 369 行目に定義があります。

8.4.3.13 increaseGain()

```
virtual void Camera::increaseGain () [inline], [virtual]
```

利得を上げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 383 行目に定義があります。

8.4.3.14 isRunning()

```
bool Camera::isRunning () const [inline]
```

キャプチャスレッドが実行中かどうか調べる

戻り値

キャプチャ中なら true

[Camera.h](#) の 281 行目に定義があります。

8.4.3.15 operator=()

```
Camera & Camera::operator= (  
    const Camera & camera) [delete]
```

代入演算子は使用しない

引数

<i>camera</i>	代入元のカメラ
---------------	---------

戻り値

代入後のこのカメラの参照

呼び出し関係図:



8.4.3.16 setPrioritizeLatency()

```
void Camera::setPrioritizeLatency (  
    bool mode) [inline]
```

レイテンシ優先モードを設定する

引数

<i>mode</i>	レイテンシを優先する場合は true
-------------	--------------------

[Camera.h](#) の 291 行目に定義があります。

8.4.3.17 start()

```
void Camera::start () [inline]
```

キャプチャスレッドを起動する

[Camera.h](#) の 158 行目に定義があります。

呼び出し関係図:



8.4.3.18 stop()

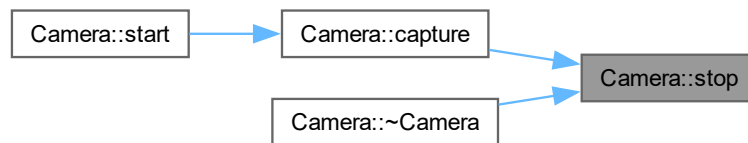
```
virtual void Camera::stop () [inline], [virtual]
```

キャプチャスレッドを停止する

CamMfで再実装されています。

Camera.h の 170 行目に定義があります。

被呼び出し関係図:



8.4.3.19 transmit() [1/3]

```
bool Camera::transmit (
    GLuint buffer) [inline]
```

キャプチャデバイスをロックしてフレームをピクセルバッファオブジェクトに転送する

引数

<i>buffer</i>	転送先のピクセルバッファオブジェクト
---------------	--------------------

戻り値

新しいフレームを転送できたら true

Camera.h の 189 行目に定義があります。

8.4.3.20 transmit() [2/3]

```
template<typename MatType>
bool Camera::transmit (
    MatType & buffer) [inline]
```

キャプチャデバイスをロックしてフレームをメモリに転送する

引数

<i>buffer</i>	転送先のメモリ (cv::Matなど)
---------------	---------------------

戻り値

新しいフレームを転送できたら true

[Camera.h](#) の 246 行目に定義があります。

8.4.3.21 transmit() [3/3]

```
void Camera::transmit (
    std::vector< GLubyte > & buffer) [inline]
```

キャプチャデバイスをロックしてフレームをメモリに転送する

引数

<i>buffer</i>	転送先のメモリ
---------------	---------

[Camera.h](#) の 219 行目に定義があります。

8.4.4 メンバ詳解

8.4.4.1 captured

```
std::atomic<bool> Camera::captured { false } [protected]
```

新しいフレームが取得されたら true

[Camera.h](#) の 86 行目に定義があります。

8.4.4.2 channels

```
int Camera::channels { 0 } [protected]
```

チャンネル数

[Camera.h](#) の 74 行目に定義があります。

8.4.4.3 frame

```
std::vector<GLubyte> Camera::frame [protected]
```

キャプチャデバイスから取得したフレーム

[Camera.h](#) の 77 行目に定義があります。

8.4.4.4 height

```
int Camera::height { 0 } [protected]
```

解像度（高さ）

[Camera.h](#) の 71 行目に定義があります。

8.4.4.5 image

```
std::vector<GLubyte> Camera::image [protected]
```

キャプチャしたフレームを GPU に送るために用いる一時メモリ

[Camera.h](#) の 80 行目に定義があります。

8.4.4.6 in

```
double Camera::in { -1.0 }
```

ムービーファイルのインポイント

[Camera.h](#) の 115 行目に定義があります。

8.4.4.7 interval

```
double Camera::interval { 10.0 } [protected]
```

キャプチャした画像のフレーム間隔

[Camera.h](#) の 65 行目に定義があります。

8.4.4.8 mtx

```
std::mutex Camera::mtx [protected]
```

キャプチャを非同期に行うためのミューテックス

[Camera.h](#) の 95 行目に定義があります。

8.4.4.9 out

```
double Camera::out { -1.0 }
```

ムービーファイルのアウトポイント

[Camera.h](#) の 118 行目に定義があります。

8.4.4.10 prioritizeLatency

```
std::atomic<bool> Camera::prioritizeLatency { false } [protected]
```

レイテンシを優先するなら true

[Camera.h](#) の 83 行目に定義があります。

8.4.4.11 reusableFrame

```
bool Camera::reusableFrame { false } [protected]
```

再転送可能なフレーム（静止画像等）なら true

[Camera.h](#) の 89 行目に定義があります。

8.4.4.12 running

```
std::atomic<bool> Camera::running { false } [protected]
```

キャプチャスレッドが実行中なら true

[Camera.h](#) の 98 行目に定義があります。

8.4.4.13 thr

```
std::thread Camera::thr [protected]
```

キャプチャを非同期に行うためのスレッド

[Camera.h](#) の 92 行目に定義があります。

8.4.4.14 total

```
double Camera::total { -1.0 } [protected]
```

ムービーファイルの総フレーム数

[Camera.h](#) の 62 行目に定義があります。

8.4.4.15 width

```
int Camera::width { 0 } [protected]
```

解像度（幅）

[Camera.h](#) の 68 行目に定義があります。

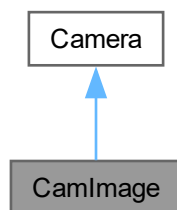
このクラス詳解は次のファイルから抽出されました:

- [Camera.h](#)

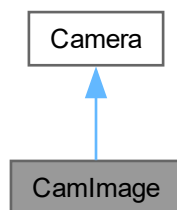
8.5 CamImage クラス

```
#include <CamImage.h>
```

CamImage の継承関係図



CamImage 連携図



公開メンバ関数

- [CamImage \(\)](#)
- [CamImage \(std::string &filename, bool flip=false\)](#)
- virtual [~CamImage \(\)](#)
- bool [open](#) (const std::string &filename, bool flip=false)
- bool [isOpened](#) () const

基底クラス [Camera](#) に属する継承公開メンバ関数

- [Camera \(\)](#)=default
- [Camera](#) (const [Camera](#) &camera)=delete
- virtual [~Camera](#) ()
- [Camera & operator=](#) (const [Camera](#) &camera)=delete
- void [start](#) ()

- virtual void `stop` ()
- bool `transmit` (GLuint buffer)
- void `transmit` (std::vector< GLubyte > &buffer)
- template<typename MatType>
bool `transmit` (MatType &buffer)
- virtual void `close` ()
- bool `isRunning` () const
- void `setPrioritizeLatency` (bool mode)
- bool `getPrioritizeLatency` () const
- std::array< int, 2 > `getSize` () const
- auto `getWidth` () const
- auto `getHeight` () const
- auto `getChannels` () const
- auto `getFrames` () const
- virtual double `getFps` () const
- virtual void `increaseExposure` ()
- virtual void `decreaseExposure` ()
- virtual void `increaseGain` ()
- virtual void `decreaseGain` ()

静的公開メンバ関数

- static bool `load` (const std::string &filename, cv::Mat &frame)
- static bool `save` (const std::string &filename, const cv::Mat &image)

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- double `in` { -1.0 }
ムービーファイルのインポイント
- double `out` { -1.0 }
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開メンバ関数

- virtual void `capture` ()

基底クラス **Camera** に属する継承限定公開変数類

- double `total` { -1.0 }
ムービーファイルの総フレーム数
- double `interval` { 10.0 }
キャプチャした画像のフレーム間隔
- int `width` { 0 }
解像度 (幅)
- int `height` { 0 }
解像度 (高さ)
- int `channels` { 0 }

- チャンネル数
- `std::vector< GLubyte > frame`
キャプチャデバイスから取得したフレーム
- `std::vector< GLubyte > image`
キャプチャしたフレームを *GPU* に送るために用いる一時メモリ
- `std::atomic< bool > prioritizeLatency { false }`
レイテンシを優先するなら *true*
- `std::atomic< bool > captured { false }`
新しいフレームが取得されたら *true*
- `bool reusableFrame { false }`
再転送可能なフレーム（静止画像等）なら *true*
- `std::thread thr`
キャプチャを非同期に行うためのスレッド
- `std::mutex mtx`
キャプチャを非同期に行うためのミューテックス
- `std::atomic< bool > running { false }`
キャプチャスレッドが実行中なら *true*

8.5.1 詳解

OpenCV を使って画像ファイルを読み込むクラス

[CamImage.h](#) の 23 行目に定義があります。

8.5.2 構築子と解体子

8.5.2.1 CamImage() [1/2]

```
CamImage::CamImage () [inline]
```

コンストラクタ

[CamImage.h](#) の 30 行目に定義があります。

8.5.2.2 CamImage() [2/2]

```
CamImage::CamImage (
    std::string & filename,
    bool flip = false) [inline]
```

画像ファイルを開いて読み込むコンストラクタ

引数

<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは <i>true</i>

[CamImage.h](#) の 40 行目に定義があります。

呼び出し関係図:



8.5.2.3 ~CamImage()

```
virtual CamImage::~CamImage () [inline], [virtual]
```

デストラクタ

[CamImage.h](#) の 48 行目に定義があります。

8.5.3 関数詳解

8.5.3.1 isOpened()

```
bool CamImage::isOpened () const [inline]
```

画像ファイルが開けたかどうか調べる

戻り値

ファイルが開けていたら true

[CamImage.h](#) の 98 行目に定義があります。

8.5.3.2 load()

```
bool CamImage::load (
    const std::string & filename,
    cv::Mat & frame) [inline], [static]
```

画像ファイルを読み込む

引数

<i>filename</i>	読み込む画像ファイルのパス
<i>frame</i>	読み込んだ画像

戻り値

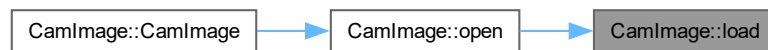
読み込みに成功したら true

[CamImage.h](#) の 111 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.3 open()

```

bool CamImage::open (
    const std::string & filename,
    bool flip = false) [inline]
  
```

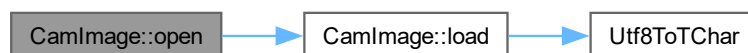
画像ファイルを開く

引数

<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは true

[CamImage.h](#) の 58 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.4 save()

```

bool CamImage::save (
    const std::string & filename,
    const cv::Mat & image) [inline], [static]
  
```

配列に格納されている画像をファイルに保存する

引数

<i>filename</i>	保存する画像ファイルのパス
<i>image</i>	保存する画像を保持している配列

戻り値

保存に成功したら true

[CamImage.h](#) の 152 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



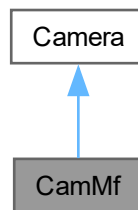
このクラス詳解は次のファイルから抽出されました:

- [CamImage.h](#)

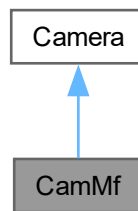
8.6 CamMf クラス

```
#include <CamMf.h>
```

CamMf の継承関係図



CamMf 連携図



公開メンバ関数

- [CamMf \(\)](#)=default
- virtual [~CamMf \(\)](#)
- bool [open](#) (int device, bool setupFormat=true)
- const auto & [getFormatList](#) () const
- bool [select](#) (int index)
- void [capture](#) ()
- virtual void [stop](#) () override
- void [close](#) ()

基底クラス **Camera** に属する継承公開メンバ関数

- **Camera** ()=default
- **Camera** (const **Camera** &camera)=delete
- virtual ~**Camera** ()
- **Camera** & operator= (const **Camera** &camera)=delete
- void **start** ()
- bool **transmit** (GLuint buffer)
- void **transmit** (std::vector< GLubyte > &buffer)
- template<typename MatType>
bool **transmit** (MatType &buffer)
- bool **isRunning** () const
- void **setPrioritizeLatency** (bool mode)
- bool **getPrioritizeLatency** () const
- std::array< int, 2 > **getSize** () const
- auto **getWidth** () const
- auto **getHeight** () const
- auto **getChannels** () const
- auto **getFrames** () const
- virtual double **getFps** () const
- virtual void **increaseExposure** ()
- virtual void **decreaseExposure** ()
- virtual void **increaseGain** ()
- virtual void **decreaseGain** ()

静的公開メンバ関数

- static const auto & **getDeviceList** ()

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- double **in** { -1.0 }
ムービーファイルのインポイント
- double **out** { -1.0 }
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開変数類

- double **total** { -1.0 }
ムービーファイルの総フレーム数
- double **interval** { 10.0 }
キャプチャした画像のフレーム間隔
- int **width** { 0 }
解像度 (幅)
- int **height** { 0 }
解像度 (高さ)
- int **channels** { 0 }
チャンネル数

- `std::vector< GLubyte > frame`
キャプチャデバイスから取得したフレーム
- `std::vector< GLubyte > image`
キャプチャしたフレームを *GPU* に送るために用いる一時メモリ
- `std::atomic< bool > prioritizeLatency { false }`
レイテンシを優先するなら *true*
- `std::atomic< bool > captured { false }`
新しいフレームが取得されたら *true*
- `bool reusableFrame { false }`
再転送可能なフレーム（静止画像等）なら *true*
- `std::thread thr`
キャプチャを非同期に行うためのスレッド
- `std::mutex mtx`
キャプチャを非同期に行うためのミューテックス
- `std::atomic< bool > running { false }`
キャプチャスレッドが実行中なら *true*

8.6.1 詳解

Microsoft Media Foundation を使ってビデオをキャプチャするクラス

[CamMf.h](#) の 25 行目に定義があります。

8.6.2 構築子と解体子

8.6.2.1 CamMf()

```
CamMf::CamMf () [default]
```

コンストラクタ

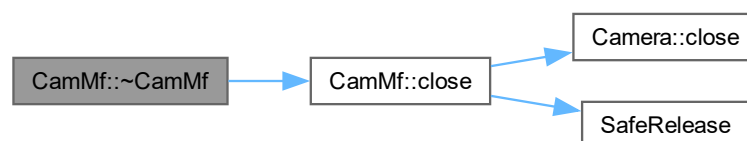
8.6.2.2 ~CamMf()

```
virtual CamMf::~CamMf () [inline], [virtual]
```

デストラクタ

[CamMf.h](#) の 229 行目に定義があります。

呼び出し関係図:



8.6.3 関数詳解

8.6.3.1 capture()

```
void CamMf::capture () [virtual]
```

フレームをキャプチャする（別スレッドでループ実行される）

[Camera](#)を再実装しています。

[CamMf.cpp](#) の 695 行目に定義があります。

呼び出し関係図:



8.6.3.2 close()

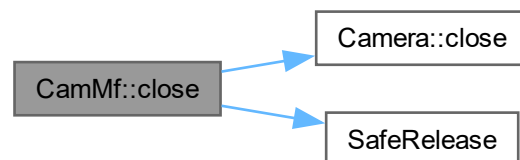
```
void CamMf::close () [virtual]
```

カメラを閉じる

[Camera](#)を再実装しています。

[CamMf.cpp](#) の 1182 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.6.3.3 getDeviceList()

```
const auto & CamMf::getDeviceList () [inline], [static]
```

Media Foundation のビデオデバイスの一覧を返す

戻り値

デバイス名のリスト

[CamMf.h](#) の 240 行目に定義があります。

被呼び出し関係図:

Config::Config



CamMf::getDeviceList

8.6.3.4 getFormatList()

```
const auto & CamMf::getFormatList () const [inline]
```

使用可能なビデオフォーマットの表示・選択情報を返す

戻り値

enumerateFormats() で作成した構造化フォーマットのリスト

[CamMf.h](#) の 259 行目に定義があります。

8.6.3.5 open()

```
bool CamMf::open (  
    int device,  
    bool setupFormat = true)
```

カメラを開く

引数

<i>device</i>	デバイスの番号
<i>setupFormat</i>	最初のフォーマットを設定するかどうか (遅延初期化時は false を指定)

戻り値

開くことができたなら true

CamMf.cpp の 628 行目に定義があります。

呼び出し関係図:



8.6.3.6 select()

```
bool CamMf::select (  
    int index)
```

フォーマットを選択して設定する

引数

<i>index</i>	選択するフォーマットのリストインデックス
--------------	----------------------

戻り値

成功したら true

CamMf.cpp の 680 行目に定義があります。

呼び出し関係図:



8.6.3.7 stop()

```
void CamMf::stop () [override], [virtual]
```

キャプチャスレッドを停止する

[Camera](#)を再実装しています。

[CamMf.cpp](#) の 1157 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [CamMf.h](#)
- [CamMf.cpp](#)

8.7 Capture クラス

```
#include <Capture.h>
```

公開メンバ関数

- [Capture](#) ()=default
- [Capture](#) (const std::string &filename)
- virtual [~Capture](#) ()
- bool [openImage](#) (const std::string &filename)
- bool [openMovie](#) (const std::string &filename, cv::VideoCaptureAPIs backend=cv::CAP_FFMPEG)
- bool [openDevice](#) (int deviceNumber, std::array< int, 2 > &size, double &fps, cv::VideoCaptureAPIs backend, char *fourcc)
- void [start](#) ()
- void [stop](#) ()
- void [close](#) ()
- [operator bool](#) () const
- bool [isOpen](#) () const
- bool [isImage](#) () const
- std::array< int, 2 > [getSize](#) () const
- double [getFps](#) () const
- bool [retrieve](#) (Buffer &buffer)
- bool [retrieve](#) (cv::Mat &frame)
- void [setPrioritizeLatency](#) (bool mode)
- bool [getPrioritizeLatency](#) () const

8.7.1 詳解

キャプチャクラス

[Capture.h](#) の 28 行目に定義があります。

8.7.2 構築子と解体子

8.7.2.1 Capture() [1/2]

```
Capture::Capture () [default]
```

キャプチャーオブジェクトのデフォルトコンストラクタ

8.7.2.2 Capture() [2/2]

```
Capture::Capture (  
    const std::string & filename) [inline]
```

キャプチャするファイルを指定するコンストラクタ

引数

<i>filename</i>	キャプチャするファイルのパス名
-----------------	-----------------

[Capture.h](#) の 53 行目に定義があります。

呼び出し関係図:



8.7.2.3 ~Capture()

```
virtual Capture::~~Capture () [inline], [virtual]
```

デストラクタ

[Capture.h](#) の 61 行目に定義があります。

呼び出し関係図:



8.7.3 関数詳解

8.7.3.1 close()

```
void Capture::close ()
```

キャプチャデバイスを閉じる

[Capture.cpp](#) の 183 行目に定義があります。

被呼び出し関係図:



8.7.3.2 getFps()

```
double Capture::getFps () const
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[Capture.cpp](#) の 205 行目に定義があります。

8.7.3.3 getPrioritizeLatency()

```
bool Capture::getPrioritizeLatency () const [inline]
```

レイテンシ優先モードかどうか調べる

戻り値

レイテンシを優先する場合は true

[Capture.h](#) の 225 行目に定義があります。

8.7.3.4 getSize()

```
std::array< int, 2 > Capture::getSize () const
```

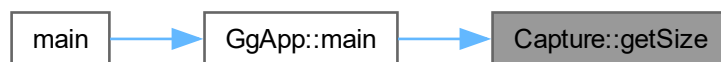
キャプチャデバイスのフレームの解像度を得る

戻り値

キャプチャデバイスのフレームの解像度

[Capture.cpp](#) の 196 行目に定義があります。

被呼び出し関係図:



8.7.3.5 isImage()

```
bool Capture::isImage () const [inline]
```

現在開いているのが静止画像かどうか

[Capture.h](#) の 174 行目に定義があります。

8.7.3.6 isOpened()

```
bool Capture::isOpened () const [inline]
```

キャプチャデバイスが有効かどうか

[Capture.h](#) の 166 行目に定義があります。

8.7.3.7 openDevice()

```
bool Capture::openDevice (  
    int deviceNumber,  
    std::array< int, 2 > & size,  
    double & fps,  
    cv::VideoCaptureAPIs backend,  
    char * fourcc)
```

キャプチャデバイスを開く (Windows以外用: OpenCV)

引数

<i>deviceNumber</i>	開くデバイス番号
<i>size</i>	キャプチャデバイスのフレームの解像度
<i>fps</i>	キャプチャデバイスのフレームレート
<i>backend</i>	バックエンドの種類
<i>fourcc</i>	コーデックの 4 文字

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 134 行目に定義があります。

8.7.3.8 openImage()

```
bool Capture::openImage (  
    const std::string & filename)
```

画像ファイルを開く

引数

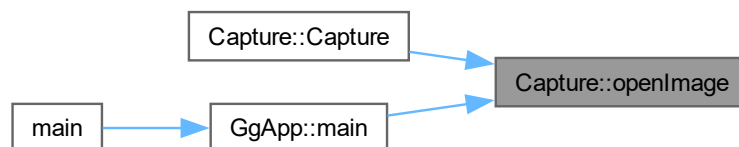
<i>filename</i>	開く画像ファイル名
-----------------	-----------

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 18 行目に定義があります。

被呼び出し関係図:



8.7.3.9 openMovie()

```
bool Capture::openMovie (  
    const std::string & filename,  
    cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG)
```

動画ファイルを開く

引数

<i>filename</i>	開く動画ファイル名
<i>backend</i>	バックエンドの種類

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 38 行目に定義があります。

8.7.3.10 operator bool()

```
Capture::operator bool () const [inline], [explicit]
```

キャプチャ中か否か

戻り値

キャプチャ中なら `true`

[Capture.h](#) の 158 行目に定義があります。

8.7.3.11 retrieve() [1/2]

```
bool Capture::retrieve (  
    Buffer & buffer)
```

新しいフレームを GPU の PBO に取得する

引数

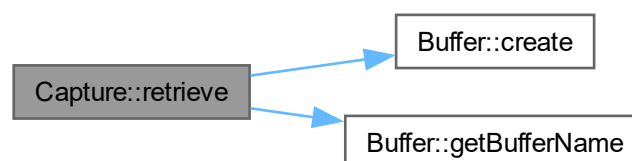
<i>buffer</i>	取得したフレームを格納するバッファ
---------------	-------------------

戻り値

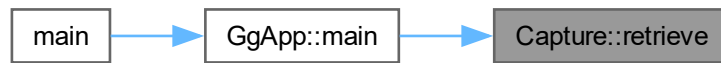
新しいフレームを取得できたなら `true`

[Capture.cpp](#) の 214 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.7.3.12 retrieve() [2/2]

```
bool Capture::retrieve (  
    cv::Mat & frame)
```

新しいフレームを CPU のメモリに取得する

引数

<i>frame</i>	取得したフレーム
--------------	----------

戻り値

新しいフレームを取得できたら true

覚え書き

OpenCV で処理する場合。

[Capture.cpp](#) の [233](#) 行目に定義があります。

8.7.3.13 setPrioritizeLatency()

```
void Capture::setPrioritizeLatency (  
    bool mode) [inline]
```

レイテンシ優先モードを設定する

引数

<i>mode</i>	レイテンシを優先する場合は true
-------------	--------------------

[Capture.h](#) の [215](#) 行目に定義があります。

8.7.3.14 start()

```
void Capture::start ()
```

キャプチャ開始

[Capture.cpp](#) の 165 行目に定義があります。

8.7.3.15 stop()

```
void Capture::stop ()
```

キャプチャ終了

[Capture.cpp](#) の 174 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Capture.h](#)
- [Capture.cpp](#)

8.8 CaptureFormat 構造体

```
#include <Camera.h>
```

公開メンバ関数

- [CaptureFormat](#) (const std::string &[resolution](#), const std::string &[fps](#), const std::string &[codec](#), int [index](#))

公開変数類

- std::string [resolution](#)
解像度の表示文字列 (例: "1920 x 1080")
- std::string [fps](#)
フレームレートの表示文字列 (例: "30.00")
- std::string [codec](#)
コーデックの表示文字列 (例: "NV12")
- int [index](#) { 0 }
バックエンドのフォーマットリストにおける選択番号

8.8.1 詳解

キャプチャデバイスが対応するビデオフォーマットの表示・選択情報

バックエンド固有のメディア型を UI が解釈し直さなくて済むよう、解像度、フレームレート、コーデックを表示用文字列として保持する。index はバックエンドが保持する実フォーマットの選択に使用する。

[Camera.h](#) の 29 行目に定義があります。

8.8.2 構築子と解体子

8.8.2.1 CaptureFormat()

```
CaptureFormat::CaptureFormat (
    const std::string & resolution,
    const std::string & fps,
    const std::string & codec,
    int index) [inline]
```

コンストラクタ

引数

<i>resolution</i>	解像度の表示文字列（例: "1920 x 1080"）
<i>fps</i>	フレームレートの表示文字列（例: "30.00"）
<i>codec</i>	コーデックの表示文字列（例: "NV12"）
<i>index</i>	バックエンドのフォーマットリストにおける選択番号

[Camera.h](#) の 44 行目に定義があります。

8.8.3 メンバ詳解

8.8.3.1 codec

```
std::string CaptureFormat::codec
```

コーデックの表示文字列（例: "NV12"）

[Camera.h](#) の 33 行目に定義があります。

8.8.3.2 fps

```
std::string CaptureFormat::fps
```

フレームレートの表示文字列（例: "30.00"）

[Camera.h](#) の 32 行目に定義があります。

8.8.3.3 index

```
int CaptureFormat::index { 0 }
```

バックエンドのフォーマットリストにおける選択番号

[Camera.h](#) の 34 行目に定義があります。

8.8.3.4 resolution

```
std::string CaptureFormat::resolution
```

解像度の表示文字列（例: "1920 x 1080"）

[Camera.h](#) の 31 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- [Camera.h](#)

8.9 Compute クラス

```
#include <Compute.h>
```

公開メンバ関数

- [Compute](#) (const char *comp)
- virtual [~Compute](#) ()
- auto [get](#) () const
- void [use](#) () const
- void [execute](#) (GLuint width, GLuint height, GLuint local_size_x=1, GLuint local_size_y=1) const

8.9.1 詳解

画像処理クラスの定義（コンピュートシェーダ版）

[Compute.h](#) の 18 行目に定義があります。

8.9.2 構築子と解体子

8.9.2.1 Compute()

```
Compute::Compute (
    const char * comp) [inline]
```

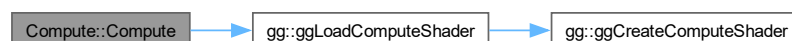
コンストラクタ

引数

<i>comp</i>	コンピュートシェーダのソースファイル名
-------------	---------------------

[Compute.h](#) の 30 行目に定義があります。

呼び出し関係図:



8.9.2.2 ~Compute()

```
virtual Compute::~Compute () [inline], [virtual]
```

デストラクタ

[Compute.h](#) の 38 行目に定義があります。

8.9.3 関数詳解

8.9.3.1 execute()

```
void Compute::execute (  
    GLuint width,  
    GLuint height,  
    GLuint local_size_x = 1,  
    GLuint local_size_y = 1) const [inline]
```

計算を実行する

引数

<i>width</i>	画像の横の画素数
<i>height</i>	画像の縦の画素数
<i>local_size_x</i>	ワークグループの横方向のスレッド数
<i>local_size_y</i>	ワークグループの縦方向のスレッド数

[Compute.h](#) の 70 行目に定義があります。

8.9.3.2 get()

```
auto Compute::get () const [inline]
```

シェーダプログラムを得る

戻り値

シェーダのプログラムオブジェクト

[Compute.h](#) の 49 行目に定義があります。

8.9.3.3 use()

```
void Compute::use () const [inline]
```

計算用のシェーダプログラムの使用を開始する

[Compute.h](#) の 57 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Compute.h](#)

8.10 Config クラス

```
#include <Config.h>
```

公開メンバ関数

- [Config](#) (const std::string &filename)
- virtual [~Config](#) ()
- void [initialize](#) ()
- bool [load](#) (const [pathString](#) &filename)
- bool [save](#) (const [pathString](#) &filename) const
- const [Settings](#) & [getSettings](#) () const
- void [setSettings](#) (const [Settings](#) &value)
- const auto & [getPreferences](#) () const
- const auto & [getBackground](#) () const
- const auto & [getMenuFont](#) () const
- auto [getMenuFontSize](#) () const
- const auto & [getTitle](#) () const
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- const auto & [getInitialImage](#) () const
- const auto & [getDictionaryName](#) () const
- const auto & [getCheckerSize](#) () const
- const auto & [getCheckerLength](#) () const
- auto [getMarkerLength](#) () const

8.10.1 詳解

構成データクラス

[Config.h](#) の 74 行目に定義があります。

8.10.2 構築子と解体子

8.10.2.1 Config()

```
Config::Config (
    const std::string & filename)
```

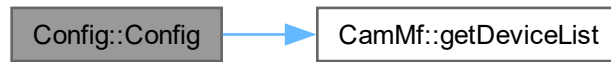
コンストラクタ

引数

<i>filename</i>	構成データの初期化に用いる JSON 形式のファイル名
-----------------	-----------------------------

[Config.cpp](#) の 26 行目に定義があります。

呼び出し関係図:



8.10.2.2 ~Config()

```
Config::~~Config () [virtual]
```

デストラクタ

[Config.cpp](#) の 67 行目に定義があります。

8.10.3 関数詳解

8.10.3.1 getBackground()

```
const auto & Config::getBackground () const [inline]
```

背景色を得る

戻り値

RGBA 形式の背景色への読み取り専用参照

[Config.h](#) の 183 行目に定義があります。

8.10.3.2 getCheckerLength()

```
const auto & Config::getCheckerLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ

[Config.h](#) の 273 行目に定義があります。

被呼び出し関係図:



8.10.3.3 getConfigCheckerSize()

```
const auto & Config::getConfigCheckerSize () const [inline]
```

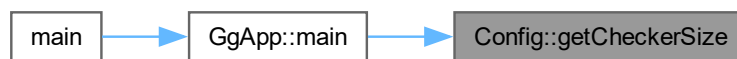
検出する ChArUco Board のマス目の横と縦の数を得る

戻り値

検出する ChArUco Board のマス目の横と縦の数

[Config.h](#) の 263 行目に定義があります。

被呼び出し関係図:



8.10.3.4 getConfigDictionaryName()

```
const auto & Config::getConfigDictionaryName () const [inline]
```

使用中の ArUco Marker の辞書名を得る

戻り値

使用中の ArUco Marker の辞書名

[Config.h](#) の 253 行目に定義があります。

被呼び出し関係図:



8.10.3.5 getHeight()

```
auto Config::getHeight () const [inline]
```

ウィンドウの高さを得る

戻り値

ウィンドウの高さ

[Config.h](#) の 233 行目に定義があります。

被呼び出し関係図:



8.10.3.6 getInitialImage()

```
const auto & Config::getInitialImage () const [inline]
```

初期画像ファイル名を得る

戻り値

初期画像ファイル名

[Config.h](#) の 243 行目に定義があります。

被呼び出し関係図:



8.10.3.7 getMarkerLength()

```
auto Config::getMarkerLength () const [inline]
```

検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Config.h](#) の 283 行目に定義があります。

8.10.3.8 getMenuFont()

```
const auto & Config::getMenuFont () const [inline]
```

メニュー用フォントのファイル名を得る

戻り値

フォントファイル名への読み取り専用参照

[Config.h](#) の 193 行目に定義があります。

8.10.3.9 getMenuFontSize()

```
auto Config::getMenuFontSize () const [inline]
```

メニュー用フォントサイズを得る

戻り値

フォントサイズ

[Config.h](#) の 203 行目に定義があります。

8.10.3.10 getPreferences()

```
const auto & Config::getPreferences () const [inline]
```

利用可能な投影方式の一覧を得る

戻り値

投影方式一覧への読み取り専用参照

[Config.h](#) の 173 行目に定義があります。

8.10.3.11 getSettings()

```
const Settings & Config::getSettings () const [inline]
```

現在の表示・較正設定を得る

戻り値

設定データへの読み取り専用参照

[Config.h](#) の 153 行目に定義があります。

8.10.3.12 getTitle()

```
const auto & Config::getTitle () const [inline]
```

ウィンドウタイトルの文字列を得る

戻り値

ウィンドウのタイトル文字列

[Config.h](#) の 213 行目に定義があります。

被呼び出し関係図:



8.10.3.13 getWidth()

```
auto Config::getWidth () const [inline]
```

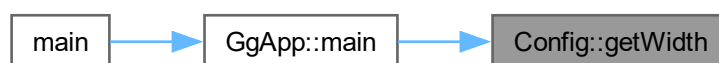
ウィンドウの横幅を得る

戻り値

ウィンドウの横幅

[Config.h](#) の 223 行目に定義があります。

被呼び出し関係図:



8.10.3.14 initialize()

```
void Config::initialize ()
```

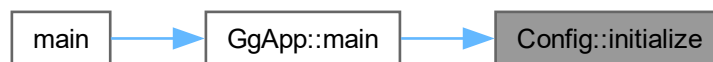
構成データを初期化する

覚え書き

OpenGL のレンダリングコンテキスト作成後に実行する

[Config.cpp](#) の 74 行目に定義があります。

被呼び出し関係図:



8.10.3.15 load()

```
bool Config::load (  
    const pathString & filename)
```

構成ファイルを読み込む

引数

<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

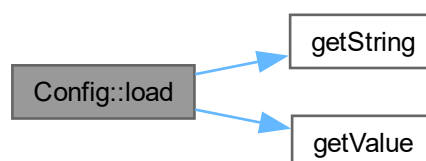
戻り値

構成ファイルの読み込みに成功したら true

ファイル内容を一時領域へ読み込んで検証し、成功した場合だけ現在の構成を置き換える。 [initialize\(\)](#) 実行後の再読み込みでは、新しい投影方式のシェーダーも構築する。

[Config.cpp](#) の 89 行目に定義があります。

呼び出し関係図:



8.10.3.16 save()

```
bool Config::save (
    const pathString & filename) const
```

構成ファイルを保存する

引数

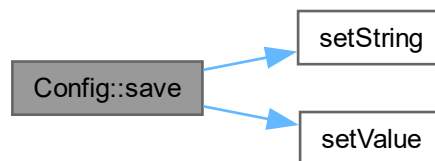
<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

戻り値

構成ファイルの保存に成功したら `true`

[Config.cpp](#) の 200 行目に定義があります。

呼び出し関係図:



8.10.3.17 setSettings()

```
void Config::setSettings (
    const Settings & value) [inline]
```

保存対象の表示・較正設定を更新する

引数

<i>value</i>	新しい設定データ
--------------	----------

[Config.h](#) の 163 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Config.h](#)
- [Config.cpp](#)
- [Menu.cpp](#)

8.11 Expand クラス

```
#include <Expand.h>
```

公開メンバ関数

- `Expand` (const std::string &vert, const std::string &frag)
- `Expand` (const `Expand` &shader)=delete
- virtual `~Expand` ()
- `Expand & operator=` (const `Expand` &shader)=delete
- auto `getProgram` () const
- std::array< GLsizei, 2 > `setup` (int samples, GLfloat aspect, const gg::GgMatrix &pose, const std::array< GLfloat, 2 > &fov, const std::array< GLfloat, 2 > ¢er, GLfloat focal, const std::array< GLfloat, 4 > &border, int unit=0) const

8.11.1 詳解

展開用シェーダクラス

`Expand.h` の 17 行目に定義があります。

8.11.2 構築子と解体子

8.11.2.1 `Expand()` [1/2]

```
Expand::Expand (
    const std::string & vert,
    const std::string & frag)
```

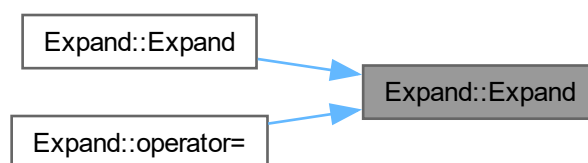
コンストラクタ

引数

<i>vert</i>	バーテックスシェーダのソースファイル名
<i>frag</i>	フラグメントシェーダのソースファイル名

`Expand.cpp` の 12 行目に定義があります。

被呼び出し関係図:



8.11.2.2 Expand() [2/2]

```
Expand::Expand (
    const Expand & shader) [delete]
```

コピーコンストラクタは使用しない

引数

<i>shader</i>	コピー元のシェーダ
---------------	-----------

呼び出し関係図:



8.11.2.3 ~Expand()

```
Expand::~Expand () [virtual]
```

デストラクタ

[Expand.cpp](#) の 29 行目に定義があります。

8.11.3 関数詳解

8.11.3.1 getProgram()

```
auto Expand::getProgram () const [inline]
```

展開用シェーダのプログラムオブジェクトを得る

戻り値

展開用シェーダのプログラムオブジェクト

[Expand.h](#) の 78 行目に定義があります。

8.11.3.2 operator=()

```
Expand & Expand::operator= (
    const Expand & shader) [delete]
```

代入演算子は使用しない

引数

<i>shader</i>	代入元のシェーダ
---------------	----------

戻り値

代入後のこのシェーダの参照

呼び出し関係図:



8.11.3.3 setup()

```

std::array< int, 2 > Expand::setup (
    int samples,
    GLfloat aspect,
    const gg::GgMatrix & pose,
    const std::array< GLfloat, 2 > & fov,
    const std::array< GLfloat, 2 > & center,
    GLfloat focal,
    const std::array< GLfloat, 4 > & border,
    int unit = 0) const
  
```

展開

引数

<i>samples</i>	展開するテクスチャをサンプリングする数
<i>aspect</i>	展開するテクスチャの縦横比
<i>pose</i>	サンプリングに用いるカメラの姿勢
<i>fov</i>	サンプリングに用いるカメラの相対画角 (単位は度)
<i>center</i>	サンプリングに用いるカメラの撮像面上の中心 (主点) 位置
<i>focal</i>	サンプリングに用いるカメラの主点とスクリーンの距離
<i>border</i>	展開後のフレームの境界色
<i>unit</i>	テクスチャユニット番号

戻り値

描画すべきメッシュの横と縦の格子点数

覚え書き

展開するテクスチャをマッピングするメッシュの解像度を `samples` と `aspect` から個々のメッシュが正方形に近くなるよう決定し、それをもとに格子の間隔を求める

展開

[Expand.cpp](#) の 37 行目に定義があります。

呼び出し関係図:



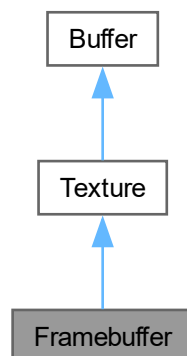
このクラス詳解は次のファイルから抽出されました:

- [Expand.h](#)
- [Expand.cpp](#)

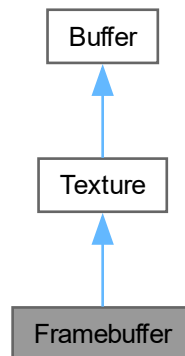
8.12 Framebuffer クラス

```
#include <Framebuffer.h>
```

Framebuffer の継承関係図



Framebuffer 連携図



公開メンバ関数

- `Framebuffer ()=default`
- `Framebuffer (GLsizei width, GLsizei height, int channels=3, GLenum attachment=GL_COLOR_ATTACHMENT0)`
- `Framebuffer (const Framebuffer &framebuffer)`
- `Framebuffer (Framebuffer &&framebuffer) noexcept`
- `virtual ~Framebuffer ()`
- `Framebuffer & operator= (const Framebuffer &framebuffer)`
- `Framebuffer & operator= (Framebuffer &&framebuffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &framebuffer) noexcept`
- `virtual void discard ()`
- `auto getFramebufferName () const`
- `virtual const std::array< GLsizei, 2 > & getSize () const`
- `virtual int getChannels () const`
- `void bindFramebuffer ()`
- `void unbindFramebuffer () const`
- `void update (const std::array< int, 2 > &size)`
- `void update (const std::array< int, 2 > &size, const Texture &frame, int unit=0)`
- `void show (GLsizei width, GLsizei height) const`

基底クラス `Texture` に属する継承公開メンバ関数

- `Texture ()=default`
- `Texture (GLsizei width, GLsizei height, int channels)`
- `Texture (const Texture &texture)`
- `Texture (Texture &&texture) noexcept`
- `virtual ~Texture ()`
- `Texture & operator= (const Texture &texture)`
- `Texture & operator= (Texture &&texture) noexcept`
- `auto getTextureName () const`

- void `bindTexture` (int unit=0) const
- void `unbindTexture` () const
- void `draw` (GLsizei width, GLsizei height, int unit=0) const
- void `readPixels` (GLuint buffer) const
- void `readPixels` (Buffer &buffer) const
- void `readPixels` () const
- void `drawPixels` (GLuint buffer) const
- void `drawPixels` (const Texture &texture)
- void `drawPixels` (const Buffer &buffer)
- void `drawPixels` () const
- void `drawPixels` (GLsizei width, GLsizei height, int channels, const void *pixels)

基底クラス **Buffer** に属する継承公開メンバ関数

- `Buffer` ()=default
- `Buffer` (GLsizei width, GLsizei height, int channels)
- `Buffer` (const Buffer &buffer)
- `Buffer` (Buffer &&buffer) noexcept
- virtual `~Buffer` ()
- `Buffer` & `operator=` (const Buffer &buffer)
- `Buffer` & `operator=` (Buffer &&buffer) noexcept
- void `resize` (GLsizei width, GLsizei height, int channels)
- void `resize` (const Buffer &buffer)
- auto `getBufferName` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

その他の継承メンバ

基底クラス **Buffer** に属する継承限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const Buffer &buffer) noexcept

基底クラス **Texture** に属する継承静的限定公開変数類

- static std::shared_ptr< Mesh > `mesh`
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

8.12.1 詳解

フレームバッファオブジェクトクラス

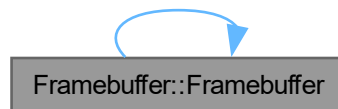
`Framebuffer.h` の 17 行目に定義があります。

8.12.2 構築子と解体子

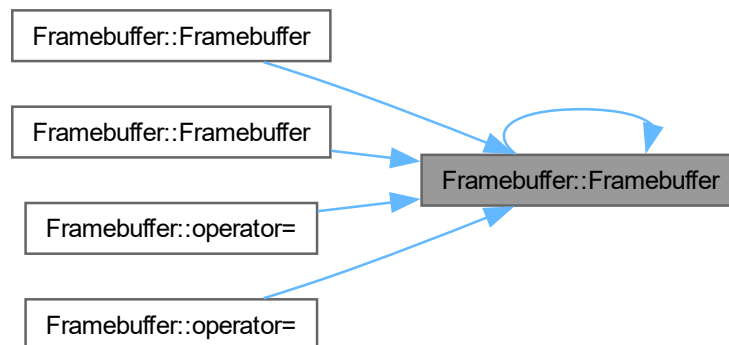
8.12.2.1 Framebuffer() [1/4]

```
Framebuffer::Framebuffer () [default]
```

デフォルトコンストラクタ呼び出し関係図:



被呼び出し関係図:



8.12.2.2 Framebuffer() [2/4]

```
Framebuffer::Framebuffer (  
    GLsizei width,  
    GLsizei height,  
    int channels = 3,  
    GLenum attachment = GL_COLOR_ATTACHMENT0)
```

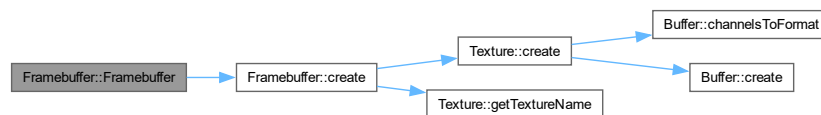
フレームバッファオブジェクトを作成するコンストラクタ

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数
<i>attachment</i>	フレームバッファオブジェクトのカラーバッファのアタッチメント

[Framebuffer.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



8.12.2.3 Framebuffer() [3/4]

```
Framebuffer::Framebuffer (
    const Framebuffer & framebuffer)
```

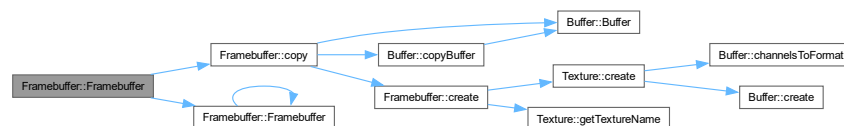
コピーコンストラクタ

引数

<i>framebuffer</i>	コピー元のフレームバッファオブジェクト
--------------------	---------------------

[Framebuffer.cpp](#) の 26 行目に定義があります。

呼び出し関係図:



8.12.2.4 Framebuffer() [4/4]

```
Framebuffer::Framebuffer (
    Framebuffer && framebuffer) [inline], [noexcept]
```

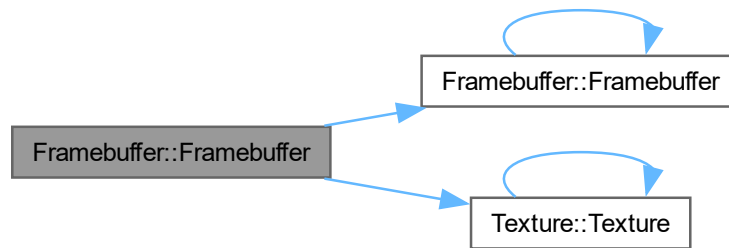
ムーブコンストラクタ

引数

<code>framebuffer</code>	ムーブ元のフレームバッファオブジェクト
--------------------------	---------------------

[Framebuffer.h](#) の 61 行目に定義があります。

呼び出し関係図:



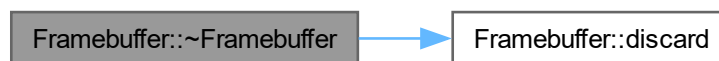
8.12.2.5 ~Framebuffer()

```
Framebuffer::~~Framebuffer () [virtual]
```

デストラクタ

[Framebuffer.cpp](#) の 36 行目に定義があります。

呼び出し関係図:



8.12.3 関数詳解

8.12.3.1 bindFramebuffer()

```
void Framebuffer::bindFramebuffer ()
```

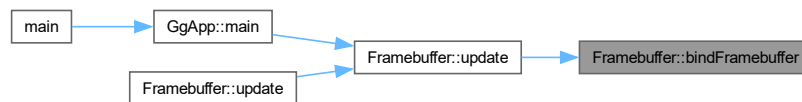
レンダリング先をフレームバッファオブジェクトに切り替える

覚え書き

この後から `unbindFramebuffer()` の前まで、レンダリング先がフレームバッファオブジェクトになる。この時点のビューポートを保存する。この間でレンダリング処理を実行する。

`Framebuffer.cpp` の 154 行目に定義があります。

被呼び出し関係図:



8.12.3.2 copy()

```
void Framebuffer::copy (
    const Buffer & framebuffer) [virtual], [noexcept]
```

フレームバッファオブジェクトをコピーする

引数

<code>framebuffer</code>	コピー元のフレームバッファオブジェクト
--------------------------	---------------------

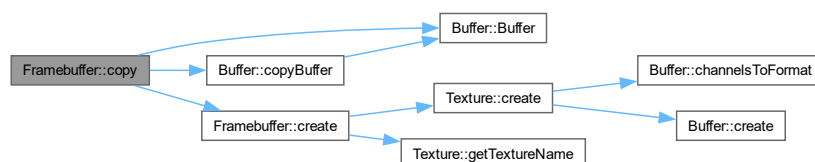
覚え書き

このフレームバッファオブジェクトのサイズがコピー元と異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直してコピーする。引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

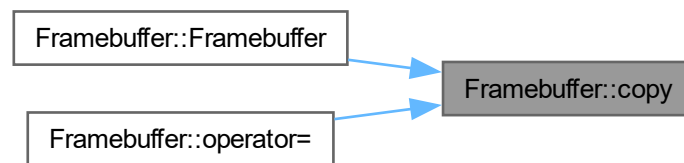
`Texture` を再実装しています。

`Framebuffer.cpp` の 141 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.3 create()

```

void Framebuffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

フレームバッファオブジェクトを作成する

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数

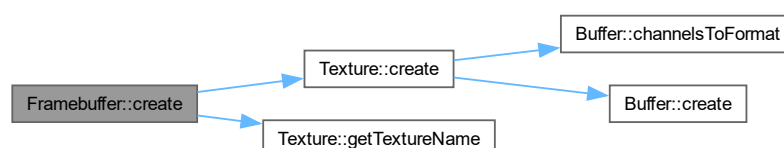
覚え書き

このフレームバッファオブジェクトのサイズが引数で指定したサイズと異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直す。

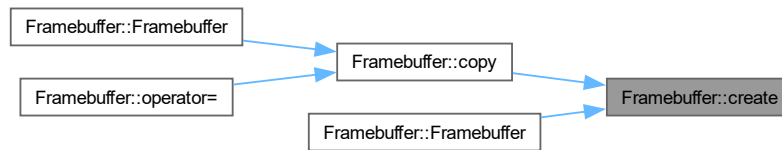
[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 110 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.4 discard()

```
void Framebuffer::discard () [virtual]
```

フレームバッファオブジェクトを破棄する

[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 92 行目に定義があります。

被呼び出し関係図:



8.12.3.5 getChannels()

```
virtual int Framebuffer::getChannels () const [inline], [virtual]
```

フレームバッファオブジェクトのチャンネル数を得る

戻り値

フレームバッファオブジェクトのチャンネル数

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 156 行目に定義があります。

被呼び出し関係図:



8.12.3.6 getFramebufferName()

```
auto Framebuffer::getFramebufferName () const [inline]
```

フレームバッファオブジェクト名を得る

戻り値

フレームバッファオブジェクト名

[Framebuffer.h](#) の 136 行目に定義があります。

8.12.3.7 getSize()

```
virtual const std::array< GLsizei, 2 > & Framebuffer::getSize () const [inline], [virtual]
```

フレームバッファオブジェクトのサイズを得る

戻り値

フレームバッファオブジェクトのサイズ

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 146 行目に定義があります。

8.12.3.8 operator=() [1/2]

```
Framebuffer & Framebuffer::operator= (
    const Framebuffer & framebuffer)
```

代入演算子

引数

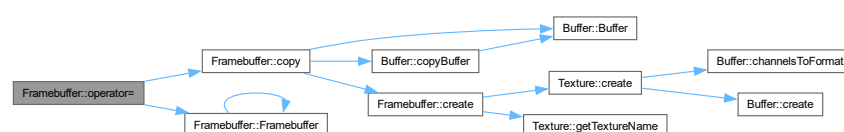
<i>framebuffer</i>	代入元のフレームバッファオブジェクト
--------------------	--------------------

戻り値

代入結果のテクスチャ

[Framebuffer.cpp](#) の 45 行目に定義があります。

呼び出し関係図:



8.12.3.9 operator=() [2/2]

```
Framebuffer & Framebuffer::operator= (  
    Framebuffer && framebuffer) [noexcept]
```

ムーブ代入演算子

引数

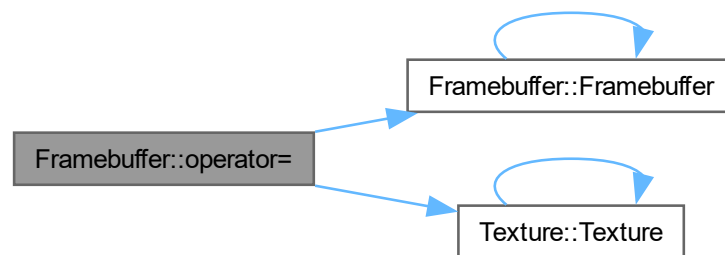
<code>framebuffer</code>	ムーブ代入元のフレームバッファオブジェクト
--------------------------	-----------------------

戻り値

ムーブ代入結果のフレームバッファオブジェクト

[Framebuffer.cpp](#) の 61 行目に定義があります。

呼び出し関係図:



8.12.3.10 show()

```
void Framebuffer::show (
    GLsizei width,
    GLsizei height) const
```

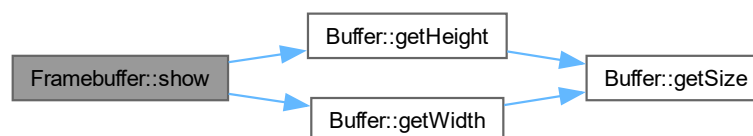
フレームバッファオブジェクトの内容を表示する

引数

<i>width</i>	フレームバッファオブジェクトの内容を表示する横の画素数
<i>height</i>	フレームバッファオブジェクトの内容を表示する縦の画素数

[Framebuffer.cpp](#) の 216 行目に定義があります。

呼び出し関係図:



8.12.3.11 unbindFramebuffer()

```
void Framebuffer::unbindFramebuffer () const
```

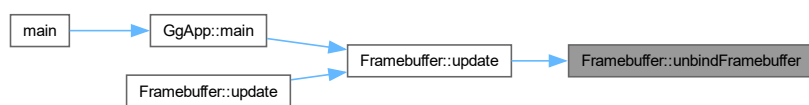
レンダリング先を通常のフレームバッファに戻す

覚え書き

`bindFramebuffer()` の後からこの前まで、レンダリング先がフレームバッファオブジェクトになる。保存したビューポートはここで復帰する。この間でレンダリング処理を実行する。

`Framebuffer.cpp` の 166 行目に定義があります。

被呼び出し関係図:



8.12.3.12 update() [1/2]

```
void Framebuffer::update (
    const std::array< int, 2 > & size)
```

テクスチャを展開してフレームバッファオブジェクトを更新する

引数

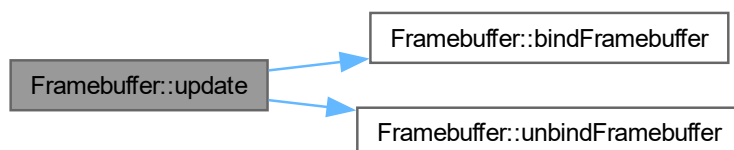
<code>size</code>	展開に用いるメッシュの分割数
-------------------	----------------

覚え書き

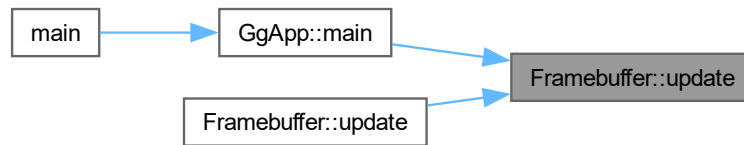
レンダリング先をフレームバッファオブジェクトに切り替えて、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択やテクスチャの結合を行う。

`Framebuffer.cpp` の 186 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.13 update() [2/2]

```

void Framebuffer::update (
    const std::array< int, 2 > & size,
    const Texture & frame,
    int unit = 0)
  
```

テクスチャを展開してフレームバッファオブジェクトを更新する

引数

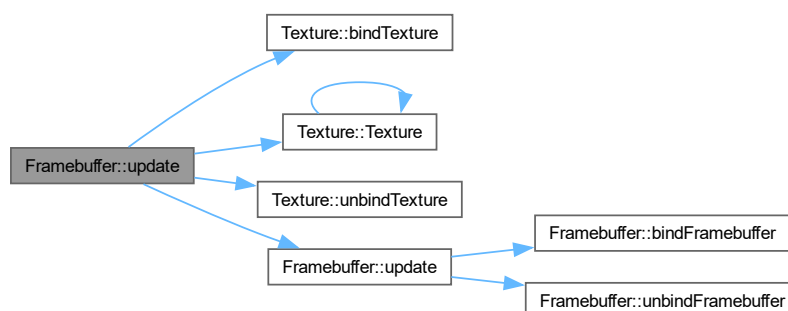
<i>size</i>	展開に用いるメッシュの分割数
<i>frame</i>	展開するフレームを格納したテクスチャ
<i>unit</i>	テクスチャのマッピングに使うテクスチャユニットの番号

覚え書き

レンダリング先をフレームバッファオブジェクトに切り替えて、テクスチャユニット *unit* を指定して *frame* に格納されたテクスチャを結合してから、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択を行う。*unit* にはシェードにおいてテクスチャのサンプラの *uniform* 変数に設定する `GL_TEXTUREi` の *i* と一致させること。

[Framebuffer.cpp](#) の 201 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Framebuffer.h](#)
- [Framebuffer.cpp](#)

8.13 GgApp クラス

```
#include <GgApp.h>
```

クラス

- class [Window](#)

公開メンバ関数

- [GgApp](#) (int major=0, int minor=1)
- [GgApp](#) (const [GgApp](#) &w)=delete
- [GgApp](#) ([GgApp](#) &&w)=default
- virtual [~GgApp](#) ()
- [GgApp](#) & [operator=](#) (const [GgApp](#) &w)=delete
- [GgApp](#) & [operator=](#) ([GgApp](#) &&w)=default
- int [main](#) (int argc, const char *const *argv)

8.13.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラス.

[GgApp.h](#) の 88 行目に定義があります。

8.13.2 構築子と解体子

8.13.2.1 GgApp() [1/3]

```
GgApp::GgApp (
    int major = 0,
    int minor = 1)
```

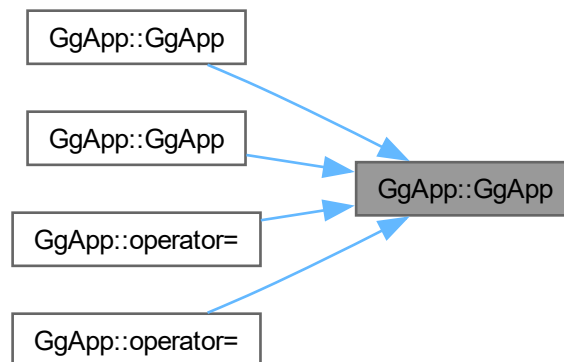
コンストラクタ.

引数

<i>major</i>	使用する OpenGL の major 番号, 0 なら無指定.
<i>minor</i>	使用する OpenGL の minor 番号, major 番号が 0 なら無視.

[GgApp.cpp](#) の 51 行目に定義があります。

被呼び出し関係図:



8.13.2.2 GgApp() [2/3]

```
GgApp::GgApp (
    const GgApp & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のオブジェクト.
----------	--------------

呼び出し関係図:



8.13.2.3 GgApp() [3/3]

```
GgApp::GgApp (
    GgApp && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ元のオブジェクト.
----------	--------------

呼び出し関係図:



8.13.2.4 ~GgApp()

```
GgApp::~GgApp () [virtual]
```

デストラクタ.

[GgApp.cpp](#) の 98 行目に定義があります。

8.13.3 関数詳解

8.13.3.1 main()

```
int GgApp::main (  
    int argc,  
    const char *const * argv)
```

アプリケーション本体.

引数

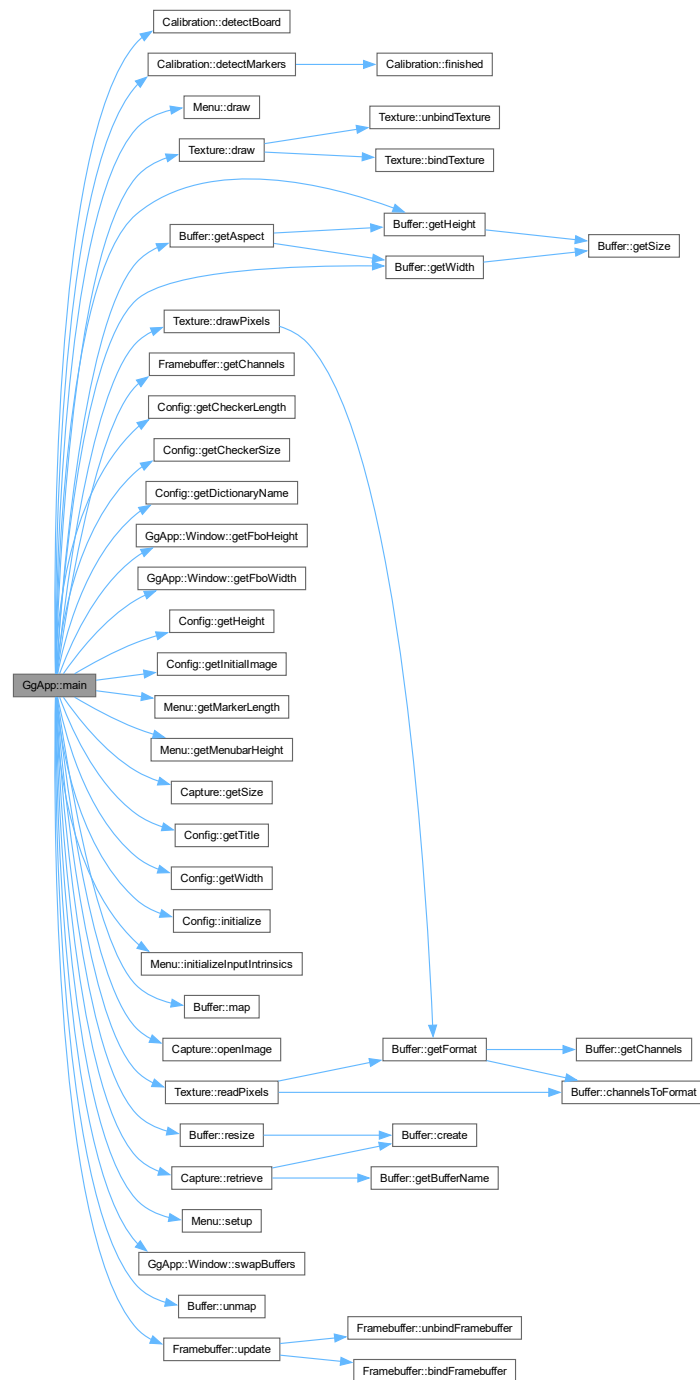
<i>argc</i>	コマンドライン引数の数.
<i>argv</i>	コマンドライン引数の配列.

戻り値

アプリケーションの終了ステータス。

`calib.cpp` の 33 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.13.3.2 operator=() [1/2]

```
GgApp & GgApp::operator= (  
    const GgApp & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のオブジェクト.
----------	-------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.13.3.3 operator=() [2/2]

```
GgApp & GgApp::operator= (  
    GgApp && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のオブジェクト.
----------	----------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [calib.cpp](#)
- [GgApp.cpp](#)

8.14 gg::GgBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgBuffer](#) (GLenum target, const T *data, GLsizei stride, GLsizei count, GLenum usage)
- [GgBuffer](#) (const GgBuffer< T > &buffer)=delete
- [GgBuffer](#) (GgBuffer< T > &&buffer)=default
- virtual [~GgBuffer](#) ()
- [GgBuffer](#)< T > & [operator=](#) (const [GgBuffer](#)< T > &buffer)=delete
- [GgBuffer](#)< T > & [operator=](#) ([GgBuffer](#)< T > &&buffer)=default
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLint first, GLsizei count) const
- void [unmap](#) () const
- void [send](#) (const T *data, GLint first, GLsizei count) const
- void [read](#) (T *data, GLint first, GLsizei count) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.14.1 詳解

```
template<typename T>
class gg::GgBuffer< T >
```

バッファオブジェクト.

覚え書き

頂点属性／頂点インデックス／ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

gg.h の 5571 行目に定義があります。

8.14.2 構築子と解体子

8.14.2.1 GgBuffer() [1/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GLenum target,
    const T * data,
    GLsizei stride,
    GLsizei count,
    GLenum usage) [inline]
```

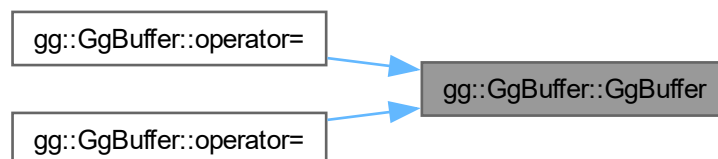
コンストラクタ.

引数

<i>target</i>	バッファオブジェクトのターゲット.
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>stride</i>	データの間隔.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5583 行目に定義があります。

被呼び出し関係図:



8.14.2.2 GgBuffer() [2/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    const GgBuffer< T > & buffer) [delete]
```

コピーコンストラクタは使用しない。

引数

<i>buffer</i>	コピー元のバッファ。
---------------	------------

8.14.2.3 GgBuffer() [3/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GgBuffer< T > && buffer) [default]
```

ムーブコンストラクタ。

引数

<i>buffer</i>	ムーブ元のバッファ。
---------------	------------

8.14.2.4 ~GgBuffer()

```
template<typename T>
virtual gg::GgBuffer< T >::~~GgBuffer () [inline], [virtual]
```

デストラクタ。

[gg.h](#) の 5583 行目に定義があります。

8.14.3 関数詳解

8.14.3.1 bind()

```
template<typename T>
void gg::GgBuffer< T >::bind () const [inline]
```

バッファオブジェクトを結合する。

[gg.h](#) の 5696 行目に定義があります。

8.14.3.2 copy()

```
template<typename T>
void gg::GgBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

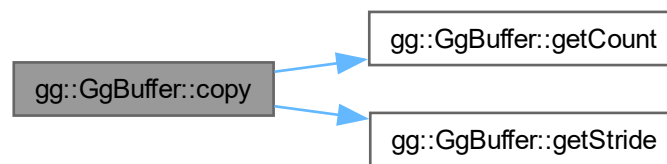
別のバッファオブジェクトからデータを複写する。

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (<i>src_buffer</i>) の先頭のデータの位置.
<i>dst_first</i>	複写先 (<code>getBuffer()</code>) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

`gg.h` の 5801 行目に定義があります。

呼び出し関係図:



8.14.3.3 getBuffer()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getBuffer () const [inline]
```

バッファオブジェクト名を取り出す。

戻り値

このバッファオブジェクト名.

`gg.h` の 5688 行目に定義があります。

8.14.3.4 getCount()

```
template<typename T>
const GLsizei & gg::GgBuffer< T >::getCount () const [inline]
```

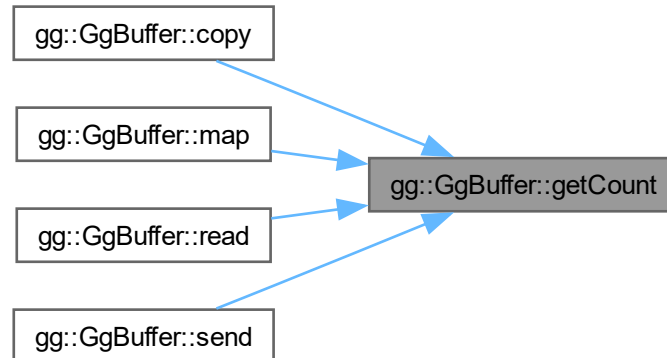
バッファオブジェクトが保持するデータの数を取り出す。

戻り値

このバッファオブジェクトが保持するデータの数.

[gg.h](#) の 5678 行目に定義があります。

被呼び出し関係図:



8.14.3.5 getStride()

```
template<typename T>  
GLsizeiPtr gg::GgBuffer< T >::getStride () const [inline]
```

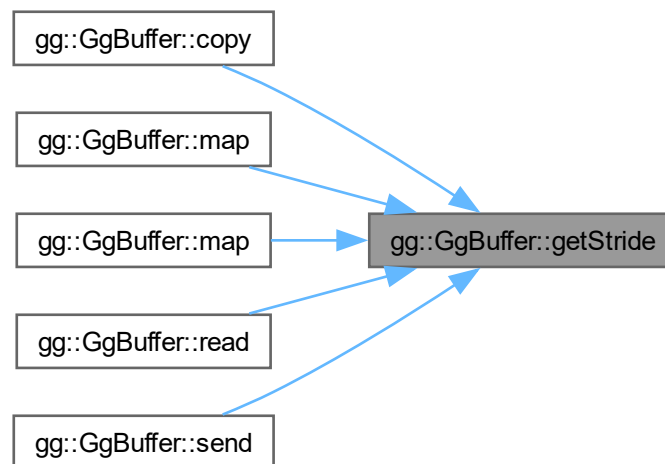
バッファオブジェクトのアライメントを考慮したデータの間隔を取り出す.

戻り値

このバッファオブジェクトのデータの間隔.

[gg.h](#) の 5668 行目に定義があります。

被呼び出し関係図:



8.14.3.6 getTarget()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getTarget () const [inline]
```

バッファオブジェクトのターゲットを取り出す.

戻り値

このバッファオブジェクトのターゲット.

[gg.h](#) の 5658 行目に定義があります。

8.14.3.7 map() [1/2]

```
template<typename T>
void * gg::GgBuffer< T >::map () const [inline]
```

バッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5714 行目に定義があります。

呼び出し関係図:



8.14.3.8 map() [2/2]

```
template<typename T>
void * gg::GgBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

バッファオブジェクトの指定した範囲をマップする.

引数

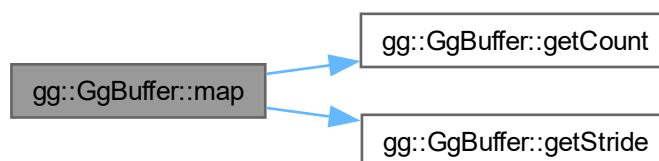
<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5731 行目に定義があります。

呼び出し関係図:



8.14.3.9 operator=() [1/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    const GgBuffer< T > & buffer) [delete]
```

代入演算子は使用しない.

引数

<i>buffer</i>	代入元のバッファ.
---------------	-----------

戻り値

代入後のこのバッファの参照.

呼び出し関係図:



8.14.3.10 operator=() [2/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    GgBuffer< T > && buffer) [default]
```

ムーブ代入演算子.

引数

<i>buffer</i>	ムーブ代入元のバッファ.
---------------	--------------

戻り値

ムーブ代入後のこのバッファの参照.

呼び出し関係図:



8.14.3.11 read()

```
template<typename T>
void gg::GgBuffer< T >::read (
    T * data,
    GLint first,
    GLsizei count) const [inline]
```

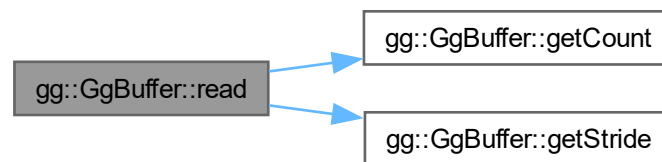
バッファオブジェクトのデータから抽出する。

引数

<i>data</i>	抽出先の領域の先頭のポインタ。
<i>first</i>	抽出元のバッファオブジェクトの取り出すデータの領域の先頭の要素番号。
<i>count</i>	抽出するデータの数 (0 ならバッファオブジェクト全体)。

gg.h の 5774 行目に定義があります。

呼び出し関係図:



8.14.3.12 send()

```
template<typename T>
void gg::GgBuffer< T >::send (
    const T * data,
    GLint first,
    GLsizei count) const [inline]
```

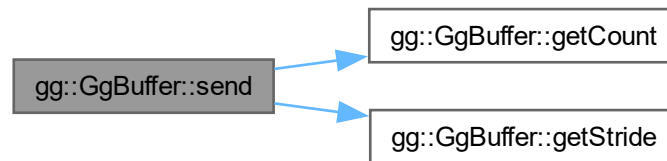
すでに確保したバッファオブジェクトにデータを転送する。

引数

<i>data</i>	転送元のデータが格納されている領域の先頭のポインタ。
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号。
<i>count</i>	転送するデータの数 (0 ならバッファオブジェクト全体)。

gg.h の 5756 行目に定義があります。

呼び出し関係図:



8.14.3.13 unbind()

```
template<typename T>
void gg::GgBuffer< T >::unbind () const [inline]
```

バッファオブジェクトを解放する。

gg.h の 5704 行目に定義があります。

8.14.3.14 unmap()

```
template<typename T>
void gg::GgBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。

gg.h の 5744 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.15 gg::GgColorTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgColorTexture](#) ()
- [GgColorTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- [GgColorTexture](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- virtual [~GgColorTexture](#) ()
- void [load](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- void [load](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)

8.15.1 詳解

カラーマップ.

覚え書き

カラー画像を読み込んでテクスチャを作成する.

[gg.h](#) の 5350 行目に定義があります。

8.15.2 構築子と解体子

8.15.2.1 GgColorTexture() [1/3]

```
gg::GgColorTexture::GgColorTexture () [inline]
```

コンストラクタ.

[gg.h](#) の 5360 行目に定義があります。

8.15.2.2 GgColorTexture() [2/3]

```
gg::GgColorTexture::GgColorTexture (  
    const GLvoid * image,  
    GLsizei width,  
    GLsizei height,  
    GLenum format = GL_RGB,  
    GLenum type = GL_UNSIGNED_BYTE,  
    GLenum internal = GL_RGB,  
    GLenum wrap = GL_CLAMP_TO_EDGE,  
    bool swizzle = true) [inline]
```

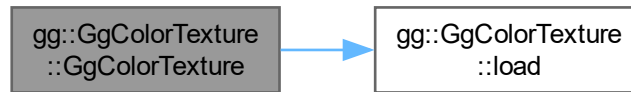
メモリ上のデータからカラーのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	読み込む画像の横の画素数.
<i>height</i>	読み込む画像の縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

[gg.h](#) の 5376 行目に定義があります。

呼び出し関係図:



8.15.2.3 GgColorTexture() [3/3]

```

gg::GgColorTexture::GgColorTexture (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE) [inline]
  
```

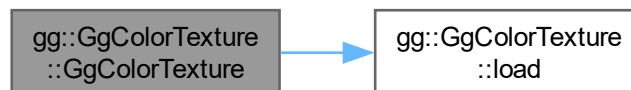
TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, GL_TEXTURE_WRAP_S および GL_TEXTURE_WRAP_T に設定する値.

gg.h の 5397 行目に定義があります。

呼び出し関係図:



8.15.2.4 ~GgColorTexture()

```

virtual gg::GgColorTexture::~~GgColorTexture () [inline], [virtual]
  
```

デストラクタ.

gg.h の 5409 行目に定義があります。

8.15.3 関数詳解

8.15.3.1 load() [1/2]

```
void gg::GgColorTexture::load (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

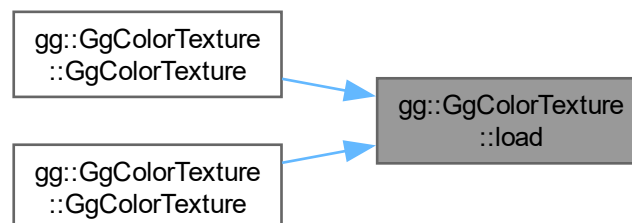
メモリ上のデータを読み込んでテクスチャを作成する。

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5425 行目に定義があります。

被呼び出し関係図:



8.15.3.2 load() [2/2]

```
void gg::GgColorTexture::load (  
    const std::string & name,  
    GLenum internal = 0,  
    GLenum wrap = GL_CLAMP_TO_EDGE)
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 ならファイルの画像フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

gg.cpp の 4028 行目に定義があります。

呼び出し関係図:



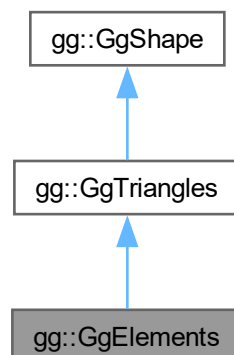
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

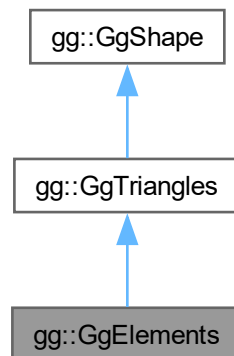
8.16 gg::GgElements クラス

```
#include <gg.h>
```

gg::GgElements の継承関係図



gg::GgElements 連携図



公開メンバ関数

- **GgElements** (GLenum mode=GL_TRIANGLES)
- **GgElements** (const **GgVertex** *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual **~GgElements** ()
- const GLsizei & **getIndexCount** () const
- const GLuint & **getIndexBuffer** () const
- void **send** (const **GgVertex** *vert, GLuint firstv, GLsizei countv, const GLuint *face=nullptr, GLuint firstf=0, GLsizei countf=0) const
- void **load** (const **GgVertex** *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum usage=GL_STATIC_DRAW)
- virtual void **draw** (GLuint first=0, GLsizei count=0) const

基底クラス **gg::GgTriangles** に属する継承公開メンバ関数

- **GgTriangles** (GLenum mode=GL_TRIANGLES)
- **GgTriangles** (const **GgVertex** *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual **~GgTriangles** ()
- const GLsizei & **getCount** () const
- const GLuint & **getBuffer** () const
- void **send** (const **GgVertex** *vert, GLuint first=0, GLsizei count=0) const
- void **load** (const **GgVertex** *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)

基底クラス **gg::GgShape** に属する継承公開メンバ関数

- **GgShape** (GLenum mode=0)
- virtual **~GgShape** ()
- virtual **operator bool** () const noexcept
- virtual bool **operator!** () const noexcept
- const GLuint & **get** () const
- void **setMode** (GLenum mode)
- const GLenum & **getMode** () const

8.16.1 詳解

三角形で表した形状データ (Elements 形式).

[gg.h](#) の 6606 行目に定義があります。

8.16.2 構築子と解体子

8.16.2.1 GgElements() [1/2]

```
gg::GgElements::GgElements (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6619 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.2.2 GgElements() [2/2]

```
gg::GgElements::GgElements (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei counth,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

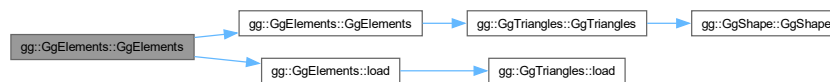
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>face</i>	三角形の頂点インデックス.
<i>countf</i>	三角形の頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6634 行目に定義があります。

呼び出し関係図:



8.16.2.3 ~GgElements()

```
virtual gg::GgElements::~~GgElements () [inline], [virtual]
```

デストラクタ.

gg.h の 6650 行目に定義があります。

8.16.3 関数詳解

8.16.3.1 draw()

```
void gg::GgElements::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

インデックスを使った三角形の描画.

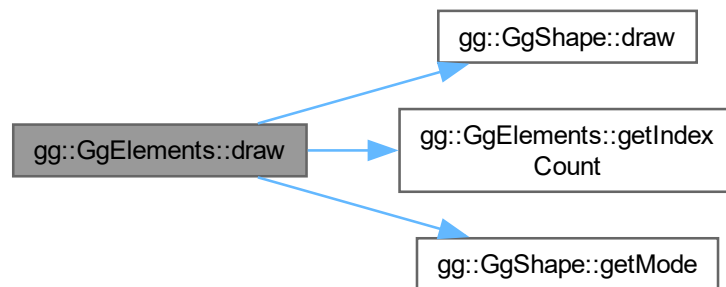
引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

gg::GgTrianglesを再実装しています。

gg.cpp の 5169 行目に定義があります。

呼び出し関係図:



8.16.3.2 getIndexBuffer()

```
const GLuint & gg::GgElements::getIndexBuffer () const [inline]
```

三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名.

[gg.h](#) の 6668 行目に定義があります。

8.16.3.3 getIndexCount()

```
const GLsizei & gg::GgElements::getIndexCount () const [inline]
```

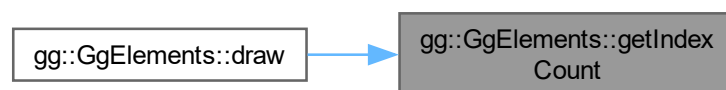
データの数を取り出す.

戻り値

この図形の三角形数.

[gg.h](#) の 6658 行目に定義があります。

被呼び出し関係図:



8.16.3.4 load()

```
void gg::GgElements::load (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

バッファオブジェクトを確保して頂点属性と三角形の頂点インデックスデータを格納する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
<i>countv</i>	頂点のデータの数 (頂点数)。
<i>face</i>	三角形の頂点インデックスデータ。
<i>countf</i>	三角形の頂点数。
<i>usage</i>	バッファオブジェクトの使い方。

gg.h の 6705 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.5 send()

```
void gg::GgElements::send (
    const GgVertex * vert,
    GLuint firstv,
    GLsizei countv,
```

```
const GLuint * face = nullptr,  
GLuint firstf = 0,  
GLsizei countf = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性と三角形の頂点インデックスデータを転送する.

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>firstv</i>	頂点属性の転送先のバッファオブジェクトの先頭の要素番号.
<i>countv</i>	頂点のデータの数 (頂点数).
<i>face</i>	三角形の頂点インデックスデータ.
<i>firstf</i>	インデックスの転送先のバッファオブジェクトの先頭の要素番号.
<i>countf</i>	三角形の頂点数.

gg.h の 6683 行目に定義があります。

呼び出し関係図:



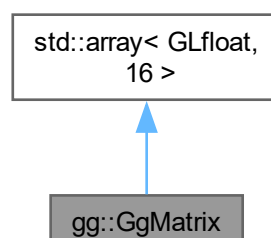
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

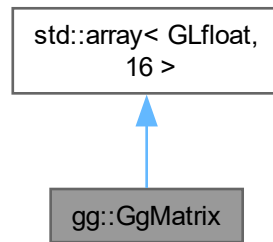
8.17 gg::GgMatrix クラス

```
#include <gg.h>
```

gg::GgMatrix の継承関係図



gg::GgMatrix 連携図



公開メンバ関数

- `GgMatrix` ()=default
- `constexpr GgMatrix` (GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03, GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13, GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23, GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33)
- `constexpr GgMatrix` (const GLfloat *a)
- `constexpr GgMatrix` (GLfloat c)
- `GgMatrix` (const GgMatrix &m)=default
- `GgMatrix` (GgMatrix &&m)=default
- `~GgMatrix` ()=default
- `GgMatrix & operator=` (const `GgMatrix` &m)=default
- `GgMatrix & operator=` (`GgMatrix` &&m)=default
- `GgMatrix & operator=` (const GLfloat *a)
- `GgMatrix operator+` (const GLfloat *a) const
- `GgMatrix operator+` (const `GgMatrix` &m) const
- `GgMatrix & operator+=` (const GLfloat *a)
- `GgMatrix & operator+=` (const `GgMatrix` &m)
- `GgMatrix operator-` (const GLfloat *a) const
- `GgMatrix operator-` (const `GgMatrix` &m) const
- `GgMatrix & operator-=` (const GLfloat *a)
- `GgMatrix & operator-=` (const `GgMatrix` &m)
- `GgMatrix operator*` (const GLfloat *a) const
- `GgMatrix operator*` (const `GgMatrix` &m) const
- `GgMatrix & operator*=` (const GLfloat *a)
- `GgMatrix & operator*=` (const `GgMatrix` &m)
- `GgMatrix operator/` (const GLfloat *a) const
- `GgMatrix operator/` (const `GgMatrix` &m) const
- `GgMatrix & operator/=` (const GLfloat *a)
- `GgMatrix & operator/=` (const `GgMatrix` &m)
- `GgMatrix & loadIdentity` ()
- `GgMatrix & loadTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix & loadTranslate` (const GLfloat *t)
- `GgMatrix & loadTranslate` (const `GgVector` &t)
- `GgMatrix & loadScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix & loadScale` (const GLfloat *s)
- `GgMatrix & loadScale` (const `GgVector` &s)

- `GgMatrix & loadRotateX` (GLfloat a)
- `GgMatrix & loadRotateY` (GLfloat a)
- `GgMatrix & loadRotateZ` (GLfloat a)
- `GgMatrix & loadRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix & loadRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r)
- `GgMatrix & loadRotate` (const `GgVector` &r)
- `GgMatrix & loadLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix & loadLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix & loadLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix & loadOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadTranspose` (const GLfloat *a)
- `GgMatrix & loadTranspose` (const `GgMatrix` &m)
- `GgMatrix & loadInvert` (const GLfloat *a)
- `GgMatrix & loadInvert` (const `GgMatrix` &m)
- `GgMatrix & loadNormal` (const GLfloat *a)
- `GgMatrix & loadNormal` (const `GgMatrix` &m)
- `GgMatrix translate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix translate` (const GLfloat *t) const
- `GgMatrix translate` (const `GgVector` &t) const
- `GgMatrix scale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix scale` (const GLfloat *s) const
- `GgMatrix scale` (const `GgVector` &s) const
- `GgMatrix rotateX` (GLfloat a) const
- `GgMatrix rotateY` (GLfloat a) const
- `GgMatrix rotateZ` (GLfloat a) const
- `GgMatrix rotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r, GLfloat a) const
- `GgMatrix rotate` (const `GgVector` &r, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r) const
- `GgMatrix rotate` (const `GgVector` &r) const
- `GgMatrix lookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz) const
- `GgMatrix lookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u) const
- `GgMatrix lookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u) const
- `GgMatrix orthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix frustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix perspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar) const
- `GgMatrix transpose` () const
- `GgMatrix invert` () const
- `GgMatrix normal` () const
- void `projection` (GLfloat *c, const GLfloat *v) const
- void `projection` (GLfloat *c, const `GgVector` &v) const
- void `projection` (`GgVector` &c, const GLfloat *v) const
- void `projection` (`GgVector` &c, const `GgVector` &v) const
- `GgVector operator*` (const `GgVector` &v) const
- const GLfloat * `get` () const
- void `get` (GLfloat *a) const

8.17.1 詳解

変換行列.

[gg.h](#) の 2126 行目に定義があります。

8.17.2 構築子と解体子

8.17.2.1 GgMatrix() [1/6]

```
gg::GgMatrix::GgMatrix () [default]
```

コンストラクタ.

8.17.2.2 GgMatrix() [2/6]

```
gg::GgMatrix::GgMatrix (  
    GLfloat m00,  
    GLfloat m01,  
    GLfloat m02,  
    GLfloat m03,  
    GLfloat m10,  
    GLfloat m11,  
    GLfloat m12,  
    GLfloat m13,  
    GLfloat m20,  
    GLfloat m21,  
    GLfloat m22,  
    GLfloat m23,  
    GLfloat m30,  
    GLfloat m31,  
    GLfloat m32,  
    GLfloat m33) [inline], [constexpr]
```

コンストラクタ.

引数

<i>m00</i>	GLfloat 型の値.
<i>m01</i>	GLfloat 型の値.
<i>m02</i>	GLfloat 型の値.
<i>m03</i>	GLfloat 型の値.
<i>m10</i>	GLfloat 型の値.
<i>m11</i>	GLfloat 型の値.
<i>m12</i>	GLfloat 型の値.
<i>m13</i>	GLfloat 型の値.
<i>m20</i>	GLfloat 型の値.
<i>m21</i>	GLfloat 型の値.

<i>m22</i>	GLfloat 型の値.
<i>m23</i>	GLfloat 型の値.
<i>m30</i>	GLfloat 型の値.
<i>m31</i>	GLfloat 型の値.
<i>m32</i>	GLfloat 型の値.
<i>m33</i>	GLfloat 型の値.

gg.h の 2161 行目に定義があります。

8.17.2.3 GgMatrix() [3/6]

```
gg::GgMatrix::GgMatrix (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

gg.h の 2176 行目に定義があります。

呼び出し関係図:



8.17.2.4 GgMatrix() [4/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

gg.h の 2186 行目に定義があります。

呼び出し関係図:



8.17.2.5 GgMatrix() [5/6]

```
gg::GgMatrix::GgMatrix (  
    const GgMatrix & m) [default]
```

コピーコンストラクタ.

引数

<i>m</i>	コピー元の変換行列.
----------	------------

呼び出し関係図:



8.17.2.6 GgMatrix() [6/6]

```
gg::GgMatrix::GgMatrix (  
    GgMatrix && m) [default]
```

ムーブコンストラクタ.

引数

<i>m</i>	ムーブ元の変換行列.
----------	------------

呼び出し関係図:



8.17.2.7 ~GgMatrix()

```
gg::GgMatrix::~~GgMatrix () [default]
```

デストラクタ.

8.17.3 関数詳解

8.17.3.1 frustum()

```
GgMatrix gg::GgMatrix::frustum (  
    GLfloat left,  
    GLfloat right,  
    GLfloat bottom,  
    GLfloat top,  
    GLfloat zNear,  
    GLfloat zFar) const [inline]
```

透視投影変換を乗じた結果を返す.

引数

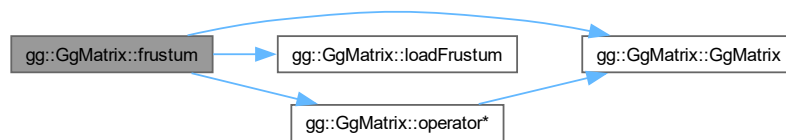
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 2969 行目に定義があります。

呼び出し関係図:



8.17.3.2 get() [1/2]

```
const GLfloat * gg::GgMatrix::get () const [inline]
```

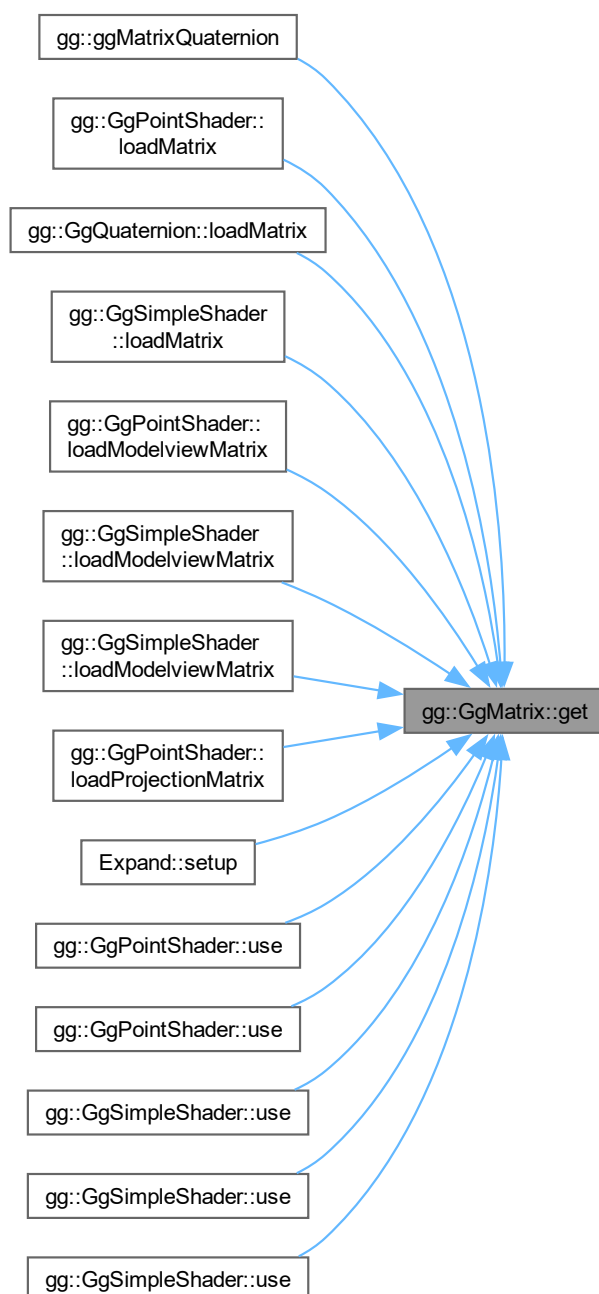
変換行列を取り出す.

戻り値

変換行列を格納した GLfloat 型の 16 要素の配列変数.

gg.h の 3092 行目に定義があります。

被呼び出し関係図:



8.17.3.3 get() [2/2]

```
void gg::GgMatrix::get (  
    GLfloat * a) const [inline]
```

変換行列を取り出す.

引数

<i>a</i>	変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

[gg.h](#) の 3102 行目に定義があります。

8.17.3.4 invert()

```
GgMatrix gg::GgMatrix::invert () const [inline]
```

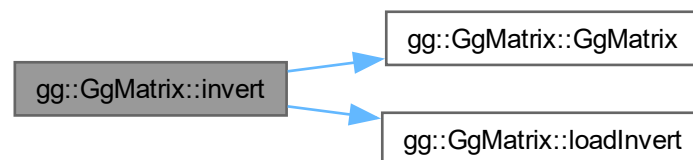
逆行列を返す.

戻り値

逆行列.

[gg.h](#) の 3013 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.5 loadFrustum()

```
gg::GgMatrix & gg::GgMatrix::loadFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

透視透視投影変換行列を格納する.

引数

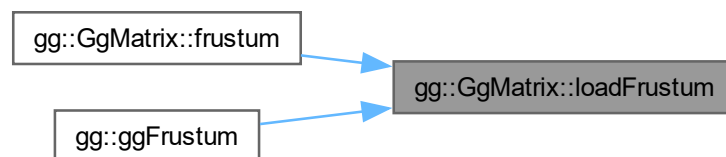
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

gg.cpp の 3129 行目に定義があります。

被呼び出し関係図:



8.17.3.6 loadIdentity()

```
gg::GgMatrix & gg::GgMatrix::loadIdentity ()
```

単位行列を格納する.

gg.cpp の 2740 行目に定義があります。

被呼び出し関係図:



8.17.3.7 loadInvert() [1/2]

```
GgMatrix & gg::GgMatrix::loadInvert (
    const GgMatrix & m) [inline]
```

逆行列を格納する.

引数

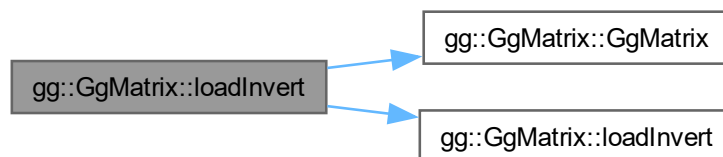
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

設定した *m* の逆行列.

`gg.h` の 2692 行目に定義があります。

呼び出し関係図:



8.17.3.8 loadInvert() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadInvert (
    const GLfloat * a)
```

逆行列を格納する.

引数

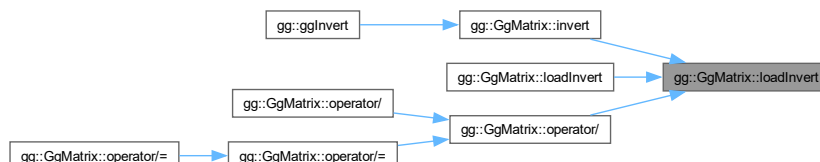
a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した a の逆行列.

gg.cpp の 2915 行目に定義があります。

被呼び出し関係図:



8.17.3.9 loadLookat() [1/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を格納する.

引数

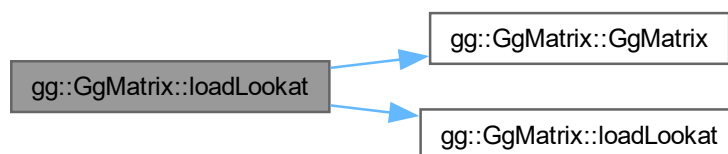
e	視点の位置の GgVector 型の変数.
t	目標点の位置の GgVector 型の変数.
u	上方向のベクトルの GgVector 型の変数.

戻り値

設定したビュー変換行列.

gg.h の 2612 行目に定義があります。

呼び出し関係図:



8.17.3.10 loadLookat() [2/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を格納する.

引数

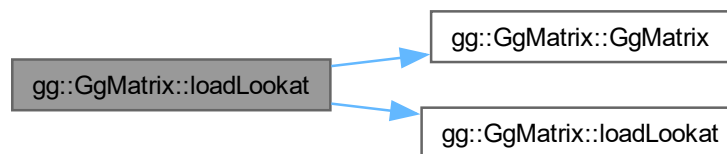
<i>e</i>	視点の位置の配列変数.
<i>t</i>	目標点の位置の配列変数.
<i>u</i>	上方向のベクトルの配列変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2599 行目に定義があります。

呼び出し関係図:



8.17.3.11 loadLookat() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz)
```

ビュー変換行列を格納する.

引数

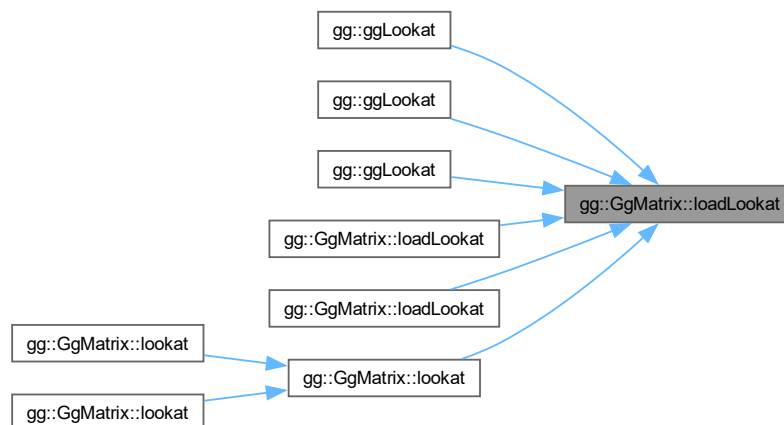
<i>ex</i>	視点の位置の <i>x</i> 座標値.
<i>ey</i>	視点の位置の <i>y</i> 座標値.
<i>ez</i>	視点の位置の <i>z</i> 座標値.
<i>tx</i>	目標点の位置の <i>x</i> 座標値.
<i>ty</i>	目標点の位置の <i>y</i> 座標値.
<i>tz</i>	目標点の位置の <i>z</i> 座標値.
<i>ux</i>	上方向のベクトルの <i>x</i> 成分.
<i>uy</i>	上方向のベクトルの <i>y</i> 成分.
<i>uz</i>	上方向のベクトルの <i>z</i> 成分.

戻り値

設定したビュー変換行列.

[gg.cpp](#) の 3029 行目に定義があります。

被呼び出し関係図:



8.17.3.12 loadNormal() [1/2]

```
GgMatrix & gg::GgMatrix::loadNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を格納する.

引数

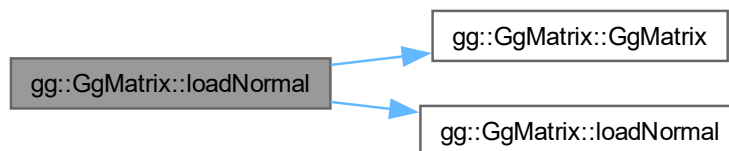
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

設定した m の法線変換行列.

[gg.h](#) の 2711 行目に定義があります。

呼び出し関係図:



8.17.3.13 loadNormal() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadNormal (
    const GLfloat * a)
```

法線変換行列を格納する.

引数

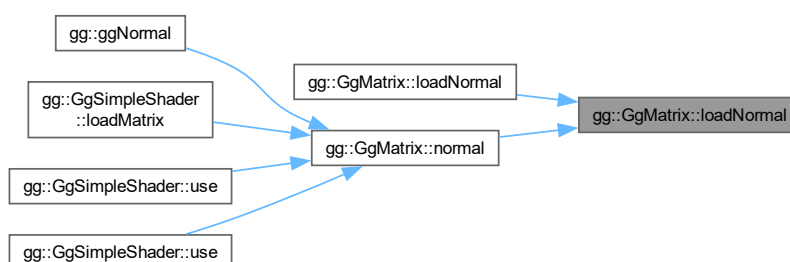
a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した m の法線変換行列.

[gg.cpp](#) の 2998 行目に定義があります。

被呼び出し関係図:



8.17.3.14 loadOrthogonal()

```
gg::GgMatrix & gg::GgMatrix::loadOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

直交投影変換行列を格納する。

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した直交投影変換行列.

gg.cpp の 3088 行目に定義があります。

被呼び出し関係図:



8.17.3.15 loadPerspective()

```
gg::GgMatrix & gg::GgMatrix::loadPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar)
```

画角を指定して透視投影変換行列を格納する。

引数

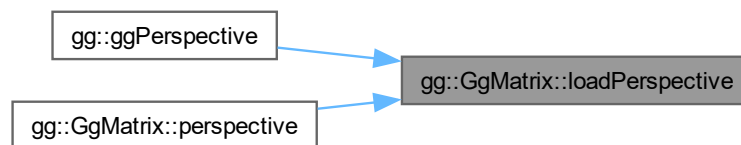
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

`gg.cpp` の 3170 行目に定義があります。

被呼び出し関係図:



8.17.3.16 loadRotate() [1/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

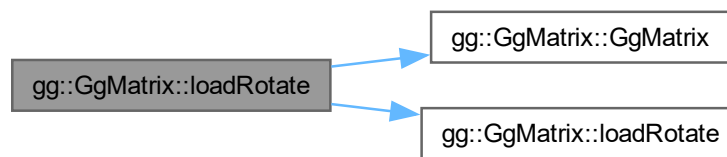
<i>r</i>	回転軸の方向ベクトルと回転角を格納した <code>GgVector</code> 型のベクトル, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

`gg.h` の 2568 行目に定義があります。

呼び出し関係図:



8.17.3.17 loadRotate() [2/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を格納する.

引数

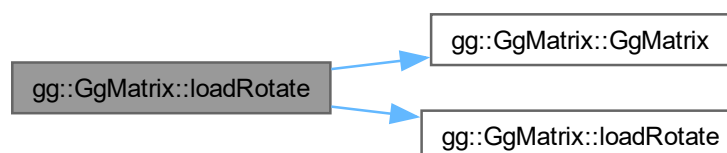
<code>r</code>	回転軸の方向ベクトルを格納した <code>GgVector</code> 型のベクトル, 第 4 要素は無視する.
<code>a</code>	回転角.

戻り値

設定した変換行列.

`gg.h` の 2546 行目に定義があります。

呼び出し関係図:



8.17.3.18 loadRotate() [3/5]

```
GgMatrix & gg::GgMatrix::loadRotate (  
    const GLfloat * r) [inline]
```

r方向のベクトルを軸とする回転の変換行列を格納する.

引数

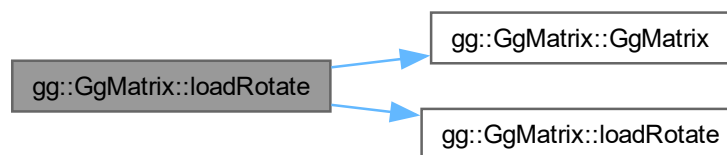
<i>r</i>	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

gg.h の 2557 行目に定義があります。

呼び出し関係図:



8.17.3.19 loadRotate() [4/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

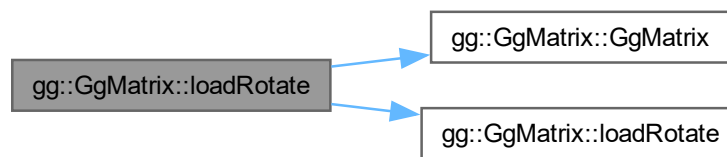
<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

設定した変換行列.

gg.h の 2534 行目に定義があります。

呼び出し関係図:



8.17.3.20 loadRotate() [5/5]

```
gg::GgMatrix & gg::GgMatrix::loadRotate (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat a)
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を格納する.

引数

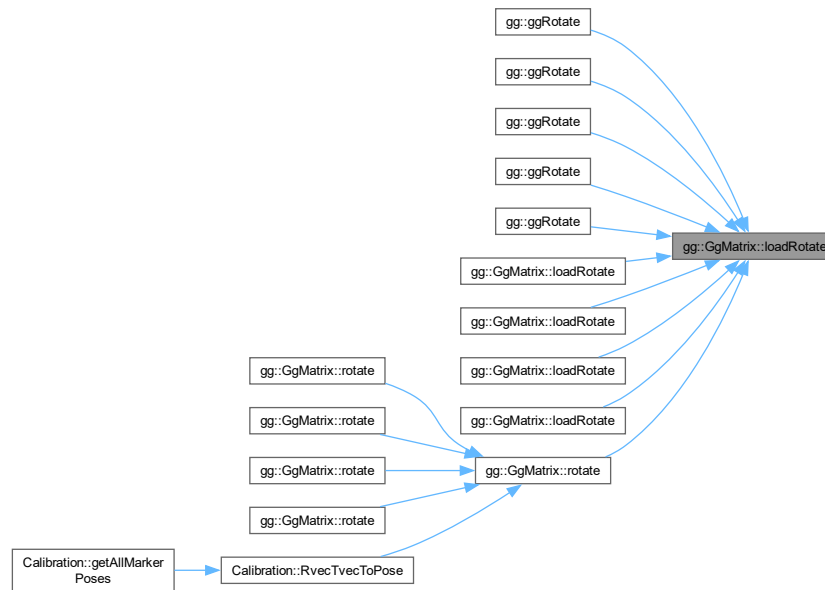
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

設定した変換行列.

[gg.cpp](#) の 2845 行目に定義があります。

被呼び出し関係図:



8.17.3.21 loadRotateX()

```
gg::GgMatrix & gg::GgMatrix::loadRotateX (
    GLfloat a)
```

x 軸中心の回転の変換行列を格納する.

引数

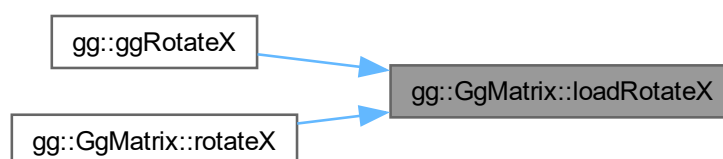
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2788 行目に定義があります。

被呼び出し関係図:



8.17.3.22 loadRotateY()

```
gg::GgMatrix & gg::GgMatrix::loadRotateY (  
    GLfloat a)
```

y 軸中心の回転の変換行列を格納する.

引数

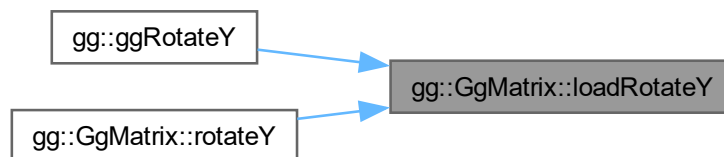
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2807 行目に定義があります。

被呼び出し関係図:



8.17.3.23 loadRotateZ()

```
gg::GgMatrix & gg::GgMatrix::loadRotateZ (  
    GLfloat a)
```

z 軸中心の回転の変換行列を格納する.

引数

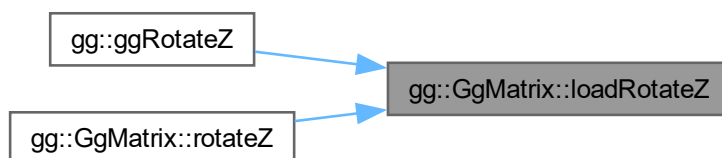
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の [2826](#) 行目に定義があります。

被呼び出し関係図:



8.17.3.24 loadScale() [1/3]

```
GgMatrix & gg::GgMatrix::loadScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を格納する.

引数

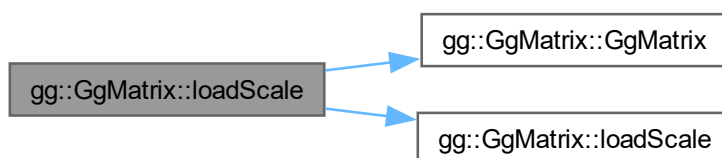
s	拡大率の GgVector 型の変数.
----------	-------------------------------------

戻り値

設定した変換行列.

[gg.h](#) の [2487](#) 行目に定義があります。

呼び出し関係図:



8.17.3.25 loadScale() [2/3]

```
GgMatrix & gg::GgMatrix::loadScale (  
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を格納する.

引数

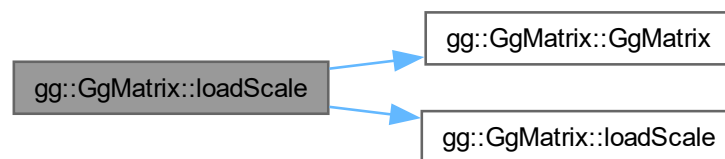
s	拡大率の GLfloat 型の配列 (x, y, z).
---	------------------------------

戻り値

設定した変換行列.

gg.h の 2476 行目に定義があります。

呼び出し関係図:



8.17.3.26 loadScale() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadScale (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w = 1.0f)
```

拡大縮小の変換行列を格納する.

引数

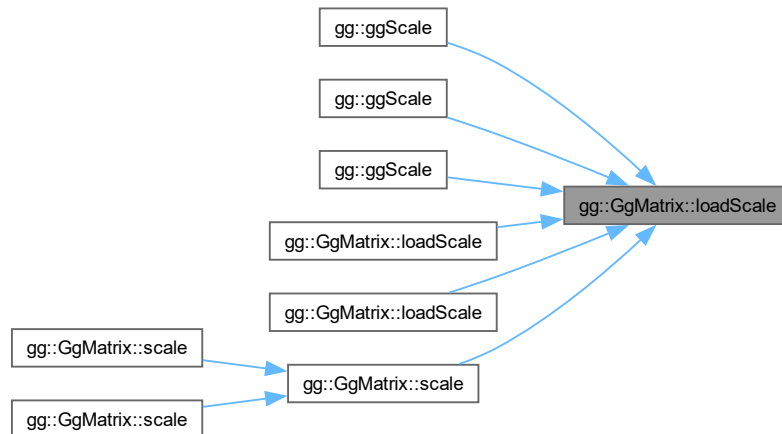
x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 拡大率のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

gg.cpp の 2772 行目に定義があります。

被呼び出し関係図:



8.17.3.27 loadTranslate() [1/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GgVector & t) [inline]
```

平行移動の変換行列を格納する.

引数

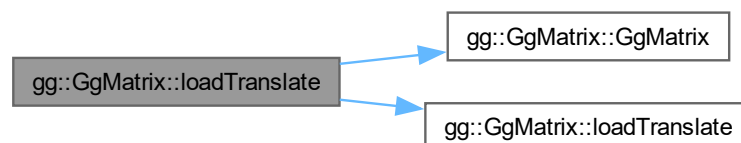
<code>t</code>	移動量の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

設定した変換行列.

gg.h の 2454 行目に定義があります。

呼び出し関係図:



8.17.3.28 loadTranslate() [2/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を格納する.

引数

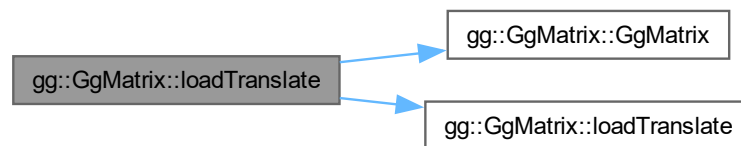
<i>t</i>	移動量の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

gg.h の 2443 行目に定義があります。

呼び出し関係図:



8.17.3.29 loadTranslate() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

平行移動の変換行列を格納する.

引数

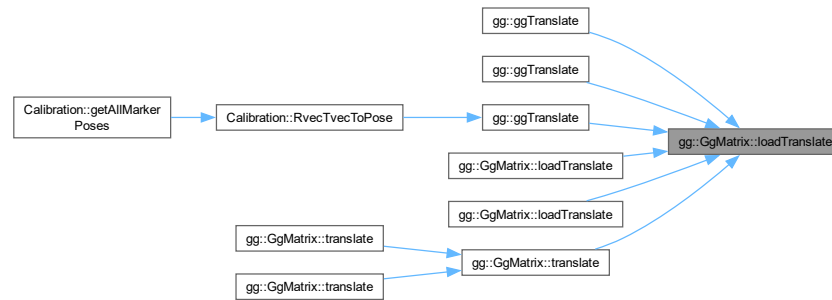
<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	w 移動量のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

gg.cpp の 2756 行目に定義があります。

被呼び出し関係図:



8.17.3.30 loadTranspose() [1/2]

```
GgMatrix & gg::GgMatrix::loadTranspose (
    const GgMatrix & m) [inline]
```

転置行列を格納する.

引数

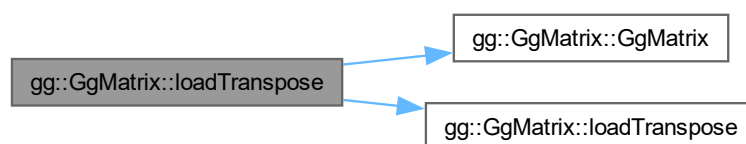
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

設定した *m* の転置行列.

gg.h の 2673 行目に定義があります。

呼び出し関係図:



8.17.3.31 loadTranspose() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadTranspose (
    const GLfloat * a)
```

転置行列を格納する.

引数

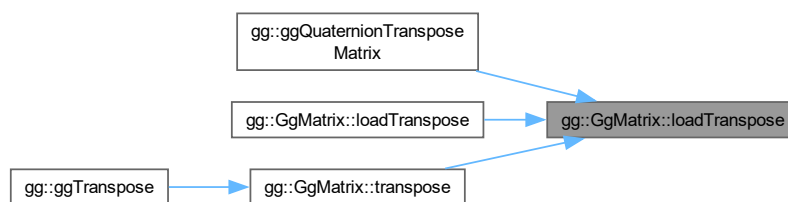
a	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した **a** の転置行列.

[gg.cpp](#) の 2887 行目に定義があります。

被呼び出し関係図:



8.17.3.32 lookat() [1/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

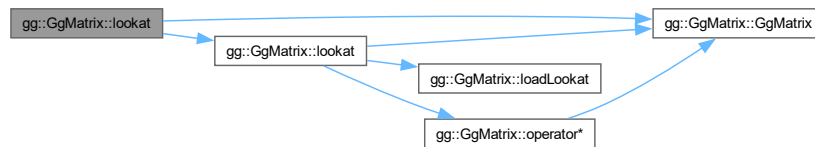
e	視点の位置を格納した GgVector 型の変数.
t	目標点の位置を格納した GgVector 型の変数.
u	上方向のベクトルを格納した GgVector 型の変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2932 行目に定義があります。

呼び出し関係図:



8.17.3.33 lookat() [2/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

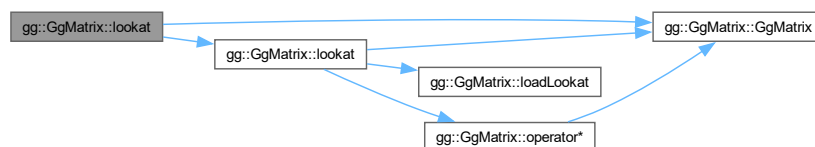
<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2919 行目に定義があります。

呼び出し関係図:



8.17.3.34 `lookat()` [3/3]

```
GgMatrix gg::GgMatrix::lookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) const [inline]
```

ビュー変換を乗じた結果を返す。

引数

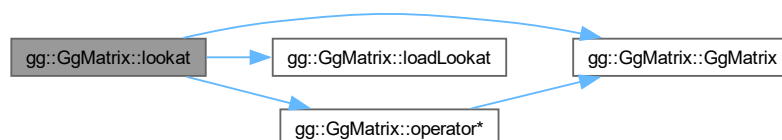
<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

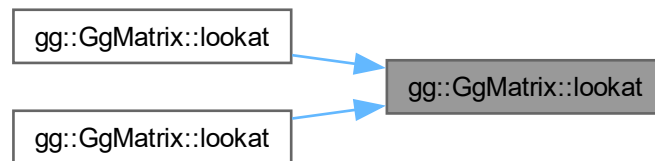
ビュー変換行列を乗じた変換行列。

[gg.h](#) の 2901 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.35 normal()

```
GgMatrix gg::GgMatrix::normal () const [inline]
```

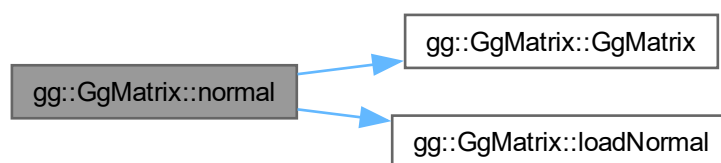
法線変換行列を返す.

戻り値

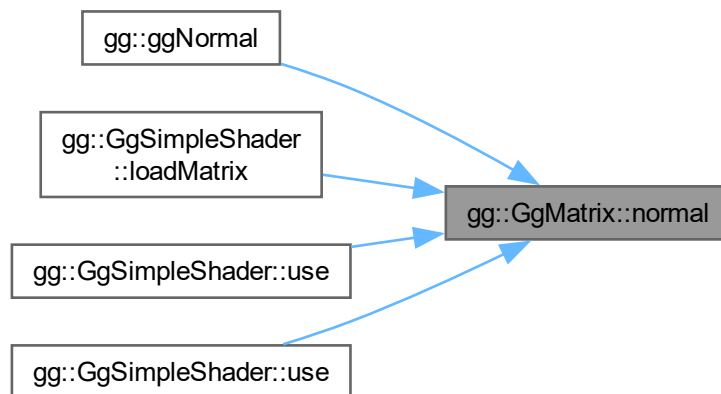
法線変換行列.

[gg.h](#) の [3024](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.36 operator*() [1/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を乗算した値を返す.

引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

変換行列に *a* を乗じた `GgMatrix` 型の変換行列.

`gg.h` の 2345 行目に定義があります。

呼び出し関係図:



8.17.3.37 operator*() [2/3]

```
GgVector gg::GgMatrix::operator* (  
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

v	元のベクトルの GgVector 型の変数.
-----	-------------------------------

戻り値

v 変換結果の **GgVector** 型のベクトル.

gg.h の 3080 行目に定義があります。

8.17.3.38 operator*() [3/3]

```
GgMatrix gg::GgMatrix::operator* (  
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を乗算した値を返す.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
-----	----------------------------------

戻り値

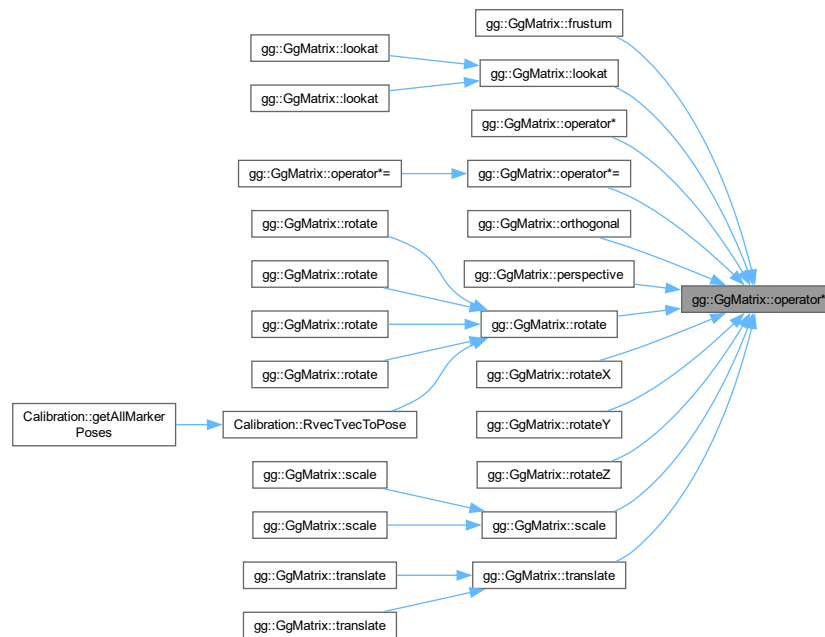
変換行列に a を乗じた **GgMatrix** 型の変換行列.

gg.h の 2332 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.39 operator*=() [1/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を乗算した結果を格納する.

引数

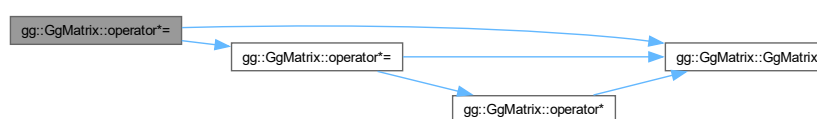
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を掛けた変換行列の参照.

gg.h の 2368 行目に定義があります。

呼び出し関係図:



8.17.3.40 operator*=() [2/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を乗算した結果を格納する.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を掛けた変換行列の参照.

gg.h の 2356 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.41 operator+() [1/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を加算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に *a* を加えた GgMatrix 型の変換行列.

gg.h の 2251 行目に定義があります。

呼び出し関係図:



8.17.3.42 operator+() [2/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を加算した値を返す.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列に *a* を加えた GgMatrix 型の変換行列.

gg.h の 2238 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.43 operator+=() [1/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を加算した結果を格納する.

引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

m を加えた変換行列の参照.

`gg.h` の 2274 行目に定義があります。

呼び出し関係図:



8.17.3.44 operator+=() [2/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を加算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を加えた変換行列の参照.

gg.h の 2262 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.45 operator-() [1/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GgMatrix & m) const [inline]
```

変換行列から別の変換行列を減算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列から `m` を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2298 行目に定義があります。

呼び出し関係図:



8.17.3.46 operator-() [2/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GLfloat * a) const [inline]
```

変換行列から配列に格納した変換行列を減算した結果を格納する.

引数

<code>a</code>	変換行列を格納した <code>GLfloat</code> 型の 16 要素の配列変数.
----------------	---

戻り値

変換行列から `a` を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2285 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.47 operator-=() [1/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を減算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を引いた変換行列の参照.

gg.h の 2321 行目に定義があります。

呼び出し関係図:



8.17.3.48 operator-=() [2/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

`a` を引いた変換行列の参照.

`gg.h` の 2309 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.49 operator/() [1/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GgMatrix & m) const [inline]
```

変換行列を変換行列で除算した値を返す.

引数

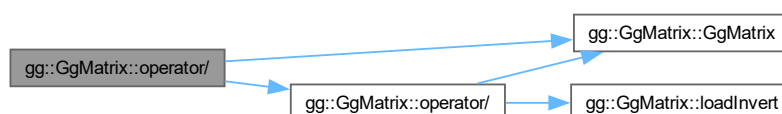
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

変換行列を `a` で割った `GgMatrix` 型の変換行列.

`gg.h` の 2393 行目に定義があります。

呼び出し関係図:



8.17.3.50 operator/() [2/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GLfloat * a) const [inline]
```

変換行列を配列に格納した変換行列で除算した値を返す.

引数

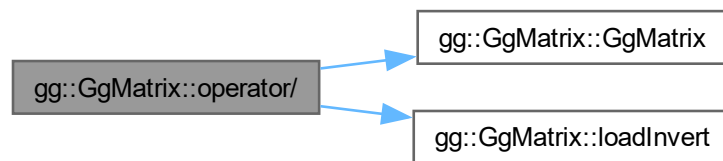
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
---	----------------------------------

戻り値

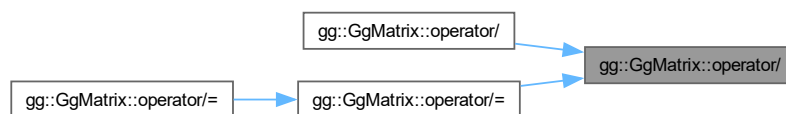
変換行列を a で割った GgMatrix 型の変換行列.

gg.h の 2379 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.51 operator/=() [1/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GgMatrix & m) [inline]
```

変換行列を別の変換行列で除算した結果を格納する.

引数

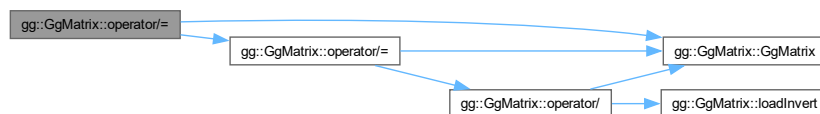
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m で割った変換行列の参照.

gg.h の 2416 行目に定義があります.

呼び出し関係図:



8.17.3.52 operator/=() [2/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GLfloat * a) [inline]
```

変換行列を配列に格納した変換行列で除算した結果を格納する.

引数

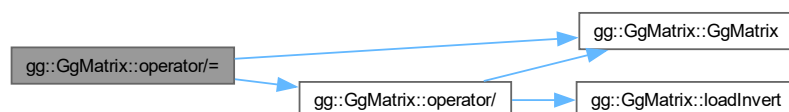
<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a で割った変換行列の参照.

gg.h の 2404 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.17.3.53 operator=() [1/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GgMatrix & m) [default]
```

代入演算子. 呼び出し関係図:



8.17.3.54 operator=() [2/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GLfloat * a) [inline]
```

配列変数の値を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

戻り値

a を代入したこのオブジェクトの参照.

[gg.h](#) の 2226 行目に定義があります。

呼び出し関係図:



8.17.3.55 operator=() [3/3]

```
GgMatrix & gg::GgMatrix::operator= (
    GgMatrix && m) [default]
```

ムーブ代入演算子. 呼び出し関係図:



8.17.3.56 orthogonal()

```
GgMatrix gg::GgMatrix::orthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

直交投影変換を乗じた結果を返す.

引数

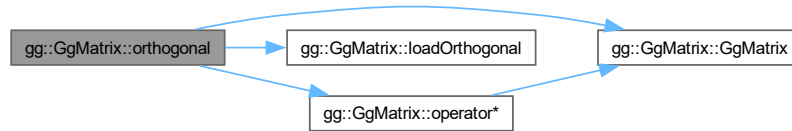
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

直交投影変換行列を乗じた変換行列.

gg.h の 2948 行目に定義があります。

呼び出し関係図:



8.17.3.57 perspective()

```

GgMatrix gg::GgMatrix::perspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) const [inline]
  
```

画角を指定して透視投影変換を乗じた結果を返す.

引数

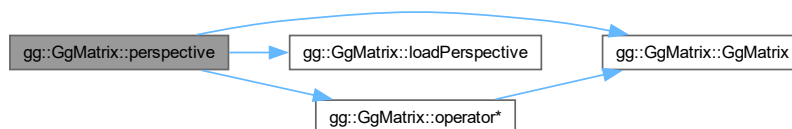
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 2988 行目に定義があります。

呼び出し関係図:



8.17.3.58 projection() [1/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3069 行目に定義があります。

8.17.3.59 projection() [2/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GLfloat 型の 4 要素の配列変数.

[gg.h](#) の 3058 行目に定義があります。

8.17.3.60 projection() [3/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GLfloat 型の 4 要素の配列変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3047 行目に定義があります。

8.17.3.61 projection() [4/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GLfloat 型の 4 要素の配列変数.
<i>v</i>	元のベクトルの GLfloat 型の 4 要素の配列変数.

gg.h の 3036 行目に定義があります。

8.17.3.62 rotate() [1/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸の方向ベクトルと回転角を格納した GgVector 型の変数).
----------	-------------------------------------

戻り値

(*r*[0], *r*[1], *r*[2]) を軸にさらに *r*[3] 回転した変換行列.

gg.h の 2882 行目に定義があります。

呼び出し関係図:



8.17.3.63 rotate() [2/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

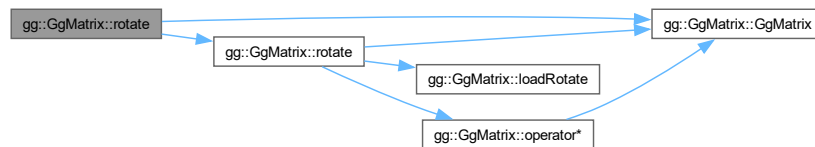
<i>r</i>	回転軸の方向ベクトルを格納した GgVector 型の変数.
<i>a</i>	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

gg.h の 2860 行目に定義があります.

呼び出し関係図:



8.17.3.64 rotate() [3/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

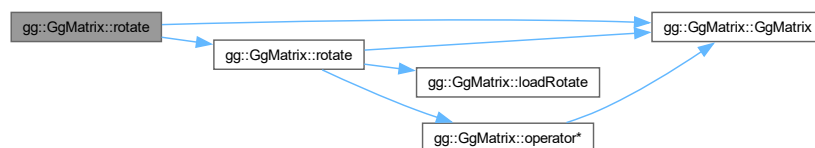
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数 (x , y , z , a).
-----	---

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

gg.h の 2871 行目に定義があります.

呼び出し関係図:



8.17.3.65 rotate() [4/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

(*r*[0], *r*[1], *r*[2]) を軸にさらに *a* 回転した変換行列.

gg.h の 2848 行目に定義があります。

呼び出し関係図:



8.17.3.66 rotate() [5/5]

```

GgMatrix gg::GgMatrix::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]

```

(x, y, z) 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

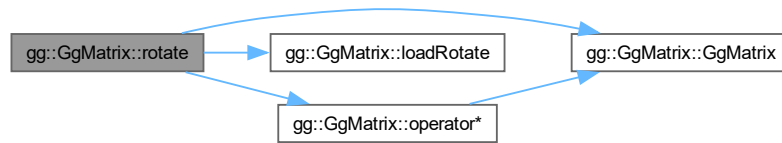
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

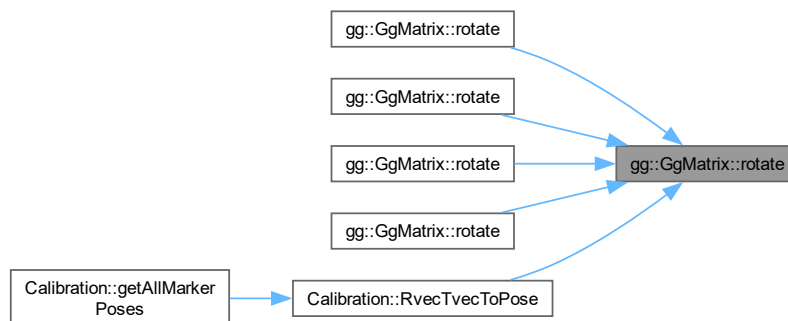
(x, y, z) を軸にさらに *a* 回転した変換行列.

gg.h の 2835 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.67 rotateX()

```
GgMatrix gg::GgMatrix::rotateX (
    GLfloat a) const [inline]
```

x 軸中心の回転変換を乗じた結果を返す.

引数

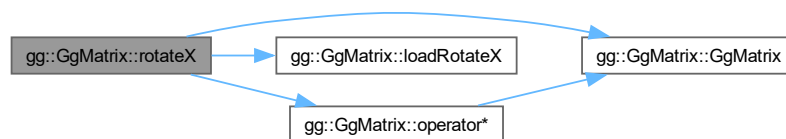
<i>a</i>	回転角.
----------	------

戻り値

x 軸中心にさらに *a* 回転した変換行列.

[gg.h](#) の 2796 行目に定義があります。

呼び出し関係図:



8.17.3.68 rotateY()

```
GgMatrix gg::GgMatrix::rotateY (
    GLfloat a) const [inline]
```

y 軸中心の回転変換を乗じた結果を返す.

引数

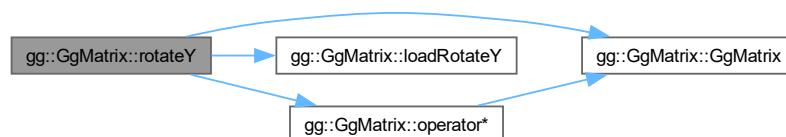
<i>a</i>	回転角.
----------	------

戻り値

y 軸中心にさらに *a* 回転した変換行列.

[gg.h](#) の 2808 行目に定義があります。

呼び出し関係図:



8.17.3.69 rotateZ()

```
GgMatrix gg::GgMatrix::rotateZ (
    GLfloat a) const [inline]
```

z 軸中心の回転変換を乗じた結果を返す.

引数

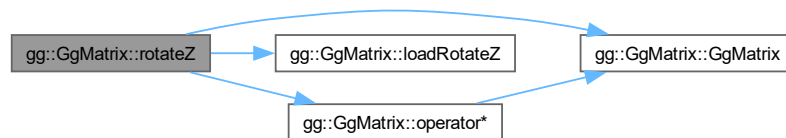
<code>a</code>	回転角.
----------------	------

戻り値

z 軸中心にさらに a 回転した変換行列.

`gg.h` の 2820 行目に定義があります。

呼び出し関係図:



8.17.3.70 scale() [1/3]

```
GgMatrix gg::GgMatrix::scale (
    const GgVector & s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

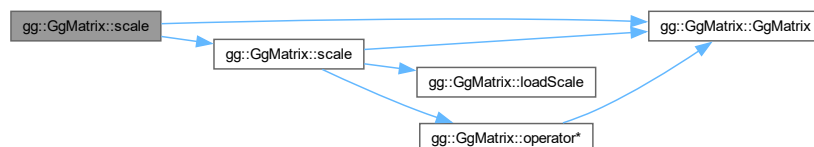
<code>s</code>	拡大率の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

拡大縮小した結果の変換行列.

`gg.h` の 2785 行目に定義があります。

呼び出し関係図:



8.17.3.71 scale() [2/3]

```
GgMatrix gg::GgMatrix::scale (  
    const GLfloat * s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

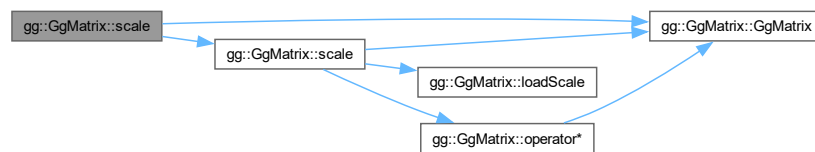
s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2774 行目に定義があります。

呼び出し関係図:



8.17.3.72 scale() [3/3]

```

GgMatrix gg::GgMatrix::scale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]

```

拡大縮小変換を乗じた結果を返す.

引数

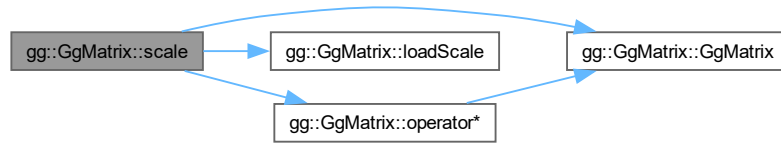
x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 移動量のスケールファクタ (= 1.0f).

戻り値

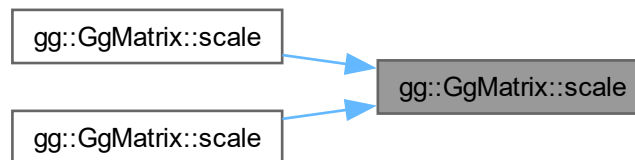
拡大縮小した結果の変換行列.

gg.h の 2762 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.73 translate() [1/3]

```
GgMatrix gg::GgMatrix::translate (
    const GgVector & t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

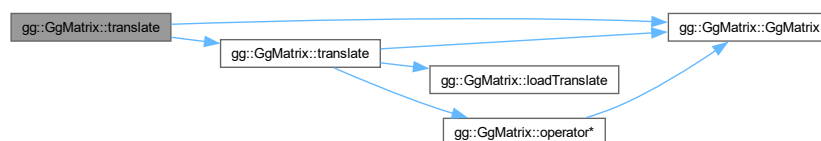
<i>t</i>	移動量の GgVector 型の変数.
----------	-------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2748 行目に定義があります。

呼び出し関係図:



8.17.3.74 translate() [2/3]

```
GgMatrix gg::GgMatrix::translate (
    const GLfloat * t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

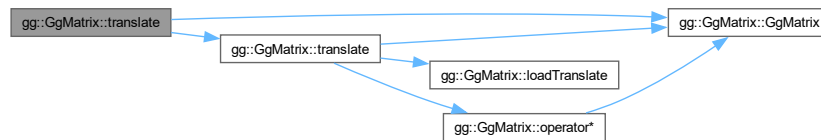
<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動した結果の変換行列.

gg.h の 2737 行目に定義があります。

呼び出し関係図:



8.17.3.75 translate() [3/3]

```
GgMatrix gg::GgMatrix::translate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

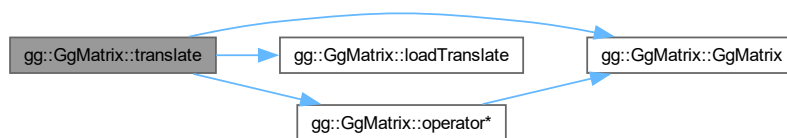
<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

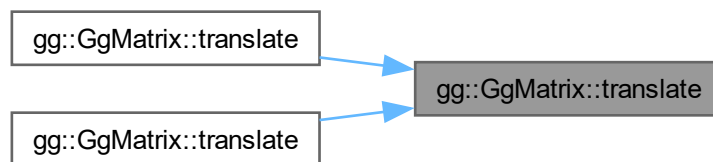
平行移動した結果の変換行列.

gg.h の 2725 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.76 transpose()

```
GgMatrix gg::GgMatrix::transpose () const [inline]
```

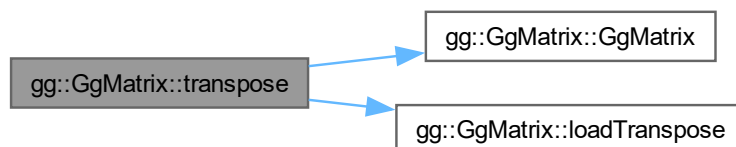
転置行列を返す.

戻り値

転置行列.

gg.h の 3002 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.18 gg::GgNormalTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgNormalTexture](#) ()
- [GgNormalTexture](#) (const GLubyte *image, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [GgNormalTexture](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- virtual [~GgNormalTexture](#) ()
- void [load](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- void [load](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)

8.18.1 詳解

法線マップ.

覚え書き

高さマップ（グレイスケール画像）を読み込んで法線マップのテクスチャを作成する.

[gg.h](#) の 5460 行目に定義があります。

8.18.2 構築子と解体子

8.18.2.1 GgNormalTexture() [1/3]

```
gg::GgNormalTexture::GgNormalTexture () [inline]
```

コンストラクタ.

[gg.h](#) の 5470 行目に定義があります。

8.18.2.2 GgNormalTexture() [2/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const GLubyte * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

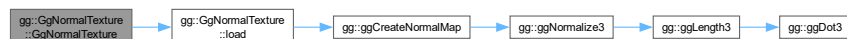
メモリ上のデータから法線マップのテクスチャを作成するコンストラクタ。

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5484 行目に定義があります。

呼び出し関係図:



8.18.2.3 GgNormalTexture() [3/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

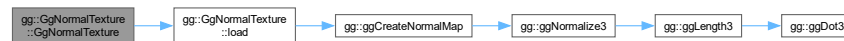
ファイルからデータを読み込んで法線マップのテクスチャを作成するコンストラクタ。

引数

<i>name</i>	画像ファイル名.
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5504 行目に定義があります。

呼び出し関係図:



8.18.2.4 ~GgNormalTexture()

```
virtual gg::GgNormalTexture::~~GgNormalTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5517 行目に定義があります。

8.18.3 関数詳解

8.18.3.1 load() [1/2]

```
void gg::GgNormalTexture::load (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

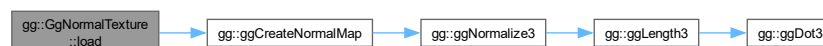
メモリ上のデータから法線マップのテクスチャを作成する.

引数

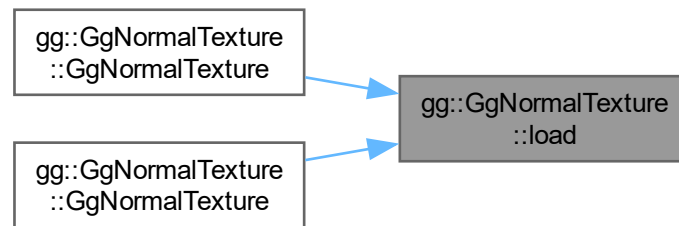
<i>hmap</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.h](#) の 5531 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.18.3.2 load() [2/2]

```

void gg::GgNormalTexture::load (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA)
  
```

TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	画像ファイル名 (1 チャンネルの TGA 画像).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.cpp の 4064 行目に定義があります。

呼び出し関係図:



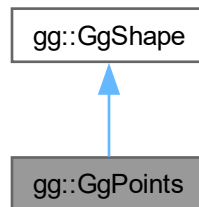
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

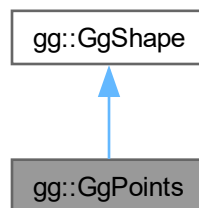
8.19 gg::GgPoints クラス

```
#include <gg.h>
```

gg::GgPoints の継承関係図



gg::GgPoints 連携図



公開メンバ関数

- **GgPoints** (GLenum mode=GL_POINTS)
- **GgPoints** (const **GgVector** *pos, GLsizei countv, GLenum mode=GL_POINTS, GLenum usage=GL_STATIC_DRAW↔_DRAW)
- virtual **~GgPoints** ()
- **operator bool** () const noexcept
- bool **operator!** () const noexcept
- const GLsizei & **getCount** () const
- const GLuint & **getBuffer** () const
- void **send** (const **GgVector** *pos, GLint first=0, GLsizei count=0) const
- void **load** (const **GgVector** *pos, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void **draw** (GLint first=0, GLsizei count=0) const

基底クラス **gg::GgShape** に属する継承公開メンバ関数

- **GgShape** (GLenum mode=0)
- virtual ~**GgShape** ()
- const GLuint & **get** () const
- void **setMode** (GLenum mode)
- const GLenum & **getMode** () const

8.19.1 詳解

点.

gg.h の 6333 行目に定義があります。

8.19.2 構築子と解体子

8.19.2.1 GgPoints() [1/2]

```
gg::GgPoints::GgPoints (
    GLenum mode = GL_POINTS) [inline]
```

コンストラクタ.

gg.h の 6344 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.19.2.2 GgPoints() [2/2]

```
gg::GgPoints::GgPoints (  
    const GgVector * pos,  
    GLsizei countv,  
    GLenum mode = GL_POINTS,  
    GLenum usage = GL_STATIC_DRAW) [inline]
```

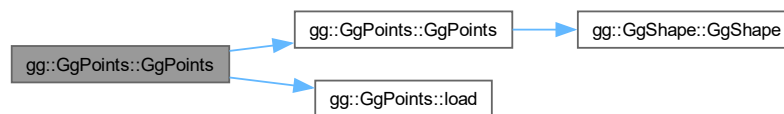
コンストラクタ.

引数

<i>pos</i>	この図形の頂点の位置のデータの配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6357 行目に定義があります。

呼び出し関係図:



8.19.2.3 ~GgPoints()

```
virtual gg::GgPoints::~GgPoints () [inline], [virtual]
```

デストラクタ.

gg.h の 6371 行目に定義があります。

8.19.3 関数詳解

8.19.3.1 draw()

```
void gg::GgPoints::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

点の描画.

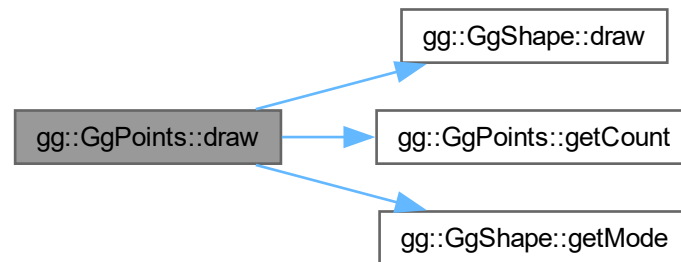
引数

<i>first</i>	描画を開始する最初の点の番号.
<i>count</i>	描画する点の数, 0 なら全部の点を描く.

gg::GgShape を再実装しています。

gg.cpp の 5126 行目に定義があります。

呼び出し関係図:



8.19.3.2 getBuffer()

```
const GLuint & gg::GgPoints::getBuffer () const [inline]
```

頂点の位置データを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点の位置データを格納した頂点バッファオブジェクト名.

[gg.h](#) の 6410 行目に定義があります。

8.19.3.3 getCount()

```
const GLsizei & gg::GgPoints::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点の位置データの数 (頂点数).

[gg.h](#) の 6400 行目に定義があります。

被呼び出し関係図:



8.19.3.4 load()

```
void gg::GgPoints::load (
    const GgVector * pos,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点の位置データを格納する。

引数

<i>pos</i>	頂点の位置データが格納されている領域の先頭のポインタ。
<i>count</i>	頂点のデータの数 (頂点数)。
<i>usage</i>	バッファオブジェクトの使い方。

gg.cpp の 5113 行目に定義があります。

被呼び出し関係図:



8.19.3.5 operator bool()

```
gg::GgPoints::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

バッファが有効かどうか調べる。

戻り値

バッファが有効なら true

gg::GgShape を再実装しています。

gg.h の 6380 行目に定義があります。

8.19.3.6 operator"!()

```
bool gg::GgPoints::operator! () const [inline], [virtual], [noexcept]
```

バッファが有効かどうかの結果を反転する。

戻り値

バッファが有効なら false, 無効なら true.

gg::GgShape を再実装しています。

gg.h の 6390 行目に定義があります。

8.19.3.7 send()

```
void gg::GgPoints::send (
    const GgVector * pos,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点の位置データを転送する.

引数

<i>pos</i>	転送元の頂点の位置データが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6422 行目に定義があります。

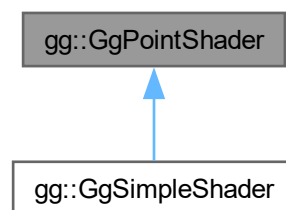
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.20 gg::GgPointShader クラス

```
#include <gg.h>
```

gg::GgPointShader の継承関係図



公開メンバ関数

- [GgPointShader](#) ()
- [GgPointShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgPointShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual [~GgPointShader](#) ()
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadProjectionMatrix](#) (const GLfloat *mp) const
- virtual void [loadProjectionMatrix](#) (const [GgMatrix](#) &mp) const
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- virtual void [use](#) () const
- void [use](#) (const GLfloat *mp) const
- void [use](#) (const [GgMatrix](#) &mp) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [unuse](#) () const
- GLuint [get](#) () const

8.20.1 詳解

点のシェーダ.

[gg.h](#) の 6977 行目に定義があります。

8.20.2 構築子と解体子

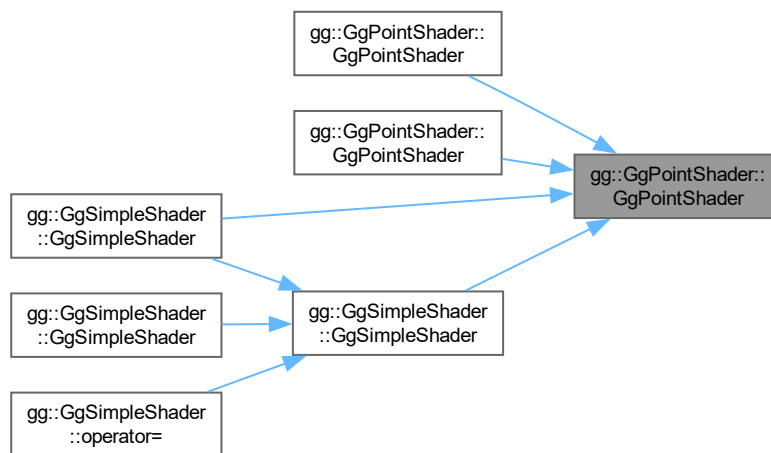
8.20.2.1 GgPointShader() [1/3]

```
gg::GgPointShader::GgPointShader () [inline]
```

コンストラクタ.

[gg.h](#) の 6993 行目に定義があります。

被呼び出し関係図:



8.20.2.2 GgPointShader() [2/3]

```

gg::GgPointShader::GgPointShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]

```

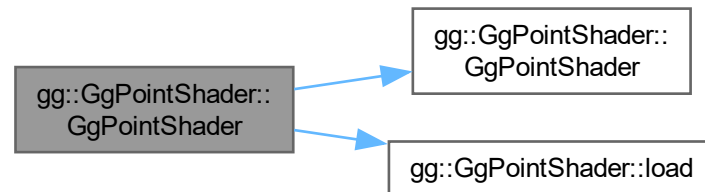
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7008 行目に定義があります。

呼び出し関係図:



8.20.2.3 GgPointShader() [3/3]

```

gg::GgPointShader::GgPointShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

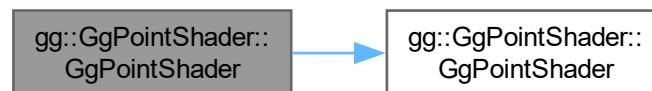
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7027 行目に定義があります。

呼び出し関係図:



8.20.2.4 ~GgPointShader()

```

virtual gg::GgPointShader::~GgPointShader () [inline], [virtual]
  
```

デストラクタ.

[gg.h](#) の 7039 行目に定義があります。

8.20.3 関数詳解

8.20.3.1 get()

```
GLuint gg::GgPointShader::get () const [inline]
```

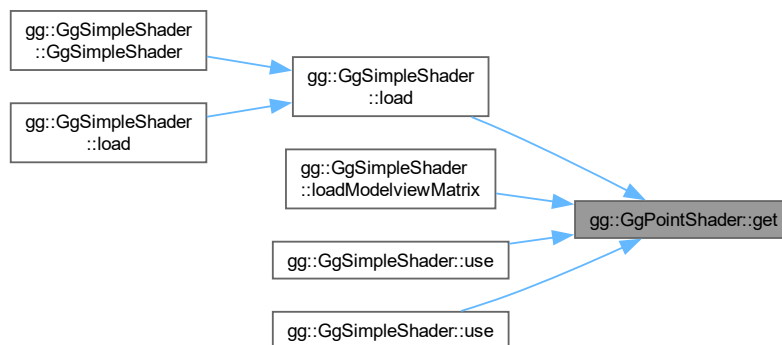
シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

[gg.h](#) の 7223 行目に定義があります。

被呼び出し関係図:



8.20.3.2 load() [1/2]

```
bool gg::GgPointShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら `true`.

[gg.h](#) の 7086 行目に定義があります。

呼び出し関係図:



8.20.3.3 load() [2/2]

```

bool gg::GgPointShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

シェーダのソースファイルを読み込む。

引数

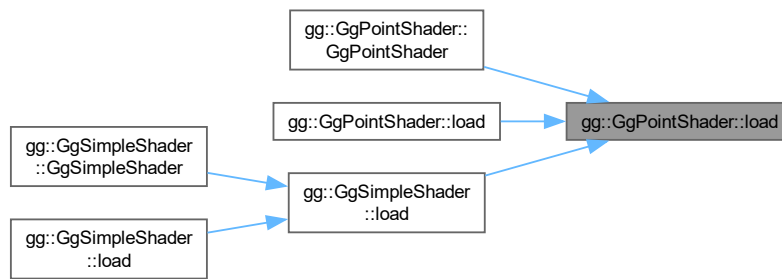
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <code>varying</code> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <code>varying</code> 変数のリスト.

戻り値

プログラムオブジェクトが作成できれば `true`.

[gg.h](#) の 7053 行目に定義があります。

被呼び出し関係図:



8.20.3.4 loadMatrix() [1/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

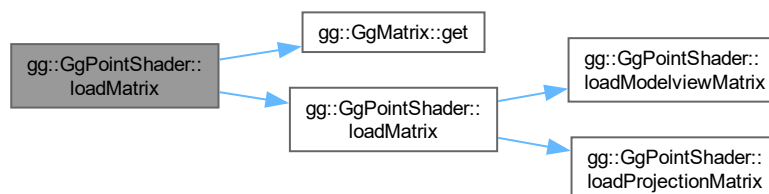
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg::GgSimpleShader](#)で再実装されています。

[gg.h](#) の 7153 行目に定義があります。

呼び出し関係図:



8.20.3.5 loadMatrix() [2/2]

```
virtual void gg::GgPointShader::loadMatrix (  
    const GLfloat * mp,  
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

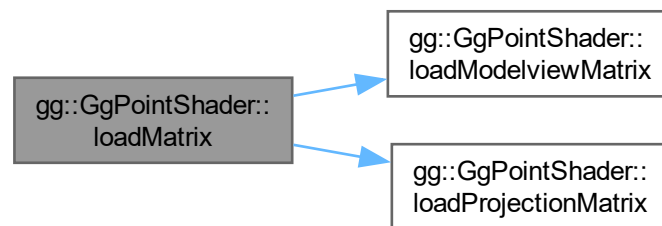
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

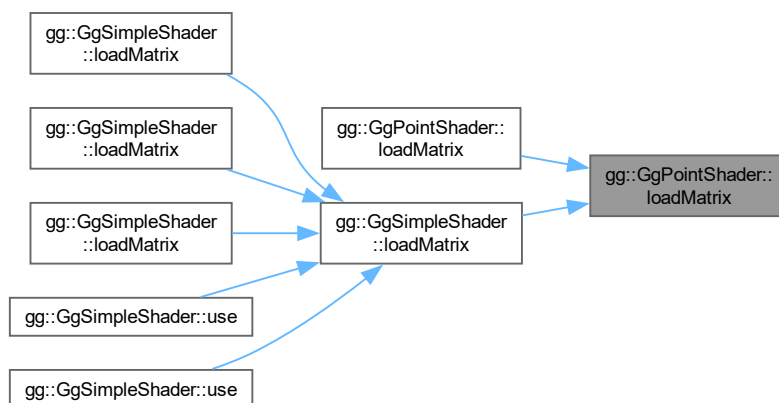
`gg::GgSimpleShader`で再実装されています。

`gg.h` の 7141 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.6 loadModelviewMatrix() [1/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

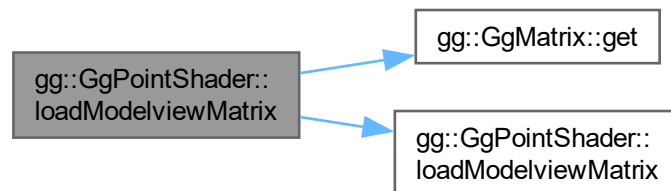
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgSimpleShaderで再実装されています。

gg.h の 7130 行目に定義があります。

呼び出し関係図:



8.20.3.7 loadModelviewMatrix() [2/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

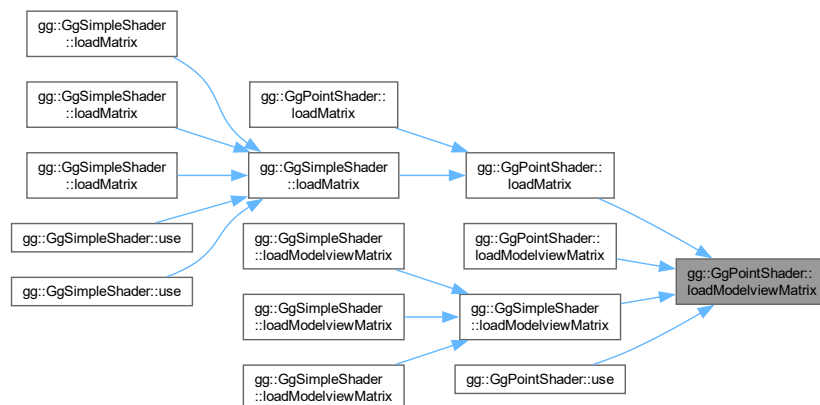
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgSimpleShaderで再実装されています。

gg.h の 7120 行目に定義があります。

被呼び出し関係図:



8.20.3.8 loadProjectionMatrix() [1/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GgMatrix & mp) const [inline], [virtual]
```

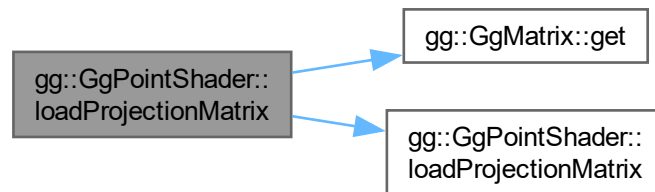
投影変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7110 行目に定義があります。

呼び出し関係図:



8.20.3.9 loadProjectionMatrix() [2/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GLfloat * mp) const [inline], [virtual]
```

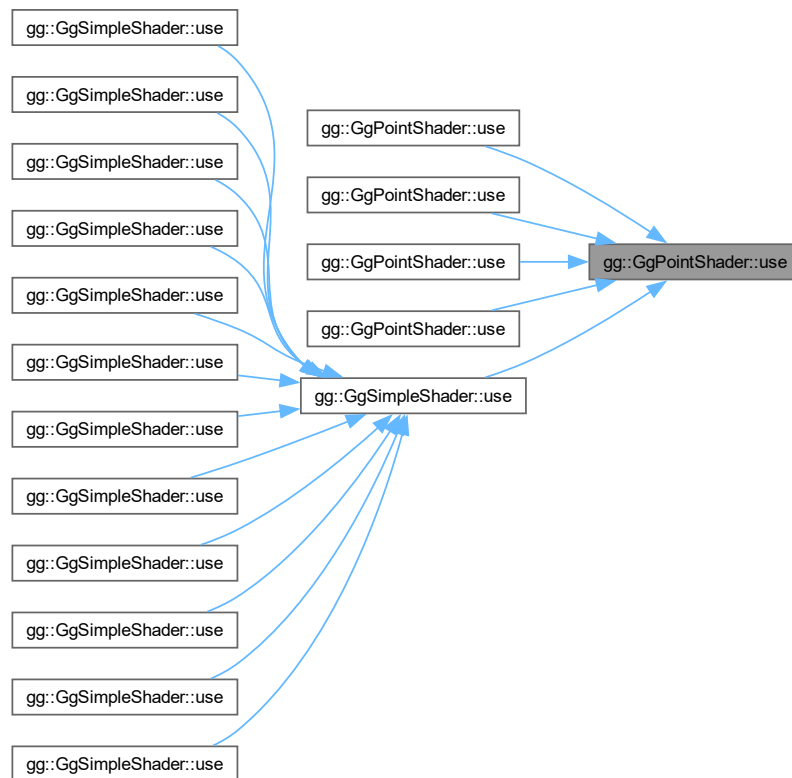
投影変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7100 行目に定義があります。

被呼び出し関係図:



8.20.3.12 use() [2/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp) const [inline]
```

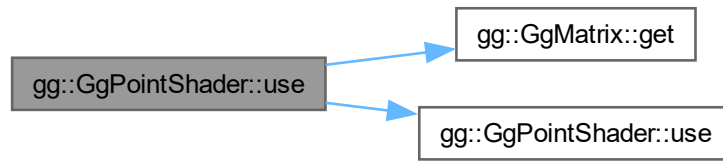
投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7182 行目に定義があります。

呼び出し関係図:



8.20.3.13 use() [3/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

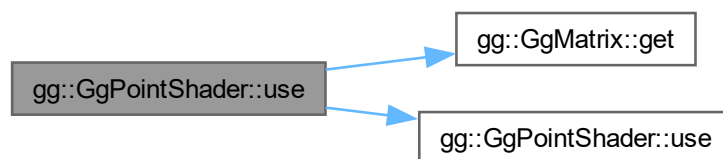
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg.h の 7205 行目に定義があります。

呼び出し関係図:



8.20.3.14 use() [4/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp) const [inline]
```

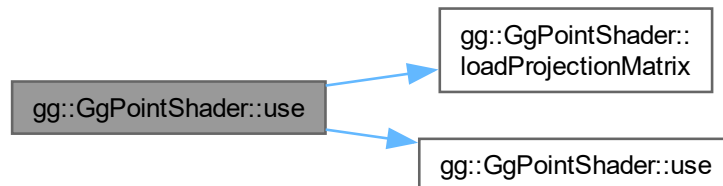
投影変換行列を設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

[gg.h](#) の 7171 行目に定義があります。

呼び出し関係図:



8.20.3.15 use() [5/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

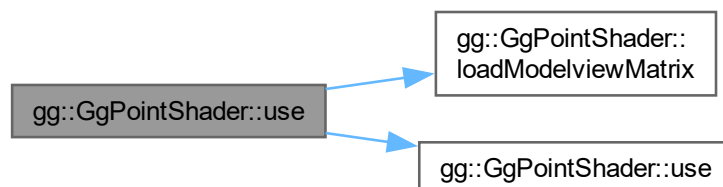
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

[gg.h](#) の 7193 行目に定義があります。

呼び出し関係図:



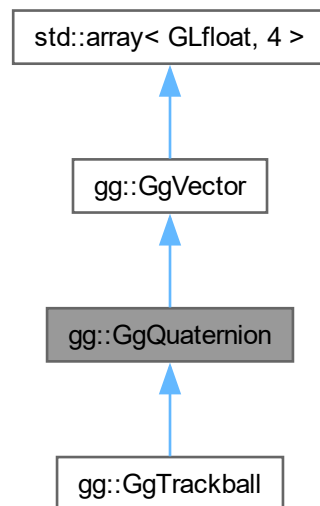
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

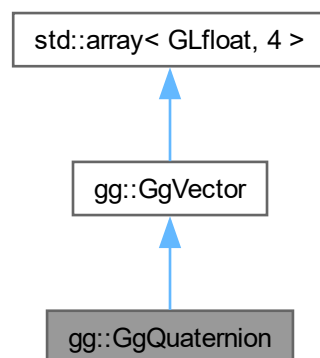
8.21 gg::GgQuaternion クラス

```
#include <gg.h>
```

gg::GgQuaternion の継承関係図



gg::GgQuaternion 連携図



公開メンバ関数

- [GgQuaternion](#) ()=default

- constexpr [GgQuaternion](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- constexpr [GgQuaternion](#) (GLfloat c)
- [GgQuaternion](#) (const GLfloat *a)
- [GgQuaternion](#) (const [GgVector](#) &v)
- [GgQuaternion](#) (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) ([GgQuaternion](#) &&q)=default
- ~[GgQuaternion](#) ()=default
- [GgQuaternion](#) & operator= (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) & operator= ([GgQuaternion](#) &&q)=default
- GLfloat [norm](#) () const
- [GgQuaternion](#) & [loadAdd](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadAdd](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadAdd](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadAdd](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadSubtract](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadSubtract](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadMultiply](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadMultiply](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadDivide](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadDivide](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadDivide](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadDivide](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) add (const GLfloat *a) const
- [GgQuaternion](#) add (const [GgVector](#) &v) const
- [GgQuaternion](#) add (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) subtract (const GLfloat *a) const
- [GgQuaternion](#) subtract (const [GgVector](#) &v) const
- [GgQuaternion](#) subtract (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) multiply (const GLfloat *a) const
- [GgQuaternion](#) multiply (const [GgVector](#) &v) const
- [GgQuaternion](#) multiply (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) divide (const GLfloat *a) const
- [GgQuaternion](#) divide (const [GgVector](#) &v) const
- [GgQuaternion](#) divide (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) & operator= (const GLfloat *a)
- [GgQuaternion](#) & operator= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator+= (const GLfloat *a)
- [GgQuaternion](#) & operator+= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator+= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator-= (const GLfloat *a)
- [GgQuaternion](#) & operator-= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator-= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator*= (const GLfloat *a)
- [GgQuaternion](#) & operator*= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator*= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator/= (const GLfloat *a)
- [GgQuaternion](#) & operator/= (const [GgVector](#) &v)

- [GgQuaternion & operator/=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion operator+](#) (const GLfloat *a) const
- [GgQuaternion operator+](#) (const [GgVector](#) &v) const
- [GgQuaternion operator+](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator-](#) (const GLfloat *a) const
- [GgQuaternion operator-](#) (const [GgVector](#) &v) const
- [GgQuaternion operator-](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator*](#) (const GLfloat *a) const
- [GgQuaternion operator*](#) (const [GgVector](#) &v) const
- [GgQuaternion operator*](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator/](#) (const GLfloat *a) const
- [GgQuaternion operator/](#) (const [GgVector](#) &v) const
- [GgQuaternion operator/](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion & loadMatrix](#) (const GLfloat *a)
- [GgQuaternion & loadMatrix](#) (const [GgMatrix](#) &m)
- [GgQuaternion & loadIdentity](#) ()
- [GgQuaternion & loadRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v)
- [GgQuaternion & loadRotateX](#) (GLfloat a)
- [GgQuaternion & loadRotateY](#) (GLfloat a)
- [GgQuaternion & loadRotateZ](#) (GLfloat a)
- [GgQuaternion rotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v) const
- [GgQuaternion rotateX](#) (GLfloat a) const
- [GgQuaternion rotateY](#) (GLfloat a) const
- [GgQuaternion rotateZ](#) (GLfloat a) const
- [GgQuaternion & loadEuler](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion & loadEuler](#) (const GLfloat *e)
- [GgQuaternion euler](#) (GLfloat heading, GLfloat pitch, GLfloat roll) const
- [GgQuaternion euler](#) (const GLfloat *e) const
- [GgQuaternion euler](#) (const [GgVector](#) &e) const
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- [GgQuaternion & loadNormalize](#) (const GLfloat *a)
- [GgQuaternion & loadNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadConjugate](#) (const GLfloat *a)
- [GgQuaternion & loadConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadInvert](#) (const GLfloat *a)
- [GgQuaternion & loadInvert](#) (const [GgQuaternion](#) &q)
- [GgQuaternion slerp](#) (GLfloat *a, GLfloat t) const
- [GgQuaternion slerp](#) (const [GgQuaternion](#) &q, GLfloat t) const
- [GgQuaternion normalize](#) () const
- [GgQuaternion conjugate](#) () const
- [GgQuaternion invert](#) () const
- void [get](#) (GLfloat *a) const
- void [getMatrix](#) (GLfloat *a) const
- void [getMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getMatrix](#) () const
- void [getConjugateMatrix](#) (GLfloat *a) const
- void [getConjugateMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getConjugateMatrix](#) () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- **GgVector** operator+ (const **GgVector** &v) const
- **GgVector** operator+ (GLfloat c) const
- **GgVector** & operator+= (const **GgVector** &v)
- **GgVector** & operator+= (GLfloat c)
- **GgVector** operator- (const **GgVector** &v) const
- **GgVector** operator- (GLfloat c) const
- **GgVector** & operator-= (const **GgVector** &v)
- **GgVector** & operator-= (GLfloat c)
- **GgVector** operator* (const **GgVector** &v) const
- **GgVector** operator* (GLfloat c) const
- **GgVector** & operator*= (const **GgVector** &v)
- **GgVector** & operator*= (GLfloat c)
- **GgVector** operator/ (const **GgVector** &v) const
- **GgVector** operator/ (GLfloat c) const
- **GgVector** & operator/= (**GgVector** &v)
- **GgVector** & operator/= (GLfloat c)
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

8.21.1 詳解

四元数.

[gg.h](#) の 3452 行目に定義があります。

8.21.2 構築子と解体子

8.21.2.1 GgQuaternion() [1/7]

```
gg::GgQuaternion::GgQuaternion () [default]
```

コンストラクタ.

8.21.2.2 GgQuaternion() [2/7]

```
gg::GgQuaternion::GgQuaternion (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline], [constexpr]
```

コンストラクタ.

引数

x	四元数の x 要素.
y	四元数の y 要素.
z	四元数の z 要素.
w	四元数の w 要素.

[gg.h](#) の 3481 行目に定義があります。

呼び出し関係図:



8.21.2.3 GgQuaternion() [3/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

c	GLfloat 型の値.
-----	--------------

[gg.h](#) の 3491 行目に定義があります。

呼び出し関係図:



8.21.2.4 GgQuaternion() [4/7]

```
gg::GgQuaternion::GgQuaternion (
    const GLfloat * a) [inline]
```

コンストラクタ.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

gg.h の 3501 行目に定義があります。

呼び出し関係図:



8.21.2.5 GgQuaternion() [5/7]

```
gg::GgQuaternion::GgQuaternion (  
    const GgVector & v) [inline]
```

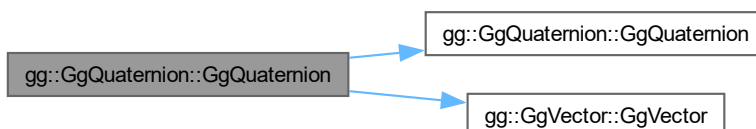
コンストラクタ.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

gg.h の 3511 行目に定義があります。

呼び出し関係図:



8.21.2.6 GgQuaternion() [6/7]

```
gg::GgQuaternion::GgQuaternion (  
    const GgQuaternion & q) [default]
```

コピーコンストラクタ.

引数

q	コピー元の四元数.
-----	-----------

呼び出し関係図:



8.21.2.7 GgQuaternion() [7/7]

```
gg::GgQuaternion::GgQuaternion (
    GgQuaternion && q) [default]
```

ムーブコンストラクタ.

引数

q	ムーブ元の四元数.
-----	-----------

呼び出し関係図:



8.21.2.8 ~GgQuaternion()

```
gg::GgQuaternion::~~GgQuaternion () [default]
```

デストラクタ.

8.21.3 関数詳解

8.21.3.1 add() [1/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

q を加えた四元数.

gg.h の 3804 行目に定義があります。

呼び出し関係図:



8.21.3.2 add() [2/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgVector & v) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

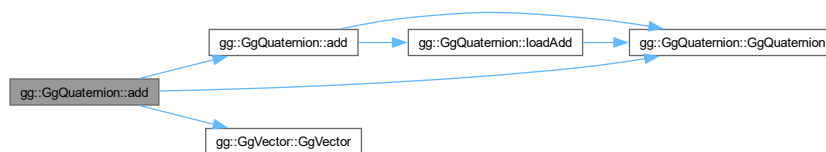
v	四元数を格納した GgVector 型の変数.
-----	-------------------------

戻り値

v を加えた四元数.

gg.h の 3793 行目に定義があります。

呼び出し関係図:



8.21.3.3 add() [3/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

<code>a</code>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	---

戻り値

`a` を加えた四元数.

`gg.h` の 3782 行目に定義があります。

呼び出し関係図:



8.21.3.4 add() [4/4]

```

GgQuaternion gg::GgQuaternion::add (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
  
```

四元数に別の四元数を加算した結果を返す.

引数

<code>x</code>	加える四元数の <code>x</code> 要素.
<code>y</code>	加える四元数の <code>y</code> 要素.
<code>z</code>	加える四元数の <code>z</code> 要素.
<code>w</code>	加える四元数の <code>w</code> 要素.

戻り値

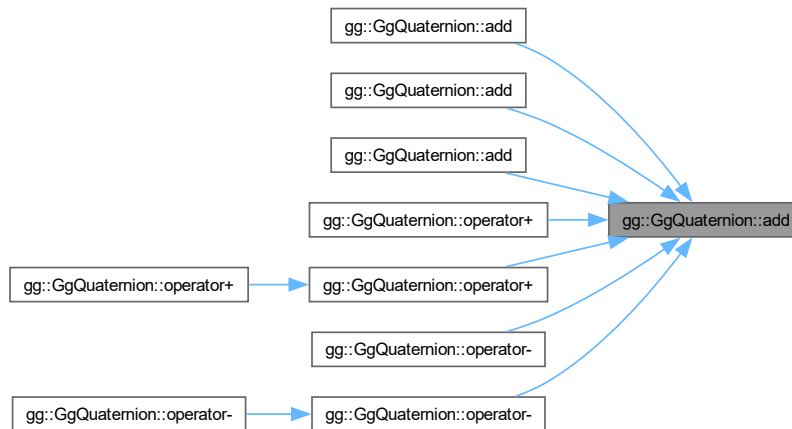
`(x, y, z, w)` を加えた四元数.

`gg.h` の 3770 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.5 conjugate()

```
GgQuaternion gg::GgQuaternion::conjugate () const [inline]
```

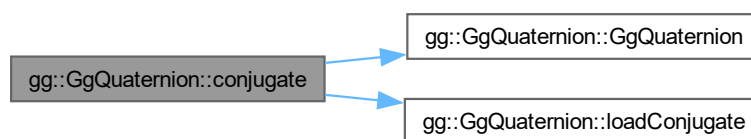
共役四元数に変換する.

戻り値

共役四元数.

[gg.h](#) の 4441 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.6 divide() [1/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgQuaternion & q) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q で割った四元数.

gg.h の 3953 行目に定義があります。

呼び出し関係図:



8.21.3.7 divide() [2/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgVector & v) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

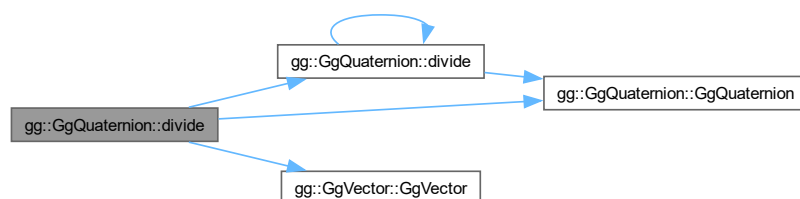
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v で割った四元数.

gg.h の 3942 行目に定義があります。

呼び出し関係図:



8.21.3.8 divide() [3/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GLfloat * a) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

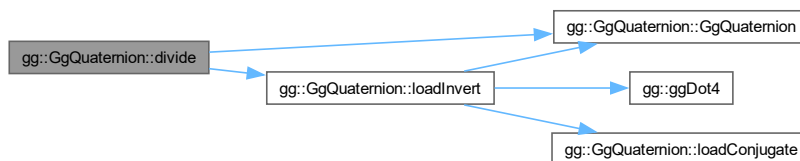
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a で割った四元数.

gg.h の 3928 行目に定義があります。

呼び出し関係図:



8.21.3.9 divide() [4/4]

```
GgQuaternion gg::GgQuaternion::divide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

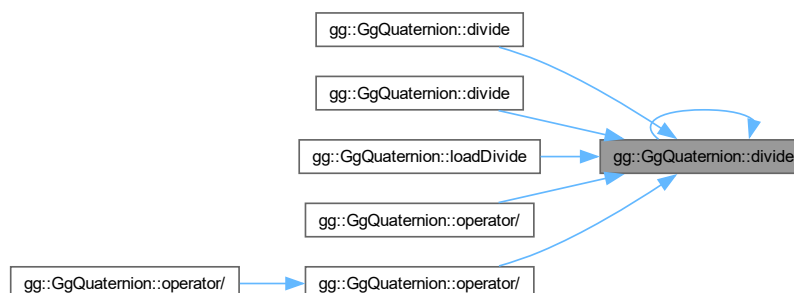
(x, y, z, w) を割った四元数.

[gg.h](#) の 3916 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.10 euler() [1/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GgVector & e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

e	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
---	--

戻り値

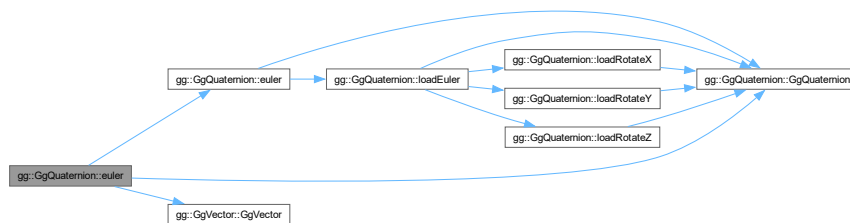
回転した四元数.

覚え書き

第 4 要素は無視する.

[gg.h](#) の 4281 行目に定義があります。

呼び出し関係図:



8.21.3.11 euler() [2/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GLfloat * e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

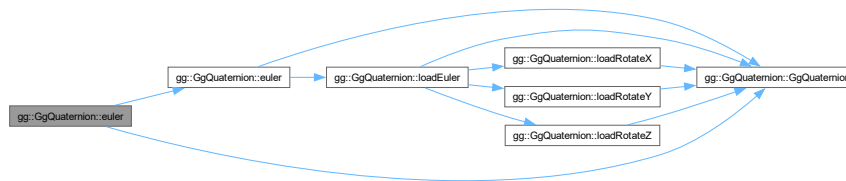
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

回転した四元数.

[gg.h](#) の [4267](#) 行目に定義があります。

呼び出し関係図:



8.21.3.12 euler() [3/3]

```

GgQuaternion gg::GgQuaternion::euler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) const [inline]
  
```

四元数をオイラー角 (heading, pitch, roll) で回転した四元数を返す.

引数

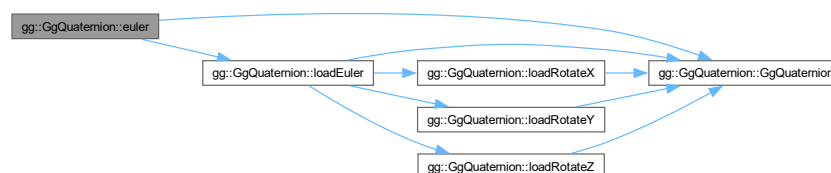
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

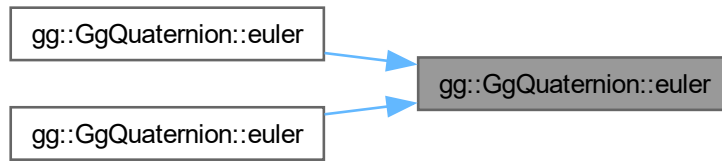
回転した四元数.

[gg.h](#) の [4255](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.13 get()

```
void gg::GgQuaternion::get (
    GLfloat * a) const [inline]
```

四元数を取り出す.

引数

<i>a</i>	四元数を格納する GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

[gg.h](#) の 4465 行目に定義があります。

8.21.3.14 getConjugateMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getConjugateMatrix () const [inline]
```

四元数の共役が表す回転の変換行列を取り出す.

戻り値

回転の変換を表す [GgMatrix](#) 型の変換行列.

[gg.h](#) の 4532 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.15 getConjugateMatrix() [2/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GgMatrix & m) const [inline]
```

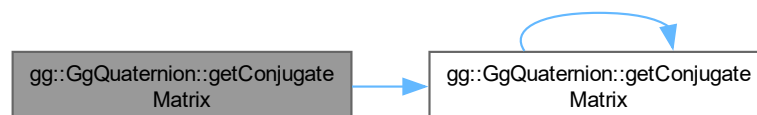
四元数の共役が表す回転の変換行列を m に求める。

引数

m	回転の変換行列を格納する <code>GgMatrix</code> 型の変数.
-----	--

`gg.h` の 4522 行目に定義があります。

呼び出し関係図:



8.21.3.16 getConjugateMatrix() [3/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GLfloat * a) const [inline]
```

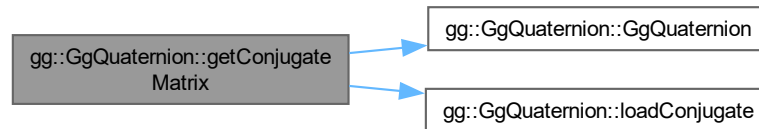
四元数の共役が表す回転の変換行列を a に求める。

引数

a	回転の変換行列を格納する <code>GLfloat</code> 型の 16 要素の配列変数.
-----	--

gg.h の 4510 行目に定義があります。

呼び出し関係図:



8.21.3.17 getMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getMatrix () const [inline]
```

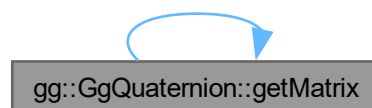
四元数が表す回転の変換行列を取り出す。

戻り値

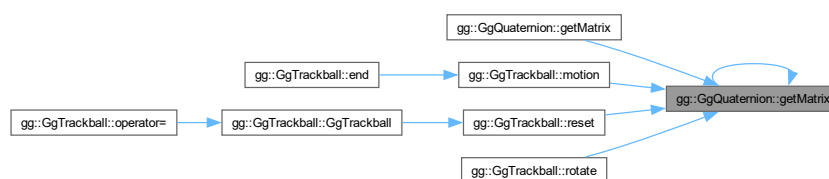
回転の変換を表す `GgMatrix` 型の変換行列。

gg.h の 4498 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.18 getMatrix() [2/3]

```
void gg::GgQuaternion::getMatrix (
    GgMatrix & m) const [inline]
```

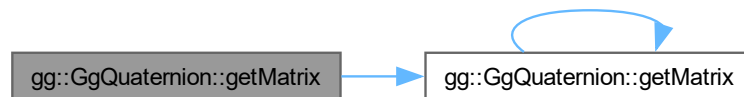
四元数が表す回転の変換行列を m に求める.

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	-----------------------------

gg.h の 4488 行目に定義があります。

呼び出し関係図:



8.21.3.19 getMatrix() [3/3]

```
void gg::GgQuaternion::getMatrix (
    GLfloat * a) const [inline]
```

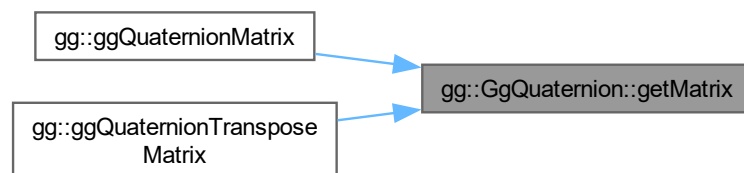
四元数が表す回転の変換行列を a に求める。

引数

<i>a</i>	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

gg.h の 4478 行目に定義があります。

被呼び出し関係図:



8.21.3.20 invert()

```
GgQuaternion gg::GgQuaternion::invert () const [inline]
```

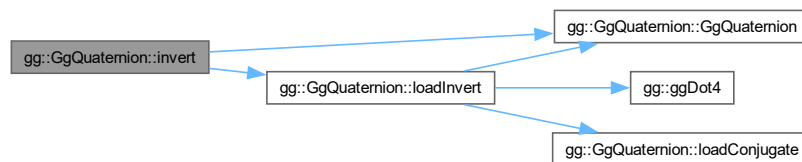
逆元に変換する.

戻り値

四元数の逆元.

gg.h の 4453 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.21 loadAdd() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

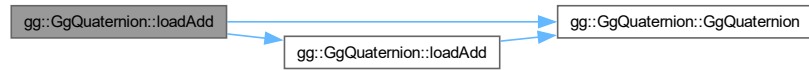
<code>q</code>	<code>GgQuaternion</code> 型の四元数.
----------------	----------------------------------

戻り値

q を加えた四元数.

gg.h の 3608 行目に定義があります。

呼び出し関係図:



8.21.3.22 loadAdd() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgVector & v) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

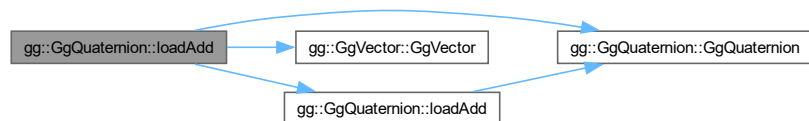
v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v を加えた四元数.

gg.h の 3597 行目に定義があります。

呼び出し関係図:



8.21.3.23 loadAdd() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GLfloat * a) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

[gg.h](#) の 3586 行目に定義があります。

呼び出し関係図:



8.21.3.24 loadAdd() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadAdd (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

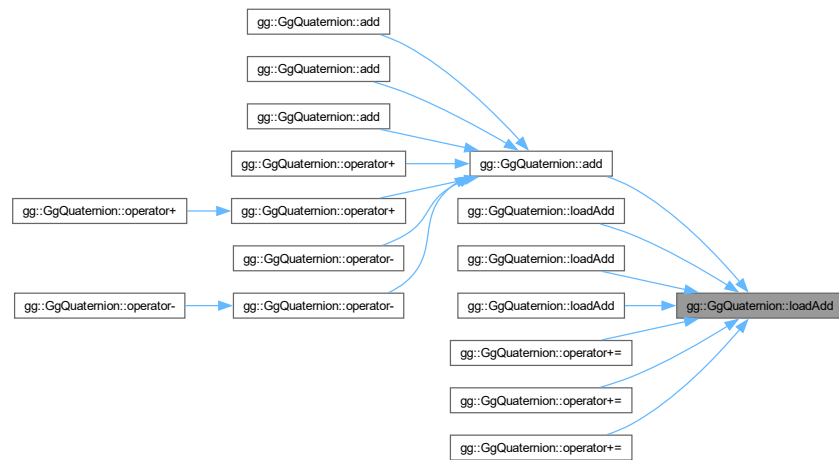
(*x*, *y*, *z*, *w*) を加えた四元数.

[gg.h](#) の 3570 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.25 loadConjugate() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の共役四元数を格納する.

引数

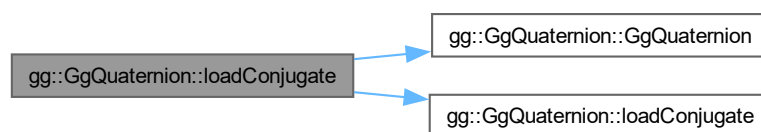
q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

共役四元数.

`gg.h` の 4372 行目に定義があります。

呼び出し関係図:



8.21.3.26 loadConjugate() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GLfloat * a)
```

引数に指定した四元数の共役四元数を格納する.

引数

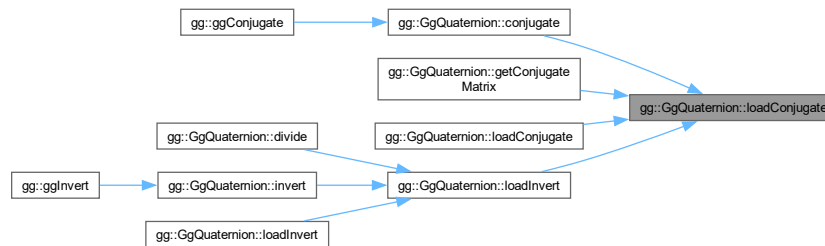
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

共役四元数.

gg.cpp の 3384 行目に定義があります。

被呼び出し関係図:



8.21.3.27 loadDivide() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgQuaternion & q) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

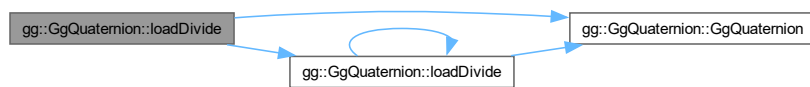
q	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q で割った四元数.

gg.h の 3756 行目に定義があります。

呼び出し関係図:



8.21.3.28 loadDivide() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgVector & v) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

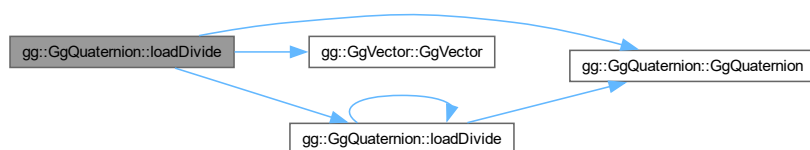
v	四元数を格納した GgVector 型の変数.
---	-------------------------

戻り値

v で割った四元数.

gg.h の 3745 行目に定義があります。

呼び出し関係図:



8.21.3.29 loadDivide() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GLfloat * a) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a で割った四元数.

[gg.h](#) の 3734 行目に定義があります。

呼び出し関係図:



8.21.3.30 loadDivide() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadDivide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元を別の四元数で除算した結果を格納する.

引数

<i>x</i>	割る四元数の <i>x</i> 要素.
<i>y</i>	割る四元数の <i>y</i> 要素.
<i>z</i>	割る四元数の <i>z</i> 要素.
<i>w</i>	割る四元数の <i>w</i> 要素.

戻り値

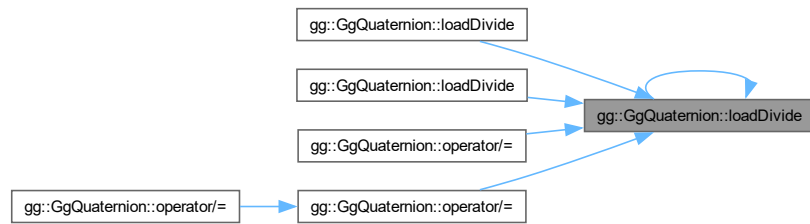
(*x*, *y*, *z*, *w*) を割った四元数.

[gg.h](#) の 3722 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.31 loadEuler() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadEuler (
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を格納する.

引数

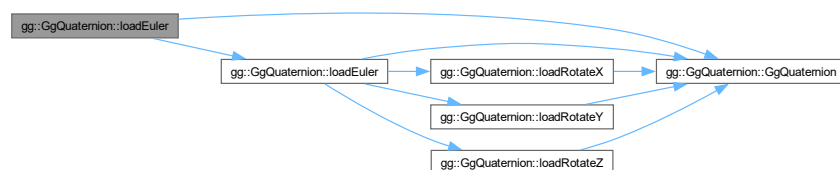
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

格納した回転を表す四元数.

gg.h の 4242 行目に定義があります。

呼び出し関係図:



8.21.3.32 loadEuler() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadEuler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll)
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を格納する.

引数

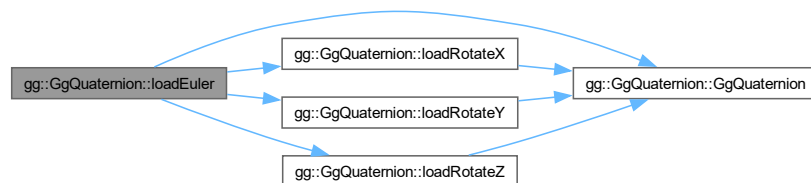
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

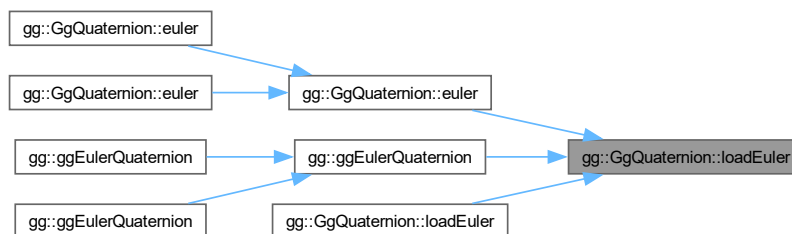
格納した回転を表す四元数.

`gg.cpp` の 3356 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.33 loadIdentity()

`GgQuaternion` & `gg::GgQuaternion::loadIdentity ()` [inline]

単位元を格納する.

戻り値

格納された単位元.

gg.h の 4092 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.34 loadInvert() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadInvert (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の逆元を格納する.

引数

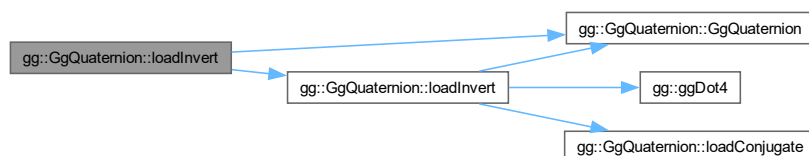
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

四元数の逆元.

gg.h の 4391 行目に定義があります。

呼び出し関係図:



8.21.3.35 loadInvert() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadInvert (
    const GLfloat * a)
```

引数に指定した四元数の逆元を格納する.

引数

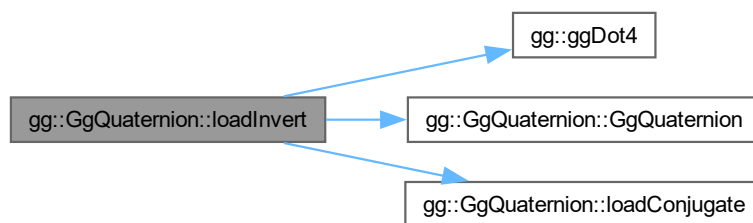
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

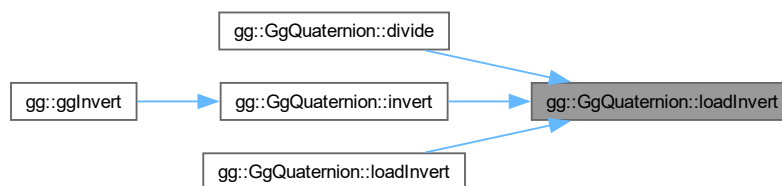
四元数の逆元.

gg.cpp の 3398 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.36 loadMatrix() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GgMatrix & m) [inline]
```

回転の変換行列 m を表す四元数を格納する.

引数

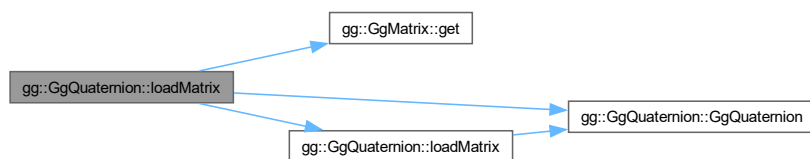
<i>m</i>	Ggmatrix 型の変換行列.
----------	------------------

戻り値

m による回転の変換に相当する四元数.

gg.h の 4082 行目に定義があります。

呼び出し関係図:



8.21.3.37 loadMatrix() [2/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GLfloat * a) [inline]
```

回転の変換行列を表す四元数を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

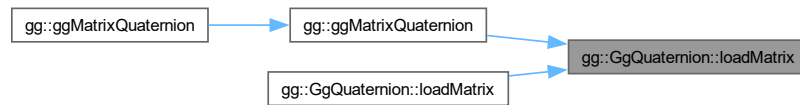
a による回転の変換に相当する四元数.

gg.h の 4070 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.38 loadMultiply() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

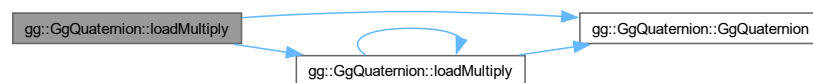
q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

q を乗じた四元数.

`gg.h` の 3708 行目に定義があります。

呼び出し関係図:



8.21.3.39 loadMultiply() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgVector & v) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

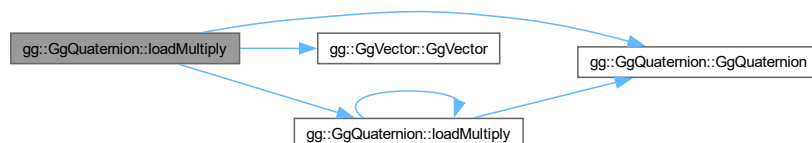
v	四元数を格納した GgVector 型の変数.
-----	--------------------------------

戻り値

v を乗じた四元数.

gg.h の 3697 行目に定義があります。

呼び出し関係図:



8.21.3.40 loadMultiply() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GLfloat * a) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

a を乗じた四元数.

gg.h の 3686 行目に定義があります。

呼び出し関係図:



8.21.3.41 loadMultiply() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を乗算した結果を格納する。

引数

<i>x</i>	掛ける四元数の <i>x</i> 要素.
<i>y</i>	掛ける四元数の <i>y</i> 要素.
<i>z</i>	掛ける四元数の <i>z</i> 要素.
<i>w</i>	掛ける四元数の <i>w</i> 要素.

戻り値

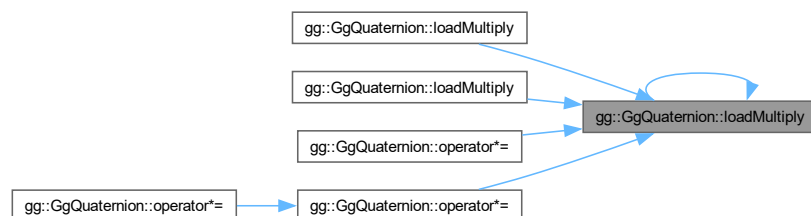
(*x*, *y*, *z*, *w*) を掛けた四元数.

[gg.h](#) の 3674 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.42 loadNormalize() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadNormalize (  
    const GgQuaternion & q) [inline]
```

引数に指定した四元数を正規化して格納する.

引数

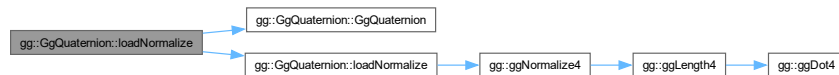
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

正規化された四元数.

gg.h の 4353 行目に定義があります。

呼び出し関係図:



8.21.3.43 loadNormalize() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GLfloat * a)
```

引数に指定した四元数を正規化して格納する.

引数

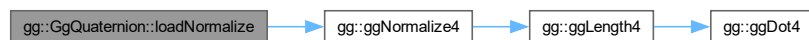
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

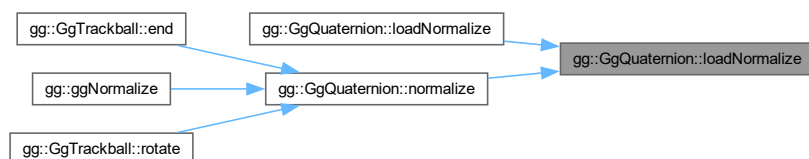
正規化された四元数.

gg.cpp の 3372 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.44 loadRotate() [1/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を格納する.

引数

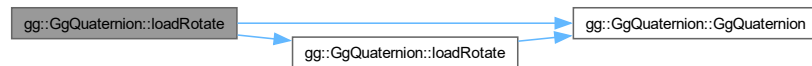
v	軸ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------------

戻り値

格納された回転を表す四元数.

gg.h の 4126 行目に定義があります。

呼び出し関係図:



8.21.3.45 loadRotate() [2/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を格納する.

引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

格納された回転を表す四元数.

gg.h の 4115 行目に定義があります。

呼び出し関係図:



8.21.3.46 loadRotate() [3/3]

```
gg::GgQuaternion & gg::GgQuaternion::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)
```

(x, y, z) を軸として角度 a 回転する四元数を格納する.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

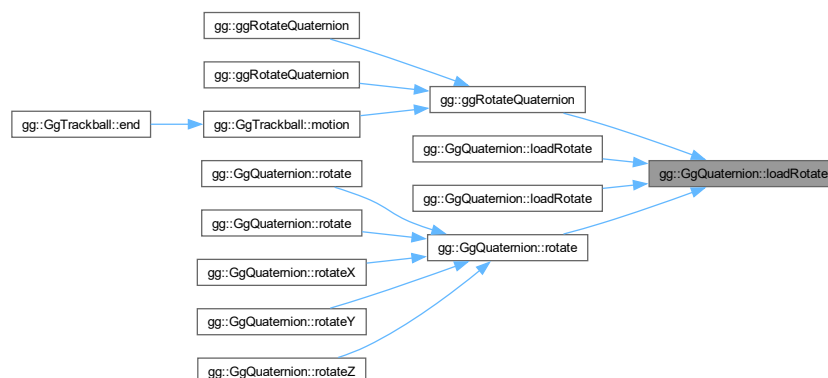
格納された回転を表す四元数.

[gg.cpp](#) の 3298 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.47 loadRotateX()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateX (
    GLfloat a)
```

x 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

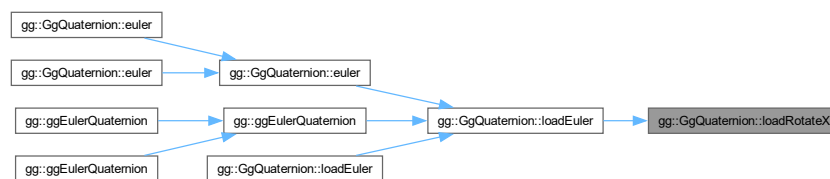
格納された回転を表す四元数.

gg.cpp の 3320 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.48 loadRotateY()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateY (
    GLfloat a)
```

y 軸中心に角度 *a* 回転する四元数を格納する.

引数

戻り値

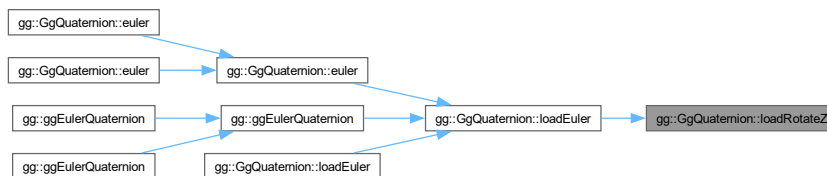
格納された回転を表す四元数.

gg.cpp の 3344 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.50 loadSlerp() [1/4]

```

GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
  
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した q, r を t で内分した四元数.

[gg.h](#) の 4308 行目に定義があります。

呼び出し関係図:



8.21.3.51 loadSlerp() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

q	GgQuaternion 型の四元数.
a	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
t	補間パラメータ.

戻り値

格納した q, a を t で内分した四元数.

[gg.h](#) の 4321 行目に定義があります。

呼び出し関係図:



8.21.3.52 loadSlerp() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した *a*, *q* を *t* で内分した四元数.

gg.h の 4334 行目に定義があります。

呼び出し関係図:



8.21.3.53 loadSlerp() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

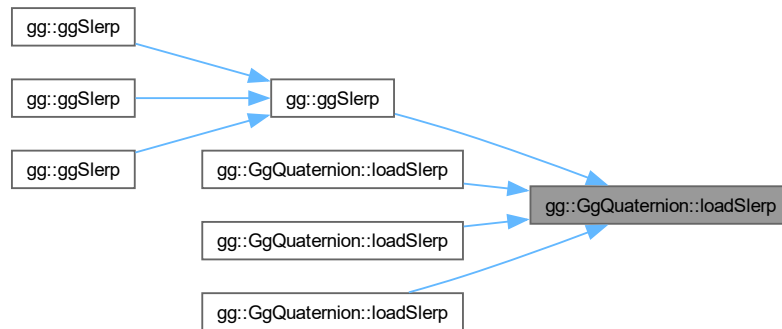
格納した *a*, *b* を *t* で内分した四元数.

gg.h の 4294 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.54 loadSubtract() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgQuaternion & q) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

q を引いた四元数.

`gg.h` の 3660 行目に定義があります。

呼び出し関係図:



8.21.3.55 loadSubtract() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgVector & v) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

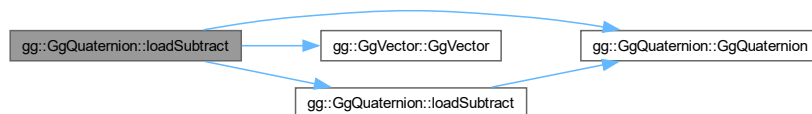
v	四元数を格納した GgVector 型の変数.
-----	--------------------------------

戻り値

v を引いた四元数.

gg.h の 3649 行目に定義があります。

呼び出し関係図:



8.21.3.56 loadSubtract() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GLfloat * a) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

a を引いた四元数.

gg.h の 3638 行目に定義があります。

呼び出し関係図:



8.21.3.57 loadSubtract() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.
w	引く四元数の w 要素.

戻り値

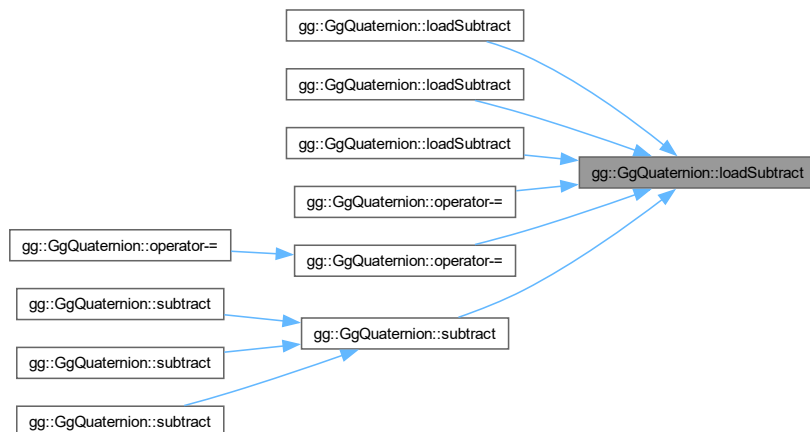
(x , y , z , w) を引いた四元数.

gg.h の 3622 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.58 multiply() [1/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

q を掛けた四元数.

`gg.h` の 3902 行目に定義があります。

呼び出し関係図:



8.21.3.59 multiply() [2/4]

```
GgQuaternion gg::GgQuaternion::multiply (  
    const GgVector & v) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

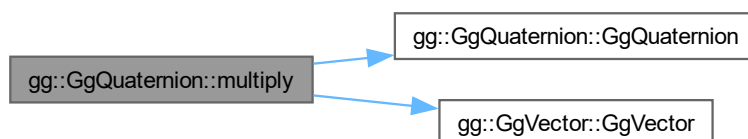
v	四元数を格納した <code>GgVector</code> 型の変数.
-----	--------------------------------------

戻り値

v を掛けた四元数.

`gg.h` の 3891 行目に定義があります。

呼び出し関係図:



8.21.3.60 multiply() [3/4]

```
GgQuaternion gg::GgQuaternion::multiply (  
    const GLfloat * a) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

<code>a</code>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	---

戻り値

`a` を掛けた四元数.

`gg.h` の 3878 行目に定義があります。

呼び出し関係図:



8.21.3.61 multiply() [4/4]

```

GgQuaternion gg::GgQuaternion::multiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
  
```

四元数に別の四元数を乗算した結果を返す.

引数

<code>x</code>	掛ける四元数の <code>x</code> 要素.
<code>y</code>	掛ける四元数の <code>y</code> 要素.
<code>z</code>	掛ける四元数の <code>z</code> 要素.
<code>w</code>	掛ける四元数の <code>w</code> 要素.

戻り値

`(x, y, z, w)` を掛けた四元数.

`gg.h` の 3866 行目に定義があります。

呼び出し関係図:



8.21.3.62 norm()

```
GLfloat gg::GgQuaternion::norm () const [inline]
```

四元数のノルムを求める。

戻り値

四元数のノルム。

gg.h の 3556 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.63 normalize()

```
GgQuaternion gg::GgQuaternion::normalize () const [inline]
```

正規化する。

戻り値

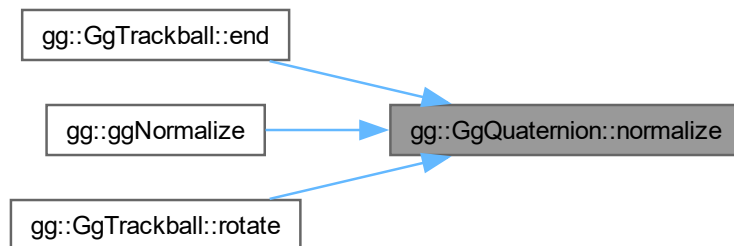
正規化された四元数.

[gg.h](#) の [4429](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.64 operator*() [1/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgQuaternion & q) const [inline]
```

[gg.h](#) の [4047](#) 行目に定義があります。

呼び出し関係図:

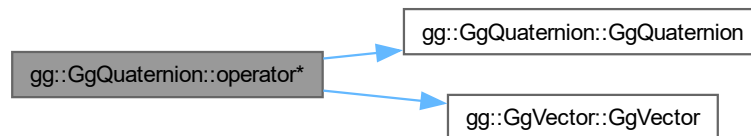


8.21.3.65 operator*() [2/3]

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GgVector & v) const [inline]
```

gg.h の 4043 行目に定義があります。

呼び出し関係図:

**8.21.3.66 operator*() [3/3]**

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GLfloat * a) const [inline]
```

gg.h の 4039 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.67 operator*=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgQuaternion & q) [inline]
```

gg.h の 3999 行目に定義があります。

呼び出し関係図:

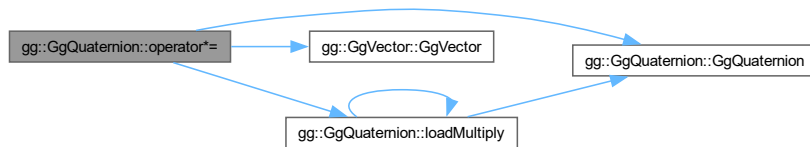


8.21.3.68 operator*=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgVector & v) [inline]
```

gg.h の 3995 行目に定義があります。

呼び出し関係図:



8.21.3.69 operator*=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GLfloat * a) [inline]
```

gg.h の 3991 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.70 operator+() [1/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgQuaternion & q) const [inline]
```

gg.h の 4023 行目に定義があります。

呼び出し関係図:

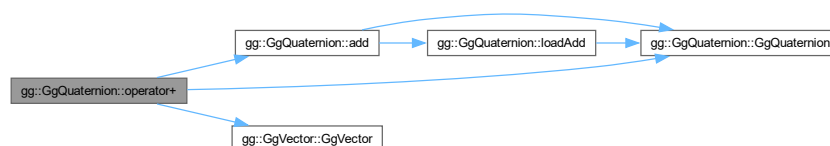


8.21.3.71 operator+() [2/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgVector & v) const [inline]
```

gg.h の 4019 行目に定義があります。

呼び出し関係図:



8.21.3.72 operator+() [3/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GLfloat * a) const [inline]
```

gg.h の 4015 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

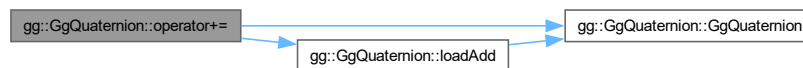


8.21.3.73 operator+=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgQuaternion & q) [inline]
```

gg.h の 3975 行目に定義があります。

呼び出し関係図:

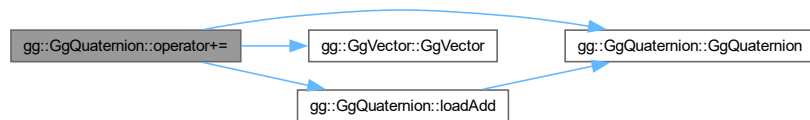


8.21.3.74 operator+=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgVector & v) [inline]
```

gg.h の 3971 行目に定義があります。

呼び出し関係図:

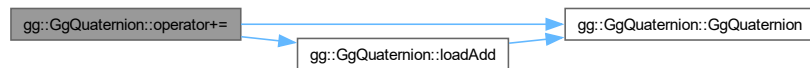


8.21.3.75 operator+=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GLfloat * a) [inline]
```

gg.h の 3967 行目に定義があります。

呼び出し関係図:

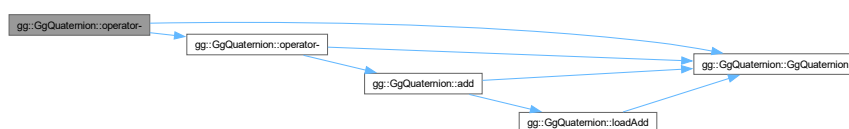


8.21.3.76 operator-() [1/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgQuaternion & q) const [inline]
```

gg.h の 4035 行目に定義があります。

呼び出し関係図:

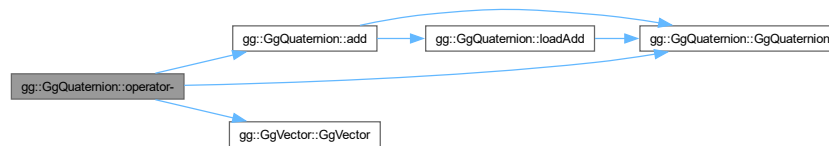


8.21.3.77 operator-() [2/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgVector & v) const [inline]
```

gg.h の 4031 行目に定義があります。

呼び出し関係図:

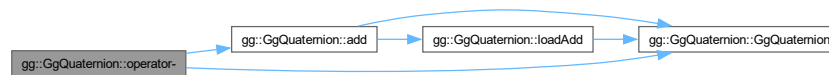


8.21.3.78 operator-() [3/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GLfloat * a) const [inline]
```

gg.h の 4027 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.79 operator-=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgQuaternion & q) [inline]
```

gg.h の 3987 行目に定義があります。

呼び出し関係図:

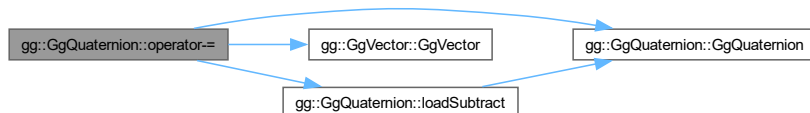


8.21.3.80 operator-=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgVector & v) [inline]
```

gg.h の 3983 行目に定義があります。

呼び出し関係図:



8.21.3.81 operator-=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GLfloat * a) [inline]
```

gg.h の 3979 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

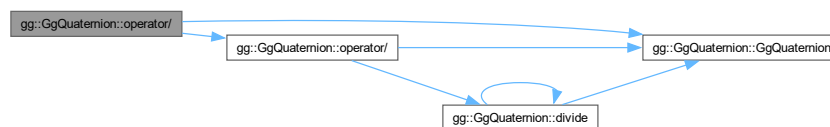


8.21.3.82 operator/() [1/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgQuaternion & q) const [inline]
```

`gg.h` の 4059 行目に定義があります。

呼び出し関係図:

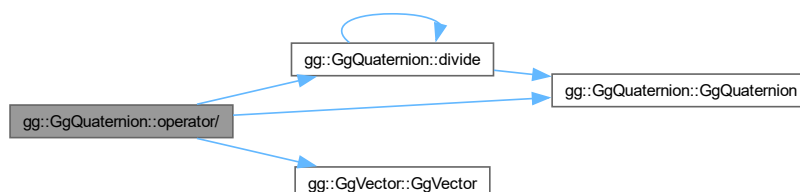


8.21.3.83 operator/() [2/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgVector & v) const [inline]
```

`gg.h` の 4055 行目に定義があります。

呼び出し関係図:

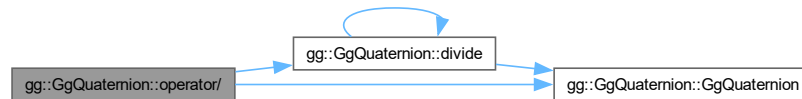


8.21.3.84 operator/() [3/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GLfloat * a) const [inline]
```

gg.h の 4051 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.85 operator/=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgQuaternion & q) [inline]
```

gg.h の 4011 行目に定義があります。

呼び出し関係図:

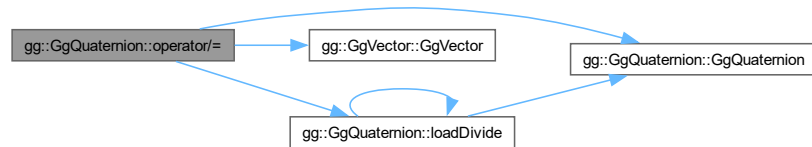


8.21.3.86 operator/=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgVector & v) [inline]
```

gg.h の 4007 行目に定義があります。

呼び出し関係図:

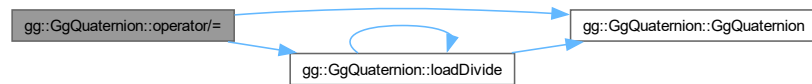


8.21.3.87 operator/=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GLfloat * a) [inline]
```

gg.h の 4003 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.88 operator=() [1/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgQuaternion & q) [default]
```

代入演算子.

引数

q	代入元の四元数.
-----	----------

戻り値

代入後のこの四元数の参照.

呼び出し関係図:



被呼び出し関係図:

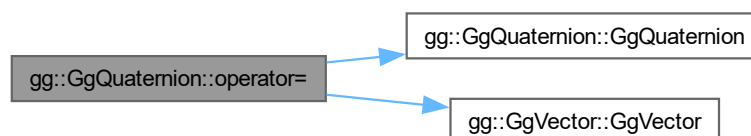


8.21.3.89 operator=() [2/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GgVector & v) [inline]
```

[gg.h](#) の 3963 行目に定義があります。

呼び出し関係図:



8.21.3.90 operator=() [3/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GLfloat * a) [inline]
```

gg.h の 3959 行目に定義があります。

呼び出し関係図:



8.21.3.91 operator=() [4/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    GgQuaternion && q) [default]
```

ムーブ代入演算子.

引数

<i>q</i>	ムーブ代入元の四元数.
----------	-------------

戻り値

ムーブ代入後のこの四元数の参照.

呼び出し関係図:



8.21.3.92 rotate() [1/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 v[3] 回転した四元数を返す.

引数

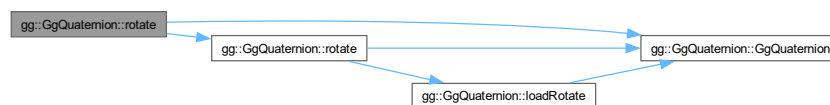
v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

回転した四元数.

gg.h の 4188 行目に定義があります。

呼び出し関係図:



8.21.3.93 rotate() [2/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v,
    GLfloat a) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 a 回転した四元数を返す.

引数

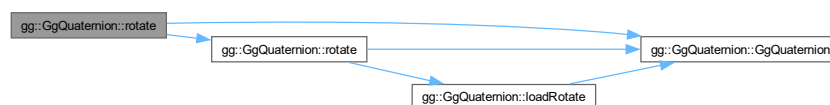
v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転した四元数.

gg.h の 4177 行目に定義があります。

呼び出し関係図:



8.21.3.94 rotate() [3/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

四元数を (x, y, z) を軸として角度 a 回転した四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

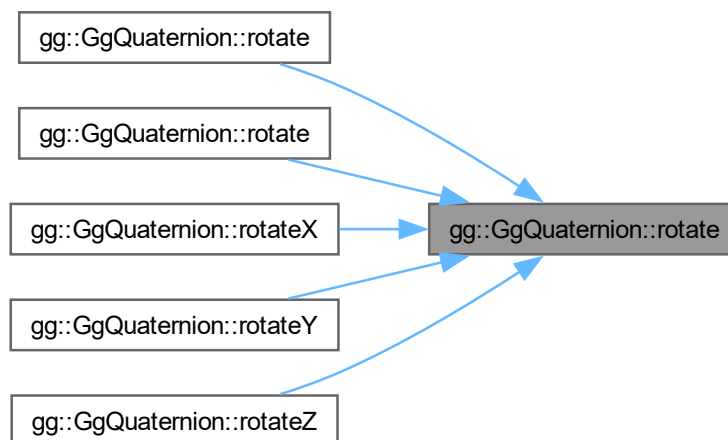
回転した四元数.

[gg.h](#) の 4164 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.95 rotateX()

```
GgQuaternion gg::GgQuaternion::rotateX (  
    GLfloat a) const [inline]
```

四元数を x 軸中心に角度 **a** 回転した四元数を返す.

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4199 行目に定義があります。

呼び出し関係図:



8.21.3.96 rotateY()

```
GgQuaternion gg::GgQuaternion::rotateY (  
    GLfloat a) const [inline]
```

四元数を y 軸中心に角度 **a** 回転した四元数を返す.

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4210 行目に定義があります。

呼び出し関係図:



8.21.3.97 rotateZ()

```
GgQuaternion gg::GgQuaternion::rotateZ (
    GLfloat a) const [inline]
```

四元数を z 軸中心に角度 **a** 回転した四元数を返す。

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4221 行目に定義があります。

呼び出し関係図:



8.21.3.98 slerp() [1/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    const GgQuaternion & q,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す。

引数

q	GgQuaternion 型の四元数.
t	補間パラメータ.

戻り値

四元数を **q** に対して **t** で内分した結果.

[gg.h](#) の 4417 行目に定義があります。

呼び出し関係図:



8.21.3.99 slerp() [2/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    GLfloat * a,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

四元数を *a* に対して *t* で内分した結果.

gg.h の 4403 行目に定義があります。

呼び出し関係図:



8.21.3.100 subtract() [1/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgQuaternion & q) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を引いた四元数.

gg.h の 3852 行目に定義があります。

呼び出し関係図:



8.21.3.101 subtract() [2/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgVector & v) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v を引いた四元数.

[gg.h](#) の 3841 行目に定義があります.

呼び出し関係図:



8.21.3.102 subtract() [3/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GLfloat * a) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a を引いた四元数.

[gg.h](#) の 3830 行目に定義があります.

呼び出し関係図:



8.21.3.103 subtract() [4/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>x</i>	引く四元数の <i>x</i> 要素.
<i>y</i>	引く四元数の <i>y</i> 要素.
<i>z</i>	引く四元数の <i>z</i> 要素.
<i>w</i>	引く四元数の <i>w</i> 要素.

戻り値

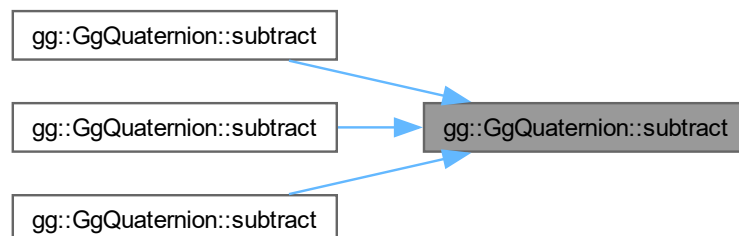
(*x*, *y*, *z*, *w*) を引いた四元数.

[gg.h](#) の 3818 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.22 gg::GgShader クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", int nvarying=0, const char *const *varyings=nullptr)
- `GgShader` (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- `GgShader` (const GgShader &shader)=delete
- `GgShader` (GgShader &&shader)=default
- virtual `~GgShader` ()
- `GgShader` & `operator=` (const `GgShader` &shader)=delete
- `GgShader` & `operator=` (`GgShader` &&shader)=default
- void `use` () const
- void `unuse` () const
- GLuint `get` () const

8.22.1 詳解

シェーダの基底クラス.

覚え書き

シェーダのクラスはこのクラスを派生して作る.

`gg.h` の 6864 行目に定義があります.

8.22.2 構築子と解体子

8.22.2.1 GgShader() [1/4]

```
gg::GgShader::GgShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

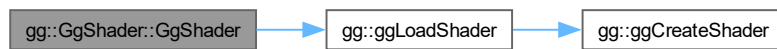
引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィールドバックする <i>varying</i> 変数の数 (0 なら不使用).

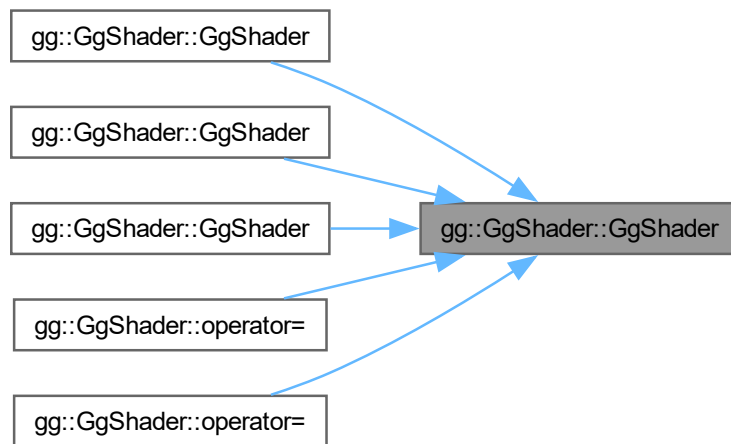
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.
-----------------	----------------------------------

gg.h の 6880 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.22.2.2 GgShader() [2/4]

```

gg::GgShader::GgShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

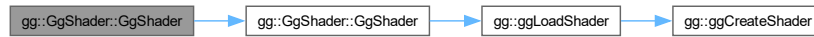
引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (<i>nullptr</i> なら不使用).
-----------------	--

[gg.h](#) の 6898 行目に定義があります。

呼び出し関係図:



8.22.2.3 GgShader() [3/4]

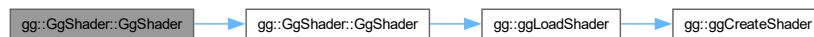
```
gg::GgShader::GgShader (
    const GgShader & shader) [delete]
```

コピーコンストラクタは使用しない。

引数

<i>shader</i>	コピー元のシェータ.
---------------	------------

呼び出し関係図:



8.22.2.4 GgShader() [4/4]

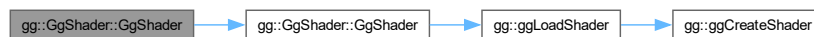
```
gg::GgShader::GgShader (
    GgShader && shader) [default]
```

ムーブコンストラクタ.

引数

<i>shader</i>	ムーブ元のシェータ.
---------------	------------

呼び出し関係図:



8.22.2.5 ~GgShader()

```
virtual gg::GgShader::~~GgShader () [inline], [virtual]
```

デストラクタ.

gg.h の 6924 行目に定義があります。

8.22.3 関数詳解

8.22.3.1 get()

```
GLuint gg::GgShader::get () const [inline]
```

シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 6968 行目に定義があります。

8.22.3.2 operator=() [1/2]

```
GgShader & gg::GgShader::operator= (
    const GgShader & shader) [delete]
```

代入演算子は使用しない.

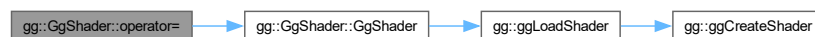
引数

<i>shader</i>	代入元のシェーダ.
---------------	-----------

戻り値

代入後のこのシェーダの参照.

呼び出し関係図:



8.22.3.3 operator=() [2/2]

```
GgShader & gg::GgShader::operator= (
    GgShader && shader) [default]
```

ムーブ代入演算子.

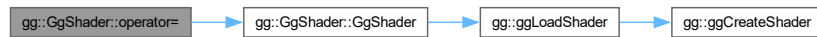
引数

<code>shader</code>	ムーブ代入元のシェーダ.
---------------------	--------------

戻り値

ムーブ代入後のこのシェーダの参照.

呼び出し関係図:



8.22.3.4 unuse()

```
void gg::GgShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する.

[gg.h](#) の [6958](#) 行目に定義があります。

8.22.3.5 use()

```
void gg::GgShader::use () const [inline]
```

シェーダプログラムの使用を開始する.

[gg.h](#) の [6950](#) 行目に定義があります。

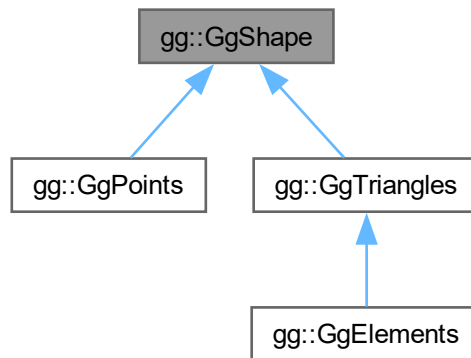
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.23 gg::GgShape クラス

```
#include <gg.h>
```

gg::GgShape の継承関係図



公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual [bool operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.23.1 詳解

形状データの基底クラス.

覚え書き

形状データのクラスはこのクラスを派生して作る. 基本図形の種類と頂点配列オブジェクトを保持する.

[gg.h](#) の 6240 行目に定義があります。

8.23.2 構築子と解体子

8.23.2.1 GgShape()

```
gg::GgShape::GgShape (
    GLenum mode = 0) [inline]
```

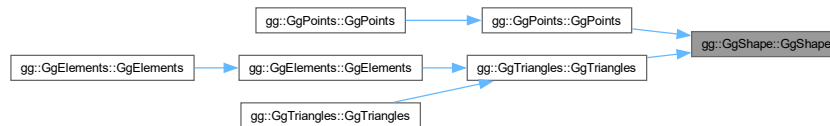
コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の 6255 行目に定義があります。

被呼び出し関係図:



8.23.2.2 ~GgShape()

```
virtual gg::GgShape::~~GgShape () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6264 行目に定義があります。

8.23.3 関数詳解

8.23.3.1 draw()

```
virtual void gg::GgShape::draw (
    GLint first = 0,
    GLsizei count = 0) const [inline], [virtual]
```

図形の描画, 派生クラスでこの手続きをオーバーライドする.

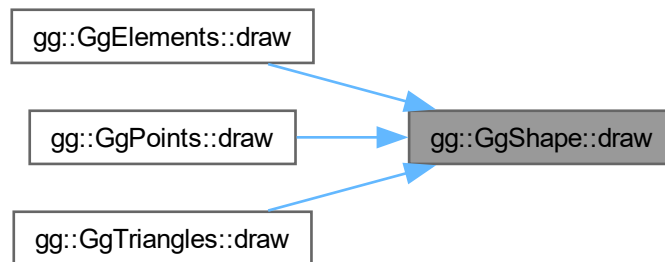
引数

<i>first</i>	描画する最初のアイテム.
<i>count</i>	描画するアイテムの数, 0 なら全部のアイテムを描画する.

[gg::GgElements](#), [gg::GgPoints](#), [gg::GgTriangles](#) で再実装されています。

[gg.h](#) の 6324 行目に定義があります。

被呼び出し関係図:



8.23.3.2 get()

```
const GLuint & gg::GgShape::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

[gg.h](#) の 6293 行目に定義があります。

被呼び出し関係図:



8.23.3.3 getMode()

```
const GLenum & gg::GgShape::getMode () const [inline]
```

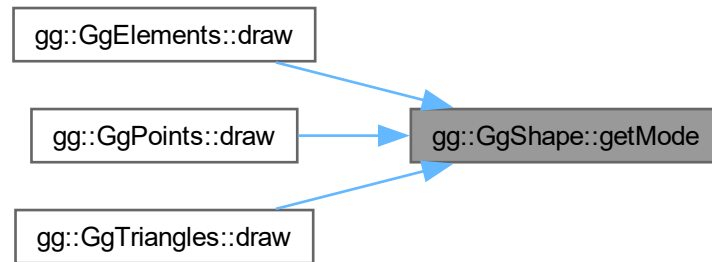
基本図形の検査.

戻り値

この頂点配列オブジェクトの基本図形の種類.

[gg.h](#) の 6313 行目に定義があります。

被呼び出し関係図:



8.23.3.4 operator bool()

```
virtual gg::GgShape::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうか調べる.

戻り値

頂点配列オブジェクトが有効なら true

[gg::GgPoints](#) で再実装されています。

[gg.h](#) の 6273 行目に定義があります。

8.23.3.5 operator!()

```
virtual bool gg::GgShape::operator! () const [inline], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうかの結果を反転する.

戻り値

頂点配列オブジェクトが有効なら false, 無効なら true.

[gg::GgPoints](#) で再実装されています。

[gg.h](#) の 6283 行目に定義があります。

8.23.3.6 setMode()

```
void gg::GgShape::setMode (
    GLenum mode) [inline]
```

基本図形の設定.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6303 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.24 gg::GgSimpleObj クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgSimpleObj](#) (const std::string &name, bool normalize=false)
- virtual [~GgSimpleObj](#) ()
デストラクタ.
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTriangles](#) * [get](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.24.1 詳解

Wavefront OBJ 形式のファイル (Arrays 形式).

gg.h の 8244 行目に定義があります。

8.24.2 構築子と解体子

8.24.2.1 GgSimpleObj()

```
gg::GgSimpleObj::GgSimpleObj (
    const std::string & name,
    bool normalize = false)
```

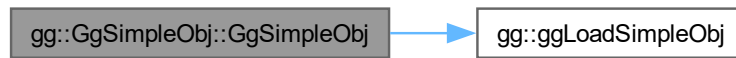
コンストラクタ.

引数

<i>name</i>	三角形分割された Alias OBJ 形式のファイルのファイル名.
<i>normalize</i>	true なら図形のサイズを [-1, 1] に正規化する.

gg.cpp の 5990 行目に定義があります。

呼び出し関係図:



8.24.2.2 ~GgSimpleObj()

```
virtual gg::GgSimpleObj::~~GgSimpleObj () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の [8267](#) 行目に定義があります。

8.24.3 関数詳解

8.24.3.1 draw()

```
void gg::GgSimpleObj::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

Wavefront OBJ 形式のデータを描画する手続き.

引数

<i>first</i>	描画する最初のパーツ番号.
<i>count</i>	描画するパーツの数, 0 なら全部のパーツを描く.

[gg.cpp](#) の [6018](#) 行目に定義があります。

8.24.3.2 get()

```
const GgTriangles * gg::GgSimpleObj::get () const [inline]
```

形状データの取り出し.

戻り値

[GgTriangles](#) 型の形状データのポインタ。

[gg.h](#) の 8296 行目に定義があります。

呼び出し関係図:



8.24.3.3 operator bool()

```
gg::GgSimpleObj::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる。

戻り値

オブジェクトが有効なら true

[gg.h](#) の 8276 行目に定義があります。

8.24.3.4 operator"!()"

```
bool gg::GgSimpleObj::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する。

戻り値

オブジェクトが有効なら false, 無効なら true.

[gg.h](#) の 8286 行目に定義があります。

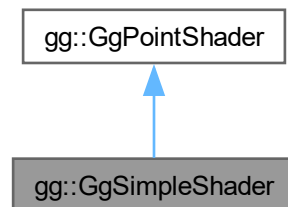
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

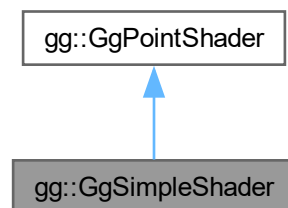
8.25 gg::GgSimpleShader クラス

```
#include <gg.h>
```

gg::GgSimpleShader の継承関係図



gg::GgSimpleShader 連携図



クラス

- struct [Light](#)
- class [LightBuffer](#)
- struct [Material](#)
- class [MaterialBuffer](#)

公開メンバ関数

- [GgSimpleShader](#) ()
- [GgSimpleShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const GgSimpleShader &o)

- virtual ~GgSimpleShader ()
- GgSimpleShader & operator= (const GgSimpleShader &o)
- bool load (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool load (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void loadModelviewMatrix (const GLfloat *mv, const GLfloat *mn) const
- virtual void loadModelviewMatrix (const GgMatrix &mv, const GgMatrix &mn) const
- virtual void loadModelviewMatrix (const GLfloat *mv) const
- virtual void loadModelviewMatrix (const GgMatrix &mv) const
- virtual void loadMatrix (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- virtual void loadMatrix (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn) const
- virtual void loadMatrix (const GLfloat *mp, const GLfloat *mv) const
- virtual void loadMatrix (const GgMatrix &mp, const GgMatrix &mv) const
- void use () const
- void use (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn) const
- void use (const GLfloat *mp, const GLfloat *mv) const
- void use (const GgMatrix &mp, const GgMatrix &mv) const
- void use (const LightBuffer *light, GLint i=0) const
- void use (const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn, const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const GLfloat *mv, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const LightBuffer &light, GLint i=0) const

基底クラス **gg::GgPointShader** に属する継承公開メンバ関数

- GgPointShader ()
- GgPointShader (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GgPointShader (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual ~GgPointShader ()
- bool load (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool load (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void loadProjectionMatrix (const GLfloat *mp) const
- virtual void loadProjectionMatrix (const GgMatrix &mp) const
- void use (const GLfloat *mp) const
- void use (const GgMatrix &mp) const
- void use (const GLfloat *mp, const GLfloat *mv) const
- void use (const GgMatrix &mp, const GgMatrix &mv) const
- void unuse () const
- GLuint get () const

8.25.1 詳解

三角形に単純な陰影付けを行うシェーダ。

[gg.h](#) の 7232 行目に定義があります。

8.25.2 構築子と解体子

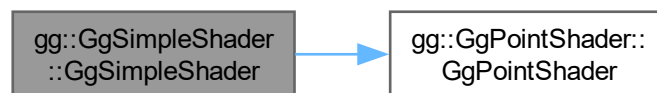
8.25.2.1 GgSimpleShader() [1/4]

```
gg::GgSimpleShader::GgSimpleShader () [inline]
```

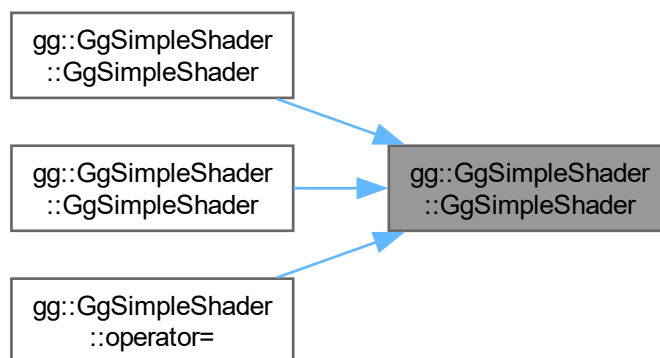
コンストラクタ.

[gg.h](#) の 7243 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.2.2 GgSimpleShader() [2/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

gg.h の 7258 行目に定義があります。

呼び出し関係図:



8.25.2.3 GgSimpleShader() [3/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

gg.h の 7276 行目に定義があります。

呼び出し関係図:



8.25.2.4 GgSimpleShader() [4/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const GgSimpleShader & o) [inline]
```

コピーコンストラクタ.

gg.h の 7288 行目に定義があります。

呼び出し関係図:



8.25.2.5 ~GgSimpleShader()

```
virtual gg::GgSimpleShader::~~GgSimpleShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7297 行目に定義があります。

8.25.3 関数詳解

8.25.3.1 load() [1/2]

```
bool gg::GgSimpleShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする varying 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする varying 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

gg.h の 7344 行目に定義があります。

呼び出し関係図:



8.25.3.2 load() [2/2]

```

bool gg::GgSimpleShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr)
  
```

シェーダのソースプログラムの文字列からプログラムオブジェクトを作成する.

引数

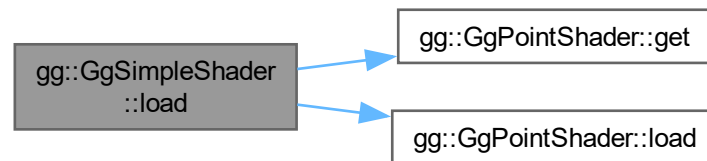
<i>vert</i>	バーテックスシェーダのソースプログラムの文字列.
<i>frag</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>geom</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

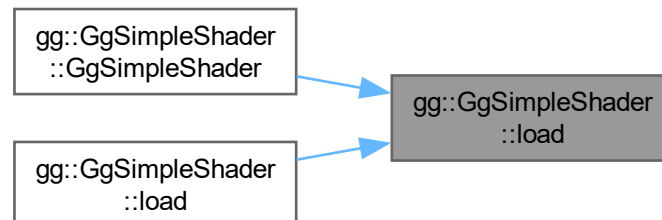
プログラムオブジェクトの作成に成功したら true.

gg.cpp の 5973 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.3 loadMatrix() [1/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
  
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する。

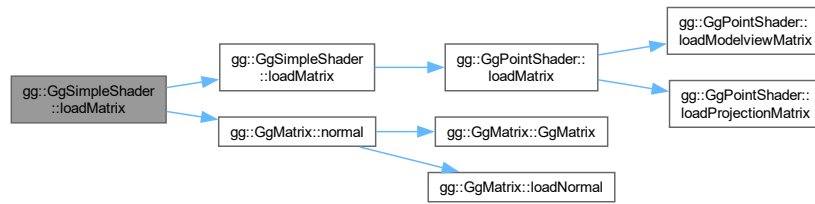
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg::GgPointShaderを再実装しています。

gg.h の 7438 行目に定義があります。

呼び出し関係図:



8.25.3.4 loadMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

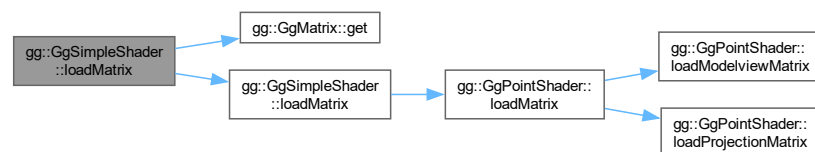
投影変換行列とモデルビュー変換行列と法線変換行列を設定する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7416 行目に定義があります。

呼び出し関係図:



8.25.3.5 loadMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

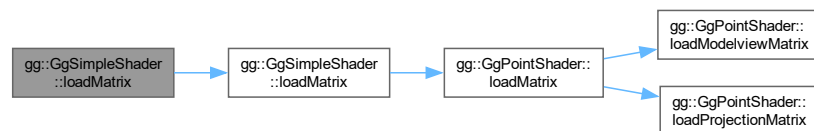
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

`gg::GgPointShader` を再実装しています。

`gg.h` の 7427 行目に定義があります。

呼び出し関係図:



8.25.3.6 loadMatrix() [4/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
  
```

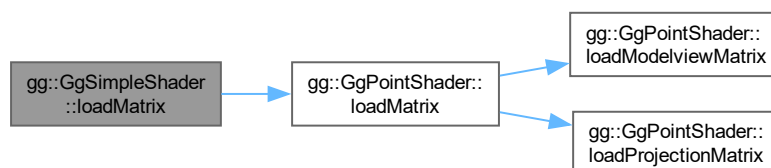
投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

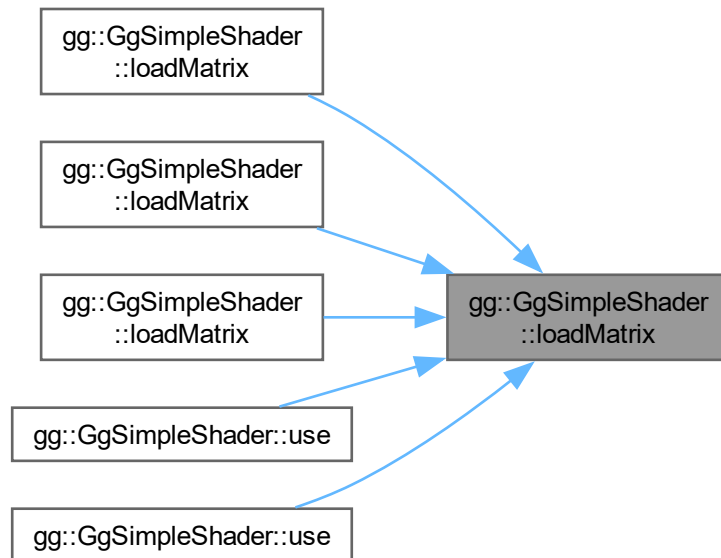
<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

`gg.h` の 7403 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.7 loadModelviewMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

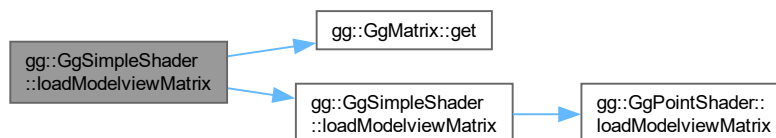
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgPointShaderを再実装しています。

gg.h の 7391 行目に定義があります。

呼び出し関係図:



8.25.3.8 loadModelviewMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

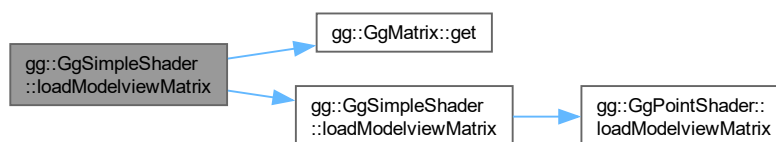
モデルビュー変換行列と法線変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7371 行目に定義があります。

呼び出し関係図:



8.25.3.9 loadModelviewMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

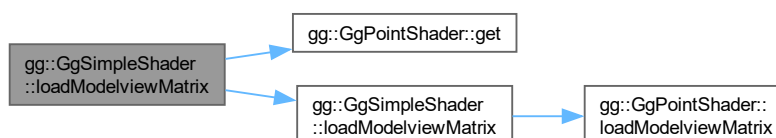
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgPointShader を再実装しています。

gg.h の 7381 行目に定義があります。

呼び出し関係図:



8.25.3.10 loadModelviewMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
```

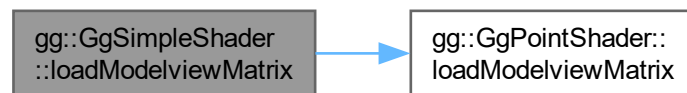
モデルビュー変換行列と法線変換行列を設定する。

引数

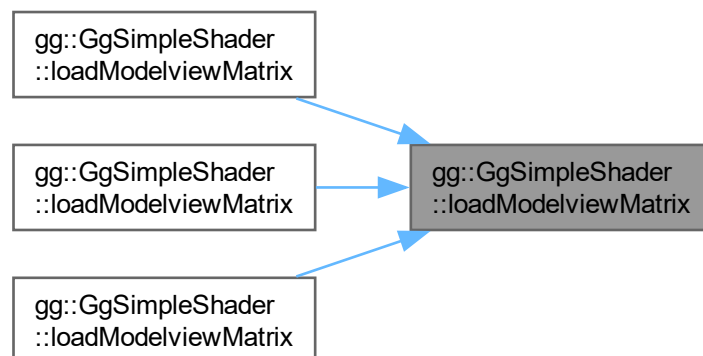
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7359 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.11 operator=()

```
GgSimpleShader & gg::GgSimpleShader::operator= (  
    const GgSimpleShader & o) [inline]
```

代入演算子.

引数

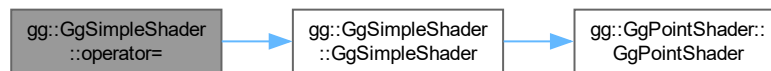
o 代入する [GgSimpleShader](#) 型のオブジェクト.

戻り値

代入後の [GgSimpleShader](#) 型のオブジェクトの参照.

[gg.h](#) の 7307 行目に定義があります。

呼び出し関係図:



8.25.3.12 use() [1/13]

```
void gg::GgSimpleShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する.

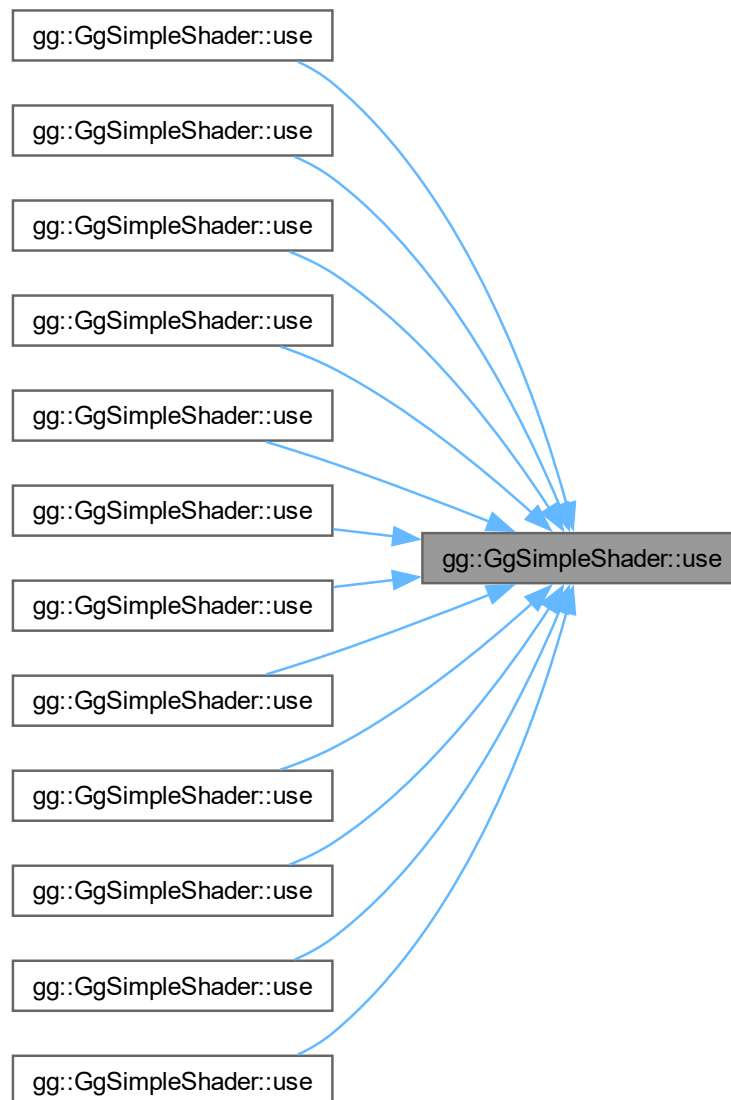
[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 8011 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.13 use() [2/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

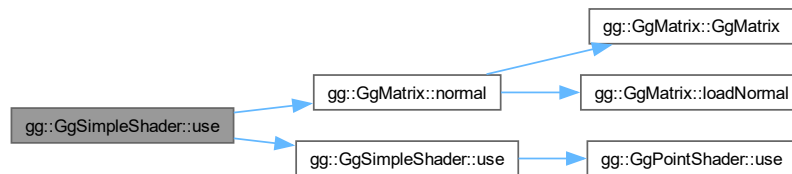
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg.h](#) の 8062 行目に定義があります。

呼び出し関係図:



8.25.3.14 use() [3/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline]
  
```

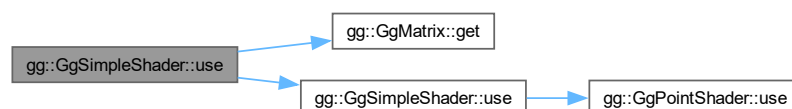
投影変換行列とモデルビュー変換行列と法線変換行列を指定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

[gg.h](#) の 8040 行目に定義があります。

呼び出し関係図:



8.25.3.15 use() [4/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

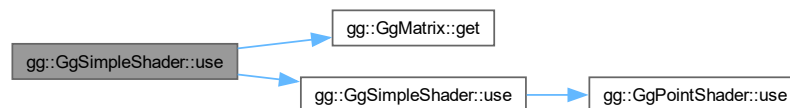
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8126 行目に定義があります。

呼び出し関係図:



8.25.3.16 use() [5/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

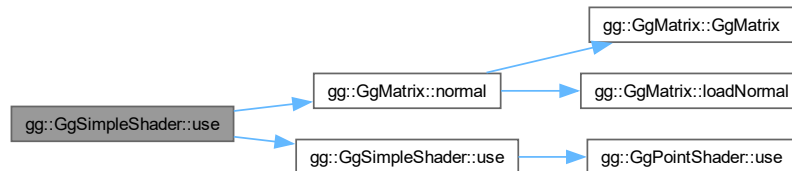
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.

<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 8163 行目に定義があります。

呼び出し関係図:



8.25.3.17 use() [6/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

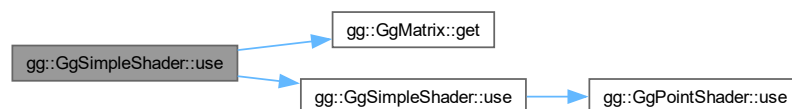
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8196 行目に定義があります。

呼び出し関係図:



8.25.3.18 use() [7/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

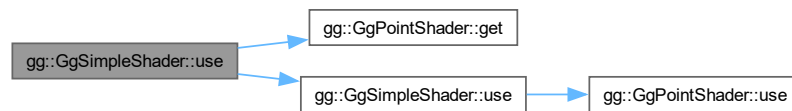
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 8051 行目に定義があります。

呼び出し関係図:



8.25.3.19 use() [8/13]

```

void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline]
  
```

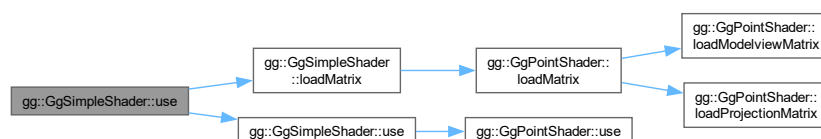
投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 8024 行目に定義があります。

呼び出し関係図:



8.25.3.20 use() [9/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

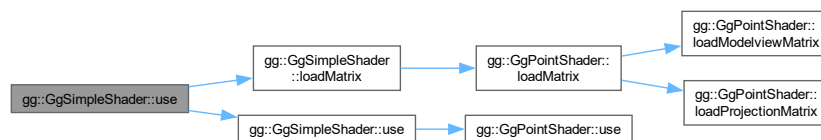
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8102 行目に定義があります。

呼び出し関係図:



8.25.3.21 use() [10/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

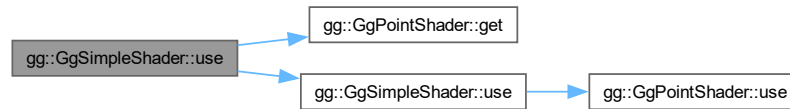
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.

<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 8145 行目に定義があります。

呼び出し関係図:



8.25.3.22 use() [11/13]

```

void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const LightBuffer * light,
    GLint i = 0) const [inline]
  
```

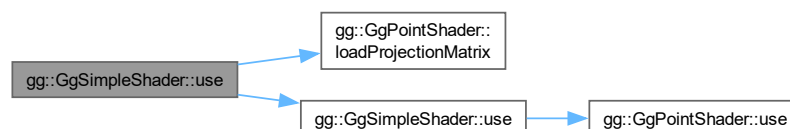
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8180 行目に定義があります。

呼び出し関係図:



8.25.3.23 use() [12/13]

```

void gg::GgSimpleShader::use (
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

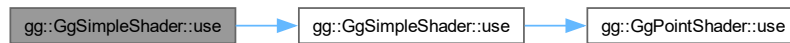
光源を指定してシェードプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の <code>gg::LightBuffer</code> 構造体.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

`gg.h` の 8088 行目に定義があります。

呼び出し関係図:



8.25.3.24 use() [13/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

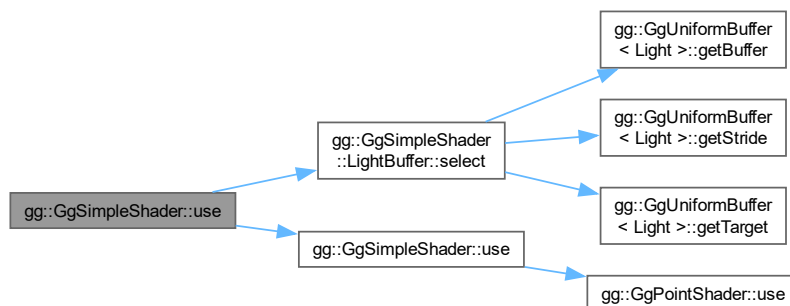
光源を指定してシェードプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の <code>gg::LightBuffer</code> 構造体のポインタ.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

`gg.h` の 8073 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

8.26 gg::GgTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- [GgTexture](#) (const GgTexture &texture)=delete
- [GgTexture](#) (GgTexture &&texture)=default
- virtual ~[GgTexture](#) ()
- [GgTexture](#) & operator= (const [GgTexture](#) &texture)=delete
- [GgTexture](#) & operator= ([GgTexture](#) &&texture)=default
- void [bind](#) () const
- void [unbind](#) () const
- void [swapRandB](#) (bool swizzle) const
- const GLsizei & [getWidth](#) () const
- const GLsizei & [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- const GLuint & [getTexture](#) () const

8.26.1 詳解

テクスチャ.

覚え書き

画像データを読み込んでテクスチャマップを作成する.

[gg.h](#) の 5193 行目に定義があります。

8.26.2 構築子と解体子

8.26.2.1 GgTexture() [1/3]

```
gg::GgTexture::GgTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [inline]
```

メモリ上のデータからテクスチャを作成するコンストラクタ.

引数

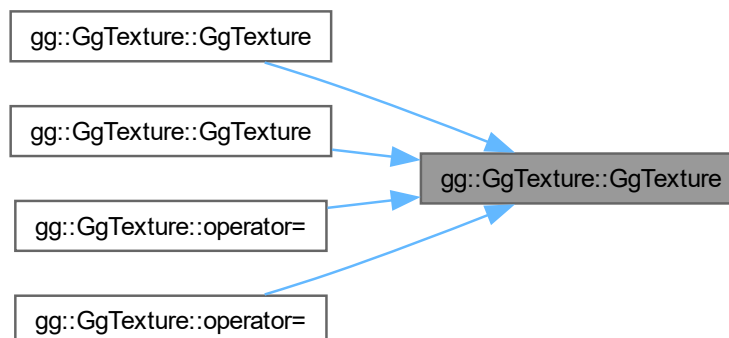
<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL.CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

gg.h の 5215 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.26.2.2 GgTexture() [2/3]

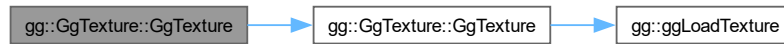
```
gg::GgTexture::GgTexture (
    const GgTexture & texture) [delete]
```

コピーコンストラクタは使用しない。

引数

<i>texture</i>	コピー元のテクスチャ.
----------------	-------------

呼び出し関係図:



8.26.2.3 GgTexture() [3/3]

```
gg::GgTexture::GgTexture (
    GgTexture && texture) [default]
```

ムーブコンストラクタ.

引数

<i>texture</i>	ムーブ元のテクスチャ.
----------------	-------------

呼び出し関係図:



8.26.2.4 ~GgTexture()

```
virtual gg::GgTexture::~~GgTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5247 行目に定義があります。

8.26.3 関数詳解

8.26.3.1 bind()

```
void gg::GgTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

[gg.h](#) の 5272 行目に定義があります。

8.26.3.2 getHeight()

```
const GLsizei & gg::GgTexture::getHeight () const [inline]
```

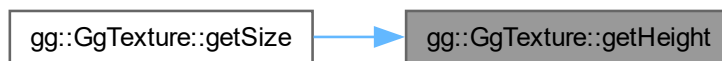
使用しているテクスチャの縦の画素数を取り出す。

戻り値

テクスチャの縦の画素数。

[gg.h](#) の 5307 行目に定義があります。

被呼び出し関係図:



8.26.3.3 getSize() [1/2]

```
const GLsizei * gg::GgTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す。

戻り値

テクスチャのサイズを格納した配列へのポインタ。

[gg.h](#) の 5328 行目に定義があります。

8.26.3.4 getSize() [2/2]

```
void gg::GgTexture::getSize (
    GLsizei * size) const [inline]
```

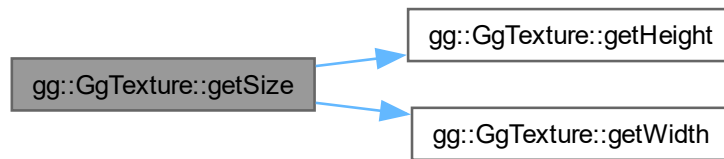
使用しているテクスチャのサイズを取り出す。

引数

size	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数。
------	--------------------------------------

[gg.h](#) の 5317 行目に定義があります。

呼び出し関係図:



8.26.3.5 getTexture()

```
const GLuint & gg::GgTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

[gg.h](#) の 5338 行目に定義があります。

8.26.3.6 getWidth()

```
const GLsizei & gg::GgTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

[gg.h](#) の 5297 行目に定義があります。

被呼び出し関係図:



8.26.3.7 operator=() [1/2]

```
GgTexture & gg::GgTexture::operator= (
    const GgTexture & texture) [delete]
```

代入演算子は使用しない.

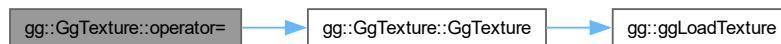
引数

<i>texture</i>	代入元のテクスチャ.
----------------	------------

戻り値

代入後のこのテクスチャの参照.

呼び出し関係図:



8.26.3.8 operator=() [2/2]

```
GgTexture & gg::GgTexture::operator= (
    GgTexture && texture) [default]
```

ムーブ代入演算子.

引数

<i>texture</i>	ムーブ代入元のテクスチャ.
----------------	---------------

戻り値

ムーブ代入後のこのテクスチャの参照.

呼び出し関係図:



8.26.3.9 swapRandB()

```
void gg::GgTexture::swapRandB (
    bool swizzle) const
```

テクスチャの赤と青を交換する

引数

swizzle	赤と青を交換するなら true
---------	-----------------

gg.cpp の 4006 行目に定義があります。

8.26.3.10 unbind()

```
void gg::GgTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

gg.h の 5280 行目に定義があります。

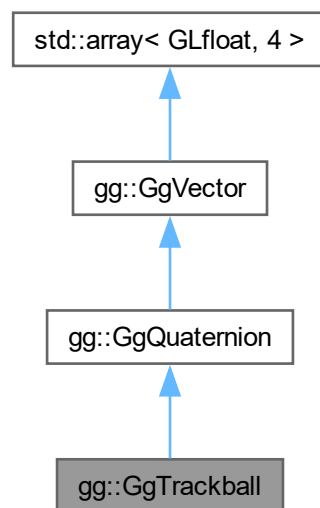
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

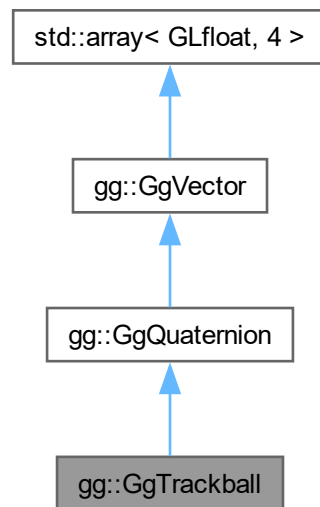
8.27 gg::GgTrackball クラス

```
#include <gg.h>
```

gg::GgTrackball の継承関係図



gg::GgTrackball 連携図



公開メンバ関数

- `GgTrackball` (`const GgQuaternion &q=gglIdentityQuaternion()`)
- `~GgTrackball` `()=default`
- `GgTrackball & operator=` (`const GgQuaternion &q`)
- `void region` (`GLfloat w, GLfloat h`)
- `void region` (`int w, int h`)
- `void begin` (`GLfloat x, GLfloat y`)
- `void motion` (`GLfloat x, GLfloat y`)
- `void rotate` (`const GgQuaternion &q`)
- `void end` (`GLfloat x, GLfloat y`)
- `void reset` (`const GgQuaternion &q=gglIdentityQuaternion()`)
- `const GLfloat * getStart` `() const`
- `const GLfloat & getStart` (`int direction`) `const`
- `void getStart` (`GLfloat *position`) `const`
- `const GLfloat * getScale` `() const`
- `const GLfloat getScale` (`int direction`) `const`
- `void getScale` (`GLfloat *factor`) `const`
- `const GgQuaternion & getQuaternion` `() const`
- `const GgMatrix & getMatrix` `() const`
- `const GLfloat * get` `() const`

基底クラス `gg::GgQuaternion` に属する継承公開メンバ関数

- `GgQuaternion` `()=default`
- `constexpr GgQuaternion` (`GLfloat x, GLfloat y, GLfloat z, GLfloat w`)
- `constexpr GgQuaternion` (`GLfloat c`)
- `GgQuaternion` (`const GLfloat *a`)

- GgQuaternion (const GgVector &v)
- GgQuaternion (const GgQuaternion &q)=default
- GgQuaternion (GgQuaternion &&q)=default
- ~GgQuaternion ()=default
- GgQuaternion & operator= (const GgQuaternion &q)=default
- GgQuaternion & operator= (GgQuaternion &&q)=default
- GLfloat norm () const
- GgQuaternion & loadAdd (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadAdd (const GLfloat *a)
- GgQuaternion & loadAdd (const GgVector &v)
- GgQuaternion & loadAdd (const GgQuaternion &q)
- GgQuaternion & loadSubtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadSubtract (const GLfloat *a)
- GgQuaternion & loadSubtract (const GgVector &v)
- GgQuaternion & loadSubtract (const GgQuaternion &q)
- GgQuaternion & loadMultiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadMultiply (const GLfloat *a)
- GgQuaternion & loadMultiply (const GgVector &v)
- GgQuaternion & loadMultiply (const GgQuaternion &q)
- GgQuaternion & loadDivide (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadDivide (const GLfloat *a)
- GgQuaternion & loadDivide (const GgVector &v)
- GgQuaternion & loadDivide (const GgQuaternion &q)
- GgQuaternion add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion add (const GLfloat *a) const
- GgQuaternion add (const GgVector &v) const
- GgQuaternion add (const GgQuaternion &q) const
- GgQuaternion subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion subtract (const GLfloat *a) const
- GgQuaternion subtract (const GgVector &v) const
- GgQuaternion subtract (const GgQuaternion &q) const
- GgQuaternion multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion multiply (const GLfloat *a) const
- GgQuaternion multiply (const GgVector &v) const
- GgQuaternion multiply (const GgQuaternion &q) const
- GgQuaternion divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion divide (const GLfloat *a) const
- GgQuaternion divide (const GgVector &v) const
- GgQuaternion divide (const GgQuaternion &q) const
- GgQuaternion & operator= (const GLfloat *a)
- GgQuaternion & operator= (const GgVector &v)
- GgQuaternion & operator+= (const GLfloat *a)
- GgQuaternion & operator+= (const GgVector &v)
- GgQuaternion & operator+= (const GgQuaternion &q)
- GgQuaternion & operator-= (const GLfloat *a)
- GgQuaternion & operator-= (const GgVector &v)
- GgQuaternion & operator-= (const GgQuaternion &q)
- GgQuaternion & operator*= (const GLfloat *a)
- GgQuaternion & operator*= (const GgVector &v)
- GgQuaternion & operator*= (const GgQuaternion &q)
- GgQuaternion & operator/= (const GLfloat *a)
- GgQuaternion & operator/= (const GgVector &v)
- GgQuaternion & operator/= (const GgQuaternion &q)
- GgQuaternion operator+ (const GLfloat *a) const
- GgQuaternion operator+ (const GgVector &v) const

- [GgQuaternion operator+](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator-](#) (const GLfloat *a) const
- [GgQuaternion operator-](#) (const [GgVector](#) &v) const
- [GgQuaternion operator-](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator*](#) (const GLfloat *a) const
- [GgQuaternion operator*](#) (const [GgVector](#) &v) const
- [GgQuaternion operator*](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator/](#) (const GLfloat *a) const
- [GgQuaternion operator/](#) (const [GgVector](#) &v) const
- [GgQuaternion operator/](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion & loadMatrix](#) (const GLfloat *a)
- [GgQuaternion & loadMatrix](#) (const [GgMatrix](#) &m)
- [GgQuaternion & loadIdentity](#) ()
- [GgQuaternion & loadRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v)
- [GgQuaternion & loadRotateX](#) (GLfloat a)
- [GgQuaternion & loadRotateY](#) (GLfloat a)
- [GgQuaternion & loadRotateZ](#) (GLfloat a)
- [GgQuaternion rotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v) const
- [GgQuaternion rotateX](#) (GLfloat a) const
- [GgQuaternion rotateY](#) (GLfloat a) const
- [GgQuaternion rotateZ](#) (GLfloat a) const
- [GgQuaternion & loadEuler](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion & loadEuler](#) (const GLfloat *e)
- [GgQuaternion euler](#) (GLfloat heading, GLfloat pitch, GLfloat roll) const
- [GgQuaternion euler](#) (const GLfloat *e) const
- [GgQuaternion euler](#) (const [GgVector](#) &e) const
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- [GgQuaternion & loadNormalize](#) (const GLfloat *a)
- [GgQuaternion & loadNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadConjugate](#) (const GLfloat *a)
- [GgQuaternion & loadConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadInvert](#) (const GLfloat *a)
- [GgQuaternion & loadInvert](#) (const [GgQuaternion](#) &q)
- [GgQuaternion slerp](#) (GLfloat *a, GLfloat t) const
- [GgQuaternion slerp](#) (const [GgQuaternion](#) &q, GLfloat t) const
- [GgQuaternion normalize](#) () const
- [GgQuaternion conjugate](#) () const
- [GgQuaternion invert](#) () const
- void [get](#) (GLfloat *a) const
- void [getMatrix](#) (GLfloat *a) const
- void [getMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getMatrix](#) () const
- void [getConjugateMatrix](#) (GLfloat *a) const
- void [getConjugateMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getConjugateMatrix](#) () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & **operator=** (const **GgVector** &v)=default
- **GgVector** & **operator=** (**GgVector** &&v)=default
- **GgVector** **operator+** (const **GgVector** &v) const
- **GgVector** **operator+** (GLfloat c) const
- **GgVector** & **operator+=** (const **GgVector** &v)
- **GgVector** & **operator+=** (GLfloat c)
- **GgVector** **operator-** (const **GgVector** &v) const
- **GgVector** **operator-** (GLfloat c) const
- **GgVector** & **operator-=** (const **GgVector** &v)
- **GgVector** & **operator-=** (GLfloat c)
- **GgVector** **operator*** (const **GgVector** &v) const
- **GgVector** **operator*** (GLfloat c) const
- **GgVector** & **operator*=** (const **GgVector** &v)
- **GgVector** & **operator*=** (GLfloat c)
- **GgVector** **operator/** (const **GgVector** &v) const
- **GgVector** **operator/** (GLfloat c) const
- **GgVector** & **operator/=** (**GgVector** &v)
- **GgVector** & **operator/=** (GLfloat c)
- GLfloat **dot3** (const **GgVector** &v) const
- GLfloat **length3** () const
- GLfloat **distance3** (const **GgVector** &v) const
- **GgVector** **normalize3** () const
- GLfloat **dot4** (const **GgVector** &v) const
- GLfloat **length4** () const
- GLfloat **distance4** (const **GgVector** &v) const
- **GgVector** **normalize4** () const

8.27.1 詳解

簡易トラックボール処理.

gg.h の 4775 行目に定義があります。

8.27.2 構築子と解体子

8.27.2.1 GgTrackball()

```
gg::GgTrackball::GgTrackball (
    const GgQuaternion & q = ggIdentityQuaternion()) [inline]
```

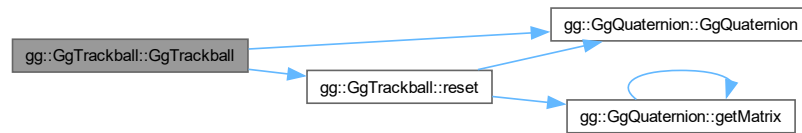
コンストラクタ.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

[gg.h](#) の 4790 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.2.2 ~GgTrackball()

```
gg::GgTrackball::~~GgTrackball () [default]
```

デストラクタ.

8.27.3 関数詳解

8.27.3.1 begin()

```
void gg::GgTrackball::begin (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を開始する.

引数

x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ開始時 (マウスボタンを押したとき) に呼び出す.

[gg.cpp](#) の 3453 行目に定義があります。

8.27.3.2 end()

```
void gg::GgTrackball::end (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を停止する.

引数

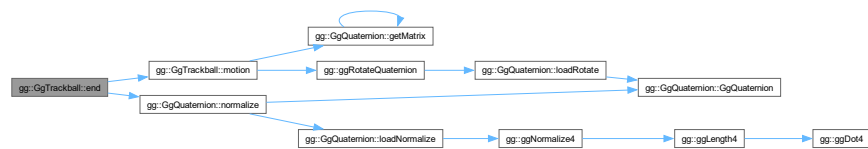
<i>x</i>	現在のマウスの <i>x</i> 座標.
<i>y</i>	現在のマウスの <i>y</i> 座標.

覚え書き

マウスのドラッグ終了時 (マウスボタンを離したとき) に呼び出す.

gg.cpp の 3519 行目に定義があります。

呼び出し関係図:



8.27.3.3 get()

```
const GLfloat * gg::GgTrackball::get () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GLfloat 型の 16 要素の配列.

gg.h の 4968 行目に定義があります。

8.27.3.4 getMatrix()

```
const GgMatrix & gg::GgTrackball::getMatrix () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GgMatrix 型の変換行列.

gg.h の 4958 行目に定義があります。

8.27.3.5 getQuaternion()

```
const GgQuaternion & gg::GgTrackball::getQuaternion () const [inline]
```

現在の回転の四元数を取り出す.

戻り値

回転の変換を表す Quaternion 型の四元数.

gg.h の 4948 行目に定義があります。

呼び出し関係図:



8.27.3.6 getScale() [1/3]

```
const GLfloat * gg::GgTrackball::getScale () const [inline]
```

トラックボール処理の換算係数を取り出す.

戻り値

トラックボールの換算係数のポインタ.

gg.h の 4917 行目に定義があります。

8.27.3.7 getScale() [2/3]

```
void gg::GgTrackball::getScale (
    GLfloat * factor) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>factor</i>	トラックボールの換算係数を格納する 2 要素の配列.
---------------	----------------------------

gg.h の 4937 行目に定義があります。

8.27.3.8 getScale() [3/3]

```
const GLfloat gg::GgTrackball::getScale (
    int direction) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

gg.h の 4927 行目に定義があります。

8.27.3.9 getStart() [1/3]

```
const GLfloat * gg::GgTrackball::getStart () const [inline]
```

トラックボール処理の開始位置を取り出す。

戻り値

トラックボールの開始位置のポインタ。

gg.h の 4888 行目に定義があります。

8.27.3.10 getStart() [2/3]

```
void gg::GgTrackball::getStart (
    GLfloat * position) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>position</i>	トラックボールの開始位置を格納する 2 要素の配列.
-----------------	----------------------------

gg.h の 4906 行目に定義があります。

8.27.3.11 getStart() [3/3]

```
const GLfloat & gg::GgTrackball::getStart (
    int direction) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

gg.h の 4896 行目に定義があります。

8.27.3.12 motion()

```
void gg::GgTrackball::motion (
    GLfloat x,
    GLfloat y)
```

回転の変換行列を計算する。

引数

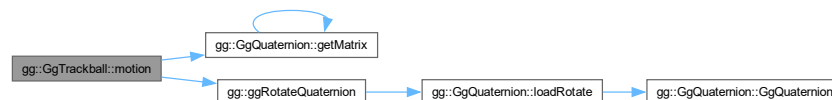
x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ中に呼び出す.

`gg.cpp` の 3469 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.3.13 operator=()

```
GgTrackball & gg::GgTrackball::operator= (
    const GgQuaternion & q) [inline]
```

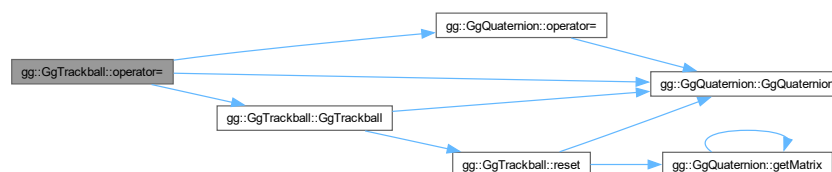
代入.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

`gg.h` の 4805 行目に定義があります。

呼び出し関係図:



8.27.3.14 region() [1/2]

```
void gg::GgTrackball::region (
    GLfloat w,
    GLfloat h)
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.cpp](#) の 3440 行目に定義があります。

被呼び出し関係図:



8.27.3.15 region() [2/2]

```
void gg::GgTrackball::region (
    int w,
    int h) [inline]
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.h](#) の [4831](#) 行目に定義があります。

呼び出し関係図:



8.27.3.16 reset()

```
void gg::GgTrackball::reset (
    const GgQuaternion & q = ggIdentityQuaternion())
```

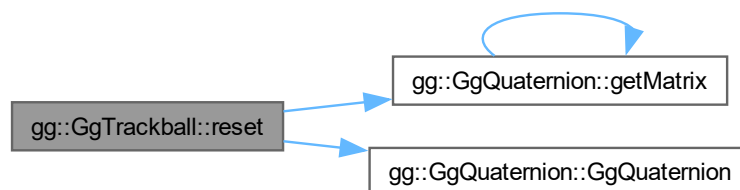
トラックボールをリセットする.

引数

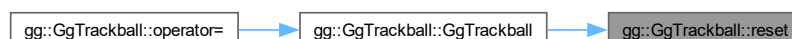
<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.cpp](#) の [3422](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.3.17 rotate()

```
void gg::GgTrackball::rotate (
    const GgQuaternion & q)
```

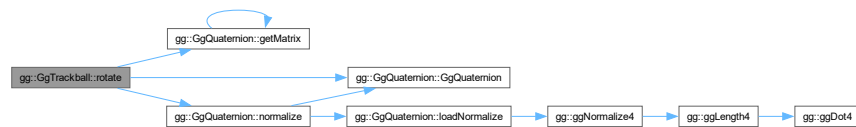
トラックボールの回転角を修正する。

引数

<i>q</i>	修正分の回転角の四元数.
----------	--------------

gg.cpp の 3498 行目に定義があります。

呼び出し関係図:



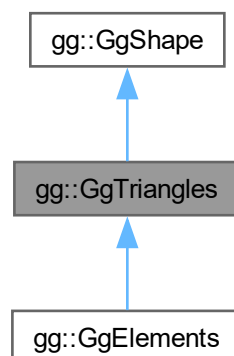
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

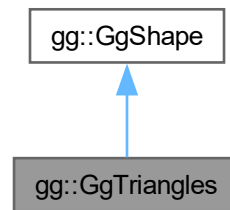
8.28 gg::GgTriangles クラス

```
#include <gg.h>
```

gg::GgTriangles の継承関係図



gg::GgTriangles 連携図



公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

8.28.1 詳解

三角形で表した形状データ (Arrays 形式).

[gg.h](#) の 6509 行目に定義があります。

8.28.2 構築子と解体子

8.28.2.1 GgTriangles() [1/2]

```
gg::GgTriangles::GgTriangles (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

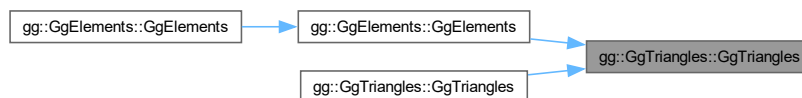
<i>mode</i>	描画する基本図形の種類.
-------------	--------------

gg.h の 6522 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.28.2.2 GgTriangles() [2/2]

```

gg::GgTriangles::GgTriangles (
    const GgVertex * vert,
    GLsizei count,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

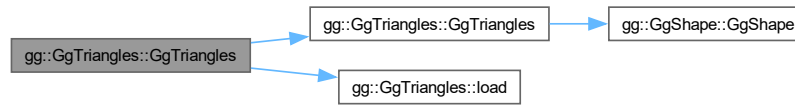
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>count</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6535 行目に定義があります。

呼び出し関係図:



8.28.2.3 ~GgTriangles()

```
virtual gg::GgTriangles::~~GgTriangles () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の [6549](#) 行目に定義があります。

8.28.3 関数詳解

8.28.3.1 draw()

```
void gg::GgTriangles::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

三角形の描画.

引数

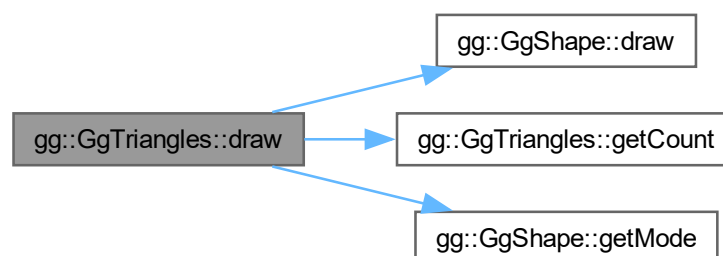
<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgShape](#) を再実装しています。

[gg::GgElements](#) で再実装されています。

[gg.cpp](#) の [5157](#) 行目に定義があります。

呼び出し関係図:



8.28.3.2 getBuffer()

```
const GLuint & gg::GgTriangles::getBuffer () const [inline]
```

頂点属性を格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の頂点属性を格納した頂点バッファオブジェクト名。

gg.h の 6568 行目に定義があります。

8.28.3.3 getCount()

```
const GLsizei & gg::GgTriangles::getCount () const [inline]
```

データの数を取り出す。

戻り値

この図形の頂点属性の数 (頂点数)。

gg.h の 6558 行目に定義があります。

被呼び出し関係図:



8.28.3.4 load()

```
void gg::GgTriangles::load (
    const GgVertex * vert,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点属性を格納する。

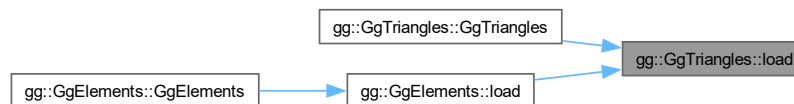
引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
-------------	-------------------------

<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

[gg.cpp](#) の 5138 行目に定義があります。

被呼び出し関係図:



8.28.3.5 send()

```
void gg::GgTriangles::send (
    const GgVertex * vert,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性を転送する。

引数

<i>vert</i>	転送元の頂点属性が格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6580 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.29 gg::GgUniformBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgUniformBuffer](#) ()
- [GgUniformBuffer](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- [GgUniformBuffer](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [load](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [load](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.29.1 詳解

```
template<typename T>
class gg::GgUniformBuffer< T >
```

ユニフォームバッファオブジェクト.

覚え書き

ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

[gg.h](#) の 5827 行目に定義があります。

8.29.2 構築子と解体子

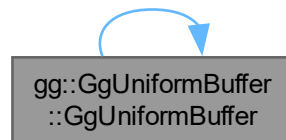
8.29.2.1 GgUniformBuffer() [1/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer () [inline]
```

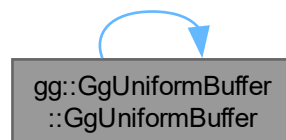
コンストラクタ.

gg.h の 5830 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.2.2 GgUniformBuffer() [2/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

ユニフォームバッファオブジェクトのブロックごとにデータを転送するコンストラクタ.

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5830 行目に定義があります。

8.29.2.3 GgUniformBuffer() [3/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

ユニフォームバッファオブジェクトの全ブロックに同じデータを格納するコンストラクタ.

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5830 行目に定義があります。

8.29.2.4 ~GgUniformBuffer()

```
template<typename T>
virtual gg::GgUniformBuffer< T >::~~GgUniformBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 5830 行目に定義があります。

8.29.3 関数詳解

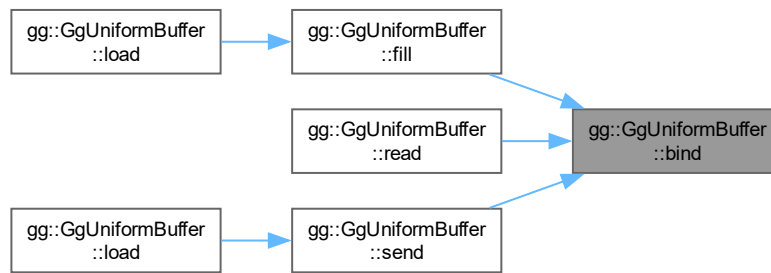
8.29.3.1 bind()

```
template<typename T>
void gg::GgUniformBuffer< T >::bind () const [inline]
```

ユニフォームバッファオブジェクトを結合する.

gg.h の 5915 行目に定義があります。

被呼び出し関係図:



8.29.3.2 copy()

```

template<typename T>
void gg::GgUniformBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
  
```

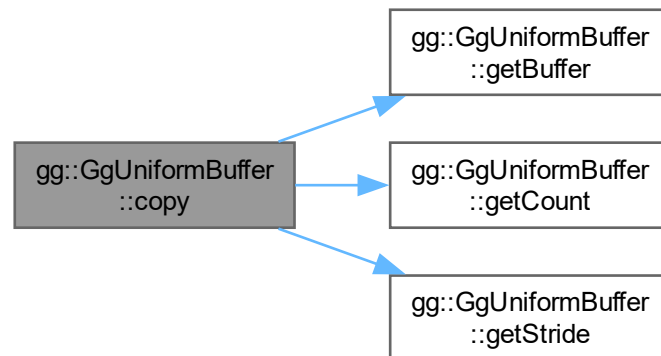
別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (buffer) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6126 行目に定義があります。

呼び出し関係図:



8.29.3.3 fill()

```

template<typename T>
void gg::GgUniformBuffer< T >::fill (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

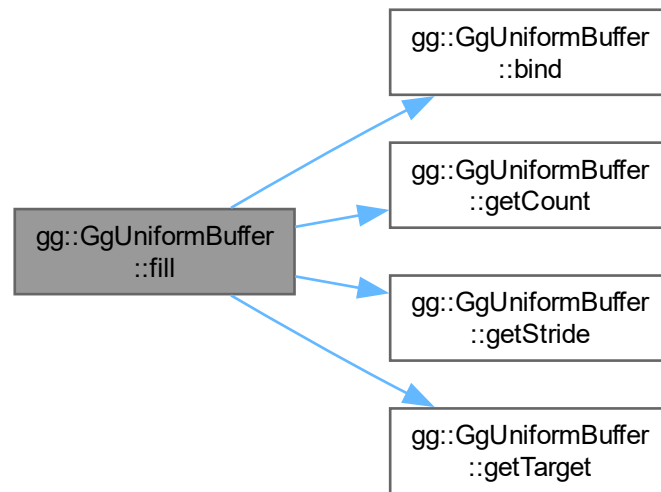
ユニフォームバッファオブジェクトの全ブロックのメンバーを同じデータを格納する.

引数

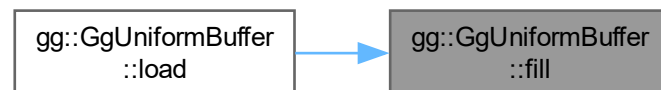
<i>data</i>	格納するデータ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

gg.h の 6043 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.3.4 getBuffer()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getBuffer () const [inline]
```

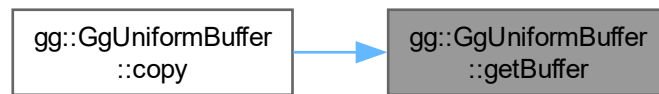
ユニフォームバッファオブジェクト名を取り出す.

戻り値

このユニフォームバッファオブジェクト名.

[gg.h](#) の 5907 行目に定義があります。

被呼び出し関係図:



8.29.3.5 getCount()

```
template<typename T>
const GLsizei & gg::GgUniformBuffer< T >::getCount () const [inline]
```

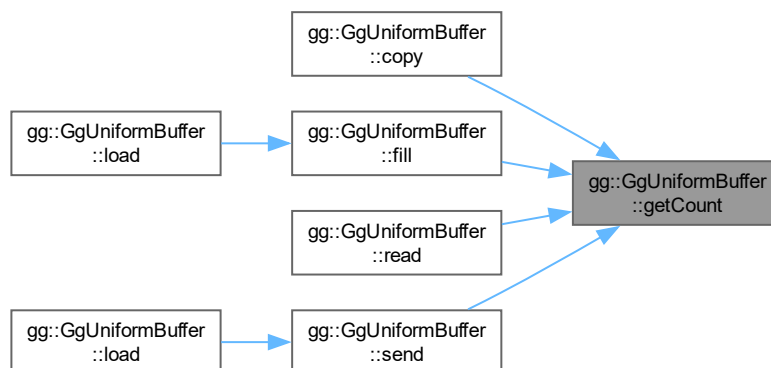
データの数を取り出す.

戻り値

このユニフォームバッファオブジェクトのデータの数.

gg.h の 5897 行目に定義があります。

被呼び出し関係図:



8.29.3.6 getStride()

```
template<typename T>
GLsizeiPtr gg::GgUniformBuffer< T >::getStride () const [inline]
```

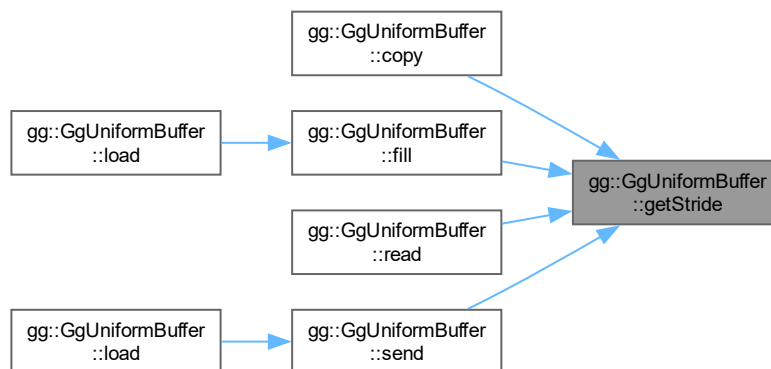
ユニフォームバッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このユニフォームバッファオブジェクトのデータの間隔。

gg.h の 5887 行目に定義があります。

被呼び出し関係図:



8.29.3.7 getTarget()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getTarget () const [inline]
```

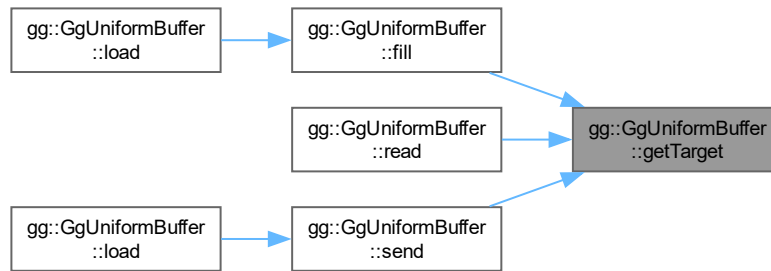
ユニフォームバッファオブジェクトのターゲットを取り出す。

戻り値

このユニフォームバッファオブジェクトのターゲット。

gg.h の 5877 行目に定義があります。

被呼び出し関係図:



8.29.3.8 load() [1/2]

```

template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

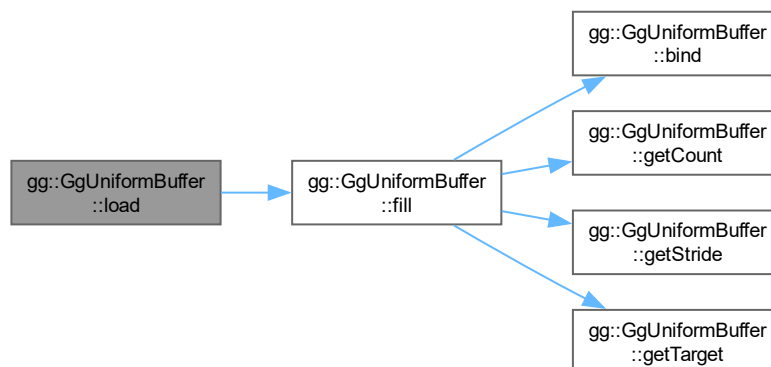
ユニフォームバッファオブジェクトを確保して全てのブロックに同じデータを格納する。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5984 行目に定義があります。

呼び出し関係図:



8.29.3.9 load() [2/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

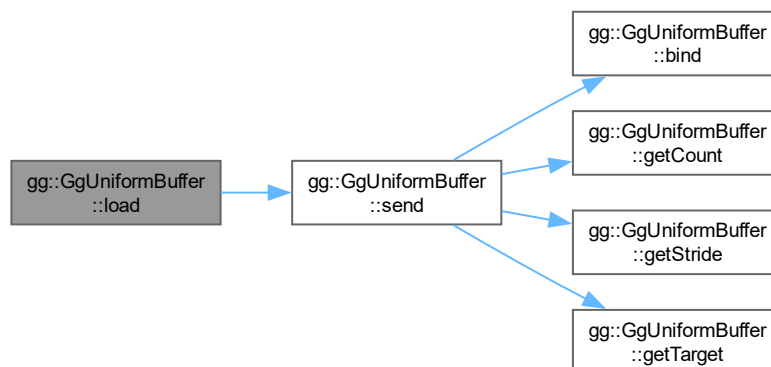
ユニフォームバッファオブジェクトを確保してブロックごとにデータを転送する。

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5965 行目に定義があります。

呼び出し関係図:



8.29.3.10 map() [1/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map () const [inline]
```

ユニフォームバッファオブジェクトをマップする。

戻り値

マップしたメモリの先頭のポインタ。

gg.h の 5933 行目に定義があります。

8.29.3.11 map() [2/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

ユニフォームバッファオブジェクトの指定した範囲をマップする。

引数

<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

[gg.h](#) の 5945 行目に定義があります。

8.29.3.12 read()

```
template<typename T>
void gg::GgUniformBuffer< T >::read (
    GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

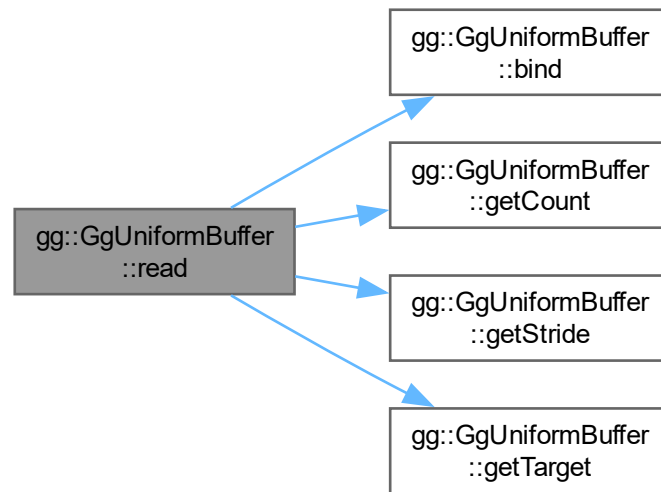
ユニフォームバッファオブジェクトからデータを抽出する。

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>offset</i>	抽出元のユニフォームバッファオブジェクトのメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	抽出するデータの一個あたりのバイト数.
<i>first</i>	抽出元のユニフォームバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	抽出するデータの数 (0 ならユニフォームバッファオブジェクト全体).

[gg.h](#) の 6078 行目に定義があります。

呼び出し関係図:



8.29.3.13 send()

```

template<typename T>
void gg::GgUniformBuffer< T >::send (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

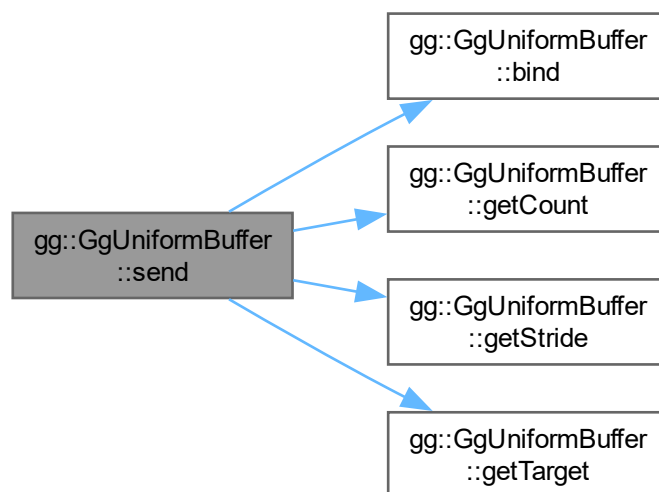
ユニフォームバッファオブジェクトを確保してユニフォームバッファオブジェクトのブロックごとのメンバを同じデータで埋める。

引数

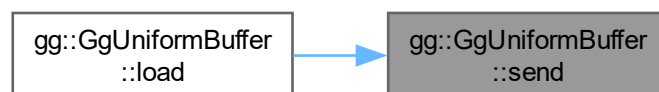
<i>data</i>	データが格納されている領域の先頭のポインタ。
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット。
<i>size</i>	格納するデータの一個あたりのバイト数。
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号。
<i>count</i>	格納するデータの数。

`gg.h` の 6005 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.3.14 unbind()

```
template<typename T>
void gg::GgUniformBuffer< T >::unbind () const [inline]
```

ユニフォームバッファオブジェクトを解放する.

[gg.h](#) の 5923 行目に定義があります。

8.29.3.15 unmap()

```
template<typename T>
void gg::GgUniformBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする.

[gg.h](#) の 5953 行目に定義があります。

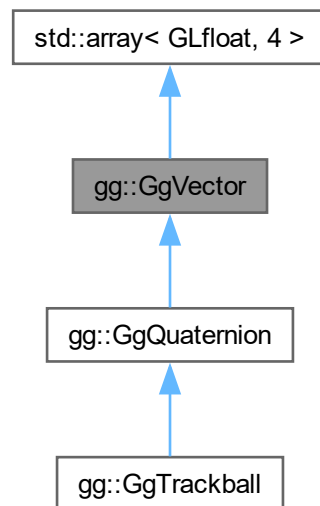
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

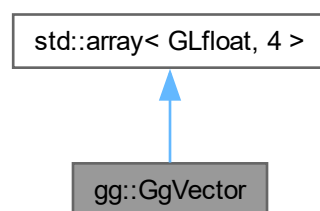
8.30 gg::GgVector クラス

```
#include <gg.h>
```

gg::GgVector の継承関係図



gg::GgVector 連携図



公開メンバ関数

- `GgVector()`=default
- `constexpr GgVector` (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- `constexpr GgVector` (const GLfloat *a)
- `constexpr GgVector` (GLfloat c)
- `GgVector` (const GgVector &v)=default

- `GgVector` (`GgVector &&v`)=default
- `~GgVector` ()=default
- `GgVector & operator=` (const `GgVector` &v)=default
- `GgVector & operator=` (`GgVector` &&v)=default
- `GgVector operator+` (const `GgVector` &v) const
- `GgVector operator+` (GLfloat c) const
- `GgVector & operator+=` (const `GgVector` &v)
- `GgVector & operator+=` (GLfloat c)
- `GgVector operator-` (const `GgVector` &v) const
- `GgVector operator-` (GLfloat c) const
- `GgVector & operator-=` (const `GgVector` &v)
- `GgVector & operator-=` (GLfloat c)
- `GgVector operator*` (const `GgVector` &v) const
- `GgVector operator*` (GLfloat c) const
- `GgVector & operator*=` (const `GgVector` &v)
- `GgVector & operator*=` (GLfloat c)
- `GgVector operator/` (const `GgVector` &v) const
- `GgVector operator/` (GLfloat c) const
- `GgVector & operator/=` (`GgVector` &v)
- `GgVector & operator/=` (GLfloat c)
- GLfloat `dot3` (const `GgVector` &v) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &v) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &v) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &v) const
- `GgVector normalize4` () const

8.30.1 詳解

単精度実数の 4 要素のベクトル。

`gg.h` の 1571 行目に定義があります。

8.30.2 構築子と解体子

8.30.2.1 GgVector() [1/6]

```
gg::GgVector::GgVector () [default]
```



```
GLfloat v2,
GLfloat v3) [inline], [constexpr]
```

コンストラクタ.

引数

<i>v0</i>	GLfloat 型の値.
<i>v1</i>	GLfloat 型の値.
<i>v2</i>	GLfloat 型の値.
<i>v3</i>	GLfloat 型の値.

gg.h の 1588 行目に定義があります。

8.30.2.3 GgVector() [3/6]

```
gg::GgVector::GgVector (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

gg.h の 1598 行目に定義があります。

呼び出し関係図:



8.30.2.4 GgVector() [4/6]

```
gg::GgVector::GgVector (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

c	GLfloat 型の値.
---	--------------

gg.h の 1608 行目に定義があります。

呼び出し関係図:



8.30.2.5 GgVector() [5/6]

```
gg::GgVector::GgVector (  
    const GgVector & v) [default]
```

コピーコンストラクタ.

引数

v	コピー元のベクトル.
---	------------

呼び出し関係図:



8.30.2.6 GgVector() [6/6]

```
gg::GgVector::GgVector (  
    GgVector && v) [default]
```

ムーブコンストラクタ.

引数

v	ムーブ元のベクトル.
-----	------------

呼び出し関係図:



8.30.2.7 ~GgVector()

```
gg::GgVector::~~GgVector () [default]
```

デストラクタ.

8.30.3 関数詳解

8.30.3.1 distance3()

```
GLfloat gg::GgVector::distance3 (
    const GgVector & v) const [inline]
```

[GgVector](#) 型の 3 要素の距離.

引数

v	GgVector 型のベクトル.
-----	----------------------------------

戻り値

オブジェクトと v の 3 要素の距離.

[gg.h](#) の 1852 行目に定義があります。

呼び出し関係図:



8.30.3.2 distance4()

```
GLfloat gg::GgVector::distance4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の距離.

引数

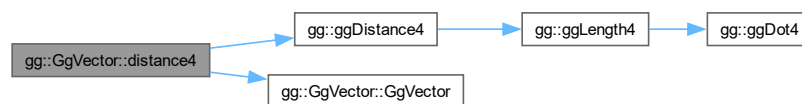
v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** の 4 要素の距離.

gg.h の 1896 行目に定義があります。

呼び出し関係図:



8.30.3.3 dot3()

```
GLfloat gg::GgVector::dot3 (  
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の内積.

引数

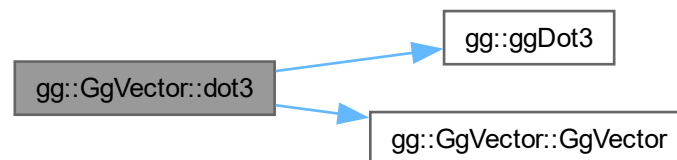
v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと v のそれぞれの 3 要素の内積.

gg.h の 1831 行目に定義があります。

呼び出し関係図:



8.30.3.4 dot4()

```
GLfloat gg::GgVector::dot4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の内積.

引数

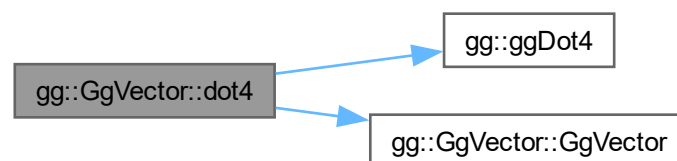
v	GgVector 型のベクトル.
-----	------------------

戻り値

オブジェクトと v のそれぞれの 4 要素の内積.

gg.h の 1875 行目に定義があります。

呼び出し関係図:



8.30.3.5 length3()

```
GLfloat gg::GgVector::length3 () const [inline]
```

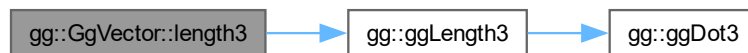
GgVector 型の 3 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1841 行目に定義があります。

呼び出し関係図:



8.30.3.6 length4()

```
GLfloat gg::GgVector::length4 () const [inline]
```

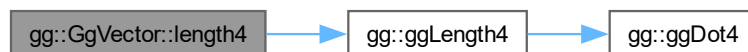
GgVector 型の 4 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1885 行目に定義があります。

呼び出し関係図:



8.30.3.7 normalize3()

```
GgVector gg::GgVector::normalize3 () const [inline]
```

GgVector 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

gg.h の 1862 行目に定義があります。

呼び出し関係図:



8.30.3.8 normalize4()

```
GgVector gg::GgVector::normalize4 () const [inline]
```

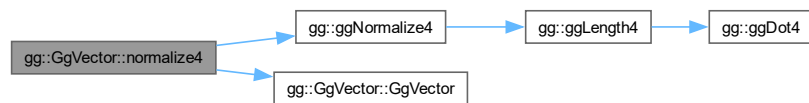
GgVector 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

gg.h の 1906 行目に定義があります。

呼び出し関係図:



8.30.3.9 operator*() [1/2]

```
GgVector gg::GgVector::operator* (  
    const GgVector & v) const [inline]
```

GgVector 型の積を返す.

引数

v	GgVector 型の変数.
----------	-----------------------

戻り値

オブジェクトの各要素と **v** の各要素の要素ごとの積のオブジェクト.

gg.h の 1740 行目に定義があります.

呼び出し関係図:



8.30.3.10 operator*() [2/2]

```
GgVector gg::GgVector::operator* (  
    GLfloat c) const [inline]
```

GgVector 型の各要素にスカラーを乗じた積を返す.

引数

c	GLfloat 型の変数.
----------	----------------------

戻り値

オブジェクトの各要素に **c** を乗じたオブジェクト.

gg.h の 1751 行目に定義があります.

呼び出し関係図:



8.30.3.11 operator*=() [1/2]

```
GgVector & gg::GgVector::operator*= (  
    const GgVector & v) [inline]
```

GgVector 型を乗算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ乗算したオブジェクトの参照.

`gg.h` の 1761 行目に定義があります.

呼び出し関係図:



8.30.3.12 operator*=() [2/2]

```
GgVector & gg::GgVector::operator*= (  
    GLfloat c) [inline]
```

`GgVector` 型の各要素にスカラーを乗じる.

引数

<code>c</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクトの参照.

`gg.h` の 1773 行目に定義があります.

呼び出し関係図:



8.30.3.13 operator+() [1/2]

```
GgVector gg::GgVector::operator+ (  
    const GgVector & v) const [inline]
```

GgVector 型の和を返す.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの和のオブジェクト.

`gg.h` の 1648 行目に定義があります.

呼び出し関係図:



8.30.3.14 operator+() [2/2]

```
GgVector gg::GgVector::operator+ (
    GLfloat c) const [inline]
```

`GgVector` 型の各要素にスカラーを足した和を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

`a` の各要素に `b` を足した和のオブジェクト.

`gg.h` の 1659 行目に定義があります.

呼び出し関係図:



8.30.3.15 operator+=() [1/2]

```
GgVector & gg::GgVector::operator+= (  
    const GgVector & v) [inline]
```

GgVector 型を加算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ加算したオブジェクトの参照.

`gg.h` の 1670 行目に定義があります.

呼び出し関係図:



8.30.3.16 operator+=() [2/2]

```
GgVector & gg::GgVector::operator+= (
    GLfloat c) [inline]
```

`GgVector` 型の各要素にスカラーを加算する.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

オブジェクトの各要素に `c` を足したオブジェクトの参照.

`gg.h` の 1682 行目に定義があります.

呼び出し関係図:



8.30.3.17 operator-() [1/2]

```
GgVector gg::GgVector::operator- (
    const GgVector & v) const [inline]
```

GgVector 型の差を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの差のオブジェクト.

`gg.h` の 1694 行目に定義があります.

呼び出し関係図:



8.30.3.18 operator-() [2/2]

```
GgVector gg::GgVector::operator- (  
    GLfloat c) const [inline]
```

`GgVector` 型の各要素からスカラーを引いた差を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクト.

`gg.h` の 1705 行目に定義があります.

呼び出し関係図:



8.30.3.19 operator-=() [1/2]

```
GgVector & gg::GgVector::operator-= (  
    const GgVector & v) [inline]
```

GgVector 型を減算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ減算したオブジェクトの参照.

`gg.h` の 1716 行目に定義があります.

呼び出し関係図:



8.30.3.20 operator-=() [2/2]

```
GgVector & gg::GgVector::operator-= (
    GLfloat c) [inline]
```

`GgVector` 型の各要素からスカラーを引く.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクトの参照.

`gg.h` の 1728 行目に定義があります.

呼び出し関係図:



8.30.3.21 operator/() [1/2]

```
GgVector gg::GgVector::operator/ (
    const GgVector & v) const [inline]
```

GgVector 型の商を返す.

引数

v	GgVector 型の変数.
----------	-----------------------

戻り値

オブジェクトの各要素を **v** の各要素で要素ごとに割った結果のオブジェクト.

gg.h の 1785 行目に定義があります.

呼び出し関係図:



8.30.3.22 operator/() [2/2]

```
GgVector gg::GgVector::operator/ (  
    GLfloat c) const [inline]
```

GgVector 型の各要素をスカラーで割った商を返す.

引数

c	GLfloat 型の変数.
----------	----------------------

戻り値

オブジェクトの各要素を **c** で割ったオブジェクト.

gg.h の 1796 行目に定義があります.

呼び出し関係図:



8.30.3.23 operator/=() [1/2]

```
GgVector & gg::GgVector::operator/= (  
    GgVector & v) [inline]
```

GgVector 型を除算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ乗算したオブジェクトの参照.

`gg.h` の 1807 行目に定義があります.

呼び出し関係図:



8.30.3.24 operator/=() [2/2]

```
GgVector & gg::GgVector::operator/= (
    GLfloat c) [inline]
```

`GgVector` 型の各要素をスカラーで割る.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素を `c` で割ったオブジェクトの参照.

`gg.h` の 1819 行目に定義があります.

呼び出し関係図:



8.30.3.25 operator=() [1/2]

```
GgVector & gg::GgVector::operator= (  
    const GgVector & v) [default]
```

代入演算子. 呼び出し関係図:



8.30.3.26 operator=() [2/2]

```
GgVector & gg::GgVector::operator= (  
    GgVector && v) [default]
```

ムーブ代入演算子. 呼び出し関係図:



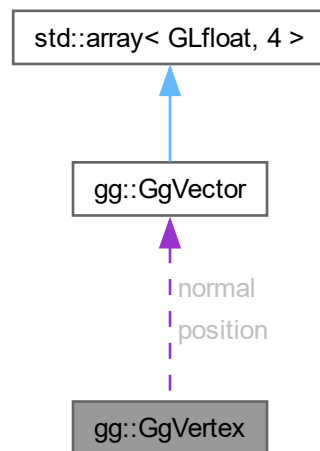
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.31 gg::GgVertex 構造体

```
#include <gg.h>
```

gg::GgVertex 連携図



公開メンバ関数

- `GgVertex ()`
- `GgVertex (const GgVector &pos, const GgVector &norm)`
- `GgVertex (GLfloat px, GLfloat py, GLfloat pz, GLfloat nx, GLfloat ny, GLfloat nz)`
- `GgVertex (const GLfloat *pos, const GLfloat *norm)`

公開変数類

- `GgVector position`
位置.
- `GgVector normal`
法線.

8.31.1 詳解

三角形の頂点データ.

`gg.h` の 6448 行目に定義があります。

8.31.2 構築子と解体子

8.31.2.1 GgVertex() [1/4]

```
gg::GgVertex::GgVertex () [inline]
```

コンストラクタ.

[gg.h](#) の 6459 行目に定義があります。

被呼び出し関係図:



8.31.2.2 GgVertex() [2/4]

```
gg::GgVertex::GgVertex (
    const GgVector & pos,
    const GgVector & norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	GgVector 型の位置データ.
<i>norm</i>	GgVector 型の法線データ.

[gg.h](#) の 6469 行目に定義があります。

8.31.2.3 GgVertex() [3/4]

```
gg::GgVertex::GgVertex (
    GLfloat px,
    GLfloat py,
    GLfloat pz,
    GLfloat nx,
    GLfloat ny,
    GLfloat nz) [inline]
```

コンストラクタ.

引数

<i>px</i>	GgVector 型の位置データの x 成分.
<i>py</i>	GgVector 型の位置データの y 成分.
<i>pz</i>	GgVector 型の位置データの z 成分.
<i>nx</i>	GgVector 型の法線データの x 成分.
<i>ny</i>	GgVector 型の法線データの y 成分.
<i>nz</i>	GgVector 型の法線データの z 成分.

gg.h の 6485 行目に定義があります。

8.31.2.4 GgVertex() [4/4]

```
gg::GgVertex::GgVertex (
    const GLfloat * pos,
    const GLfloat * norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	3 要素の GLfloat 型の位置データのポインタ.
<i>norm</i>	3 要素の GLfloat 型の法線データのポインタ.

gg.h の 6500 行目に定義があります。

呼び出し関係図:



8.31.3 メンバ詳解

8.31.3.1 normal

GgVector gg::GgVertex::normal

法線.

gg.h の 6454 行目に定義があります。

8.31.3.2 position

`GgVector` gg::GgVertex::position

位置.

gg.h の 6451 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- gg.h

8.32 gg::GgVertexArray クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgVertexArray` (GLenum mode=0)
- `GgVertexArray` (const GgVertexArray &array)=delete
- `GgVertexArray` (GgVertexArray &&array)=default
- virtual `~GgVertexArray` ()
- `GgVertexArray & operator=` (const `GgVertexArray` &array)=delete
- `GgVertexArray & operator=` (`GgVertexArray` &&array)=default
- const GLuint & `get` () const
- void `bind` () const

8.32.1 詳解

頂点配列 クラス.

gg.h の 6157 行目に定義があります。

8.32.2 構築子と解体子

8.32.2.1 GgVertexArray() [1/3]

```
gg::GgVertexArray::GgVertexArray (
    GLenum mode = 0) [inline]
```

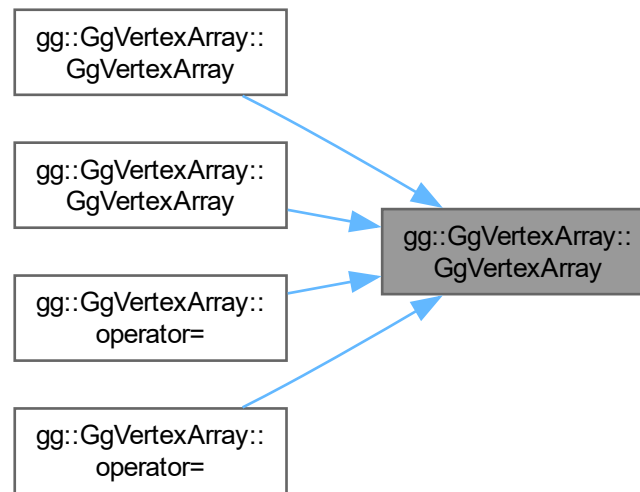
コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6169 行目に定義があります。

被呼び出し関係図:



8.32.2.2 GgVertexArray() [2/3]

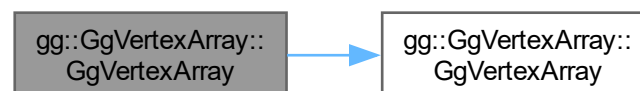
```
gg::GgVertexArray::GgVertexArray (
    const GgVertexArray & array) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>array</i>	コピー元の頂点配列 オブジェクト.
--------------	-------------------

呼び出し関係図:



8.32.2.3 GgVertexArray() [3/3]

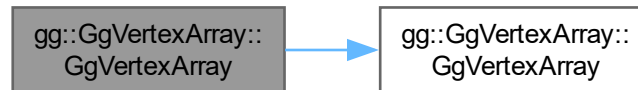
```
gg::GgVertexArray::GgVertexArray (  
    GgVertexArray && array) [default]
```

ムーブコンストラクタ.

引数

<code>array</code>	ムーブ元の頂点配列オブジェクト.
--------------------	------------------

呼び出し関係図:



8.32.2.4 ~GgVertexArray()

```
virtual gg::GgVertexArray::~~GgVertexArray () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6192 行目に定義があります。

8.32.3 関数詳解

8.32.3.1 bind()

```
void gg::GgVertexArray::bind () const [inline]
```

頂点配列オブジェクトを結合する.

[gg.h](#) の 6227 行目に定義があります。

8.32.3.2 get()

```
const GLuint & gg::GgVertexArray::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

[gg.h](#) の 6219 行目に定義があります。

8.32.3.3 operator=() [1/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    const GgVertexArray & array) [delete]
```

代入演算子を使用しない.

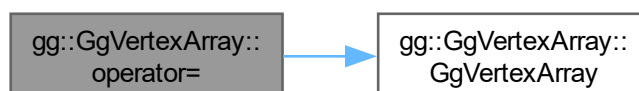
引数

<code>array</code>	代入元の頂点配列オブジェクト.
--------------------	-----------------

戻り値

代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



8.32.3.4 operator=() [2/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    GgVertexArray && array) [default]
```

ムーブ代入演算子.

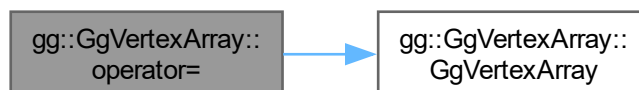
引数

<code>array</code>	ムーブ代入元の頂点配列オブジェクト.
--------------------	--------------------

戻り値

ムーブ代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.33 Intrinsic 構造体

```
#include <Intrinsic.h>
```

公開メンバ関数

- `Intrinsic()`=default
- `Intrinsic` (const std::array< float, 2 > &`fov`, const std::array< float, 2 > &`center`, const std::array< int, 2 > `size`, double `fps`)
- `Intrinsic` (const picojson::object &object)
- void `setFov` (float focal)
- void `setFov` (float fovx, float fovy)
- void `setCenter` (float x, float y)
- void `setSize` (int width, int height)
- void `setFps` (double frequency)

公開変数類

- std::array< float, 2 > `fov` { 50.03f, 38.58f }
キャプチャデバイスのレンズの縦横の画角
- std::array< float, 2 > `center` { 0.0f, 0.0f }
キャプチャデバイスのレンズの中心 (主点) の位置
- std::array< int, 2 > `size` { 640, 480 }
キャプチャデバイスの解像度
- double `fps` { 30.0 }
キャプチャデバイスのフレームレート

静的公開変数類

- static constexpr auto `sensorSize` { 35.0f }
展開時のセンサーサイズ

8.33.1 詳解

キャプチャデバイス固有のパラメータ

`Intrinsic.h` の 20 行目に定義があります。

8.33.2 構築子と解体子

8.33.2.1 Intrinsic() [1/3]

```
Intrinsic::Intrinsic () [default]
```

キャプチャデバイス固有のパラメータの構造体のデフォルトコンストラクタ

覚え書き

構成ファイルが読めなかったときにしか使わない

8.33.2.2 Intrinsics() [2/3]

```
Intrinsics::Intrinsics (  
    const std::array< float, 2 > & fov,  
    const std::array< float, 2 > & center,  
    const std::array< int, 2 > size,  
    double fps) [inline]
```

パラメータを指定するときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ

引数

<i>fov</i>	キャプチャデバイスのレンズの縦横の画角
<i>center</i>	キャプチャデバイスのレンズの中心 (主点) の位置
<i>size</i>	キャプチャデバイスの解像度
<i>fps</i>	キャプチャデバイスのフレームレート

[Intrinsics.h](#) の 50 行目に定義があります。

8.33.2.3 Intrinsics() [3/3]

```
Intrinsics::Intrinsics (  
    const picojson::object & object)
```

構成ファイルを読み込むときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ

[Intrinsics.cpp](#) の 14 行目に定義があります。

呼び出し関係図:



8.33.3 関数詳解

8.33.3.1 setCenter()

```
void Intrinsics::setCenter (  
    float x,  
    float y) [inline]
```

キャプチャデバイスのレンズの中心の位置を変更する

引数

<i>x</i>	キャプチャデバイスのレンズの中心の位置の <i>x</i> 座標
<i>y</i>	キャプチャデバイスのレンズの中心の位置の <i>y</i> 座標

[Intrinsics.h](#) の 94 行目に定義があります。

8.33.3.2 setFov() [1/2]

```
void Intrinsics::setFov (
    float focal)
```

スクリーンの高さをもとにしてキャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>focal</i>	撮像面の対角線長が <i>sensorSize</i> のときの焦点距離
--------------	--------------------------------------

[Intrinsics.cpp](#) の 32 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.33.3.3 setFov() [2/2]

```
void Intrinsics::setFov (
    float fovx,
    float fovy) [inline]
```

キャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>fovx</i>	キャプチャデバイスのレンズの <i>fovx</i> 方向の画角
<i>fovy</i>	キャプチャデバイスのレンズの <i>fovy</i> 方向の画角

[Intrinsics.h](#) の 81 行目に定義があります。

8.33.3.4 setFps()

```
void Intrinsics::setFps (  
    double frequency) [inline]
```

キャプチャデバイスのフレームレートを変更する

引数

<i>frequency</i>	フレームレート
------------------	---------

[Intrinsics.h](#) の 121 行目に定義があります。

8.33.3.5 setSize()

```
void Intrinsics::setSize (  
    int width,  
    int height) [inline]
```

キャプチャデバイスの解像度を変更する

引数

<i>width</i>	キャプチャデバイスのフレームの横の画素数
<i>height</i>	キャプチャデバイスのフレームの縦の画素数

[Intrinsics.h](#) の 108 行目に定義があります。

8.33.4 メンバ詳解

8.33.4.1 center

```
std::array<float, 2> Intrinsics::center { 0.0f, 0.0f }
```

キャプチャデバイスのレンズの中心 (主点) の位置

[Intrinsics.h](#) の 26 行目に定義があります。

8.33.4.2 fov

```
std::array<float, 2> Intrinsics::fov { 50.03f, 38.58f }
```

キャプチャデバイスのレンズの縦横の画角

[Intrinsics.h](#) の 23 行目に定義があります。

8.33.4.3 fps

```
double Intrinsics::fps { 30.0 }
```

キャプチャデバイスのフレームレート

[Intrinsics.h](#) の 32 行目に定義があります。

8.33.4.4 sensorSize

```
auto Intrinsics::sensorSize { 35.0f } [static], [constexpr]
```

展開時のセンサーサイズ

[Intrinsics.h](#) の 66 行目に定義があります。

8.33.4.5 size

```
std::array<int, 2> Intrinsics::size { 640, 480 }
```

キャプチャデバイスの解像度

[Intrinsics.h](#) の 29 行目に定義があります。

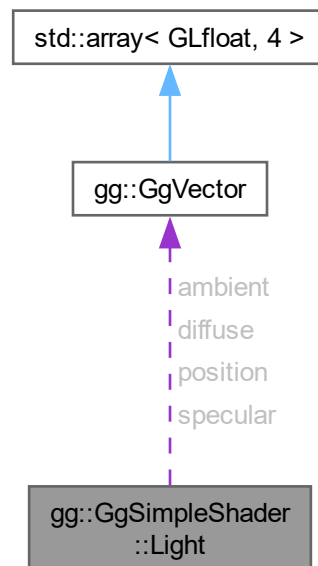
この構造体詳解は次のファイルから抽出されました:

- [Intrinsics.h](#)
- [Intrinsics.cpp](#)

8.34 gg::GgSimpleShader::Light 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Light 連携図



公開変数類

- `GgVector ambient`
光源強度の環境光成分.
- `GgVector diffuse`
光源強度の拡散反射光成分.
- `GgVector specular`
光源強度の鏡面反射光成分.
- `GgVector position`
光源の位置.

8.34.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データ.

`gg.h` の 7446 行目に定義があります。

8.34.2 メンバ詳解

8.34.2.1 ambient

```
GgVector gg::GgSimpleShader::Light::ambient
```

光源強度の環境光成分.

[gg.h](#) の 7448 行目に定義があります。

8.34.2.2 diffuse

```
GgVector gg::GgSimpleShader::Light::diffuse
```

光源強度の拡散反射光成分.

[gg.h](#) の 7449 行目に定義があります。

8.34.2.3 position

```
GgVector gg::GgSimpleShader::Light::position
```

光源の位置.

[gg.h](#) の 7451 行目に定義があります。

8.34.2.4 specular

```
GgVector gg::GgSimpleShader::Light::specular
```

光源強度の鏡面反射光成分.

[gg.h](#) の 7450 行目に定義があります。

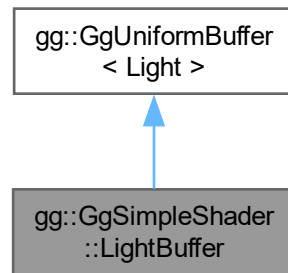
この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

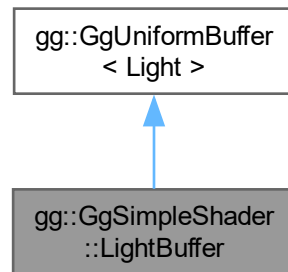
8.35 gg::GgSimpleShader::LightBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::LightBuffer の継承関係図



gg::GgSimpleShader::LightBuffer 連携図



公開メンバ関数

- [LightBuffer](#) (const [Light](#) *light=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [LightBuffer](#) (const [Light](#) &light, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [LightBuffer](#) ([GgVector](#) ambient, [GgVector](#) diffuse, [GgVector](#) specular, [GgVector](#) position, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual ~[LightBuffer](#) ()
- void [loadAmbient](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const [GgVector](#) &ambient, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (const [GgVector](#) &diffuse, GLint first=0, GLsizei count=1) const

- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GgVector &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadColor` (const Light &color, GLint first=0, GLsizei count=1) const
- void `loadPosition` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector &position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GLfloat *position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector *position, GLint first=0, GLsizei count=1) const
- void `load` (const Light *light, GLint first=0, GLsizei count=1) const
- void `load` (const Light &light, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Light >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const Light *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.35.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データのユニフォームバッファオブジェクト。

`gg.h` の 7457 行目に定義があります。

8.35.2 構築子と解体子

8.35.2.1 LightBuffer() [1/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light * light = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ。

引数

<i>light</i>	GgSimpleShader::Light 型の光源データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7469 行目に定義があります。

呼び出し関係図:



8.35.2.2 LightBuffer() [2/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light & light,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>light</i>	GgSimpleShader::Light 型の光源データ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7485 行目に定義があります。

呼び出し関係図:



8.35.2.3 LightBuffer() [3/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GgVector position,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	光源強度の環境光成分.
<i>diffuse</i>	光源強度の拡散反射光成分.
<i>specular</i>	光源強度の鏡面反射光成分.
<i>position</i>	光源の位置.
<i>count</i>	バッファ中の <code>GgSimpleShader::Light</code> 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

`gg.h` の 7504 行目に定義があります.

呼び出し関係図:



8.35.2.4 ~LightBuffer()

```
virtual gg::GgSimpleShader::LightBuffer::~~LightBuffer () [inline], [virtual]
```

デストラクタ.

`gg.h` の 7519 行目に定義があります.

8.35.3 関数詳解

8.35.3.1 load() [1/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light & light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

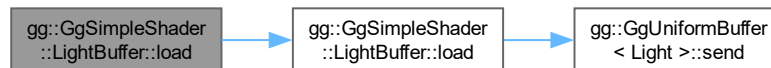
光源の色と位置を設定する.

引数

<i>light</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7711 行目に定義があります。

呼び出し関係図:



8.35.3.2 load() [2/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light * light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の色と位置を設定する。

引数

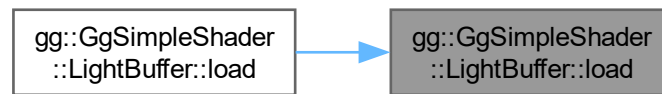
<i>light</i>	光源の特性の GgSimpleShader::Light 構造体のポインタ.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7699 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.35.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

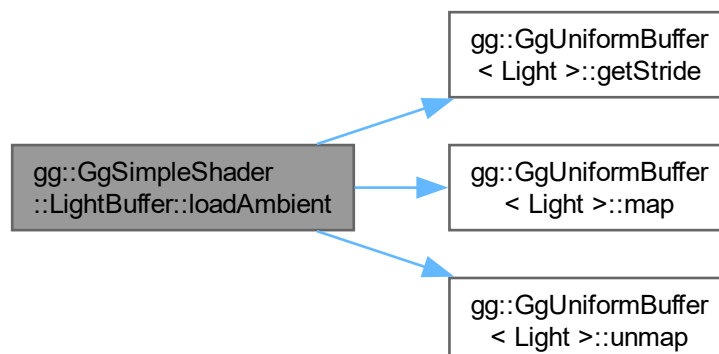
光源の強度の環境光成分を設定する.

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5483 行目に定義があります。

呼び出し関係図:



8.35.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7554 行目に定義があります。

呼び出し関係図:



8.35.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

光源の強度の環境光成分を設定する。

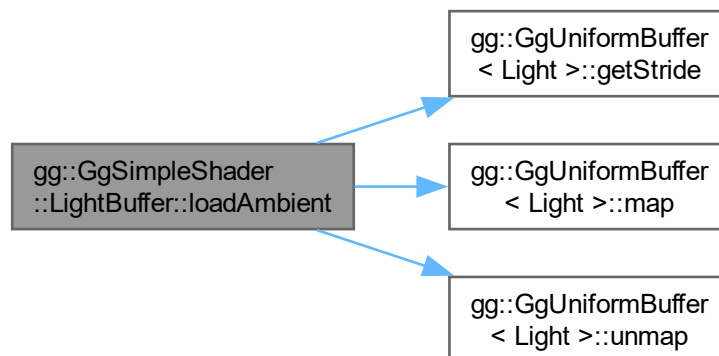
引数

<i>r</i>	光源の強度の環境光成分の赤成分.
<i>g</i>	光源の強度の環境光成分の緑成分.
<i>b</i>	光源の強度の環境光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.

<i>count</i>	値を設定する光源データの数, デフォルトは 1.
--------------	--------------------------

`gg.cpp` の 5457 行目に定義があります。

呼び出し関係図:



8.35.3.6 loadColor()

```
void gg::GgSimpleShader::LightBuffer::loadColor (
    const Light & color,
    GLint first = 0,
    GLsizei count = 1) const
```

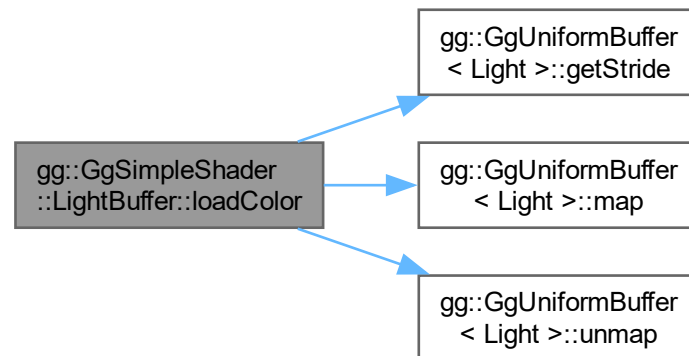
光源の色を設定するが位置は変更しない。

引数

<i>color</i>	光源の特性の <code>GgSimpleShader::Light</code> 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5610 行目に定義があります。

呼び出し関係図:



8.35.3.7 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

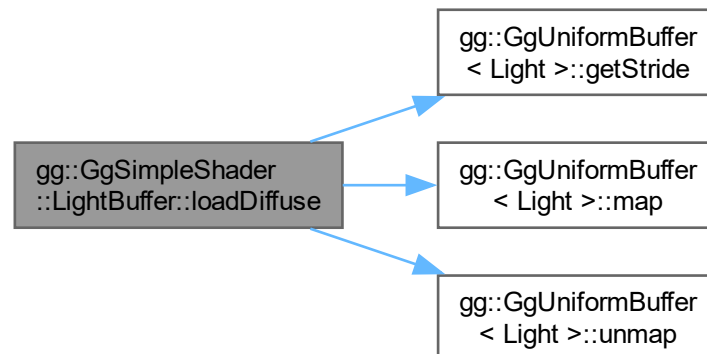
光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5535 行目に定義があります。

呼び出し関係図:



8.35.3.8 loadDiffuse() [2/3]

```

void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7591 行目に定義があります。

呼び出し関係図:



8.35.3.9 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

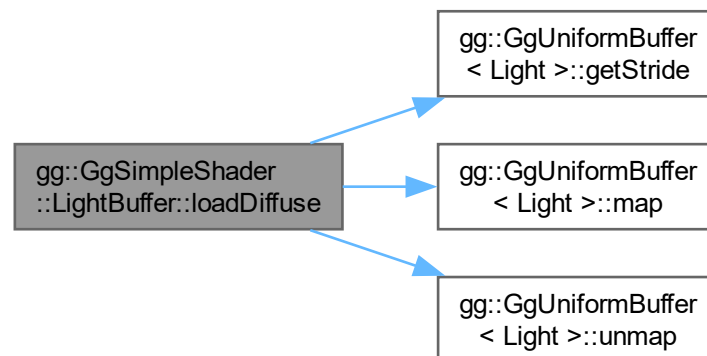
光源の強度の拡散反射光成分を設定する。

引数

<i>r</i>	光源の強度の拡散反射光成分の赤成分.
<i>g</i>	光源の強度の拡散反射光成分の緑成分.
<i>b</i>	光源の強度の拡散反射光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5509 行目に定義があります。

呼び出し関係図:



8.35.3.10 loadPosition() [1/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector & position,
    GLint first = 0,
    GLsizei count = 1) const
```

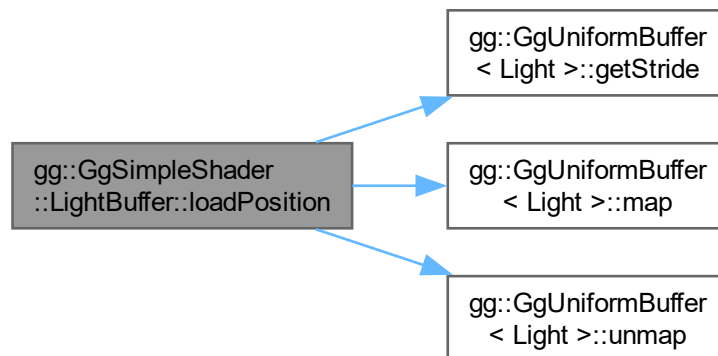
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5664 行目に定義があります。

呼び出し関係図:



8.35.3.11 loadPosition() [2/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

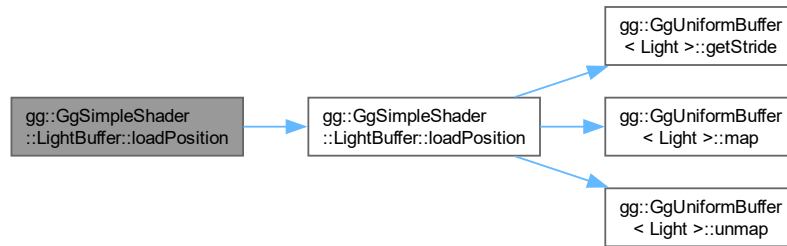
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の配列.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7687 行目に定義があります。

呼び出し関係図:



8.35.3.12 loadPosition() [3/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GLfloat * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

光源の位置を設定する.

引数

<i>position</i>	光源の位置の同次座標を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7674 行目に定義があります。

呼び出し関係図:



8.35.3.13 loadPosition() [4/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

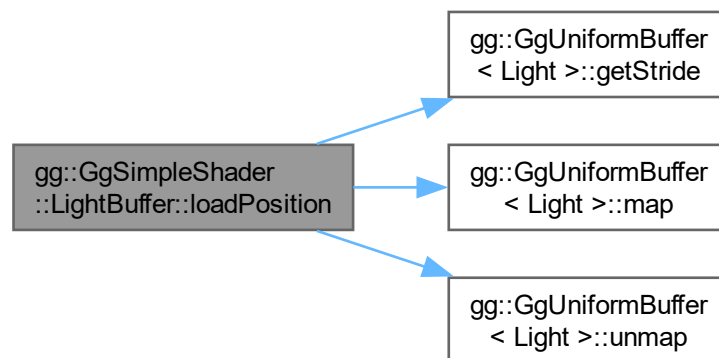
光源の位置を設定する。

引数

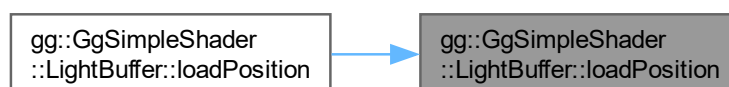
<i>x</i>	光源の位置の <i>x</i> 座標.
<i>y</i>	光源の位置の <i>y</i> 座標.
<i>z</i>	光源の位置の <i>z</i> 座標.
<i>w</i>	光源の位置の <i>w</i> 座標, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5638 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.35.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

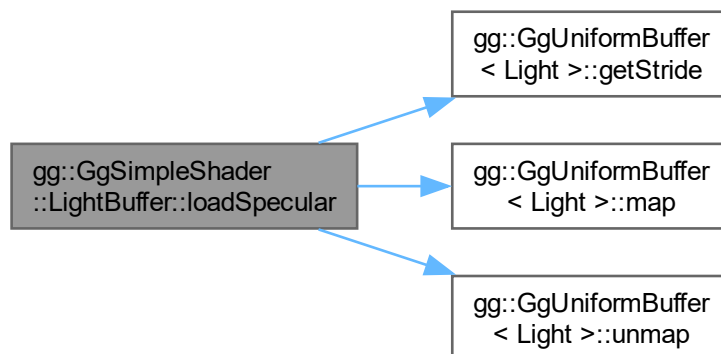
光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5587 行目に定義があります。

呼び出し関係図:



8.35.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GLfloat 型の 4 要素の配列変数.
-----------------	---

<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7628 行目に定義があります。

呼び出し関係図:



8.35.3.16 loadSpecular() [3/3]

```

void gg::GgSimpleShader::LightBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

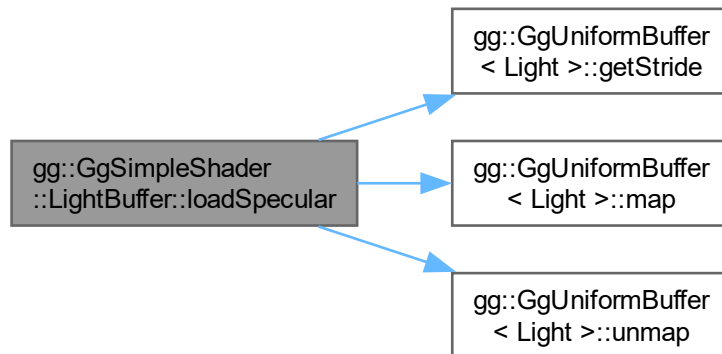
光源の強度の鏡面反射光成分を設定する。

引数

<i>r</i>	光源の強度の鏡面反射光成分の赤成分.
<i>g</i>	光源の強度の鏡面反射光成分の緑成分.
<i>b</i>	光源の強度の鏡面反射光成分の青成分.
<i>a</i>	光源の強度の鏡面反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5561 行目に定義があります。

呼び出し関係図:



8.35.3.17 select()

```
void gg::GgSimpleShader::LightBuffer::select (
    GLint i = 0) const [inline]
```

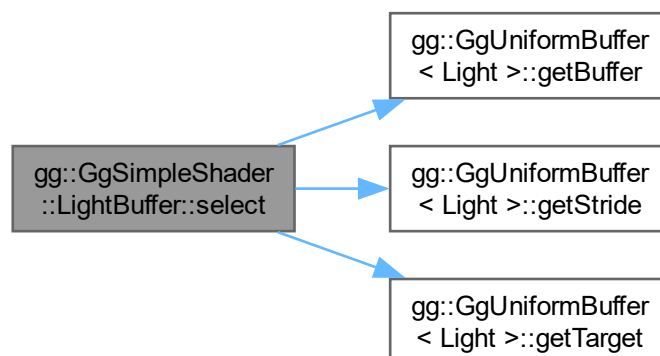
光源を選択する.

引数

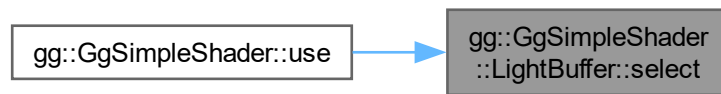
<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

[gg.h](#) の 7721 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



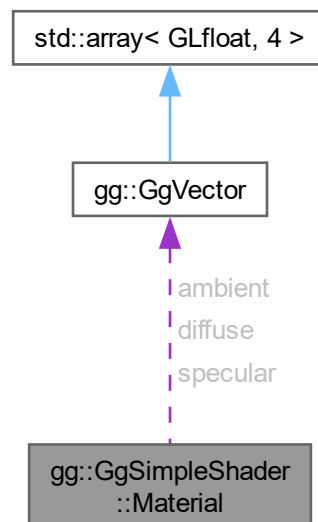
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.36 gg::GgSimpleShader::Material 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Material 連携図



公開変数類

- [GgVector ambient](#)
環境光に対する反射係数.
- [GgVector diffuse](#)
拡散反射係数.
- [GgVector specular](#)
鏡面反射係数.
- [GLfloat shininess](#)
輝き係数.

8.36.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データ.

[gg.h](#) の 7732 行目に定義があります。

8.36.2 メンバ詳解

8.36.2.1 ambient

```
GgVector gg::GgSimpleShader::Material::ambient
```

環境光に対する反射係数.

[gg.h](#) の 7734 行目に定義があります。

8.36.2.2 diffuse

```
GgVector gg::GgSimpleShader::Material::diffuse
```

拡散反射係数.

[gg.h](#) の 7735 行目に定義があります。

8.36.2.3 shininess

```
GLfloat gg::GgSimpleShader::Material::shininess
```

輝き係数.

[gg.h](#) の 7737 行目に定義があります。

8.36.2.4 specular

```
GgVector gg::GgSimpleShader::Material::specular
```

鏡面反射係数.

[gg.h](#) の 7736 行目に定義があります。

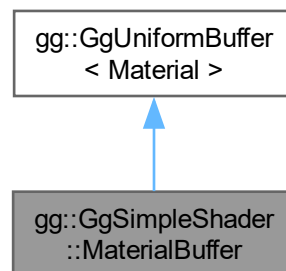
この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

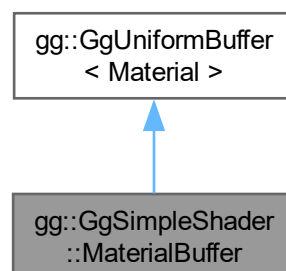
8.37 gg::GgSimpleShader::MaterialBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::MaterialBuffer の継承関係図



gg::GgSimpleShader::MaterialBuffer 連携図



公開メンバ関数

- [MaterialBuffer](#) (const [Material](#) *material=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [MaterialBuffer](#) (const [Material](#) &material, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [MaterialBuffer](#) ([GgVector](#) ambient, [GgVector](#) diffuse, [GgVector](#) specular, GLfloat shininess, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual [~MaterialBuffer](#) ()
- void [loadAmbient](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const [GgVector](#) &ambient, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (const [GgVector](#) &diffuse, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void [loadAmbientAndDiffuse](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadAmbientAndDiffuse](#) (const [GgVector](#) &color, GLint first=0, GLsizei count=1) const
- void [loadAmbientAndDiffuse](#) (const GLfloat *color, GLint first=0, GLsizei count=1) const
- void [loadSpecular](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadSpecular](#) (const [GgVector](#) &specular, GLint first=0, GLsizei count=1) const
- void [loadSpecular](#) (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void [loadShininess](#) (GLfloat shininess, GLint first=0, GLsizei count=1) const
- void [loadShininess](#) (const GLfloat *shininess, GLint first=0, GLsizei count=1) const
- void [load](#) (const [Material](#) *material, GLint first=0, GLsizei count=1) const
- void [load](#) (const [Material](#) &material, GLint first=0, GLsizei count=1) const
- void [select](#) (GLint i=0) const

基底クラス [gg::GgUniformBuffer< Material >](#) に属する継承公開メンバ関数

- [GgUniformBuffer](#) ()
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void [unmap](#) () const
- void [load](#) (const [Material](#) *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof([Material](#)), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof([Material](#)), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof([Material](#)), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.37.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データのユニフォームバッファオブジェクト。

[gg.h](#) の 7743 行目に定義があります。

8.37.2 構築子と解体子

8.37.2.1 MaterialBuffer() [1/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material * material = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

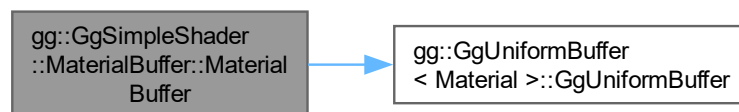
デフォルトコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7755 行目に定義があります.

呼び出し関係図:



8.37.2.2 MaterialBuffer() [2/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material & material,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

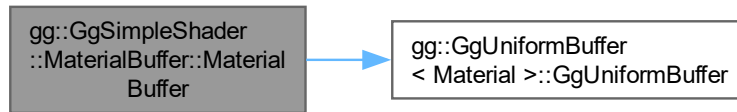
同じデータで埋めるコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7771 行目に定義があります.

呼び出し関係図:



8.37.2.3 MaterialBuffer() [3/3]

```

gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GLfloat shininess,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

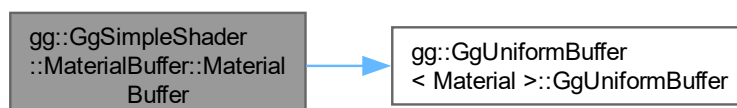
同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	環境光に対する反射係数.
<i>diffuse</i>	拡散反射係数.
<i>specular</i>	鏡面反射係数.
<i>shininess</i>	輝き係数.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7790 行目に定義があります。

呼び出し関係図:



8.37.2.4 ~MaterialBuffer()

```
virtual gg::GgSimpleShader::MaterialBuffer::~MaterialBuffer () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 7805 行目に定義があります。

8.37.3 関数詳解

8.37.3.1 load() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material & material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

材質を設定する.

引数

<i>material</i>	光源の特性の GgSimpleShader::Material 構造体.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7990 行目に定義があります。

呼び出し関係図:



8.37.3.2 load() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material * material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

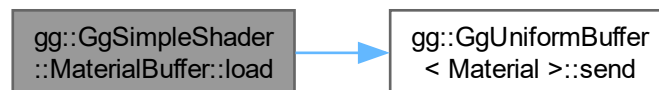
材質を設定する.

引数

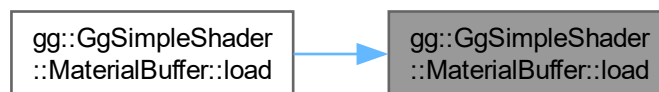
<i>material</i>	光源の特性の GgSimpleShader::Material 構造体のポインタ.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7978 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.37.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

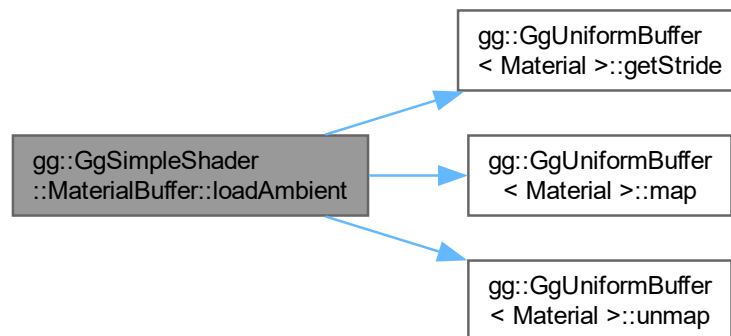
三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5716 行目に定義があります。

呼び出し関係図:



8.37.3.4 loadAmbient() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

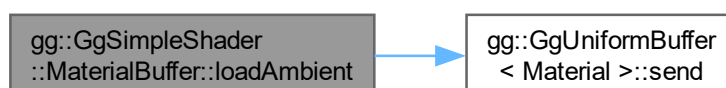
環境光に対する反射係数を設定する.

引数

<i>ambient</i>	環境光に対する反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7840 行目に定義があります。

呼び出し関係図:



8.37.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

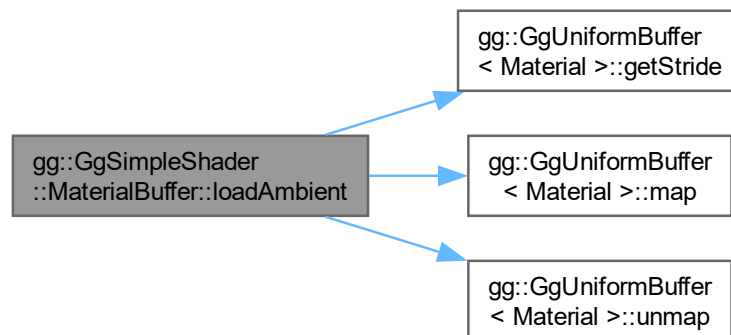
環境光に対する反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数の赤成分.
<i>g</i>	環境光に対する反射係数の緑成分.
<i>b</i>	環境光に対する反射係数の青成分.
<i>a</i>	環境光に対する反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5690 行目に定義があります。

呼び出し関係図:



8.37.3.6 loadAmbientAndDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GgVector & color,
    GLint first = 0,
    GLsizei count = 1) const
```

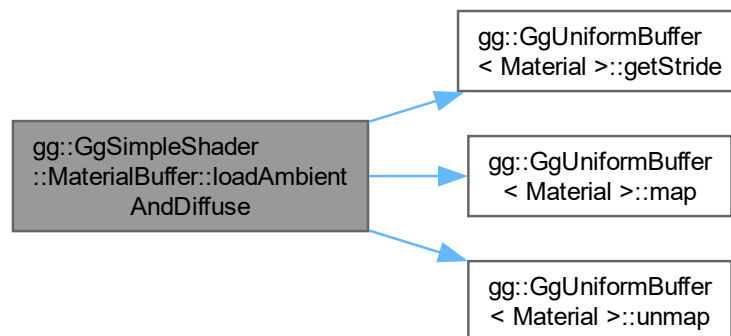
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5820 行目に定義があります。

呼び出し関係図:



8.37.3.7 loadAmbientAndDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GLfloat * color,
    GLint first = 0,
    GLsizei count = 1) const
```

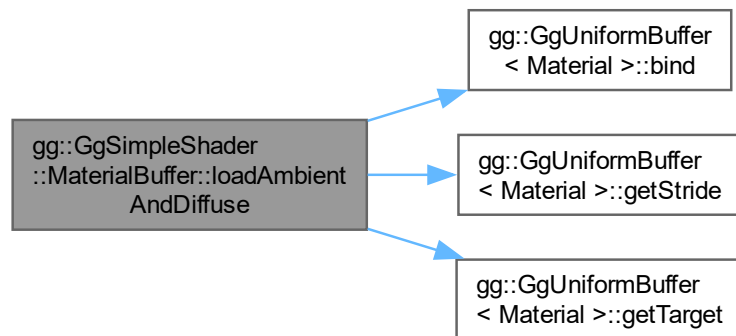
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5843 行目に定義があります。

呼び出し関係図:



8.37.3.8 loadAmbientAndDiffuse() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

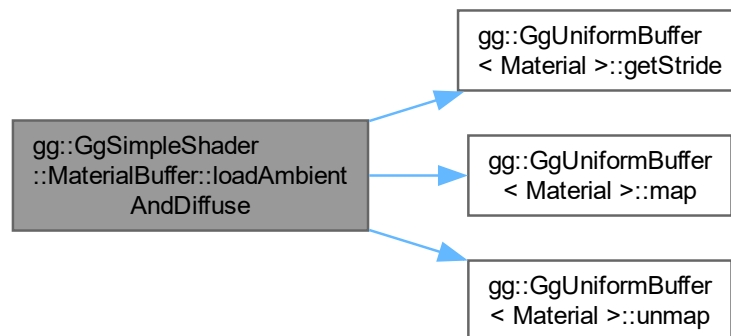
環境光に対する反射係数と拡散反射係数を設定する.

引数

<i>r</i>	環境光に対する反射係数と拡散反射係数の赤成分.
<i>g</i>	環境光に対する反射係数と拡散反射係数の緑成分.
<i>b</i>	環境光に対する反射係数と拡散反射係数の青成分.
<i>a</i>	環境光に対する反射係数と拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5794 行目に定義があります。

呼び出し関係図:



8.37.3.9 loadDiffuse() [1/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
  
```

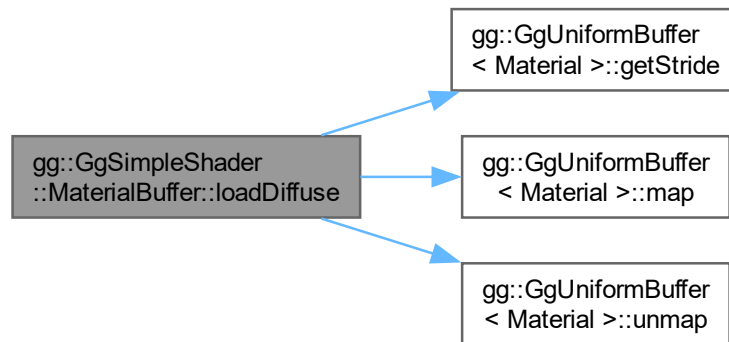
三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する。

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5768 行目に定義があります。

呼び出し関係図:



8.37.3.10 loadDiffuse() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

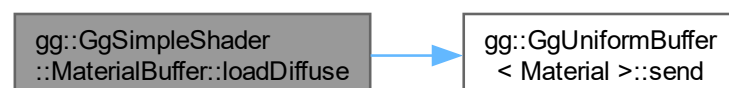
拡散反射係数を設定する.

引数

<i>diffuse</i>	拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7877 行目に定義があります。

呼び出し関係図:



8.37.3.11 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

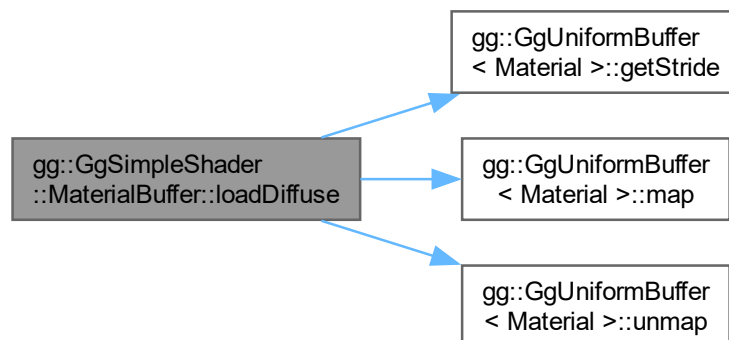
拡散反射係数を設定する。

引数

<i>r</i>	拡散反射係数の赤成分.
<i>g</i>	拡散反射係数の緑成分.
<i>b</i>	拡散反射係数の青成分.
<i>a</i>	拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5742 行目に定義があります。

呼び出し関係図:



8.37.3.12 loadShininess() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    const GLfloat * shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

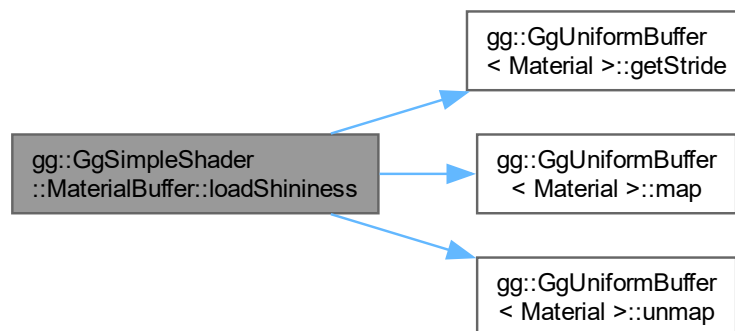
輝き係数を設定する。

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5956 行目に定義があります。

呼び出し関係図:



8.37.3.13 loadShininess() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    GLfloat shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

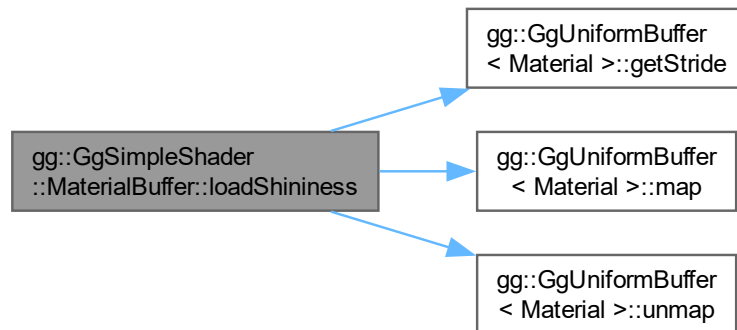
輝き係数を設定する.

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5935 行目に定義があります。

呼び出し関係図:



8.37.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

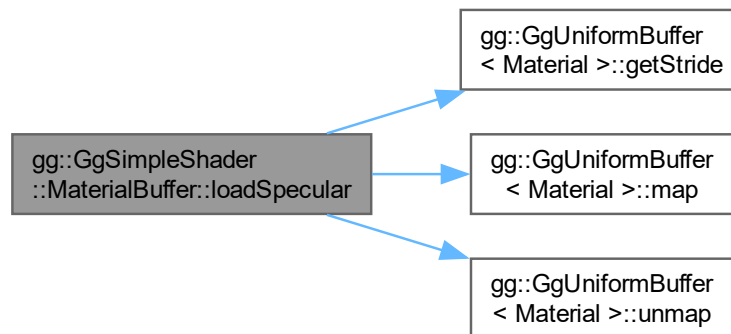
三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する。

引数

<i>specular</i>	鏡面反射係数を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5912 行目に定義があります。

呼び出し関係図:



8.37.3.15 loadSpecular() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

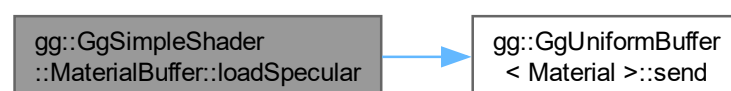
鏡面反射係数を設定する.

引数

<i>specular</i>	鏡面反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 7947 行目に定義があります。

呼び出し関係図:



8.37.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

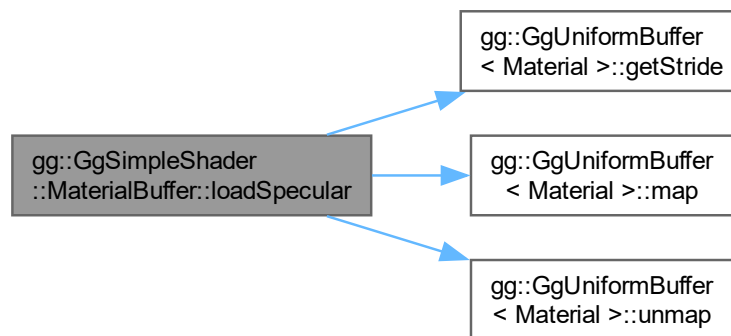
鏡面反射係数を設定する。

引数

<i>r</i>	鏡面反射係数の赤成分.
<i>g</i>	鏡面反射係数の緑成分.
<i>b</i>	鏡面反射係数の青成分.
<i>a</i>	鏡面反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5886 行目に定義があります。

呼び出し関係図:



8.37.3.17 select()

```
void gg::GgSimpleShader::MaterialBuffer::select (
    GLint i = 0) const [inline]
```

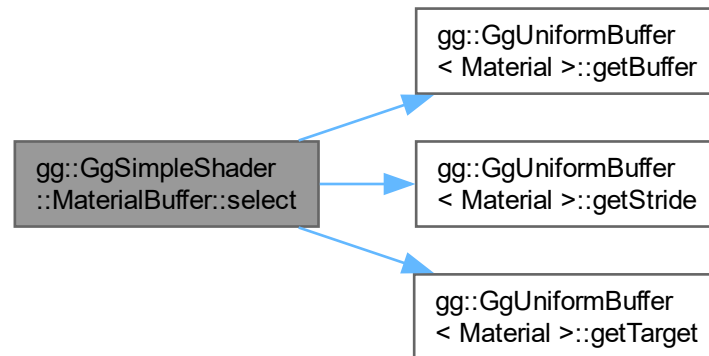
材質を選択する。

引数

i	材質データの uniform block のインデックス.
---	-------------------------------

gg.h の 8000 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.38 Menu クラス

```
#include <Menu.h>
```

公開メンバ関数

- [Menu](#) ([Config](#) &config, [Capture](#) &capture, [Calibration](#) &calibration)
- [Menu](#) (const [Menu](#) &menu)=delete
- virtual [~Menu](#) ()
- [Menu](#) & [operator=](#) (const [Menu](#) &menu)=delete
- [operator bool](#) () const
- const auto & [getPose](#) () const
- auto [getMenubarHeight](#) () const
- void [initializeInputIntrinsics](#) (const std::array< int, 2 > &size)
- const auto & [getCheckerSize](#) () const
- const auto & [getCheckerLength](#) () const
- auto [getMarkerLength](#) () const
- std::array< GLsizei, 2 > [setup](#) (GLfloat aspect) const
- void [draw](#) ()
- void [saveImage](#) (const cv::Mat &image, const std::string &filename="*.jpg") const

公開変数類

- bool `prioritizeLatency` { false }
レイテンシを優先するなら *true*
- bool `detectMarker` { false }
ArUco Marker を検出するなら *true*
- bool `detectBoard` { false }
ChArUco Board を検出するなら *true*

8.38.1 詳解

メニューの描画

`Menu.h` の 23 行目に定義があります。

8.38.2 構築子と解体子

8.38.2.1 `Menu()` [1/2]

```
Menu::Menu (
    Config & config,
    Capture & capture,
    Calibration & calibration)
```

コンストラクタ

引数

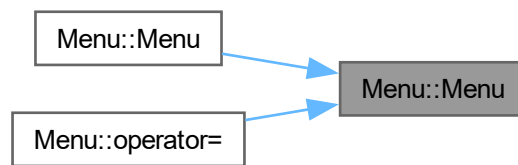
<code>config</code>	構成データ
<code>capture</code>	入力フレームを取得するキャプチャデバイス
<code>calibration</code>	較正オブジェクト

`Menu.cpp` の 447 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.38.2.2 Menu() [2/2]

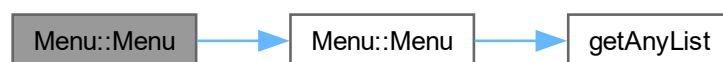
```
Menu::Menu (  
    const Menu & menu) [delete]
```

コピーコンストラクタは使用しない

引数

<i>menu</i>	コピー元
-------------	------

呼び出し関係図:



8.38.2.3 ~Menu()

```
Menu::~Menu () [virtual]
```

デストラクタ

[Menu.cpp](#) の 501 行目に定義があります。

8.38.3 関数詳解

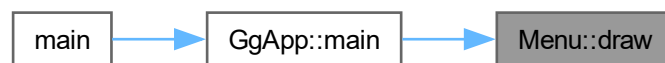
8.38.3.1 draw()

```
void Menu::draw ()
```

メニューを描画する

[Menu.cpp](#) の 1169 行目に定義があります。

被呼び出し関係図:



8.38.3.2 getCheckerLength()

```
const auto & Menu::getCheckerLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さ

[Menu.h](#) の 353 行目に定義があります。

8.38.3.3 getCheckerSize()

```
const auto & Menu::getCheckerSize () const [inline]
```

検出する ChArUco Board のマス目の横と縦の数を得る

戻り値

検出する ChArUco Board のマス目の横と縦の数

[Menu.h](#) の 343 行目に定義があります。

8.38.3.4 getMarkerLength()

```
auto Menu::getMarkerLength () const [inline]
```

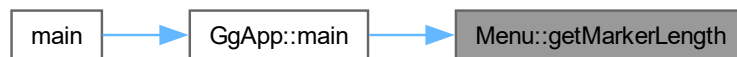
検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Menu.h](#) の 363 行目に定義があります。

被呼び出し関係図:



8.38.3.5 getMenubarHeight()

```
auto Menu::getMenubarHeight () const [inline]
```

メニューバーの高さを得る

戻り値

メニューバーの高さ

[Menu.h](#) の 323 行目に定義があります。

被呼び出し関係図:



8.38.3.6 getPose()

```
const auto & Menu::getPose () const [inline]
```

キャプチャデバイスの姿勢を得る

戻り値

図形の姿勢

[Menu.h](#) の 313 行目に定義があります。

8.38.3.7 initializeInputIntrinsics()

```
void Menu::initializeInputIntrinsics (
    const std::array< int, 2 > & size)
```

入力画像に合わせて内部パラメータを初期化する

引数

<i>size</i>	開かれた入力フレームの解像度
-------------	----------------

実解像度を反映し、現在の焦点距離から画像全体が見やすい初期画角を計算する。中心位置は投影方式の設定値を維持する。

[Menu.cpp](#) の 516 行目に定義があります。

被呼び出し関係図:



8.38.3.8 operator bool()

```
Menu::operator bool () const [inline], [explicit]
```

処理を継続するかどうか調べる

戻り値

処理を継続するなら true

[Menu.h](#) の 303 行目に定義があります。

8.38.3.9 operator=()

```
Menu & Menu::operator= (
    const Menu & menu) [delete]
```

代入演算子は使用しない

引数

<i>menu</i>	代入元のメニュー
-------------	----------

戻り値

代入後のこのメニューの参照

呼び出し関係図:



8.38.3.10 saveImage()

```
void Menu::saveImage (
    const cv::Mat & image,
    const std::string & filename = "*.jpg") const
```

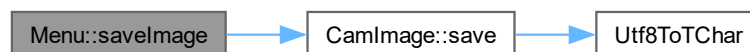
画像の保存

引数

<i>image</i>	保存する画像データ
<i>filename</i>	保存する画像ファイル名のテンプレート

[Menu.cpp](#) の 1188 行目に定義があります。

呼び出し関係図:



8.38.3.11 setup()

```
std::array< GLsizei, 2 > Menu::setup (
    GLfloat aspect) const
```

シェーダを設定する

引数

<i>aspect</i>	表示領域の縦横比
---------------	----------

戻り値

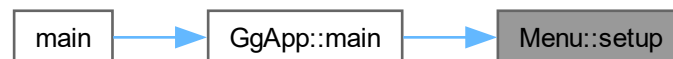
描画すべきメッシュの横と縦の格子点数

覚え書き

格子点数は画角 *aspect* と展開用メッシュのサンプル点数 *samples* から求める。

[Menu.cpp](#) の 607 行目に定義があります。

被呼び出し関係図:



8.38.4 メンバ詳解

8.38.4.1 detectBoard

```
bool Menu::detectBoard { false }
```

ChArUco Board を検出するなら true

[Menu.h](#) の 267 行目に定義があります。

8.38.4.2 detectMarker

```
bool Menu::detectMarker { false }
```

ArUco Marker を検出するなら true

[Menu.h](#) の 264 行目に定義があります。

8.38.4.3 prioritizeLatency

```
bool Menu::prioritizeLatency { false }
```

レイテンシを優先するなら true

[Menu.h](#) の 261 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Menu.h](#)
- [Menu.cpp](#)

8.39 Mesh クラス

```
#include <Mesh.h>
```

公開メンバ関数

- [Mesh \(\)](#)
- [Mesh \(const Mesh &mesh\)=delete](#)
- [virtual ~Mesh \(\)](#)
- [Mesh & operator= \(const Mesh &mesh\)=delete](#)
- [void draw \(const std::array< GLfloat, 2 > &scale, GLint unit=0\) const](#)
- [void drawMesh \(const std::array< GLsizei, 2 > &size\) const](#)

8.39.1 詳解

メッシュの描画クラス

[Mesh.h](#) の 20 行目に定義があります。

8.39.2 構築子と解体子

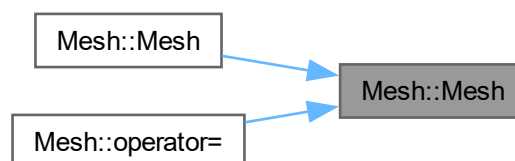
8.39.2.1 Mesh() [1/2]

```
Mesh::Mesh () [inline]
```

コンストラクタ

[Mesh.h](#) の 39 行目に定義があります。

被呼び出し関係図:



8.39.2.2 Mesh() [2/2]

```
Mesh::Mesh (
    const Mesh & mesh) [delete]
```

コピーコンストラクタは使用しない呼び出し関係図:



8.39.2.3 ~Mesh()

```
virtual Mesh::~Mesh () [inline], [virtual]
```

デストラクタ

[Mesh.h](#) の 60 行目に定義があります。

8.39.3 関数詳解

8.39.3.1 draw()

```
void Mesh::draw (
    const std::array< GLfloat, 2 > & scale,
    GLint unit = 0) const [inline]
```

シェーダを指定して矩形を描画する

引数

<i>scale</i>	マッピングするテクスチャのスケール
<i>unit</i>	使用するテクスチャユニット

覚え書き

この前にテクスチャユニット `GL_TEXTURE0 + unit` を指定し、テクスチャを結合しておく必要がある。

[Mesh.h](#) の 84 行目に定義があります。

8.39.3.2 drawMesh()

```
void Mesh::drawMesh (
    const std::array< GLsizei, 2 > & size) const [inline]
```

メッシュを描画する

引数

<i>size</i>	描画するメッシュの横と縦の格子点数
-------------	-------------------

覚え書き

この前に別途シェードを指定し、メッシュの格子点数を得ておく必要がある。

[Mesh.h](#) の 116 行目に定義があります。

8.39.3.3 operator=()

```
Mesh & Mesh::operator= (
    const Mesh & mesh) [delete]
```

代入演算子は使用しない

引数

<i>mesh</i>	代入元のオブジェクト
-------------	------------

戻り値

代入後のこのオブジェクトの参照

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Mesh.h](#)

8.40 Preference クラス

```
#include <Preference.h>
```

公開メンバ関数

- [Preference](#) ()
- [Preference](#) (const std::string &description, const std::string &vert, const std::string &frag, const [Intrinsics](#) &intrinsics=[Intrinsics](#){})
- [Preference](#) (const picojson::object &object)
- virtual ~[Preference](#) ()
- void [buildShader](#) ()
- const auto & [getDescription](#) () const
- const auto & [getIntrinsics](#) () const
- const auto & [getShader](#) () const
- void [setPreference](#) (picojson::object &object) const

8.40.1 詳解

キャプチャデバイスの構成データ

キャプチャデバイスごとの特性を記述する. キャプチャデバイス固有のパラメータはカメラの内部パラメータなどである. キャプチャデバイスの投影方式は平面展開用のシェードとして記述する.

[Preference.h](#) の 24 行目に定義があります。

8.40.2 構築子と解体子

8.40.2.1 [Preference\(\)](#) [1/3]

```
Preference::Preference ()
```

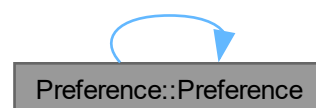
キャプチャデバイスの構成データのデフォルトコンストラクタ

覚え書き

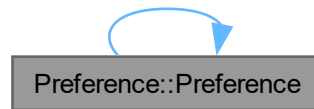
構成ファイルが読めなかったときにしか使わない.

[Preference.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.40.2.2 Preference() [2/3]

```

Preference::Preference (
    const std::string & description,
    const std::string & vert,
    const std::string & frag,
    const Intrinsic & intrinsics = Intrinsic{})
  
```

シェーダのソースファイルを指定するときに使う キャプチャデバイスの構成データのコンストラクタ

引数

<i>description</i>	このキャプチャデバイスの説明の文字列
<i>vert</i>	展開用のバーテックスシェーダのソースファイル名
<i>frag</i>	展開用のフラグメントシェーダのソースファイル名
<i>intrinsics</i>	キャプチャデバイス固有のパラメータ

[Preference.cpp](#) の 22 行目に定義があります。

8.40.2.3 Preference() [3/3]

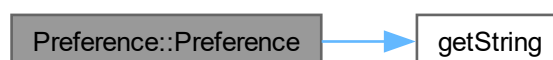
```

Preference::Preference (
    const picojson::object & object)
  
```

構成ファイルを読み込むときに使う キャプチャデバイスの構成データのコンストラクタ

[Preference.cpp](#) の 35 行目に定義があります。

呼び出し関係図:



8.40.2.4 ~Preference()

```
Preference::~~Preference () [virtual]
```

デストラクタ

[Preference.cpp](#) の 48 行目に定義があります。

8.40.3 関数詳解

8.40.3.1 buildShader()

```
void Preference::buildShader ()
```

展開用のシェーダをビルドする

[Preference.cpp](#) の 57 行目に定義があります。

8.40.3.2 getDescription()

```
const auto & Preference::getDescription () const [inline]
```

説明の文字列を取り出す

戻り値

構成の説明文

[Preference.h](#) の 84 行目に定義があります。

8.40.3.3 getIntrinsics()

```
const auto & Preference::getIntrinsics () const [inline]
```

キャプチャデバイス固有のパラメータを取り出す

戻り値

キャプチャデバイス固有のパラメータ

[Preference.h](#) の 94 行目に定義があります。

8.40.3.4 getShader()

```
const auto & Preference::getShader () const [inline]
```

シェーダを取り出す

戻り値

この構成のシェーダの参照

[Preference.h](#) の 104 行目に定義があります。

8.40.3.5 setPreference()

```
void Preference::setPreference (  
    picojson::object & object) const
```

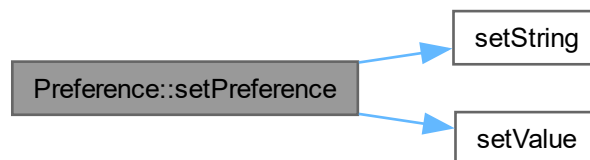
JSON オブジェクトに構成を格納する

引数

<i>object</i>	格納先の JSON オブジェクト
---------------	------------------

[Preference.cpp](#) の 69 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Preference.h](#)
- [Preference.cpp](#)

8.41 Settings 構造体

```
#include <Config.h>
```

公開メンバ関数

- auto [getFocal](#) () const

公開変数類

- int [samples](#) { 57600 }
展開に用いるメッシュのサンプル数
- std::array< float, 3 > [euler](#) { [defaultEuler](#) }
キャプチャデバイスの姿勢のオイラー角
- float [focal](#) { [defaultFocal](#) }
描画時の焦点距離
- std::array< float, 2 > [focalRange](#) { [defaultFocalRange](#) }
描画時の焦点距離の範囲
- std::string [dictionaryName](#) { "DICT_4X4_50" }
使用中の *ArUco Marker* 辞書名
- std::array< int, 2 > [checkerSize](#) { [defaultCheckerSize](#) }
検出する *ChArUco Board* のマス目の横と縦の数
- std::array< float, 2 > [checkerLength](#) { [defaultCheckerLength](#) }
検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 *cm*)
- float [markerLength](#) { [defaultMarkerLength](#) }
検出する *ArUco Marker* の一辺の長さ (単位 *cm*)

静的公開変数類

- `static constexpr decltype(euler) defaultEuler { 0.0f, 0.0f, 0.0f }`
キャプチャデバイスの姿勢のデフォルト値
- `static constexpr decltype(focal) defaultFocal { 50.0f }`
展開時の焦点距離のデフォルト値
- `static constexpr decltype(focalRange) defaultFocalRange { 10.0f, 200.0f }`
描画時の焦点距離の範囲のデフォルト値
- `static constexpr decltype(checkerSize) defaultCheckerSize { 10, 7 }`
検出する *ChArUco Board* のマス目の横と縦の数のデフォルト値
- `static constexpr decltype(checkerLength) defaultCheckerLength { 4.0f, 2.0f }`
検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さのデフォルト値
- `static constexpr decltype(markerLength) defaultMarkerLength { 5.0f }`
検出する *ArUco Marker* の一辺の長さのデフォルト値

8.41.1 詳解

表示関連の設定データ

[Config.h](#) の 17 行目に定義があります。

8.41.2 関数詳解

8.41.2.1 getFocal()

```
auto Settings::getFocal () const [inline]
```

正規化デバイス座標系における焦点距離を求める

[Config.h](#) の 64 行目に定義があります。

8.41.3 メンバ詳解

8.41.3.1 checkerLength

```
std::array<float, 2> Settings::checkerLength { defaultCheckerLength }
```

検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 cm)

[Config.h](#) の 50 行目に定義があります。

8.41.3.2 checkerSize

```
std::array<int, 2> Settings::checkerSize { defaultCheckerSize }
```

検出する *ChArUco Board* のマス目の横と縦の数

[Config.h](#) の 44 行目に定義があります。

8.41.3.3 defaultCheckerLength

```
decltype(checkerLength) Settings::defaultCheckerLength { 4.0f, 2.0f } [static], [constexpr]
```

検出する ChArUco Board のマス目一辺の長さと ArUco Marker の一辺の長さのデフォルト値

[Config.h](#) の 53 行目に定義があります。

8.41.3.4 defaultCheckerSize

```
decltype(checkerSize) Settings::defaultCheckerSize { 10, 7 } [static], [constexpr]
```

検出する ChArUco Board のマス目の横と縦の数のデフォルト値

[Config.h](#) の 47 行目に定義があります。

8.41.3.5 defaultEuler

```
decltype(euler) Settings::defaultEuler { 0.0f, 0.0f, 0.0f } [static], [constexpr]
```

キャプチャデバイスの姿勢のデフォルト値

[Config.h](#) の 26 行目に定義があります。

8.41.3.6 defaultFocal

```
decltype(focal) Settings::defaultFocal { 50.0f } [static], [constexpr]
```

展開時の焦点距離のデフォルト値

[Config.h](#) の 32 行目に定義があります。

8.41.3.7 defaultFocalRange

```
decltype(focalRange) Settings::defaultFocalRange { 10.0f, 200.0f } [static], [constexpr]
```

描画時の焦点距離の範囲のデフォルト値

[Config.h](#) の 38 行目に定義があります。

8.41.3.8 defaultMarkerLength

```
decltype(markerLength) Settings::defaultMarkerLength { 5.0f } [static], [constexpr]
```

検出する ArUco Marker の一辺の長さのデフォルト値

[Config.h](#) の 59 行目に定義があります。

8.41.3.9 dictionaryName

```
std::string Settings::dictionaryName { "DICT_4X4_50" }
```

使用中の ArUco Marker 辞書名

[Config.h](#) の 41 行目に定義があります。

8.41.3.10 euler

```
std::array<float, 3> Settings::euler { defaultEuler }
```

キャプチャデバイスの姿勢のオイラー角

[Config.h](#) の 23 行目に定義があります。

8.41.3.11 focal

```
float Settings::focal { defaultFocal }
```

描画時の焦点距離

[Config.h](#) の 29 行目に定義があります。

8.41.3.12 focalRange

```
std::array<float, 2> Settings::focalRange { defaultFocalRange }
```

描画時の焦点距離の範囲

[Config.h](#) の 35 行目に定義があります。

8.41.3.13 markerLength

```
float Settings::markerLength { defaultMarkerLength }
```

検出する ArUco Marker の一辺の長さ (単位 cm)

[Config.h](#) の 56 行目に定義があります。

8.41.3.14 samples

```
int Settings::samples { 57600 }
```

展開に用いるメッシュのサンプル数

[Config.h](#) の 20 行目に定義があります。

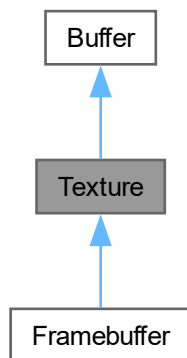
この構造体詳解は次のファイルから抽出されました:

- [Config.h](#)

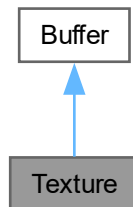
8.42 Texture クラス

```
#include <Texture.h>
```

Texture の継承関係図



Texture 連携図



公開メンバ関数

- `Texture ()`=default
- `Texture (GLsizei width, GLsizei height, int channels)`
- `Texture (const Texture &texture)`
- `Texture (Texture &&texture) noexcept`
- `virtual ~Texture ()`
- `Texture & operator= (const Texture &texture)`
- `Texture & operator= (Texture &&texture) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &texture) noexcept`
- `virtual void discard ()`

- auto `getTextureName` () const
- virtual const std::array< GLsizei, 2 > & `getSize` () const
- virtual int `getChannels` () const
- void `bindTexture` (int unit=0) const
- void `unbindTexture` () const
- void `draw` (GLsizei width, GLsizei height, int unit=0) const
- void `readPixels` (GLuint buffer) const
- void `readPixels` (`Buffer` &buffer) const
- void `readPixels` () const
- void `drawPixels` (GLuint buffer) const
- void `drawPixels` (const `Texture` &texture)
- void `drawPixels` (const `Buffer` &buffer)
- void `drawPixels` () const
- void `drawPixels` (GLsizei width, GLsizei height, int channels, const void *pixels)

基底クラス `Buffer` に属する継承公開メンバ関数

- `Buffer` ()=default
- `Buffer` (GLsizei width, GLsizei height, int channels)
- `Buffer` (const `Buffer` &buffer)
- `Buffer` (`Buffer` &&buffer) noexcept
- virtual ~`Buffer` ()
- `Buffer` & `operator=` (const `Buffer` &buffer)
- `Buffer` & `operator=` (`Buffer` &&buffer) noexcept
- void `resize` (GLsizei width, GLsizei height, int channels)
- void `resize` (const `Buffer` &buffer)
- auto `getBufferName` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

静的限定公開変数類

- static std::shared_ptr< `Mesh` > `mesh`
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

その他の継承メンバ

基底クラス `Buffer` に属する継承限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const `Buffer` &buffer) noexcept

8.42.1 詳解

テクスチャクラス

[Texture.h](#) の 20 行目に定義があります。

8.42.2 構築子と解体子

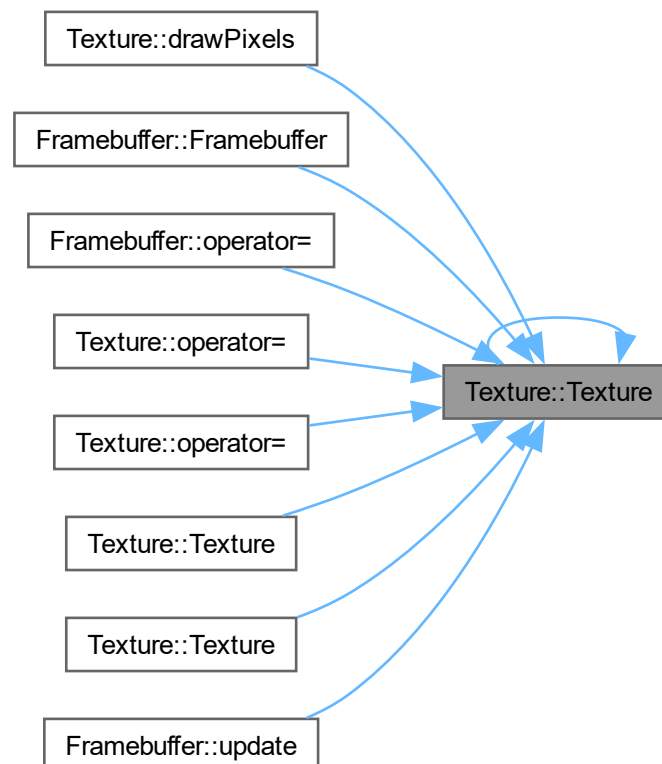
8.42.2.1 Texture() [1/4]

```
Texture::Texture () [default]
```

テクスチャのデフォルトコンストラクタ 呼び出し関係図:



被呼び出し関係図:



8.42.2.2 Texture() [2/4]

```
Texture::Texture (
    GLsizei width,
    GLsizei height,
    int channels)
```

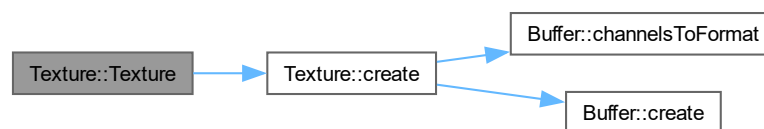
テクスチャを作成するコンストラクタ

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

[Texture.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



8.42.2.3 Texture() [3/4]

```
Texture::Texture (
    const Texture & texture)
```

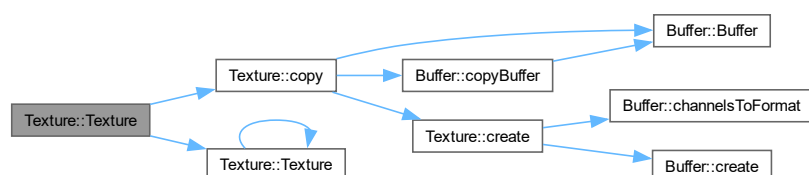
コピーコンストラクタ

引数

<i>texture</i>	コピー元のテクスチャ
----------------	------------

[Texture.cpp](#) の 22 行目に定義があります。

呼び出し関係図:



8.42.2.4 Texture() [4/4]

```
Texture::Texture (  
    Texture && texture) [inline], [noexcept]
```

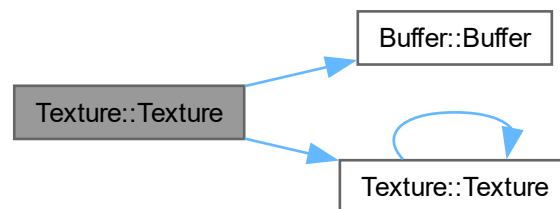
ムーブコンストラクタ

引数

<i>texture</i>	ムーブ元のテクスチャ
----------------	------------

[Texture.h](#) の 64 行目に定義があります。

呼び出し関係図:



8.42.2.5 ~Texture()

```
Texture::~Texture () [virtual]
```

デストラクタ

[Texture.cpp](#) の 31 行目に定義があります。

呼び出し関係図:



8.42.3 関数詳解

8.42.3.1 bindTexture()

```
void Texture::bindTexture (  
    int unit = 0) const [inline]
```

テクスチャユニットを指定してテクスチャを結合する

引数

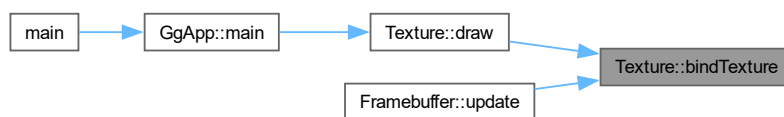
<i>unit</i>	テクスチャユニット番号
-------------	-------------

覚え書き

`unit` にはシェードにおいてテクスチャのサンプラの `uniform` 変数に設定する `GL_TEXTUREi` の `i` と一致させること。

[Texture.h](#) の 165 行目に定義があります。

被呼び出し関係図:



8.42.3.2 copy()

```
void Texture::copy (
    const Buffer & texture) [virtual], [noexcept]
```

テクスチャをコピーする

引数

<i>texture</i>	コピー元のテクスチャ
----------------	------------

覚え書き

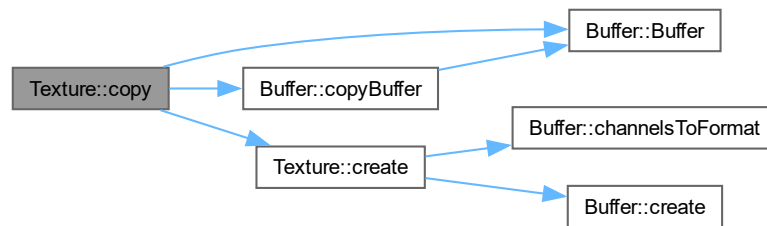
このテクスチャのサイズがコピー元と異なれば、このテクスチャを削除して新しいテクスチャを作り直してコピーする。引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

`Buffer` を再実装しています。

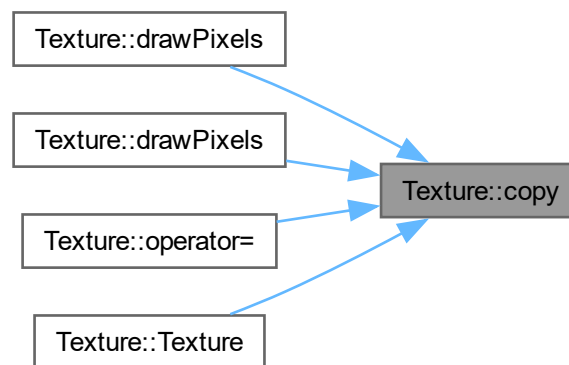
`Framebuffer` で再実装されています。

[Texture.cpp](#) の 121 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.3 create()

```

void Texture::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

テクスチャを作成する

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

覚え書き

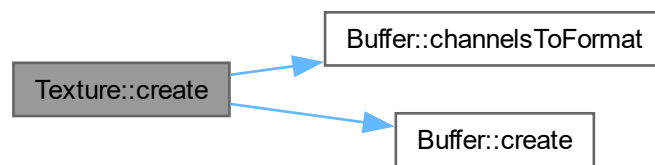
このテクスチャのサイズが引数で指定したサイズと異なれば、このテクスチャを削除して新しいテクスチャを作り直す。

[Buffer](#)を再実装しています。

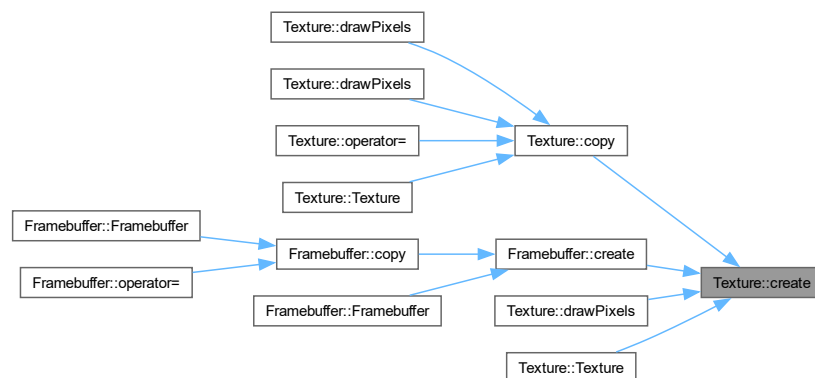
[Framebuffer](#)で再実装されています。

[Texture.cpp](#) の 85 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.4 discard()

```
void Texture::discard () [virtual]
```

テクスチャを破棄する

[Buffer](#)を再実装しています。

[Framebuffer](#)で再実装されています。

[Texture.cpp](#) の 134 行目に定義があります。

被呼び出し関係図:



8.42.3.5 draw()

```
void Texture::draw (  
    GLsizei width,  
    GLsizei height,  
    int unit = 0) const
```

このテクスチャをマッピングして矩形を描画する

引数

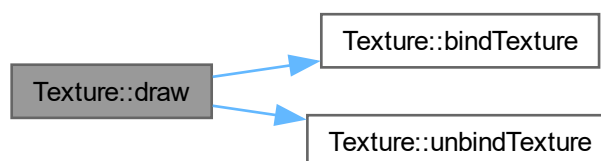
<i>width</i>	表示領域の横の画素数
<i>height</i>	表示領域の縦の画素数
<i>unit</i>	使用するテクスチャユニット番号

覚え書き

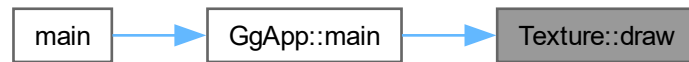
フレームバッファ全体を覆うメッシュを描画する。unit にはシェードにおいてテクスチャのサンブラの uniform 変数に設定する GL_TEXTUREi の i と一致させること。

[Texture.cpp](#) の 151 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



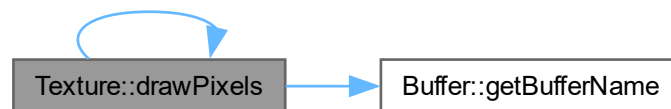
8.42.3.6 drawPixels() [1/5]

```
void Texture::drawPixels () const [inline]
```

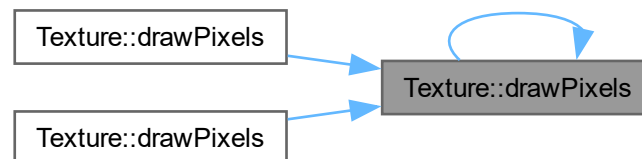
ピクセルバッファオブジェクトからテクスチャにデータをコピーする

[Texture.h](#) の 301 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.7 drawPixels() [2/5]

```
void Texture::drawPixels (
    const Buffer & buffer) [inline]
```

指定したオブジェクトからテクスチャにデータをコピーする

引数

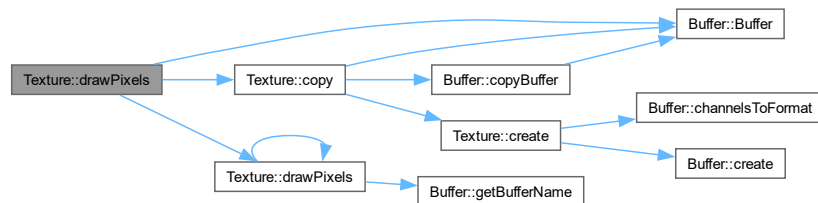
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

テクスチャのサイズをコピー元のオブジェクトのバッファに合わせる。

[Texture.h](#) の 289 行目に定義があります。

呼び出し関係図:



8.42.3.8 drawPixels() [3/5]

```
void Texture::drawPixels (
    const Texture & texture) [inline]
```

指定したテクスチャからからデータをコピーする

引数

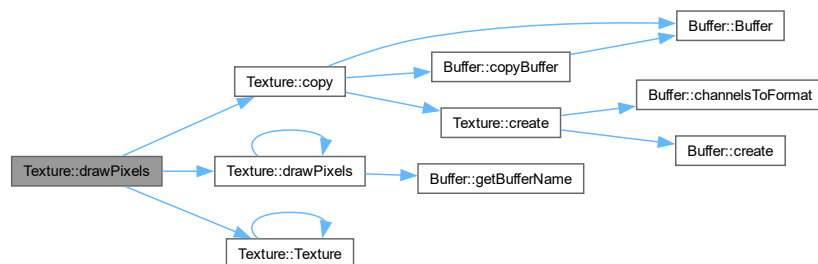
<i>texture</i>	コピー元のテクスチャ
----------------	------------

覚え書き

テクスチャのサイズをコピー元のテクスチャに合わせる。

[Texture.h](#) の 272 行目に定義があります。

呼び出し関係図:



8.42.3.9 drawPixels() [4/5]

```
void Texture::drawPixels (
    GLsizei width,
    GLsizei height,
    int channels,
    const void * pixels)
```

CPU メモリ上の画像をテクスチャへ転送する

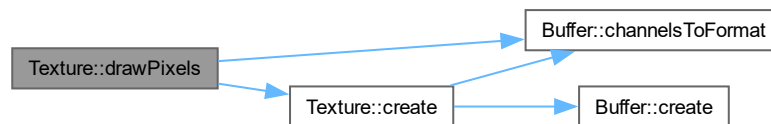
引数

<i>width</i>	画像の幅
<i>height</i>	画像の高さ
<i>channels</i>	画像のチャンネル数
<i>pixels</i>	画像の先頭画素へのポインタ

OpenCV で補正した `cv::Mat` を PBO を介さずGPUへ転送するために使う。

[Texture.cpp](#) の 275 行目に定義があります。

呼び出し関係図:



8.42.3.10 drawPixels() [5/5]

```
void Texture::drawPixels (
    GLuint buffer) const
```

指定したピクセルバッファオブジェクトからテクスチャにデータをコピーする

引数

<i>buffer</i>	コピーするテクスチャを格納したピクセルバッファオブジェクト名
---------------	--------------------------------

覚え書き

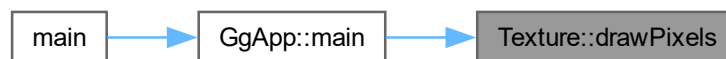
コピーするデータのサイズが一致している必要がある。

`Texture.cpp` の 237 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.11 getChannels()

```
virtual int Texture::getChannels () const [inline], [virtual]
```

テクスチャのチャンネル数を得る

`Buffer` を再実装しています。

`Framebuffer` で再実装されています。

`Texture.h` の 151 行目に定義があります。

8.42.3.12 getSize()

```
virtual const std::array< GLsizei, 2 > & Texture::getSize () const [inline], [virtual]
```

テクスチャのサイズを得る

`Buffer` を再実装しています。

`Framebuffer` で再実装されています。

`Texture.h` の 143 行目に定義があります。

8.42.3.13 `getTextureName()`

```
auto Texture::getTextureName () const [inline]
```

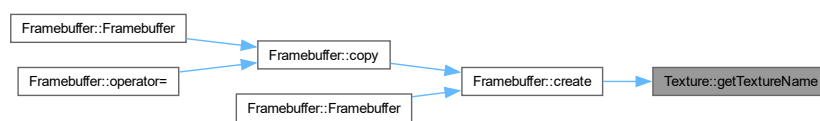
テクスチャ名を得る

戻り値

テクスチャ名

[Texture.h](#) の 135 行目に定義があります。

被呼び出し関係図:

8.42.3.14 `operator=()` [1/2]

```
Texture & Texture::operator= (
    const Texture & texture)
```

代入演算子

引数

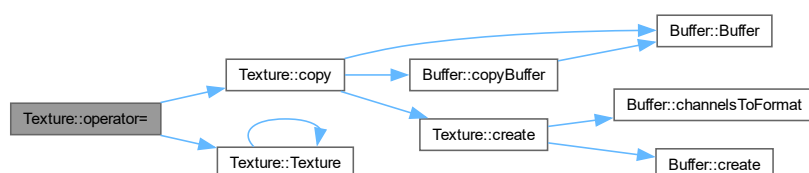
<i>texture</i>	代入元のテクスチャ
----------------	-----------

戻り値

代入結果のテクスチャ

[Texture.cpp](#) の 40 行目に定義があります。

呼び出し関係図:



8.42.3.15 operator=() [2/2]

```
Texture & Texture::operator= (  
    Texture && texture) [noexcept]
```

ムーブ代入演算子

引数

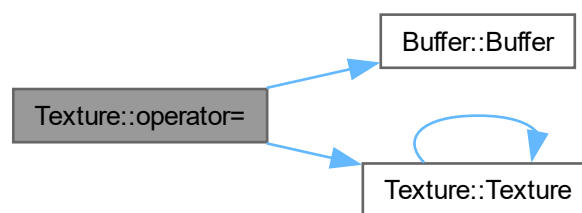
<code>texture</code>	ムーブ代入元のテクスチャ
----------------------	--------------

戻り値

ムーブ代入結果のテクスチャ

[Texture.cpp](#) の 55 行目に定義があります。

呼び出し関係図:



8.42.3.16 readPixels() [1/3]

```
void Texture::readPixels () const [inline]
```

テクスチャからピクセルバッファオブジェクトにデータをコピーする

[Texture.h](#) の 239 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.17 readPixels() [2/3]

```
void Texture::readPixels (
    Buffer & buffer) const [inline]
```

テクスチャから指定したオブジェクトのバッファにデータをコピーする

引数

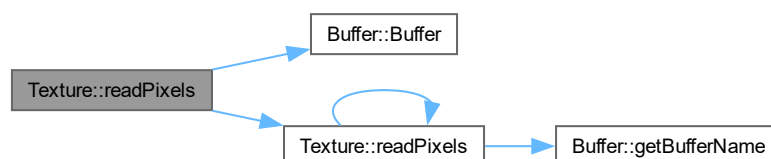
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

コピーする先のオブジェクトのサイズをテクスチャに合わせる。

[Texture.h](#) の 224 行目に定義があります。

呼び出し関係図:



8.42.3.18 readPixels() [3/3]

```
void Texture::readPixels (
    GLuint buffer) const
```

テクスチャから指定したピクセルバッファオブジェクトにデータをコピーする

引数

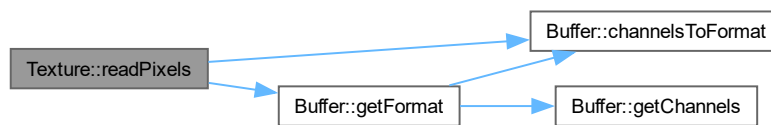
<i>buffer</i>	テクスチャをコピーする先のピクセルバッファオブジェクト名
---------------	------------------------------

覚え書き

コピーするデータのサイズが一致している必要がある。

[Texture.cpp](#) の 178 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



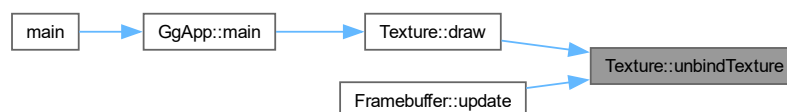
8.42.3.19 unbindTexture()

```
void Texture::unbindTexture () const [inline]
```

テクスチャの結合を解除する

[Texture.h](#) の 177 行目に定義があります。

被呼び出し関係図:



8.42.4 メンバ詳解

8.42.4.1 mesh

```
std::shared_ptr< Mesh > Texture::mesh [static], [protected]
```

テクスチャ展開に用いるメッシュの頂点配列オブジェクト

[Texture.h](#) の 34 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Texture.h](#)
- [Texture.cpp](#)

8.43 GgApp::Window クラス

```
#include <GgApp.h>
```

公開メンバ関数

- [Window](#) (const std::string &title="GLFW Window", int width=640, int height=480, int fullscreen=0, GLFWwindow *share=nullptr)
- [Window](#) (const Window &w)=delete
- [Window](#) (Window &&w)=default
- virtual ~[Window](#) ()
- [Window](#) & operator= (const [Window](#) &w)=delete
- [Window](#) & operator= ([Window](#) &&w)=default
- auto * [get](#) () const
- void [setClose](#) (int flag=GLFW_TRUE) const
- bool [shouldClose](#) () const
- [operator](#) bool ()
- void [swapBuffers](#) () const
- void [restoreViewport](#) () const
- void [updateViewport](#) ()
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- auto [getFboWidth](#) () const
- auto [getFboHeight](#) () const
- const auto & [getSize](#) () const
- void [getSize](#) (GLsizei *size) const
- const auto & [getFboSize](#) () const
- void [getFboSize](#) (GLsizei *fboSize) const
- auto [getAspect](#) () const
- bool [getKey](#) (int key) const
- void [selectInterface](#) (int no)
- void [setVelocity](#) (GLfloat vx, GLfloat vy, GLfloat vz=0.1f)
- int [getLastKey](#) ()
- auto [getArrow](#) (int direction=0, int mods=0) const
- auto [getArrowX](#) (int mods=0) const
- auto [getArrowY](#) (int mods=0) const

- void [getArrow](#) (GLfloat *arrow, int mods=0) const
- auto [getShiftArrowX](#) () const
- auto [getShiftArrowY](#) () const
- void [getShiftArrow](#) (GLfloat *shift_arrow) const
- auto [getControlArrowX](#) () const
- auto [getControlArrowY](#) () const
- void [getControlArrow](#) (GLfloat *control_arrow) const
- auto [getAltArrowX](#) () const
- auto [getAltArrowY](#) () const
- void [getAltArrow](#) (GLfloat *alt_arrow) const
- const auto * [getMouse](#) () const
- void [getMouse](#) (GLfloat *position) const
- auto [getMouse](#) (int direction) const
- auto [getMouseX](#) () const
- auto [getMouseY](#) () const
- const auto * [getWheel](#) () const
- void [getWheel](#) (GLfloat *rotation) const
- auto [getWheel](#) (int direction) const
- auto [getWheelX](#) () const
- auto [getWheelY](#) () const
- const auto & [getTranslation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getTranslationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getScrollMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- void [resetRotation](#) ()
- void [resetTranslation](#) ()
- void [reset](#) ()
- void * [getUserPointer](#) () const
- void [setUserPointer](#) (void *pointer)
- void [setResizeFunc](#) (void(*func)(const [Window](#) *window, int width, int height))
- void [setKeyboardFunc](#) (void(*func)(const [Window](#) *window, int key, int scancode, int action, int mods))
- void [setMouseFunc](#) (void(*func)(const [Window](#) *window, int button, int action, int mods))
- void [setWheelFunc](#) (void(*func)(const [Window](#) *window, double x, double y))

8.43.1 詳解

ウィンドウ関連の処理.

覚え書き

GLFW を使って OpenGL のウィンドウを操作するラッパークラス.

[GgApp.h](#) の 150 行目に定義があります。

8.43.2 構築子と解体子

8.43.2.1 Window() [1/3]

```
GgApp::Window::Window (  
    const std::string & title = "GLFW Window",  
    int width = 640,  
    int height = 480,  
    int fullscreen = 0,  
    GLFWwindow * share = nullptr)
```

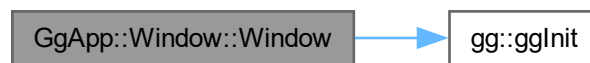
コンストラクタ.

引数

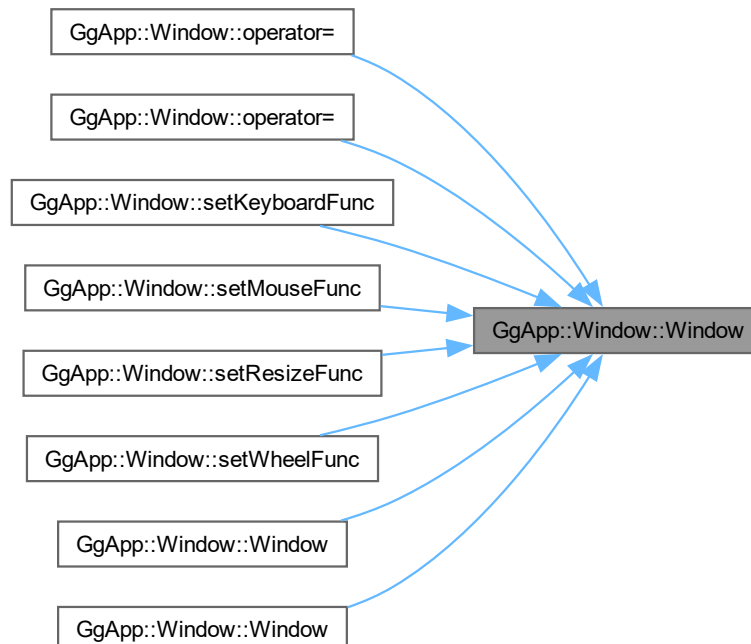
<i>title</i>	ウィンドウタイトルの文字列.
<i>width</i>	開くウィンドウの幅, フルスクリーン時は無視され実際のディスプレイの幅が使われる.
<i>height</i>	開くウィンドウの高さ, フルスクリーン時は無視され実際のディスプレイの高さが使われる.
<i>fullscreen</i>	フルスクリーン表示を行うディスプレイ番号, 0 ならフルスクリーン表示を行わない.
<i>share</i>	共有するコンテキスト, nullptr ならコンテキストを共有しない.

GgApp.cpp の 351 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.2.2 Window() [2/3]

```
GgApp::Window::Window (
    const Window & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のウィンドウ.
----------	-------------

呼び出し関係図:



8.43.2.3 Window() [3/3]

```
GgApp::Window::Window (
    Window && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

呼び出し関係図:



8.43.2.4 ~Window()

```
virtual GgApp::Window::~~Window () [inline], [virtual]
```

デストラクタ.

[GgApp.h](#) の 279 行目に定義があります。

8.43.3 関数詳解

8.43.3.1 get()

```
auto * GgApp::Window::get () const [inline]
```

ウィンドウの識別子のポインタを取得する.

戻り値

GLFWwindow 型のウィンドウ識別子のポインタ.

[GgApp.h](#) の 309 行目に定義があります。

8.43.3.2 getAltArrowX()

```
auto GgApp::Window::getAltArrowX () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 633 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.3 getAltArrowY()

```
auto GgApp::Window::getAltArrowY () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 643 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.4 getAltArrow()

```
void GgApp::Window::getAltArrow (
    GLfloat * alt_arrow) const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<code>alt_arrow</code>	ALT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
------------------------	---

[GgApp.h](#) の 653 行目に定義があります。

呼び出し関係図:



8.43.3.5 getArrow() [1/2]

```
void GgApp::Window::getArrow (
    GLfloat * arrow,
    int mods = 0) const [inline]
```

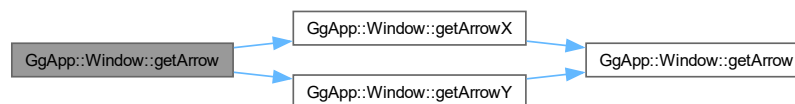
矢印キーの現在の値を得る.

引数

<i>arrow</i>	矢印キーの値を格納する GLfloat[2] の配列.
<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).

[GgApp.h](#) の 560 行目に定義があります。

呼び出し関係図:



8.43.3.6 getArrow() [2/2]

```
auto GgApp::Window::getArrow (
    int direction = 0,
    int mods = 0) const [inline]
```

矢印キーの現在の値を得る.

引数

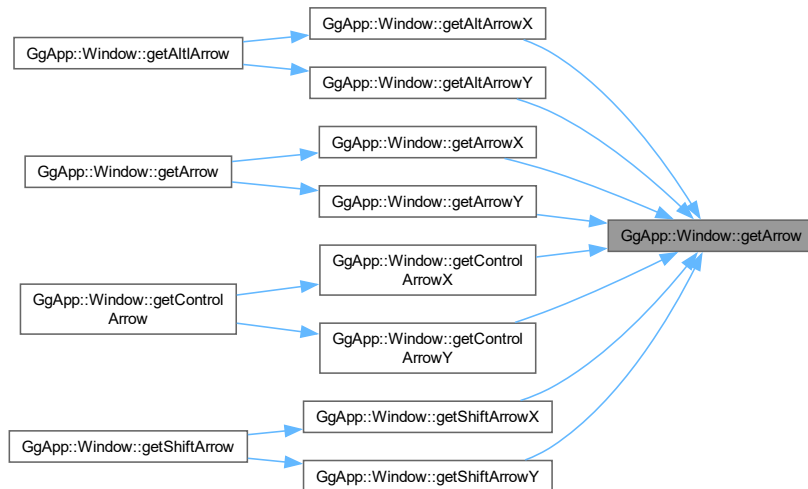
<i>direction</i>	方向 (0: X, 1:Y).
<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).

戻り値

矢印キーの値.

[GgApp.h](#) の 526 行目に定義があります。

被呼び出し関係図:



8.43.3.7 getArrowX()

```
auto GgApp::Window::getArrowX (
    int mods = 0) const [inline]
```

矢印キーの現在の X 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの X 値.

[GgApp.h](#) の 538 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.8 getArrowY()

```
auto GgApp::Window::getArrowY (  
    int mods = 0) const [inline]
```

矢印キーの現在の Y 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの Y 値.

[GgApp.h](#) の [549](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.9 getAspect()

```
auto GgApp::Window::getAspect () const [inline]
```

ウィンドウの縦横比を得る.

戻り値

ウィンドウの縦横比.

[GgApp.h](#) の 463 行目に定義があります。

8.43.3.10 getControlArrow()

```
void GgApp::Window::getControlArrow (
    GLfloat * control_arrow) const [inline]
```

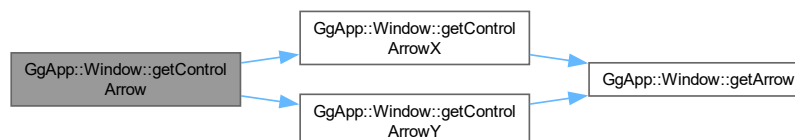
CTRL キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>control_arrow</i>	CTRL キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
----------------------	--

[GgApp.h](#) の 622 行目に定義があります。

呼び出し関係図:



8.43.3.11 getControlArrowX()

```
auto GgApp::Window::getControlArrowX () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

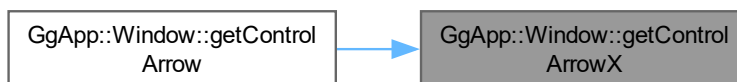
CTRL キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 602 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.12 getControlArrowY()

```
auto GgApp::Window::getControlArrowY () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

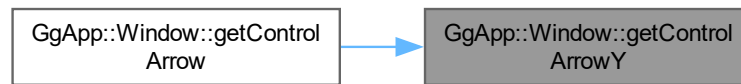
CTRL キーを押しながら矢印キーを押したときの現在の Y 値.

GgApp.h の 612 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.13 getFboHeight()

```
auto GgApp::Window::getFboHeight () const [inline]
```

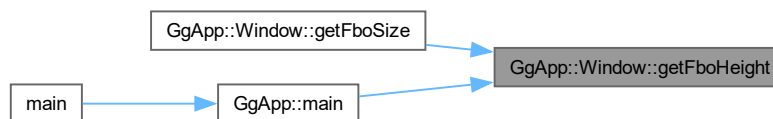
FBO の高さを得る.

戻り値

FBO の高さ.

[GgApp.h](#) の 411 行目に定義があります。

被呼び出し関係図:



8.43.3.14 getFboSize() [1/2]

```
const auto & GgApp::Window::getFboSize () const [inline]
```

FBO のサイズを得る.

戻り値

FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.

[GgApp.h](#) の 442 行目に定義があります。

8.43.3.15 getFboSize() [2/2]

```
void GgApp::Window::getFboSize (
    GLsizei * fboSize) const [inline]
```

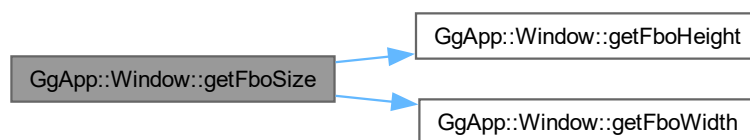
FBO のサイズを得る.

引数

<i>fboSize</i>	FBO の幅と高さを格納する GLsizei 型の 2 要素の配列.
----------------	------------------------------------

GgApp.h の 452 行目に定義があります。

呼び出し関係図:



8.43.3.16 getFboWidth()

```
auto GgApp::Window::getFboWidth () const [inline]
```

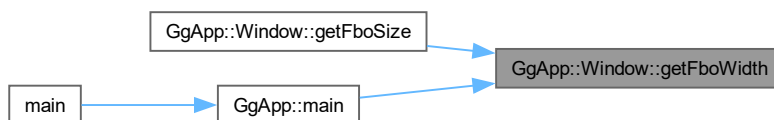
FBO の横幅を得る.

戻り値

FBO の横幅.

GgApp.h の 401 行目に定義があります。

被呼び出し関係図:



8.43.3.17 getHeight()

```
auto GgApp::Window::getHeight () const [inline]
```

ウィンドウの高さを得る.

戻り値

ウィンドウの高さ.

[GgApp.h](#) の 391 行目に定義があります。

被呼び出し関係図:



8.43.3.18 getKey()

```
bool GgApp::Window::getKey (
    int key) const [inline]
```

キーが押されているかどうかを判定する.

戻り値

キーが押されていれば true.

[GgApp.h](#) の 473 行目に定義があります。

8.43.3.19 getLastKey()

```
int GgApp::Window::getLastKey () [inline]
```

最後にタイプしたキーを得る.

戻り値

最後にタイプしたキーの文字.

[GgApp.h](#) の 511 行目に定義があります。

8.43.3.20 getMouse() [1/3]

```
const auto * GgApp::Window::getMouse () const [inline]
```

マウスカーソルの現在位置を得る.

戻り値

マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の 664 行目に定義があります。

8.43.3.21 getMouse() [2/3]

```
void GgApp::Window::getMouse (
    GLfloat * position) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>position</i>	マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.
-----------------	---------------------------------------

[GgApp.h](#) の 675 行目に定義があります。

8.43.3.22 getMouse() [3/3]

```
auto GgApp::Window::getMouse (
    int direction) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスカーソルの現在位置.

[GgApp.h](#) の 688 行目に定義があります。

8.43.3.23 getMouseX()

```
auto GgApp::Window::getMouseX () const [inline]
```

マウスカーソルの現在位置の X 座標を得る.

戻り値

direction 方向のマウスカーソルの X 方向の現在位置.

[GgApp.h](#) の 699 行目に定義があります。

8.43.3.24 getMouseY()

```
auto GgApp::Window::getMouseY () const [inline]
```

マウスカーソルの現在位置の Y 座標を得る.

戻り値

direction 方向のマウスカーソルの Y 方向の現在位置.

[GgApp.h](#) の 710 行目に定義があります。

8.43.3.25 getRotation()

```
auto GgApp::Window::getRotation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う [GgQuaternion](#) 型の四元数.

[GgApp.h](#) の 838 行目に定義があります。

8.43.3.26 getRotationMatrix()

```
auto GgApp::Window::getRotationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う [GgMarix](#) 型の変換行列.

[GgApp.h](#) の 851 行目に定義があります。

8.43.3.27 getScrollMatrix()

```
auto GgApp::Window::getScrollMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによってオブジェクトの平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

クリッピング座標系で平行移動を行う [GgMatrix](#) 型の変換行列.

[GgApp.h](#) の 815 行目に定義があります。

8.43.3.28 getShiftArrow()

```
void GgApp::Window::getShiftArrow (
    GLfloat * shift_arrow) const [inline]
```

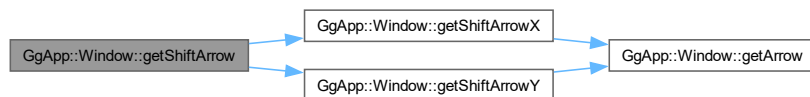
SHIFT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<code>shift_arrow</code>	SHIFT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
--------------------------	---

GgApp.h の 591 行目に定義があります。

呼び出し関係図:



8.43.3.29 getShiftArrowX()

```
auto GgApp::Window::getShiftArrowX () const [inline]
```

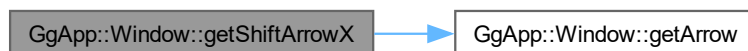
SHIFT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 571 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.30 getShiftArrowY()

```
auto GgApp::Window::getShiftArrowY () const [inline]
```

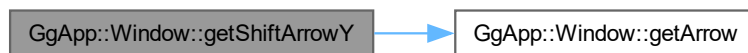
SHIFT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 581 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.31 getSize() [1/2]

```
const auto & GgApp::Window::getSize () const [inline]
```

ウィンドウのサイズを得る.

戻り値

ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列の参照.

[GgApp.h](#) の 421 行目に定義があります。

8.43.3.32 getSize() [2/2]

```
void GgApp::Window::getSize (  
    GLsizei * size) const [inline]
```

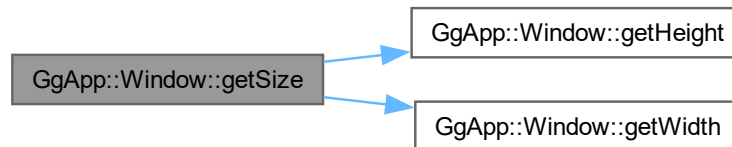
ウィンドウのサイズを得る.

引数

<i>size</i>	ウィンドウの幅と高さを格納した <code>GLsizei</code> 型の 2 要素の配列.
-------------	--

[GgApp.h](#) の 431 行目に定義があります。

呼び出し関係図:



8.43.3.33 getTranslation()

```
const auto & GgApp::Window::getTranslation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボール処理を考慮したマウスによるスクロールの変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (<code>GLFW_MOUSE_BUTTON_1</code>).
---------------	--

戻り値

平行移動量を格納した `GLfloat[3]` の配列のポインタ.

[GgApp.h](#) の 779 行目に定義があります。

8.43.3.34 getTranslationMatrix()

```
auto GgApp::Window::getTranslationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによって視点の平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (<code>GLFW_MOUSE_BUTTON_1</code>).
---------------	--

戻り値

視点座標系で平行移動を行う `GgMatrix` 型の変換行列.

[GgApp.h](#) の 792 行目に定義があります。

8.43.3.35 getUserPointer()

```
void * GgApp::Window::getUserPointer () const [inline]
```

ユーザーポインタを取り出す.

戻り値

保存されているユーザポインタ.

[GgApp.h](#) の [893](#) 行目に定義があります.

8.43.3.36 getWheel() [1/3]

```
const auto * GgApp::Window::getWheel () const [inline]
```

マウスホイールの回転量を得る.

戻り値

マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の [721](#) 行目に定義があります.

8.43.3.37 getWheel() [2/3]

```
void GgApp::Window::getWheel (
    GLfloat * rotation) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>rotation</i>	マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.
-----------------	--------------------------------------

[GgApp.h](#) の [732](#) 行目に定義があります.

8.43.3.38 getWheel() [3/3]

```
auto GgApp::Window::getWheel (
    int direction) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスホイールの回転量.

[GgApp.h](#) の 745 行目に定義があります。

8.43.3.39 `getWheelX()`

```
auto GgApp::Window::getWheelX () const [inline]
```

マウスホイールの X 方向の回転量を得る.

戻り値

マウスホイールの X 方向の回転量.

[GgApp.h](#) の 756 行目に定義があります。

8.43.3.40 `getWheelY()`

```
auto GgApp::Window::getWheelY () const [inline]
```

マウスホイールの Y 方向の回転量を得る.

戻り値

マウスホイールの Y 方向の回転量.

[GgApp.h](#) の 767 行目に定義があります。

8.43.3.41 `getWidth()`

```
auto GgApp::Window::getWidth () const [inline]
```

ウィンドウの横幅を得る.

戻り値

ウィンドウの横幅.

[GgApp.h](#) の 381 行目に定義があります。

被呼び出し関係図:



8.43.3.42 operator bool()

```
GgApp::Window::operator bool () [explicit]
```

イベントを取得してループを継続すべきかどうか調べる。

戻り値

ループを継続すべきなら true.

[GgApp.cpp](#) の 431 行目に定義があります。

呼び出し関係図:



8.43.3.43 operator=() [1/2]

```
Window & GgApp::Window::operator= (
    const Window & w) [delete]
```

代入演算子は使用しない。

引数

<i>w</i>	代入元のウィンドウ.
----------	------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.43.3.44 operator=() [2/2]

```
Window & GgApp::Window::operator= (
    Window && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



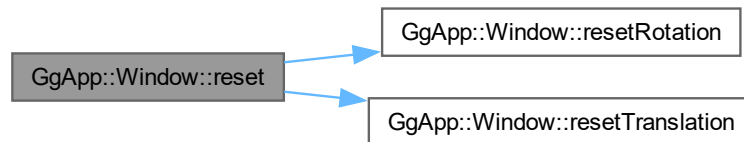
8.43.3.45 reset()

```
void GgApp::Window::reset () [inline]
```

トラックボール・マウスホイール・矢印キーの値を初期化する.

[GgApp.h](#) の 879 行目に定義があります。

呼び出し関係図:



8.43.3.46 resetRotation()

```
void GgApp::Window::resetRotation () [inline]
```

トラックボール処理を初期化する.

[GgApp.h](#) の 861 行目に定義があります。

被呼び出し関係図:



8.43.3.47 resetTranslation()

```
void GgApp::Window::resetTranslation () [inline]
```

平行移動量を初期化する。

[GgApp.h](#) の 870 行目に定義があります。

被呼び出し関係図:



8.43.3.48 restoreViewport()

```
void GgApp::Window::restoreViewport () const [inline]
```

ビューポートを元のサイズに復帰する。

[GgApp.h](#) の 350 行目に定義があります。

被呼び出し関係図:



8.43.3.49 selectInterface()

```
void GgApp::Window::selectInterface (  
    int no) [inline]
```

インタフェースを選択する。

引数

<i>no</i>	インターフェース番号.
-----------	-------------

[GgApp.h](#) の 488 行目に定義があります。

8.43.3.50 setClose()

```
void GgApp::Window::setClose (
    int flag = GLFW_TRUE) const [inline]
```

ウィンドウのクローズフラグを設定する.

引数

<i>flag</i>	クローズフラグ, 0 (GLFW_FALSE) 以外ならウィンドウを閉じる.
-------------	--

[GgApp.h](#) の 319 行目に定義があります。

8.43.3.51 setKeyboardFunc()

```
void GgApp::Window::setKeyboardFunc (  
    void(* func ) (const Window *window, int key, int scancode, int action, int mods))  
[inline]
```

ユーザ定義の keyboard 関数を設定する.

引数

<i>func</i>	ユーザ定義の keyboard 関数, キーボードの操作時に呼び出される.
-------------	---------------------------------------

[GgApp.h](#) の 923 行目に定義があります。

呼び出し関係図:



8.43.3.52 setMouseFunc()

```
void GgApp::Window::setMouseFunc (  
    void(* func ) (const Window *window, int button, int action, int mods)) [inline]
```

ユーザ定義の mouse 関数を設定する.

引数

<i>func</i>	ユーザ定義の mouse 関数, マウスボタンの操作時に呼び出される.
-------------	-------------------------------------

[GgApp.h](#) の 933 行目に定義があります。

呼び出し関係図:



8.43.3.53 setResizeFunc()

```
void GgApp::Window::setResizeFunc (
    void(* func )(const Window *window, int width, int height)) [inline]
```

ユーザ定義の `resize` 関数を設定する.

引数

<i>func</i>	ユーザ定義の <code>resize</code> 関数, ウィンドウのサイズ変更時に呼び出される.
-------------	---

[GgApp.h](#) の 913 行目に定義があります。

呼び出し関係図:



8.43.3.54 setUserPointer()

```
void GgApp::Window::setUserPointer (
    void * pointer) [inline]
```

任意のユーザポインタを保存する.

引数

<i>pointer</i>	保存するユーザポインタ.
----------------	--------------

[GgApp.h](#) の 903 行目に定義があります。

8.43.3.55 setVelocity()

```
void GgApp::Window::setVelocity (
    GLfloat vx,
    GLfloat vy,
    GLfloat vz = 0.1f) [inline]
```

マウスの移動速度を設定する.

引数

<code>vx</code>	x 方向の移動速度.
<code>vy</code>	y 方向の移動速度.
<code>vz</code>	z 方向の移動速度.

[GgApp.h](#) の 501 行目に定義があります。

8.43.3.56 setWheelFunc()

```
void GgApp::Window::setWheelFunc (
    void(* func ) (const Window *window, double x, double y)) [inline]
```

ユーザ定義の wheel 関数を設定する.

引数

<code>func</code>	ユーザ定義の wheel 関数, マウスホイールの操作時に呼び出される.
-------------------	--------------------------------------

[GgApp.h](#) の 943 行目に定義があります。

呼び出し関係図:



8.43.3.57 shouldClose()

```
bool GgApp::Window::shouldClose () const [inline]
```

ウィンドウを閉じるべきかどうか調べる.

戻り値

ウィンドウを閉じるべきなら true.

[GgApp.h](#) の 329 行目に定義があります。

被呼び出し関係図:



8.43.3.58 swapBuffers()

```
void GgApp::Window::swapBuffers () const
```

カラーバッファを入れ替える。

[GgApp.cpp](#) の 482 行目に定義があります。

被呼び出し関係図:



8.43.3.59 updateViewport()

```
void GgApp::Window::updateViewport ()
```

ビューポートのサイズを更新する。

[GgApp.cpp](#) の 501 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

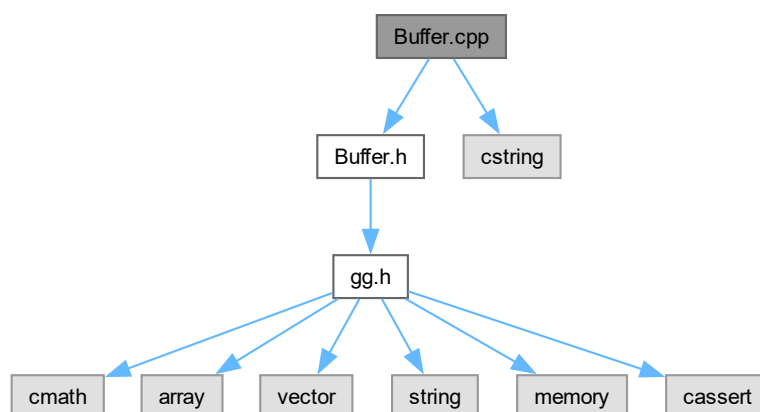
- [GgApp.h](#)
- [GgApp.cpp](#)

Chapter 9

ファイル詳解

9.1 Buffer.cpp ファイル

```
#include "Buffer.h"  
#include <cstring>  
Buffer.cpp の依存先関係図:
```



9.1.1 詳解

バッファクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Buffer.cpp](#) に定義があります。

9.2 Buffer.cpp

[詳解]

```

00001
00008 #include "Buffer.h"
00009
00010 // 標準ライブラリ
00011 #include <cstring>
00012
00013 //
00014 // フォーマットからチャネル数を求める
00015 //
00016 int Buffer::formatToChannels(GLenum format) const
00017 {
00018     switch (format)
00019     {
00020     case GL_RED:
00021         return 1;
00022     case GL_RG:
00023         return 2;
00024     case GL_RGB:
00025         return 3;
00026     case GL_RGBA:
00027         return 4;
00028     default:
00029         assert(false);
00030         break;
00031     }
00032
00033     return 0;
00034 };
00035
00036 //
00037 // 既存のバッファにコピーする
00038 //
00039 void Buffer::copyBuffer(const Buffer& buffer) noexcept
00040 {
00041     #if defined(USE_PIXEL_BUFFER_OBJECT)
00042         // コピー元のピクセルバッファオブジェクト
00043         glBindBuffer(GL_COPY_READ_BUFFER, buffer.bufferName);
00044
00045         // コピー先のピクセルバッファオブジェクト
00046         glBindBuffer(GL_COPY_WRITE_BUFFER, bufferName);
00047
00048         // バッファオブジェクトをコピーする
00049         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
00050                             0, 0, std::min(bufferLength, buffer.bufferLength));
00051     #else
00052         // フレームのデータをコピーする
00053         memcpy(bufferName.data(), buffer.bufferName.data(), bufferName.size());
00054     #endif
00055 }
00056
00057 //
00058 // バッファを作成する
00059 //
00060 void Buffer::create(GLsizei width, GLsizei height, int channels)
00061 {
00062     // 指定したサイズが保持しているフレームのサイズと同じなら何もしない
00063     if (width == bufferSize[0] && height == bufferSize[1]
00064         && channels == bufferChannels) return;
00065
00066     // フレームのサイズとチャネル数を記録する
00067     bufferSize = std::array<int, 2>{ width, height };
00068     bufferChannels = channels;
00069
00070     #if defined(USE_PIXEL_BUFFER_OBJECT)
00071         // フレームの保存に必要なメモリ量を求める
00072         bufferLength = width * height * channels;
00073
00074         // 以前のピクセルバッファオブジェクトを破棄する
00075         glDeleteBuffers(1, &bufferName);
00076
00077         // 新しいピクセルバッファオブジェクトを作成する
00078         glGenBuffers(1, &bufferName);
00079
00080         // ピクセルバッファオブジェクトのメモリを確保してデータを転送する
00081         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00082         glBufferData(GL_PIXEL_PACK_BUFFER, bufferLength, nullptr, GL_DYNAMIC_COPY);
00083         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00084     #else
00085         // メモリを確保する
00086         bufferName.resize(width * height * channels);
00087     #endif
00088 }

```

```

00089
00090 //
00091 // バッファをコピーする
00092 //
00093 void Buffer::copy(const Buffer& buffer) noexcept
00094 {
00095     // コピー元と同じサイズの空のバッファを作る
00096     Buffer::create(buffer.bufferSize[0], buffer.bufferSize[1],
00097         buffer.bufferChannels);
00098
00099     // 作成されたバッファにコピーする
00100     copyBuffer(buffer);
00101 }
00102
00103 //
00104 // 代入演算子
00105 //
00106 Buffer& Buffer::operator=(const Buffer& buffer)
00107 {
00108     // 代入元と代入先が同じなら何もしない
00109     if (&buffer == this) return *this;
00110
00111     // 引数のバッファをバッファをこのバッファにコピーする
00112     Buffer::copy(buffer);
00113
00114     // このバッファを返す
00115     return *this;
00116 }
00117
00118 //
00119 // ムーブ代入演算子
00120 //
00121 Buffer& Buffer::operator=(Buffer&& buffer) noexcept
00122 {
00123     // ムーブ代入元とムーブ代入先が同じなら何もしない
00124     if (&buffer == this) return *this;
00125
00126     // ムーブ代入元のバッファのメンバをムーブする
00127     bufferSize = buffer.bufferSize;
00128     buffer.bufferSize = { 0, 0 };
00129     buffer.channels = buffer.bufferChannels;
00130     buffer.bufferChannels = 0;
00131 #if defined(USE_PIXEL_BUFFER_OBJECT)
00132     buffer.length = buffer.bufferLength;
00133     buffer.bufferLength = 0;
00134
00135     // ピクセルバッファオブジェクトを削除する
00136     glDeleteBuffers(1, &bufferName);
00137
00138     // ムーブ代入元のピクセルバッファオブジェクト名をムーブ代入先に移す
00139     bufferName = buffer.bufferName;
00140
00141     // ムーブ代入元のデストラクタでバッファが削除されないよう 0 にする
00142     buffer.bufferName = 0;
00143 #endif
00144
00145     // このバッファを返す
00146     return *this;
00147 }
00148
00149 //
00150 // ピクセルバッファオブジェクトをマップする
00151 //
00152 GLvoid* Buffer::map()
00153 #if defined(USE_PIXEL_BUFFER_OBJECT)
00154 const
00155 #endif
00156 {
00157     #if defined(USE_PIXEL_BUFFER_OBJECT)
00158         // ピクセルバッファオブジェクトを参照する
00159         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00160
00161         // ピクセルバッファオブジェクトをマップする
00162         return glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, bufferLength,
00163             GL_MAP_READ_BIT | GL_MAP_WRITE_BIT);
00164     #else
00165         // データの格納場所を返す
00166         return bufferName.data();
00167     #endif
00168 }
00169
00170 //
00171 // ピクセルバッファオブジェクトをアンマップする
00172 //
00173 void Buffer::unmap() const
00174 {
00175     #if defined(USE_PIXEL_BUFFER_OBJECT)

```

```

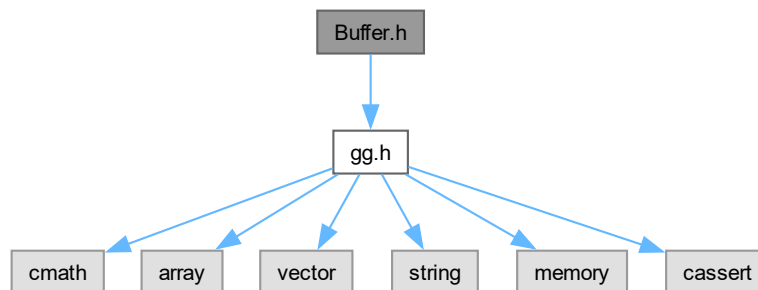
00176 // ピクセルバッファオブジェクトのマップを解除する
00177 glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
00178
00179 // アップロード先のピクセルバッファオブジェクトの結合を解除する
00180 glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00181 #endif
00182 }
00183
00184 //
00185 // バッファを破棄する
00186 //
00187 void Buffer::discard()
00188 {
00189     #if defined(USE_PIXEL_BUFFER_OBJECT)
00190         // ピクセルバッファオブジェクトの結合を解除する
00191         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00192         glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00193
00194         // ピクセルバッファオブジェクトを削除する
00195         glDeleteBuffers(1, &bufferName);
00196         bufferName = 0;
00197     #else
00198         // メモリを消去する
00199         bufferName.clear();
00200     #endif
00201
00202     // バッファのサイズを 0 にする
00203     bufferSize = std::array<int, 2>{ 0, 0 };
00204     bufferChannels = 0;
00205 }

```

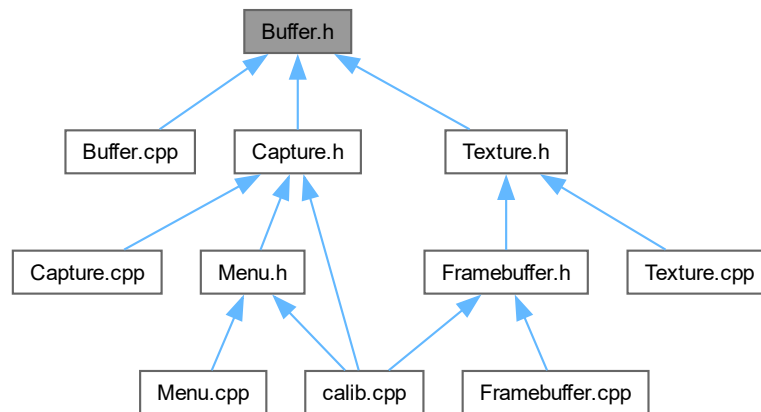
9.3 Buffer.h ファイル

```
#include "gg.h"
```

Buffer.h の依存先関係図:



被依存関係図:



クラス

- class [Buffer](#)

マクロ定義

- `#define` [USE_PIXEL_BUFFER_OBJECT](#)

9.3.1 詳解

バッファクラスの定義

著者

Kohe Tokoi

日付

Aplil 3, 2023

[Buffer.h](#) に定義があります。

9.3.2 マクロ定義詳解

9.3.2.1 USE_PIXEL_BUFFER_OBJECT

```
#define USE_PIXEL_BUFFER_OBJECT
```

[Buffer.h](#) の 16 行目に定義があります。

9.4 Buffer.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00015 // ピクセルバッファオブジェクトを使うとき
00016 #define USE_PIXEL_BUFFER_OBJECT
00017
00024 class Buffer
00025 {
00026     std::array<int, 2> bufferSize{ 0, 0 };
00028
00030     int bufferChannels{ 0 };
00031
00032 #if defined(USE_PIXEL_BUFFER_OBJECT)
00034     GLsizei bufferLength{ 0 };
00035
00037     GLuint
00038 #else
00040     std::vector<GLubyte>
00041 #endif
00042     bufferName{ 0 };
00043
00044 protected:
00045
00052     auto channelsToFormat(int channels) const
00053     {
00054         // OpenGL のテクスチャフォーマットのリスト
00055         static constexpr GLenum toFormat[] { GL_RED, GL_RG, GL_RGB, GL_RGBA };
00056
00057         // フォーマットを返す
00058         return toFormat[(channels - 1) & 3];
00059     }
00060
00067     int formatToChannels(GLenum format) const;
00068
00074     void copyBuffer(const Buffer& buffer) noexcept;
00075
00076 public:
00077
00081     Buffer() = default;
00082
00094     Buffer(GLsizei width, GLsizei height, int channels)
00095     {
00096         Buffer::create(width, height, channels);
00097     }
00098
00104     Buffer(const Buffer& buffer)
00105     {
00106         Buffer::copy(buffer);
00107     }
00108
00114     Buffer(Buffer&& buffer) noexcept
00115         : bufferSize{ buffer.bufferSize }
00116         , bufferChannels{ buffer.bufferChannels }
00117 #if defined(USE_PIXEL_BUFFER_OBJECT)
00118         , bufferLength{ buffer.bufferLength }
00119         , bufferName{ buffer.bufferName }
00120 #endif
00121     {
00122         buffer.bufferSize = { 0, 0 };
00123         buffer.bufferChannels = 0;
00124 #if defined(USE_PIXEL_BUFFER_OBJECT)
00125         buffer.bufferLength = 0;
00126
00127         // ムーブ元のデストラクタでバッファが削除されないよう 0 にする
00128         buffer.bufferName = 0;
00129 #endif
00130     }
00131
00135     virtual ~Buffer()
00136     {
00137         Buffer::discard();
00138     }
00139
00146     Buffer& operator=(const Buffer& buffer);
00147
00154     Buffer& operator=(Buffer&& buffer) noexcept;
00155
00167     virtual void create(GLsizei width, GLsizei height, int channels);

```

```

00168
00180 void resize(GLsizei width, GLsizei height, int channels)
00181 {
00182     create(width, height, channels);
00183 }
00184
00194 void resize(const Buffer& buffer)
00195 {
00196     create(buffer.getWidth(), buffer.getHeight(), buffer.getChannels());
00197 }
00198
00208 virtual void copy(const Buffer& buffer) noexcept;
00209
00213 virtual void discard();
00214
00220 #if defined(USE_PIXEL_BUFFER_OBJECT)
00221 auto getBufferName() const
00222 #else
00223 auto& getBufferName()
00224 #endif
00225 {
00226     return bufferName;
00227 }
00228
00234 virtual const std::array<GLsizei, 2>& getSize() const
00235 {
00236     return bufferSize;
00237 }
00238
00244 GLsizei getWidth() const
00245 {
00246     return getSize()[0];
00247 }
00248
00254 GLsizei getHeight() const
00255 {
00256     return getSize()[1];
00257 }
00258
00264 virtual int getChannels() const
00265 {
00266     return bufferChannels;
00267 }
00268
00274 auto getFormat() const
00275 {
00276     return channelsToFormat(getChannels());
00277 }
00278
00284 auto getAspect() const
00285 {
00286     return static_cast<GLfloat>(getWidth()) / static_cast<GLfloat>(getHeight());
00287 }
00288
00294 void bindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00295 {
00296     #if defined(USE_PIXEL_BUFFER_OBJECT)
00297         glBindBuffer(target, bufferName);
00298     #endif
00299 }
00300
00306 void unbindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00307 {
00308     #if defined(USE_PIXEL_BUFFER_OBJECT)
00309         glBindBuffer(target, 0);
00310     #endif
00311 }
00312
00318 GLvoid* map()
00319 #if defined(USE_PIXEL_BUFFER_OBJECT)
00320     const
00321 #endif
00322     ;
00323
00327 void unmap() const;
00328 };

```

9.5 calib.cpp ファイル

```

#include "GgApp.h"
#include "Config.h"

```


9.6 calib.cpp

[詳解]

```

00001
00008
00009 // ウィンドウ関連の処理
00010 #include "GgApp.h"
00011
00012 // 構成データ
00013 #include "Config.h"
00014
00015 // キャプチャデバイス
00016 #include "Capture.h"
00017
00018 // 校正
00019 #include "Calibration.h"
00020
00021 // メニュー
00022 #include "Menu.h"
00023
00024 // フレームバッファオブジェクト
00025 #include "Framebuffer.h"
00026
00027 // 構成ファイル名
00028 #define CONFIG_FILE PROJECT_NAME "_config.json"
00029
00030 //
00031 // アプリケーション本体
00032 //
00033 int GgApp::main(int argc, const char* const* argv)
00034 {
00035     // 構成ファイルを読み込む
00036     Config config{ CONFIG_FILE };
00037
00038     // 構成にもとづいてウィンドウを作成する
00039     GgApp::Window window{ config.getTitle(), config.getWidth(), config.getHeight() };
00040
00041     // 開いたウィンドウに対して初期化処理を実行する
00042     config.initialize();
00043
00044     // キャプチャデバイスを作る
00045     Capture capture;
00046
00047     // 校正オブジェクトを作成する
00048     Calibration calibration{ config.getDictionaryName(), config.getCheckerSize(),
00049                             config.getCheckerLength() };
00049
00050     // メニューを作る
00051     Menu menu{ config, capture, calibration };
00052
00053     // キャプチャデバイスで初期画像を開く
00054     if (!capture.openImage(config.getInitialImage())) throw std::runtime_error("Cannot open initial
00055 image.");
00056
00057     // 初期画像の実解像度と焦点距離から、見やすい初期画角を設定する
00058     menu.initializeInputIntrinsics(capture.getSize());
00059
00060     // キャプチャしたフレームを保持するテクスチャ
00061     Texture frame;
00062
00063     // 画像の展開に用いるフレームバッファオブジェクトのサイズを初期ウィンドウに合わせる
00064     Framebuffer framebuffer{ config.getWidth(), config.getHeight() };
00065
00066     // ウィンドウが開いている間繰り返す
00067     while (window && menu)
00068     {
00069         // メニューを表示して設定を更新する
00070         menu.draw();
00071
00072         // 選択しているキャプチャデバイスから1フレーム取得する
00073         capture.retrieve(frame);
00074
00075         // ピクセルバッファオブジェクトの内容をテクスチャに転送する
00076         frame.drawPixels();
00077
00078         // フレームバッファオブジェクトのサイズをキャプチャしたフレームに合わせる
00079         framebuffer.resize(frame);
00080
00081         // シェーダの設定を行う
00082         const auto&& size{ menu.setup(framebuffer.getAspect()) };
00083
00084         // フレームバッファオブジェクトにフレームを展開する
00085         framebuffer.update(size, frame);
00086
00087         // ArUco Marker を検出するなら

```

```

00087     if (menu.detectMarker || menu.detectBoard)
00088     {
00089         // フレームバッファオブジェクトの内容をピクセルバッファオブジェクトに転送する
00090         framebuffer.readPixels();
00091
00092         // 入力画像のサイズを調べる
00093         const auto size{ cv::Size{ framebuffer.getWidth(), framebuffer.getHeight() } };
00094
00095         // ピクセルバッファオブジェクトを CPU のメモリ空間にマップする
00096         cv::Mat image{ size, CV_8UC(framebuffer.getChannels()), framebuffer.map() };
00097
00098         // ChArUco Board を認識するなら
00099         if (menu.detectBoard)
00100         {
00101             // ChArUco Board を検出する
00102             calibration.detectBoard(image);
00103         }
00104         else
00105         {
00106             // ArUco Marker を検出する
00107             calibration.detectMarkers(image, menu.getMarkerLength());
00108         }
00109
00110         // ピクセルバッファオブジェクトのマップを解除する
00111         framebuffer.unmap();
00112
00113         // ピクセルバッファオブジェクトの内容をフレームバッファオブジェクトに書き戻す
00114         framebuffer.drawPixels();
00115     }
00116
00117     // 表示するウィンドウのビューポートを再設定する
00118     window.setMenuBarHeight(menu.getMenuBarHeight());
00119
00120     // シェーダーでBGRAをRGBAへ変換し、縦横比を維持して実Framebuffer領域へ中央表示する
00121     framebuffer.draw(window.getFboWidth(), window.getFboHeight());
00122
00123     // カラーバッファを入れ替えてイベントを取り出す
00124     window.swapBuffers();
00125 }
00126
00127 return 0;
00128 }

```

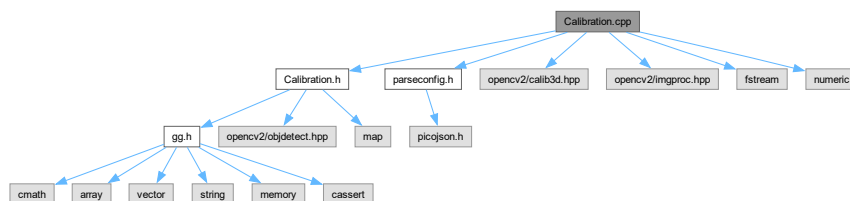
9.7 Calibration.cpp ファイル

```

#include "Calibration.h"
#include "parseconfig.h"
#include <opencv2/calib3d.hpp>
#include <opencv2/imgproc.hpp>
#include <fstream>
#include <numeric>

```

Calibration.cpp の依存先関係図:



マクロ定義

- `#define USE_RODRIGUES`

9.7.1 詳解

較正用フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Calibration.cpp](#) に定義があります。

9.7.2 マクロ定義詳解

9.7.2.1 USE_RODRIGUES

```
#define USE_RODRIGUES
```

[Calibration.cpp](#) の 23 行目に定義があります。

9.8 Calibration.cpp

[[詳解](#)]

```
00001
00008 #include "Calibration.h"
00009
00010 // 構成ファイルの読み取り補助
00011 #include "parseconfig.h"
00012
00013 // OpenCV
00014 #pragma warning(disable:4819)
00015 #include <opencv2/calib3d.hpp>
00016 #include <opencv2/imgproc.hpp>
00017
00018 // 標準ライブラリ
00019 #include <fstream>
00020 #include <numeric>
00021
00022 // cv::Rodrigues() を使う
00023 #define USE_RODRIGUES
00024
00025
00026 //
00027 // コンストラクタ
00028 //
00029 Calibration::Calibration(const std::string& dictionaryName,
00030                          const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength)
00031 {
00032     // ArUco Marker の辞書を選択する
00033     setDictionary(dictionaryName, checkerSize, checkerLength);
00034 }
00035
00036 //
00037 // デストラクタ
00038 //
00039 Calibration::~Calibration()
00040 {
00041 }
00042
00043 //
00044 // ChArUco Board を作成する
```

```

00045 //
00046 void Calibration::createBoard(const std::array<int, 2>& checkerSize,
00047                               const std::array<float, 2>& checkerLength)
00048 {
00049     // キャリブレーション用の ChArUco Board を作成する
00050     board = new cv::aruco::CharucoBoard(cv::Size{ checkerSize[0], checkerSize[1] },
00051     checkerLength[0] * 0.01f, checkerLength[1] * 0.01f, dictionary);
00052
00053     // キャリブレーション用の ChArUco Board の検出器を作成する
00054     boardDetector = new cv::aruco::CharucoDetector(*board);
00055
00056     // board は boardDetector->getBoard() で取り出すことができるが
00057     // 実行中に board を作り直すことがあるので cv::Ptr に持たせる
00058
00059     // 校正結果を再利用しない
00060     calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00061 }
00062
00063 //
00064 // ArUco Marker の辞書と検出器を設定する
00065 //
00066 void Calibration::setDictionary(const std::string& dictionaryName,
00067                                 const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength)
00068 {
00069     // ArUco Marker の辞書を検索する
00070     auto dictionaryItem{ dictionaryList.find(dictionaryName) };
00071
00072     // ArUco Marker の辞書が見つからなかったら辞書リストの最初の辞書を使う
00073     if (dictionaryItem == dictionaryList.end()) dictionaryItem = dictionaryList.begin();
00074
00075     // ArUco Marker の辞書を設定する
00076     dictionary = cv::aruco::getPredefinedDictionary(dictionaryItem->second);
00077
00078     // ArUco Marker の検出器を作成する
00079     cv::aruco::DetectorParameters detectorParams = cv::aruco::DetectorParameters();
00080     detector = new cv::aruco::ArucoDetector(dictionary, detectorParams);
00081
00082     // キャリブレーション用の ChArUco Board を作成する
00083     createBoard(checkerSize, checkerLength);
00084 }
00085
00086 //
00087 // ChArUco Board を描く
00088 //
00089 void Calibration::drawBoard(cv::Mat& boardImage, int width, int height)
00090 {
00091     board->generateImage(cv::Size{ width, height }, boardImage, 10, 1);
00092 }
00093
00094 //
00095 // ChArUco Board を検出する
00096 //
00097 void Calibration::detectBoard(cv::Mat& image)
00098 {
00099     // 画像のサイズを保存しておく
00100     size = image.size();
00101
00102     // 4 チャンネル画像の場合は一時的に3チャンネル画像を作成する
00103     cv::Mat tempImage;
00104     const auto isFourChannels{ image.channels() == 4 };
00105     if (isFourChannels)
00106     {
00107         cv::cvtColor(image, tempImage, cv::COLOR_BGRA2BGR);
00108     }
00109     else
00110     {
00111         tempImage = image;
00112     }
00113
00114     // ChArUco Board のコーナーを検出する
00115     boardDetector->detectBoard(tempImage, charucoCorners, charucoIds);
00116
00117     // コーナーが見つからなかったら何もしない
00118     if (charucoCorners.empty()) return;
00119
00120     // ChArUco Board のコーナーの位置を表示に描き込む
00121     cv::aruco::drawDetectedCornersCharuco(tempImage, charucoCorners, charucoIds, cv::Scalar(0, 0, 255));
00122
00123     // 4 チャンネル画像の場合は結果を書き戻す
00124     if (isFourChannels)
00125     {
00126         cv::cvtColor(tempImage, image, cv::COLOR_BGR2BGRA);
00127     }
00128 }
00129
00130 //
00131 // ArUco Marker を検出する

```



```

00132 //
00133 void Calibration::detectMarkers(cv::Mat& image, float markerLength)
00134 {
00135     // 4 チャンネル画像の場合は一時的に3チャンネル画像を作成する
00136     cv::Mat tempImage;
00137     const auto isFourChannels{ image.channels() == 4 };
00138     if (isFourChannels)
00139     {
00140         cv::cvtColor(image, tempImage, cv::COLOR_BGRA2BGR);
00141     }
00142     else
00143     {
00144         tempImage = image;
00145     }
00146
00147     // ArUco Marker のコーナーを検出する
00148     detector->detectMarkers(tempImage, corners, ids, rejected);
00149
00150     // コーナーが見つからなければ戻る
00151     if (corners.empty()) return;
00152
00153     // キャリブレーションが完了していれば
00154     if (finished())
00155     {
00156         // 各マーカに対応する3次元空間の点
00157         const float markerCenter{ markerLength * 0.5f };
00158         std::vector<cv::Point3f> markerObjPoints;
00159         markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00160         markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00161         markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00162         markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00163
00164         // 個々のマーカーについて
00165         for (size_t i = 0; i < corners.size(); ++i)
00166         {
00167             // マーカーのコーナー検出位置から3次元姿勢(回転・平行移動)を推定する
00168             cv::Vec3d rvec, tvec;
00169             cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00170                 cv::SOLVEPNP_IPPE_SQUARE);
00171
00172             // 座標軸を描く
00173             cv::drawFrameAxes(tempImage, cameraMatrix, distCoeffs, rvec, tvec, markerLength);
00174         }
00175     }
00176     else
00177     {
00178         // ArUco Marker の場所に矩形と番号を描き込む
00179         cv::aruco::drawDetectedMarkers(tempImage, corners, ids);
00180     }
00181
00182     // 4 チャンネル画像の場合は結果を書き戻す
00183     if (isFourChannels)
00184     {
00185         cv::cvtColor(tempImage, image, cv::COLOR_BGR2BGRA);
00186     }
00187 }
00188 //
00189 // 標本を取得する
00190 //
00191 void Calibration::recordCorners()
00192 {
00193     // ChArUco Board のコーナーが4 つ以上見つければ
00194     if (charucoCorners.size() >= 4)
00195     {
00196         // ChArUco Board のレイアウトと検出されたコーナーから
00197         // ChArUco Board 上の点と対応する画像上の点を求める
00198         board->matchImagePoints(charucoCorners, charucoIds, objectPoints, imagePoints);
00199
00200         // ChArUco Board 上の点と対応する画像上の点が見つければ
00201         if (!imagePoints.empty() && !objectPoints.empty())
00202         {
00203             // ChArUco Board のコーナーを記録する
00204             allCorners.push_back(charucoCorners);
00205             allIds.push_back(charucoIds);
00206             allImagePoints.push_back(imagePoints);
00207             allObjectPoints.push_back(objectPoints);
00208
00209             // 記録したコーナーの数の合計を求める
00210             totalCorners += static_cast<int>(charucoCorners.size());
00211         }
00212     }
00213 #if defined(_DEBUG)
00214     std::cerr << "charucoCorners = " << charucoCorners.size()
00215         << ", allCorners = " << allCorners.size() << "\n";
00216 #endif
00217 }

```

```

00218
00219 //
00220 // 標本と校正結果を破棄する
00221 //
00222 void Calibration::discardCorners()
00223 {
00224     // 記録した標本を消去する
00225     allCorners.clear();
00226     allIds.clear();
00227
00228     // 校正結果を消去する
00229     cameraMatrix.release();
00230     distCoeffs.release();
00231
00232     // 検出したコーナー数の合計を 0 にする
00233     totalCorners = 0;
00234
00235     // 校正の計算結果を再利用しない
00236     calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00237 }
00238
00239 //
00240 // 校正する
00241 //
00242 bool Calibration::calibrate()
00243 {
00244     // 再投影誤差はとりあえず 0 にしておく
00245     repError = 0.0f;
00246
00247     // コーナーを合計 6 つ以上検出できていれば
00248     if (allCorners.size() >= 6) try
00249     {
00250         // CALIB_USE_INTRINSIC_GUESS が設定されていない場合に、
00251         // fx と fy をしてしたアスペクト比に強制する
00252         if (calibrationFlags & cv::CALIB_FIX_ASPECT_RATIO)
00253         {
00254             const auto aspect{ static_cast<double>(size.width) / size.height };
00255             cameraMatrix = cv::Mat_<double>(3, 3, CV_64F);
00256             cameraMatrix.at<double>(0, 0) = aspect;
00257         }
00258
00259         // ChArUco Board の姿勢 (使わないので捨ててしまう)
00260         std::vector<cv::Mat> boardRvecs, boardTvecs;
00261
00262         // 取得した全てのコーナーからカメラパラメータを推定する
00263         repError = cv::calibrateCamera(allObjectPoints, allImagePoints, size,
00264             cameraMatrix, distCoeffs, boardRvecs, boardTvecs, cv::noArray(),
00265             cv::noArray(), cv::noArray(), calibrationFlags);
00266     }
00267     catch (const cv::Exception&)
00268     {
00269         // 校正に失敗した場合は計算結果を捨てる
00270         discardCorners();
00271
00272         // 校正に失敗したことを報告する
00273         return false;
00274     }
00275
00276     // 校正の計算結果を再利用する
00277     calibrationFlags |= cv::CALIB_USE_INTRINSIC_GUESS;
00278
00279     // 校正に成功したことを報告する
00280     return true;
00281 }
00282
00283 //
00284 // 回転ベクトルから姿勢の変換行列を求める
00285 //
00286 GgMatrix Calibration::RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec)
00287 {
00288     #if defined(USE_RODRIGUES)
00289         // 回転軸と回転角から回転の変換行列を求める
00290         cv::Mat_<double> r(3, 3);
00291         cv::Rodrigues(rvec, r);
00292
00293         // 姿勢の変換行列
00294         return GgMatrix
00295         {
00296             static_cast<GLfloat>(r[0][0]),
00297             static_cast<GLfloat>(r[1][0]),
00298             static_cast<GLfloat>(r[2][0]),
00299             0.0f,
00300             static_cast<GLfloat>(r[0][1]),
00301             static_cast<GLfloat>(r[1][1]),
00302             static_cast<GLfloat>(r[2][1]),
00303             0.0f,
00304             static_cast<GLfloat>(r[0][2]),

```

```

00305     static_cast<GLfloat>(r[1][2]),
00306     static_cast<GLfloat>(r[2][2]),
00307     0.0f,
00308     static_cast<GLfloat>(tvec[0]),
00309     static_cast<GLfloat>(tvec[1]),
00310     static_cast<GLfloat>(tvec[2]),
00311     1.0f
00312 };
00313 #else
00314 // 回転角
00315 const auto d{ cv::norm(rvec) };
00316
00317 // 回転軸ベクトル
00318 const auto rx{ static_cast<GLfloat>(rvec[0] / d) };
00319 const auto ry{ static_cast<GLfloat>(rvec[1] / d) };
00320 const auto rz{ static_cast<GLfloat>(rvec[2] / d) };
00321
00322 // 平行移動量
00323 const auto tx{ static_cast<GLfloat>(tvec[0]) };
00324 const auto ty{ static_cast<GLfloat>(tvec[1]) };
00325 const auto tz{ static_cast<GLfloat>(tvec[2]) };
00326
00327 // 姿勢の変換行列
00328 return ggTranslate(tx, ty, tz).rotate(rx, ry, rz, static_cast<GLfloat>(d));
00329 #endif
00330 }
00331
00332 //
00333 // ArUco Marker の3次元姿勢の変換行列を求める
00334 //
00335 void Calibration::getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses)
00336 {
00337     // 各マーカに対応する3次元空間の点
00338     const float markerCenter = markerLength * 0.5f;
00339     std::vector<cv::Point3f> markerObjPoints;
00340     markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00341     markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00342     markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00343     markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00344
00345     // 個々のマーカについて
00346     for (size_t i = 0; i < corners.size(); ++i)
00347     {
00348         // マーカのコーナー検出位置から3次元姿勢(回転・平行移動)を推定する
00349         cv::Vec3d rvec, tvec;
00350         cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00351             cv::SOLVEPNP_IPPE_SQUARE);
00352
00353         // 各マーカの姿勢の変換行列を求める
00354         poses[ids[i]] = RvecTvecToPose(rvec, tvec);
00355     }
00356 }
00357 //
00358 // カメラパラメータのJSON オブジェクトから数値の配列を取得する
00359 //
00360 static bool getMatrix(const picojson::object& object,
00361     const std::string& key, cv::Mat& mat, int cols, int rows)
00362 {
00363     // key に一致するオブジェクトを探す
00364     const auto&& value{ object.find(key) };
00365
00366     // オブジェクトが無い配列でなかったら戻る
00367     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00368
00369     // 配列を取り出す
00370     const auto& array{ value->second.get<picojson::array>() };
00371
00372     // 配列の要素数とデータの格納先の行列の要素数が一致していなければ戻る
00373     if (static_cast<size_t>(cols) * rows != array.size()) return false;
00374
00375     // カメラ行列の要素を確保する
00376     mat = cv::Mat::zeros(rows, cols, CV_64F);
00377
00378     // 配列の要素について
00379     for (size_t i = 0; i < array.size(); ++i)
00380     {
00381         // 行列の要素の位置
00382         const auto x{ static_cast<int>(i % cols) };
00383         const auto y{ static_cast<int>(i / cols) };
00384
00385         // 要素が数値なら格納する
00386         if (array[i].is<double>()) mat.at<double>(y, x) = array[i].get<double>();
00387     }
00388
00389     return true;
00390 }

```

```

00391
00392 //
00393 // カメラパラメータの JSON オブジェクトから数値の配列を取得する
00394 //
00395 static void setMatrix(picojson::object& object,
00396     const std::string& key, const cv::Mat& mat)
00397 {
00398     // picojson の配列
00399     picojson::array array;
00400
00401     // 配列の要素について
00402     for (size_t i = 0; i < mat.total(); ++i)
00403     {
00404         // 行列の要素の位置
00405         const auto x{ static_cast<int>(i % mat.cols) };
00406         const auto y{ static_cast<int>(i / mat.cols) };
00407
00408         // 要素を picojson::array に追加する
00409         array.emplace_back(picojson::value(mat.at<double>(y, x)));
00410     }
00411
00412     // オブジェクトに追加する
00413     object.emplace(key, array);
00414 }
00415
00416 //
00417 // ファイルからキャリブレーションパラメータを読み込む
00418 //
00419 bool Calibration::loadParameters(const std::string& filename)
00420 {
00421     // パラメータファイルの読み込み
00422     std::ifstream json{ Utf8ToTChar(filename) };
00423     if (!json) return false;
00424
00425     // JSON の読み込み
00426     picojson::value value;
00427     json >> value;
00428     json.close();
00429
00430     // 構成内容の取り出し
00431     const auto& object{ value.get<picojson::object>() };
00432
00433     // オブジェクトが空だったらエラー
00434     if (object.empty()) return false;
00435
00436     // カメラ行列
00437     if (!getMatrix(object, "camera matrix", cameraMatrix, 3, 3)) return false;
00438
00439     // 歪み定数
00440     if (!getMatrix(object, "distortion", distCoeffs, 5, 1)) return false;
00441
00442     // 再投影誤差
00443     getValue(object, "error", repError);
00444
00445     // 較正の計算結果を再利用しない
00446     calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00447
00448     // キャリブレーションパラメータの読み込み
00449     return true;
00450 }
00451
00452 //
00453 // キャリブレーションパラメータをファイルに保存する
00454 //
00455 bool Calibration::saveParameters(const std::string& filename) const
00456 {
00457     // 設定値を保存する
00458     std::ofstream config{ Utf8ToTChar(filename) };
00459     if (!config) return false;
00460
00461     // オブジェクト
00462     picojson::object object;
00463
00464     // カメラ行列
00465     setMatrix(object, "camera matrix", cameraMatrix);
00466
00467     // 歪み定数
00468     setMatrix(object, "distortion", distCoeffs);
00469
00470     // 再投影誤差
00471     setValue(object, "error", repError);
00472
00473     // 構成をシリアライズして保存
00474     picojson::value v{ object };
00475     config << v.serialize(true);
00476     config.close();
00477

```

```

00478 // キャリブレーションパラメータの書き込み
00479 return true;
00480 }
00481
00482 // ArUco Marker 辞書のリスト
00483 const std::map<const std::string, const cv::aruco::PredefinedDictionaryType>
Calibration::dictionaryList
00484 {
00485     { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
00486     { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
00487     { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
00488     { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
00489     { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
00490     { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
00491     { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
00492     { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
00493     { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
00494     { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
00495     { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
00496     { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
00497     { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
00498     { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
00499     { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
00500     { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
00501     { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
00502     { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
00503     { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
00504     { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
00505     { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 },
00506 };

```

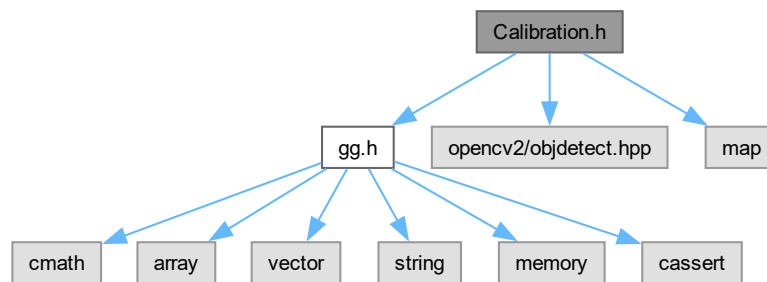
9.9 Calibration.h ファイル

```

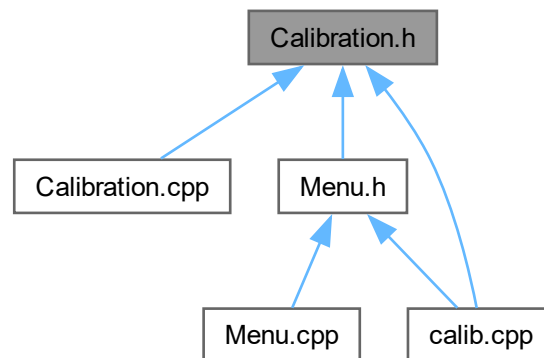
#include "gg.h"
#include <opencv2/objdetect.hpp>
#include <map>

```

Calibration.h の依存先関係図:



被依存関係図:



クラス

- class [Calibration](#)

9.9.1 詳解

校正クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Calibration.h](#) に定義があります。

9.10 Calibration.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00015 // OpenCV ArUco & ChArUco (modern OpenCV 4.7+)
00016 #include <opencv2/objdetect.hpp>
00017
00018 // 標準ライブラリ
00019 #include <map>
```

```

00020
00024 class Calibration
00025 {
00027     cv::aruco::Dictionary dictionary;
00028
00030     cv::Ptr<cv::aruco::ArucoDetector> detector;
00031
00033     cv::Ptr<cv::aruco::CharucoBoard> board;
00034
00036     cv::Ptr<cv::aruco::CharucoDetector> boardDetector;
00037
00039     std::vector<std::vector<cv::Point2f>> corners, rejected;
00040     std::vector<int> ids;
00041
00043     std::vector<cv::Point2f> charucoCorners;
00044     std::vector<int> charucoIds;
00045     std::vector<cv::Point3f> objectPoints;
00046     std::vector<cv::Point2f> imagePoints;
00047
00049     std::vector<std::vector<cv::Point2f>> allCorners;
00050     std::vector<std::vector<int>> allIds;
00051     std::vector<std::vector<cv::Point2f>> allImagePoints;
00052     std::vector<std::vector<cv::Point3f>> allObjectPoints;
00053
00055     cv::Mat cameraMatrix;
00056
00058     cv::Mat distCoeffs;
00059
00061     cv::Size size{ 0, 0 };
00062
00064     double repError{ 0.0 };
00065
00067     int totalCorners{ 0 };
00068
00070     int calibrationFlags
00071     {
00072         0
00073         ///| cv::CALIB_USE_INTRINSIC_GUESS // cameraMatrix contains valid initial values of fx, fy, cx, cy
that are optimized further. Otherwise, (cx, cy) is initially set to the image center (imageSize is
used), and focal distances are computed in a least - squares fashion. Note, that if intrinsic
parameters are known, there is no need to use this function just to estimate extrinsic parameters. Use
solvePnP instead.
00074         ///| cv::CALIB_FIX_PRINCIPAL_POINT // The principal point is not changed during the global
optimization. It stays at the center or at a different location specified when CALIB_USE_INTRINSIC_GUESS
is set too.
00075         ///| cv::CALIB_FIX_ASPECT_RATIO // The functions consider only fy as a free parameter. The ratio
fx / fy stays the same as in the input cameraMatrix. When CALIB_USE_INTRINSIC_GUESS is not set, the
actual input values of fx and fy are ignored, only their ratio is computed and used further.
00076         ///| cv::CALIB_ZERO_TANGENT_DIST // Tangential distortion coefficients (p1, p2) are set to zeros
and stay zero.
00077         ///| cv::CALIB_FIX_FOCAL_LENGTH // The focal length is not changed during the global optimization
if CALIB_USE_INTRINSIC_GUESS is set.
00078         ///| cv::CALIB_FIX_K1 // , ..., CALIB_FIX_K6 The corresponding radial distortion
coefficient is not changed during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient
from the supplied distCoeffs matrix is used. Otherwise, it is set to 0.
00079         ///| cv::CALIB_RATIONAL_MODEL // Coefficients k4, k5, and k6 are enabled. To provide the
backward compatibility, this extra flag should be explicitly specified to make the calibration
function use the rational model and return 8 coefficients or more.
00080         ///| cv::CALIB_THIN_PRISM_MODEL // Coefficients s1, s2, s3 and s4 are enabled. To provide the
backward compatibility, this extra flag should be explicitly specified to make the calibration
function use the thin prism model and return 12 coefficients or more.
00081         ///| cv::CALIB_FIX_S1_S2_S3_S4 // The thin prism distortion coefficients are not changed during
the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied distCoeffs
matrix is used. Otherwise, it is set to 0.
00082         ///| cv::CALIB_TILTED_MODEL // Coefficients tauX and tauY are enabled. To provide the backward
compatibility, this extra flag should be explicitly specified to make the calibration function use the
tilted sensor model and return 14 coefficients.
00083         ///| cv::CALIB_FIX_TAUX_TAUY // The coefficients of the tilted sensor model are not changed
during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied
distCoeffs matrix is used. Otherwise, it is set to 0.
00084     };
00085
00086 public:
00087
00095     Calibration(const std::string& dictionaryName,
00096                 const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);
00097
00103     Calibration(const Calibration& calibration) = delete;
00104
00108     virtual ~Calibration();
00109
00115     Calibration& operator=(const Calibration& calibration) = delete;
00116
00123     void createBoard(const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);
00124
00132     void setDictionary(const std::string& dictionaryName,
00133                       const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);

```

```

00134
00142 void drawBoard(cv::Mat& boardImage, int width, int height);
00143
00149 void detectBoard(cv::Mat& image);
00150
00157 void detectMarkers(cv::Mat& image, float markerLength);
00158
00162 void recordCorners();
00163
00167 void discardCorners();
00168
00174 bool calibrate();
00175
00181 auto getCornersCount() const
00182 {
00183     return static_cast<int>(corners.size());
00184 }
00185
00191 auto getSampleCount() const
00192 {
00193     return static_cast<int>(allCorners.size());
00194 }
00195
00201 auto getTotalCount() const
00202 {
00203     return totalCorners;
00204 }
00205
00211 const auto& getCameraMatrix() const
00212 {
00213     return cameraMatrix;
00214 }
00215
00221 const auto& getDistortionCoefficients() const
00222 {
00223     return distCoeffs;
00224 }
00225
00231 auto getReprojectionError() const
00232 {
00233     return repError;
00234 }
00235
00241 auto finished()
00242 {
00243     return cameraMatrix.total() == 9 && distCoeffs.total() == 5;
00244 }
00245
00253 GgMatrix RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec);
00254
00264 void getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses);
00265
00272 bool loadParameters(const std::string& filename);
00273
00280 bool saveParameters(const std::string& filename) const;
00281
00283 static const std::map<const std::string, const cv::aruco::PredefinedDictionaryType> dictionaryList;
00284 };

```

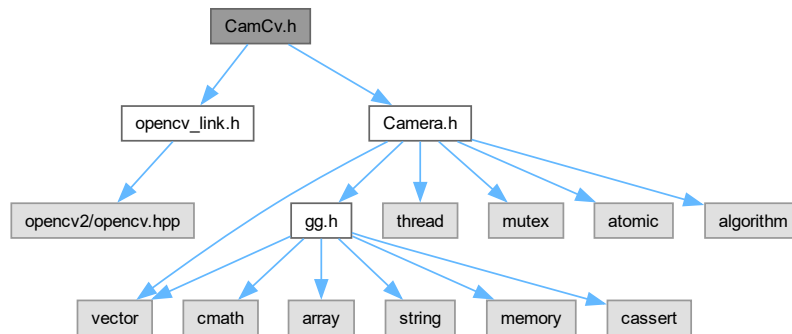
9.11 CamCv.h ファイル

```

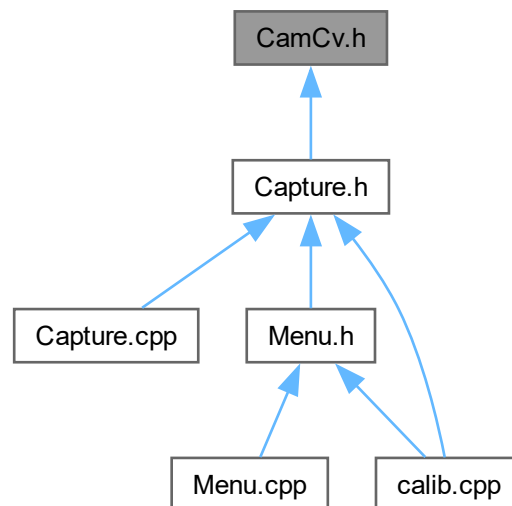
#include "opencv_link.h"
#include "Camera.h"

```


CamCv.h の依存先関係図:



被依存関係図:



クラス

- class [CamCv](#)

9.11.1 詳解

OpenCV を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 27, 2022

[CamCv.h](#) に定義があります。

9.12 CamCv.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // OpenCV へのリンクとインクルード
00012 #include "opencv_link.h"
00013
00014 // カメラ関連の処理
00015 #include "Camera.h"
00016
00020 class CamCv : public Camera
00021 {
00023     cv::VideoCapture camera;
00024
00026     cv::Mat cvFrame;
00027
00029     cv::Mat cvImage;
00030
00032     double elapsedTime{ 0.0 };
00033
00035     int exposure{ 0 }, gain{ 0 };
00036
00046     bool init(int initial_width, int initial_height, double initial_fps, const char* fourcc = "")
00047     {
00048         // カメラのコーデック・解像度・フレームレートを設定する
00049         if (fourcc[0] != '\0') camera.set(cv::CAP_PROP_FOURCC,
00050             cv::VideoWriter::fourcc(fourcc[0], fourcc[1], fourcc[2], fourcc[3]));
00051         if (initial_width > 0) camera.set(cv::CAP_PROP_FRAME_WIDTH, initial_width);
00052         if (initial_height > 0) camera.set(cv::CAP_PROP_FRAME_HEIGHT, initial_height);
00053         if (initial_fps > 0.0) camera.set(cv::CAP_PROP_FPS, initial_fps);
00054
00055         // fps が 0 なら逆数をカメラの遅延に使う
00056         const auto fps{ camera.get(cv::CAP_PROP_FPS) };
00057         if (fps > 0.0) interval = 1000.0 / fps;
00058
00059         // ムービーファイルのインポイント・アウトポイントの初期値とフレーム数
00060         in = camera.get(cv::CAP_PROP_POS_FRAMES);
00061         out = total = camera.get(cv::CAP_PROP_FRAME_COUNT);
00062
00063         // 経過時間
00064         elapsedTime = 0.0;
00065
00066         // カメラから最初のフレームをキャプチャできなかったらカメラは使えない
00067         if (!camera.grab()) return false;
00068
00069         // カメラの利得と露出を取得する
00070         gain = static_cast<GLsizei>(camera.get(cv::CAP_PROP_GAIN));
00071         exposure = static_cast<GLsizei>(camera.get(cv::CAP_PROP_EXPOSURE) * 10.0);
00072
00073         // フレームを取り出してキャプチャ用のメモリを確保する
00074         camera.retrieve(cvFrame);
00075
00076         #if defined(DEBUG)
00077         char codec[5]{ 0, 0, 0, 0, 0 };
00078         getCodec(codec);
00079         std::cerr << "in:" << in << ", out:" << out
00080             << ", width:" << cvFrame.cols << ", height:" << cvFrame.rows
00081             << ", fourcc: " << codec << "\n";
00082         #endif
00083
00084         // キャプチャしたデータを一時メモリにコピーする
00085         cvFrame.copyTo(cvImage);
00086     }
```

```

00087 // 基底クラスのバッファとメンバを更新
00088 width = cvFrame.cols;
00089 height = cvFrame.rows;
00090 channels = cvFrame.channels();
00091 {
00092     const size_t size = cvFrame.total() * cvFrame.elemSize();
00093     frame.resize(size);
00094     memcpy(frame.data(), cvFrame.data, size);
00095     image.resize(size);
00096     memcpy(image.data(), cvImage.data, size);
00097 }
00098
00099 // フレームがキャプチャされたことを記録する
00100 captured = true;
00101
00102 // カメラが使える
00103 return true;
00104 }
00105
00109 void capture()
00110 {
00111     // 再生開始時刻
00112     auto startTime{ glfwGetTime() };
00113
00114     // スレッドが実行可の間
00115     while (running)
00116     {
00117         // フレームを取り出せたら true
00118         auto status{ (total <= 0.0 || camera.get(cv::CAP_PROP_POS_FRAMES) < out) && camera.grab() };
00119
00120         // ムービーファイルでないかムービーファイルの終端でなければ次のフレームを取り出して
00121         if (status && camera.retrieve(cvFrame))
00122         {
00123             // 一時メモリをロックしてから
00124             std::lock_guard<std::mutex> lock{ mtx };
00125
00126             // キャプチャしたデータを一時メモリにコピーして
00127             cvFrame.copyTo(cvImage);
00128
00129             // 基底クラスのバッファとメンバを更新
00130             width = cvFrame.cols;
00131             height = cvFrame.rows;
00132             channels = cvFrame.channels();
00133             {
00134                 const size_t size = cvFrame.total() * cvFrame.elemSize();
00135                 frame.resize(size);
00136                 memcpy(frame.data(), cvFrame.data, size);
00137                 image.resize(size);
00138                 memcpy(image.data(), cvImage.data, size);
00139             }
00140
00141             // 新しいフレームがキャプチャされたことを通知する
00142             captured = true;
00143         }
00144
00145         // 遅延時間
00146         auto deferred{ 0.0 };
00147
00148         // ムービーファイルから入力しているとき
00149         if (total > 0.0)
00150         {
00151             // ムービーファイルの終端に到達していたら
00152             if (!status)
00153             {
00154                 // インポイントまで巻き戻す
00155                 camera.set(cv::CAP_PROP_POS_FRAMES, in);
00156
00157                 // 経過時間を戻す
00158                 elapsedTime = 0.0;
00159
00160                 // 開始時間を更新する
00161                 startTime = glfwGetTime();
00162             }
00163             else
00164             {
00165                 // 現在のフレームのインポイントからの経過時間
00166                 const auto pos{ camera.get(cv::CAP_PROP_POS_MSEC) - in };
00167
00168                 // 再生位置の次のフレームの時刻に対する経過時間
00169                 const auto now{ (elapsedTime + glfwGetTime() - startTime) * 1000.0 + interval };
00170
00171                 // 遅延時間はフレームの経過時間と現在の経過時間の差
00172                 deferred = pos - now;
00173             }
00174         }
00175
00176         // 遅延時間あれば

```

```

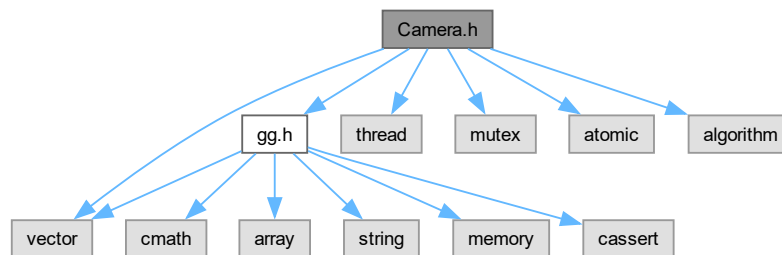
00177         if (deferred > 0.0)
00178         {
00179             // 待つ
00180             std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<int>(deferred)));
00181         }
00182     }
00183
00184     // 再生停止までの経過時間を積算する
00185     elapsedTime += glfwGetTime() - startTime;
00186 }
00187
00188 public:
00189
00190     CamCv() = default;
00191
00192     virtual ~CamCv()
00193     {
00194     }
00195
00196     auto open(int device, int width = 0, int height = 0, double fps = 0.0, const char* fourcc = "", int
00197     pref = cv::CAP_ANY)
00198     {
00199         // カメラを開いて初期化する
00200         return camera.open(device, pref) && init(width, height, fps, fourcc);
00201     }
00202
00203     void close()
00204     {
00205         // キャプチャデバイスを開放する
00206         camera.release();
00207
00208         // キャプチャデバイスを閉じる
00209         Camera::close();
00210     }
00211
00212     auto open(const std::string& file, int width = 0, int height = 0, double fps = 0.0, const char*
00213     fourcc = "", int pref = cv::CAP_ANY)
00214     {
00215         // ファイル／ネットワークを開いて初期化する
00216         return camera.open(file, pref) && init(width, height, fps, fourcc);
00217     }
00218
00219     virtual double getFps() const
00220     {
00221         return camera.get(cv::CAP_PROP_FPS);
00222     }
00223
00224     auto getCodec() const
00225     {
00226         return static_cast<unsigned int>(camera.get(cv::CAP_PROP_FOURCC));
00227     }
00228
00229     void getCodec(char* fourcc) const
00230     {
00231         auto cc{ getCodec() };
00232         for (int i = 0; i < 4; ++i)
00233         {
00234             fourcc[i] = static_cast<char>(cc & 0x7f);
00235             if (!isalnum(fourcc[i])) fourcc[i] = '?';
00236             cc >>= 8;
00237         }
00238     }
00239
00240     auto getPosition() const
00241     {
00242         return camera.get(cv::CAP_PROP_POS_FRAMES);
00243     }
00244
00245     void setPosition(double frame)
00246     {
00247         camera.set(cv::CAP_PROP_POS_FRAMES, frame);
00248     }
00249
00250     void setExposure(double exposure)
00251     {
00252         if (camera.isOpened()) camera.set(cv::CAP_PROP_EXPOSURE, exposure);
00253     }
00254
00255     void increaseExposure()
00256     {
00257         setExposure(++exposure * 0.1);
00258     }
00259
00260     void decreaseExposure()
00261     {
00262         setExposure(--exposure * 0.1);
00263     }

```

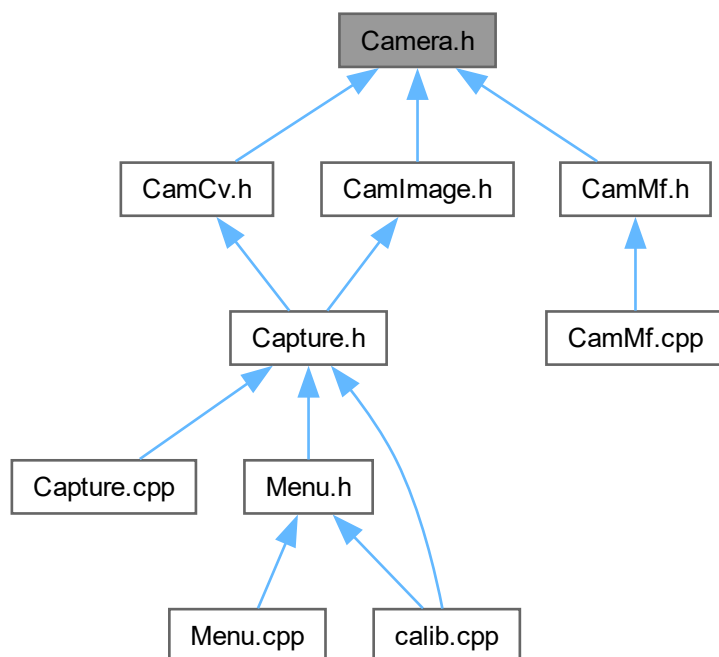
```
00329
00335 void setGain(double gain)
00336 {
00337     if (camera.isOpened()) camera.set(cv::CAP_PROP_GAIN, gain);
00338 }
00339
00343 void increaseGain()
00344 {
00345     setGain(++gain);
00346 }
00347
00351 void decreaseGain()
00352 {
00353     setGain(--gain);
00354 }
00355 };
```

9.13 Camera.h ファイル

```
#include "gg.h"
#include <vector>
#include <thread>
#include <mutex>
#include <atomic>
#include <algorithm>
Camera.h の依存先関係図:
```



被依存関係図:



クラス

- struct [CaptureFormat](#)
- class [Camera](#)

9.13.1 詳解

キャプチャデバイス関連の基底クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Camera.h](#) に定義があります。

9.14 Camera.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 #include <vector>
00014
00015 // 非同期処理
00016 #include <thread>
00017 #include <mutex>
00018 #include <atomic>
00019 #include <algorithm>
00020
00029 struct CaptureFormat
00030 {
00031     std::string resolution;
00032     std::string fps;
00033     std::string codec;
00034     int index{ 0 };
00035
00044     CaptureFormat(const std::string& resolution, const std::string& fps,
00045                   const std::string& codec, int index)
00046         : resolution{ resolution }
00047         , fps{ fps }
00048         , codec{ codec }
00049         , index{ index }
00050     {
00051     }
00052 };
00053
00057 class Camera
00058 {
00059 protected:
00060
00062     double total{ -1.0 };
00063
00065     double interval{ 10.0 };
00066
00068     int width{ 0 };
00069
00071     int height{ 0 };
00072
00074     int channels{ 0 };
00075
00077     std::vector<GLubyte> frame;
00078
00080     std::vector<GLubyte> image;
00081
00083     std::atomic<bool> prioritizeLatency{ false };
00084
00086     std::atomic<bool> captured{ false };
00087
00089     bool reusableFrame{ false };
00090
00092     std::thread thr;
00093
00095     std::mutex mtx;
00096
00098     std::atomic<bool> running{ false };
00099
00106     virtual void capture()
00107     {
00108         // 継承されていなければスレッドを起動しない
00109         stop();
00110     }
00111
00112 public:
00113
00115     double in{ -1.0 };
00116
00118     double out{ -1.0 };
00119
00123     Camera() = default;
00124
00130     Camera(const Camera& camera) = delete;
00131
00135     virtual ~Camera()
00136     {
00137         // キャプチャスレッドを停止する
00138         stop();
00139
00140         // キャプチャデバイスの使用を終了する

```

```

00141     close();
00142
00143     // 一時メモリを空にする
00144     image.clear();
00145 }
00146
00153 Camera& operator=(const Camera& camera) = delete;
00154
00158 void start()
00159 {
00160     // スレッドが起動状態であることを記録しておく
00161     running = true;
00162
00163     // スレッドを起動する
00164     thr = std::thread([&] { this->capture(); });
00165 }
00166
00170 virtual void stop()
00171 {
00172     // キャプチャスレッドが実行中なら
00173     if (running)
00174     {
00175         // キャプチャスレッドのループを止めて
00176         running = false;
00177
00178         // 合流する
00179         thr.join();
00180     }
00181 }
00182
00189 bool transmit(GLuint buffer)
00190 {
00191     // 新しいフレームが取得されているときカメラのロックが成功したら
00192     std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00193     if (lock.owns_lock() && captured)
00194     {
00195         // データの長さを計算して
00196         const auto expected_length{ static_cast<size_t>(width) * height * channels };
00197         const auto length{ std::min(image.size(), expected_length) * sizeof(GLubyte) };
00198
00199         // フレームをピクセルバッファオブジェクトに転送して
00200         glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer);
00201         glBufferSubData(GL_PIXEL_PACK_BUFFER, 0, length, image.data());
00202         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00203
00204         // 動画・カメラ入力では次のフレームを待つ。静止画像は表示方式を
00205         // 切り替えた後も同じ内容を再転送できるよう取得済みの状態を維持する。
00206         if (!reusableFrame) captured = false;
00207         return true;
00208     }
00209
00210     // カメラがロックできなかった
00211     return false;
00212 }
00213
00219 void transmit(std::vector<GLubyte>& buffer)
00220 {
00221     // 新しいフレームが取得されているときカメラのロックが成功したら
00222     std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00223     if (lock.owns_lock() && captured)
00224     {
00225         // データの長さを計算して
00226         const auto length{ image.size() };
00227
00228         // 呼び出し先のサイズを合わせてから
00229         buffer.resize(length);
00230
00231         // フレームを呼び出し元にコピーして
00232         memcpy(buffer.data(), image.data(), length);
00233
00234         // 静止画像でなければ次のフレームの取得を待つ
00235         if (!reusableFrame) captured = false;
00236     }
00237 }
00238
00245 template <typename MatType>
00246 bool transmit(MatType& buffer)
00247 {
00248     // 新しいフレームが取得されているときカメラのロックが成功したら
00249     std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00250     if (lock.owns_lock() && captured)
00251     {
00252         // 呼び出し元にコピーして
00253         // cv::Mat の型番号 (CV_8UC1~CV_8UC4) は (channels - 1) << 3 で表されます
00254         buffer.create(height, width, ((channels - 1) << 3));
00255         const auto expected_length{ static_cast<size_t>(width) * height * channels };
00256         memcpy(buffer.data, image.data(), std::min(image.size(), expected_length));

```



```

00257
00258     // 静止画像でなければ次のフレームの取得を待つ
00259     if (!reusableFrame) captured = false;
00260     return true;
00261 }
00262
00263     // カメラがロックできなかった
00264     return false;
00265 }
00266
00270 virtual void close()
00271 {
00272     // フレームを取得していないことにする
00273     captured = false;
00274 }
00275
00281 bool isRunning() const
00282 {
00283     return running;
00284 }
00285
00291 void setPrioritizeLatency(bool mode)
00292 {
00293     prioritizeLatency = mode;
00294 }
00295
00301 bool getPrioritizeLatency() const
00302 {
00303     return prioritizeLatency;
00304 }
00305
00311 std::array<int, 2> getSize() const
00312 {
00313     return std::array<int, 2>{ width, height };
00314 }
00315
00321 auto getWidth() const
00322 {
00323     return width;
00324 }
00325
00331 auto getHeight() const
00332 {
00333     return height;
00334 }
00335
00341 auto getChannels() const
00342 {
00343     return channels;
00344 }
00345
00351 auto getFrames() const
00352 {
00353     return total;
00354 }
00355
00361 virtual double getFps() const
00362 {
00363     return 1000.0 / interval;
00364 }
00365
00369 virtual void increaseExposure()
00370 {
00371 }
00372
00376 virtual void decreaseExposure()
00377 {
00378 }
00379
00383 virtual void increaseGain()
00384 {
00385 }
00386
00390 virtual void decreaseGain()
00391 {
00392 }
00393 };

```

9.15 CamImage.h ファイル

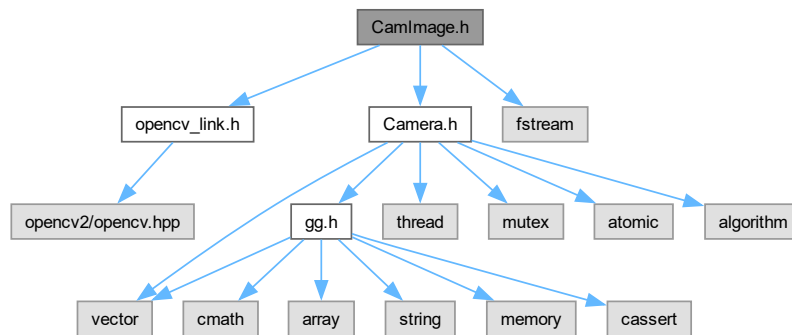
```

#include "opencv_link.h"
#include "Camera.h"

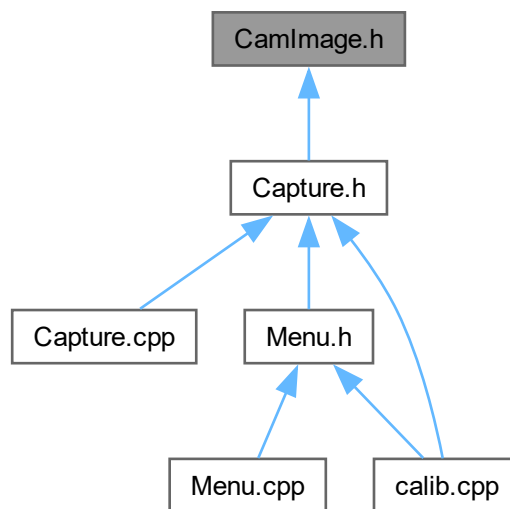
```

```
#include "fstream"
```

CamImage.h の依存先関係図:



被依存関係図:



クラス

- class [CamImage](#)

9.15.1 詳解

OpenCV を使って画像ファイルを読み込むクラス

著者

Kohe Tokoi

日付

November 15, 2022

[CamImage.h](#) に定義があります。

9.16 CamImage.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // OpenCV へのリンクとインクルード
00012 #include "opencv_link.h"
00013
00014 // カメラ関連の処理
00015 #include "Camera.h"
00016
00017 // ファイル入出力
00018 #include "fstream"
00019
00023 class CamImage : public Camera
00024 {
00025 public:
00026
00030 CamImage()
00031 {
00032 }
00033
00040 CamImage(std::string& filename, bool flip = false)
00041 {
00042     open(filename, flip);
00043 }
00044
00048 virtual ~CamImage()
00049 {
00050 }
00051
00058 bool open(const std::string& filename, bool flip = false)
00059 {
00060     // 画像ファイルを読み込む
00061     cv::Mat cvFrame;
00062     if (!load(filename, cvFrame)) return false;
00063
00064     // 必要なら上下を反転する
00065     if (flip) cv::flip(cvFrame, cvFrame, 1);
00066
00067     // 読み出したデータを一時メモリにコピーする
00068     cv::Mat cvImage;
00069     cvFrame.copyTo(cvImage);
00070
00071     // 基底クラスのバッファとメンバを更新
00072     width = cvFrame.cols;
00073     height = cvFrame.rows;
00074     channels = cvFrame.channels();
00075     {
00076         const size_t size = cvFrame.total() * cvFrame.elemSize();
00077         frame.resize(size);
00078         memcpy(frame.data(), cvFrame.data, size);
00079         image.resize(size);
00080         memcpy(image.data(), cvImage.data, size);
00081     }
00082
00083     // 静止画像は表示方式を切り替えた後も同じフレームを再利用する
00084     reusableFrame = true;
00085
00086     // 画像が読み込まれたことを記録する
00087     captured = true;
00088
00089     // 画像ファイルが開けた
00090     return true;
```

```

00091     }
00092
00098     bool isOpened() const
00099     {
00100         // 画像が読み込めていたら true
00101         return !image.empty();
00102     }
00103
00111     static bool load(const std::string& filename, cv::Mat& frame)
00112     {
00113         // 画像ファイルをバイナリモードで開いて読み込み位置を最後に移動する
00114         std::ifstream file(Utf8ToTChar(filename),
00115             std::ifstream::in |
00116             std::ifstream::binary |
00117             std::ifstream::ate);
00118
00119         // 画像ファイルが開けたら
00120         if (file.is_open())
00121         {
00122             // 画像ファイルを読み込むメモリを確保する
00123             std::vector<char> buffer(static_cast<std::vector<char>::size_type>(file.tellg()));
00124
00125             // 画像ファイルを先頭から全部読み込む
00126             file.seekg(0, std::ifstream::beg);
00127             file.read(buffer.data(), buffer.size());
00128
00129             // 画像ファイルを閉じる
00130             file.close();
00131
00132             // 画像ファイルが読み込めたら
00133             if (file.good())
00134             {
00135                 // 読み込んだ画像データを復号して返す
00136                 frame = cv::imdecode(buffer, cv::IMREAD_COLOR);
00137                 return true;
00138             }
00139         }
00140
00141         // 読み込めなかった
00142         return false;
00143     }
00144
00152     static bool save(const std::string& filename, const cv::Mat& image)
00153     {
00154         // ファイル名の拡張子の場所を取り出す
00155         const auto pos{ filename.find_last_of('.') };
00156
00157         // 拡張子があればそれを取り出し、無ければ ".png" にする
00158         const std::string ext{ pos != std::string::npos ? filename.substr(pos) : ".png" };
00159
00160         // 画像の符号化に失敗したら戻る
00161         std::vector<uchar> buffer;
00162         if (!cv::imencode(ext, image, buffer)) return false;
00163
00164         // 画像ファイルをバイナリモードで開く
00165         std::ofstream file(Utf8ToTChar(filename),
00166             std::ofstream::out |
00167             std::ofstream::binary);
00168
00169         // 画像ファイルが開けなかったら戻る
00170         if (file.fail()) return false;
00171
00172         // 画像データをファイルに書き込む
00173         const auto ptr{ reinterpret_cast<char*>(buffer.data()) };
00174         file.write(ptr, buffer.size());
00175
00176         // 画像ファイルを閉じる
00177         file.close();
00178
00179         // 読み込めたかどうかを返す
00180         return file.good();
00181     }
00182 };

```

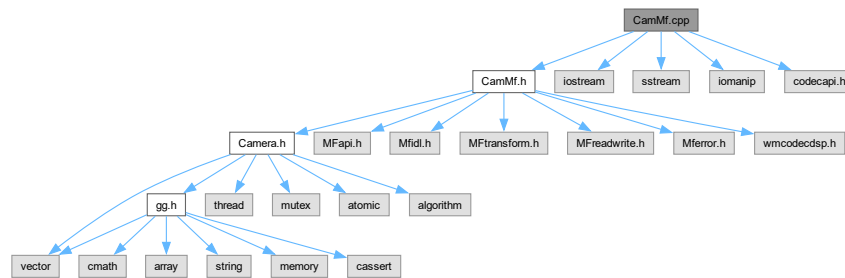
9.17 CamMf.cpp ファイル

```

#include "CamMf.h"
#include <iostream>
#include <sstream>
#include <iomanip>

```

#include <codecapi.h>
CamMf.cpp の依存先関係図:



関数

- template<class T>
void [SafeRelease](#) (T **ppT)
- std::string [SubTypeToName](#) (const GUID &subType)

9.17.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.cpp](#) に定義があります。

9.17.2 関数詳解

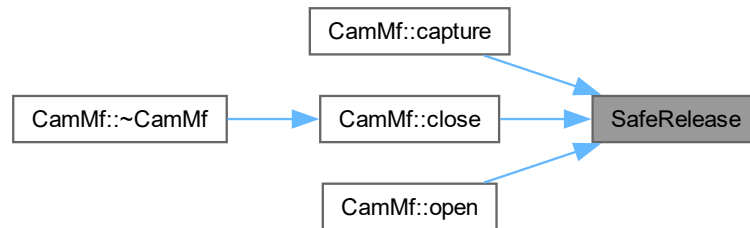
9.17.2.1 SafeRelease()

```

template<class T>
void SafeRelease (
    T ** ppT)
  
```

[CamMf.cpp](#) の 27 行目に定義があります。

被呼び出し関係図:



9.17.2.2 SubTypeToName()

```
std::string SubTypeToName (
    const GUID & subType)
```

CamMf.cpp の 39 行目に定義があります。

9.18 CamMf.cpp

[詳解]

```

00001
00008 #include "CamMf.h"
00009 #include <iostream>
00010 #include <sstream>
00011 #include <iomanip>
00012 #include <codecapi.h>
00013
00014 // Microsoft Media Foundation
00015 #pragma comment(lib, "MF.lib")
00016 #pragma comment(lib, "MFplat.lib")
00017 #pragma comment(lib, "MFuuid.lib")
00018 #pragma comment(lib, "MFreadwrite.lib")
00019 #pragma comment(lib, "wmcodecdspuuid.lib")
00020
00021 // COM ライブラリの初期化と終了を行うオブジェクト
00022 CamMf::ComInitializer CamMf::ComInitializer::instance;
00023
00024 //
00025 // メモリの解放
00026 //
00027 template <class T> void SafeRelease(T** ppT)
00028 {
00029     if (*ppT)
00030     {
00031         (*ppT)->Release();
00032         *ppT = nullptr;
00033     }
00034 }
00035
00036 //
00037 // GUID から人間が読める形式の名前を返すヘルパー関数
00038 //
00039 std::string SubTypeToName(const GUID& subType)
00040 {
00041     if (subType == MFVideoFormat_YUY2) return "YUY2";
00042     if (subType == MFVideoFormat_NV12) return "NV12";
00043     if (subType == MFVideoFormat_MJPEG) return "MJPG";
00044     if (subType == MFVideoFormat_H264) return "H264";
00045 }
```

```

00046 // TODO: 他に使用するフォーマットがあればここに追加
00047 //if (subType == MFVideoFormat_RGB24) return "RGB24";
00048 if (subType == MFVideoFormat_RGB32) return "RGB32";
00049 return "";
00050 }
00051 }
00052 //
00053 //
00054 // COM ライブラリの初期化と終了を行うクラスのコンストラクタ
00055 //
00056 CamMf::ComInitializer::ComInitializer() = default;
00057 //
00058 //
00059 // COM ライブラリの初期化と終了を行うクラスのデストラクタ
00060 //
00061 CamMf::ComInitializer::~ComInitializer()
00062 {
00063 // メディアソースのリストを取得していれば
00064 if (ppSourceActivate)
00065 {
00066 // すべてのメディアソースを解放して
00067 for (DWORD i = 0; i < cSourceActivate; ++i) SafeRelease(&ppSourceActivate[i]);
00068 // メディアソースのリストに使ったメモリを解放して
00069 CoTaskMemFree(ppSourceActivate);
00070 ppSourceActivate = nullptr;
00071 // ビデオキャプチャデバイスの表示名のリストを空にする
00072 deviceList.clear();
00073 }
00074 // Media Foundation が起動していればシャットダウンする
00075 if (mfStarted) MFShutdown();
00076 // COM ライブラリが初期化されていれば終了する
00077 if (coInitialized) CoUninitialize();
00078 }
00079 //
00080 // 初期化
00081 //
00082 const char* CamMf::ComInitializer::initialize()
00083 {
00084 // エラーメッセージ
00085 const char* message{ nullptr };
00086 // 検索条件を保持する属性ストア
00087 IMFAttributes* pAttributes{ nullptr };
00088 // COM ライブラリを初期化する
00089 if (FAILED(CoInitializeEx(nullptr, COINIT_MULTITHREADED)))
00090 {
00091 // COM ライブラリの初期化に失敗した
00092 message = "Failed to initialize COM library.";
00093 goto done;
00094 }
00095 // COM ライブラリの初期化に成功した
00096 coInitialized = true;
00097 // Media Foundation を起動する
00098 if (FAILED(MFStartup(MF_VERSION, MFSTARTUP_FULL)))
00099 {
00100 // Media Foundation の起動に失敗した
00101 message = "Failed to start Media Foundation.";
00102 goto done;
00103 }
00104 // Media Foundation の起動に成功した
00105 mfStarted = true;
00106 // 検索条件を保持する属性ストアを作成する
00107 if (FAILED(MFCreateAttributes(&pAttributes, 1)))
00108 {
00109 // 属性ストアの作成失敗した
00110 message = "Failed to create attribute store.";
00111 goto done;
00112 }
00113 // 属性ストアにビデオキャプチャデバイスの属性を設定する
00114 if (FAILED(pAttributes->SetGUID(
00115 MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE,
00116 MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE_VIDCAP_GUID)))
00117 {
00118 // ビデオキャプチャデバイスの属性の設定に失敗した
00119 message = "Failed to set attribute for video capture device.";
00120 goto done;
00121 }
00122 }

```

```

00133     }
00134
00135     // メディアソースを列挙する
00136     if (FAILED(MFEnumDeviceSources(pAttributes,
00137         &ppSourceActivate, &cSourceActivate)))
00138     {
00139         // メディアソースの列挙に失敗した
00140         message = "Failed to enumerate media sources.";
00141         goto done;
00142     }
00143
00144     // すべてのメディアソースについて
00145     for (DWORD i = 0; i < cSourceActivate; ++i)
00146     {
00147         // メディアソースの表示名の文字列ポインタ
00148         WCHAR* szFriendlyName{ nullptr };
00149
00150         // メディアソースの表示名の文字数
00151         UINT32 cFriendlyName{ 0 };
00152
00153         // メディアソースの表示名を取得する
00154         if (SUCCEEDED(ppSourceActivate[i]->GetAllocatedString(
00155             MF.DEVSOURCE_ATTRIBUTE_FRIENDLY_NAME, &szFriendlyName, &cFriendlyName)))
00156         {
00157             // ビデオキャプチャデバイス名を作る
00158             std::stringstream ss;
00159             ss << TCharToUtf8(szFriendlyName) << "##" << i;
00160
00161             // ビデオキャプチャデバイス名をリストに追加する
00162             deviceList.emplace_back(ss.str());
00163         }
00164
00165         // 表示名の文字列に使ったメモリを解放する
00166         CoTaskMemFree(szFriendlyName);
00167     }
00168
00169 done:
00170
00171     // 属性ストアはもう使わないので解放する
00172     SafeRelease(&pAttributes);
00173
00174     // エラーメッセージを返す
00175     return message;
00176 }
00177
00178 //
00179 // COM ライブラリのシングルトンインスタンスを返す
00180 //
00181 const CamMf::ComInitializer& CamMf::ComInitializer::GetInstance()
00182 {
00183     // COM ライブラリが初期化され Media Foundation が起動されていなければ
00184     if (!instance.ppSourceActivate)
00185     {
00186         // COM ライブラリを初期化して Media Foundation を起動する
00187         auto message{ instance.initialize() };
00188
00189         // COM ライブラリの初期化と Media Foundation の起動に失敗したら例外を投げる
00190         if (message) throw std::runtime_error(message);
00191     }
00192
00193     // COM ライブラリのシングルトンインスタンスへの参照を返す
00194     return instance;
00195 }
00196
00197 //
00198 // 有効化
00199 //
00200 bool CamMf::ComInitializer::activate(int device, IMFMediaSource** pMediaSource)
00201 {
00202     // メディアソースを作成して結果を返す
00203     return device >= 0 && static_cast<UINT32>(device) < instance.cSourceActivate
00204         && SUCCEEDED(instance.ppSourceActivate[device]->ActivateObject(IID_PPV_ARGS(pMediaSource)))
00205         && pMediaSource;
00206 }
00207
00208 //
00209 // ビデオキャプチャデバイスの表示名のリストを返す
00210 //
00211 const std::vector<std::string>& CamMf::ComInitializer::getDeviceList()
00212 {
00213     // ビデオキャプチャデバイスの表示名のリストを返す
00214     return getInstance().deviceList;
00215 }
00216
00217 //
00218 // 使用可能な解像度、フレームレート、コーデックのリストを作成する
00219 //

```



```

00220 bool CamMf::enumerateFormats()
00221 {
00222     // Source Reader が作成されていなければ戻る
00223     if (!pSourceReader) return false;
00224
00225     // 以前に作成したリストをクリアする
00226     availableFormats.clear();
00227     formatList.clear();
00228
00229     // ネイティブメディアタイプを一つずつ列挙する
00230     IMFMediaType* pMediaType{ nullptr };
00231     for (DWORD dwMediaTypeIndex = 0;
00232          SUCCEEDED(pSourceReader->GetNativeMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00233              dwMediaTypeIndex, &pMediaType));
00234          ++dwMediaTypeIndex
00235      )
00236      {
00237          // スコープを抜けるときにメディアタイプを解放する
00238          struct MediaTypeReleaser { IMFMediaType*& p; ~MediaTypeReleaser() { SafeRelease(&p); } } releaser{
00239              pMediaType };
00240
00241          // メディアタイプを取得する
00242          GUID majorType{};
00243          if (FAILED(pMediaType->GetGUID(MF_MT_MAJOR_TYPE, &majorType))) continue;
00244
00245          // メディアタイプがビデオで無ければ次へ
00246          if (majorType != MFMediaType_Video) continue;
00247
00248          // ビデオフォーマットを取得する
00249          GUID subType{};
00250          if (FAILED(pMediaType->GetGUID(MF_MT_SUBTYPE, &subType))) continue;
00251
00252          // 解像度を取得する
00253          UINT32 width{ 0 }, height{ 0 };
00254          if (FAILED(MFGetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, &width, &height))) continue;
00255
00256          // フレームレートを取得する
00257          UINT32 numerator{ 0 }, denominator{ 0 };
00258          if (FAILED(MFGetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, &numerator, &denominator))) continue;
00259
00260          // コーデックの文字列を取り出す
00261          const auto& codecName{ SubTypeToName(subType) };
00262
00263          // 対応できないコーデックかフレームレートに問題があれば次へ
00264          if (codecName.empty() || denominator == 0 || numerator == 0) continue;
00265
00266          // フレームレートを求める
00267          const double fps{ static_cast<double>(numerator) / static_cast<double>(denominator) };
00268
00269          // フレームレートが 5 未満なら次へ
00270          if (fps < 5.0) continue;
00271
00272          // UI が Media Foundation 固有の型を扱わずに済むよう、解像度と fps を表示文字列に変換する
00273          std::ostringstream resolution;
00274          resolution << width << " x " << height;
00275          std::ostringstream frameRate;
00276          frameRate << std::fixed << std::setprecision(2) << fps;
00277
00278          // availableFormats と同じ並び順の選択番号を関連付けて UI 用リストへ追加する
00279          formatList.push_back({ resolution.str(), frameRate.str(), codecName,
00280              static_cast<int>(availableFormats.size()) });
00281
00282          // 使用可能なビデオフォーマットのリストに追加する
00283          availableFormats.emplace_back(width, height, numerator, denominator, subType);
00284      }
00285
00286     // リストが空でなければ成功
00287     return !formatList.empty();
00288 }
00289
00290 // 指定されたサブタイプに対応するビデオデコーダを探す
00291 //
00292 HRESULT CamMf::findVideoDecoder(
00293     const GUID& subtype,
00294     IMFTransform** ppDecoder,
00295     BOOL bAllowAsync,
00296     BOOL bAllowHardware,
00297     BOOL bAllowTranscode
00298 ) const
00299 {
00300     // 結果
00301     HRESULT hr{ S_OK };
00302
00303     // 見つかったデコーダの数
00304     UINT32 count{ 0 };
00305

```

```

00306 // デコーダのアクティベーションオブジェクトのリスト
00307 IMFActivate** ppActivate{ nullptr };
00308
00309 // 検索条件
00310 MFT_REGISTER_TYPE_INFO inputInfo{ MFMediaType_Video, subtype };
00311 MFT_REGISTER_TYPE_INFO outputInfo{ MFMediaType_Video, MFVideoFormat_NV12 };
00312
00313 // 列挙のフラグ
00314 UINT32 unFlags
00315 {
00316     MFT_ENUM_FLAG_SYNCMFT
00317     | MFT_ENUM_FLAG_LOCALMFT
00318     | MFT_ENUM_FLAG_SORTANDFILTER
00319 };
00320
00321 // 非同期デコードも含めるなら
00322 if (bAllowAsync) unFlags |= MFT_ENUM_FLAG_ASYNCMFT;
00323
00324 // ハードウェアデコードも含めるなら
00325 if (bAllowHardware) unFlags |= MFT_ENUM_FLAG_HARDWARE;
00326
00327 // トランスコード専用のデコードも含めるなら
00328 if (bAllowTranscode) unFlags |= MFT_ENUM_FLAG_TRANSCODE_ONLY;
00329
00330 // 条件に合う MFT のアクティベーションオブジェクトを列挙する
00331 hr = MFTEnumEx(MFT_CATEGORY_VIDEO_DECODER,
00332     unFlags, &inputInfo, &outputInfo, &ppActivate, &count);
00333
00334 // デコーダが見つからなかったらエラー
00335 if (SUCCEEDED(hr) && count == 0) hr = MF_E_TOPO_CODEC_NOT_FOUND;
00336
00337 // リストの最初のデコーダをインスタンス化する
00338 if (SUCCEEDED(hr)) hr = ppActivate[0]->ActivateObject(IID_PPV_ARGS(ppDecoder));
00339
00340 // デコーダのアクティベーションオブジェクトのリストを解放する
00341 for (UINT32 i = 0; i < count; i++) ppActivate[i]->Release();
00342
00343 // デコーダのアクティベーションオブジェクトのリストに使ったメモリを解放する
00344 CoTaskMemFree(ppActivate);
00345
00346 // 結果を返す
00347 return hr;
00348 }
00349
00350 //
00351 // MFT のセットアップと接続を行う
00352 //
00353 HRESULT CamMf::setUpPipeline(IMFTransform* pTransform, const VideoFormat& format, const GUID& subType)
00354 {
00355     #if defined(_DEBUG)
00356         std::cerr << SubTypeToName(format.subType) << " -> " << SubTypeToName(subType) << std::endl;
00357     #endif
00358
00359     // 入力と出力のメディアタイプ
00360     IMFMediaType* pInputType{ nullptr };
00361     IMFMediaType* pOutputType{ nullptr };
00362
00363     // 結果
00364     HRESULT hr{ S_OK };
00365
00366     // MFT の入力タイプの設定
00367     hr = MFCreateMediaType(&pInputType);
00368     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00369     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_SUBTYPE, format.subType);
00370     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pInputType, MF_MT_FRAME_SIZE, format.width,
format.height);
00371     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pInputType, MF_MT_FRAME_RATE, format.fpsNum,
format.fpsDenom);
00372
00373     // MFT に入力タイプを設定する
00374     if (SUCCEEDED(hr)) hr = pTransform->SetInputType(0, pInputType, 0);
00375     if (FAILED(hr)) goto done;
00376
00377     // MFT の出力タイプの設定
00378     hr = MFCreateMediaType(&pOutputType);
00379     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00380     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_SUBTYPE, subType);
00381     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pOutputType, MF_MT_FRAME_SIZE, format.width,
format.height);
00382     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pOutputType, MF_MT_FRAME_RATE, format.fpsNum,
format.fpsDenom);
00383
00384     // MFT に出力タイプを設定する
00385     if (SUCCEEDED(hr)) hr = pTransform->SetOutputType(0, pOutputType, 0);
00386     if (FAILED(hr)) goto done;
00387

```

```

00388 // MFT をアクティブにする
00389 hr = pTransform->ProcessMessage(MFT_MESSAGE_COMMAND_FLUSH, NULL);
00390 if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_START_OF_STREAM, NULL);
00391 if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_BEGIN_STREAMING, NULL);
00392
00393 done:
00394
00395 // メディアタイプの取得に使ったメモリを解放する
00396 SafeRelease(&pInputType);
00397 SafeRelease(&pOutputType);
00398
00399 // 結果を返す
00400 return hr;
00401 }
00402
00403 //
00404 // MFT を解放する
00405 //
00406 void CamMf::cleanUpTransform(IMFTransform** pTransform) const
00407 {
00408 // MFT が作成されていれば
00409 if (*pTransform)
00410 {
00411 // MFT のストリーミングの終了を通知する
00412 (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_STREAMING, NULL);
00413 (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_OF_STREAM, NULL);
00414
00415 // MFT を解放する
00416 (*pTransform)->Release();
00417 *pTransform = nullptr;
00418 }
00419 }
00420
00421 //
00422 // デコーダの出力バッファを作成する
00423 //
00424 HRESULT CamMf::createDecoderBuffer()
00425 {
00426 // デコーダの出力ストリーム情報を取得する
00427 MFT_OUTPUT_STREAM_INFO streamInfo{ 0 };
00428 if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo))) {}
00429
00430 // NV12 は通常 16 ピクセル単位に切り上げられたサイズなので幅を 16 の倍数に丸める
00431 const auto alignedWidth{ static_cast<UINT32>((width + 15) & ~15) };
00432 const auto alignedHeight{ static_cast<UINT32>((height + 15) & ~15) };
00433
00434 // NV12 フォーマットの理論上の必要バッファサイズを計算する
00435 const auto cbDecoderCalc{ static_cast<UINT32>((alignedWidth * alignedHeight * 3) / 2) };
00436
00437 // MFT が要求するサイズと理論計算値の大きい方を採用する
00438 const auto cbDecoder{ static_cast<UINT32>((streamInfo.cbSize > cbDecoderCalc) ? streamInfo.cbSize :
cbDecoderCalc) };
00439
00440 // MFT が要求するメモリアライメントを取得する
00441 const auto alignmentDecoder{ static_cast<DWORD>((streamInfo.cbAlignment > 0) ?
(streamInfo.cbAlignment - 1) : 63) };
00442
00443 // 現在の出力フォーマット用に確保されている出力バッファがあれば解放する
00444 SafeRelease(&pDecoderBuffer);
00445
00446 // デコーダの出力フレームを書き込むためのアライメント付きメモリバッファを作成する
00447 return MFCreatAlignedMemoryBuffer(cbDecoder, alignmentDecoder, &pDecoderBuffer);
00448 }
00449
00450 //
00451 // カラーコンバータの出力バッファを作成する
00452 //
00453 HRESULT CamMf::createConverterBuffer()
00454 {
00455 // カラーコンバータの出力ストリーム情報を取得する
00456 MFT_OUTPUT_STREAM_INFO convStreamInfo{ 0 };
00457 if (SUCCEEDED(pConverter->GetOutputStreamInfo(0, &convStreamInfo))) {}
00458
00459 // RGB32 フォーマットの理論上の必要バッファサイズを計算する
00460 UINT32 cbConverterCalc{ 0 };
00461 MFCalculateImageSize(MFVideoFormat_RGB32, width, height, &cbConverterCalc);
00462
00463 // MFT が要求するサイズと理論計算値の大きい方を採用する
00464 UINT32 cbConverter = (convStreamInfo.cbSize > cbConverterCalc) ? convStreamInfo.cbSize :
cbConverterCalc;
00465
00466 // MFT が要求するメモリアライメントを取得する
00467 DWORD alignmentConverter = (convStreamInfo.cbAlignment > 0) ? (convStreamInfo.cbAlignment - 1) : 63;
00468
00469 // 既存の出力バッファがあれば解放する
00470 SafeRelease(&pConverterBuffer);
00471

```

```

00472 // カラーコンバータが要求するサイズとアラインメントを満たす RGB32 用の出力バッファを作成する
00473 return MFCreateAlignedMemoryBuffer(cbConverter, alignmentConverter, &pConverterBuffer);
00474 }
00475 //
00476 //
00477 // Source Reader の出力フォーマットを設定し、基底クラスの frame を初期化する
00478 //
00479 bool CamMf::setFormat(int index)
00480 {
00481 // 使用可能なフォーマットのリストから選択されたフォーマットを取得する
00482 VideoFormat selectedFormat{ availableFormats[index] };
00483 // デコード方法
00484 enum class DecodeMethod { None = 0, Convert, Decode } decodeMethod
00485 {
00486     selectedFormat.subType == MFVideoFormat_MJPG ? DecodeMethod::Decode :
00487     selectedFormat.subType == MFVideoFormat_H264 ? DecodeMethod::Decode :
00488     selectedFormat.subType == MFVideoFormat_NV12 ? DecodeMethod::Convert :
00489     selectedFormat.subType == MFVideoFormat_YUY2 ? DecodeMethod::Convert :
00490     DecodeMethod::None
00491 };
00492 // デコーダとカラーコンバータを未設定にする
00493 cleanUpTransform(&pDecoder);
00494 cleanUpTransform(&pConverter);
00495 // 結果
00496 HRESULT hr{ S_OK };
00497 // 新しいメディアタイプを作成する
00498 IMFMediaType* pMediaType{ nullptr };
00499 hr = MFCreateMediaType(&pMediaType);
00500 if (FAILED(hr)) goto done;
00501 // メジャータイプにビデオを指定する
00502 hr = pMediaType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00503 if (FAILED(hr)) goto done;
00504 // ビデオフォーマットを指定する
00505 hr = pMediaType->SetGUID(MF_MT_SUBTYPE, selectedFormat.subType);
00506 if (FAILED(hr)) goto done;
00507 // 解像度を設定する
00508 hr = MFSetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, selectedFormat.width, selectedFormat.height);
00509 if (FAILED(hr)) goto done;
00510 // フレームレートを設定する
00511 hr = MFSetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, selectedFormat.fpsNum,
00512 selectedFormat.fpsDenom);
00513 if (FAILED(hr)) goto done;
00514 // Source Reader の出力メディアタイプを設定する
00515 hr = pSourceReader->SetCurrentMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM, nullptr, pMediaType);
00516 if (FAILED(hr)) goto done;
00517 // 基底クラスの解像度とチャンネル数を設定し、バッファサイズを設定する
00518 width = selectedFormat.width;
00519 height = selectedFormat.height;
00520 channels = 4;
00521 frame.resize(width * height * channels);
00522 image.resize(width * height * channels);
00523 // インターバルを計算する
00524 interval = (selectedFormat.fpsDenom != 0)
00525 ? (1000.0 * selectedFormat.fpsDenom / selectedFormat.fpsNum)
00526 : 10.0;
00527 // デコードまたはカラー変換が必要なとき
00528 if (decodeMethod != DecodeMethod::None)
00529 {
00530 // デコードが必要なら
00531 if (decodeMethod == DecodeMethod::Decode)
00532 {
00533 // 選択されたビデオフォーマットのデコーダを探す
00534 hr = findVideoDecoder(selectedFormat.subType, &pDecoder);
00535 if (FAILED(hr)) goto done;
00536 // デコーダから ICodecAPI インターフェイスの取得に成功したら
00537 ICodecAPI* pCodecAPI{ nullptr };
00538 if (SUCCEEDED(pDecoder->QueryInterface(IID_PPV_ARGS(&pCodecAPI))))
00539 {
00540 // コーデック設定値を渡すための VARIANT を初期化する
00541 VARIANT var;
00542 VariantInit(&var);
00543 // Low Latency Mode を VT_UI4 型で有効にする
00544 var.vt = VT_UI4;

```

```

00558
00559     // 1 = Low Latency Mode
00560     var.ulVal = 1;
00561
00562     // デコーダに Low Latency Mode を要求する
00563     hr = pCodecAPI->SetValue(&CODECAPI_AVLowLatencyMode, &var);
00564
00565     // VT_UI4 型で Low Latency Mode を有効にできなかったら
00566     if (FAILED(hr))
00567     {
00568         // Low Latency Mode を VT_BOOL 型で有効にする
00569         var.vt = VT_BOOL;
00570
00571         // VARIANT_TRUE = Low Latency Mode
00572         var.boolVal = VARIANT_TRUE;
00573
00574         // デコーダに Low Latency Mode を要求する
00575         pCodecAPI->SetValue(&CODECAPI_AVLowLatencyMode, &var);
00576     }
00577
00578     // VARIANT が保持しているリソースを解放する38
00579     VariantClear(&var);
00580
00581     // ICodecAPI インターフェイスを解放する
00582     SafeRelease(&pCodecAPI);
00583 }
00584
00585     // MFT デコーダのセットアップと接続を行う
00586     hr = setUpPipeline(pDecoder, selectedFormat, MFVideoFormat_NV12);
00587     if (FAILED(hr)) goto done;
00588
00589     // デコード後のビデオフォーマットは NV12 にしている
00590     selectedFormat.subType = MFVideoFormat_NV12;
00591
00592     // デコーダの出力バッファを作成する
00593     hr = createDecoderBuffer();
00594     if (FAILED(hr)) goto done;
00595 }
00596
00597     // MFT をインスタンス化する
00598     hr = CoCreateInstance(CLSID_CColorConvertDMO, nullptr, CLSCTX_INPROC_SERVER,
IID_PPV_ARGS(&pConverter));
00599     if (FAILED(hr)) goto done;
00600
00601     // MFT カラーコンバータのセットアップと接続を行う
00602     hr = setUpPipeline(pConverter, selectedFormat, MFVideoFormat_RGB32);
00603     if (FAILED(hr)) goto done;
00604
00605     // カラー変換用の出力バッファを作成する
00606     hr = createConverterBuffer();
00607     if (FAILED(hr)) goto done;
00608 }
00609
00610 done:
00611
00612     // メディアタイプはもう使わないので解放する
00613     SafeRelease(&pMediaType);
00614
00615     // 成功したら true を返す
00616     if (SUCCEEDED(hr)) return true;
00617
00618     // 失敗したときはカメラを閉じて
00619     close();
00620
00621     // false を返す
00622     return false;
00623 }
00624
00625 //
00626 // カメラを開く
00627 //
00628 bool CamMf::open(int device, bool setupFormat)
00629 {
00630     // メディアソースを作成する
00631     if (!ComInitializer::activate(device, &pMediaSource)) return false;
00632
00633     // 結果
00634     HRESULT hr{ S_OK };
00635
00636     // Source Reader の属性ストアを作成する
00637     IMFAttributes* pAttributes{ nullptr };
00638     hr = MFCreateAttributes(&pAttributes, 1);
00639     if (FAILED(hr)) goto done;
00640
00641     // Source Reader の属性ストアにデコード能力を設定する
00642     pAttributes->SetUINT32(MF_READWRITE_ENABLE_HARDWARE_TRANSFORMS, TRUE);
00643     pAttributes->SetUINT32(MF_READWRITE_DISABLE_CONVERTERS, FALSE);

```

```

00644 pAttributes->SetUINT32(MF_SOURCE_READER_ENABLE_VIDEO_PROCESSING, FALSE);
00645
00646 // Source Reader に低遅延モードを要求する
00647 pAttributes->SetUINT32(MF_LOW_LATENCY, TRUE);
00648
00649 // Source Reader の解放時に Media Source をシャットダウンするようにする
00650 pAttributes->SetUINT32(MF_SOURCE_READER_DISCONNECT_MEDIASOURCE_ON_SHUTDOWN, TRUE);
00651
00652 // Source Reader を作成する
00653 hr = MFCreateSourceReaderFromMediaSource(pMediaSource, pAttributes, &pSourceReader);
00654 if (FAILED(hr)) goto done;
00655
00656 // 属性ストアはもう必要ないので解放する
00657 SafeRelease(&pAttributes);
00658
00659 // 使用可能なフォーマットを列挙して最初のフォーマットを選択する
00660 if (enumerateFormats())
00661 {
00662     // 要求された場合のみ最初のフォーマットを設定する
00663     if (!setupFormat || setFormat(0)) return true;
00664 }
00665
00666 done:
00667
00668 // フォーマットの設定に失敗したら全てを解放して戻る
00669 SafeRelease(&pSourceReader);
00670 SafeRelease(&pAttributes);
00671 availableFormats.clear();
00672 formatList.clear();
00673
00674 return false;
00675 }
00676
00677 //
00678 // フォーマットを選択して設定する
00679 //
00680 bool CamMf::select(int index)
00681 {
00682     // カメラが開かれていないかインデックスが範囲外なら戻る
00683     if (!pSourceReader || index < 0 || index >= availableFormats.size()) return false;
00684
00685     // キャプチャスレッドが実行中であれば安全のために停止する
00686     stop();
00687
00688     // 選択されたフォーマットを設定する
00689     return setFormat(index);
00690 }
00691
00692 //
00693 // フレームをキャプチャする
00694 //
00695 void CamMf::capture()
00696 {
00697     // スレッドが実行可の間
00698     while (running)
00699     {
00700         // サンプルを読み出して
00701         DWORD dwStreamIndex{ 0 };
00702         DWORD dwStreamFlags{ 0 };
00703         LONGLONG llTimestamp{ 0 };
00704         IMFSample* pSample{ nullptr };
00705         if (FAILED(pSourceReader->ReadSample(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00706             0, &dwStreamIndex, &dwStreamFlags, &llTimestamp, &pSample))) continue;
00707
00708         // サンプルが取得できていなければ戻る
00709         if (!pSample) continue;
00710
00711         // サンプルから取り出したメディアバッファ
00712         IMFMediaBuffer* pBuffer{ nullptr };
00713
00714         // デコーダの出力フレームを保持するサンプル
00715         IMFSample* pDecodedSample{ nullptr };
00716
00717         // カラーコンバータの出力フレームを保持するサンプル
00718         IMFSample* pConvertedSample{ nullptr };
00719
00720         // ProcessOutput の呼び出しの状態
00721         DWORD dwStatus{ 0 };
00722
00723         // デコーダが設定されていれば (MJPG か H264 の場合)
00724         if (pDecoder)
00725         {
00726             // サンプルをデコーダに渡す
00727             HRESULT hr{ pDecoder->ProcessInput(0, pSample, 0) };
00728
00729             // デコーダへの入力に失敗したら
00730             if (FAILED(hr))

```

```

00731     {
00732     #if defined(_DEBUG)
00733         if (hr != MF_E_NOTACCEPTING)
00734         {
00735             std::cerr << "Decoder process input failed: ";
00736             switch (hr)
00737             {
00738             case E_INVALIDARG:
00739                 std::cerr << "Invalid argument." << std::endl; break;
00740             case E_UNEXPECTED:
00741                 std::cerr << "Unexpected error." << std::endl; break;
00742             case E_FAIL:
00743                 std::cerr << "Unspecified error." << std::endl; break;
00744             case MF_E_INVALIDSTREAMNUMBER:
00745                 std::cerr << "Invalid stream number." << std::endl; break;
00746             case MF_E_NO_SAMPLE_DURATION:
00747                 std::cerr << "No sample duration." << std::endl; break;
00748             case MF_E_NO_SAMPLE_TIMESTAMP:
00749                 std::cerr << "No sample timestamp." << std::endl; break;
00750             case MF_E_NOTACCEPTING:
00751                 std::cerr << "Not accepting input." << std::endl; break;
00752             case MF_E_TRANSFORM_TYPE_NOT_SET:
00753                 std::cerr << "Transform type not set." << std::endl; break;
00754             case MF_E_UNSUPPORTED_D3D_TYPE:
00755                 std::cerr << "Unsupported D3D type." << std::endl; break;
00756             default:
00757                 std::cerr << "Code: " << std::hex << hr << std::endl; break;
00758             }
00759         }
00760     #endif
00761     goto done;
00762     }
00763
00764     // デコーダの出力ストリーム情報を取得し、誰がサンプルを提供するべきかを判定する
00765     MFT_OUTPUT_STREAM_INFO streamInfo{};
00766     bool mftProvidesSamples{ false };
00767     if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo)))
00768     {
00769         mftProvidesSamples = (streamInfo.dwFlags & MFT_OUTPUT_STREAM_PROVIDES_SAMPLES) != 0;
00770     }
00771
00772     // デコーダから出力サンプルを取り出す
00773     IMFSample* pLatestDecodedSample{ nullptr };
00774     bool hasOutput{ false };
00775
00776     while (true)
00777     {
00778         DWORD prevLength = 0;
00779
00780         if (mftProvidesSamples)
00781         {
00782             // MFT が自前でサンプルを割り当てるので nullptr を渡す
00783             pDecodedSample = nullptr;
00784         }
00785         else
00786         {
00787             // 呼び出し側がサンプルを提供する
00788             hr = MFCreateSample(&pDecodedSample);
00789             if (FAILED(hr)) goto done;
00790
00791             // クラスメンバの pDecoderBuffer が有効であることを確認
00792             if (!pDecoderBuffer)
00793             {
00794                 hr = createDecoderBuffer();
00795                 if (FAILED(hr))
00796                 {
00797                     SafeRelease(&pDecodedSample);
00798                     goto done;
00799                 }
00800             }
00801
00802             // バッファの有効データ長を 0 に設定して、MFT が先頭から書き込めるようにする
00803             pDecoderBuffer->GetCurrentLength(&prevLength);
00804             pDecoderBuffer->SetCurrentLength(0);
00805
00806             hr = pDecodedSample->AddBuffer(pDecoderBuffer);
00807             if (FAILED(hr))
00808             {
00809                 SafeRelease(&pDecodedSample);
00810                 goto done;
00811             }
00812         }
00813
00814         MFT_OUTPUT_DATA_BUFFER decodedBuffer{ 0, pDecodedSample, 0, nullptr };
00815         hr = pDecoder->ProcessOutput(0, 1, &decodedBuffer, &dwStatus);
00816
00817         // ストリームのフォーマットが変化した場合の処理

```

```

00818         if (hr == MF_E_TRANSFORM_STREAM_CHANGE)
00819         {
00820             #if defined(_DEBUG)
00821                 std::cerr << "Transform stream change." << std::endl;
00822             #endif
00823             // 1. 利用可能な出力タイプから NV12 を探索・選択する
00824             IMFMediaType* pNewOutputType{ nullptr };
00825             GUID subtype{ 0 };
00826             DWORD dwTypeIndex = 0;
00827             bool foundNV12 = false;
00828             hr = S_OK; // hr を S_OK にリセット
00829             while (SUCCEEDED(pDecoder->GetOutputAvailableType(0, dwTypeIndex++, &pNewOutputType))
00830             {
00831                 GUID subType{};
00832                 if (SUCCEEDED(pNewOutputType->GetGUID(MF_MT_SUBTYPE, &subType)) && subType ==
MFVideoFormat_NV12)
00833                 {
00834                     foundNV12 = true;
00835                     subtype = subType;
00836                     #if defined(_DEBUG)
00837                         std::cerr << "Found NV12 output type at index " << (dwTypeIndex - 1) << std::endl;
00838                     #endif
00839                     break;
00840                 }
00841                 SafeRelease(&pNewOutputType);
00842             }
00843             // 見つからなかった場合はインデックス 0 を取得してサブタイプを NV12 に書きする
00844             if (!foundNV12)
00845             {
00846                 #if defined(_DEBUG)
00847                     std::cerr << "NV12 output type not found. Forcing fallback to index 0." << std::endl;
00848                 #endif
00849                 hr = pDecoder->GetOutputAvailableType(0, 0, &pNewOutputType);
00850                 if (SUCCEEDED(hr))
00851                 {
00852                     hr = pNewOutputType->SetGUID(MF_MT_SUBTYPE, MFVideoFormat_NV12);
00853                     subtype = MFVideoFormat_NV12;
00854                 }
00855             }
00856         }
00857         // 2. 現在の入力メディアタイプから解像度などの属性をコピーする
00858         if (SUCCEEDED(hr) && pNewOutputType)
00859         {
00860             IMFMediaType* pInputType{ nullptr };
00861             if (SUCCEEDED(pDecoder->GetInputCurrentType(0, &pInputType)))
00862             {
00863                 UINT32 w{ 0 }, h{ 0 };
00864                 if (SUCCEEDED(MFGetAttributeSize(pInputType, MF_MT_FRAME_SIZE, &w, &h)))
00865                 {
00866                     MFSetAttributeSize(pNewOutputType, MF_MT_FRAME_SIZE, w, h);
00867                 }
00868                 UINT32 fpsNum{ 0 }, fpsDenom{ 0 };
00869                 if (SUCCEEDED(MFGetAttributeRatio(pInputType, MF_MT_FRAME_RATE, &fpsNum, &fpsDenom)))
00870                 {
00871                     MFSetAttributeRatio(pNewOutputType, MF_MT_FRAME_RATE, fpsNum, fpsDenom);
00872                 }
00873                 UINT32 aspectNum{ 0 }, aspectDenom{ 0 };
00874                 if (SUCCEEDED(MFGetAttributeRatio(pInputType, MF_MT_PIXEL_ASPECT_RATIO, &aspectNum,
&aspectDenom)))
00875                 {
00876                     MFSetAttributeRatio(pNewOutputType, MF_MT_PIXEL_ASPECT_RATIO, aspectNum, aspectDenom);
00877                 }
00878                 PROPVARIANT var;
00879                 PropVariantInit(&var);
00880                 if (SUCCEEDED(pInputType->GetItem(MF_MT_INTERLACE_MODE, &var)))
00881                 {
00882                     if (var.vt == VT_UI4) pNewOutputType->SetUINT32(MF_MT_INTERLACE_MODE, var.ulVal);
00883                     PropVariantClear(&var);
00884                 }
00885                 SafeRelease(&pInputType);
00886             }
00887             // 新しい出力メディアタイプを設定する
00888             hr = pDecoder->SetOutputType(0, pNewOutputType, 0);
00889             #if defined(_DEBUG)
00890                 if (FAILED(hr)) std::cerr << "pDecoder->SetOutputType failed: " << std::hex << hr <<
std::endl;
00891             #endif
00892             else std::cerr << "pDecoder->SetOutputType succeeded." << std::endl;
00893         }
00894         #if defined(_DEBUG)
00895             if (FAILED(hr))
00896             {
00897                 std::cerr << "pDecoder->SetOutputType failed: " << std::hex << hr <<
std::endl;
00898             }
00899             else if (SUCCEEDED(hr))
00900             {
00901                 std::cerr << "pDecoder->SetOutputType succeeded." << std::endl;
00902             }
00903         #endif

```



```

00902         hr = E_FAIL;
00903 #if defined(_DEBUG)
00904         std::cerr << "No output type available to set!" << std::endl;
00905 #endif
00906     }
00907
00908     // 3. 設定された出力タイプから属性を取得してバッファとカラーコンバータを更新する
00909     if (SUCCEEDED(hr) && pNewOutputType)
00910     {
00911         UINT32 newWidth{ 0 }, newHeight{ 0 };
00912         if (SUCCEEDED(MFGetAttributeSize(pNewOutputType, MF_MT_FRAME_SIZE, &newWidth, &newHeight))
00913             && newWidth > 0 && newHeight > 0)
00914         {
00915             width = newWidth;
00916             height = newHeight;
00917         }
00918
00919         UINT32 fpsNum{ 0 }, fpsDenom{ 0 };
00920         MFGetAttributeRatio(pNewOutputType, MF_MT_FRAME_RATE, &fpsNum, &fpsDenom);
00921
00922         // デコーダの出力バッファ要件の取得と再作成
00923         hr = createDecoderBuffer();
00924
00925         // カラーコンバータの再設定
00926         if (SUCCEEDED(hr) && pConverter)
00927         {
00928             VideoFormat newFormat{ static_cast<UINT32>(width), static_cast<UINT32>(height), fpsNum,
00929                                     fpsDenom, subtype };
00930
00931             // カラーコンバータのセットアップ
00932             hr = setUpPipeline(pConverter, newFormat, MFVideoFormat_RGB32);
00933
00934             if (SUCCEEDED(hr))
00935             {
00936                 hr = createConverterBuffer();
00937             }
00938
00939             // 基底クラスのバッファリサイズ
00940             if (SUCCEEDED(hr))
00941             {
00942                 frame.resize(width * height * channels);
00943                 image.resize(width * height * channels);
00944             }
00945
00946             // 出力サンプルとイベントを破棄する
00947             if (!mftProvidesSamples) SafeRelease(&pDecodedSample);
00948             else SafeRelease(&decodedBuffer.pSample);
00949             SafeRelease(&decodedBuffer.pEvents);
00950
00951             // リソースの解放
00952             SafeRelease(&pNewOutputType);
00953
00954             if (FAILED(hr))
00955             {
00956 #if defined(_DEBUG)
00957                 std::cerr << "Failed to handle stream change: " << std::hex << hr << std::endl;
00958 #endif
00959                 goto done;
00960             }
00961
00962             // ストリーム情報を再取得する (ストリームチェンジによってフラグが変わる可能性があるため)
00963             if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo)))
00964             {
00965                 mftProvidesSamples = (streamInfo.dwFlags & MFT_OUTPUT_STREAM_PROVIDES_SAMPLES) != 0;
00966 #if defined(_DEBUG)
00967                 std::cerr << "After stream change, mftProvidesSamples = " << (mftProvidesSamples ? "true"
00968                                         : "false") << ", cbSize = " << streamInfo.cbSize << std::endl;
00969 #endif
00970             }
00971
00972 #if defined(_DEBUG)
00973             std::cerr << "Stream change handled successfully, retrying ProcessOutput." << std::endl;
00974 #endif
00975
00976             // ストリーム変更を行ったので、今回の ProcessOutput はやり直す
00977             if (!mftProvidesSamples && pDecoderBuffer)
00978             {
00979                 pDecoderBuffer->SetCurrentLength(prevLength);
00980             }
00981             continue;
00982
00983             // デコーダがさらなる入力を要求している場合 (正常動作)
00984             if (hr == MF_E_TRANSFORM_NEED_MORE_INPUT)
00985             {
00986                 if (!mftProvidesSamples)

```

```

00986         {
00987             if (pDecoderBuffer) pDecoderBuffer->SetCurrentLength(prevLength);
00988             SafeRelease(&pDecodedSample);
00989         }
00990         else
00991         {
00992             SafeRelease(&decodedBuffer.pSample);
00993         }
00994         break;
00995     }
00996
00997     // その他のエラー
00998     if (FAILED(hr))
00999     {
01000 #if defined(_DEBUG)
01001         std::cerr << "Decoder process output failed (loop bottom): " << std::hex << hr << std::endl;
01002 #endif
01003         if (!mftProvidesSamples)
01004         {
01005             if (pDecoderBuffer) pDecoderBuffer->SetCurrentLength(prevLength);
01006             SafeRelease(&pDecodedSample);
01007         }
01008         else
01009         {
01010             SafeRelease(&decodedBuffer.pSample);
01011         }
01012         SafeRelease(&decodedBuffer.pEvents);
01013         goto done;
01014     }
01015
01016     // デコード成功!
01017     SafeRelease(&pLatestDecodedSample);
01018     if (mftProvidesSamples)
01019     {
01020         pLatestDecodedSample = decodedBuffer.pSample;
01021     }
01022     else
01023     {
01024         pLatestDecodedSample = pDecodedSample;
01025     }
01026     pDecodedSample = nullptr;
01027     SafeRelease(&decodedBuffer.pEvents);
01028     hasOutput = true;
01029
01030     // レイテンシ優先なら
01031     if (prioritizeLatency)
01032     {
01033         // 最新を取得し続けるためループを継続する
01034         continue;
01035     }
01036     else
01037     {
01038         // 全フレーム処理モードならループを抜ける
01039         break;
01040     }
01041 }
01042
01043 if (hasOutput && pLatestDecodedSample)
01044 {
01045     // 元のサンプルはもう使わないので解放する
01046     pSample->Release();
01047
01048     // デコードに成功したらデコードされたサンプルを使う
01049     pSample = pLatestDecodedSample;
01050     pLatestDecodedSample = nullptr;
01051 }
01052 else
01053 {
01054     // 今回の入力ではデコード完了フレームが得られなかったため、
01055     // 入力サンプルを解放して次のフレームの読み込みに進む
01056     pSample->Release();
01057     pSample = nullptr;
01058     continue;
01059 }
01060 }
01061
01062 // カラーコンバータが設定されていれば (NV12 か YUY2 か MJPG か H264 の場合)
01063 if (pConverter)
01064 {
01065     // サンプルをカラーコンバータに渡す
01066     HRESULT hr{ pConverter->ProcessInput(0, pSample, 0) };
01067     if (FAILED(hr)) goto done;
01068
01069     // カラーコンバータの出力サンプルを作成する
01070     hr = MFCreateSample(&pConvertedSample);
01071     if (FAILED(hr)) goto done;
01072

```

```

01073 // バッファの長さを 0 に設定して書き込み可能にする
01074 pConverterBuffer->SetCurrentLength(0);
01075
01076 // カラーコンバータの出力サンプルにバッファを追加する
01077 hr = pConvertedSample->AddBuffer(pConverterBuffer);
01078 if (FAILED(hr)) goto done;
01079
01080 // カラーコンバータの出力バッファ
01081 MFT_OUTPUT_DATA_BUFFER convertedBuffer{ 0, pConvertedSample, 0, nullptr };
01082
01083 // カラー変換処理を実行
01084 hr = pConverter->ProcessOutput(0, 1, &convertedBuffer, &dwStatus);
01085 if (FAILED(hr)) goto done;
01086
01087 // 元のサンプルはもう使わないので解放する
01088 pSample->Release();
01089
01090 // コンバートに成功したらコンバートされたサンプルを使う
01091 pSample = pConvertedSample;
01092 pConvertedSample = nullptr;
01093 }
01094
01095 // サンプルからメディアバッファを取得して
01096 if (FAILED(pSample->GetBufferByIndex(0, &pBuffer))) goto done;
01097
01098 // メディアバッファが取得できていれば
01099 if (pBuffer)
01100 {
01101     // メディアバッファから取り出したデータ
01102     BYTE* pData{ nullptr };
01103
01104     // メディアバッファから取り出したデータの長さ
01105     DWORD cbDataLength{ 0 };
01106
01107     // メディアバッファからフレームの情報を取得できたら
01108     if (SUCCEEDED(pBuffer->Lock(&pData, nullptr, &cbDataLength)) && pData)
01109     {
01110         // 全フレーム処理モードなら
01111         if (!prioritizeLatency)
01112         {
01113             // キャプチャしている間は
01114             while (running && captured)
01115             {
01116                 // スレッドを一時停止して CPU リソースを解放する
01117                 std::this_thread::yield();
01118             }
01119         }
01120
01121         // 一時メモリをロックして
01122         std::lock_guard<std::mutex> lock{ mtx };
01123
01124         // キャプチャしたデータを一時メモリにコピーしたら
01125         if (image.size() < cbDataLength) image.resize(cbDataLength);
01126         memcpy(image.data(), pData, cbDataLength);
01127
01128         // 新しいフレームがキャプチャされたことを通知して
01129         captured = true;
01130
01131         // メディアバッファのロックを解除する
01132         pBuffer->Unlock();
01133     }
01134
01135     // メディアバッファを解放する
01136     pBuffer->Release();
01137     pBuffer = nullptr;
01138 }
01139
01140 done:
01141
01142 // デコードの出力サンプルとバッファを解放する
01143 if (pDecodedSample) pDecodedSample->Release();
01144
01145 // カラーコンバータの出力サンプルバッファを解放する
01146 if (pConvertedSample) pConvertedSample->Release();
01147
01148 // サンプルを解放する
01149 pSample->Release();
01150 pSample = nullptr;
01151 }
01152 }
01153
01154 //
01155 // キャプチャスレッドを停止する
01156 //
01157 void CamMf::stop()
01158 {
01159     // キャプチャスレッドが実行中なら

```

```

01160     if (running)
01161     {
01162         // キャプチャスレッドのループを止めて
01163         running = false;
01164
01165         // ReadSample のブロッキングを解除する
01166         if (pSourceReader)
01167         {
01168             pSourceReader->Flush(MF_SOURCE_READER_FIRST_VIDEO_STREAM);
01169         }
01170
01171         // 合流する
01172         if (thr.joinable())
01173         {
01174             thr.join();
01175         }
01176     }
01177 }
01178
01179 //
01180 // カメラを閉じる
01181 //
01182 void CamMf::close()
01183 {
01184     // MFT デコーダを解放する
01185     cleanUpTransform(&pDecoder);
01186
01187     // MFT カラーコンバータを解放する
01188     cleanUpTransform(&pConverter);
01189
01190     // MFT デコーダのバッファを解放する
01191     SafeRelease(&pDecoderBuffer);
01192
01193     // MFT カラーコンバータのバッファを解放する
01194     SafeRelease(&pConverterBuffer);
01195
01196     // Source Reader を解放する
01197     SafeRelease(&pSourceReader);
01198
01199     // Media Source 解放する
01200     SafeRelease(&pMediaSource);
01201
01202     // フォーマットリストをクリアする
01203     availableFormats.clear();
01204     formatList.clear();
01205
01206     // 基底クラスの close を呼び出す
01207     Camera::close();
01208 }

```

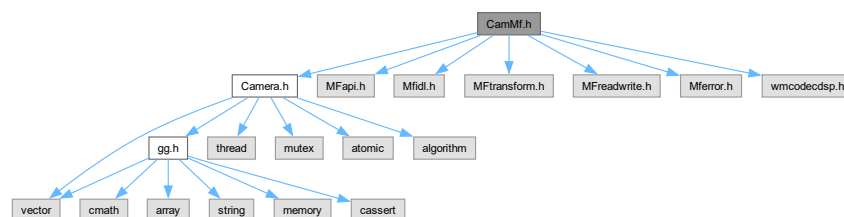
9.19 CamMf.h ファイル

```

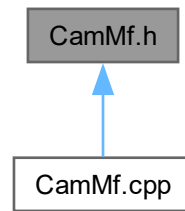
#include "Camera.h"
#include <MFapi.h>
#include <Mfidl.h>
#include <MFtransform.h>
#include <MFreadwrite.h>
#include <Mferror.h>
#include <wmcodecdsp.h>

```

CamMf.h の依存先関係図:



被依存関係図:



クラス

- class [CamMf](#)

9.19.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.h](#) に定義があります。

9.20 CamMf.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // カメラ関連の処理
00012 #include "Camera.h"
00013
00014 // Microsoft Media Foundation
00015 #include <MFapi.h>
00016 #include <Mfidl.h>
00017 #include <MFtransform.h>
00018 #include <MFreadwrite.h>
00019 #include <Mferror.h>
00020 #include <wmcodecdsp.h>
00021
00025 class CamMf : public Camera
00026 {
00030     struct VideoFormat
00031     {
00032         UINT32 width;
```

```

00033     UINT32 height;
00034     UINT32 fpsNum;
00035     UINT32 fpsDenom;
00036     GUID subType;
00037
00047     VideoFormat(UINT32 width, UINT32 height,
00048         UINT32 fpsNum, UINT32 fpsDenom, GUID subType)
00049         : width{ width }
00050         , height{ height }
00051         , fpsNum{ fpsNum }
00052         , fpsDenom{ fpsDenom }
00053         , subType{ subType }
00054     {
00055     }
00056 };
00057
00061 class ComInitializer
00062 {
00064     static ComInitializer instance;
00065
00067     IMFActivate** ppSourceActivate{ nullptr };
00068
00070     UINT32 cSourceActivate{ 0 };
00071
00073     std::vector<std::string> deviceList;
00074
00076     bool coInitialized{ false };
00077
00079     bool mfStarted{ false };
00080
00084     ComInitializer();
00085
00089     ~ComInitializer();
00090
00096     const char* initialize();
00097
00098 public:
00099
00100     // シングルトンなのでコピーは作らせない
00101     ComInitializer(const ComInitializer& com) = delete;
00102     ComInitializer(ComInitializer&& com) = delete;
00103     ComInitializer& operator=(const ComInitializer& com) = delete;
00104     ComInitializer& operator=(ComInitializer&&) = delete;
00105
00111     static const ComInitializer& getInstance();
00112
00120     static bool activate(int device, IMFMediaSource** pMediaSource);
00121
00127     static const std::vector<std::string>& getDeviceList();
00128 };
00129
00131 IMFMediaSource* pMediaSource{ nullptr };
00132
00134 IMFSourceReader* pSourceReader{ nullptr };
00135
00137 IMFTransform* pDecoder{ nullptr };
00138
00140 IMFMediaBuffer* pDecoderBuffer{ nullptr };
00141
00143 IMFTransform* pConverter{ nullptr };
00144
00146 IMFMediaBuffer* pConverterBuffer{ nullptr };
00147
00149 std::vector<VideoFormat> availableFormats;
00150
00152 std::vector<CaptureFormat> formatList;
00153
00159 bool enumerateFormats();
00160
00171 HRESULT findVideoDecoder(
00172     const GUID& subtype,
00173     IMFTransform** ppDecoder,
00174     BOOL bAllowAsync = FALSE,
00175     BOOL bAllowHardware = FALSE,
00176     BOOL bAllowTranscode = FALSE
00177 ) const;
00178
00187 HRESULT setUpPipeline(IMFTransform* pTransform,
00188     const VideoFormat& format, const GUID& subType) const;
00189
00195 void cleanUpTransform(IMFTransform** pTransform) const;
00196
00202 HRESULT createDecoderBuffer();
00203
00209 HRESULT createConverterBuffer();
00210
00217 bool setFormat(int index);

```

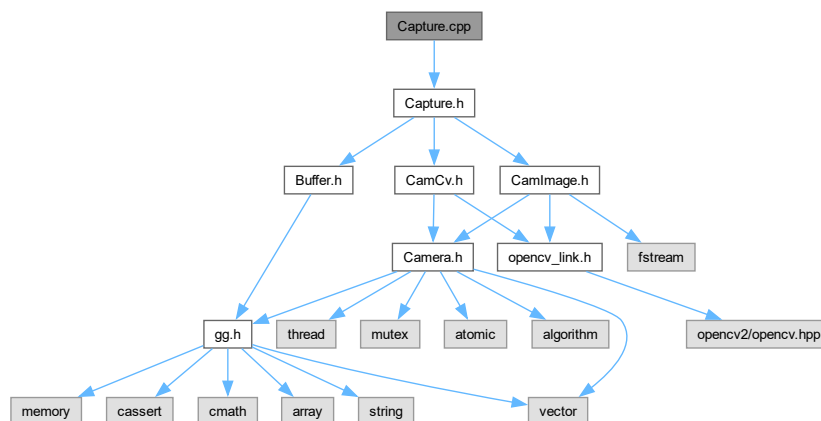
```

00218
00219 public:
00220
00224   CamMf() = default;
00225
00229   virtual ~CamMf()
00230   {
00231       // カメラを閉じる
00232       close();
00233   }
00234
00240   static const auto& getDeviceList()
00241   {
00242       return ComInitializer::getDeviceList();
00243   }
00244
00252   bool open(int device, bool setupFormat = true);
00253
00259   const auto& getFormatList() const
00260   {
00261       return formatList;
00262   }
00263
00270   bool select(int index);
00271
00275   void capture();
00276
00280   virtual void stop() override;
00281
00285   void close();
00286 };

```

9.21 Capture.cpp ファイル

#include "Capture.h"
 Capture.cpp の依存先関係図:



9.21.1 詳解

キャプチャクラスの実装

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.cpp](#) に定義があります。

9.22 Capture.cpp

[詳解]

```
00001
00008 #include "Capture.h"
00009
00010 #if defined(_WIN32)
00012 const std::vector<CaptureFormat> Capture::emptyFormatList;
00013 #endif
00014
00015 //
00016 // 画像ファイルを開く
00017 //
00018 bool Capture::openImage(const std::string& filename)
00019 {
00020     // 新しいキャプチャデバイスを作成したら
00021     auto camImage{ std::make_unique<CamImage>() };
00022
00023     // キャプチャデバイスを開く
00024     if (camImage->open(filename))
00025     {
00026         // このキャプチャデバイスを使うことにする
00027         camera = std::move(camImage);
00028         return true;
00029     }
00030
00031     // 開けなかった
00032     return false;
00033 }
00034
00035 //
00036 // 動画ファイルを開く
00037 //
00038 bool Capture::openMovie(const std::string& filename,
00039 cv::VideoCaptureAPIs backend)
00040 {
00041     // 新しいキャプチャデバイスを作成したら
00042     auto camCv{ std::make_unique<CamCv>() };
00043
00044     // キャプチャデバイスを開く
00045     if (camCv->open(filename, 0, 0, 0.0, "", backend))
00046     {
00047         // このキャプチャデバイスを使うことにする
00048         camera = std::move(camCv);
00049         return true;
00050     }
00051
00052     // 開けなかった
00053     return false;
00054 }
00055
00056 #if defined(_WIN32)
00057 //
00058 // デバイスを開く (Windows用: MSMF)
00059 //
00060 bool Capture::openDevice(int deviceNumber)
00061 {
00062     // 既にカメラが有効なら一旦閉じる
00063     if (camera) camera->close();
00064
00065     // 新しいキャプチャデバイスを作成したら
00066     auto camMf{ std::make_unique<CamMf>() };
00067
00068     // このデバイスをデバイス番号で開いて
00069     if (camMf->open(deviceNumber, false))
00070     {
00071         // このキャプチャデバイスを使うことにする
00072         camera = std::move(camMf);
00073
00074         // 開けた
00075         return true;
00076     }
00077 }
```



```

00077
00078 // カメラを無効にしておく
00079 camera.reset();
00080
00081 // 開けなかった
00082 return false;
00083 }
00084
00085 //
00086 // ビデオフォーマット選択
00087 //
00088 bool Capture::select(int index)
00089 {
00090     // カメラが有効でなければ戻る
00091     if (!camera) return false;
00092
00093     // バックエンドが Microsoft Media Foundation でなければ戻る
00094     auto camMf{ dynamic_cast<CamMf*>(camera.get()) };
00095     if (!camMf) return true; // Media Foundation 以外なら常に true
00096
00097     // ビデオフォーマットを選択する
00098     return camMf->select(index);
00099 }
00100
00101 void Capture::updateFormatList(int deviceNumber)
00102 {
00103     // 実際の入力状態を変更せずに選択肢だけ取得する一時カメラ
00104     CamMf temp;
00105
00106     // デバイスを遅延初期化で開く
00107     if (temp.open(deviceNumber, false))
00108     {
00109         // 開けたら列挙されたフォーマットリストを保存する
00110         deviceFormatList = temp.getFormatList();
00111         temp.close();
00112     }
00113     else
00114     {
00115         // 開けなかったらフォーマットリストを空にする
00116         deviceFormatList.clear();
00117     }
00118 }
00119
00120 //
00121 // フォーマットリストを取り出す
00122 //
00123 const std::vector<CaptureFormat>& Capture::getFormatList() const
00124 {
00125     // 開いている Media Foundation カメラを優先し、なければ事前取得した一覧を返す
00126     auto camMf{ dynamic_cast<const CamMf*>(camera.get()) };
00127     return camMf ? camMf->getFormatList() : deviceFormatList;
00128 }
00129
00130 #else
00131 //
00132 // デバイスを開く (Windows以外用: OpenCV)
00133 //
00134 bool Capture::openDevice(int deviceNumber, std::array<int, 2>& size, double& fps,
00135 cv::VideoCaptureAPIs backend, char* fourcc)
00136 {
00137     // 既にカメラが有効なら一旦閉じる
00138     if (camera) camera->close();
00139
00140     // 新しいキャプチャデバイスを作成したら
00141     auto camCv{ std::make_unique<CamCv>() };
00142
00143     // このデバイスをデバイス番号で開いて
00144     if (camCv->open(deviceNumber, size[0], size[1], fps, fourcc, backend))
00145     {
00146         // 実際に開いた設定を書き戻す
00147         size[0] = camCv->getWidth();
00148         size[1] = camCv->getHeight();
00149         fps = camCv->getFps();
00150         camCv->getCodec(fourcc);
00151
00152         // このキャプチャデバイスを使うことにする
00153         camera = std::move(camCv);
00154         return true;
00155     }
00156
00157     // 開けなかった
00158     return false;
00159 }
00160 #endif
00161
00162 //
00163 // キャプチャ開始

```

```

00164 //
00165 void Capture::start()
00166 {
00167     // キャプチャデバイスが有効ならキャプチャスレッドを起動する
00168     if (camera) camera->start();
00169 }
00170
00171 //
00172 // キャプチャ終了
00173 //
00174 void Capture::stop()
00175 {
00176     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00177     if (camera) camera->stop();
00178 }
00179
00180 //
00181 // キャプチャデバイスを閉じる
00182 //
00183 void Capture::close()
00184 {
00185     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00186     if (camera)
00187     {
00188         camera->close();
00189         camera.reset();
00190     }
00191 }
00192
00193 //
00194 // キャプチャデバイスの解像度とフレームレートを取得する
00195 //
00196 std::array<int, 2> Capture::getSize() const
00197 {
00198     // キャプチャデバイスが有効ならその解像度を返す
00199     return camera ? camera->getSize() : std::array<int, 2>{ 0, 0 };
00200 }
00201
00202 //
00203 // キャプチャデバイスのフレームレートを取得する
00204 //
00205 double Capture::getFps() const
00206 {
00207     // キャプチャデバイスが有効ならそのフレームレートを返す
00208     return camera ? camera->getFps() : 0.0;
00209 }
00210
00211 //
00212 // 新しいフレームを GPU の PBO に取得する
00213 //
00214 bool Capture::retrieve(Buffer& buffer)
00215 {
00216     // キャプチャデバイスが有効なら
00217     if (camera)
00218     {
00219         // バッファのサイズを取得したフレームのサイズに合わせて
00220         buffer.create(camera->getWidth(), camera->getHeight(), camera->getChannels());
00221
00222         // 取得したフレームをフレームを転送する
00223         return camera->transmit(buffer.getBufferName());
00224     }
00225
00226     // 転送失敗
00227     return false;
00228 }
00229
00230 //
00231 // 新しいフレームを CPU のメモリに取得する
00232 //
00233 bool Capture::retrieve(cv::Mat& frame)
00234 {
00235     // OpenCV 補正を選んだときだけ使用し、PBO へ送る前の画像を cv::Mat として得る。
00236     return camera && camera->transmit(frame);
00237 }

```

9.23 Capture.h ファイル

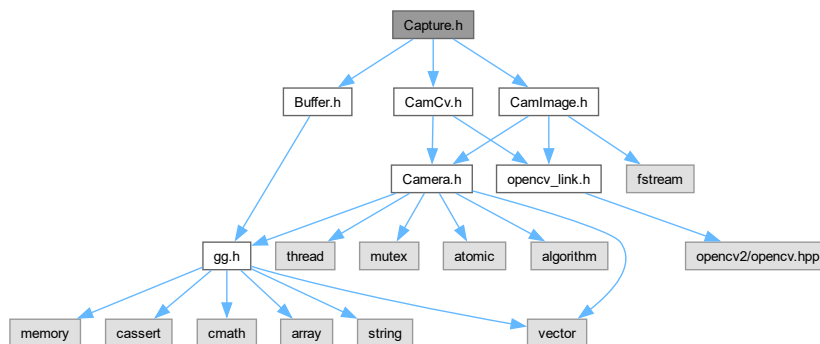
```

#include "Buffer.h"
#include "CamImage.h"

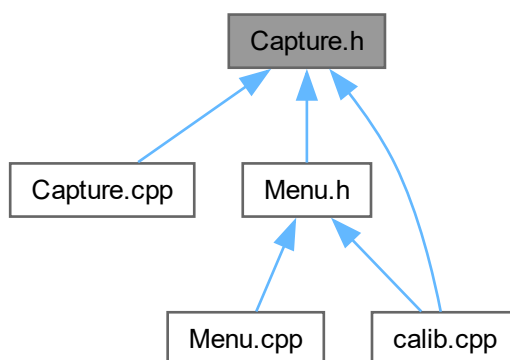
```

```
#include "CamCv.h"
```

Capture.h の依存先関係図:



被依存関係図:



クラス

- class [Capture](#)

9.23.1 詳解

キャプチャクラスの定義

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.h](#) に定義があります。

9.24 Capture.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // バッファクラス
00012 #include "Buffer.h"
00013
00014 // OpenCV による画像ファイルの入力
00015 #include "CamImage.h"
00016
00017 // OpenCV による動画の入力
00018 #include "CamCv.h"
00019
00020 // Windows のみ Microsoft Media Foundation による動画の入力
00021 #if defined(_WIN32)
00022 #include "CamMf.h"
00023 #endif
00024
00028 class Capture
00029 {
00031     std::unique_ptr<Camera> camera{};
00032
00033     #if defined(_WIN32)
00035         std::vector<CaptureFormat> deviceFormatList;
00036
00038         static const std::vector<CaptureFormat> emptyFormatList;
00039     #endif
00040
00041 public:
00042
00046     Capture() = default;
00047
00053     Capture(const std::string& filename)
00054     {
00055         openImage(filename);
00056     }
00057
00061     virtual ~Capture()
00062     {
00063         close();
00064     }
00065
00072     bool openImage(const std::string& filename);
00073
00081     bool openMovie(const std::string& filename,
00082 #if defined(_WIN32)
00083         cv::VideoCaptureAPIs backend = cv::CAP_ANY;
00084 #else
00085         cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG;
00086 #endif
00087
00088 #if defined(_WIN32)
00095         bool openDevice(int deviceNumber);
00096
00104         const std::vector<CaptureFormat>& getFormatList() const;
00105
00111         bool select(int index);
00112
00120         void updateFormatList(int deviceNumber);
00121 #else
00132         bool openDevice(int deviceNumber,
00133             std::array<int, 2>& size, double& fps,
00134             cv::VideoCaptureAPIs backend,
00135             char* fourcc);
00136 #endif
00137
00141         void start();
00142
00146         void stop();
00147
00151         void close();
00152
00158         explicit operator bool() const
00159         {
00160             return camera && camera->isRunning();
00161         }
00162
00166         bool isOpened() const
00167         {
00168             return bool(camera);
00169         }
00170
00174         bool isImage() const

```

```

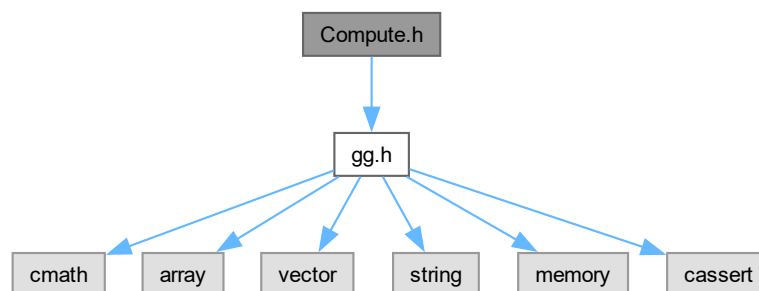
00175 {
00176     return camera && dynamic_cast<const CamImage*>(camera.get()) != nullptr;
00177 }
00178
00184 std::array<int, 2> getSize() const;
00185
00191 double getFps() const;
00192
00199 bool retrieve(Buffer& buffer);
00200
00208 bool retrieve(cv::Mat& frame);
00209
00215 void setPrioritizeLatency(bool mode)
00216 {
00217     if (camera) camera->setPrioritizeLatency(mode);
00218 }
00219
00225 bool getPrioritizeLatency() const
00226 {
00227     return camera ? camera->getPrioritizeLatency() : false;
00228 }
00229 };

```

9.25 Compute.h ファイル

```
#include "gg.h"
```

Compute.h の依存先関係図:



クラス

- class [Compute](#)

9.25.1 詳解

画像処理クラスの定義（コンピュートシェード版）

著者

Kohe Tokoi

日付

November 15, 2022

[Compute.h](#) に定義があります。

9.26 Compute.h

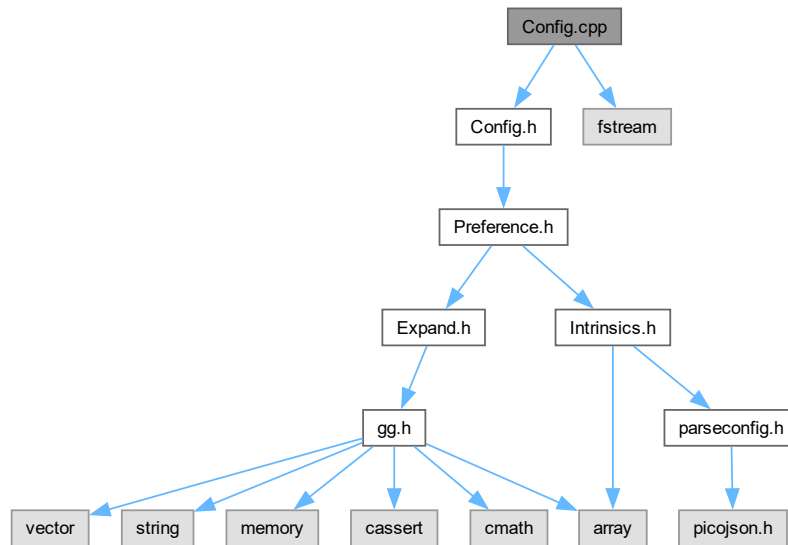
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00018 class Compute
00019 {
00021     const GLuint program;
00022
00023 public:
00024
00030     Compute(const char* comp)
00031         : program(ggLoadComputeShader(comp))
00032     {
00033     }
00034
00038     virtual ~Compute()
00039     {
00040         // シェーダプログラムを削除する
00041         glDeleteShader(program);
00042     }
00043
00049     auto get() const
00050     {
00051         return program;
00052     }
00053
00057     void use() const
00058     {
00059         glUseProgram(program);
00060     }
00061
00070     void execute(GLuint width, GLuint height, GLuint local_size_x = 1, GLuint local_size_y = 1) const
00071     {
00072         glDispatchCompute((width + local_size_x - 1) / local_size_x, (height + local_size_y - 1) /
00073             local_size_y, 1);
00074     };
00074 };
```

9.27 Config.cpp ファイル

```
#include "Config.h"
#include <fstream>
```

Config.cpp の依存先関係図:



9.27.1 詳解

構成データクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Config.cpp](#) に定義があります。

9.28 Config.cpp

[詳解]

```

00001
00008 #include "Config.h"
00009
00010 // 標準ライブラリ
00011 #include <fstream>
00012
00013 #if defined(_WIN32)
00014 // Microsoft Media Foundation によるキャプチャ
00015 #include "CamMf.h"
00016 #endif
00017
00018 #if !defined(_DEBUG) && defined(_WIN32)
00019 // appData のパスを得るときに使う
00020 #include <shlobj_core.h>

```

```

00021 #endif
00022
00023 //
00024 // コンストラクタ
00025 //
00026 Config::Config(const std::string& filename)
00027 #if defined(_WIN32)
00028 : deviceList{ CamMf::getDeviceList() }
00029 #endif
00030 {
00031     // 構成ファイルの保存場所を決定する
00032     #if defined(DEBUG)
00033         const auto path{ Utf8ToTChar(filename) };
00034     #else
00035         # if defined(_WIN32)
00036             // 構成ファイルの保存先のパス
00037             wchar_t appDataPath[MAX_PATH];
00038
00039             // AppData のパスを取得する
00040             SHGetSpecialFolderPathW(NULL, appDataPath, CSIDL_APPDATA, 0);
00041
00042             // AppData のパスに構成ファイル名を連結する
00043             const auto path{ CString(appDataPath) + TEXT("\\") + Utf8ToTChar(filename) };
00044         # else
00045             // パスワードファイルのエントリを取得する
00046             const passwd* pw{ getpwuid(getuid()) };
00047
00048             // ホームディレクトリのパスに構成ファイル名を連結する
00049             const auto path{ std::string(pw->pw_dir) + "/" + filename };
00050         # endif
00051     #endif
00052
00053     // 構成ファイルを読み込む
00054     if (!load(path))
00055     {
00056         // デフォルトの設定を追加する
00057         if (!load(Utf8ToTChar(filename))) preferenceList.emplace_back();
00058
00059         // デフォルトの設定を入れた構成ファイルを作成する
00060         save(path);
00061     }
00062 }
00063
00064 //
00065 // デストラクタ
00066 //
00067 Config::~Config()
00068 {
00069 }
00070
00071 //
00072 // 構成データを初期化する
00073 //
00074 void Config::initialize()
00075 {
00076     // 構成リストのすべて構成についてシェーダをビルドする
00077     for (auto& preference : preferenceList) preference.buildShader();
00078
00079     // 初期化済みであることを記録しておく
00080     initialized = true;
00081
00082     // 背景色を設定する
00083     glClearColor(background[0], background[1], background[2], background[3]);
00084 }
00085
00086 //
00087 // 構成ファイルを読み込む
00088 //
00089 bool Config::load(const pathString& filename)
00090 {
00091     // 構成ファイルの読み込み
00092     std::ifstream json{ filename };
00093     if (!json) return false;
00094
00095     // JSON の読み込み
00096     picojson::value value;
00097     json >> value;
00098     json.close();
00099
00100     // ルートが JSON オブジェクトでなければ、既存構成を変更せず失敗とする
00101     if (!value.is<picojson::object>()) return false;
00102
00103     // 構成内容の取り出し
00104     const auto& object{ value.get<picojson::object>() };
00105
00106     // オブジェクトが空だったらエラー
00107     if (object.empty()) return false;

```



```

00108
00109 // 読み込みに失敗しても現在の構成を壊さないよう、一時領域へ読み込む
00110 auto loadedWindowSize{ windowSize };
00111 auto loadedBackground{ background };
00112 auto loadedSettings{ settings };
00113 auto loadedInitialImage{ initialImage };
00114 std::vector<Preference> loadedPreferences;
00115
00116 // ウィンドウのサイズ
00117 getValue(object, "size", loadedWindowSize);
00118
00119 // ウィンドウの背景色
00120 getValue(object, "background", loadedBackground);
00121
00122 // メッシュのサンプル数
00123 getValue(object, "samples", loadedSettings.samples);
00124
00125 // キャプチャデバイスの姿勢
00126 getValue(object, "pose", loadedSettings.euler);
00127
00128 // 描画時の焦点距離
00129 getValue(object, "focal", loadedSettings.focal);
00130
00131 // 描画時の焦点距離の範囲
00132 getValue(object, "range", loadedSettings.focalRange);
00133
00134 // ArUco Marker の辞書名
00135 getString(object, "dictionary", loadedSettings.dictionaryName);
00136
00137 // ChArUco Board のマス目の横と縦の数
00138 if (!getValue(object, "squares", loadedSettings.checkerSize))
00139     getValue(object, "checkerSize", loadedSettings.checkerSize);
00140
00141 // ChArUco Board のマス目とマーカーの一辺の長さ
00142 if (!getValue(object, "length", loadedSettings.checkerLength))
00143     getValue(object, "checkerLength", loadedSettings.checkerLength);
00144
00145 // ArUco Marker の一辺の長さ
00146 if (!getValue(object, "marker", loadedSettings.markerLength))
00147     getValue(object, "markerLength", loadedSettings.markerLength);
00148
00149 // 初期表示画像
00150 getString(object, "initial", loadedInitialImage);
00151
00152 // キャプチャデバイスの構成を探す
00153 const auto& camera{ object.find("camera") };
00154
00155 // キャプチャデバイスの構成の配列が見つからなければエラー
00156 if (camera == object.end() || !camera->second.is<picojson::array>()) return false;
00157
00158 // 配列の個々の要素について
00159 for (const auto& value : camera->second.get<picojson::array>())
00160 {
00161     // 配列中に投影方式以外の値があれば、不完全な構成を適用せず失敗とする
00162     if (!value.is<picojson::object>()) return false;
00163
00164     // キャプチャデバイスの構成のオブジェクトを取り出す
00165     const auto& preference{ value.get<picojson::object>() };
00166
00167     // キャプチャデバイスの構成をリストに追加
00168     loadedPreferences.emplace_back(preference);
00169 }
00170
00171 // キャプチャデバイスの構成が一つも読み取れなければエラー
00172 if (loadedPreferences.empty()) return false;
00173
00174 // 実行時の再読み込みでは、新しい構成のシェードも準備する
00175 if (initialized)
00176 {
00177     // OpenGL 初期化後の再読み込みなので、置換前に全投影方式を描画可能にする
00178     for (auto& preference : loadedPreferences) preference.buildShader();
00179 }
00180
00181 // 読み込みと検証がすべて成功してから現在の構成を置き換える
00182 windowSize = loadedWindowSize;
00183 background = loadedBackground;
00184 settings = loadedSettings;
00185 initialImage = std::move(loadedInitialImage);
00186 preferenceList.swap(loadedPreferences);
00187
00188 if (initialized)
00189 {
00190     // 実行中の再読み込みでは、新しい背景色を現在の OpenGL 状態にも反映する
00191     glClearColor(background[0], background[1], background[2], background[3]);
00192 }
00193
00194 return true;

```

```

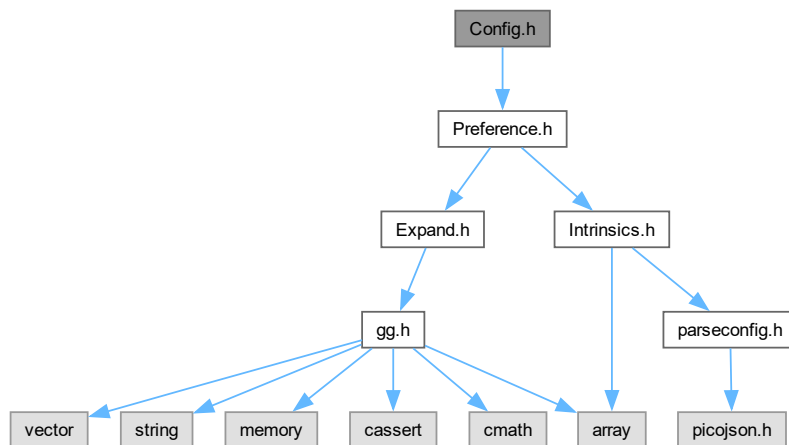
00195 }
00196
00197 //
00198 // 構成ファイルを保存する
00199 //
00200 bool Config::save(const pathString& filename) const
00201 {
00202     // 設定値を保存する
00203     std::ofstream config{ filename };
00204     if (!config) return false;
00205
00206     // オブジェクト
00207     picojson::object object;
00208
00209     // ウィンドウのサイズ
00210     setValue(object, "size", windowSize);
00211
00212     // ウィンドウの背景色
00213     setValue(object, "background", background);
00214
00215     // メッシュのサンプル数
00216     setValue(object, "samples", settings.samples);
00217
00218     // キャプチャデバイスの姿勢
00219     setValue(object, "pose", settings.euler);
00220
00221     // 描画時の焦点距離
00222     setValue(object, "focal", settings.focal);
00223
00224     // 描画時の焦点距離の範囲
00225     setValue(object, "range", settings.focalRange);
00226
00227     // ArUco Marker 辞書名
00228     setString(object, "dictionary", settings.dictionaryName);
00229
00230     // ChArUco Board のマス目の横と縦の数
00231     setValue(object, "squares", settings.checkerSize);
00232
00233     // ChArUco Board のマス目とマーカの一辺の長さ
00234     setValue(object, "length", settings.checkerLength);
00235
00236     // ArUco Marker の一辺の長さ
00237     setValue(object, "marker", settings.markerLength);
00238
00239     // 初期表示画像
00240     setString(object, "initial", initialImage);
00241
00242     // 配列
00243     picojson::array array;
00244
00245     // 構成リストのすべて構成について
00246     for (const auto& preference : preferenceList)
00247     {
00248         // 構成の要素
00249         picojson::object camera;
00250
00251         // 構成を JSON オブジェクトに格納する
00252         preference.setPreference(camera);
00253
00254         // 要素を picojson::array に追加する
00255         array.emplace_back(picojson::value(camera));
00256     }
00257
00258     // オブジェクトに追加する
00259     object.emplace("camera", array);
00260
00261     // 構成をシリアライズして保存
00262     picojson::value v{ object };
00263     config << v.serialize(true);
00264     config.close();
00265
00266     // 構成データの書き込み成功
00267     return true;
00268 }

```

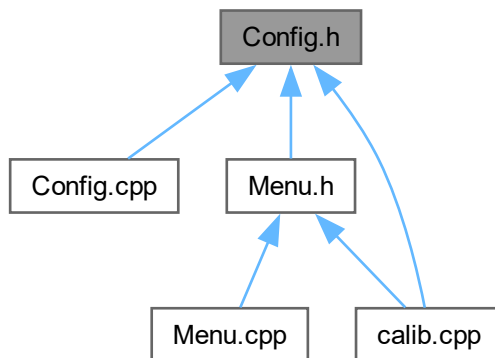
9.29 Config.h ファイル

```
#include "Preference.h"
```

Config.h の依存先関係図:



被依存関係図:



クラス

- struct [Settings](#)
- class [Config](#)

9.29.1 詳解

構成データクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Config.h](#) に定義があります。

9.30 Config.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // キャプチャデバイスの構成
00012 #include "Preference.h"
00013
00017 struct Settings
00018 {
00020     int samples{ 57600 };
00021
00023     std::array<float, 3> euler{ defaultEuler };
00024
00026     static constexpr decltype(euler) defaultEuler{ 0.0f, 0.0f, 0.0f };
00027
00029     float focal{ defaultFocal };
00030
00032     static constexpr decltype(focal) defaultFocal{ 50.0f };
00033
00035     std::array<float, 2> focalRange{ defaultFocalRange };
00036
00038     static constexpr decltype(focalRange) defaultFocalRange{ 10.0f, 200.0f };
00039
00041     std::string dictionaryName{ "DICT_4X4_50" };
00042
00044     std::array<int, 2> checkerSize{ defaultCheckerSize };
00045
00047     static constexpr decltype(checkerSize) defaultCheckerSize{ 10, 7 };
00048
00050     std::array<float, 2> checkerLength{ defaultCheckerLength };
00051
00053     static constexpr decltype(checkerLength) defaultCheckerLength{ 4.0f, 2.0f };
00054
00056     float markerLength{ defaultMarkerLength };
00057
00059     static constexpr decltype(markerLength) defaultMarkerLength{ 5.0f };
00060
00064     auto getFocal() const
00065     {
00066         // 投影面の対角線長は 35mm (17.5mm × 2) とする
00067         return focal / 17.5f;
00068     }
00069 };
00070
00074 class Config
00075 {
00077     std::string title{ PROJECT_NAME };
00078
00080     std::array<GLsizei, 2> windowSize{ 1280, 720 };
00081
00083     std::array<GLfloat, 4> background{ 0.2f, 0.3f, 0.4f, 1.0f };
00084
00086     Settings settings{};
00087
00089     std::string menuFont{ "Mplus1-Regular.ttf" };
```

```
00090
00092     float menuFontSize{ 20.0f };
00093
00095     static std::string initialImage;
00096
00098     std::vector<Preference> preferenceList;
00099
00101     bool initialized{ false };
00102
00103     #if defined(_WIN32)
00105     const std::vector<std::string>& deviceList;
00106     #endif
00107
00108     public:
00109
00115     Config(const std::string& filename);
00116
00120     virtual ~Config();
00121
00127     void initialize();
00128
00138     bool load(const pathString& filename);
00139
00146     bool save(const pathString& filename) const;
00147
00153     const Settings& getSettings() const
00154     {
00155         return settings;
00156     }
00157
00163     void setSettings(const Settings& value)
00164     {
00165         settings = value;
00166     }
00167
00173     const auto& getPreferences() const
00174     {
00175         return preferenceList;
00176     }
00177
00183     const auto& getBackground() const
00184     {
00185         return background;
00186     }
00187
00193     const auto& getMenuFont() const
00194     {
00195         return menuFont;
00196     }
00197
00203     auto getMenuFontSize() const
00204     {
00205         return menuFontSize;
00206     }
00207
00213     const auto& getTitle() const
00214     {
00215         return title;
00216     }
00217
00223     auto getWidth() const
00224     {
00225         return windowSize[0];
00226     }
00227
00233     auto getHeight() const
00234     {
00235         return windowSize[1];
00236     }
00237
00243     const auto& getInitialImage() const
00244     {
00245         return initialImage;
00246     }
00247
00253     const auto& getDictionaryName() const
00254     {
00255         return settings.dictionaryName;
00256     }
00257
00263     const auto& getCheckerSize() const
00264     {
00265         return settings.checkerSize;
00266     }
00267
00273     const auto& getCheckerLength() const
00274     {
```

```

00275     return settings.checkerLength;
00276 }
00277
00283 auto getMarkerLength() const
00284 {
00285     return settings.markerLength;
00286 }
00287
00288 #if defined(_WIN32)
00294 const auto& getDeviceList() const
00295 {
00296     return deviceList;
00297 }
00298
00305 const auto& getDeviceName(int number) const
00306 {
00307     static const std::string empty{};
00308     return deviceList.empty() ? empty : deviceList[number];
00309 }
00310 #endif
00311 };

```

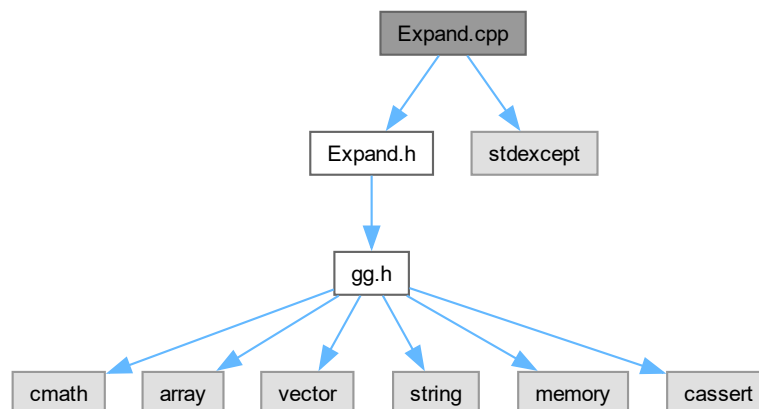
9.31 Expand.cpp ファイル

```

#include "Expand.h"
#include <stdexcept>

```

Expand.cpp の依存先関係図:



9.32 Expand.cpp

[詳解]

```

00001 //
00002 // 展開用シェーダ
00003 //
00004 #include "Expand.h"
00005
00006 // 標準ライブラリ
00007 #include <stdexcept>
00008
00009 //
00010 // コンストラクタ
00011 //
00012 Expand::Expand(const std::string& vert, const std::string& frag)
00013 : program{ gg::ggLoadShader(vert, frag) }

```

```

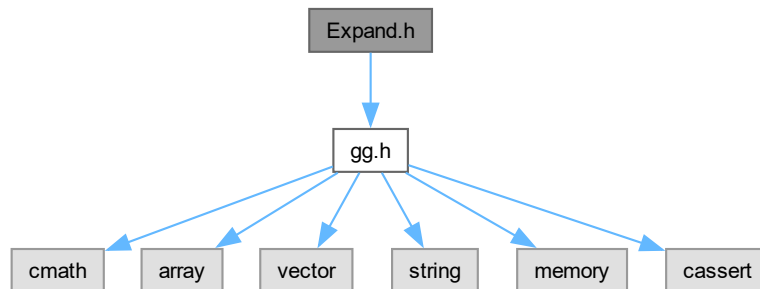
00014 , imageLoc{ glGetUniformLocation(program, "image") }
00015 , screenLoc{ glGetUniformLocation(program, "screen") }
00016 , focalLoc{ glGetUniformLocation(program, "focal") }
00017 , rotationLoc{ glGetUniformLocation(program, "rotation") }
00018 , circleLoc{ glGetUniformLocation(program, "circle") }
00019 , borderLoc{ glGetUniformLocation(program, "border") }
00020 , gapLoc{ glGetUniformLocation(program, "gap") }
00021 {
00022     // プログラムオブジェクトが作れなかったら落とす
00023     if (program == 0) throw std::runtime_error("Cannot create one of the expand shader.");
00024 }
00025
00026 //
00027 // デストラクタ
00028 //
00029 Expand::~Expand()
00030 {
00031     glDeleteProgram(program);
00032 }
00033
00037 std::array<int, 2> Expand::setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00038     const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00039     const std::array<GLfloat, 4>& border, int unit) const
00040 {
00041     // プログラムオブジェクトの指定
00042     glUseProgram(program);
00043
00044     // テクスチャユニットの指定
00045     glUniform1i(imageLoc, unit);
00046
00047     // 境界色
00048     glUniform4fv(borderLoc, 1, border.data());
00049
00050     // 投影像の画角（度）と中心位置
00051     glUniform4f(circleLoc, fov[0], fov[1], center[0], center[1]);
00052
00053     // スクリーンのサイズと中心位置
00054     // screen[0] = (right - left) / 2
00055     // screen[1] = (top - bottom) / 2
00056     // screen[2] = (right + left) / 2
00057     // screen[3] = (top + bottom) / 2
00058     const GLfloat screen[]{ aspect, 1.0f, 0.0f, 0.0f };
00059     glUniform4fv(screenLoc, 1, screen);
00060
00061     // レンズの主点とスクリーンの距離
00062     glUniform1f(focalLoc, focal);
00063
00064     // 背景に対する視線の回転行列
00065     glUniformMatrix4fv(rotationLoc, 1, GL_FALSE, pose.get());
00066
00067     // メッシュの横の格子点数
00068     // 標本点の数（頂点数） samples = w * h とするとき、これに縦横比
00069     // aspect = w / h をかければ aspect * samples = w * h となるから、
00070     // w = sqrt(aspect * samples), h = samples / w で求められる
00071     const auto w{ static_cast<int>(sqrt(aspect * samples)) };
00072     const auto h{ samples / w };
00073
00074     // 格子間隔
00075     // クリッピング空間全体を埋める四角形は [-1, 1] の範囲すなわち
00076     // 縦横 2 の大きさだから、それを縦横の（格子数 - 1）で割る
00077     std::array<GLfloat, 2> gap{ 2.0f / (w - 1), 2.0f / (h - 1) };
00078     glUniform2fv(gapLoc, 1, gap.data());
00079
00080     // 描画するメッシュの横と縦の格子点数を返す
00081     return std::array<int, 2>{ w, h };
00082 }

```

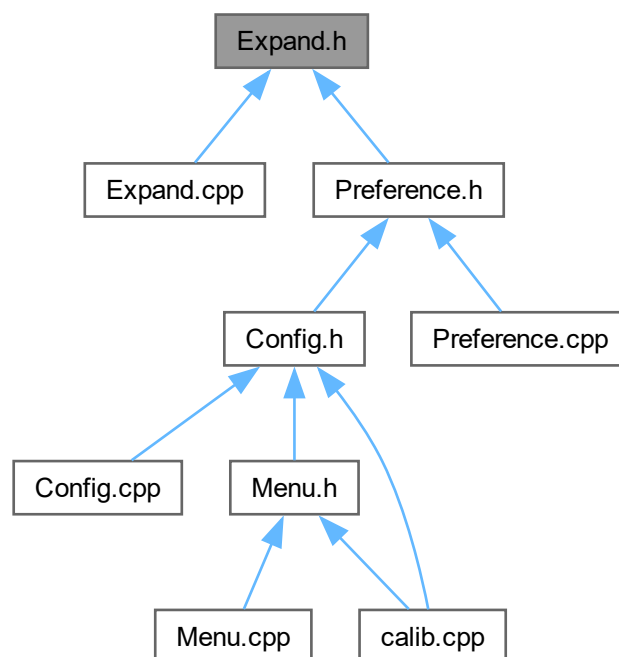
9.33 Expand.h ファイル

```
#include "gg.h"
```

Expand.h の依存先関係図:



被依存関係図:



クラス

- class [Expand](#)

9.33.1 詳解

展開用シェーダクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Expand.h](#) に定義があります。

9.34 Expand.h

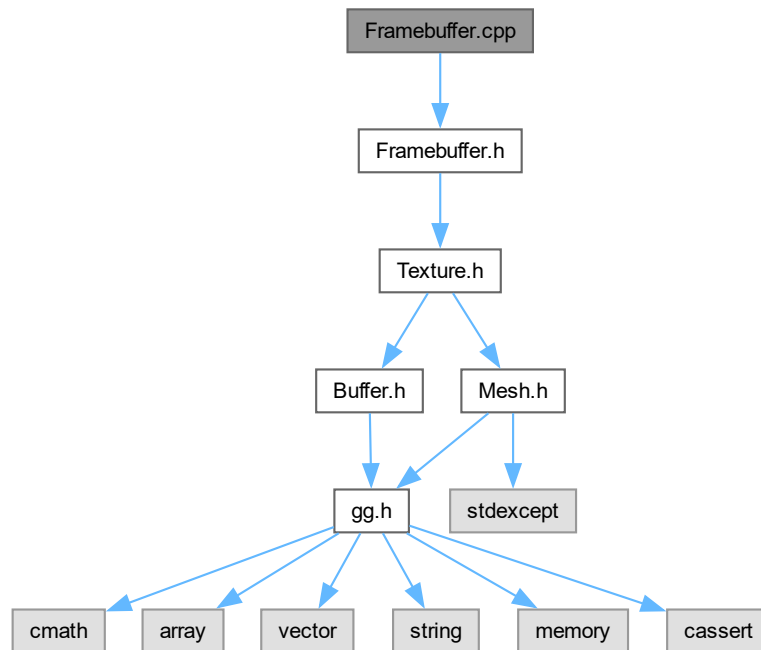
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00017 class Expand
00018 {
00020     const GLuint program;
00021
00023     const GLint imageLoc;
00024
00026     const GLint screenLoc;
00027
00029     const GLint focalLoc;
00030
00032     const GLint rotationLoc;
00033
00035     const GLint circleLoc;
00036
00038     const GLint borderLoc;
00039
00041     const GLint gapLoc;
00042
00043 public:
00044
00051     Expand(const std::string& vert, const std::string& frag);
00052
00058     Expand(const Expand& shader) = delete;
00059
00063     virtual ~Expand();
00064
00071     Expand& operator=(const Expand& shader) = delete;
00072
00078     auto getProgram() const
00079     {
00080         return program;
00081     }
00082
00100     std::array<GLsizei, 2> setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00101         const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00102         const std::array<GLfloat, 4>& border, int unit = 0) const;
00103 };
```

9.35 Framebuffer.cpp ファイル

```
#include "Framebuffer.h"
```

Framebuffer.cpp の依存先関係図:



9.35.1 詳解

フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Framebuffer.cpp](#) に定義があります。

9.36 Framebuffer.cpp

[詳解]

```

00001
00008 #include "Framebuffer.h"
00009
00010 //
00011 // フレームバッファオブジェクトを作成するコンストラクタ
00012 //
00013 Framebuffer::Framebuffer(GLsizei width, GLsizei height, int channels,
00014                           GLenum attachment)
00015     : attachment{ attachment }
00016 {
00017     // フレームバッファオブジェクトを作る
00018     Framebuffer::create(width, height, channels);
00019 }
00020
00026 Framebuffer::Framebuffer(const Framebuffer& framebuffer)
00027     : attachment{ framebuffer.attachment }
00028 {
00029     // フレームバッファオブジェクトをコピーして作成する
00030     Framebuffer::copy(framebuffer);
00031 }
00032
00033 //
00034 // デストラクタ
00035 //
00036 Framebuffer::~Framebuffer()
00037 {
00038     // フレームバッファオブジェクトを破棄する
00039     Framebuffer::discard();
00040 }
00041
00042 //
00043 // 代入演算子
00044 //
00045 Framebuffer& Framebuffer::operator=(const Framebuffer& framebuffer)
00046 {
00047     // 代入元と代入先が同じでなければ
00048     if (&framebuffer != this)
00049     {
00050         // 引数のフレームバッファオブジェクトをコピーする
00051         Framebuffer::copy(framebuffer);
00052     }
00053
00054     // このフレームバッファオブジェクトを返す
00055     return *this;
00056 }
00057
00058 //
00059 // ムーブ代入演算子
00060 //
00061 Framebuffer& Framebuffer::operator=(Framebuffer&& framebuffer) noexcept
00062 {
00063     // ムーブ代入元とムーブ代入先が同じなら何もしない
00064     if (&framebuffer == this) return *this;
00065
00066     // ムーブ代入元の基底クラスをムーブする
00067     static_cast<Texture&>(*this) = std::move(framebuffer);
00068
00069     // ムーブ代入元のテクスチャのメンバをムーブする
00070     framebufferSize = framebuffer.framebufferSize;
00071     framebuffer.framebufferSize = { 0, 0 };
00072     framebufferChannels = framebuffer.framebufferChannels;
00073     framebuffer.framebufferChannels = 0;
00074     framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00075
00076     // ムーブ代入先のフレームバッファを削除する
00077     glDeleteFramebuffers(1, &framebufferName);
00078
00079     // ムーブ代入元のフレームバッファ名をムーブ代入先に移す
00080     framebufferName = framebuffer.framebufferName;
00081
00082     // ムーブ代入元のデストラクタでフレームバッファが削除されないよう 0 にする
00083     framebuffer.framebufferName = 0;
00084
00085     // このフレームバッファオブジェクトを返す
00086     return *this;
00087 }
00088
00089 //
00090 // フレームバッファオブジェクトを破棄する
00091 //
00092 void Framebuffer::discard()
00093 {

```

```

00094 // デフォルトのフレームバッファオブジェクトに戻す
00095 glBindFramebuffer(GL_FRAMEBUFFER, 0);
00096
00097 // フレームバッファオブジェクトを削除する
00098 glDeleteFramebuffers(1, &framebufferName);
00099 framebufferName = 0;
00100
00101 // バッファオブジェクトのメンバを初期化する
00102 framebufferSize = std::array<int, 2>{ 0, 0 };
00103 framebufferChannels = 0;
00104 attachment = GL_COLOR_ATTACHMENT0;
00105 }
00106
00107 //
00108 // フレームバッファオブジェクトを作成する
00109 //
00110 void Framebuffer::create(GLsizei width, GLsizei height, int channels)
00111 {
00112     // 既存のテクスチャを破棄して新しいテクスチャを作成する
00113     Texture::create(width, height, channels);
00114
00115     // まだメッシュの頂点配列オブジェクトが作られていなければ作る
00116     if (!mesh) mesh = std::make_shared<Mesh>();
00117
00118     // 指定したサイズがフレームバッファオブジェクトのサイズと同じなら何もしない
00119     if (width == framebufferSize[0] && height == framebufferSize[1]
00120         && channels == framebufferChannels) return;
00121
00122     // フレームバッファオブジェクトのサイズとチャンネル数を記録する
00123     framebufferSize = std::array<int, 2>{ width, height };
00124     framebufferChannels = channels;
00125
00126     // 以前のフレームバッファオブジェクトを削除する
00127     glDeleteFramebuffers(1, &framebufferName);
00128
00129     // フレームバッファオブジェクトを作成する
00130     glGenFramebuffers(1, &framebufferName);
00131     glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00132     glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, getTextureName(), 0);
00133     glDrawBuffers(1, &attachment);
00134     glReadBuffer(attachment);
00135     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00136 }
00137
00138 //
00139 // フレームバッファオブジェクトをコピーする
00140 //
00141 void Framebuffer::copy(const Buffer& framebuffer) noexcept
00142 {
00143     // コピー元と同じサイズの空のフレームバッファオブジェクトを作る
00144     Framebuffer::create(framebuffer.getWidth(), framebuffer.getHeight(),
00145         framebuffer.getChannels());
00146
00147     // 作成したフレームバッファオブジェクトのバッファにコピーする
00148     copyBuffer(framebuffer);
00149 }
00150
00151 //
00152 // レンダリング先をフレームバッファオブジェクトに切り替える
00153 //
00154 void Framebuffer::bindFramebuffer()
00155 {
00156     // 描画先をフレームバッファオブジェクトに切り替える
00157     glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00158
00159     // ビューポートをフレームバッファオブジェクトに合わせる
00160     glViewport(0, 0, framebufferSize[0], framebufferSize[1]);
00161 }
00162
00163 //
00164 // レンダリング先を通常のフレームバッファに戻す
00165 //
00166 void Framebuffer::unbindFramebuffer() const
00167 {
00168     // 描画先を通常のフレームバッファに戻す
00169     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00170
00171     // 読み書きを通常のフレームバッファのバックバッファに対して行う
00172     static constexpr GLenum bufs{
00173 #if defined(GL_GLES_PROTOTYPES)
00174         GL_BACK
00175 #else
00176         GL_BACK_LEFT
00177 #endif
00178     };
00179     glDrawBuffers(1, &bufs);
00180     glReadBuffer(GL_BACK);

```

```

00181 }
00182
00183 //
00184 // テクスチャを展開してフレームバッファオブジェクトを更新する
00185 //
00186 void Framebuffer::update(const std::array<int, 2>& size)
00187 {
00188     // 描画先をフレームバッファオブジェクトに切り替える
00189     glBindFramebuffer();
00190
00191     // テクスチャをフレームバッファオブジェクトに展開する
00192     mesh->drawMesh(size);
00193
00194     // 描画先を通常のフレームバッファに戻す
00195     glBindFramebuffer();
00196 }
00197
00198 //
00199 // テクスチャを展開してフレームバッファオブジェクトを更新する
00200 //
00201 void Framebuffer::update(const std::array<int, 2>& size, const Texture& frame, int unit)
00202 {
00203     // 展開するするテクスチャを指定する
00204     frame.bindTexture(unit);
00205
00206     // テクスチャをフレームバッファオブジェクトに展開する
00207     update(size);
00208
00209     // 展開するテクスチャの指定を解除する
00210     frame.unbindTexture();
00211 }
00212
00213 //
00214 // フレームバッファオブジェクトの表示
00215 //
00216 void Framebuffer::show(GLsizei width, GLsizei height) const
00217 {
00218     // フレームバッファオブジェクトの縦横比
00219     const auto f{ static_cast<float>(framebufferSize[0] * height) };
00220
00221     // ウィンドウの表示領域の縦横比
00222     const auto d{ static_cast<float>(framebufferSize[1] * width) };
00223
00224     // 実際に描画する領域
00225     GLint dx0, dy0, dx1, dy1;
00226
00227     // 表示領域の右上端の位置を求める
00228     --width;
00229     --height;
00230
00231     // フレームバッファオブジェクトの縦横比が大きかったら
00232     if (f > d)
00233     {
00234         // ウィンドウの表示領域の高さを求める
00235         const auto h{ static_cast<GLint>(d / getWidth() + 0.5f) };
00236
00237         // 表示が横長なので表示領域の横幅いっぱいに表示する
00238         dx0 = 0;
00239         dx1 = width;
00240
00241         // 高さは縦横比を維持して描画する領域の中央に描く
00242         dy0 = (height - h) / 2;
00243         dy1 = dy0 + h;
00244     }
00245     else
00246     {
00247         // ウィンドウの表示領域の幅を求める
00248         const auto w{ static_cast<GLint>(f / getHeight() + 0.5f) };
00249
00250         // 表示が縦長なので表示領域の高さいっぱいに表示する
00251         dy0 = 0;
00252         dy1 = height;
00253
00254         // 横幅は縦横比を維持して描画する領域の中央に描く
00255         dx0 = (width - w) / 2;
00256         dx1 = dx0 + w;
00257     }
00258
00259     // フレームバッファオブジェクトを読み込み元にする
00260     glBindFramebuffer(GL_READ_FRAMEBUFFER, framebufferName);
00261     glReadBuffer(GL_ATTACHMENT0);
00262
00263     // 現在のフレームバッファを背景色で塗りつぶす
00264     glClear(GL_COLOR_BUFFER_BIT);
00265
00266     // フレームバッファオブジェクトの内容を通常のフレームバッファに書き込む
00267     glBlitFramebuffer(0, 0, getWidth() - 1, getHeight() - 1,

```

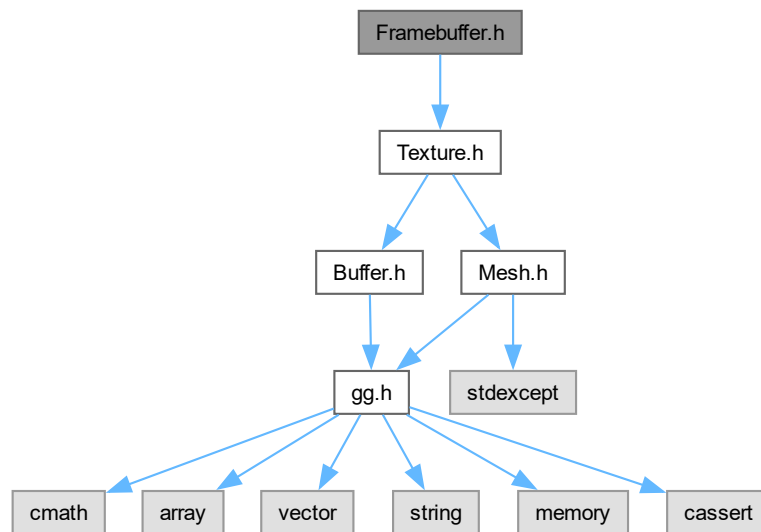
```

00268     dx0, dy1, dx1, dy0, GL_COLOR_BUFFER_BIT, GL_NEAREST);
00269
00270     // 読み込み元を通常のフレームバッファに戻す
00271     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00272     glReadBuffer(GL_BACK);
00273 }

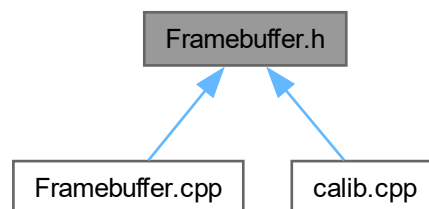
```

9.37 Framebuffer.h ファイル

#include "Texture.h"
 Framebuffer.h の依存先関係図:



被依存関係図:



クラス

- class [Framebuffer](#)

9.37.1 詳解

フレームバッファオブジェクトクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Framebuffer.h](#) に定義があります。

9.38 Framebuffer.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // テクスチャ
00012 #include "Texture.h"
00013
00017 class Framebuffer : public Texture
00018 {
00020     std::array<int, 2> framebufferSize{ 0, 0 };
00021
00023     int framebufferChannels{ 0 };
00024
00026     GLuint framebufferName{ 0 };
00027
00029     GLenum attachment{ GL_COLOR_ATTACHMENT0 };
00030
00031 public:
00032
00036     Framebuffer() = default;
00037
00046     Framebuffer(GLsizei width, GLsizei height, int channels = 3,
00047                 GLenum attachment = GL_COLOR_ATTACHMENT0);
00048
00054     Framebuffer(const Framebuffer& framebuffer);
00055
00061     Framebuffer(Framebuffer&& framebuffer) noexcept
00062         : Texture{ std::move(framebuffer) }
00063         , framebufferSize{ framebuffer.framebufferSize }
00064         , framebufferChannels{ framebuffer.framebufferChannels }
00065         , framebufferName{ framebuffer.framebufferName }
00066         , attachment{ framebuffer.attachment }
00067     {
00068         framebuffer.framebufferSize = { 0, 0 };
00069         framebuffer.framebufferChannels = 0;
00070
00071         // ムーブ元のデストラクタでフレームバッファが削除されないよう 0 にする
00072         framebuffer.framebufferName = 0;
00073         framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00074     }
00075
00079     virtual ~Framebuffer();
00080
00087     Framebuffer& operator=(const Framebuffer& framebuffer);
00088
00095     Framebuffer& operator=(Framebuffer&& framebuffer) noexcept;
00096
00109     virtual void create(GLsizei width, GLsizei height, int channels);
00110
00124     virtual void copy(const Buffer& framebuffer) noexcept;
00125
00129     virtual void discard();
00130
00136     auto getFramebufferName() const
00137     {

```

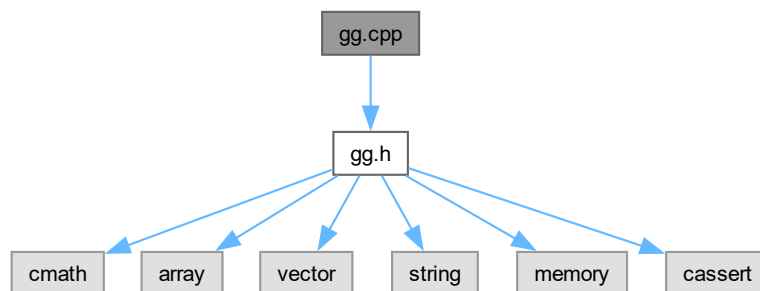
```

00138     return framebufferName;
00139 }
00140
00146 virtual const std::array<GLsizei, 2>& getSize() const
00147 {
00148     return framebufferSize;
00149 }
00150
00156 virtual int getChannels() const
00157 {
00158     return framebufferChannels;
00159 }
00160
00170 void bindFramebuffer();
00171
00181 void unbindFramebuffer() const;
00182
00193 void update(const std::array<int, 2>& size);
00194
00210 void update(const std::array<int, 2>& size, const Texture& frame, int unit = 0);
00211
00218 void show(GLsizei width, GLsizei height) const;
00219 };

```

9.39 gg.cpp ファイル

#include "gg.h"
gg.cpp の依存先関係図:



9.39.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の定義.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.cpp](#) に定義があります。

9.40 gg.cpp

[詳解]

```

00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 #include "gg.h"
00026
00027 // 標準ライブラリ
00028 #include <cmath>
00029 #include <cstdlib>
00030 #include <iostream>
00031 #include <fstream>
00032 #include <sstream>
00033 #include <limits>
00034 #include <map>
00035
00036 #define READ_TEXTURE_COORDINATE_FROM_OBJ 0
00037
00038 // Windows (Visual Studio) のとき
00039 #if defined(_WIN32)
00040 // デバッグビルドかどうか調べて
00041 # if defined(DEBUG)
00042 // デバッグビルドならそのことを示す記号定数を別に定義して
00043 #   define DEBUG
00044 // デバッグビルド用のライブラリをリンクする
00045 #   pragma comment(lib, "glfw3.lib")
00046 # else
00047 // リリースビルドならコンソールにメッセージを出さないようにして
00048 #   pragma comment(linker, "/subsystem:\"windows\" /entry:\"mainCRTStartup\"")
00049 // リリースビルド用のライブラリをリンクする
00050 #   pragma comment(lib, "glfw3.lib")
00051 # endif
00052 #endif
00053
00054 //
00055 // For VC++ MFC Convert UTF-8 to TCHAR, or Convert TCHAR to UTF-8. VC++ MFC用 UTF-8⇔TCHARの変換処理
00056 //
00057 // Original author: mt-u
00058 // Copyright (c) 2013 mt-u
00059 // https://gist.github.com/mt-u/6878251
00060 //
00061 // Modified by: Kohe Tokoi
00062 //
00063 //
00064 // UTF-8 文字列を CString に変換する
00065 //
00066 //
00067 pathString Utf8ToTChar(const std::string& string)
00068 {
00069     // UTF-8 文字列を UTF-16 に変換した後の文字列の長さを求める
00070     const INT length{ MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, NULL, 0) };
00071
00072     // 変換結果の格納に必要な長さのメモリを確保する
00073     std::vector<WCHAR> utf16(length);
00074
00075     // UTF-8 文字列を UTF-16 に変換する
00076     MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, utf16.data(), length);
00077
00078     // 変換した文字列を CString にして返す
00079     return CString{ utf16.data(), static_cast<int>(wcslen(utf16.data())) };
00080 }
00081
00082
00083

```

```

00092 // CString を UTF-8 文字列に変換する
00093 //
00094 std::string TCharToUtf8(const pathString& cstring)
00095 {
00096     // 与えられた文字列を一旦 UTF-16 に変換する
00097     CStringW cstringw{ cstring };
00098
00099     // UTF-16 を UTF-8 に変換した後の文字列の長さを求める
00100     const INT length{ WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, NULL, 0, NULL, NULL) };
00101
00102     // 変換結果の格納に必要な長さのメモリを確保する
00103     std::vector<CHAR> utf8(length);
00104
00105     // UTF-16 を UTF-8 に変換する
00106     WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, utf8.data(), length, NULL, NULL);
00107
00108     // 変換した文字列を std::string にして返す
00109     return std::string{ utf8.data(), strlen(utf8.data()) };
00110 }
00111 #endif
00112
00113 // OpenGL 3.2 の API のエントリポイント
00114 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00115 PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00116 PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00117 PFNGLACTIVEVETEXTUREPROC glActiveTexture;
00118 PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00119 PFNGLATTACHSHADERPROC glAttachShader;
00120 PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00121 PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00122 PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00123 PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00124 PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00125 PFNGLBEGINQUERYPROC glBeginQuery;
00126 PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00127 PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00128 PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00129 PFNGLBINDBUFFERPROC glBindBuffer;
00130 PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00131 PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00132 PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00133 PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00134 PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00135 PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00136 PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00137 PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00138 PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00139 PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00140 PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00141 PFNGLBINDSAMPLERPROC glBindSampler;
00142 PFNGLBINDSAMPLERSPROC glBindSamplers;
00143 PFNGLBINDTEXTUREPROC glBindTexture;
00144 PFNGLBINDTEXTURESPROC glBindTextures;
00145 PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00146 PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00147 PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00148 PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00149 PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00150 PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00151 PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00152 PFNGLBLENDCOLORPROC glBlendColor;
00153 PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00154 PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00155 PFNGLBLEND EQUATIONPROC glBlendEquation;
00156 PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00157 PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00158 PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00159 PFNGLBLENDFUNC IARBPROC glBlendFunciARB;
00160 PFNGLBLENDFUNCIPROC glBlendFunci;
00161 PFNGLBLENDFUNCPROC glBlendFunc;
00162 PFNGLBLENDFUNCSEPARATEIARBPROC glBlendFuncSeparateiARB;
00163 PFNGLBLENDFUNCSEPARATEIPROC glBlendFuncSeparatei;
00164 PFNGLBLENDFUNCSEPARATEPROC glBlendFuncSeparate;
00165 PFNGLBLENDPARAMETERINVPROC glBlendParameteriNV;
00166 PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00167 PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00168 PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00169 PFNGLBUFFERDATAPROC glBufferData;
00170 PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00171 PFNGLBUFFERSTORAGEPROC glBufferStorage;
00172 PFNGLBUFFERSUBDATAPROC glBufferSubData;
00173 PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00174 PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00175 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00176 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00177 PFNGLCLAMP COLORPROC glClampColor;
00178 PFNGLCLEARBUFFERDATAPROC glClearBufferData;

```

```
00179 PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00180 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00181 PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00182 PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00183 PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00184 PFNGLCLEARCOLORPROC glClearColor;
00185 PFNGLCLEARDEPTHFPROC glClearDepthf;
00186 PFNGLCLEARDEPTHPROC glClearDepth;
00187 PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00188 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00189 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00190 PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00191 PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00192 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00193 PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00194 PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00195 PFNGLCLEARPROC glClear;
00196 PFNGLCLEARSTENCILPROC glClearStencil;
00197 PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00198 PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00199 PFNGLCLIENTATTRIBDEFAULTEXTPROC glClientAttribDefaultEXT;
00200 PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00201 PFNGLCLIPCONTROLPROC glClipControl;
00202 PFNGLCOLORFORMATNVPROC glColorFormatNV;
00203 PFNGLCOLORMASKIPROC glColorMaski;
00204 PFNGLCOLORMASKPROC glColorMask;
00205 PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00206 PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00207 PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00208 PFNGLCOMPILESHADERPROC glCompileShader;
00209 PFNGLCOMPRESSEDMULTITEXIMAGE1DEXTPROC glCompressedMultiTexImage1D;
00210 PFNGLCOMPRESSEDMULTITEXIMAGE2DEXTPROC glCompressedMultiTexImage2D;
00211 PFNGLCOMPRESSEDMULTITEXIMAGE3DEXTPROC glCompressedMultiTexImage3D;
00212 PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DEXTPROC glCompressedMultiTexSubImage1D;
00213 PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DEXTPROC glCompressedMultiTexSubImage2D;
00214 PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DEXTPROC glCompressedMultiTexSubImage3D;
00215 PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00216 PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00217 PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00218 PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00219 PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00220 PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00221 PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC glCompressedTextureImage1D;
00222 PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC glCompressedTextureImage2D;
00223 PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC glCompressedTextureImage3D;
00224 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC glCompressedTextureSubImage1D;
00225 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00226 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC glCompressedTextureSubImage2D;
00227 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00228 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC glCompressedTextureSubImage3D;
00229 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00230 PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00231 PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00232 PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00233 PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00234 PFNGLCOPYMULTITEXIMAGE1DEXTPROC glCopyMultiTexImage1D;
00235 PFNGLCOPYMULTITEXIMAGE2DEXTPROC glCopyMultiTexImage2D;
00236 PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC glCopyMultiTexSubImage1D;
00237 PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC glCopyMultiTexSubImage2D;
00238 PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC glCopyMultiTexSubImage3D;
00239 PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00240 PFNGLCOPYPATHNVPROC glCopyPathNV;
00241 PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00242 PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00243 PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00244 PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00245 PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00246 PFNGLCOPYTEXTUREIMAGE1DEXTPROC glCopyTextureImage1D;
00247 PFNGLCOPYTEXTUREIMAGE2DEXTPROC glCopyTextureImage2D;
00248 PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC glCopyTextureSubImage1D;
00249 PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00250 PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC glCopyTextureSubImage2D;
00251 PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00252 PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC glCopyTextureSubImage3D;
00253 PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00254 PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00255 PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00256 PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00257 PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00258 PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00259 PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00260 PFNGLCREATEBUFFERSPROC glCreateBuffers;
00261 PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00262 PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00263 PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00264 PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00265 PFNGLCREATEPROGRAMPROC glCreateProgram;
```

```
00266 PFNGLCREATEQUERIESPROC glCreateQueries;
00267 PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00268 PFNGLCREATESAMPLERSPROC glCreateSamplers;
00269 PFNGLCREATESHADERPROC glCreateShader;
00270 PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00271 PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00272 PFNGLCREATESTATESNVPROC glCreateStatesNV;
00273 PFNGLCREATESENDERFROMCLEVENTARBPROC glCreateSyncFromCLEventARB;
00274 PFNGLCREATETEXTURESPROC glCreateTextures;
00275 PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00276 PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00277 PFNGLCULLFACEPROC glCullFace;
00278 PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00279 PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00280 PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00281 PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00282 PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00283 PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00284 PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00285 PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00286 PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00287 PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00288 PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00289 PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00290 PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00291 PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00292 PFNGLDELETEPROGRAMPROC glDeleteProgram;
00293 PFNGLDELETEQUERIESPROC glDeleteQueries;
00294 PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00295 PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00296 PFNGLDELETESHADERPROC glDeleteShader;
00297 PFNGLDELETESSTATESNVPROC glDeleteStatesNV;
00298 PFNGLDELETESYNCPROC glDeleteSync;
00299 PFNGLDELETETEXTURESPROC glDeleteTextures;
00300 PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00301 PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00302 PFNGLDEPTHFUNCPROC glDepthFunc;
00303 PFNGLDEPTHMASKPROC glDepthMask;
00304 PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00305 PFNGLDEPTHRANGEFPROC glDepthRangeF;
00306 PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00307 PFNGLDEPTHRANGEPROC glDepthRange;
00308 PFNGLDETACHSHADERPROC glDetachShader;
00309 PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00310 PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00311 PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00312 PFNGLDISABLEIPROC glDisablei;
00313 PFNGLDISABLEPROC glDisable;
00314 PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00315 PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00316 PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00317 PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00318 PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00319 PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00320 PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00321 PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00322 PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00323 PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00324 PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00325 PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00326 PFNGLDRAWARRAYSPROC glDrawArrays;
00327 PFNGLDRAWBUFFERPROC glDrawBuffer;
00328 PFNGLDRAWBUFFERSPROC glDrawBuffers;
00329 PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00330 PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00331 PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00332 PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00333 PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00334 PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00335 PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00336 PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00337 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC glDrawElementsInstancedBaseVertexBaseInstance;
00338 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00339 PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00340 PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00341 PFNGLDRAWELEMENTSPROC glDrawElements;
00342 PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00343 PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00344 PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00345 PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00346 PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00347 PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00348 PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00349 PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00350 PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00351 PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00352 PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
```

```
00353 PFNGLENABLEIPROC glEnablei;
00354 PFNGLENABLEPROC glEnable;
00355 PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00356 PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00357 PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00358 PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00359 PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00360 PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00361 PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00362 PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00363 PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00364 PFNGLENDQUERYPROC glEndQuery;
00365 PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00366 PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00367 PFNGLFENCESYNCPROC glFenceSync;
00368 PFNGLFINISHPROC glFinish;
00369 PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00370 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00371 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00372 PFNGLFLUSHPROC glFlush;
00373 PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00374 PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00375 PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00376 PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00377 PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00378 PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00379 PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00380 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00381 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00382 PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00383 PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00384 PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00385 PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00386 PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00387 PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00388 PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00389 PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00390 PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00391 PFNGLFRONTFACEPROC glFrontFace;
00392 PFNGLGENBUFFERSPROC glGenBuffers;
00393 PFNGLGENERATEMIPMAPPROC glGenMipmap;
00394 PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenMultiTexMipmapEXT;
00395 PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenTextureMipmapEXT;
00396 PFNGLGENERATETEXTUREMIPMAPPROC glGenTextureMipmap;
00397 PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00398 PFNGLGENPATHSNVPROC glGenPathsNV;
00399 PFNGLGENPERFMONITORAMDPROC glGenPerfMonitorsAMD;
00400 PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00401 PFNGLGENQUERIESPROC glGenQueries;
00402 PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00403 PFNGLGENSAMPLERSPROC glGenSamplers;
00404 PFNGLGENTEXTURESPROC glGenTextures;
00405 PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00406 PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00407 PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00408 PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00409 PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00410 PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00411 PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00412 PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00413 PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00414 PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00415 PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00416 PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00417 PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00418 PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00419 PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00420 PFNGLGETBOOLEANIVPROC glGetBooleani.v;
00421 PFNGLGETBOOLEANVPROC glGetBooleanv;
00422 PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00423 PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00424 PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00425 PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00426 PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00427 PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00428 PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00429 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTexImage;
00430 PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00431 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00432 PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00433 PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00434 PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00435 PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00436 PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00437 PFNGLGETDOUBLEI.VEXTPROC glGetDoublei.vEXT;
00438 PFNGLGETDOUBLEIVPROC glGetDoublei.v;
00439 PFNGLGETDOUBLEVPROC glGetDoublev;
```

```
00440 PFNGLGETERRORPROC glGetError;
00441 PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00442 PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00443 PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00444 PFNGLGETFLOATI_VPROC glGetFloati_v;
00445 PFNGLGETFLOATVPROC glGetFloatv;
00446 PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00447 PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00448 PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00449 PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00450 PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00451 PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00452 PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00453 PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00454 PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00455 PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00456 PFNGLGETINTEGER64VPROC glGetInteger64v;
00457 PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00458 PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00459 PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00460 PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00461 PFNGLGETINTEGERVPROC glGetIntegerv;
00462 PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00463 PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00464 PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00465 PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00466 PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00467 PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00468 PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00469 PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00470 PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00471 PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00472 PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00473 PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00474 PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00475 PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00476 PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00477 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00478 PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00479 PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00480 PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00481 PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00482 PFNGLGETNAMEDBUFFERPOINTINTERVEXTPROC glGetNamedBufferPointervEXT;
00483 PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00484 PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00485 PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00486 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC glGetNamedFramebufferAttachmentParameterivEXT;
00487 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00488 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00489 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00490 PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00491 PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00492 PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00493 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00494 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00495 PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00496 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00497 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00498 PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00499 PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00500 PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00501 PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00502 PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00503 PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00504 PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00505 PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvarb;
00506 PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00507 PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvarb;
00508 PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00509 PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64varb;
00510 PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00511 PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00512 PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64varb;
00513 PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivarb;
00514 PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00515 PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00516 PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00517 PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00518 PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00519 PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00520 PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00521 PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00522 PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00523 PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00524 PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00525 PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00526 PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
```

```
00527 PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00528 PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00529 PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00530 PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00531 PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00532 PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00533 PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00534 PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00535 PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00536 PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00537 PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00538 PFNGLGETPOINTERIVEXTPROC glGetPointerivEXT;
00539 PFNGLGETPOINTERVPROC glGetPointerv;
00540 PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00541 PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00542 PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00543 PFNGLGETPROGRAMIVPROC glGetProgramiv;
00544 PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00545 PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00546 PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00547 PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00548 PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00549 PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00550 PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00551 PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00552 PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00553 PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00554 PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00555 PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00556 PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00557 PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00558 PFNGLGETQUERYIVPROC glGetQueryiv;
00559 PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00560 PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00561 PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00562 PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00563 PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00564 PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00565 PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00566 PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00567 PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00568 PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00569 PFNGLGETSHADERIVPROC glGetShaderiv;
00570 PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00571 PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00572 PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00573 PFNGLGETSTRINGIPROC glGetStringi;
00574 PFNGLGETSTRINGPROC glGetString;
00575 PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00576 PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00577 PFNGLGETSYNClVPROC glGetSynciv;
00578 PFNGLGETTEXIMAGEPROC glGetTexImage;
00579 PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00580 PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00581 PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00582 PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00583 PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00584 PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00585 PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00586 PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00587 PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00588 PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00589 PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00590 PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00591 PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00592 PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00593 PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00594 PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00595 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00596 PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00597 PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00598 PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00599 PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00600 PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00601 PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00602 PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00603 PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00604 PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00605 PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00606 PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00607 PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00608 PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00609 PFNGLGETUNIFORMDVPROC glGetUniformdv;
00610 PFNGLGETUNIFORMFVPROC glGetUniformfv;
00611 PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00612 PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00613 PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
```

```
00614 PFNGLGETUNIFORMIVPROC glGetUniformiv;
00615 PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00616 PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00617 PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00618 PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00619 PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00620 PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00621 PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00622 PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00623 PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayIntegerivEXT;
00624 PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00625 PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00626 PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointerivEXT;
00627 PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00628 PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00629 PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00630 PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00631 PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00632 PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribLdv;
00633 PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00634 PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00635 PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00636 PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointeriv;
00637 PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00638 PFNGLHINTPROC glHint;
00639 PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00640 PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00641 PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00642 PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00643 PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00644 PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00645 PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00646 PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00647 PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00648 PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00649 PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00650 PFNGLISBUFFERPROC glIsBuffer;
00651 PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00652 PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00653 PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00654 PFNGLISENABLEDIPROC glIsEnabledi;
00655 PFNGLISENABLEDPROC glIsEnabled;
00656 PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00657 PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00658 PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00659 PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00660 PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00661 PFNGLISPATHNVPROC glIsPathNV;
00662 PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00663 PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00664 PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00665 PFNGLISPROGRAMPROC glIsProgram;
00666 PFNGLISQUERYPROC glIsQuery;
00667 PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00668 PFNGLISSAMPLERPROC glIsSampler;
00669 PFNGLISSHADERPROC glIsShader;
00670 PFNGLISSTATENVPROC glIsStateNV;
00671 PFNGLISSYNCPROC glIsSync;
00672 PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00673 PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00674 PFNGLISTEXTUREPROC glIsTexture;
00675 PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00676 PFNGLISVERTEXARRAYPROC glIsVertexArray;
00677 PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00678 PFNGLLINEWIDTHPROC glLineWidth;
00679 PFNGLLINKPROGRAMPROC glLinkProgram;
00680 PFNGLLISTDRAWCOMMANDSSTATESCIENTNVPROC glListDrawCommandsStatesClientNV;
00681 PFNGLLOGICOPPROC glLogicOp;
00682 PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00683 PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00684 PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00685 PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00686 PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00687 PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00688 PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00689 PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00690 PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00691 PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00692 PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00693 PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00694 PFNGLMAPBUFFERPROC glMapBuffer;
00695 PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00696 PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00697 PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00698 PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00699 PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00700 PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
```



```
00701 PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00702 PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00703 PFNGLMATRIXLOADDEXTPROC glMatrixLoadEXT;
00704 PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00705 PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00706 PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00707 PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00708 PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00709 PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00710 PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00711 PFNGLMATRIXMULTDEXTPROC glMatrixMultdEXT;
00712 PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00713 PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00714 PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00715 PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00716 PFNGLMATRIXORTHOREXTPROC glMatrixOrthoEXT;
00717 PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00718 PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00719 PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00720 PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00721 PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00722 PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00723 PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00724 PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00725 PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00726 PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00727 PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00728 PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00729 PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00730 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00731 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00732 PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00733 PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00734 PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00735 PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00736 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00737 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00738 PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00739 PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00740 PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00741 PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00742 PFNGLMULTITEXCOORDPOINTEREEXTPROC glMultiTexCoordPointerEXT;
00743 PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00744 PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00745 PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00746 PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00747 PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00748 PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00749 PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00750 PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00751 PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00752 PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00753 PFNGLMULTITEXIMAGEDEXTPROC glMultiTexImage1DEXT;
00754 PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00755 PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00756 PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00757 PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00758 PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00759 PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00760 PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00761 PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00762 PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00763 PFNGLMULTITEXSUBIMAGEDEXTPROC glMultiTexSubImage1DEXT;
00764 PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00765 PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00766 PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00767 PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00768 PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00769 PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00770 PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00771 PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00772 PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00773 PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00774 PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00775 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00776 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00777 PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00778 PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00779 PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00780 PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00781 PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00782 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00783 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00784 PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00785 PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00786 PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00787 PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
```

```
00788 PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00789 PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00790 PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00791 PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00792 PFNGLNAMEDPROGRAMLOCALPARAMETER4DXTPROC glNamedProgramLocalParameter4dEXT;
00793 PFNGLNAMEDPROGRAMLOCALPARAMETER4DVXTPROC glNamedProgramLocalParameter4dvEXT;
00794 PFNGLNAMEDPROGRAMLOCALPARAMETER4FXTPROC glNamedProgramLocalParameter4fEXT;
00795 PFNGLNAMEDPROGRAMLOCALPARAMETER4FVXTPROC glNamedProgramLocalParameter4fvEXT;
00796 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00797 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVXTPROC glNamedProgramLocalParameterI4ivEXT;
00798 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00799 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVXTPROC glNamedProgramLocalParameterI4uivEXT;
00800 PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVXTPROC glNamedProgramLocalParameters4fvEXT;
00801 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVXTPROC glNamedProgramLocalParametersI4ivEXT;
00802 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVXTPROC glNamedProgramLocalParametersI4uivEXT;
00803 PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00804 PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00805 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
glNamedRenderbufferStorageMultisampleCoverageEXT;
00806 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00807 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00808 PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00809 PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00810 PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00811 PFNGLOBBJECTLABELPROC glObjectLabel;
00812 PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00813 PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00814 PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00815 PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00816 PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00817 PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00818 PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00819 PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00820 PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00821 PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00822 PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00823 PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00824 PFNGLPATHPARAMETERFNVPROC glPathParameterfvNV;
00825 PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00826 PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00827 PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00828 PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00829 PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00830 PFNGLPATHSTRINGNVPROC glPathStringNV;
00831 PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00832 PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00833 PFNGLPAUSESETTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00834 PFNGLPIXELSTOREFPROC glPixelStoref;
00835 PFNGLPIXELSTOREIPROC glPixelStorei;
00836 PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00837 PFNGLPOINTPARAMETERFPROC glPointParameterf;
00838 PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00839 PFNGLPOINTPARAMETERIPROC glPointParameteri;
00840 PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00841 PFNGLPOINTSIZEPROC glPointSize;
00842 PFNGLPOLYGONMODEPROC glPolygonMode;
00843 PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00844 PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00845 PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00846 PFNGLPOPDEBUGGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00847 PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00848 PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00849 PFNGLPROGRAMBINARYPROC glProgramBinary;
00850 PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00851 PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00852 PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00853 PFNGLPROGRAMUNIFORM1DXTPROC glProgramUniform1dEXT;
00854 PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00855 PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00856 PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00857 PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00858 PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00859 PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00860 PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00861 PFNGLPROGRAMUNIFORM1I4ARBPROC glProgramUniform1i64ARB;
00862 PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00863 PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00864 PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00865 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00866 PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00867 PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00868 PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00869 PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00870 PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00871 PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00872 PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00873 PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
```

```
00874 PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00875 PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00876 PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00877 PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00878 PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00879 PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00880 PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00881 PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00882 PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00883 PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00884 PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00885 PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00886 PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00887 PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00888 PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00889 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00890 PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00891 PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00892 PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00893 PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00894 PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00895 PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00896 PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00897 PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00898 PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00899 PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00900 PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00901 PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00902 PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00903 PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00904 PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00905 PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00906 PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00907 PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00908 PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00909 PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00910 PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00911 PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00912 PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00913 PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00914 PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00915 PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00916 PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00917 PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00918 PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00919 PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00920 PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00921 PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00922 PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00923 PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00924 PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00925 PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00926 PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00927 PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00928 PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00929 PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00930 PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00931 PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00932 PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00933 PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00934 PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00935 PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00936 PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00937 PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00938 PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00939 PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00940 PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00941 PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00942 PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00943 PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00944 PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00945 PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00946 PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00947 PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00948 PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00949 PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00950 PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00951 PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00952 PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00953 PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00954 PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00955 PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00956 PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00957 PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00958 PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00959 PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00960 PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
```

```
00961 PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00962 PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00963 PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00964 PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00965 PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00966 PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00967 PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00968 PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00969 PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00970 PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00971 PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00972 PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00973 PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00974 PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00975 PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00976 PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00977 PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00978 PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00979 PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00980 PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00981 PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00982 PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00983 PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00984 PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00985 PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00986 PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00987 PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00988 PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00989 PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00990 PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00991 PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00992 PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC glPushClientAttribDefaultEXT;
00993 PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00994 PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00995 PFNGLQUERYCOUNTERPROC glQueryCounter;
00996 PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00997 PFNGLREADBUFFERPROC glReadBuffer;
00998 PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00999 PFNGLREADNPIXELSPROC glReadnPixels;
01000 PFNGLREADPIXELSPROC glReadPixels;
01001 PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
01002 PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
01003 PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
01004 PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
01005 PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01006 PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01007 PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01008 PFNGLSAMPLEMASKIPROC glSampleMaski;
01009 PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01010 PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01011 PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01012 PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01013 PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01014 PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01015 PFNGLSCISSORARRAYVPROC glScissorArrayv;
01016 PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01017 PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01018 PFNGLSCISSORPROC glScissor;
01019 PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01020 PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01021 PFNGLSHADERBINARYPROC glShaderBinary;
01022 PFNGLSHADERSOURCEPROC glShaderSource;
01023 PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01024 PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01025 PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01026 PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01027 PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01028 PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01029 PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01030 PFNGLSTENCILFUNCPROC glStencilFunc;
01031 PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01032 PFNGLSTENCILMASKPROC glStencilMask;
01033 PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01034 PFNGLSTENCILOPPROC glStencilOp;
01035 PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01036 PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01037 PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01038 PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01039 PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01040 PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01041 PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01042 PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01043 PFNGLTEXBUFFERARBPROC glTexBufferARB;
01044 PFNGLTEXBUFFERPROC glTexBuffer;
01045 PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01046 PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01047 PFNGLTEXIMAGE1DPROC glTexImage1D;
```

```
01048 PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01049 PFNGLTEXIMAGE2DPROC glTexImage2D;
01050 PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01051 PFNGLTEXIMAGE3DPROC glTexImage3D;
01052 PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01053 PFNGLTEXPARAMETERFPROC glTexParameterf;
01054 PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01055 PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01056 PFNGLTEXPARAMETERIPROC glTexParameteri;
01057 PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01058 PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01059 PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01060 PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01061 PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01062 PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01063 PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01064 PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01065 PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01066 PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01067 PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01068 PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01069 PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01070 PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01071 PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01072 PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01073 PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01074 PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01075 PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01076 PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01077 PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01078 PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01079 PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01080 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01081 PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01082 PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01083 PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01084 PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01085 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01086 PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01087 PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01088 PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01089 PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01090 PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01091 PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01092 PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01093 PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01094 PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01095 PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01096 PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01097 PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01098 PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01099 PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01100 PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01101 PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01102 PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01103 PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01104 PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01105 PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01106 PFNGLTEXTUREVIEWPROC glTextureView;
01107 PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01108 PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01109 PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01110 PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01111 PFNGLUNIFORM1DPROC glUniform1d;
01112 PFNGLUNIFORM1DVPROC glUniform1dv;
01113 PFNGLUNIFORM1FPROC glUniform1f;
01114 PFNGLUNIFORM1FVPROC glUniform1fv;
01115 PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01116 PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01117 PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01118 PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01119 PFNGLUNIFORM1IPROC glUniform1i;
01120 PFNGLUNIFORM1IVPROC glUniform1iv;
01121 PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01122 PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01123 PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01124 PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01125 PFNGLUNIFORM1UIPROC glUniform1ui;
01126 PFNGLUNIFORM1UIVPROC glUniform1uiv;
01127 PFNGLUNIFORM2DPROC glUniform2d;
01128 PFNGLUNIFORM2DVPROC glUniform2dv;
01129 PFNGLUNIFORM2FPROC glUniform2f;
01130 PFNGLUNIFORM2FVPROC glUniform2fv;
01131 PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01132 PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01133 PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01134 PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
```

```
01135 PFNGLUNIFORM2IPROC glUniform2i;
01136 PFNGLUNIFORM2IVPROC glUniform2iv;
01137 PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01138 PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01139 PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01140 PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01141 PFNGLUNIFORM2UIPROC glUniform2ui;
01142 PFNGLUNIFORM2UIVPROC glUniform2uiv;
01143 PFNGLUNIFORM3DPROC glUniform3d;
01144 PFNGLUNIFORM3DVPROC glUniform3dv;
01145 PFNGLUNIFORM3FPROC glUniform3f;
01146 PFNGLUNIFORM3FVPROC glUniform3fv;
01147 PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01148 PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01149 PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01150 PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01151 PFNGLUNIFORM3IPROC glUniform3i;
01152 PFNGLUNIFORM3IVPROC glUniform3iv;
01153 PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01154 PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01155 PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01156 PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01157 PFNGLUNIFORM3UIPROC glUniform3ui;
01158 PFNGLUNIFORM3UIVPROC glUniform3uiv;
01159 PFNGLUNIFORM4DPROC glUniform4d;
01160 PFNGLUNIFORM4DVPROC glUniform4dv;
01161 PFNGLUNIFORM4FPROC glUniform4f;
01162 PFNGLUNIFORM4FVPROC glUniform4fv;
01163 PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01164 PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01165 PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01166 PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01167 PFNGLUNIFORM4IPROC glUniform4i;
01168 PFNGLUNIFORM4IVPROC glUniform4iv;
01169 PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01170 PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01171 PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01172 PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01173 PFNGLUNIFORM4UIPROC glUniform4ui;
01174 PFNGLUNIFORM4UIVPROC glUniform4uiv;
01175 PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01176 PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01177 PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01178 PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01179 PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01180 PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01181 PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01182 PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01183 PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01184 PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01185 PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01186 PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01187 PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01188 PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01189 PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01190 PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01191 PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01192 PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01193 PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01194 PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01195 PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01196 PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01197 PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01198 PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01199 PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01200 PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01201 PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01202 PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01203 PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01204 PFNGLUSEPROGRAMPROC glUseProgram;
01205 PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01206 PFNGLUSESHADERPROGRAMEXTPROC glUseProgramProgramEXT;
01207 PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01208 PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01209 PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01210 PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01211 PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01212 PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01213 PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01214 PFNGLVERTEXARRAYINDIRECTVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01215 PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01216 PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01217 PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01218 PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01219 PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01220 PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01221 PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
```

```
01222 PFNGLVERTEXARRAYSECONDARYCOLOROFFSEXTPROC glVertexArraySecondaryColorOffsetEXT;
01223 PFNGLVERTEXARRAYTEXCOORDOFFSEXTPROC glVertexArrayTexCoordOffsetEXT;
01224 PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01225 PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01226 PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01227 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01228 PFNGLVERTEXARRAYVERTEXATTRIBIOFFSEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01229 PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01230 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01231 PFNGLVERTEXARRAYVERTEXATTRIBOFFSEXTPROC glVertexArrayVertexAttribOffsetEXT;
01232 PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01233 PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01234 PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01235 PFNGLVERTEXARRAYVERTEXOFFSEXTPROC glVertexArrayVertexOffsetEXT;
01236 PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01237 PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01238 PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01239 PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01240 PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01241 PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01242 PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01243 PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01244 PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01245 PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01246 PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01247 PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01248 PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01249 PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01250 PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01251 PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01252 PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01253 PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01254 PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01255 PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01256 PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01257 PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01258 PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01259 PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01260 PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01261 PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01262 PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01263 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01264 PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01265 PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01266 PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01267 PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01268 PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01269 PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01270 PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01271 PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01272 PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01273 PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01274 PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01275 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01276 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01277 PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01278 PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01279 PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01280 PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01281 PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01282 PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01283 PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01284 PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01285 PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01286 PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01287 PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01288 PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01289 PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01290 PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01291 PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01292 PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01293 PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01294 PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01295 PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01296 PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01297 PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01298 PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01299 PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01300 PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01301 PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01302 PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01303 PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01304 PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01305 PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01306 PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01307 PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01308 PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
```

```

01309 PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01310 PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01311 PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01312 PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01313 PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01314 PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01315 PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01316 PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01317 PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01318 PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01319 PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01320 PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01321 PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01322 PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01323 PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01324 PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01325 PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01326 PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01327 PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01328 PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01329 PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01330 PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01331 PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01332 PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01333 PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01334 PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01335 PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01336 PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01337 PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01338 PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01339 PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01340 PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01341 PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01342 PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01343 PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01344 PFNGLVIEWPORTPROC glViewport;
01345 PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01346 PFNGLWAITSYNCPROC glWaitSync;
01347 PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01348 PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01349 PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01350 #endif
01351
01353
01355 GLint gg::ggBufferAlignment (0);
01356
01357 //
01358 // ゲームグラフィックス特論の都合にもとづく初期化
01359 //
01360 void gg::ggInit ()
01361 {
01362     //すでにこの関数が実行されていたら以降の処理を行わない
01363     if (ggBufferAlignment) return;
01364
01365     // macOS 以外で OpenGL 3.2 以降の API を取得する
01366     #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
01367         glActiveProgramEXT = PFNGLACTIVEPROGRAMEXTPROC (glfwGetProcAddress ("glActiveProgramEXT"));
01368         glActiveShaderProgram = PFNGLACTIVESHADERPROGRAMPROC (glfwGetProcAddress ("glActiveShaderProgram"));
01369         glActiveTexture = PFNGLACTIVETEXTUREPROC (glfwGetProcAddress ("glActiveTexture"));
01370         glApplyFramebufferAttachmentCMAAINTEL =
01371             PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC (glfwGetProcAddress ("glApplyFramebufferAttachmentCMAAINTEL"));
01372         glAttachShader = PFNGLATTACHSHADERPROC (glfwGetProcAddress ("glAttachShader"));
01373         glBeginConditionalRender =
01374             PFNGLBEGINCONDITIONALRENDERPROC (glfwGetProcAddress ("glBeginConditionalRender"));
01375         glBeginConditionalRenderNV =
01376             PFNGLBEGINCONDITIONALRENDERNVPROC (glfwGetProcAddress ("glBeginConditionalRenderNV"));
01377         glBeginPerfMonitorAMD = PFNGLBEGINPERFMONITORAMDPROC (glfwGetProcAddress ("glBeginPerfMonitorAMD"));
01378         glBeginPerfQueryINTEL = PFNGLBEGINPERFQUERYINTELPROC (glfwGetProcAddress ("glBeginPerfQueryINTEL"));
01379         glBeginQuery = PFNGLBEGINQUERYPROC (glfwGetProcAddress ("glBeginQuery"));
01380         glBeginQueryIndexed = PFNGLBEGINQUERYINDEXEDPROC (glfwGetProcAddress ("glBeginQueryIndexed"));
01381         glBeginTransformFeedback =
01382             PFNGLBEGINTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glBeginTransformFeedback"));
01383         glBindAttribLocation = PFNGLBINDATTRIBLOCATIONPROC (glfwGetProcAddress ("glBindAttribLocation"));
01384         glBindBuffer = PFNGLBINDBUFFERPROC (glfwGetProcAddress ("glBindBuffer"));
01385         glBindBufferBase = PFNGLBINDBUFFERBASEPROC (glfwGetProcAddress ("glBindBufferBase"));
01386         glBindBufferRange = PFNGLBINDBUFFERRANGEPROC (glfwGetProcAddress ("glBindBufferRange"));
01387         glBindBuffersBase = PFNGLBINDBUFFERSBASEPROC (glfwGetProcAddress ("glBindBuffersBase"));
01388         glBindBuffersRange = PFNGLBINDBUFFERSRANGEPROC (glfwGetProcAddress ("glBindBuffersRange"));
01389         glBindFragDataLocation =
01390             PFNGLBINDFRAGDATALOCATIONPROC (glfwGetProcAddress ("glBindFragDataLocation"));
01391         glBindFragDataLocationIndexed =
01392             PFNGLBINDFRAGDATALOCATIONINDEXEDPROC (glfwGetProcAddress ("glBindFragDataLocationIndexed"));
01393         glBindFramebuffer = PFNGLBINDFRAMEBUFFERPROC (glfwGetProcAddress ("glBindFramebuffer"));
01394         glBindImageTexture = PFNGLBINDIMAGETEXTUREPROC (glfwGetProcAddress ("glBindImageTexture"));
01395         glBindImageTextures = PFNGLBINDIMAGETEXTURESPROC (glfwGetProcAddress ("glBindImageTextures"));
01396         glBindMultiTextureEXT = PFNGLBINDMULTITEXTUREEXTPROC (glfwGetProcAddress ("glBindMultiTextureEXT"));
01397         glBindProgramPipeline = PFNGLBINDPROGRAMPIPELINEPROC (glfwGetProcAddress ("glBindProgramPipeline"));

```



```

01392     glBindRenderbuffer = PFNGLBINDRENDERBUFFERPROC (glfwGetProcAddress ("glBindRenderbuffer"));
01393     glBindSampler = PFNGLBINDSAMPLERPROC (glfwGetProcAddress ("glBindSampler"));
01394     glBindSamplers = PFNGLBINDSAMPLERSPROC (glfwGetProcAddress ("glBindSamplers"));
01395     glBindTexture = PFNGLBINDTEXTUREPROC (glfwGetProcAddress ("glBindTexture"));
01396     glBindTextureUnit = PFNGLBINDTEXTUREUNITPROC (glfwGetProcAddress ("glBindTextureUnit"));
01397     glBindTextures = PFNGLBINDTEXTURESPROC (glfwGetProcAddress ("glBindTextures"));
01398     glBindTransformFeedback =
PFNGLBINDTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glBindTransformFeedback"));
01399     glBindVertexArray = PFNGLBINDVERTEXARRAYPROC (glfwGetProcAddress ("glBindVertexArray"));
01400     glBindVertexBuffer = PFNGLBINDVERTEXBUFFERPROC (glfwGetProcAddress ("glBindVertexBuffer"));
01401     glBindVertexBuffers = PFNGLBINDVERTEXBUFFERSPROC (glfwGetProcAddress ("glBindVertexBuffers"));
01402     glBlendBarrierKHR = PFNGLBLENDARRIERKHRPROC (glfwGetProcAddress ("glBlendBarrierKHR"));
01403     glBlendBarrierNV = PFNGLBLENDBARRIERNVPROC (glfwGetProcAddress ("glBlendBarrierNV"));
01404     glBlendColor = PFNGLBLENDCOLORPROC (glfwGetProcAddress ("glBlendColor"));
01405     glBlendEquation = PFNGLBLEND EQUATIONPROC (glfwGetProcAddress ("glBlendEquation"));
01406     glBlendEquationSeparate =
PFNGLBLEND EQUATIONSEPARATEPROC (glfwGetProcAddress ("glBlendEquationSeparate"));
01407     glBlendEquationSeparatei =
PFNGLBLEND EQUATIONSEPARATEIPROC (glfwGetProcAddress ("glBlendEquationSeparatei"));
01408     glBlendEquationSeparateiARB =
PFNGLBLEND EQUATIONSEPARATEIARBPROC (glfwGetProcAddress ("glBlendEquationSeparateiARB"));
01409     glBlendEquationi = PFNGLBLEND EQUATIONIPROC (glfwGetProcAddress ("glBlendEquationi"));
01410     glBlendEquationiARB = PFNGLBLEND EQUATIONIARBPROC (glfwGetProcAddress ("glBlendEquationiARB"));
01411     glBlendFunc = PFNGLBLEND FUNCPROC (glfwGetProcAddress ("glBlendFunc"));
01412     glBlendFuncSeparate = PFNGLBLEND FUNCSEPARATEPROC (glfwGetProcAddress ("glBlendFuncSeparate"));
01413     glBlendFuncSeparatei = PFNGLBLEND FUNCSEPARATEIPROC (glfwGetProcAddress ("glBlendFuncSeparatei"));
01414     glBlendFuncSeparateiARB =
PFNGLBLEND FUNCSEPARATEIARBPROC (glfwGetProcAddress ("glBlendFuncSeparateiARB"));
01415     glBlendFunci = PFNGLBLEND FUNCIPROC (glfwGetProcAddress ("glBlendFunci"));
01416     glBlendFunciARB = PFNGLBLEND FUNCIARBPROC (glfwGetProcAddress ("glBlendFunciARB"));
01417     glBlendParameteriNV = PFNGLBLEND PARAMETERINVPROC (glfwGetProcAddress ("glBlendParameteriNV"));
01418     glBlitFramebuffer = PFNGLBLITFRAMEBUFFERPROC (glfwGetProcAddress ("glBlitFramebuffer"));
01419     glBlitNamedFramebuffer =
PFNGLBLITNAMEDFRAMEBUFFERPROC (glfwGetProcAddress ("glBlitNamedFramebuffer"));
01420     glBufferAddressRangeNV =
PFNGLBUFFERADDRESSRANGENVPROC (glfwGetProcAddress ("glBufferAddressRangeNV"));
01421     glBufferData = PFNGLBUFFERDATAPROC (glfwGetProcAddress ("glBufferData"));
01422     glBufferPageCommitmentARB =
PFNGLBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress ("glBufferPageCommitmentARB"));
01423     glBufferStorage = PFNGLBUFFERSTORAGEPROC (glfwGetProcAddress ("glBufferStorage"));
01424     glBufferSubData = PFNGLBUFFERSUBDATAPROC (glfwGetProcAddress ("glBufferSubData"));
01425     glCallCommandListNV = PFNGLCALLCOMMANDLISTNVPROC (glfwGetProcAddress ("glCallCommandListNV"));
01426     glCheckFramebufferStatus =
PFNGLCHECKFRAMEBUFFERSTATUSPROC (glfwGetProcAddress ("glCheckFramebufferStatus"));
01427     glCheckNamedFramebufferStatus =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress ("glCheckNamedFramebufferStatus"));
01428     glCheckNamedFramebufferStatusEXT =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC (glfwGetProcAddress ("glCheckNamedFramebufferStatusEXT"));
01429     glClampColor = PFNGLCLAMP COLORPROC (glfwGetProcAddress ("glClampColor"));
01430     glClear = PFNGLCLEARPROC (glfwGetProcAddress ("glClear"));
01431     glClearBufferData = PFNGLCLEARBUFFERDATAPROC (glfwGetProcAddress ("glClearBufferData"));
01432     glClearBufferSubData = PFNGLCLEARBUFFERSUBDATAPROC (glfwGetProcAddress ("glClearBufferSubData"));
01433     glClearBufferfi = PFNGLCLEARBUFFERFIPROC (glfwGetProcAddress ("glClearBufferfi"));
01434     glClearBufferfv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress ("glClearBufferfv"));
01435     glClearBufferiv = PFNGLCLEARBUFFERIVPROC (glfwGetProcAddress ("glClearBufferiv"));
01436     glClearBufferuiv = PFNGLCLEARBUFFERUIVPROC (glfwGetProcAddress ("glClearBufferuiv"));
01437     glClearColor = PFNGLCLEAR COLORPROC (glfwGetProcAddress ("glClearColor"));
01438     glClearDepth = PFNGLCLEAR DEPTHPROC (glfwGetProcAddress ("glClearDepth"));
01439     glClearDepthf = PFNGLCLEAR DEPTHFPROC (glfwGetProcAddress ("glClearDepthf"));
01440     glClearNamedBufferData =
PFNGLCLEARNAMEDBUFFERDATAPROC (glfwGetProcAddress ("glClearNamedBufferData"));
01441     glClearNamedBufferDataEXT =
PFNGLCLEARNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress ("glClearNamedBufferDataEXT"));
01442     glClearNamedBufferSubData =
PFNGLCLEARNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glClearNamedBufferSubData"));
01443     glClearNamedBufferSubDataEXT =
PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glClearNamedBufferSubDataEXT"));
01444     glClearNamedFramebufferfi =
PFNGLCLEARNAMEDFRAMEBUFFERFIPROC (glfwGetProcAddress ("glClearNamedFramebufferfi"));
01445     glClearNamedFramebufferfv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress ("glClearNamedFramebufferfv"));
01446     glClearNamedFramebufferiv =
PFNGLCLEARNAMEDFRAMEBUFFERIVPROC (glfwGetProcAddress ("glClearNamedFramebufferiv"));
01447     glClearNamedFramebufferuiv =
PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC (glfwGetProcAddress ("glClearNamedFramebufferuiv"));
01448     glClearStencil = PFNGLCLEARSTENCILPROC (glfwGetProcAddress ("glClearStencil"));
01449     glClearTexImage = PFNGLCLEARTEXIMAGEPROC (glfwGetProcAddress ("glClearTexImage"));
01450     glClearTexSubImage = PFNGLCLEARTEXSUBIMAGEPROC (glfwGetProcAddress ("glClearTexSubImage"));
01451     glClientAttribDefaultEXT =
PFNGLCLIENTATTRIBDEFAULTTEXTPROC (glfwGetProcAddress ("glClientAttribDefaultEXT"));
01452     glClientWaitSync = PFNGLCLIENTWAITSYNCPROC (glfwGetProcAddress ("glClientWaitSync"));
01453     glClipControl = PFNGLCLIPCONTROLPROC (glfwGetProcAddress ("glClipControl"));
01454     glColorFormatNV = PFNGLCOLORFORMATNVPROC (glfwGetProcAddress ("glColorFormatNV"));
01455     glColorMask = PFNGLCOLORMASKPROC (glfwGetProcAddress ("glColorMask"));
01456     glColorMaski = PFNGLCOLORMASKIPROC (glfwGetProcAddress ("glColorMaski"));
01457     glCommandListSegmentsNV =
PFNGLCOMMANDLISTSEGMENTSNVPROC (glfwGetProcAddress ("glCommandListSegmentsNV"));

```

```

01458     glCompileCommandListNV =
PFNGLCOMPILECOMMANDLISTNVPROC (glfwGetProcAddress ("glCompileCommandListNV"));
01459     glCompileShader = PFNGLCOMPILESHADERPROC (glfwGetProcAddress ("glCompileShader"));
01460     glCompileShaderIncludeARB =
PFNGLCOMPILESHADERINCLUDEARBPROC (glfwGetProcAddress ("glCompileShaderIncludeARB"));
01461     glCompressedMultiTexImage1DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage1DEXT"));
01462     glCompressedMultiTexImage2DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage2DEXT"));
01463     glCompressedMultiTexImage3DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage3DEXT"));
01464     glCompressedMultiTexSubImage1DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage1DEXT"));
01465     glCompressedMultiTexSubImage2DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage2DEXT"));
01466     glCompressedMultiTexSubImage3DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage3DEXT"));
01467     glCompressedTexImage1D =
PFNGLCOMPRESSEDTEXIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexImage1D"));
01468     glCompressedTexImage2D =
PFNGLCOMPRESSEDTEXIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexImage2D"));
01469     glCompressedTexImage3D =
PFNGLCOMPRESSEDTEXIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexImage3D"));
01470     glCompressedTexSubImage1D =
PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexSubImage1D"));
01471     glCompressedTexSubImage2D =
PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexSubImage2D"));
01472     glCompressedTexSubImage3D =
PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexSubImage3D"));
01473     glCompressedTextureImage1DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage1DEXT"));
01474     glCompressedTextureImage2DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage2DEXT"));
01475     glCompressedTextureImage3DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage3DEXT"));
01476     glCompressedTextureSubImage1D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTextureSubImage1D"));
01477     glCompressedTextureSubImage1DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage1DEXT"));
01478     glCompressedTextureSubImage2D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTextureSubImage2D"));
01479     glCompressedTextureSubImage2DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage2DEXT"));
01480     glCompressedTextureSubImage3D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTextureSubImage3D"));
01481     glCompressedTextureSubImage3DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage3DEXT"));
01482     glConservativeRasterParameterfNV =
PFNGLCONSERVATIVERASTERPARAMETERFNVPROC (glfwGetProcAddress ("glConservativeRasterParameterfNV"));
01483     glConservativeRasterParameteriNV =
PFNGLCONSERVATIVERASTERPARAMETERINVPROC (glfwGetProcAddress ("glConservativeRasterParameteriNV"));
01484     glCopyBufferSubData = PFNGLCOPYBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyBufferSubData"));
01485     glCopyImageSubData = PFNGLCOPYIMAGESUBDATAPROC (glfwGetProcAddress ("glCopyImageSubData"));
01486     glCopyMultiTexImage1DEXT =
PFNGLCOPYMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage1DEXT"));
01487     glCopyMultiTexImage2DEXT =
PFNGLCOPYMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage2DEXT"));
01488     glCopyMultiTexSubImage1DEXT =
PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage1DEXT"));
01489     glCopyMultiTexSubImage2DEXT =
PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage2DEXT"));
01490     glCopyMultiTexSubImage3DEXT =
PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage3DEXT"));
01491     glCopyNamedBufferSubData =
PFNGLCOPYNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyNamedBufferSubData"));
01492     glCopyPathNV = PFNGLCOPYPATHNVPROC (glfwGetProcAddress ("glCopyPathNV"));
01493     glCopyTexImage1D = PFNGLCOPYTEXIMAGE1DPROC (glfwGetProcAddress ("glCopyTexImage1D"));
01494     glCopyTexImage2D = PFNGLCOPYTEXIMAGE2DPROC (glfwGetProcAddress ("glCopyTexImage2D"));
01495     glCopyTexSubImage1D = PFNGLCOPYTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTexSubImage1D"));
01496     glCopyTexSubImage2D = PFNGLCOPYTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTexSubImage2D"));
01497     glCopyTexSubImage3D = PFNGLCOPYTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTexSubImage3D"));
01498     glCopyTextureImage1DEXT =
PFNGLCOPYTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureImage1DEXT"));
01499     glCopyTextureImage2DEXT =
PFNGLCOPYTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureImage2DEXT"));
01500     glCopyTextureSubImage1D =
PFNGLCOPYTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTextureSubImage1D"));
01501     glCopyTextureSubImage1DEXT =
PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage1DEXT"));
01502     glCopyTextureSubImage2D =
PFNGLCOPYTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTextureSubImage2D"));
01503     glCopyTextureSubImage2DEXT =
PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage2DEXT"));
01504     glCopyTextureSubImage3D =
PFNGLCOPYTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTextureSubImage3D"));
01505     glCopyTextureSubImage3DEXT =
PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage3DEXT"));

```

```
01506     glCoverFillPathInstancedNV =
PFNGLCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glCoverFillPathInstancedNV"));
01507     glCoverFillPathNV = PFNGLCOVERFILLPATHNVPROC (glfwGetProcAddress("glCoverFillPathNV"));
01508     glCoverStrokePathInstancedNV =
PFNGLCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glCoverStrokePathInstancedNV"));
01509     glCoverStrokePathNV = PFNGLCOVERSTROKEPATHNVPROC (glfwGetProcAddress("glCoverStrokePathNV"));
01510     glCoverageModulationNV =
PFNGLCOVERAGEMODULATIONNVPROC (glfwGetProcAddress("glCoverageModulationNV"));
01511     glCoverageModulationTableNV =
PFNGLCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress("glCoverageModulationTableNV"));
01512     glCreateBuffers = PFNGLCREATEBUFFERSPROC (glfwGetProcAddress("glCreateBuffers"));
01513     glCreateCommandListsNV =
PFNGLCREATECOMMANDLISTSNVPROC (glfwGetProcAddress("glCreateCommandListsNV"));
01514     glCreateFramebuffers = PFNGLCREATEFRAMEBUFFERSPROC (glfwGetProcAddress("glCreateFramebuffers"));
01515     glCreatePerfQueryINTEL =
PFNGLCREATEPERFQUERYINTELPROC (glfwGetProcAddress("glCreatePerfQueryINTEL"));
01516     glCreateProgram = PFNGLCREATEPROGRAMPROC (glfwGetProcAddress("glCreateProgram"));
01517     glCreateProgramPipelines =
PFNGLCREATEPROGRAMPIPELINESPROC (glfwGetProcAddress("glCreateProgramPipelines"));
01518     glCreateQueries = PFNGLCREATEQUERIESPROC (glfwGetProcAddress("glCreateQueries"));
01519     glCreateRenderbuffers = PFNGLCREATERENDERBUFFERSPROC (glfwGetProcAddress("glCreateRenderbuffers"));
01520     glCreateSamplers = PFNGLCREATESAMPLERSPROC (glfwGetProcAddress("glCreateSamplers"));
01521     glCreateShader = PFNGLCREATESHADERPROC (glfwGetProcAddress("glCreateShader"));
01522     glCreateShaderProgramEXT =
PFNGLCREATESHADERPROGRAMEXTPROC (glfwGetProcAddress("glCreateShaderProgramEXT"));
01523     glCreateShaderProgramv =
PFNGLCREATESHADERPROGRAMVPROC (glfwGetProcAddress("glCreateShaderProgramv"));
01524     glCreateStatesNV = PFNGLCREATESTATESNVPROC (glfwGetProcAddress("glCreateStatesNV"));
01525     glCreateSyncFromCLeventARB =
PFNGLCREATESYNCFROMCLEVENTARBPROC (glfwGetProcAddress("glCreateSyncFromCLeventARB"));
01526     glCreateTextures = PFNGLCREATETEXTURESPROC (glfwGetProcAddress("glCreateTextures"));
01527     glCreateTransformFeedbacks =
PFNGLCREATETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glCreateTransformFeedbacks"));
01528     glCreateVertexArrays = PFNGLCREATEVERTEXARRAYSPROC (glfwGetProcAddress("glCreateVertexArrays"));
01529     glCullFace = PFNGLCULLFACEPROC (glfwGetProcAddress("glCullFace"));
01530     glDebugMessageCallback =
PFNGLDEBUGMESSAGECALLBACKPROC (glfwGetProcAddress("glDebugMessageCallback"));
01531     glDebugMessageCallbackARB =
PFNGLDEBUGMESSAGECALLBACKARBPROC (glfwGetProcAddress("glDebugMessageCallbackARB"));
01532     glDebugMessageControl = PFNGLDEBUGMESSAGECONTROLPROC (glfwGetProcAddress("glDebugMessageControl"));
01533     glDebugMessageControlARB =
PFNGLDEBUGMESSAGECONTROLARBPROC (glfwGetProcAddress("glDebugMessageControlARB"));
01534     glDebugMessageInsert = PFNGLDEBUGMESSAGEINSERTPROC (glfwGetProcAddress("glDebugMessageInsert"));
01535     glDebugMessageInsertARB =
PFNGLDEBUGMESSAGEINSERTARBPROC (glfwGetProcAddress("glDebugMessageInsertARB"));
01536     glDeleteBuffers = PFNGLDELETEBUFFERSPROC (glfwGetProcAddress("glDeleteBuffers"));
01537     glDeleteCommandListsNV =
PFNGLDELETECOMMANDLISTSNVPROC (glfwGetProcAddress("glDeleteCommandListsNV"));
01538     glDeleteFramebuffers = PFNGLDELETEFRAMEBUFFERSPROC (glfwGetProcAddress("glDeleteFramebuffers"));
01539     glDeleteNamedStringARB =
PFNGLDELETENAMEDSTRINGARBPROC (glfwGetProcAddress("glDeleteNamedStringARB"));
01540     glDeletePathsNV = PFNGLDELETEPATHSNVPROC (glfwGetProcAddress("glDeletePathsNV"));
01541     glDeletePerfMonitorsAMD =
PFNGLDELETEPERFMONITORSSAMDPROC (glfwGetProcAddress("glDeletePerfMonitorsAMD"));
01542     glDeletePerfQueryINTEL =
PFNGLDELETEPERFQUERYINTELPROC (glfwGetProcAddress("glDeletePerfQueryINTEL"));
01543     glDeleteProgram = PFNGLDELETEPROGRAMPROC (glfwGetProcAddress("glDeleteProgram"));
01544     glDeleteProgramPipelines =
PFNGLDELETEPROGRAMPIPELINESPROC (glfwGetProcAddress("glDeleteProgramPipelines"));
01545     glDeleteQueries = PFNGLDELETEQUERIESPROC (glfwGetProcAddress("glDeleteQueries"));
01546     glDeleteRenderbuffers = PFNGLDELETERENDERBUFFERSPROC (glfwGetProcAddress("glDeleteRenderbuffers"));
01547     glDeleteSamplers = PFNGLDELETESAMPLERSPROC (glfwGetProcAddress("glDeleteSamplers"));
01548     glDeleteShader = PFNGLDELETESHADERPROC (glfwGetProcAddress("glDeleteShader"));
01549     glDeleteStatesNV = PFNGLDELETESATESNVPROC (glfwGetProcAddress("glDeleteStatesNV"));
01550     glDeleteSync = PFNGLDELETESYNCPROC (glfwGetProcAddress("glDeleteSync"));
01551     glDeleteTextures = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteTextures"));
01552     glDeleteTransformFeedbacks =
PFNGLDELETETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glDeleteTransformFeedbacks"));
01553     glDeleteVertexArrays = PFNGLDELETEVERTEXARRAYSPROC (glfwGetProcAddress("glDeleteVertexArrays"));
01554     glDepthFunc = PFNGLDEPTHFUNCPROC (glfwGetProcAddress("glDepthFunc"));
01555     glDepthMask = PFNGLDEPTHMASKPROC (glfwGetProcAddress("glDepthMask"));
01556     glDepthRange = PFNGLDEPTHRANGEPROC (glfwGetProcAddress("glDepthRange"));
01557     glDepthRangeArrayv = PFNGLDEPTHRANGEARRAYVPROC (glfwGetProcAddress("glDepthRangeArrayv"));
01558     glDepthRangeIndexed = PFNGLDEPTHRANGEINDEXEDPROC (glfwGetProcAddress("glDepthRangeIndexed"));
01559     glDepthRangef = PFNGLDEPTHRANGEFPROC (glfwGetProcAddress("glDepthRangef"));
01560     glDetachShader = PFNGLDETACHSHADERPROC (glfwGetProcAddress("glDetachShader"));
01561     glDisable = PFNGLDISABLEPROC (glfwGetProcAddress("glDisable"));
01562     glDisableClientStateIndexedEXT =
PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glDisableClientStateIndexedEXT"));
01563     glDisableClientStateiEXT =
PFNGLDISABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glDisableClientStateiEXT"));
01564     glDisableIndexedEXT = PFNGLDISABLEINDEXEDEXTPROC (glfwGetProcAddress("glDisableIndexedEXT"));
01565     glDisableVertexArrayAttrib =
PFNGLDISABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glDisableVertexArrayAttrib"));
01566     glDisableVertexArrayAttribEXT =
PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glDisableVertexArrayAttribEXT"));
01567     glDisableVertexArrayEXT =
```

```

    PFNGLDISABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glDisableVertexArrayEXT"));
01568   glDisableVertexAttribArray =
    PFNGLDISABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glDisableVertexAttribArray"));
01569   glDisablei = PFNGLDISABLEIPROC (glfwGetProcAddress("glDisablei"));
01570   glDispatchCompute = PFNGLDISPATCHCOMPUTEPROC (glfwGetProcAddress("glDispatchCompute"));
01571   glDispatchComputeGroupSizeARB =
    PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC (glfwGetProcAddress("glDispatchComputeGroupSizeARB"));
01572   glDispatchComputeIndirect =
    PFNGLDISPATCHCOMPUTEINDIRECTPROC (glfwGetProcAddress("glDispatchComputeIndirect"));
01573   glDrawArrays = PFNGLDRAWARRAYSPROC (glfwGetProcAddress("glDrawArrays"));
01574   glDrawArraysIndirect = PFNGLDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glDrawArraysIndirect"));
01575   glDrawArraysInstanced = PFNGLDRAWARRAYSINSTANCEDPROC (glfwGetProcAddress("glDrawArraysInstanced"));
01576   glDrawArraysInstancedARB =
    PFNGLDRAWARRAYSINSTANCEDARBPROC (glfwGetProcAddress("glDrawArraysInstancedARB"));
01577   glDrawArraysInstancedBaseInstance =
    PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawArraysInstancedBaseInstance"));
01578   glDrawArraysInstancedEXT =
    PFNGLDRAWARRAYSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawArraysInstancedEXT"));
01579   glDrawBuffer = PFNGLDRAWBUFFERPROC (glfwGetProcAddress("glDrawBuffer"));
01580   glDrawBuffers = PFNGLDRAWBUFFERSPROC (glfwGetProcAddress("glDrawBuffers"));
01581   glDrawCommandsAddressNV =
    PFNGLDRAWCOMMANDSADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsAddressNV"));
01582   glDrawCommandsNV = PFNGLDRAWCOMMANDSNVPROC (glfwGetProcAddress("glDrawCommandsNV"));
01583   glDrawCommandsStatesAddressNV =
    PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsStatesAddressNV"));
01584   glDrawCommandsStatesNV =
    PFNGLDRAWCOMMANDSSTATESNVPROC (glfwGetProcAddress("glDrawCommandsStatesNV"));
01585   glDrawElements = PFNGLDRAWELEMENTSPROC (glfwGetProcAddress("glDrawElements"));
01586   glDrawElementsBaseVertex =
    PFNGLDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsBaseVertex"));
01587   glDrawElementsIndirect =
    PFNGLDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress("glDrawElementsIndirect"));
01588   glDrawElementsInstanced =
    PFNGLDRAWELEMENTSINSTANCEDPROC (glfwGetProcAddress("glDrawElementsInstanced"));
01589   glDrawElementsInstancedARB =
    PFNGLDRAWELEMENTSINSTANCEDARBPROC (glfwGetProcAddress("glDrawElementsInstancedARB"));
01590   glDrawElementsInstancedBaseInstance =
    PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseInstance"));
01591   glDrawElementsInstancedBaseVertex =
    PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertex"));
01592   glDrawElementsInstancedBaseVertexBaseInstance =
    PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertexBaseInstance"));
01593   glDrawElementsInstancedEXT =
    PFNGLDRAWELEMENTSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawElementsInstancedEXT"));
01594   glDrawRangeElements = PFNGLDRAWRANGEELEMENTSPROC (glfwGetProcAddress("glDrawRangeElements"));
01595   glDrawRangeElementsBaseVertex =
    PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawRangeElementsBaseVertex"));
01596   glDrawTransformFeedback =
    PFNGLDRAWTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glDrawTransformFeedback"));
01597   glDrawTransformFeedbackInstanced =
    PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackInstanced"));
01598   glDrawTransformFeedbackStream =
    PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC (glfwGetProcAddress("glDrawTransformFeedbackStream"));
01599   glDrawTransformFeedbackStreamInstanced =
    PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackStreamInstanced"));
01600   glDrawVkImageNV = PFNGLDRAWVKIMAGENVPROC (glfwGetProcAddress("glDrawVkImageNV"));
01601   glEdgeFlagFormatNV = PFNGLEDGEFLAGFORMATNVPROC (glfwGetProcAddress("glEdgeFlagFormatNV"));
01602   glEnable = PFNGLENABLEPROC (glfwGetProcAddress("glEnable"));
01603   glEnableClientStateIndexedEXT =
    PFNGLENABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glEnableClientStateIndexedEXT"));
01604   glEnableClientStateiEXT =
    PFNGLENABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glEnableClientStateiEXT"));
01605   glEnableIndexedEXT = PFNGLENABLEINDEXEDEXTPROC (glfwGetProcAddress("glEnableIndexedEXT"));
01606   glEnableVertexArrayAttrib =
    PFNGLENABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glEnableVertexArrayAttrib"));
01607   glEnableVertexArrayAttribEXT =
    PFNGLENABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glEnableVertexArrayAttribEXT"));
01608   glEnableVertexArrayEXT =
    PFNGLENABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glEnableVertexArrayEXT"));
01609   glEnableVertexAttribArray =
    PFNGLENABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glEnableVertexAttribArray"));
01610   glEnablei = PFNGLENABLEIPROC (glfwGetProcAddress("glEnablei"));
01611   glEndConditionalRender =
    PFNGLENDCONDITIONALRENDERPROC (glfwGetProcAddress("glEndConditionalRender"));
01612   glEndConditionalRenderNV =
    PFNGLENDCONDITIONALRENDERNVPROC (glfwGetProcAddress("glEndConditionalRenderNV"));
01613   glEndPerfMonitorAMD = PFNGLENDPERFMONITORAMDPROC (glfwGetProcAddress("glEndPerfMonitorAMD"));
01614   glEndPerfQueryINTEL = PFNGLENDPERFQUERYINTELPROC (glfwGetProcAddress("glEndPerfQueryINTEL"));
01615   glEndQuery = PFNGLENDQUERYPROC (glfwGetProcAddress("glEndQuery"));
01616   glEndQueryIndexed = PFNGLENDQUERYINDEXEDPROC (glfwGetProcAddress("glEndQueryIndexed"));
01617   glEndTransformFeedback =
    PFNGLENDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glEndTransformFeedback"));
01618   glEvaluateDepthValuesARB =
    PFNGLEVALUATEDEPTHVALUESARBPROC (glfwGetProcAddress("glEvaluateDepthValuesARB"));
01619   glFenceSync = PFNGLFENCESYNCPROC (glfwGetProcAddress("glFenceSync"));
01620   glFinish = PFNGLFINISHPROC (glfwGetProcAddress("glFinish"));
01621   glFlush = PFNGLFLUSHPROC (glfwGetProcAddress("glFlush"));

```



```

01622     glFlushMappedBufferRange =
PFNGLFLUSHMAPPEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedBufferRange"));
01623     glFlushMappedNamedBufferRange =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedNamedBufferRange"));
01624     glFlushMappedNamedBufferRangeEXT =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glFlushMappedNamedBufferRangeEXT"));
01625     glFogCoordFormatNV = PFNGLFOGCOORDFORMATNVPROC (glfwGetProcAddress("glFogCoordFormatNV"));
01626     glFragmentCoverageColorNV =
PFNGLFRAGMENTCOVERAGECOLORNVPROC (glfwGetProcAddress("glFragmentCoverageColorNV"));
01627     glFramebufferDrawBufferEXT =
PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC (glfwGetProcAddress("glFramebufferDrawBufferEXT"));
01628     glFramebufferDrawBuffersEXT =
PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC (glfwGetProcAddress("glFramebufferDrawBuffersEXT"));
01629     glFramebufferParameteri =
PFNGLFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress("glFramebufferParameteri"));
01630     glFramebufferReadBufferEXT =
PFNGLFRAMEBUFFERREADBUFFEREXTPROC (glfwGetProcAddress("glFramebufferReadBufferEXT"));
01631     glFramebufferRenderbuffer =
PFNGLFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress("glFramebufferRenderbuffer"));
01632     glFramebufferSampleLocationsfvARB =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress("glFramebufferSampleLocationsfvARB"));
01633     glFramebufferSampleLocationsfvNV =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress("glFramebufferSampleLocationsfvNV"));
01634     glFramebufferTexture = PFNGLFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress("glFramebufferTexture"));
01635     glFramebufferTexture1D =
PFNGLFRAMEBUFFERTEXTURE1DPROC (glfwGetProcAddress("glFramebufferTexture1D"));
01636     glFramebufferTexture2D =
PFNGLFRAMEBUFFERTEXTURE2DPROC (glfwGetProcAddress("glFramebufferTexture2D"));
01637     glFramebufferTexture3D =
PFNGLFRAMEBUFFERTEXTURE3DPROC (glfwGetProcAddress("glFramebufferTexture3D"));
01638     glFramebufferTextureARB =
PFNGLFRAMEBUFFERTEXTUREARBPROC (glfwGetProcAddress("glFramebufferTextureARB"));
01639     glFramebufferTextureFaceARB =
PFNGLFRAMEBUFFERTEXTUREFACEARBPROC (glfwGetProcAddress("glFramebufferTextureFaceARB"));
01640     glFramebufferTextureLayer =
PFNGLFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress("glFramebufferTextureLayer"));
01641     glFramebufferTextureLayerARB =
PFNGLFRAMEBUFFERTEXTURELAYERARBPROC (glfwGetProcAddress("glFramebufferTextureLayerARB"));
01642     glFramebufferTextureMultiviewOVR =
PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC (glfwGetProcAddress("glFramebufferTextureMultiviewOVR"));
01643     glFrontFace = PFNGLFRONTFACEPROC (glfwGetProcAddress("glFrontFace"));
01644     glGenBuffers = PFNGLGENBUFFERSPROC (glfwGetProcAddress("glGenBuffers"));
01645     glGenFramebuffers = PFNGLGENFRAMEBUFFERSPROC (glfwGetProcAddress("glGenFramebuffers"));
01646     glGenPathsNV = PFNGLGENPATHSNVPROC (glfwGetProcAddress("glGenPathsNV"));
01647     glGenPerfMonitorsAMD = PFNGLGENPERFMONITORSAMDPROC (glfwGetProcAddress("glGenPerfMonitorsAMD"));
01648     glGenProgramPipelines = PFNGLGENPROGRAMPIPELINESPROC (glfwGetProcAddress("glGenProgramPipelines"));
01649     glGenQueries = PFNGLGENQUERIESPROC (glfwGetProcAddress("glGenQueries"));
01650     glGenRenderbuffers = PFNGLGENRENDERBUFFERSPROC (glfwGetProcAddress("glGenRenderbuffers"));
01651     glGenSamplers = PFNGLGENSAMPLERSPROC (glfwGetProcAddress("glGenSamplers"));
01652     glGenTextures = PFNGLGENTEXTURESPROC (glfwGetProcAddress("glGenTextures"));
01653     glGenTransformFeedbacks =
PFNGLGENTRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glGenTransformFeedbacks"));
01654     glGenVertexArrays = PFNGLGENVERTEXARRAYSPROC (glfwGetProcAddress("glGenVertexArrays"));
01655     glGenerateMipmap = PFNGLGENERATEMIPMAPPROC (glfwGetProcAddress("glGenerateMipmap"));
01656     glGenerateMultiTexMipmapEXT =
PFNGLGENERATEMULTITEXMIPMAPEXTPROC (glfwGetProcAddress("glGenerateMultiTexMipmapEXT"));
01657     glGenerateTextureMipmap =
PFNGLGENERATETEXTUREMIPMAPPROC (glfwGetProcAddress("glGenerateTextureMipmap"));
01658     glGenerateTextureMipmapEXT =
PFNGLGENERATETEXTUREMIPMAPEXTPROC (glfwGetProcAddress("glGenerateTextureMipmapEXT"));
01659     glGetActiveAtomicCounterBufferiv =
PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC (glfwGetProcAddress("glGetActiveAtomicCounterBufferiv"));
01660     glGetActiveAttrib = PFNGLGETACTIVEATTRIBPROC (glfwGetProcAddress("glGetActiveAttrib"));
01661     glGetActiveSubroutineName =
PFNGLGETACTIVESUBROUTINENAMEPROC (glfwGetProcAddress("glGetActiveSubroutineName"));
01662     glGetActiveSubroutineUniformName =
PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC (glfwGetProcAddress("glGetActiveSubroutineUniformName"));
01663     glGetActiveSubroutineUniformiv =
PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC (glfwGetProcAddress("glGetActiveSubroutineUniformiv"));
01664     glGetActiveUniform = PFNGLGETACTIVEUNIFORMPROC (glfwGetProcAddress("glGetActiveUniform"));
01665     glGetActiveUniformBlockName =
PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC (glfwGetProcAddress("glGetActiveUniformBlockName"));
01666     glGetActiveUniformBlockiv =
PFNGLGETACTIVEUNIFORMBLOCKIVPROC (glfwGetProcAddress("glGetActiveUniformBlockiv"));
01667     glGetActiveUniformName =
PFNGLGETACTIVEUNIFORMNAMEPROC (glfwGetProcAddress("glGetActiveUniformName"));
01668     glGetActiveUniformsiv = PFNGLGETACTIVEUNIFORMSIVPROC (glfwGetProcAddress("glGetActiveUniformsiv"));
01669     glGetAttachedShaders = PFNGLGETATTACHEDSHADERSPROC (glfwGetProcAddress("glGetAttachedShaders"));
01670     glGetAttribLocation = PFNGLGETATTRIBLOCATIONPROC (glfwGetProcAddress("glGetAttribLocation"));
01671     glGetBooleanIndexedvEXT =
PFNGLGETBOOLEANINDEXEDVEXTPROC (glfwGetProcAddress("glGetBooleanIndexedvEXT"));
01672     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress("glGetBooleani_v"));
01673     glGetBooleani_v = PFNGLGETBOOLEANVPROC (glfwGetProcAddress("glGetBooleani_v"));
01674     glGetBufferParameteri64v =
PFNGLGETBUFFERPARAMETERI64VPROC (glfwGetProcAddress("glGetBufferParameteri64v"));
01675     glGetBufferParameteriv =
PFNGLGETBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetBufferParameteriv"));

```

```

01676     glGetBufferParameterui64vNV =
PFNGLGETBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetBufferParameterui64vNV"));
01677     glGetBufferPointerv = PFNGLGETBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetBufferPointerv"));
01678     glGetBufferSubData = PFNGLGETBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetBufferSubData"));
01679     glGetCommandHeaderNV = PFNGLGETCOMMANDHEADERNVPROC (glfwGetProcAddress ("glGetCommandHeaderNV"));
01680     glGetCompressedMultiTexImageEXT =
PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedMultiTexImageEXT"));
01681     glGetCompressedTexImage =
PFNGLGETCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress ("glGetCompressedTexImage"));
01682     glGetCompressedTextureImage =
PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureImage"));
01683     glGetCompressedTextureImageEXT =
PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedTextureImageEXT"));
01684     glGetCompressedTextureSubImage =
PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureSubImage"));
01685     glGetCoverageModulationTableNV =
PFNGLGETCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress ("glGetCoverageModulationTableNV"));
01686     glGetDebugMessageLog = PFNGLGETDEBUGMESSAGELOGPROC (glfwGetProcAddress ("glGetDebugMessageLog"));
01687     glGetDebugMessageLogARB =
PFNGLGETDEBUGMESSAGELOGARBPROC (glfwGetProcAddress ("glGetDebugMessageLogARB"));
01688     glGetDoubleIndexeddvEXT =
PFNGLGETDOUBLEINDEXEDVEXTPROC (glfwGetProcAddress ("glGetDoubleIndexeddvEXT"));
01689     glGetDoublei_v = PFNGLGETDOUBLEI_VPROC (glfwGetProcAddress ("glGetDoublei_v"));
01690     glGetDoublei_vEXT = PFNGLGETDOUBLEI_VEXTPROC (glfwGetProcAddress ("glGetDoublei_vEXT"));
01691     glGetDoublev = PFNGLGETDOUBLEVPROC (glfwGetProcAddress ("glGetDoublev"));
01692     glGetError = PFNGLGETERRORPROC (glfwGetProcAddress ("glGetError"));
01693     glGetFirstPerfQueryIdINTEL =
PFNGLGETFIRSTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetFirstPerfQueryIdINTEL"));
01694     glGetFloatIndexeddvEXT = PFNGLGETFLOATINDEXEDVEXTPROC (glfwGetProcAddress ("glGetFloatIndexeddvEXT"));
01695     glGetFloati_v = PFNGLGETFLOATI_VPROC (glfwGetProcAddress ("glGetFloati_v"));
01696     glGetFloati_vEXT = PFNGLGETFLOATI_VEXTPROC (glfwGetProcAddress ("glGetFloati_vEXT"));
01697     glGetFloatv = PFNGLGETFLOATVPROC (glfwGetProcAddress ("glGetFloatv"));
01698     glGetFragDataIndex = PFNGLGETFRAGDATAINDEXPROC (glfwGetProcAddress ("glGetFragDataIndex"));
01699     glGetFragDataLocation = PFNGLGETFRAGDATALOCATIONPROC (glfwGetProcAddress ("glGetFragDataLocation"));
01700     glGetFramebufferAttachmentParameteriv =
PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferAttachmentParameteriv"));
01701     glGetFramebufferParameteriv =
PFNGLGETFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferParameteriv"));
01702     glGetFramebufferParameterivEXT =
PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetFramebufferParameterivEXT"));
01703     glGetGraphicsResetStatus =
PFNGLGETGRAPHICSRESETSTATUSPROC (glfwGetProcAddress ("glGetGraphicsResetStatus"));
01704     glGetGraphicsResetStatusARB =
PFNGLGETGRAPHICSRESETSTATUSARBPROC (glfwGetProcAddress ("glGetGraphicsResetStatusARB"));
01705     glGetImageHandleARB = PFNGLGETIMAGEHANDLEARBPROC (glfwGetProcAddress ("glGetImageHandleARB"));
01706     glGetImageHandleNV = PFNGLGETIMAGEHANDLENVPROC (glfwGetProcAddress ("glGetImageHandleNV"));
01707     glGetInteger64i_v = PFNGLGETINTEGER64I_VPROC (glfwGetProcAddress ("glGetInteger64i_v"));
01708     glGetInteger64v = PFNGLGETINTEGER64VPROC (glfwGetProcAddress ("glGetInteger64v"));
01709     glGetIntegerIndexeddvEXT =
PFNGLGETINTEGERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetIntegerIndexeddvEXT"));
01710     glGetIntegeri_v = PFNGLGETINTEGERI_VPROC (glfwGetProcAddress ("glGetIntegeri_v"));
01711     glGetIntegerui64i_vNV = PFNGLGETINTEGERUI64I_VNVPROC (glfwGetProcAddress ("glGetIntegerui64i_vNV"));
01712     glGetIntegerui64vNV = PFNGLGETINTEGERUI64VNVPROC (glfwGetProcAddress ("glGetIntegerui64vNV"));
01713     glGetIntegerv = PFNGLGETINTEGERVPROC (glfwGetProcAddress ("glGetIntegerv"));
01714     glGetInternalformatSampleivNV =
PFNGLGETINTERNALFORMATSAMPLEIVNVPROC (glfwGetProcAddress ("glGetInternalformatSampleivNV"));
01715     glGetInternalformati64v =
PFNGLGETINTERNALFORMATI64VPROC (glfwGetProcAddress ("glGetInternalformati64v"));
01716     glGetInternalformativ = PFNGLGETINTERNALFORMATIVPROC (glfwGetProcAddress ("glGetInternalformativ"));
01717     glGetMultiTexEnvfvEXT = PFNGLGETMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvfvEXT"));
01718     glGetMultiTexEnvivEXT = PFNGLGETMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvivEXT"));
01719     glGetMultiTexGendvEXT = PFNGLGETMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glGetMultiTexGendvEXT"));
01720     glGetMultiTexGenfvEXT = PFNGLGETMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenfvEXT"));
01721     glGetMultiTexGenivEXT = PFNGLGETMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenivEXT"));
01722     glGetMultiTexImageEXT = PFNGLGETMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetMultiTexImageEXT"));
01723     glGetMultiTexLevelParameterfvEXT =
PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterfvEXT"));
01724     glGetMultiTexLevelParameterivEXT =
PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterivEXT"));
01725     glGetMultiTexParameterIivEXT =
PFNGLGETMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIivEXT"));
01726     glGetMultiTexParameterIuivEXT =
PFNGLGETMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIuivEXT"));
01727     glGetMultiTexParameterfvEXT =
PFNGLGETMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterfvEXT"));
01728     glGetMultiTexParameterivEXT =
PFNGLGETMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterivEXT"));
01729     glGetMultisamplefv = PFNGLGETMULTISAMPLEFVPROC (glfwGetProcAddress ("glGetMultisamplefv"));
01730     glGetNamedBufferParameteri64v =
PFNGLGETNAMEDBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetNamedBufferParameteri64v"));
01731     glGetNamedBufferParameteriv =
PFNGLGETNAMEDBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedBufferParameteriv"));
01732     glGetNamedBufferParameterivEXT =
PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedBufferParameterivEXT"));
01733     glGetNamedBufferParameterui64vNV =
PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetNamedBufferParameterui64vNV"));
01734     glGetNamedBufferPointerv =

```

```
PFNGLGETNAMEDBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetNamedBufferPointerv"));
01735 glGetNamedBufferPointervEXT =
PFNGLGETNAMEDBUFFERPOINTERVEXTPROC (glfwGetProcAddress ("glGetNamedBufferPointervEXT"));
01736 glGetNamedBufferSubData =
PFNGLGETNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetNamedBufferSubData"));
01737 glGetNamedBufferSubDataEXT =
PFNGLGETNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glGetNamedBufferSubDataEXT"));
01738 glGetNamedFramebufferAttachmentParameteriv =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameteriv"));
01739 glGetNamedFramebufferAttachmentParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameterivEXT"));
01740 glGetNamedFramebufferParameteriv =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferParameteriv"));
01741 glGetNamedFramebufferParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferParameterivEXT"));
01742 glGetNamedProgramLocalParameterIivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIivEXT"));
01743 glGetNamedProgramLocalParameterIuivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIuivEXT"));
01744 glGetNamedProgramLocalParameterdvEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterdvEXT"));
01745 glGetNamedProgramLocalParameterfvEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterfvEXT"));
01746 glGetNamedProgramStringEXT =
PFNGLGETNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glGetNamedProgramStringEXT"));
01747 glGetNamedProgramivEXT =
PFNGLGETNAMEDPROGRAMIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramivEXT"));
01748 glGetNamedRenderbufferParameteriv =
PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameteriv"));
01749 glGetNamedRenderbufferParameterivEXT =
PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameterivEXT"));
01750 glGetNamedStringARB = PFNGLGETNAMEDSTRINGARBPROC (glfwGetProcAddress ("glGetNamedStringARB"));
01751 glGetNamedStringivARB = PFNGLGETNAMEDSTRINGIVARBPROC (glfwGetProcAddress ("glGetNamedStringivARB"));
01752 glGetNextPerfQueryIdINTEL =
PFNGLGETNEXTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetNextPerfQueryIdINTEL"));
01753 glGetObjectLabel = PFNGLGETOBJECTLABELPROC (glfwGetProcAddress ("glGetObjectLabel"));
01754 glGetObjectLabelEXT = PFNGLGETOBJECTLABELEXTPROC (glfwGetProcAddress ("glGetObjectLabelEXT"));
01755 glGetObjectPtrLabel = PFNGLGETOBJECTPTRLABELPROC (glfwGetProcAddress ("glGetObjectPtrLabel"));
01756 glGetPathCommandsNV = PFNGLGETPATHCOMMANDSNVPROC (glfwGetProcAddress ("glGetPathCommandsNV"));
01757 glGetPathCoordsNV = PFNGLGETPATHCOORDSNVPROC (glfwGetProcAddress ("glGetPathCoordsNV"));
01758 glGetPathDashArrayNV = PFNGLGETPATHDASHARRAYNVPROC (glfwGetProcAddress ("glGetPathDashArrayNV"));
01759 glGetPathLengthNV = PFNGLGETPATHLENGTHNVPROC (glfwGetProcAddress ("glGetPathLengthNV"));
01760 glGetPathMetricRangeNV =
PFNGLGETPATHMETRICRANGENVPROC (glfwGetProcAddress ("glGetPathMetricRangeNV"));
01761 glGetPathMetricsNV = PFNGLGETPATHMETRICSNVPROC (glfwGetProcAddress ("glGetPathMetricsNV"));
01762 glGetPathParameterfvNV =
PFNGLGETPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glGetPathParameterfvNV"));
01763 glGetPathParameterivNV =
PFNGLGETPATHPARAMETERIVNVPROC (glfwGetProcAddress ("glGetPathParameterivNV"));
01764 glGetPathSpacingNV = PFNGLGETPATHSPACINGNVPROC (glfwGetProcAddress ("glGetPathSpacingNV"));
01765 glGetPerfCounterInfoINTEL =
PFNGLGETPERFCOUNTERINFOINTELPROC (glfwGetProcAddress ("glGetPerfCounterInfoINTEL"));
01766 glGetPerfMonitorCounterDataAMD =
PFNGLGETPERFMONITORCOUNTERDATAAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterDataAMD"));
01767 glGetPerfMonitorCounterInfoAMD =
PFNGLGETPERFMONITORCOUNTERINFOAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterInfoAMD"));
01768 glGetPerfMonitorCounterStringAMD =
PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterStringAMD"));
01769 glGetPerfMonitorCountersAMD =
PFNGLGETPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCountersAMD"));
01770 glGetPerfMonitorGroupStringAMD =
PFNGLGETPERFMONITORGROUPSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupStringAMD"));
01771 glGetPerfMonitorGroupsAMD =
PFNGLGETPERFMONITORGROUPSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupsAMD"));
01772 glGetPerfQueryDataINTEL =
PFNGLGETPERFQUERYDATAINTELPROC (glfwGetProcAddress ("glGetPerfQueryDataINTEL"));
01773 glGetPerfQueryIdByNameINTEL =
PFNGLGETPERFQUERYIDBYNAMEINTELPROC (glfwGetProcAddress ("glGetPerfQueryIdByNameINTEL"));
01774 glGetPerfQueryInfoINTEL =
PFNGLGETPERFQUERYINFOINTELPROC (glfwGetProcAddress ("glGetPerfQueryInfoINTEL"));
01775 glGetPointerIndexedvEXT =
PFNGLGETPOINTERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetPointerIndexedvEXT"));
01776 glGetPointerivEXT = PFNGLGETPOINTERIVEXTPROC (glfwGetProcAddress ("glGetPointerivEXT"));
01777 glGetPointerv = PFNGLGETPOINTERVPROC (glfwGetProcAddress ("glGetPointerv"));
01778 glGetProgramBinary = PFNGLGETPROGRAMBINARYPROC (glfwGetProcAddress ("glGetProgramBinary"));
01779 glGetProgramInfoLog = PFNGLGETPROGRAMINFOLOGPROC (glfwGetProcAddress ("glGetProgramInfoLog"));
01780 glGetProgramInterfaceiv =
PFNGLGETPROGRAMINTERFACEIVPROC (glfwGetProcAddress ("glGetProgramInterfaceiv"));
01781 glGetProgramPipelineInfoLog =
PFNGLGETPROGRAMPIPELINEINFOLOGPROC (glfwGetProcAddress ("glGetProgramPipelineInfoLog"));
01782 glGetProgramPipelineiv =
PFNGLGETPROGRAMPIPELINEIVPROC (glfwGetProcAddress ("glGetProgramPipelineiv"));
01783 glGetProgramResourceIndex =
PFNGLGETPROGRAMRESOURCEINDEXPROC (glfwGetProcAddress ("glGetProgramResourceIndex"));
01784 glGetProgramResourceLocation =
PFNGLGETPROGRAMRESOURCELOCATIONPROC (glfwGetProcAddress ("glGetProgramResourceLocation"));
01785 glGetProgramResourceLocationIndex =
```

```

    PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC (glfwGetProcAddress("glGetProgramResourceLocationIndex"));
01786   glGetProgramResourceName =
    PFNGLGETPROGRAMRESOURCEENAMEPROC (glfwGetProcAddress("glGetProgramResourceName"));
01787   glGetProgramResourcefvNV =
    PFNGLGETPROGRAMRESOURCEFVNVPROC (glfwGetProcAddress("glGetProgramResourcefvNV"));
01788   glGetProgramResourceiv =
    PFNGLGETPROGRAMRESOURCEIIVPROC (glfwGetProcAddress("glGetProgramResourceiv"));
01789   glGetProgramStageiv = PFNGLGETPROGRAMSTAGEIIVPROC (glfwGetProcAddress("glGetProgramStageiv"));
01790   glGetProgramiv = PFNGLGETPROGRAMIIVPROC (glfwGetProcAddress("glGetProgramiv"));
01791   glGetQueryBufferObjecti64v =
    PFNGLGETQUERYBUFFEROBJECTI64VPROC (glfwGetProcAddress("glGetQueryBufferObjecti64v"));
01792   glGetQueryBufferObjectiv =
    PFNGLGETQUERYBUFFEROBJECTIIVPROC (glfwGetProcAddress("glGetQueryBufferObjectiv"));
01793   glGetQueryBufferObjectui64v =
    PFNGLGETQUERYBUFFEROBJECTUI64VPROC (glfwGetProcAddress("glGetQueryBufferObjectui64v"));
01794   glGetQueryBufferObjectuiv =
    PFNGLGETQUERYBUFFEROBJECTUIVPROC (glfwGetProcAddress("glGetQueryBufferObjectuiv"));
01795   glGetQueryIndexediv = PFNGLGETQUERYINDEXEDIVPROC (glfwGetProcAddress("glGetQueryIndexediv"));
01796   glGetQueryObjecti64v = PFNGLGETQUERYOBJECTI64VPROC (glfwGetProcAddress("glGetQueryObjecti64v"));
01797   glGetQueryObjectiv = PFNGLGETQUERYOBJECTIIVPROC (glfwGetProcAddress("glGetQueryObjectiv"));
01798   glGetQueryObjectui64v = PFNGLGETQUERYOBJECTUI64VPROC (glfwGetProcAddress("glGetQueryObjectui64v"));
01799   glGetQueryObjectuiv = PFNGLGETQUERYOBJECTUIVPROC (glfwGetProcAddress("glGetQueryObjectuiv"));
01800   glGetQueryiv = PFNGLGETQUERYIIVPROC (glfwGetProcAddress("glGetQueryiv"));
01801   glGetRenderbufferParameteriv =
    PFNGLGETRENDERBUFFERPARAMETERIIVPROC (glfwGetProcAddress("glGetRenderbufferParameteriv"));
01802   glGetSamplerParameterIiv =
    PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress("glGetSamplerParameterIiv"));
01803   glGetSamplerParameterIuiv =
    PFNGLGETSAMPLERPARAMETERUIVPROC (glfwGetProcAddress("glGetSamplerParameterIuiv"));
01804   glGetSamplerParameterfv =
    PFNGLGETSAMPLERPARAMETERFVPROC (glfwGetProcAddress("glGetSamplerParameterfv"));
01805   glGetSamplerParameteriv =
    PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress("glGetSamplerParameteriv"));
01806   glGetShaderInfoLog = PFNGLGETSHADERINFOLOGPROC (glfwGetProcAddress("glGetShaderInfoLog"));
01807   glGetShaderPrecisionFormat =
    PFNGLGETSHADERPRECISIONFORMATPROC (glfwGetProcAddress("glGetShaderPrecisionFormat"));
01808   glGetShaderSource = PFNGLGETSHADERSOURCEPROC (glfwGetProcAddress("glGetShaderSource"));
01809   glGetShaderiv = PFNGLGETSHADERIIVPROC (glfwGetProcAddress("glGetShaderiv"));
01810   glGetStageIndexNV = PFNGLGETSTAGEINDEXNVPROC (glfwGetProcAddress("glGetStageIndexNV"));
01811   glGetString = PFNGLGETSTRINGPROC (glfwGetProcAddress("glGetString"));
01812   glGetStringi = PFNGLGETSTRINGIPROC (glfwGetProcAddress("glGetStringi"));
01813   glGetSubroutineIndex = PFNGLGETSUBROUTINEINDEXPROC (glfwGetProcAddress("glGetSubroutineIndex"));
01814   glGetSubroutineUniformLocation =
    PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC (glfwGetProcAddress("glGetSubroutineUniformLocation"));
01815   glGetSynciv = PFNGLGETSYNClVPROC (glfwGetProcAddress("glGetSynciv"));
01816   glGetTexImage = PFNGLGETTEXIMAGEPROC (glfwGetProcAddress("glGetTexImage"));
01817   glGetTexLevelParameterfv =
    PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress("glGetTexLevelParameterfv"));
01818   glGetTexLevelParameteriv =
    PFNGLGETTEXTURELEVELPARAMETERIIVPROC (glfwGetProcAddress("glGetTexLevelParameteriv"));
01819   glGetTexParameterIiv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress("glGetTexParameterIiv"));
01820   glGetTexParameterIuiv = PFNGLGETTEXPARAMETERUIVPROC (glfwGetProcAddress("glGetTexParameterIuiv"));
01821   glGetTexParameterfv = PFNGLGETTEXPARAMETERFVPROC (glfwGetProcAddress("glGetTexParameterfv"));
01822   glGetTexParameteriv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress("glGetTexParameteriv"));
01823   glGetTextureHandleARB = PFNGLGETTEXTUREHANDLEARBPROC (glfwGetProcAddress("glGetTextureHandleARB"));
01824   glGetTextureHandleNV = PFNGLGETTEXTUREHANDLENVPROC (glfwGetProcAddress("glGetTextureHandleNV"));
01825   glGetTextureImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress("glGetTextureImage"));
01826   glGetTextureImageEXT = PFNGLGETTEXTUREIMAGEEXTPROC (glfwGetProcAddress("glGetTextureImageEXT"));
01827   glGetTextureLevelParameterfv =
    PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress("glGetTextureLevelParameterfv"));
01828   glGetTextureLevelParameterfvEXT =
    PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC (glfwGetProcAddress("glGetTextureLevelParameterfvEXT"));
01829   glGetTextureLevelParameteriv =
    PFNGLGETTEXTURELEVELPARAMETERIIVPROC (glfwGetProcAddress("glGetTextureLevelParameteriv"));
01830   glGetTextureLevelParameterivEXT =
    PFNGLGETTEXTURELEVELPARAMETERIIVEXTPROC (glfwGetProcAddress("glGetTextureLevelParameterivEXT"));
01831   glGetTextureParameterIiv =
    PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glGetTextureParameterIiv"));
01832   glGetTextureParameterIivEXT =
    PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glGetTextureParameterIivEXT"));
01833   glGetTextureParameterIuiv =
    PFNGLGETTEXTUREPARAMETERUIVPROC (glfwGetProcAddress("glGetTextureParameterIuiv"));
01834   glGetTextureParameterIuivEXT =
    PFNGLGETTEXTUREPARAMETERUIVEXTPROC (glfwGetProcAddress("glGetTextureParameterIuivEXT"));
01835   glGetTextureParameterfv =
    PFNGLGETTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glGetTextureParameterfv"));
01836   glGetTextureParameterfvEXT =
    PFNGLGETTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glGetTextureParameterfvEXT"));
01837   glGetTextureParameteriv =
    PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glGetTextureParameteriv"));
01838   glGetTextureParameterivEXT =
    PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glGetTextureParameterivEXT"));
01839   glGetTextureSamplerHandleARB =
    PFNGLGETTEXTURESAMPLERHANDLEARBPROC (glfwGetProcAddress("glGetTextureSamplerHandleARB"));
01840   glGetTextureSamplerHandleNV =
    PFNGLGETTEXTURESAMPLERHANDLENVPROC (glfwGetProcAddress("glGetTextureSamplerHandleNV"));
01841   glGetTextureSubImage = PFNGLGETTEXTURESUBIMAGEPROC (glfwGetProcAddress("glGetTextureSubImage"));

```



```
01842     glGetTransformFeedbackVarying =
PFNGLGETTRANSFORMFEEDBACKVARYINGPROC (glfwGetProcAddress("glGetTransformFeedbackVarying"));
01843     glGetTransformFeedbacki64_v =
PFNGLGETTRANSFORMFEEDBACKI64_VPROC (glfwGetProcAddress("glGetTransformFeedbacki64_v"));
01844     glGetTransformFeedbacki_v =
PFNGLGETTRANSFORMFEEDBACKI_VPROC (glfwGetProcAddress("glGetTransformFeedbacki_v"));
01845     glGetTransformFeedbackiv =
PFNGLGETTRANSFORMFEEDBACKIVPROC (glfwGetProcAddress("glGetTransformFeedbackiv"));
01846     glGetUniformBlockIndex =
PFNGLGETUNIFORMBLOCKINDEXPROC (glfwGetProcAddress("glGetUniformBlockIndex"));
01847     glGetUniformIndices = PFNGLGETUNIFORMINDICESPROC (glfwGetProcAddress("glGetUniformIndices"));
01848     glGetUniformLocation = PFNGLGETUNIFORMLOCATIONPROC (glfwGetProcAddress("glGetUniformLocation"));
01849     glGetUniformSubroutineuiv =
PFNGLGETUNIFORMSUBROUTINEUIVPROC (glfwGetProcAddress("glGetUniformSubroutineuiv"));
01850     glGetUniformdv = PFNGLGETUNIFORMDVPROC (glfwGetProcAddress("glGetUniformdv"));
01851     glGetUniformfv = PFNGLGETUNIFORMFVPROC (glfwGetProcAddress("glGetUniformfv"));
01852     glGetUniformi64vARB = PFNGLGETUNIFORMI64VARBPROC (glfwGetProcAddress("glGetUniformi64vARB"));
01853     glGetUniformi64vNV = PFNGLGETUNIFORMI64VNVPROC (glfwGetProcAddress("glGetUniformi64vNV"));
01854     glGetUniformiv = PFNGLGETUNIFORMIVPROC (glfwGetProcAddress("glGetUniformiv"));
01855     glGetUniformi64vARB = PFNGLGETUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetUniformui64vARB"));
01856     glGetUniformi64vNV = PFNGLGETUNIFORMUI64VNVPROC (glfwGetProcAddress("glGetUniformui64vNV"));
01857     glGetUniformuiv = PFNGLGETUNIFORMUIVPROC (glfwGetProcAddress("glGetUniformuiv"));
01858     glGetVertexArrayIndexed64iv =
PFNGLGETVERTEXARRAYINDEXED64IVPROC (glfwGetProcAddress("glGetVertexArrayIndexed64iv"));
01859     glGetVertexArrayIndexediv =
PFNGLGETVERTEXARRAYINDEXEDIVPROC (glfwGetProcAddress("glGetVertexArrayIndexediv"));
01860     glGetVertexArrayIntegeri_vEXT =
PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayIntegeri_vEXT"));
01861     glGetVertexArrayIntegervEXT =
PFNGLGETVERTEXARRAYINTEGERVEXTPROC (glfwGetProcAddress("glGetVertexArrayIntegervEXT"));
01862     glGetVertexArrayPointeri_vEXT =
PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayPointeri_vEXT"));
01863     glGetVertexArrayPointervEXT =
PFNGLGETVERTEXARRAYPOINTERVEXTPROC (glfwGetProcAddress("glGetVertexArrayPointervEXT"));
01864     glGetVertexArrayiv = PFNGLGETVERTEXARRAYIVPROC (glfwGetProcAddress("glGetVertexArrayiv"));
01865     glGetVertexAttribiiv = PFNGLGETVERTEXATTRIBIIVPROC (glfwGetProcAddress("glGetVertexAttribiiv"));
01866     glGetVertexAttribiuiiv = PFNGLGETVERTEXATTRIBUIVPROC (glfwGetProcAddress("glGetVertexAttribiuiiv"));
01867     glGetVertexAttribLdv = PFNGLGETVERTEXATTRIBLDVPROC (glfwGetProcAddress("glGetVertexAttribLdv"));
01868     glGetVertexAttribLi64vNV =
PFNGLGETVERTEXATTRIBLI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLi64vNV"));
01869     glGetVertexAttribLui64vARB =
PFNGLGETVERTEXATTRIBLUI64VARBPROC (glfwGetProcAddress("glGetVertexAttribLui64vARB"));
01870     glGetVertexAttribLui64vNV =
PFNGLGETVERTEXATTRIBLUI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLui64vNV"));
01871     glGetVertexAttribPointerv =
PFNGLGETVERTEXATTRIBPOINTERVPROC (glfwGetProcAddress("glGetVertexAttribPointerv"));
01872     glGetVertexAttribdv = PFNGLGETVERTEXATTRIBDVPROC (glfwGetProcAddress("glGetVertexAttribdv"));
01873     glGetVertexAttribfv = PFNGLGETVERTEXATTRIBFVPROC (glfwGetProcAddress("glGetVertexAttribfv"));
01874     glGetVertexAttribiv = PFNGLGETVERTEXATTRIBIVPROC (glfwGetProcAddress("glGetVertexAttribiv"));
01875     glGetVkProcAddrNV = PFNGLGETVKPROCADDRNVPROC (glfwGetProcAddress("glGetVkProcAddrNV"));
01876     glGetnCompressedTexImage =
PFNGLGETNCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetnCompressedTexImage"));
01877     glGetnCompressedTexImageARB =
PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC (glfwGetProcAddress("glGetnCompressedTexImageARB"));
01878     glGetnTexImage = PFNGLGETNTEXIMAGEPROC (glfwGetProcAddress("glGetnTexImage"));
01879     glGetnTexImageARB = PFNGLGETNTEXIMAGEARBPROC (glfwGetProcAddress("glGetnTexImageARB"));
01880     glGetnUniformdv = PFNGLGETNUNIFORMDVPROC (glfwGetProcAddress("glGetnUniformdv"));
01881     glGetnUniformdvARB = PFNGLGETNUNIFORMDVARBPROC (glfwGetProcAddress("glGetnUniformdvARB"));
01882     glGetnUniformfv = PFNGLGETNUNIFORMFVPROC (glfwGetProcAddress("glGetnUniformfv"));
01883     glGetnUniformfvARB = PFNGLGETNUNIFORMFVARBPROC (glfwGetProcAddress("glGetnUniformfvARB"));
01884     glGetnUniformi64vARB = PFNGLGETNUNIFORMI64VARBPROC (glfwGetProcAddress("glGetnUniformi64vARB"));
01885     glGetnUniformiv = PFNGLGETNUNIFORMIVPROC (glfwGetProcAddress("glGetnUniformiv"));
01886     glGetnUniformivARB = PFNGLGETNUNIFORMIVARBPROC (glfwGetProcAddress("glGetnUniformivARB"));
01887     glGetnUniformi64vARB = PFNGLGETNUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetnUniformui64vARB"));
01888     glGetnUniformuiv = PFNGLGETNUNIFORMUIVPROC (glfwGetProcAddress("glGetnUniformuiv"));
01889     glGetnUniformuivARB = PFNGLGETNUNIFORMUIVARBPROC (glfwGetProcAddress("glGetnUniformuivARB"));
01890     glHint = PFNGLHINTPROC (glfwGetProcAddress("glHint"));
01891     glIndexFormatNV = PFNGLINDEXFORMATNVPROC (glfwGetProcAddress("glIndexFormatNV"));
01892     glInsertEventMarkerEXT =
PFNGLINSERTEVENTMARKEREXTPROC (glfwGetProcAddress("glInsertEventMarkerEXT"));
01893     glInterpolatePathsNV = PFNGLINTERPOLATEPATHSNVPROC (glfwGetProcAddress("glInterpolatePathsNV"));
01894     glInvalidateBufferData =
PFNGLINVALIDATEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateBufferData"));
01895     glInvalidateBufferSubData =
PFNGLINVALIDATEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateBufferSubData"));
01896     glInvalidateFramebuffer =
PFNGLINVALIDATEFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateFramebuffer"));
01897     glInvalidateNamedFramebufferData =
PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferData"));
01898     glInvalidateNamedFramebufferSubData =
PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferSubData"));
01899     glInvalidateSubFramebuffer =
PFNGLINVALIDATESUBFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateSubFramebuffer"));
01900     glInvalidateTexImage = PFNGLINVALIDATETEXIMAGEPROC (glfwGetProcAddress("glInvalidateTexImage"));
01901     glInvalidateTexSubImage =
PFNGLINVALIDATETEXSUBIMAGEPROC (glfwGetProcAddress("glInvalidateTexSubImage"));
01902     glIsBuffer = PFNGLISBUFFERPROC (glfwGetProcAddress("glIsBuffer"));
```

```

01903   glIsBufferResidentNV = PFNGLISBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsBufferResidentNV"));
01904   glIsCommandListNV = PFNGLISCOMMANDLISTNVPROC (glfwGetProcAddress("glIsCommandListNV"));
01905   glIsEnabled = PFNGLISENABLEDPROC (glfwGetProcAddress("glIsEnabled"));
01906   glIsEnabledIndexedEXT = PFNGLISENABLEDINDEXEDEXTPROC (glfwGetProcAddress("glIsEnabledIndexedEXT"));
01907   glIsEnabledi = PFNGLISENABLEDIPROC (glfwGetProcAddress("glIsEnabledi"));
01908   glIsFramebuffer = PFNGLISFRAMEBUFFERPROC (glfwGetProcAddress("glIsFramebuffer"));
01909   glIsImageHandleResidentARB =
PFNGLISIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsImageHandleResidentARB"));
01910   glIsImageHandleResidentNV =
PFNGLISIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsImageHandleResidentNV"));
01911   glIsNamedBufferResidentNV =
PFNGLISNAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsNamedBufferResidentNV"));
01912   glIsNamedStringARB = PFNGLISNAMEDSTRINGARBPROC (glfwGetProcAddress("glIsNamedStringARB"));
01913   glIsPathNV = PFNGLISPATHNVPROC (glfwGetProcAddress("glIsPathNV"));
01914   glIsPointInFillPathNV = PFNGLISPOINTINFILLPATHNVPROC (glfwGetProcAddress("glIsPointInFillPathNV"));
01915   glIsPointInStrokePathNV =
PFNGLISPOINTINSTROKEPATHNVPROC (glfwGetProcAddress("glIsPointInStrokePathNV"));
01916   glIsProgram = PFNGLISPROGRAMPROC (glfwGetProcAddress("glIsProgram"));
01917   glIsProgramPipeline = PFNGLISPROGRAMPIPELINEPROC (glfwGetProcAddress("glIsProgramPipeline"));
01918   glIsQuery = PFNGLISQUERYPROC (glfwGetProcAddress("glIsQuery"));
01919   glIsRenderbuffer = PFNGLISRENDERBUFFERPROC (glfwGetProcAddress("glIsRenderbuffer"));
01920   glIsSampler = PFNGLISSAMPLERPROC (glfwGetProcAddress("glIsSampler"));
01921   glIsShader = PFNGLISSHADERPROC (glfwGetProcAddress("glIsShader"));
01922   glIsStateNV = PFNGLISSTATENVPROC (glfwGetProcAddress("glIsStateNV"));
01923   glIsSync = PFNGLISSYNCPROC (glfwGetProcAddress("glIsSync"));
01924   glIsTexture = PFNGLISTEXTUREPROC (glfwGetProcAddress("glIsTexture"));
01925   glIsTextureHandleResidentARB =
PFNGLISTEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsTextureHandleResidentARB"));
01926   glIsTextureHandleResidentNV =
PFNGLISTEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsTextureHandleResidentNV"));
01927   glIsTransformFeedback = PFNGLISTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glIsTransformFeedback"));
01928   glIsVertexArray = PFNGLISVERTEXARRAYPROC (glfwGetProcAddress("glIsVertexArray"));
01929   glLabelObjectEXT = PFNGLLABELOBJECTEXTPROC (glfwGetProcAddress("glLabelObjectEXT"));
01930   glLineWidth = PFNGLLINEWIDTHPROC (glfwGetProcAddress("glLineWidth"));
01931   glLinkProgram = PFNGLLINKPROGRAMPROC (glfwGetProcAddress("glLinkProgram"));
01932   glListDrawCommandsStatesClientNV =
PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC (glfwGetProcAddress("glListDrawCommandsStatesClientNV"));
01933   glLogicOp = PFNGLLOGICOPPROC (glfwGetProcAddress("glLogicOp"));
01934   glMakeBufferNonResidentNV =
PFNGLMAKEBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferNonResidentNV"));
01935   glMakeBufferResidentNV =
PFNGLMAKEBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferResidentNV"));
01936   glMakeImageHandleNonResidentARB =
PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleNonResidentARB"));
01937   glMakeImageHandleNonResidentNV =
PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleNonResidentNV"));
01938   glMakeImageHandleResidentARB =
PFNGLMAKEIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleResidentARB"));
01939   glMakeImageHandleResidentNV =
PFNGLMAKEIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleResidentNV"));
01940   glMakeNamedBufferNonResidentNV =
PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferNonResidentNV"));
01941   glMakeNamedBufferResidentNV =
PFNGLMAKENAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferResidentNV"));
01942   glMakeTextureHandleNonResidentARB =
PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentARB"));
01943   glMakeTextureHandleNonResidentNV =
PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentNV"));
01944   glMakeTextureHandleResidentARB =
PFNGLMAKETEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleResidentARB"));
01945   glMakeTextureHandleResidentNV =
PFNGLMAKETEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleResidentNV"));
01946   glMapBuffer = PFNGLMAPBUFFERPROC (glfwGetProcAddress("glMapBuffer"));
01947   glMapBufferRange = PFNGLMAPBUFFERRANGEPROC (glfwGetProcAddress("glMapBufferRange"));
01948   glMapNamedBuffer = PFNGLMAPNAMEDBUFFERPROC (glfwGetProcAddress("glMapNamedBuffer"));
01949   glMapNamedBufferEXT = PFNGLMAPNAMEDBUFFEREEXTPROC (glfwGetProcAddress("glMapNamedBufferEXT"));
01950   glMapNamedBufferRange = PFNGLMAPNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glMapNamedBufferRange"));
01951   glMapNamedBufferRangeEXT =
PFNGLMAPNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glMapNamedBufferRangeEXT"));
01952   glMatrixFrustumEXT = PFNGLMATRIXFRUSTUMEXTPROC (glfwGetProcAddress("glMatrixFrustumEXT"));
01953   glMatrixLoad3x2fNV = PFNGLMATRIXLOAD3X2FNVPROC (glfwGetProcAddress("glMatrixLoad3x2fNV"));
01954   glMatrixLoad3x3fNV = PFNGLMATRIXLOAD3X3FNVPROC (glfwGetProcAddress("glMatrixLoad3x3fNV"));
01955   glMatrixLoadIdentityEXT =
PFNGLMATRIXLOADIDENTITYEXTPROC (glfwGetProcAddress("glMatrixLoadIdentityEXT"));
01956   glMatrixLoadTranspose3x3fNV =
PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC (glfwGetProcAddress("glMatrixLoadTranspose3x3fNV"));
01957   glMatrixLoadTransposedEXT =
PFNGLMATRIXLOADTRANSPOSEDEXTPROC (glfwGetProcAddress("glMatrixLoadTransposedEXT"));
01958   glMatrixLoadTransposefEXT =
PFNGLMATRIXLOADTRANSPOSEFEEXTPROC (glfwGetProcAddress("glMatrixLoadTransposefEXT"));
01959   glMatrixLoaddeEXT = PFNGLMATRIXLOADDEXTPROC (glfwGetProcAddress("glMatrixLoaddeEXT"));
01960   glMatrixLoadfEXT = PFNGLMATRIXLOADFEEXTPROC (glfwGetProcAddress("glMatrixLoadfEXT"));
01961   glMatrixMult3x2fNV = PFNGLMATRIXMULT3X2FNVPROC (glfwGetProcAddress("glMatrixMult3x2fNV"));
01962   glMatrixMult3x3fNV = PFNGLMATRIXMULT3X3FNVPROC (glfwGetProcAddress("glMatrixMult3x3fNV"));
01963   glMatrixMultTranspose3x3fNV =
PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC (glfwGetProcAddress("glMatrixMultTranspose3x3fNV"));
01964   glMatrixMultTransposedEXT =

```

```
PFNGLMATRIXMULTTRANPOSEDEXTPROC (glfwGetProcAddress("glMatrixMultTransposedEXT"));
01965 glMatrixMultTransposefEXT =
PFNGLMATRIXMULTTRANPOSEFEFEXTPROC (glfwGetProcAddress("glMatrixMultTransposefEXT"));
01966 glMatrixMultdEXT = PFNGLMATRIXMULTDEXTPROC (glfwGetProcAddress("glMatrixMultdEXT"));
01967 glMatrixMultfEXT = PFNGLMATRIXMULTFEEXTPROC (glfwGetProcAddress("glMatrixMultfEXT"));
01968 glMatrixOrthoEXT = PFNGLMATRIXORTHOEXTPROC (glfwGetProcAddress("glMatrixOrthoEXT"));
01969 glMatrixPopEXT = PFNGLMATRIXPOPEXTPROC (glfwGetProcAddress("glMatrixPopEXT"));
01970 glMatrixPushEXT = PFNGLMATRIXPUSHEXTPROC (glfwGetProcAddress("glMatrixPushEXT"));
01971 glMatrixRotatedEXT = PFNGLMATRIXROTATEDEXTPROC (glfwGetProcAddress("glMatrixRotatedEXT"));
01972 glMatrixRotatefEXT = PFNGLMATRIXROTATEFEEXTPROC (glfwGetProcAddress("glMatrixRotatefEXT"));
01973 glMatrixScaledEXT = PFNGLMATRIXSCALEDEXTPROC (glfwGetProcAddress("glMatrixScaledEXT"));
01974 glMatrixScalefEXT = PFNGLMATRIXSCALEFEEXTPROC (glfwGetProcAddress("glMatrixScalefEXT"));
01975 glMatrixTranslatedEXT = PFNGLMATRIXTRANSLATEDEXTPROC (glfwGetProcAddress("glMatrixTranslatedEXT"));
01976 glMatrixTranslatefEXT = PFNGLMATRIXTRANSLATEFEEXTPROC (glfwGetProcAddress("glMatrixTranslatefEXT"));
01977 glMaxShaderCompilerThreadsARB =
PFNGLMAXSHADERCOMPILERTHREADSARBPROC (glfwGetProcAddress("glMaxShaderCompilerThreadsARB"));
01978 glMemoryBarrier = PFNGLMEMORYBARRIERPROC (glfwGetProcAddress("glMemoryBarrier"));
01979 glMemoryBarrierByRegion =
PFNGLMEMORYBARRIERBYREGIONPROC (glfwGetProcAddress("glMemoryBarrierByRegion"));
01980 glMinSampleShading = PFNGLMINSAMPLESHADINGPROC (glfwGetProcAddress("glMinSampleShading"));
01981 glMinSampleShadingARB = PFNGLMINSAMPLESHADINGARBPROC (glfwGetProcAddress("glMinSampleShadingARB"));
01982 glMultiDrawArrays = PFNGLMULTIDRAWARRAYSPROC (glfwGetProcAddress("glMultiDrawArrays"));
01983 glMultiDrawArraysIndirect =
PFNGLMULTIDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glMultiDrawArraysIndirect"));
01984 glMultiDrawArraysIndirectBindlessCountNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress("glMultiDrawArraysIndirectBindlessCountNV"));
01985 glMultiDrawArraysIndirectBindlessNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC (glfwGetProcAddress("glMultiDrawArraysIndirectBindlessNV"));
01986 glMultiDrawArraysIndirectCountARB =
PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC (glfwGetProcAddress("glMultiDrawArraysIndirectCountARB"));
01987 glMultiDrawElements = PFNGLMULTIDRAWELEMENTSPROC (glfwGetProcAddress("glMultiDrawElements"));
01988 glMultiDrawElementsBaseVertex =
PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glMultiDrawElementsBaseVertex"));
01989 glMultiDrawElementsIndirect =
PFNGLMULTIDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress("glMultiDrawElementsIndirect"));
01990 glMultiDrawElementsIndirectBindlessCountNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress("glMultiDrawElementsIndirectBindlessCountNV"));
01991 glMultiDrawElementsIndirectBindlessNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC (glfwGetProcAddress("glMultiDrawElementsIndirectBindlessNV"));
01992 glMultiDrawElementsIndirectCountARB =
PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC (glfwGetProcAddress("glMultiDrawElementsIndirectCountARB"));
01993 glMultiTexBufferEXT = PFNGLMULTITEXBUFFEREXTPROC (glfwGetProcAddress("glMultiTexBufferEXT"));
01994 glMultiTexCoordPointerEXT =
PFNGLMULTITEXCOORDPOINTEREXTPROC (glfwGetProcAddress("glMultiTexCoordPointerEXT"));
01995 glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress("glMultiTexEnvfEXT"));
01996 glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress("glMultiTexEnvfvEXT"));
01997 glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress("glMultiTexEnvfEXT"));
01998 glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress("glMultiTexEnvfvEXT"));
01999 glMultiTexGendEXT = PFNGLMULTITEXGENDEXTPROC (glfwGetProcAddress("glMultiTexGendEXT"));
02000 glMultiTexGendvEXT = PFNGLMULTITEXGENDVEXTPROC (glfwGetProcAddress("glMultiTexGendvEXT"));
02001 glMultiTexGenfEXT = PFNGLMULTITEXGENFEXTPROC (glfwGetProcAddress("glMultiTexGenfEXT"));
02002 glMultiTexGenfvEXT = PFNGLMULTITEXGENFVEXTPROC (glfwGetProcAddress("glMultiTexGenfvEXT"));
02003 glMultiTexGeniEXT = PFNGLMULTITEXGENIEXTPROC (glfwGetProcAddress("glMultiTexGeniEXT"));
02004 glMultiTexGenivEXT = PFNGLMULTITEXGENIVEXTPROC (glfwGetProcAddress("glMultiTexGenivEXT"));
02005 glMultiTexImage1DEXT = PFNGLMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexImage1DEXT"));
02006 glMultiTexImage2DEXT = PFNGLMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexImage2DEXT"));
02007 glMultiTexImage3DEXT = PFNGLMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexImage3DEXT"));
02008 glMultiTexParameterIivEXT =
PFNGLMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIivEXT"));
02009 glMultiTexParameterIuivEXT =
PFNGLMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIuivEXT"));
02010 glMultiTexParameterfEXT =
PFNGLMULTITEXPARAMETERFEXTPROC (glfwGetProcAddress("glMultiTexParameterfEXT"));
02011 glMultiTexParameterfvEXT =
PFNGLMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress("glMultiTexParameterfvEXT"));
02012 glMultiTexParameteriEXT =
PFNGLMULTITEXPARAMETERIEXTPROC (glfwGetProcAddress("glMultiTexParameteriEXT"));
02013 glMultiTexParameterivEXT =
PFNGLMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress("glMultiTexParameterivEXT"));
02014 glMultiTexRenderbufferEXT =
PFNGLMULTITEXRENDERBUFFEREXTPROC (glfwGetProcAddress("glMultiTexRenderbufferEXT"));
02015 glMultiTexSubImage1DEXT =
PFNGLMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexSubImage1DEXT"));
02016 glMultiTexSubImage2DEXT =
PFNGLMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexSubImage2DEXT"));
02017 glMultiTexSubImage3DEXT =
PFNGLMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexSubImage3DEXT"));
02018 glNamedBufferData = PFNGLNAMEDBUFFERDATAPROC (glfwGetProcAddress("glNamedBufferData"));
02019 glNamedBufferDataEXT = PFNGLNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glNamedBufferDataEXT"));
02020 glNamedBufferPageCommitmentARB =
PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glNamedBufferPageCommitmentARB"));
02021 glNamedBufferPageCommitmentEXT =
PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glNamedBufferPageCommitmentEXT"));
02022 glNamedBufferStorage = PFNGLNAMEDBUFFERSTORAGEPROC (glfwGetProcAddress("glNamedBufferStorage"));
02023 glNamedBufferStorageEXT =
PFNGLNAMEDBUFFERSTORAGEEXTPROC (glfwGetProcAddress("glNamedBufferStorageEXT"));
02024 glNamedBufferSubData = PFNGLNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glNamedBufferSubData"));
```



```

02025     glNamedBufferSubDataEXT =
PFNGLNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedBufferSubDataEXT"));
02026     glNamedCopyBufferSubDataEXT =
PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedCopyBufferSubDataEXT"));
02027     glNamedFramebufferDrawBuffer =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferDrawBuffer"));
02028     glNamedFramebufferDrawBuffers =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC (glfwGetProcAddress ("glNamedFramebufferDrawBuffers"));
02029     glNamedFramebufferParameteri =
PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glNamedFramebufferParameteri"));
02030     glNamedFramebufferParameteriEXT =
PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC (glfwGetProcAddress ("glNamedFramebufferParameteriEXT"));
02031     glNamedFramebufferReadBuffer =
PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferReadBuffer"));
02032     glNamedFramebufferRenderbuffer =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferRenderbuffer"));
02033     glNamedFramebufferRenderbufferEXT =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC (glfwGetProcAddress ("glNamedFramebufferRenderbufferEXT"));
02034     glNamedFramebufferSampleLocationsfvARB =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvARB"));
02035     glNamedFramebufferSampleLocationsfvNV =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvNV"));
02036     glNamedFramebufferTexture =
PFNGLNAMEDFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glNamedFramebufferTexture"));
02037     glNamedFramebufferTexture1DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE1DXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture1DEXT"));
02038     glNamedFramebufferTexture2DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE2DXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture2DEXT"));
02039     glNamedFramebufferTexture3DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE3DXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture3DEXT"));
02040     glNamedFramebufferTextureEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureEXT"));
02041     glNamedFramebufferTextureFaceEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureFaceEXT"));
02042     glNamedFramebufferTextureLayer =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayer"));
02043     glNamedFramebufferTextureLayerEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayerEXT"));
02044     glNamedProgramLocalParameter4dEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dEXT"));
02045     glNamedProgramLocalParameter4dvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dvEXT"));
02046     glNamedProgramLocalParameter4fEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fEXT"));
02047     glNamedProgramLocalParameter4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fvEXT"));
02048     glNamedProgramLocalParameterI4iEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4iEXT"));
02049     glNamedProgramLocalParameterI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4ivEXT"));
02050     glNamedProgramLocalParameterI4uiEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uiEXT"));
02051     glNamedProgramLocalParameterI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uivEXT"));
02052     glNamedProgramLocalParameters4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameters4fvEXT"));
02053     glNamedProgramLocalParametersI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4ivEXT"));
02054     glNamedProgramLocalParametersI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4uivEXT"));
02055     glNamedProgramStringEXT =
PFNGLNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glNamedProgramStringEXT"));
02056     glNamedRenderbufferStorage =
PFNGLNAMEDRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedRenderbufferStorage"));
02057     glNamedRenderbufferStorageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageEXT"));
02058     glNamedRenderbufferStorageMultisample =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisample"));
02059     glNamedRenderbufferStorageMultisampleCoverageEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleCoverageEXT"));
02060     glNamedRenderbufferStorageMultisampleEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleEXT"));
02061     glNamedStringARB = PFNGLNAMEDSTRINGARBPROC (glfwGetProcAddress ("glNamedStringARB"));
02062     glNormalFormatNV = PFNGLNORMALFORMATNVPROC (glfwGetProcAddress ("glNormalFormatNV"));
02063     glObjectLabel = PFNGLOBJECTLABELPROC (glfwGetProcAddress ("glObjectLabel"));
02064     glObjectPtrLabel = PFNGLOBJECTPTRLABELPROC (glfwGetProcAddress ("glObjectPtrLabel"));
02065     glPatchParameterfv = PFNGLPATCHPARAMETERFVPROC (glfwGetProcAddress ("glPatchParameterfv"));
02066     glPatchParameteri = PFNGLPATCHPARAMETERIPROC (glfwGetProcAddress ("glPatchParameteri"));
02067     glPathCommandsNV = PFNGLPATHCOMMANDSNVPROC (glfwGetProcAddress ("glPathCommandsNV"));
02068     glPathCoordsNV = PFNGLPATHCOORDSNVPROC (glfwGetProcAddress ("glPathCoordsNV"));
02069     glPathCoverDepthFuncNV =
PFNGLPATHCOVERDEPTHFUNCNVPROC (glfwGetProcAddress ("glPathCoverDepthFuncNV"));
02070     glPathDashArrayNV = PFNGLPATHDASHARRAYNVPROC (glfwGetProcAddress ("glPathDashArrayNV"));
02071     glPathGlyphIndexArrayNV =
PFNGLPATHGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathGlyphIndexArrayNV"));
02072     glPathGlyphIndexRangeNV =
PFNGLPATHGLYPHINDEXRANGENVPROC (glfwGetProcAddress ("glPathGlyphIndexRangeNV"));

```

```
02073     glPathGlyphRangeNV = PFNGLPATHGLYPHRANGENVPROC (glfwGetProcAddress ("glPathGlyphRangeNV"));
02074     glPathGlyphsNV = PFNGLPATHGLYPHSNVPROC (glfwGetProcAddress ("glPathGlyphsNV"));
02075     glPathMemoryGlyphIndexArrayNV =
PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathMemoryGlyphIndexArrayNV"));
02076     glPathParameterfNV = PFNGLPATHPARAMETERFNVPROC (glfwGetProcAddress ("glPathParameterfNV"));
02077     glPathParameterfVNV = PFNGLPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glPathParameterfVNV"));
02078     glPathParameteriNV = PFNGLPATHPARAMETERINVPROC (glfwGetProcAddress ("glPathParameteriNV"));
02079     glPathParameteriVNV = PFNGLPATHPARAMETERIVNVPROC (glfwGetProcAddress ("glPathParameteriVNV"));
02080     glPathStencilDepthOffsetNV =
PFNGLPATHSTENCILDEPTHOFFSETNVPROC (glfwGetProcAddress ("glPathStencilDepthOffsetNV"));
02081     glPathStencilFuncNV = PFNGLPATHSTENCILFUNCNVPROC (glfwGetProcAddress ("glPathStencilFuncNV"));
02082     glPathStringNV = PFNGLPATHSTRINGNVPROC (glfwGetProcAddress ("glPathStringNV"));
02083     glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress ("glPathSubCommandsNV"));
02084     glPathSubCoordsNV = PFNGLPATHSUBCOORDSNVPROC (glfwGetProcAddress ("glPathSubCoordsNV"));
02085     glPauseTransformFeedback =
PFNGLPAUSETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glPauseTransformFeedback"));
02086     glPixelStoref = PFNGLPIXELSTOREFPROC (glfwGetProcAddress ("glPixelStoref"));
02087     glPixelStorei = PFNGLPIXELSTOREIPROC (glfwGetProcAddress ("glPixelStorei"));
02088     glPointAlongPathNV = PFNGLPOINTALONGPATHNVPROC (glfwGetProcAddress ("glPointAlongPathNV"));
02089     glPointParameterf = PFNGLPOINTPARAMETERFPROC (glfwGetProcAddress ("glPointParameterf"));
02090     glPointParameterfV = PFNGLPOINTPARAMETERFVPROC (glfwGetProcAddress ("glPointParameterfV"));
02091     glPointParameteri = PFNGLPOINTPARAMETERIPROC (glfwGetProcAddress ("glPointParameteri"));
02092     glPointParameteriV = PFNGLPOINTPARAMETERIVPROC (glfwGetProcAddress ("glPointParameteriV"));
02093     glPointSize = PFNGLPOINTSIZEPROC (glfwGetProcAddress ("glPointSize"));
02094     glPolygonMode = PFNGLPOLYGONMODEPROC (glfwGetProcAddress ("glPolygonMode"));
02095     glPolygonOffset = PFNGLPOLYGONOFFSETPROC (glfwGetProcAddress ("glPolygonOffset"));
02096     glPolygonOffsetClampEXT =
PFNGLPOLYGONOFFSETCLAMPEXTPROC (glfwGetProcAddress ("glPolygonOffsetClampEXT"));
02097     glPopDebugGroup = PFNGLPOPDEBUGGROUPPROC (glfwGetProcAddress ("glPopDebugGroup"));
02098     glPopGroupMarkerEXT = PFNGLPOPGROUPMARKEREXTPROC (glfwGetProcAddress ("glPopGroupMarkerEXT"));
02099     glPrimitiveBoundingBoxARB =
PFNGLPRIMITIVEBOUNDINGBOXARBPROC (glfwGetProcAddress ("glPrimitiveBoundingBoxARB"));
02100     glPrimitiveRestartIndex =
PFNGLPRIMITIVERESTARTINDEXPROC (glfwGetProcAddress ("glPrimitiveRestartIndex"));
02101     glProgramBinary = PFNGLPROGRAMBINARYPROC (glfwGetProcAddress ("glProgramBinary"));
02102     glProgramParameteri = PFNGLPROGRAMPARAMETERIPROC (glfwGetProcAddress ("glProgramParameteri"));
02103     glProgramParameteriARB =
PFNGLPROGRAMPARAMETERIARBPROC (glfwGetProcAddress ("glProgramParameteriARB"));
02104     glProgramPathFragmentInputGenNV =
PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC (glfwGetProcAddress ("glProgramPathFragmentInputGenNV"));
02105     glProgramUniformId = PFNGLPROGRAMUNIFORMIDPROC (glfwGetProcAddress ("glProgramUniformId"));
02106     glProgramUniformIdEXT = PFNGLPROGRAMUNIFORMIDEXTPROC (glfwGetProcAddress ("glProgramUniformIdEXT"));
02107     glProgramUniformIdv = PFNGLPROGRAMUNIFORMIDVPROC (glfwGetProcAddress ("glProgramUniformIdv"));
02108     glProgramUniformIdvEXT =
PFNGLPROGRAMUNIFORMIDVEXTPROC (glfwGetProcAddress ("glProgramUniformIdvEXT"));
02109     glProgramUniformI = PFNGLPROGRAMUNIFORMIIPROC (glfwGetProcAddress ("glProgramUniformI"));
02110     glProgramUniformIEXT = PFNGLPROGRAMUNIFORMIEXTPROC (glfwGetProcAddress ("glProgramUniformIEXT"));
02111     glProgramUniformIv = PFNGLPROGRAMUNIFORMIVPROC (glfwGetProcAddress ("glProgramUniformIv"));
02112     glProgramUniformIvEXT =
PFNGLPROGRAMUNIFORMIVEXTPROC (glfwGetProcAddress ("glProgramUniformIvEXT"));
02113     glProgramUniformli = PFNGLPROGRAMUNIFORMLIIPROC (glfwGetProcAddress ("glProgramUniformli"));
02114     glProgramUniformli64ARB =
PFNGLPROGRAMUNIFORMLI64ARBPROC (glfwGetProcAddress ("glProgramUniformli64ARB"));
02115     glProgramUniformli64NV =
PFNGLPROGRAMUNIFORMLI64NVPROC (glfwGetProcAddress ("glProgramUniformli64NV"));
02116     glProgramUniformli64vARB =
PFNGLPROGRAMUNIFORMLI64VARBPROC (glfwGetProcAddress ("glProgramUniformli64vARB"));
02117     glProgramUniformli64vNV =
PFNGLPROGRAMUNIFORMLI64VNVPROC (glfwGetProcAddress ("glProgramUniformli64vNV"));
02118     glProgramUniformliEXT = PFNGLPROGRAMUNIFORMLIEXTPROC (glfwGetProcAddress ("glProgramUniformliEXT"));
02119     glProgramUniformliv = PFNGLPROGRAMUNIFORMLIVPROC (glfwGetProcAddress ("glProgramUniformliv"));
02120     glProgramUniformlivEXT =
PFNGLPROGRAMUNIFORMLIVEXTPROC (glfwGetProcAddress ("glProgramUniformlivEXT"));
02121     glProgramUniformlui = PFNGLPROGRAMUNIFORMLUIPROC (glfwGetProcAddress ("glProgramUniformlui"));
02122     glProgramUniformlui64ARB =
PFNGLPROGRAMUNIFORMLUI64ARBPROC (glfwGetProcAddress ("glProgramUniformlui64ARB"));
02123     glProgramUniformlui64NV =
PFNGLPROGRAMUNIFORMLUI64NVPROC (glfwGetProcAddress ("glProgramUniformlui64NV"));
02124     glProgramUniformlui64vARB =
PFNGLPROGRAMUNIFORMLUI64VARBPROC (glfwGetProcAddress ("glProgramUniformlui64vARB"));
02125     glProgramUniformlui64vNV =
PFNGLPROGRAMUNIFORMLUI64VNVPROC (glfwGetProcAddress ("glProgramUniformlui64vNV"));
02126     glProgramUniformluiEXT =
PFNGLPROGRAMUNIFORMLUIEXTPROC (glfwGetProcAddress ("glProgramUniformluiEXT"));
02127     glProgramUniformluiv = PFNGLPROGRAMUNIFORMLUIVPROC (glfwGetProcAddress ("glProgramUniformluiv"));
02128     glProgramUniformluivEXT =
PFNGLPROGRAMUNIFORMLUIVEXTPROC (glfwGetProcAddress ("glProgramUniformluivEXT"));
02129     glProgramUniform2d = PFNGLPROGRAMUNIFORM2DPROC (glfwGetProcAddress ("glProgramUniform2d"));
02130     glProgramUniform2dEXT = PFNGLPROGRAMUNIFORM2DEXTPROC (glfwGetProcAddress ("glProgramUniform2dEXT"));
02131     glProgramUniform2dv = PFNGLPROGRAMUNIFORM2DVPROC (glfwGetProcAddress ("glProgramUniform2dv"));
02132     glProgramUniform2dvEXT =
PFNGLPROGRAMUNIFORM2DVEXTPROC (glfwGetProcAddress ("glProgramUniform2dvEXT"));
02133     glProgramUniform2f = PFNGLPROGRAMUNIFORM2FPROC (glfwGetProcAddress ("glProgramUniform2f"));
02134     glProgramUniform2fEXT = PFNGLPROGRAMUNIFORM2FEXTPROC (glfwGetProcAddress ("glProgramUniform2fEXT"));
02135     glProgramUniform2fv = PFNGLPROGRAMUNIFORM2FVPROC (glfwGetProcAddress ("glProgramUniform2fv"));
02136     glProgramUniform2fvEXT =
PFNGLPROGRAMUNIFORM2FVEXTPROC (glfwGetProcAddress ("glProgramUniform2fvEXT"));
```

```

02137     glProgramUniform2i = PFNGLPROGRAMUNIFORM2IPROC (glfwGetProcAddress ("glProgramUniform2i"));
02138     glProgramUniform2i64ARB =
PFNGLPROGRAMUNIFORM2I64ARBPROC (glfwGetProcAddress ("glProgramUniform2i64ARB"));
02139     glProgramUniform2i64NV =
PFNGLPROGRAMUNIFORM2I64NVPROC (glfwGetProcAddress ("glProgramUniform2i64NV"));
02140     glProgramUniform2i64vARB =
PFNGLPROGRAMUNIFORM2I64VARBPROC (glfwGetProcAddress ("glProgramUniform2i64vARB"));
02141     glProgramUniform2i64vNV =
PFNGLPROGRAMUNIFORM2I64VNVPROC (glfwGetProcAddress ("glProgramUniform2i64vNV"));
02142     glProgramUniform2iEXT = PFNGLPROGRAMUNIFORM2IEXTPROC (glfwGetProcAddress ("glProgramUniform2iEXT"));
02143     glProgramUniform2iv = PFNGLPROGRAMUNIFORM2IVPROC (glfwGetProcAddress ("glProgramUniform2iv"));
02144     glProgramUniform2ivEXT =
PFNGLPROGRAMUNIFORM2IVEXTPROC (glfwGetProcAddress ("glProgramUniform2ivEXT"));
02145     glProgramUniform2ui = PFNGLPROGRAMUNIFORM2UIPROC (glfwGetProcAddress ("glProgramUniform2ui"));
02146     glProgramUniform2ui64ARB =
PFNGLPROGRAMUNIFORM2UI64ARBPROC (glfwGetProcAddress ("glProgramUniform2ui64ARB"));
02147     glProgramUniform2ui64NV =
PFNGLPROGRAMUNIFORM2UI64NVPROC (glfwGetProcAddress ("glProgramUniform2ui64NV"));
02148     glProgramUniform2ui64vARB =
PFNGLPROGRAMUNIFORM2UI64VARBPROC (glfwGetProcAddress ("glProgramUniform2ui64vARB"));
02149     glProgramUniform2ui64vNV =
PFNGLPROGRAMUNIFORM2UI64VNVPROC (glfwGetProcAddress ("glProgramUniform2ui64vNV"));
02150     glProgramUniform2uiEXT =
PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02151     glProgramUniform2uiv = PFNGLPROGRAMUNIFORM2UIVPROC (glfwGetProcAddress ("glProgramUniform2uiv"));
02152     glProgramUniform2uivEXT =
PFNGLPROGRAMUNIFORM2UIVEXTPROC (glfwGetProcAddress ("glProgramUniform2uivEXT"));
02153     glProgramUniform3d = PFNGLPROGRAMUNIFORM3DPROC (glfwGetProcAddress ("glProgramUniform3d"));
02154     glProgramUniform3dEXT = PFNGLPROGRAMUNIFORM3DEXTPROC (glfwGetProcAddress ("glProgramUniform3dEXT"));
02155     glProgramUniform3dv = PFNGLPROGRAMUNIFORM3DVPROC (glfwGetProcAddress ("glProgramUniform3dv"));
02156     glProgramUniform3dvEXT =
PFNGLPROGRAMUNIFORM3DVEXTPROC (glfwGetProcAddress ("glProgramUniform3dvEXT"));
02157     glProgramUniform3f = PFNGLPROGRAMUNIFORM3FPROC (glfwGetProcAddress ("glProgramUniform3f"));
02158     glProgramUniform3fEXT = PFNGLPROGRAMUNIFORM3FEXTPROC (glfwGetProcAddress ("glProgramUniform3fEXT"));
02159     glProgramUniform3fv = PFNGLPROGRAMUNIFORM3FVPROC (glfwGetProcAddress ("glProgramUniform3fv"));
02160     glProgramUniform3fvEXT =
PFNGLPROGRAMUNIFORM3FVEXTPROC (glfwGetProcAddress ("glProgramUniform3fvEXT"));
02161     glProgramUniform3i = PFNGLPROGRAMUNIFORM3IPROC (glfwGetProcAddress ("glProgramUniform3i"));
02162     glProgramUniform3i64ARB =
PFNGLPROGRAMUNIFORM3I64ARBPROC (glfwGetProcAddress ("glProgramUniform3i64ARB"));
02163     glProgramUniform3i64NV =
PFNGLPROGRAMUNIFORM3I64NVPROC (glfwGetProcAddress ("glProgramUniform3i64NV"));
02164     glProgramUniform3i64vARB =
PFNGLPROGRAMUNIFORM3I64VARBPROC (glfwGetProcAddress ("glProgramUniform3i64vARB"));
02165     glProgramUniform3i64vNV =
PFNGLPROGRAMUNIFORM3I64VNVPROC (glfwGetProcAddress ("glProgramUniform3i64vNV"));
02166     glProgramUniform3iEXT = PFNGLPROGRAMUNIFORM3IEXTPROC (glfwGetProcAddress ("glProgramUniform3iEXT"));
02167     glProgramUniform3iv = PFNGLPROGRAMUNIFORM3IVPROC (glfwGetProcAddress ("glProgramUniform3iv"));
02168     glProgramUniform3ivEXT =
PFNGLPROGRAMUNIFORM3IVEXTPROC (glfwGetProcAddress ("glProgramUniform3ivEXT"));
02169     glProgramUniform3ui = PFNGLPROGRAMUNIFORM3UIPROC (glfwGetProcAddress ("glProgramUniform3ui"));
02170     glProgramUniform3ui64ARB =
PFNGLPROGRAMUNIFORM3UI64ARBPROC (glfwGetProcAddress ("glProgramUniform3ui64ARB"));
02171     glProgramUniform3ui64NV =
PFNGLPROGRAMUNIFORM3UI64NVPROC (glfwGetProcAddress ("glProgramUniform3ui64NV"));
02172     glProgramUniform3ui64vARB =
PFNGLPROGRAMUNIFORM3UI64VARBPROC (glfwGetProcAddress ("glProgramUniform3ui64vARB"));
02173     glProgramUniform3ui64vNV =
PFNGLPROGRAMUNIFORM3UI64VNVPROC (glfwGetProcAddress ("glProgramUniform3ui64vNV"));
02174     glProgramUniform3uiEXT =
PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02175     glProgramUniform3uiv = PFNGLPROGRAMUNIFORM3UIVPROC (glfwGetProcAddress ("glProgramUniform3uiv"));
02176     glProgramUniform3uivEXT =
PFNGLPROGRAMUNIFORM3UIVEXTPROC (glfwGetProcAddress ("glProgramUniform3uivEXT"));
02177     glProgramUniform4d = PFNGLPROGRAMUNIFORM4DPROC (glfwGetProcAddress ("glProgramUniform4d"));
02178     glProgramUniform4dEXT = PFNGLPROGRAMUNIFORM4DEXTPROC (glfwGetProcAddress ("glProgramUniform4dEXT"));
02179     glProgramUniform4dv = PFNGLPROGRAMUNIFORM4DVPROC (glfwGetProcAddress ("glProgramUniform4dv"));
02180     glProgramUniform4dvEXT =
PFNGLPROGRAMUNIFORM4DVEXTPROC (glfwGetProcAddress ("glProgramUniform4dvEXT"));
02181     glProgramUniform4f = PFNGLPROGRAMUNIFORM4FPROC (glfwGetProcAddress ("glProgramUniform4f"));
02182     glProgramUniform4fEXT = PFNGLPROGRAMUNIFORM4FEXTPROC (glfwGetProcAddress ("glProgramUniform4fEXT"));
02183     glProgramUniform4fv = PFNGLPROGRAMUNIFORM4FVPROC (glfwGetProcAddress ("glProgramUniform4fv"));
02184     glProgramUniform4fvEXT =
PFNGLPROGRAMUNIFORM4FVEXTPROC (glfwGetProcAddress ("glProgramUniform4fvEXT"));
02185     glProgramUniform4i = PFNGLPROGRAMUNIFORM4IPROC (glfwGetProcAddress ("glProgramUniform4i"));
02186     glProgramUniform4i64ARB =
PFNGLPROGRAMUNIFORM4I64ARBPROC (glfwGetProcAddress ("glProgramUniform4i64ARB"));
02187     glProgramUniform4i64NV =
PFNGLPROGRAMUNIFORM4I64NVPROC (glfwGetProcAddress ("glProgramUniform4i64NV"));
02188     glProgramUniform4i64vARB =
PFNGLPROGRAMUNIFORM4I64VARBPROC (glfwGetProcAddress ("glProgramUniform4i64vARB"));
02189     glProgramUniform4i64vNV =
PFNGLPROGRAMUNIFORM4I64VNVPROC (glfwGetProcAddress ("glProgramUniform4i64vNV"));
02190     glProgramUniform4iEXT = PFNGLPROGRAMUNIFORM4IEXTPROC (glfwGetProcAddress ("glProgramUniform4iEXT"));
02191     glProgramUniform4iv = PFNGLPROGRAMUNIFORM4IVPROC (glfwGetProcAddress ("glProgramUniform4iv"));
02192     glProgramUniform4ivEXT =
PFNGLPROGRAMUNIFORM4IVEXTPROC (glfwGetProcAddress ("glProgramUniform4ivEXT"));

```

```
02193     glProgramUniform4ui = PFNGLPROGRAMUNIFORM4UIPROC (glfwGetProcAddress("glProgramUniform4ui"));
02194     glProgramUniform4i64ARB =
PFNGLPROGRAMUNIFORM4UI64ARBPROC (glfwGetProcAddress("glProgramUniform4i64ARB"));
02195     glProgramUniform4i64NV =
PFNGLPROGRAMUNIFORM4UI64NVPROC (glfwGetProcAddress("glProgramUniform4i64NV"));
02196     glProgramUniform4i64vARB =
PFNGLPROGRAMUNIFORM4UI64VARBPROC (glfwGetProcAddress("glProgramUniform4i64vARB"));
02197     glProgramUniform4i64vNV =
PFNGLPROGRAMUNIFORM4UI64VNVPROC (glfwGetProcAddress("glProgramUniform4i64vNV"));
02198     glProgramUniform4iEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress("glProgramUniform4iEXT"));
02199     glProgramUniform4uiv = PFNGLPROGRAMUNIFORM4UIVPROC (glfwGetProcAddress("glProgramUniform4uiv"));
02200     glProgramUniform4uivEXT =
PFNGLPROGRAMUNIFORM4UIVEXTPROC (glfwGetProcAddress("glProgramUniform4uivEXT"));
02201     glProgramUniformHandleui64ARB =
PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glProgramUniformHandleui64ARB"));
02202     glProgramUniformHandleui64NV =
PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glProgramUniformHandleui64NV"));
02203     glProgramUniformHandleui64vARB =
PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glProgramUniformHandleui64vARB"));
02204     glProgramUniformHandleui64vNV =
PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glProgramUniformHandleui64vNV"));
02205     glProgramUniformMatrix2dv =
PFNGLPROGRAMUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glProgramUniformMatrix2dv"));
02206     glProgramUniformMatrix2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2dvEXT"));
02207     glProgramUniformMatrix2fv =
PFNGLPROGRAMUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glProgramUniformMatrix2fv"));
02208     glProgramUniformMatrix2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2fvEXT"));
02209     glProgramUniformMatrix2x3dv =
PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glProgramUniformMatrix2x3dv"));
02210     glProgramUniformMatrix2x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2x3dvEXT"));
02211     glProgramUniformMatrix2x3fv =
PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glProgramUniformMatrix2x3fv"));
02212     glProgramUniformMatrix2x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2x3fvEXT"));
02213     glProgramUniformMatrix2x4dv =
PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glProgramUniformMatrix2x4dv"));
02214     glProgramUniformMatrix2x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2x4dvEXT"));
02215     glProgramUniformMatrix2x4fv =
PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glProgramUniformMatrix2x4fv"));
02216     glProgramUniformMatrix2x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix2x4fvEXT"));
02217     glProgramUniformMatrix3dv =
PFNGLPROGRAMUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glProgramUniformMatrix3dv"));
02218     glProgramUniformMatrix3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3dvEXT"));
02219     glProgramUniformMatrix3fv =
PFNGLPROGRAMUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glProgramUniformMatrix3fv"));
02220     glProgramUniformMatrix3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3fvEXT"));
02221     glProgramUniformMatrix3x2dv =
PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glProgramUniformMatrix3x2dv"));
02222     glProgramUniformMatrix3x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3x2dvEXT"));
02223     glProgramUniformMatrix3x2fv =
PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glProgramUniformMatrix3x2fv"));
02224     glProgramUniformMatrix3x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3x2fvEXT"));
02225     glProgramUniformMatrix3x4dv =
PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glProgramUniformMatrix3x4dv"));
02226     glProgramUniformMatrix3x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3x4dvEXT"));
02227     glProgramUniformMatrix3x4fv =
PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glProgramUniformMatrix3x4fv"));
02228     glProgramUniformMatrix3x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix3x4fvEXT"));
02229     glProgramUniformMatrix4dv =
PFNGLPROGRAMUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glProgramUniformMatrix4dv"));
02230     glProgramUniformMatrix4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix4dvEXT"));
02231     glProgramUniformMatrix4fv =
PFNGLPROGRAMUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glProgramUniformMatrix4fv"));
02232     glProgramUniformMatrix4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix4fvEXT"));
02233     glProgramUniformMatrix4x2dv =
PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress("glProgramUniformMatrix4x2dv"));
02234     glProgramUniformMatrix4x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix4x2dvEXT"));
02235     glProgramUniformMatrix4x2fv =
PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress("glProgramUniformMatrix4x2fv"));
02236     glProgramUniformMatrix4x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC (glfwGetProcAddress("glProgramUniformMatrix4x2fvEXT"));
02237     glProgramUniformMatrix4x3dv =
```



```

PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dv"));
02238 glProgramUniformMatrix4x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dvEXT"));
02239 glProgramUniformMatrix4x3fv =
PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fv"));
02240 glProgramUniformMatrix4x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fvEXT"));
02241 glProgramUniformi64NV =
PFNGLPROGRAMUNIFORMUI64NVPROC (glfwGetProcAddress ("glProgramUniformui64NV"));
02242 glProgramUniformi64vNV =
PFNGLPROGRAMUNIFORMUI64VNVPROC (glfwGetProcAddress ("glProgramUniformui64vNV"));
02243 glProvokingVertex = PFNGLPROVOKINGVERTEXPROC (glfwGetProcAddress ("glProvokingVertex"));
02244 glPushClientAttribDefaultEXT =
PFNGLPUSHCLIENTATTRIBDEFAULTPROC (glfwGetProcAddress ("glPushClientAttribDefaultEXT"));
02245 glPushDebugGroup = PFNGLPUSHDEBUGGROUPPROC (glfwGetProcAddress ("glPushDebugGroup"));
02246 glPushGroupMarkerEXT = PFNGLPUSHGROUPMARKEREXTPROC (glfwGetProcAddress ("glPushGroupMarkerEXT"));
02247 glQueryCounter = PFNGLQUERYCOUNTERPROC (glfwGetProcAddress ("glQueryCounter"));
02248 glRasterSamplesEXT = PFNGLRASTERSAMPLESEXTPROC (glfwGetProcAddress ("glRasterSamplesEXT"));
02249 glReadBuffer = PFNGLREADBUFFERPROC (glfwGetProcAddress ("glReadBuffer"));
02250 glReadPixels = PFNGLREADPIXELSPROC (glfwGetProcAddress ("glReadPixels"));
02251 glReadnPixels = PFNGLREADNPIXELSPROC (glfwGetProcAddress ("glReadnPixels"));
02252 glReadnPixelsARB = PFNGLREADNPIXELSARBPROC (glfwGetProcAddress ("glReadnPixelsARB"));
02253 glReleaseShaderCompiler =
PFNGLRELEASESHADERCOMPILERPROC (glfwGetProcAddress ("glReleaseShaderCompiler"));
02254 glRenderbufferStorage = PFNGLRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glRenderbufferStorage"));
02255 glRenderbufferStorageMultisample =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress ("glRenderbufferStorageMultisample"));
02256 glRenderbufferStorageMultisampleCoverageNV =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEVPROC (glfwGetProcAddress ("glRenderbufferStorageMultisampleCoverageNV"));
02257 glResolveDepthValuesNV =
PFNGLRESOLVEDEPTHVALUESVPROC (glfwGetProcAddress ("glResolveDepthValuesNV"));
02258 glResumeTransformFeedback =
PFNGLRESUMETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glResumeTransformFeedback"));
02259 glSampleCoverage = PFNGLSAMPLECOVERAGEPROC (glfwGetProcAddress ("glSampleCoverage"));
02260 glSampleMaski = PFNGLSAMPLEMASKIPROC (glfwGetProcAddress ("glSampleMaski"));
02261 glSamplerParameterIiv = PFNGLSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glSamplerParameterIiv"));
02262 glSamplerParameterIuiv =
PFNGLSAMPLERPARAMETERUIVPROC (glfwGetProcAddress ("glSamplerParameterIuiv"));
02263 glSamplerParameterf = PFNGLSAMPLERPARAMETERFPROC (glfwGetProcAddress ("glSamplerParameterf"));
02264 glSamplerParameterfv = PFNGLSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glSamplerParameterfv"));
02265 glSamplerParameteri = PFNGLSAMPLERPARAMETERIPROC (glfwGetProcAddress ("glSamplerParameteri"));
02266 glSamplerParameteriv = PFNGLSAMPLERPARAMETERIVPROC (glfwGetProcAddress ("glSamplerParameteriv"));
02267 glScissor = PFNGLSCISSORPROC (glfwGetProcAddress ("glScissor"));
02268 glScissorArrayv = PFNGLSCISSORARRAYVPROC (glfwGetProcAddress ("glScissorArrayv"));
02269 glScissorIndexed = PFNGLSCISSORINDEXEDPROC (glfwGetProcAddress ("glScissorIndexed"));
02270 glScissorIndexedv = PFNGLSCISSORINDEXEDVPROC (glfwGetProcAddress ("glScissorIndexedv"));
02271 glSecondaryColorFormatNV =
PFNGLSECONDARYCOLORFORMATNVPROC (glfwGetProcAddress ("glSecondaryColorFormatNV"));
02272 glSelectPerfMonitorCountersAMD =
PFNGLSELECTPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glSelectPerfMonitorCountersAMD"));
02273 glShaderBinary = PFNGLSHADERBINARYPROC (glfwGetProcAddress ("glShaderBinary"));
02274 glShaderSource = PFNGLSHADERSOURCEPROC (glfwGetProcAddress ("glShaderSource"));
02275 glShaderStorageBlockBinding =
PFNGLSHADERSTORAGEBLOCKBINDINGPROC (glfwGetProcAddress ("glShaderStorageBlockBinding"));
02276 glSignalVkFenceNV = PFNGLSIGNALVKFENCEVPROC (glfwGetProcAddress ("glSignalVkFenceNV"));
02277 glSignalVkSemaphoreNV = PFNGLSIGNALVKSEMAPHORENVPROC (glfwGetProcAddress ("glSignalVkSemaphoreNV"));
02278 glSpecializeShaderARB = PFNGLSPECIALIZEShaderARBPROC (glfwGetProcAddress ("glSpecializeShaderARB"));
02279 glStateCaptureNV = PFNGLSTATECAPTURENVPROC (glfwGetProcAddress ("glStateCaptureNV"));
02280 glStencilFillPathInstancedNV =
PFNGLSTENCILFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilFillPathInstancedNV"));
02281 glStencilFillPathNV = PFNGLSTENCILFILLPATHNVPROC (glfwGetProcAddress ("glStencilFillPathNV"));
02282 glStencilFunc = PFNGLSTENCILFUNCPROC (glfwGetProcAddress ("glStencilFunc"));
02283 glStencilFuncSeparate = PFNGLSTENCILFUNCSEPARATEPROC (glfwGetProcAddress ("glStencilFuncSeparate"));
02284 glStencilMask = PFNGLSTENCILMASKPROC (glfwGetProcAddress ("glStencilMask"));
02285 glStencilMaskSeparate = PFNGLSTENCILMASKSEPARATEPROC (glfwGetProcAddress ("glStencilMaskSeparate"));
02286 glStencilOp = PFNGLSTENCILOPPROC (glfwGetProcAddress ("glStencilOp"));
02287 glStencilOpSeparate = PFNGLSTENCILOPSEPARATEPROC (glfwGetProcAddress ("glStencilOpSeparate"));
02288 glStencilStrokePathInstancedNV =
PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilStrokePathInstancedNV"));
02289 glStencilStrokePathNV = PFNGLSTENCILSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilStrokePathNV"));
02290 glStencilThenCoverFillPathInstancedNV =
PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathInstancedNV"));
02291 glStencilThenCoverFillPathNV =
PFNGLSTENCILTHENCOVERFILLPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathNV"));
02292 glStencilThenCoverStrokePathInstancedNV =
PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathInstancedNV"));
02293 glStencilThenCoverStrokePathNV =
PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathNV"));
02294 glSubpixelPrecisionBiasNV =
PFNGLSUBPIXELPRECISIONBIASNVPROC (glfwGetProcAddress ("glSubpixelPrecisionBiasNV"));
02295 glTexBuffer = PFNGLTEXBUFFERPROC (glfwGetProcAddress ("glTexBuffer"));
02296 glTexBufferARB = PFNGLTEXBUFFERARBPROC (glfwGetProcAddress ("glTexBufferARB"));
02297 glTexBufferRange = PFNGLTEXBUFFERRANGEPROC (glfwGetProcAddress ("glTexBufferRange"));
02298 glTexCoordFormatNV = PFNGLTEXCOORDFORMATNVPROC (glfwGetProcAddress ("glTexCoordFormatNV"));
02299 glTexImage1D = PFNGLTEXIMAGE1DPROC (glfwGetProcAddress ("glTexImage1D"));
02300 glTexImage2D = PFNGLTEXIMAGE2DPROC (glfwGetProcAddress ("glTexImage2D"));
02301 glTexImage2DMultisample =

```



```
PFNGLTEXTIMAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage2DMultisample"));
02302   glTexImage3D = PFNGLTEXTIMAGE3DPROC (glfwGetProcAddress("glTexImage3D"));
02303   glTexImage3DMultisample =
PFNGLTEXTIMAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage3DMultisample"));
02304   glTexPageCommitmentARB =
PFNGLTEXPAGECOMMITMENTARBPROC (glfwGetProcAddress("glTexPageCommitmentARB"));
02305   glTexParameterIiv = PFNGLTEXPARAMETERIIVPROC (glfwGetProcAddress("glTexParameterIiv"));
02306   glTexParameterIuiv = PFNGLTEXPARAMETERIUIVPROC (glfwGetProcAddress("glTexParameterIuiv"));
02307   glTexParameterf = PFNGLTEXPARAMETERFPROC (glfwGetProcAddress("glTexParameterf"));
02308   glTexParameterfv = PFNGLTEXPARAMETERFVPROC (glfwGetProcAddress("glTexParameterfv"));
02309   glTexParameteri = PFNGLTEXPARAMETERIPROC (glfwGetProcAddress("glTexParameteri"));
02310   glTexParameteriv = PFNGLTEXPARAMETERIVPROC (glfwGetProcAddress("glTexParameteriv"));
02311   glTexStorage1D = PFNGLTEXSTORAGE1DPROC (glfwGetProcAddress("glTexStorage1D"));
02312   glTexStorage2D = PFNGLTEXSTORAGE2DPROC (glfwGetProcAddress("glTexStorage2D"));
02313   glTexStorage2DMultisample =
PFNGLTEXSTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage2DMultisample"));
02314   glTexStorage3D = PFNGLTEXSTORAGE3DPROC (glfwGetProcAddress("glTexStorage3D"));
02315   glTexStorage3DMultisample =
PFNGLTEXSTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage3DMultisample"));
02316   glTexSubImage1D = PFNGLTEXSUBIMAGE1DPROC (glfwGetProcAddress("glTexSubImage1D"));
02317   glTexSubImage2D = PFNGLTEXSUBIMAGE2DPROC (glfwGetProcAddress("glTexSubImage2D"));
02318   glTexSubImage3D = PFNGLTEXSUBIMAGE3DPROC (glfwGetProcAddress("glTexSubImage3D"));
02319   glTextureBarrier = PFNGLTEXTUREBARRIERPROC (glfwGetProcAddress("glTextureBarrier"));
02320   glTextureBarrierNV = PFNGLTEXTUREBARRIERNVPROC (glfwGetProcAddress("glTextureBarrierNV"));
02321   glTextureBuffer = PFNGLTEXTUREBUFFERPROC (glfwGetProcAddress("glTextureBuffer"));
02322   glTextureBufferEXT = PFNGLTEXTUREBUFFEREXTPROC (glfwGetProcAddress("glTextureBufferEXT"));
02323   glTextureBufferRange = PFNGLTEXTUREBUFFERRANGEPROC (glfwGetProcAddress("glTextureBufferRange"));
02324   glTextureBufferRangeEXT =
PFNGLTEXTUREBUFFERRANGEEXTPROC (glfwGetProcAddress("glTextureBufferRangeEXT"));
02325   glTextureImage1DEXT = PFNGLTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glTextureImage1DEXT"));
02326   glTextureImage2DEXT = PFNGLTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glTextureImage2DEXT"));
02327   glTextureImage3DEXT = PFNGLTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glTextureImage3DEXT"));
02328   glTexturePageCommitmentEXT =
PFNGLTEXTUREPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glTexturePageCommitmentEXT"));
02329   glTextureParameterIiv = PFNGLTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glTextureParameterIiv"));
02330   glTextureParameterIivEXT =
PFNGLTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glTextureParameterIivEXT"));
02331   glTextureParameterIuiv =
PFNGLTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress("glTextureParameterIuiv"));
02332   glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));
02333   glTextureParameterf = PFNGLTEXTUREPARAMETERFPROC (glfwGetProcAddress("glTextureParameterf"));
02334   glTextureParameterfEXT =
PFNGLTEXTUREPARAMETERFEXTPROC (glfwGetProcAddress("glTextureParameterfEXT"));
02335   glTextureParameterfv = PFNGLTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glTextureParameterfv"));
02336   glTextureParameterfvEXT =
PFNGLTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glTextureParameterfvEXT"));
02337   glTextureParameteri = PFNGLTEXTUREPARAMETERIPROC (glfwGetProcAddress("glTextureParameteri"));
02338   glTextureParameteriEXT =
PFNGLTEXTUREPARAMETERIEXTPROC (glfwGetProcAddress("glTextureParameteriEXT"));
02339   glTextureParameteriv = PFNGLTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glTextureParameteriv"));
02340   glTextureParameterivEXT =
PFNGLTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glTextureParameterivEXT"));
02341   glTextureRenderbufferEXT =
PFNGLTEXTURERENDERBUFFEREXTPROC (glfwGetProcAddress("glTextureRenderbufferEXT"));
02342   glTextureStorage1D = PFNGLTEXTURESTORAGE1DPROC (glfwGetProcAddress("glTextureStorage1D"));
02343   glTextureStorage1DEXT = PFNGLTEXTURESTORAGE1DEXTPROC (glfwGetProcAddress("glTextureStorage1DEXT"));
02344   glTextureStorage2D = PFNGLTEXTURESTORAGE2DPROC (glfwGetProcAddress("glTextureStorage2D"));
02345   glTextureStorage2DEXT = PFNGLTEXTURESTORAGE2DEXTPROC (glfwGetProcAddress("glTextureStorage2DEXT"));
02346   glTextureStorage2DMultisample =
PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage2DMultisample"));
02347   glTextureStorage2DMultisampleEXT =
PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage2DMultisampleEXT"));
02348   glTextureStorage3D = PFNGLTEXTURESTORAGE3DPROC (glfwGetProcAddress("glTextureStorage3D"));
02349   glTextureStorage3DEXT = PFNGLTEXTURESTORAGE3DEXTPROC (glfwGetProcAddress("glTextureStorage3DEXT"));
02350   glTextureStorage3DMultisample =
PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage3DMultisample"));
02351   glTextureStorage3DMultisampleEXT =
PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage3DMultisampleEXT"));
02352   glTextureSubImage1D = PFNGLTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glTextureSubImage1D"));
02353   glTextureSubImage1DEXT =
PFNGLTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glTextureSubImage1DEXT"));
02354   glTextureSubImage2D = PFNGLTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glTextureSubImage2D"));
02355   glTextureSubImage2DEXT =
PFNGLTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress("glTextureSubImage2DEXT"));
02356   glTextureSubImage3D = PFNGLTEXTURESUBIMAGE3DPROC (glfwGetProcAddress("glTextureSubImage3D"));
02357   glTextureSubImage3DEXT =
PFNGLTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glTextureSubImage3DEXT"));
02358   glTextureView = PFNGLTEXTUREVIEWPROC (glfwGetProcAddress("glTextureView"));
02359   glTransformFeedbackBufferBase =
PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC (glfwGetProcAddress("glTransformFeedbackBufferBase"));
02360   glTransformFeedbackBufferRange =
PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC (glfwGetProcAddress("glTransformFeedbackBufferRange"));
02361   glTransformFeedbackVaryings =
PFNGLTRANSFORMFEEDBACKVARYINGSPROC (glfwGetProcAddress("glTransformFeedbackVaryings"));
02362   glTransformPathNV = PFNGLTRANSFORMPATHNVPROC (glfwGetProcAddress("glTransformPathNV"));
02363   glUniform1d = PFNGLUNIFORM1DPROC (glfwGetProcAddress("glUniform1d"));
```

```
02364 glUniformldv = PFNGLUNIFORM1DVPROC (glfwGetProcAddress("glUniformldv"));
02365 glUniformlf = PFNGLUNIFORM1FPROC (glfwGetProcAddress("glUniformlf"));
02366 glUniformlfv = PFNGLUNIFORM1FVPROC (glfwGetProcAddress("glUniformlfv"));
02367 glUniformli = PFNGLUNIFORM1IPROC (glfwGetProcAddress("glUniformli"));
02368 glUniformli64ARB = PFNGLUNIFORM1I64ARBPROC (glfwGetProcAddress("glUniformli64ARB"));
02369 glUniformli64NV = PFNGLUNIFORM1I64NVPROC (glfwGetProcAddress("glUniformli64NV"));
02370 glUniformli64vARB = PFNGLUNIFORM1I64VARBPROC (glfwGetProcAddress("glUniformli64vARB"));
02371 glUniformli64vNV = PFNGLUNIFORM1I64VNVPROC (glfwGetProcAddress("glUniformli64vNV"));
02372 glUniformliv = PFNGLUNIFORM1IVPROC (glfwGetProcAddress("glUniformliv"));
02373 glUniformlui = PFNGLUNIFORM1UIPROC (glfwGetProcAddress("glUniformlui"));
02374 glUniformlui64ARB = PFNGLUNIFORM1UI64ARBPROC (glfwGetProcAddress("glUniformlui64ARB"));
02375 glUniformlui64NV = PFNGLUNIFORM1UI64NVPROC (glfwGetProcAddress("glUniformlui64NV"));
02376 glUniformlui64vARB = PFNGLUNIFORM1UI64VARBPROC (glfwGetProcAddress("glUniformlui64vARB"));
02377 glUniformlui64vNV = PFNGLUNIFORM1UI64VNVPROC (glfwGetProcAddress("glUniformlui64vNV"));
02378 glUniformluiv = PFNGLUNIFORM1UIVPROC (glfwGetProcAddress("glUniformluiv"));
02379 glUniform2d = PFNGLUNIFORM2DPROC (glfwGetProcAddress("glUniform2d"));
02380 glUniform2dv = PFNGLUNIFORM2DVPROC (glfwGetProcAddress("glUniform2dv"));
02381 glUniform2f = PFNGLUNIFORM2FPROC (glfwGetProcAddress("glUniform2f"));
02382 glUniform2fv = PFNGLUNIFORM2FVPROC (glfwGetProcAddress("glUniform2fv"));
02383 glUniform2i = PFNGLUNIFORM2IPROC (glfwGetProcAddress("glUniform2i"));
02384 glUniform2i64ARB = PFNGLUNIFORM2I64ARBPROC (glfwGetProcAddress("glUniform2i64ARB"));
02385 glUniform2i64NV = PFNGLUNIFORM2I64NVPROC (glfwGetProcAddress("glUniform2i64NV"));
02386 glUniform2i64vARB = PFNGLUNIFORM2I64VARBPROC (glfwGetProcAddress("glUniform2i64vARB"));
02387 glUniform2i64vNV = PFNGLUNIFORM2I64VNVPROC (glfwGetProcAddress("glUniform2i64vNV"));
02388 glUniform2iv = PFNGLUNIFORM2IVPROC (glfwGetProcAddress("glUniform2iv"));
02389 glUniform2ui = PFNGLUNIFORM2UIPROC (glfwGetProcAddress("glUniform2ui"));
02390 glUniform2ui64ARB = PFNGLUNIFORM2UI64ARBPROC (glfwGetProcAddress("glUniform2ui64ARB"));
02391 glUniform2ui64NV = PFNGLUNIFORM2UI64NVPROC (glfwGetProcAddress("glUniform2ui64NV"));
02392 glUniform2ui64vARB = PFNGLUNIFORM2UI64VARBPROC (glfwGetProcAddress("glUniform2ui64vARB"));
02393 glUniform2ui64vNV = PFNGLUNIFORM2UI64VNVPROC (glfwGetProcAddress("glUniform2ui64vNV"));
02394 glUniform2uiv = PFNGLUNIFORM2UIVPROC (glfwGetProcAddress("glUniform2uiv"));
02395 glUniform3d = PFNGLUNIFORM3DPROC (glfwGetProcAddress("glUniform3d"));
02396 glUniform3dv = PFNGLUNIFORM3DVPROC (glfwGetProcAddress("glUniform3dv"));
02397 glUniform3f = PFNGLUNIFORM3FPROC (glfwGetProcAddress("glUniform3f"));
02398 glUniform3fv = PFNGLUNIFORM3FVPROC (glfwGetProcAddress("glUniform3fv"));
02399 glUniform3i = PFNGLUNIFORM3IPROC (glfwGetProcAddress("glUniform3i"));
02400 glUniform3i64ARB = PFNGLUNIFORM3I64ARBPROC (glfwGetProcAddress("glUniform3i64ARB"));
02401 glUniform3i64NV = PFNGLUNIFORM3I64NVPROC (glfwGetProcAddress("glUniform3i64NV"));
02402 glUniform3i64vARB = PFNGLUNIFORM3I64VARBPROC (glfwGetProcAddress("glUniform3i64vARB"));
02403 glUniform3i64vNV = PFNGLUNIFORM3I64VNVPROC (glfwGetProcAddress("glUniform3i64vNV"));
02404 glUniform3iv = PFNGLUNIFORM3IVPROC (glfwGetProcAddress("glUniform3iv"));
02405 glUniform3ui = PFNGLUNIFORM3UIPROC (glfwGetProcAddress("glUniform3ui"));
02406 glUniform3ui64ARB = PFNGLUNIFORM3UI64ARBPROC (glfwGetProcAddress("glUniform3ui64ARB"));
02407 glUniform3ui64NV = PFNGLUNIFORM3UI64NVPROC (glfwGetProcAddress("glUniform3ui64NV"));
02408 glUniform3ui64vARB = PFNGLUNIFORM3UI64VARBPROC (glfwGetProcAddress("glUniform3ui64vARB"));
02409 glUniform3ui64vNV = PFNGLUNIFORM3UI64VNVPROC (glfwGetProcAddress("glUniform3ui64vNV"));
02410 glUniform3uiv = PFNGLUNIFORM3UIVPROC (glfwGetProcAddress("glUniform3uiv"));
02411 glUniform4d = PFNGLUNIFORM4DPROC (glfwGetProcAddress("glUniform4d"));
02412 glUniform4dv = PFNGLUNIFORM4DVPROC (glfwGetProcAddress("glUniform4dv"));
02413 glUniform4f = PFNGLUNIFORM4FPROC (glfwGetProcAddress("glUniform4f"));
02414 glUniform4fv = PFNGLUNIFORM4FVPROC (glfwGetProcAddress("glUniform4fv"));
02415 glUniform4i = PFNGLUNIFORM4IPROC (glfwGetProcAddress("glUniform4i"));
02416 glUniform4i64ARB = PFNGLUNIFORM4I64ARBPROC (glfwGetProcAddress("glUniform4i64ARB"));
02417 glUniform4i64NV = PFNGLUNIFORM4I64NVPROC (glfwGetProcAddress("glUniform4i64NV"));
02418 glUniform4i64vARB = PFNGLUNIFORM4I64VARBPROC (glfwGetProcAddress("glUniform4i64vARB"));
02419 glUniform4i64vNV = PFNGLUNIFORM4I64VNVPROC (glfwGetProcAddress("glUniform4i64vNV"));
02420 glUniform4iv = PFNGLUNIFORM4IVPROC (glfwGetProcAddress("glUniform4iv"));
02421 glUniform4ui = PFNGLUNIFORM4UIPROC (glfwGetProcAddress("glUniform4ui"));
02422 glUniform4ui64ARB = PFNGLUNIFORM4UI64ARBPROC (glfwGetProcAddress("glUniform4ui64ARB"));
02423 glUniform4ui64NV = PFNGLUNIFORM4UI64NVPROC (glfwGetProcAddress("glUniform4ui64NV"));
02424 glUniform4ui64vARB = PFNGLUNIFORM4UI64VARBPROC (glfwGetProcAddress("glUniform4ui64vARB"));
02425 glUniform4ui64vNV = PFNGLUNIFORM4UI64VNVPROC (glfwGetProcAddress("glUniform4ui64vNV"));
02426 glUniform4uiv = PFNGLUNIFORM4UIVPROC (glfwGetProcAddress("glUniform4uiv"));
02427 glUniformBlockBinding = PFNGLUNIFORMBLOCKBINDINGPROC (glfwGetProcAddress("glUniformBlockBinding"));
02428 glUniformHandleui64ARB =
PFNGLUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glUniformHandleui64ARB"));
02429 glUniformHandleui64NV = PFNGLUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glUniformHandleui64NV"));
02430 glUniformHandleui64vARB =
PFNGLUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glUniformHandleui64vARB"));
02431 glUniformHandleui64vNV =
PFNGLUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glUniformHandleui64vNV"));
02432 glUniformMatrix2dv = PFNGLUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glUniformMatrix2dv"));
02433 glUniformMatrix2fv = PFNGLUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glUniformMatrix2fv"));
02434 glUniformMatrix2x3dv = PFNGLUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glUniformMatrix2x3dv"));
02435 glUniformMatrix2x3fv = PFNGLUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glUniformMatrix2x3fv"));
02436 glUniformMatrix2x4dv = PFNGLUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glUniformMatrix2x4dv"));
02437 glUniformMatrix2x4fv = PFNGLUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glUniformMatrix2x4fv"));
02438 glUniformMatrix3dv = PFNGLUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glUniformMatrix3dv"));
02439 glUniformMatrix3fv = PFNGLUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glUniformMatrix3fv"));
02440 glUniformMatrix3x2dv = PFNGLUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glUniformMatrix3x2dv"));
02441 glUniformMatrix3x2fv = PFNGLUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glUniformMatrix3x2fv"));
02442 glUniformMatrix3x4dv = PFNGLUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glUniformMatrix3x4dv"));
02443 glUniformMatrix3x4fv = PFNGLUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glUniformMatrix3x4fv"));
02444 glUniformMatrix4dv = PFNGLUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glUniformMatrix4dv"));
02445 glUniformMatrix4fv = PFNGLUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glUniformMatrix4fv"));
02446 glUniformMatrix4x2dv = PFNGLUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress("glUniformMatrix4x2dv"));
02447 glUniformMatrix4x2fv = PFNGLUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress("glUniformMatrix4x2fv"));
```

```
02448     glUniformMatrix4x3dv = PFNGLUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress("glUniformMatrix4x3dv"));
02449     glUniformMatrix4x3fv = PFNGLUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress("glUniformMatrix4x3fv"));
02450     glUniformSubroutinesuiv =
PFNGLUNIFORMSUBROUTINESUIVPROC (glfwGetProcAddress("glUniformSubroutinesuiv"));
02451     glUniformui64NV = PFNGLUNIFORMUI64NVPROC (glfwGetProcAddress("glUniformui64NV"));
02452     glUniformui64vNV = PFNGLUNIFORMUI64VNVPROC (glfwGetProcAddress("glUniformui64vNV"));
02453     glUnmapBuffer = PFNGLUNMAPBUFFERPROC (glfwGetProcAddress("glUnmapBuffer"));
02454     glUnmapNamedBuffer = PFNGLUNMAPNAMEDBUFFERPROC (glfwGetProcAddress("glUnmapNamedBuffer"));
02455     glUnmapNamedBufferEXT = PFNGLUNMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress("glUnmapNamedBufferEXT"));
02456     glUseProgram = PFNGLUSEPROGRAMPROC (glfwGetProcAddress("glUseProgram"));
02457     glUseProgramStages = PFNGLUSEPROGRAMSTAGESPROC (glfwGetProcAddress("glUseProgramStages"));
02458     glUseProgramEXT = PFNGLUSESHADERPROGRAMEXTPROC (glfwGetProcAddress("glUseProgramEXT"));
02459     glValidateProgram = PFNGLVALIDATEPROGRAMPROC (glfwGetProcAddress("glValidateProgram"));
02460     glValidateProgramPipeline =
PFNGLVALIDATEPROGRAMPIPELINEPROC (glfwGetProcAddress("glValidateProgramPipeline"));
02461     glVertexArrayAttribBinding =
PFNGLVERTEXARRAYATTRIBBINDINGPROC (glfwGetProcAddress("glVertexArrayAttribBinding"));
02462     glVertexArrayAttribFormat =
PFNGLVERTEXARRAYATTRIBFORMATPROC (glfwGetProcAddress("glVertexArrayAttribFormat"));
02463     glVertexArrayAttribIFormat =
PFNGLVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress("glVertexArrayAttribIFormat"));
02464     glVertexArrayAttribLFormat =
PFNGLVERTEXARRAYATTRIBLFORMATPROC (glfwGetProcAddress("glVertexArrayAttribLFormat"));
02465     glVertexArrayBindVertexBufferEXT =
PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC (glfwGetProcAddress("glVertexArrayBindVertexBufferEXT"));
02466     glVertexArrayBindingDivisor =
PFNGLVERTEXARRAYBINDINGDIVISORPROC (glfwGetProcAddress("glVertexArrayBindingDivisor"));
02467     glVertexArrayColorOffsetEXT =
PFNGLVERTEXARRAYCOLOROFFSETPROC (glfwGetProcAddress("glVertexArrayColorOffsetEXT"));
02468     glVertexArrayEdgeFlagOffsetEXT =
PFNGLVERTEXARRAYEDGEFLAGOFFSETPROC (glfwGetProcAddress("glVertexArrayEdgeFlagOffsetEXT"));
02469     glVertexArrayElementBuffer =
PFNGLVERTEXARRAYELEMENTBUFFERPROC (glfwGetProcAddress("glVertexArrayElementBuffer"));
02470     glVertexArrayFogCoordOffsetEXT =
PFNGLVERTEXARRAYFOGCOORDOFFSETPROC (glfwGetProcAddress("glVertexArrayFogCoordOffsetEXT"));
02471     glVertexArrayIndexOffsetEXT =
PFNGLVERTEXARRAYINDEXOFFSETPROC (glfwGetProcAddress("glVertexArrayIndexOffsetEXT"));
02472     glVertexArrayMultiTexCoordOffsetEXT =
PFNGLVERTEXARRAYMULTITEXCOORDOFFSETPROC (glfwGetProcAddress("glVertexArrayMultiTexCoordOffsetEXT"));
02473     glVertexArrayNormalOffsetEXT =
PFNGLVERTEXARRAYNORMALOFFSETPROC (glfwGetProcAddress("glVertexArrayNormalOffsetEXT"));
02474     glVertexArraySecondaryColorOffsetEXT =
PFNGLVERTEXARRAYSECONDARYCOLOROFFSETPROC (glfwGetProcAddress("glVertexArraySecondaryColorOffsetEXT"));
02475     glVertexArrayTexCoordOffsetEXT =
PFNGLVERTEXARRAYTEXCOORDOFFSETPROC (glfwGetProcAddress("glVertexArrayTexCoordOffsetEXT"));
02476     glVertexArrayVertexAttribBindingEXT =
PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC (glfwGetProcAddress("glVertexArrayVertexAttribBindingEXT"));
02477     glVertexArrayVertexAttribDivisorEXT =
PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC (glfwGetProcAddress("glVertexArrayVertexAttribDivisorEXT"));
02478     glVertexArrayVertexAttribFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC (glfwGetProcAddress("glVertexArrayVertexAttribFormatEXT"));
02479     glVertexArrayVertexAttribIFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC (glfwGetProcAddress("glVertexArrayVertexAttribIFormatEXT"));
02480     glVertexArrayVertexAttribIOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETPROC (glfwGetProcAddress("glVertexArrayVertexAttribIOffsetEXT"));
02481     glVertexArrayVertexAttribLFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC (glfwGetProcAddress("glVertexArrayVertexAttribLFormatEXT"));
02482     glVertexArrayVertexAttribLOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETPROC (glfwGetProcAddress("glVertexArrayVertexAttribLOffsetEXT"));
02483     glVertexArrayVertexAttribOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBOFFSETPROC (glfwGetProcAddress("glVertexArrayVertexAttribOffsetEXT"));
02484     glVertexArrayVertexBindingDivisorEXT =
PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC (glfwGetProcAddress("glVertexArrayVertexBindingDivisorEXT"));
02485     glVertexArrayVertexBuffer =
PFNGLVERTEXARRAYVERTEXBUFFERPROC (glfwGetProcAddress("glVertexArrayVertexBuffer"));
02486     glVertexArrayVertexBuffers =
PFNGLVERTEXARRAYVERTEXBUFFERSPROC (glfwGetProcAddress("glVertexArrayVertexBuffers"));
02487     glVertexArrayVertexOffsetEXT =
PFNGLVERTEXARRAYVERTEXOFFSETPROC (glfwGetProcAddress("glVertexArrayVertexOffsetEXT"));
02488     glVertexAttrib1d = PFNGLVERTEXATTRIB1DPROC (glfwGetProcAddress("glVertexAttrib1d"));
02489     glVertexAttrib1dv = PFNGLVERTEXATTRIB1DVPROC (glfwGetProcAddress("glVertexAttrib1dv"));
02490     glVertexAttrib1f = PFNGLVERTEXATTRIB1FPROC (glfwGetProcAddress("glVertexAttrib1f"));
02491     glVertexAttrib1fv = PFNGLVERTEXATTRIB1FVPROC (glfwGetProcAddress("glVertexAttrib1fv"));
02492     glVertexAttrib1s = PFNGLVERTEXATTRIB1SPROC (glfwGetProcAddress("glVertexAttrib1s"));
02493     glVertexAttrib1sv = PFNGLVERTEXATTRIB1SVPROC (glfwGetProcAddress("glVertexAttrib1sv"));
02494     glVertexAttrib2d = PFNGLVERTEXATTRIB2DPROC (glfwGetProcAddress("glVertexAttrib2d"));
02495     glVertexAttrib2dv = PFNGLVERTEXATTRIB2DVPROC (glfwGetProcAddress("glVertexAttrib2dv"));
02496     glVertexAttrib2f = PFNGLVERTEXATTRIB2FPROC (glfwGetProcAddress("glVertexAttrib2f"));
02497     glVertexAttrib2fv = PFNGLVERTEXATTRIB2FVPROC (glfwGetProcAddress("glVertexAttrib2fv"));
02498     glVertexAttrib2s = PFNGLVERTEXATTRIB2SPROC (glfwGetProcAddress("glVertexAttrib2s"));
02499     glVertexAttrib2sv = PFNGLVERTEXATTRIB2SVPROC (glfwGetProcAddress("glVertexAttrib2sv"));
02500     glVertexAttrib3d = PFNGLVERTEXATTRIB3DPROC (glfwGetProcAddress("glVertexAttrib3d"));
02501     glVertexAttrib3dv = PFNGLVERTEXATTRIB3DVPROC (glfwGetProcAddress("glVertexAttrib3dv"));
02502     glVertexAttrib3f = PFNGLVERTEXATTRIB3FPROC (glfwGetProcAddress("glVertexAttrib3f"));
02503     glVertexAttrib3fv = PFNGLVERTEXATTRIB3FVPROC (glfwGetProcAddress("glVertexAttrib3fv"));
02504     glVertexAttrib3s = PFNGLVERTEXATTRIB3SPROC (glfwGetProcAddress("glVertexAttrib3s"));
02505     glVertexAttrib3sv = PFNGLVERTEXATTRIB3SVPROC (glfwGetProcAddress("glVertexAttrib3sv"));
```



```

02506     glVertexAttrib4Nbv = PFNGLVERTEXATTRIB4NBVPROC (glfwGetProcAddress ("glVertexAttrib4Nbv"));
02507     glVertexAttrib4Niv = PFNGLVERTEXATTRIB4NIVPROC (glfwGetProcAddress ("glVertexAttrib4Niv"));
02508     glVertexAttrib4Nsv = PFNGLVERTEXATTRIB4NSVPROC (glfwGetProcAddress ("glVertexAttrib4Nsv"));
02509     glVertexAttrib4Nub = PFNGLVERTEXATTRIB4NUBPROC (glfwGetProcAddress ("glVertexAttrib4Nub"));
02510     glVertexAttrib4Nubv = PFNGLVERTEXATTRIB4NUBVPROC (glfwGetProcAddress ("glVertexAttrib4Nubv"));
02511     glVertexAttrib4Nuiv = PFNGLVERTEXATTRIB4NUIVPROC (glfwGetProcAddress ("glVertexAttrib4Nuiv"));
02512     glVertexAttrib4Nusv = PFNGLVERTEXATTRIB4NUSVPROC (glfwGetProcAddress ("glVertexAttrib4Nusv"));
02513     glVertexAttrib4bv = PFNGLVERTEXATTRIB4BVPROC (glfwGetProcAddress ("glVertexAttrib4bv"));
02514     glVertexAttrib4d = PFNGLVERTEXATTRIB4DPROC (glfwGetProcAddress ("glVertexAttrib4d"));
02515     glVertexAttrib4dv = PFNGLVERTEXATTRIB4DVPROC (glfwGetProcAddress ("glVertexAttrib4dv"));
02516     glVertexAttrib4f = PFNGLVERTEXATTRIB4FPROC (glfwGetProcAddress ("glVertexAttrib4f"));
02517     glVertexAttrib4fv = PFNGLVERTEXATTRIB4FVPROC (glfwGetProcAddress ("glVertexAttrib4fv"));
02518     glVertexAttrib4iv = PFNGLVERTEXATTRIB4IVPROC (glfwGetProcAddress ("glVertexAttrib4iv"));
02519     glVertexAttrib4s = PFNGLVERTEXATTRIB4SPROC (glfwGetProcAddress ("glVertexAttrib4s"));
02520     glVertexAttrib4sv = PFNGLVERTEXATTRIB4SVPROC (glfwGetProcAddress ("glVertexAttrib4sv"));
02521     glVertexAttrib4ubv = PFNGLVERTEXATTRIB4UBVPROC (glfwGetProcAddress ("glVertexAttrib4ubv"));
02522     glVertexAttrib4uiv = PFNGLVERTEXATTRIB4UIVPROC (glfwGetProcAddress ("glVertexAttrib4uiv"));
02523     glVertexAttrib4usv = PFNGLVERTEXATTRIB4USVPROC (glfwGetProcAddress ("glVertexAttrib4usv"));
02524     glVertexAttribBinding = PFNGLVERTEXATTRIBBINDINGPROC (glfwGetProcAddress ("glVertexAttribBinding"));
02525     glVertexAttribDivisor = PFNGLVERTEXATTRIBDIVISORPROC (glfwGetProcAddress ("glVertexAttribDivisor"));
02526     glVertexAttribDivisorARB =
PFNGLVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress ("glVertexAttribDivisorARB"));
02527     glVertexAttribFormat = PFNGLVERTEXATTRIBFORMATPROC (glfwGetProcAddress ("glVertexAttribFormat"));
02528     glVertexAttribFormatNV =
PFNGLVERTEXATTRIBFORMATNVPROC (glfwGetProcAddress ("glVertexAttribFormatNV"));
02529     glVertexAttribI1i = PFNGLVERTEXATTRIBI1IPROC (glfwGetProcAddress ("glVertexAttribI1i"));
02530     glVertexAttribI1iv = PFNGLVERTEXATTRIBI1IVPROC (glfwGetProcAddress ("glVertexAttribI1iv"));
02531     glVertexAttribI1ui = PFNGLVERTEXATTRIBI1UIPROC (glfwGetProcAddress ("glVertexAttribI1ui"));
02532     glVertexAttribI1uiv = PFNGLVERTEXATTRIBI1UIVPROC (glfwGetProcAddress ("glVertexAttribI1uiv"));
02533     glVertexAttribI2i = PFNGLVERTEXATTRIBI2IPROC (glfwGetProcAddress ("glVertexAttribI2i"));
02534     glVertexAttribI2iv = PFNGLVERTEXATTRIBI2IVPROC (glfwGetProcAddress ("glVertexAttribI2iv"));
02535     glVertexAttribI2ui = PFNGLVERTEXATTRIBI2UIPROC (glfwGetProcAddress ("glVertexAttribI2ui"));
02536     glVertexAttribI2uiv = PFNGLVERTEXATTRIBI2UIVPROC (glfwGetProcAddress ("glVertexAttribI2uiv"));
02537     glVertexAttribI3i = PFNGLVERTEXATTRIBI3IPROC (glfwGetProcAddress ("glVertexAttribI3i"));
02538     glVertexAttribI3iv = PFNGLVERTEXATTRIBI3IVPROC (glfwGetProcAddress ("glVertexAttribI3iv"));
02539     glVertexAttribI3ui = PFNGLVERTEXATTRIBI3UIPROC (glfwGetProcAddress ("glVertexAttribI3ui"));
02540     glVertexAttribI3uiv = PFNGLVERTEXATTRIBI3UIVPROC (glfwGetProcAddress ("glVertexAttribI3uiv"));
02541     glVertexAttribI4bv = PFNGLVERTEXATTRIBI4BVPROC (glfwGetProcAddress ("glVertexAttribI4bv"));
02542     glVertexAttribI4i = PFNGLVERTEXATTRIBI4IPROC (glfwGetProcAddress ("glVertexAttribI4i"));
02543     glVertexAttribI4iv = PFNGLVERTEXATTRIBI4IVPROC (glfwGetProcAddress ("glVertexAttribI4iv"));
02544     glVertexAttribI4sv = PFNGLVERTEXATTRIBI4SVPROC (glfwGetProcAddress ("glVertexAttribI4sv"));
02545     glVertexAttribI4ubv = PFNGLVERTEXATTRIBI4UBVPROC (glfwGetProcAddress ("glVertexAttribI4ubv"));
02546     glVertexAttribI4ui = PFNGLVERTEXATTRIBI4UIPROC (glfwGetProcAddress ("glVertexAttribI4ui"));
02547     glVertexAttribI4uiv = PFNGLVERTEXATTRIBI4UIVPROC (glfwGetProcAddress ("glVertexAttribI4uiv"));
02548     glVertexAttribI4usv = PFNGLVERTEXATTRIBI4USVPROC (glfwGetProcAddress ("glVertexAttribI4usv"));
02549     glVertexAttribIFormat = PFNGLVERTEXATTRIBIFORMATPROC (glfwGetProcAddress ("glVertexAttribIFormat"));
02550     glVertexAttribIFormatNV =
PFNGLVERTEXATTRIBIFORMATNVPROC (glfwGetProcAddress ("glVertexAttribIFormatNV"));
02551     glVertexAttribIPointer =
PFNGLVERTEXATTRIBIPOINTERPROC (glfwGetProcAddress ("glVertexAttribIPointer"));
02552     glVertexAttribL1d = PFNGLVERTEXATTRIBL1DPROC (glfwGetProcAddress ("glVertexAttribL1d"));
02553     glVertexAttribL1dv = PFNGLVERTEXATTRIBL1DVPROC (glfwGetProcAddress ("glVertexAttribL1dv"));
02554     glVertexAttribL1i64NV = PFNGLVERTEXATTRIBL1I64NVPROC (glfwGetProcAddress ("glVertexAttribL1i64NV"));
02555     glVertexAttribL1i64vNV =
PFNGLVERTEXATTRIBL1I64VNVPROC (glfwGetProcAddress ("glVertexAttribL1i64vNV"));
02556     glVertexAttribL1ui64ARB =
PFNGLVERTEXATTRIBL1UI64ARBPROC (glfwGetProcAddress ("glVertexAttribL1ui64ARB"));
02557     glVertexAttribL1ui64NV =
PFNGLVERTEXATTRIBL1UI64NVPROC (glfwGetProcAddress ("glVertexAttribL1ui64NV"));
02558     glVertexAttribL1ui64vARB =
PFNGLVERTEXATTRIBL1UI64VARBPROC (glfwGetProcAddress ("glVertexAttribL1ui64vARB"));
02559     glVertexAttribL1ui64vNV =
PFNGLVERTEXATTRIBL1UI64VNVPROC (glfwGetProcAddress ("glVertexAttribL1ui64vNV"));
02560     glVertexAttribL2d = PFNGLVERTEXATTRIBL2DPROC (glfwGetProcAddress ("glVertexAttribL2d"));
02561     glVertexAttribL2dv = PFNGLVERTEXATTRIBL2DVPROC (glfwGetProcAddress ("glVertexAttribL2dv"));
02562     glVertexAttribL2i64NV = PFNGLVERTEXATTRIBL2I64NVPROC (glfwGetProcAddress ("glVertexAttribL2i64NV"));
02563     glVertexAttribL2i64vNV =
PFNGLVERTEXATTRIBL2I64VNVPROC (glfwGetProcAddress ("glVertexAttribL2i64vNV"));
02564     glVertexAttribL2ui64NV =
PFNGLVERTEXATTRIBL2UI64NVPROC (glfwGetProcAddress ("glVertexAttribL2ui64NV"));
02565     glVertexAttribL2ui64vNV =
PFNGLVERTEXATTRIBL2UI64VNVPROC (glfwGetProcAddress ("glVertexAttribL2ui64vNV"));
02566     glVertexAttribL3d = PFNGLVERTEXATTRIBL3DPROC (glfwGetProcAddress ("glVertexAttribL3d"));
02567     glVertexAttribL3dv = PFNGLVERTEXATTRIBL3DVPROC (glfwGetProcAddress ("glVertexAttribL3dv"));
02568     glVertexAttribL3i64NV = PFNGLVERTEXATTRIBL3I64NVPROC (glfwGetProcAddress ("glVertexAttribL3i64NV"));
02569     glVertexAttribL3i64vNV =
PFNGLVERTEXATTRIBL3I64VNVPROC (glfwGetProcAddress ("glVertexAttribL3i64vNV"));
02570     glVertexAttribL3ui64NV =
PFNGLVERTEXATTRIBL3UI64NVPROC (glfwGetProcAddress ("glVertexAttribL3ui64NV"));
02571     glVertexAttribL3ui64vNV =
PFNGLVERTEXATTRIBL3UI64VNVPROC (glfwGetProcAddress ("glVertexAttribL3ui64vNV"));
02572     glVertexAttribL4d = PFNGLVERTEXATTRIBL4DPROC (glfwGetProcAddress ("glVertexAttribL4d"));
02573     glVertexAttribL4dv = PFNGLVERTEXATTRIBL4DVPROC (glfwGetProcAddress ("glVertexAttribL4dv"));
02574     glVertexAttribL4i64NV = PFNGLVERTEXATTRIBL4I64NVPROC (glfwGetProcAddress ("glVertexAttribL4i64NV"));
02575     glVertexAttribL4i64vNV =
PFNGLVERTEXATTRIBL4I64VNVPROC (glfwGetProcAddress ("glVertexAttribL4i64vNV"));
02576     glVertexAttribL4ui64NV =

```

```

    PFNGLVERTEXATTRIBL4UI64NVPROC (glfwGetProcAddress("glVertexAttribL4ui64NV"));
02577     glVertexAttribL4ui64vNV =
    PFNGLVERTEXATTRIBL4UI64VNVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
02578     glVertexAttribLFormat = PFNGLVERTEXATTRIBLFORMATPROC (glfwGetProcAddress("glVertexAttribLFormat"));
02579     glVertexAttribLFormatNV =
    PFNGLVERTEXATTRIBLFORMATNVPROC (glfwGetProcAddress("glVertexAttribLFormatNV"));
02580     glVertexAttribLPointer =
    PFNGLVERTEXATTRIBLPOINTERPROC (glfwGetProcAddress("glVertexAttribLPointer"));
02581     glVertexAttribP1ui = PFNGLVERTEXATTRIBP1UIPROC (glfwGetProcAddress("glVertexAttribP1ui"));
02582     glVertexAttribP1uiv = PFNGLVERTEXATTRIBP1UIVPROC (glfwGetProcAddress("glVertexAttribP1uiv"));
02583     glVertexAttribP2ui = PFNGLVERTEXATTRIBP2UIPROC (glfwGetProcAddress("glVertexAttribP2ui"));
02584     glVertexAttribP2uiv = PFNGLVERTEXATTRIBP2UIVPROC (glfwGetProcAddress("glVertexAttribP2uiv"));
02585     glVertexAttribP3ui = PFNGLVERTEXATTRIBP3UIPROC (glfwGetProcAddress("glVertexAttribP3ui"));
02586     glVertexAttribP3uiv = PFNGLVERTEXATTRIBP3UIVPROC (glfwGetProcAddress("glVertexAttribP3uiv"));
02587     glVertexAttribP4ui = PFNGLVERTEXATTRIBP4UIPROC (glfwGetProcAddress("glVertexAttribP4ui"));
02588     glVertexAttribP4uiv = PFNGLVERTEXATTRIBP4UIVPROC (glfwGetProcAddress("glVertexAttribP4uiv"));
02589     glVertexAttribPointer = PFNGLVERTEXATTRIBPOINTERPROC (glfwGetProcAddress("glVertexAttribPointer"));
02590     glVertexBindingDivisor =
    PFNGLVERTEXBINDINGDIVISORPROC (glfwGetProcAddress("glVertexBindingDivisor"));
02591     glVertexFormatNV = PFNGLVERTEXFORMATNVPROC (glfwGetProcAddress("glVertexFormatNV"));
02592     glViewport = PFNGLVIEWPORTPROC (glfwGetProcAddress("glViewport"));
02593     glViewportArrayv = PFNGLVIEWPORTARRAYVPROC (glfwGetProcAddress("glViewportArrayv"));
02594     glViewportIndexedf = PFNGLVIEWPORTINDEXEDFPROC (glfwGetProcAddress("glViewportIndexedf"));
02595     glViewportIndexedfv = PFNGLVIEWPORTINDEXEDFVPROC (glfwGetProcAddress("glViewportIndexedfv"));
02596     glViewportPositionWScaleNV =
    PFNGLVIEWPORTPOSITIONWSCALENVPROC (glfwGetProcAddress("glViewportPositionWScaleNV"));
02597     glViewportSwizzleNV = PFNGLVIEWPORTSWIZZLENVPROC (glfwGetProcAddress("glViewportSwizzleNV"));
02598     glWaitSync = PFNGLWAITSYNCPROC (glfwGetProcAddress("glWaitSync"));
02599     glWaitVkSemaphoreNV = PFNGLWAITVKSEMAPHORENVPROC (glfwGetProcAddress("glWaitVkSemaphoreNV"));
02600     glWeightPathsNV = PFNGLWEIGHTPATHSNVPROC (glfwGetProcAddress("glWeightPathsNV"));
02601     glWindowRectanglesEXT = PFNGLWINDOWRECTANGLESEXTPROC (glfwGetProcAddress("glWindowRectanglesEXT"));
02602 #endif
02603
02604 // 使用している GPU のバッファアライメントを調べる
02605 glGetInteger(GL_UNIFORM_BUFFER_OFFSET_ALIGNMENT, &ggBufferAlignment);
02606 }
02607
02608 //
02609 // OpenGL のエラーをチェックする
02610 //
02611 // OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02612 //
02613 // msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02614 //
02615 void gg::ggError(const std::string& name, unsigned int line)
02616 {
02617     const GLenum error{ glGetError() };
02618
02619     if (error != GL_NO_ERROR)
02620     {
02621         if (!name.empty())
02622         {
02623             std::cerr << name;
02624             if (line > 0) std::cerr << " (" << line << ")";
02625             std::cerr << ": ";
02626         }
02627
02628         switch (error)
02629         {
02630             case GL_INVALID_ENUM:
02631                 std::cerr << "An unacceptable value is specified for an enumerated argument" << std::endl;
02632                 break;
02633             case GL_INVALID_VALUE:
02634                 std::cerr << "A numeric argument is out of range" << std::endl;
02635                 break;
02636             case GL_INVALID_OPERATION:
02637                 std::cerr << "The specified operation is not allowed in the current state" << std::endl;
02638                 break;
02639             case GL_OUT_OF_MEMORY:
02640                 std::cerr << "There is not enough memory left to execute the command" << std::endl;
02641                 break;
02642             case GL_INVALID_FRAMEBUFFER_OPERATION:
02643                 std::cerr << "The specified operation is not allowed current frame buffer" << std::endl;
02644                 break;
02645             default:
02646                 std::cerr << "An OpenGL error has occurred: " << std::hex << std::showbase << error << std::endl;
02647                 break;
02648         }
02649     }
02650 }
02651
02652 //
02653 // FBO のエラーをチェックする
02654 //
02655 // FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02656 //
02657 // msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない

```

```

02658 //
02659 void gg::ggFBOError(const std::string& name, unsigned int line)
02660 {
02661     const auto status{ glCheckFramebufferStatus(GL_FRAMEBUFFER) };
02662
02663     if (status != GL_FRAMEBUFFER_COMPLETE)
02664     {
02665         if (!name.empty())
02666         {
02667             std::cerr << name;
02668             if (line > 0) std::cerr << " (" << line << ")";
02669             std::cerr << ": ";
02670         }
02671
02672         switch (status)
02673         {
02674             case GL_FRAMEBUFFER_UNDEFINED:
02675                 std::cerr << "the default framebuffer does not exist" << std::endl;
02676                 break;
02677             case GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT:
02678                 std::cerr << "Framebuffer incomplete, duplicate attachment" << std::endl;
02679                 break;
02680             case GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT:
02681                 std::cerr << "Framebuffer incomplete, missing attachment" << std::endl;
02682                 break;
02683             case GL_FRAMEBUFFER_UNSUPPORTED:
02684                 std::cerr << "Unsupported framebuffer internal" << std::endl;
02685                 break;
02686             case GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE:
02687                 std::cerr << "The value of GL_RENDERBUFFER_SAMPLES is not"
02688                     " the same for all attached renderbuffers or,"
02689                     " if the attached images are a mix of renderbuffers and textures,"
02690                     " the value of GL_RENDERBUFFER_SAMPLES is not zero" << std::endl;
02691                 break;
02692             #if !defined(GL_GLES_PROTOTYPES)
02693             case GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS:
02694                 std::cerr << "Any framebuffer attachment is layered,"
02695                     " and any populated attachment is not layered,"
02696                     " or if all populated color attachments are not from textures"
02697                     " of the same target" << std::endl;
02698                 break;
02699             case GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER:
02700                 std::cerr << "Framebuffer incomplete, missing draw buffer" << std::endl;
02701                 break;
02702             case GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER:
02703                 std::cerr << "Framebuffer incomplete, missing read buffer" << std::endl;
02704                 break;
02705             #endif
02706             default:
02707                 std::cerr << "Programming error; will fail on all hardware: " << std::hex << std::showbase <<
02708                     status << std::endl;
02709                 break;
02710         }
02711     }
02712
02713     //
02714     // 変換行列：行列とベクトルの積  $c \leftarrow a \times b$ 
02715     //
02716     void gg::GgMatrix::projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02717     {
02718         for (int i = 0; i < 4; ++i)
02719         {
02720             c[i] = a[0 + i] * b[0] + a[4 + i] * b[1] + a[8 + i] * b[2] + a[12 + i] * b[3];
02721         }
02722     }
02723
02724     //
02725     // 変換行列：行列と行列の積  $c \leftarrow a \times b$ 
02726     //
02727     void gg::GgMatrix::multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02728     {
02729         for (int i = 0; i < 16; ++i)
02730         {
02731             int j = i & 3, k = i & ~3;
02732
02733             c[i] = a[0 + j] * b[k + 0] + a[4 + j] * b[k + 1] + a[8 + j] * b[k + 2] + a[12 + j] * b[k + 3];
02734         }
02735     }
02736
02737     //
02738     // 変換行列：単位行列を設定する
02739     //
02740     gg::GgMatrix& gg::GgMatrix::loadIdentity()
02741     {
02742         *this =
02743         {

```

```

02744     1.0f, 0.0f, 0.0f, 0.0f,
02745     0.0f, 1.0f, 0.0f, 0.0f,
02746     0.0f, 0.0f, 1.0f, 0.0f,
02747     0.0f, 0.0f, 0.0f, 1.0f
02748 };
02749
02750 return *this;
02751 }
02752
02753 //
02754 // 変換行列：平行移動変換行列を設定する
02755 //
02756 gg::GgMatrix& gg::GgMatrix::loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02757 {
02758     *this =
02759     {
02760         w,    0.0f, 0.0f, 0.0f,
02761         0.0f, w,    0.0f, 0.0f,
02762         0.0f, 0.0f, w,    0.0f,
02763         x,    y,    z,    w
02764     };
02765
02766     return *this;
02767 }
02768
02769 //
02770 // 変換行列：拡大縮小変換行列を設定する
02771 //
02772 gg::GgMatrix& gg::GgMatrix::loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02773 {
02774     *this =
02775     {
02776         x,    0.0f, 0.0f, 0.0f,
02777         0.0f, y,    0.0f, 0.0f,
02778         0.0f, 0.0f, z,    0.0f,
02779         0.0f, 0.0f, 0.0f, w
02780     };
02781
02782     return *this;
02783 }
02784
02785 //
02786 // 変換行列：x 軸中心の回転変換行列を設定する
02787 //
02788 gg::GgMatrix& gg::GgMatrix::loadRotateX(GLfloat a)
02789 {
02790     const auto c{ cosf(a) };
02791     const auto s{ sinf(a) };
02792
02793     *this =
02794     {
02795         1.0f, 0.0f, 0.0f, 0.0f,
02796         0.0f, c,    s,    0.0f,
02797         0.0f, -s,   c,    0.0f,
02798         0.0f, 0.0f, 0.0f, 1.0f
02799     };
02800
02801     return *this;
02802 }
02803
02804 //
02805 // 変換行列：y 軸中心の回転変換行列を設定する
02806 //
02807 gg::GgMatrix& gg::GgMatrix::loadRotateY(GLfloat a)
02808 {
02809     const auto c{ cosf(a) };
02810     const auto s{ sinf(a) };
02811
02812     *this =
02813     {
02814         c,    0.0f, -s,    0.0f,
02815         0.0f, 1.0f, 0.0f, 0.0f,
02816         s,    0.0f, c,    0.0f,
02817         0.0f, 0.0f, 0.0f, 1.0f
02818     };
02819
02820     return *this;
02821 }
02822
02823 //
02824 // 変換行列：z 軸中心の回転変換行列を設定する
02825 //
02826 gg::GgMatrix& gg::GgMatrix::loadRotateZ(GLfloat a)
02827 {
02828     const auto c{ cosf(a) };
02829     const auto s{ sinf(a) };
02830

```

```

02831     *this =
02832     {
02833         c,      s,      0.0f, 0.0f,
02834         -s,     c,      0.0f, 0.0f,
02835         0.0f, 0.0f, 1.0f, 0.0f,
02836         0.0f, 0.0f, 0.0f, 1.0f
02837     };
02838
02839     return *this;
02840 }
02841
02842 //
02843 // 変換行列：任意軸中心の回転変換行列を設定する
02844 //
02845 gg::GgMatrix& gg::GgMatrix::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
02846 {
02847     const auto d{ sqrtf(x * x + y * y + z * z) };
02848
02849     if (d > 0.0f)
02850     {
02851         const auto l{ x / d }, m{ y / d }, n{ z / d };
02852         const auto l2{ l * l }, m2{ m * m }, n2{ n * n };
02853         const auto lm{ l * m }, mn{ m * n }, nl{ n * l };
02854         const auto c{ cosf(a) }, cl{ 1.0f - c };
02855         const auto s{ sinf(a) };
02856
02857         *this =
02858         {
02859             (1.0f - l2) * c + l2,
02860             lm * cl + n * s,
02861             nl * cl - m * s,
02862             0.0f,
02863
02864             lm * cl - n * s,
02865             (1.0f - m2) * c + m2,
02866             mn * cl + l * s,
02867             0.0f,
02868
02869             nl * cl + m * s,
02870             mn * cl - l * s,
02871             (1.0f - n2) * c + n2,
02872             0.0f,
02873
02874             0.0f,
02875             0.0f,
02876             0.0f,
02877             1.0f,
02878         };
02879     }
02880
02881     return *this;
02882 }
02883
02884 //
02885 // 変換行列：転置行列を設定する
02886 //
02887 gg::GgMatrix& gg::GgMatrix::loadTranspose(const GLfloat* marray)
02888 {
02889     *this =
02890     {
02891         marray[ 0],
02892         marray[ 4],
02893         marray[ 8],
02894         marray[12],
02895         marray[ 1],
02896         marray[ 5],
02897         marray[ 9],
02898         marray[13],
02899         marray[ 2],
02900         marray[ 6],
02901         marray[10],
02902         marray[14],
02903         marray[ 3],
02904         marray[ 7],
02905         marray[11],
02906         marray[15]
02907     };
02908
02909     return *this;
02910 }
02911
02912 //
02913 // 変換行列：逆行列を設定する
02914 //
02915 gg::GgMatrix& gg::GgMatrix::loadInvert(const GLfloat* marray)
02916 {
02917     GLfloat lu[20];

```



```

02918   GLfloat* plu[4];
02919
02920   // j 行の要素の値の絶対値の最大値を plu[j][4] に求める
02921   for (int j = 0; j < 4; ++j)
02922   {
02923       auto max{ fabs(*(plu[j] = lu + 5 * j) = *(marray++)) };
02924
02925       for (int i = 0; ++i < 4;)
02926       {
02927           auto a{ fabs(plu[j][i] = *(marray++)) };
02928           if (a > max) max = a;
02929       }
02930       if (max == 0.0f) return *this;
02931       plu[j][4] = 1.0f / max;
02932   }
02933
02934   // ピボットを考慮した LU 分解
02935   for (int j = 0; j < 4; ++j)
02936   {
02937       auto max{ fabs(plu[j][j] * plu[j][4]) };
02938       int i = j;
02939
02940       for (int k = j; ++k < 4;)
02941       {
02942           auto a{ fabs(plu[k][j] * plu[k][4]) };
02943           if (a > max)
02944           {
02945               max = a;
02946               i = k;
02947           }
02948       }
02949       if (i > j)
02950       {
02951           auto* t{ plu[j] };
02952           plu[j] = plu[i];
02953           plu[i] = t;
02954       }
02955       if (plu[j][j] == 0.0f) return *this;
02956       for (int k = j; ++k < 4;)
02957       {
02958           plu[k][j] /= plu[j][j];
02959           for (int i = j; ++i < 4;)
02960           {
02961               plu[k][i] -= plu[j][i] * plu[k][j];
02962           }
02963       }
02964   }
02965
02966   // LU 分解から逆行列を求める
02967   for (int k = 0; k < 4; ++k)
02968   {
02969       // array に単位行列を設定する
02970       for (int i = 0; i < 4; ++i)
02971       {
02972           (*this)[i * 4 + k] = (plu[i] == lu + k * 5) ? 1.0f : 0.0f;
02973       }
02974       // lu から逆行列を求める
02975       for (int i = 0; i < 4; ++i)
02976       {
02977           for (int j = i; ++j < 4;)
02978           {
02979               (*this)[j * 4 + k] -= (*this)[i * 4 + k] * plu[j][i];
02980           }
02981       }
02982       for (int i = 4; --i >= 0;)
02983       {
02984           for (int j = i; ++j < 4;)
02985           {
02986               (*this)[i * 4 + k] -= plu[i][j] * (*this)[j * 4 + k];
02987           }
02988           (*this)[i * 4 + k] /= plu[i][i];
02989       }
02990   }
02991
02992   return *this;
02993 }
02994
02995 //
02996 // 変換行列：法線変換行列を設定する
02997 //
02998 gg::GgMatrix& gg::GgMatrix::loadNormal(const GLfloat* marray)
02999 {
03000     *this =
03001     {
03002         marray[ 5] * marray[10] - marray[ 6] * marray[ 9],
03003         marray[ 6] * marray[ 8] - marray[ 4] * marray[10],
03004         marray[ 4] * marray[ 9] - marray[ 5] * marray[ 8],

```

```

03005     0.0f,
03006
03007     marray[ 9] * marray[ 2] - marray[10] * marray[ 1],
03008     marray[10] * marray[ 0] - marray[ 8] * marray[ 2],
03009     marray[ 8] * marray[ 1] - marray[ 9] * marray[ 0],
03010     0.0f,
03011
03012     marray[ 1] * marray[ 6] - marray[ 2] * marray[ 5],
03013     marray[ 2] * marray[ 4] - marray[ 0] * marray[ 6],
03014     marray[ 0] * marray[ 5] - marray[ 1] * marray[ 4],
03015     0.0f,
03016
03017     0.0f,
03018     0.0f,
03019     0.0f,
03020     1.0f
03021 };
03022
03023 return *this;
03024 }
03025
03026 //
03027 // 変換行列：ビュー変換行列を設定する
03028 //
03029 gg::GgMatrix& gg::GgMatrix::loadLookat(
03030     GLfloat ex, GLfloat ey, GLfloat ez,
03031     GLfloat tx, GLfloat ty, GLfloat tz,
03032     GLfloat ux, GLfloat uy, GLfloat uz
03033 )
03034 {
03035     // z 軸 = e - t
03036     const auto zx{ ex - tx };
03037     const auto zy{ ey - ty };
03038     const auto zz{ ez - tz };
03039
03040     // z 軸の長さ
03041     const auto z{ sqrtf(zx * zx + zy * zy + zz * zz) };
03042
03043     // x 軸 = u × z 軸
03044     const auto xx{ uy * zz - uz * zy };
03045     const auto xy{ uz * zx - ux * zz };
03046     const auto xz{ ux * zy - uy * zx };
03047
03048     // x 軸の長さ
03049     const auto x{ sqrtf(xx * xx + xy * xy + xz * xz) };
03050
03051     // y 軸 = z 軸 × x 軸
03052     const auto yx{ zy * xz - zz * xy };
03053     const auto yy{ zz * xx - zx * xz };
03054     const auto yz{ zx * xy - zy * xx };
03055
03056     // y 軸の長さ
03057     const auto y{ sqrtf(yx * yx + yy * yy + yz * yz) };
03058
03059     // y 軸の長さをチェック
03060     if (fabs(y) < std::numeric_limits<float>::epsilon()) return *this;
03061
03062     (*this)[ 0] = xx / x;
03063     (*this)[ 1] = yx / y;
03064     (*this)[ 2] = zx / z;
03065     (*this)[ 3] = 0.0f;
03066
03067     (*this)[ 4] = xy / x;
03068     (*this)[ 5] = yy / y;
03069     (*this)[ 6] = zy / z;
03070     (*this)[ 7] = 0.0f;
03071
03072     (*this)[ 8] = xz / x;
03073     (*this)[ 9] = yz / y;
03074     (*this)[10] = zz / z;
03075     (*this)[11] = 0.0f;
03076
03077     (*this)[12] = -(ex * (*this)[ 0] + ey * (*this)[ 4] + ez * (*this)[ 8]);
03078     (*this)[13] = -(ex * (*this)[ 1] + ey * (*this)[ 5] + ez * (*this)[ 9]);
03079     (*this)[14] = -(ex * (*this)[ 2] + ey * (*this)[ 6] + ez * (*this)[10]);
03080     (*this)[15] = 1.0f;
03081
03082     return *this;
03083 }
03084
03085 //
03086 // 変換行列：平行投影変換行列を設定する
03087 //
03088 gg::GgMatrix& gg::GgMatrix::loadOrthogonal(
03089     GLfloat left, GLfloat right,
03090     GLfloat bottom, GLfloat top,
03091     GLfloat zNear, GLfloat zFar

```

```

03092 )
03093 {
03094     const auto dx{ right - left };
03095     const auto dy{ top - bottom };
03096     const auto dz{ zFar - zNear };
03097
03098     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03099
03100     *this =
03101     {
03102         2.0f / dx,
03103         0.0f,
03104         0.0f,
03105         0.0f,
03106
03107         0.0f,
03108         2.0f / dy,
03109         0.0f,
03110         0.0f,
03111
03112         0.0f,
03113         0.0f,
03114         -2.0f / dz,
03115         0.0f,
03116
03117         -(right + left) / dx,
03118         -(top + bottom) / dy,
03119         -(zFar + zNear) / dz,
03120         1.0f
03121     };
03122
03123     return *this;
03124 }
03125
03126 //
03127 // 変換行列：透視投影変換行列を設定する
03128 //
03129 gg::GgMatrix& gg::GgMatrix::loadFrustum(
03130     GLfloat left, GLfloat right,
03131     GLfloat bottom, GLfloat top,
03132     GLfloat zNear, GLfloat zFar
03133 )
03134 {
03135     const auto dx{ right - left };
03136     const auto dy{ top - bottom };
03137     const auto dz{ zFar - zNear };
03138
03139     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03140
03141     *this =
03142     {
03143         2.0f * zNear / dx,
03144         0.0f,
03145         0.0f,
03146         0.0f,
03147
03148         0.0f,
03149         2.0f * zNear / dy,
03150         0.0f,
03151         0.0f,
03152
03153         (right + left) / dx,
03154         (top + bottom) / dy,
03155         -(zFar + zNear) / dz,
03156         -1.0f,
03157
03158         0.0f,
03159         0.0f,
03160         -2.0f * zFar * zNear / dz,
03161         0.0f
03162     };
03163
03164     return *this;
03165 }
03166
03167 //
03168 // 変換行列：画角から透視投影変換行列を設定する
03169 //
03170 gg::GgMatrix& gg::GgMatrix::loadPerspective(
03171     GLfloat fovy, GLfloat aspect,
03172     GLfloat zNear, GLfloat zFar
03173 )
03174 {
03175     const auto dz{ zFar - zNear };
03176
03177     if (dz == 0.0f) return *this;
03178

```

```

03179     const auto f{ 1.0f / tanf(fovy * 0.5f) };
03180
03181     *this =
03182     {
03183         f / aspect,
03184         0.0f,
03185         0.0f,
03186         0.0f,
03187
03188         0.0f,
03189         f,
03190         0.0f,
03191         0.0f,
03192
03193         0.0f,
03194         0.0f,
03195         -(zFar + zNear) / dz,
03196         -1.0f,
03197
03198         0.0f,
03199         0.0f,
03200         -2.0f * zFar * zNear / dz,
03201         0.0f
03202     };
03203
03204     return *this;
03205 }
03206
03207 //
03208 // 四元数: GgQuaternion 型の四元数 p, q の積を r に求める
03209 //
03210 void gg::GgQuaternion::multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const
03211 {
03212     r[0] = p[1] * q[2] - p[2] * q[1] + p[0] * q[3] + p[3] * q[0];
03213     r[1] = p[2] * q[0] - p[0] * q[2] + p[1] * q[3] + p[3] * q[1];
03214     r[2] = p[0] * q[1] - p[1] * q[0] + p[2] * q[3] + p[3] * q[2];
03215     r[3] = p[3] * q[3] - p[0] * q[0] - p[1] * q[1] - p[2] * q[2];
03216 }
03217
03218 //
03219 // 四元数: GgQuaternion 型の四元数 q が表す変換行列を m に求める
03220 //
03221 void gg::GgQuaternion::toMatrix(GLfloat* m, const GLfloat* q) const
03222 {
03223     const auto xx{ q[0] * q[0] * 2.0f };
03224     const auto yy{ q[1] * q[1] * 2.0f };
03225     const auto zz{ q[2] * q[2] * 2.0f };
03226     const auto xy{ q[0] * q[1] * 2.0f };
03227     const auto yz{ q[1] * q[2] * 2.0f };
03228     const auto zx{ q[2] * q[0] * 2.0f };
03229     const auto xw{ q[0] * q[3] * 2.0f };
03230     const auto yw{ q[1] * q[3] * 2.0f };
03231     const auto zw{ q[2] * q[3] * 2.0f };
03232
03233     m[ 0] = 1.0f - yy - zz;
03234     m[ 1] = xy + zw;
03235     m[ 2] = zx - yw;
03236     m[ 4] = xy - zw;
03237     m[ 5] = 1.0f - zz - xx;
03238     m[ 6] = yz + xw;
03239     m[ 8] = zx + yw;
03240     m[ 9] = yz - xw;
03241     m[10] = 1.0f - xx - yy;
03242     m[ 3] = m[ 7] = m[11] = m[12] = m[13] = m[14] = 0.0f;
03243     m[15] = 1.0f;
03244 }
03245
03246 //
03247 // 四元数: 回転変換行列 a が表す四元数を q に求める
03248 //
03249 void gg::GgQuaternion::toQuaternion(GLfloat* q, const GLfloat* a) const
03250 {
03251     const auto tr{ a[0] + a[5] + a[10] + a[15] };
03252
03253     if (tr > 0.0f)
03254     {
03255         q[3] = sqrtf(tr) * 0.5f;
03256         q[0] = (a[6] - a[9]) * 0.25f / q[3];
03257         q[1] = (a[8] - a[2]) * 0.25f / q[3];
03258         q[2] = (a[1] - a[4]) * 0.25f / q[3];
03259     }
03260 }
03261
03262 //
03263 // 四元数: 球面線形補間 p に q と r を t で補間した四元数を求める
03264 //
03265 void gg::GgQuaternion::slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const

```

```

03266 {
03267     const auto qr{ ggDot3(q, r) };
03268     const auto ss{ 1.0f - qr * qr };
03269
03270     if (ss == 0.0f)
03271     {
03272         if (p != q)
03273         {
03274             p[0] = q[0];
03275             p[1] = q[1];
03276             p[2] = q[2];
03277             p[3] = q[3];
03278         }
03279     }
03280     else
03281     {
03282         const auto sp{ sqrtf(ss) };
03283         const auto ph{ acosf(qr) };
03284         const auto pt{ ph * t };
03285         const auto t1{ sinf(pt) / sp };
03286         const auto t0{ sinf(ph - pt) / sp };
03287
03288         p[0] = q[0] * t0 + r[0] * t1;
03289         p[1] = q[1] * t0 + r[1] * t1;
03290         p[2] = q[2] * t0 + r[2] * t1;
03291         p[3] = q[3] * t0 + r[3] * t1;
03292     }
03293 }
03294
03295 //
03296 // 四元数：(x, y, z) を軸とし角度 a 回転する四元数を求める
03297 //
03298 gg::GgQuaternion& gg::GgQuaternion::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03299 {
03300     const auto l{ x * x + y * y + z * z };
03301     const auto w{ cosf(a * 0.5f) };
03302
03303     if (fabs(l) > std::numeric_limits<float>::epsilon())
03304     {
03305         const auto s{ sinf(a) / sqrtf(l) };
03306
03307         *this = GgQuaternion{ x * s, y * s, z * s, w };
03308     }
03309     else
03310     {
03311         *this = GgQuaternion{ 0.0f, 0.0f, 0.0f, w };
03312     }
03313
03314     return *this;
03315 }
03316
03317 //
03318 // x 軸中心に角度 a 回転する四元数を格納する
03319 //
03320 gg::GgQuaternion& gg::GgQuaternion::loadRotateX(GLfloat a)
03321 {
03322     const auto t{ a * 0.5f };
03323
03324     *this = GgQuaternion{ sinf(t), 0.0f, 0.0f, cosf(t) };
03325
03326     return *this;
03327 }
03328
03329 //
03330 // y 軸中心に角度 a 回転する四元数を格納する
03331 //
03332 gg::GgQuaternion& gg::GgQuaternion::loadRotateY(GLfloat a)
03333 {
03334     const auto t{ a * 0.5f };
03335
03336     *this = GgQuaternion{ 0.0f, sinf(t), 0.0f, cosf(t) };
03337
03338     return *this;
03339 }
03340
03341 //
03342 // z 軸中心に角度 a 回転する四元数を格納する
03343 //
03344 gg::GgQuaternion& gg::GgQuaternion::loadRotateZ(GLfloat a)
03345 {
03346     const auto t{ a * 0.5f };
03347
03348     *this = GgQuaternion{ 0.0f, 0.0f, sinf(t), cosf(t) };
03349
03350     return *this;
03351 }
03352

```

```

03353 //
03354 // 四元数：オイラー角 (heading, pitch, roll) にもとづいて四元数を求める
03355 //
03356 gg::GgQuaternion& gg::GgQuaternion::loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)
03357 {
03358     GgQuaternion h, p, r;
03359
03360     h.loadRotateY(heading);
03361     p.loadRotateX(pitch);
03362     r.loadRotateZ(roll);
03363
03364     *this = h * p * r;
03365
03366     return *this;
03367 }
03368 //
03369 // 四元数：正規化して格納する
03370 //
03371 //
03372 gg::GgQuaternion& gg::GgQuaternion::loadNormalize(const GLfloat* a)
03373 {
03374     *this = a;
03375
03376     ggNormalize4(data());
03377
03378     return *this;
03379 }
03380 //
03381 // 四元数：共役四元数を格納する
03382 //
03383 //
03384 gg::GgQuaternion& gg::GgQuaternion::loadConjugate(const GLfloat* a)
03385 {
03386     // w 要素を反転する
03387     data()[0] = a[0];
03388     data()[1] = a[1];
03389     data()[2] = a[2];
03390     data()[3] = -a[3];
03391
03392     return *this;
03393 }
03394 //
03395 // 四元数：逆元を格納する
03396 //
03397 //
03398 gg::GgQuaternion& gg::GgQuaternion::loadInvert(const GLfloat* a)
03399 {
03400     // ノルムの二乗を求める
03401     const auto l(ggDot4(a, a));
03402
03403     if (l > 0.0f)
03404     {
03405         // 共役四元数を求める
03406         GgQuaternion r;
03407         r.loadConjugate(a);
03408
03409         // ノルムの二乗で割る
03410         data()[0] = r[0] / l;
03411         data()[1] = r[1] / l;
03412         data()[2] = r[2] / l;
03413         data()[3] = r[3] / l;
03414     }
03415
03416     return *this;
03417 }
03418 //
03419 // 簡易トラックボール処理：リセット
03420 //
03421 //
03422 void gg::GgTrackball::reset(const GgQuaternion& q)
03423 {
03424     // ドラッグ中ではない
03425     drag = false;
03426
03427     // 単位クォーターニオンに初期値を与える
03428     *this = cq = q;
03429
03430     // 回転行列を初期化する
03431     GgQuaternion::getMatrix(rt);
03432 }
03433 //
03434 // 簡易トラックボール処理：トラックボールする領域の設定
03435 //
03436 //
03437 // Reshape コールバック (resize) の中で実行する
03438 // (w, h) ウィンドウサイズ
03439 //

```

```

03440 void gg::GgTrackball::region(GLfloat w, GLfloat h)
03441 {
03442     // マウスポインタ位置のウィンドウ内の相対的位置への換算用
03443     scale[0] = 2.0f / w;
03444     scale[1] = 2.0f / h;
03445 }
03446
03447 //
03448 // 簡易トラックボール処理：ドラッグ開始時の処理
03449 //
03450 //   マウスボタンを押したときに実行する
03451 //   (x, y) 現在のマウス位置
03452 //
03453 void gg::GgTrackball::begin(GLfloat x, GLfloat y)
03454 {
03455     // ドラッグ開始
03456     drag = true;
03457
03458     // ドラッグ開始点を記録する
03459     start[0] = x;
03460     start[1] = y;
03461 }
03462
03463 //
03464 // 簡易トラックボール処理：ドラッグ中の処理
03465 //
03466 //   マウスのドラッグ中に実行する
03467 //   (x, y) 現在のマウス位置
03468 //
03469 void gg::GgTrackball::motion(GLfloat x, GLfloat y)
03470 {
03471     // ドラッグ中でなければ何もしない
03472     if (!drag) return;
03473
03474     // マウスポインタの位置のドラッグ開始位置からの変位
03475     const auto dx{ (x - start[0]) * scale[0] };
03476     const auto dy{ (y - start[1]) * scale[1] };
03477
03478     // マウスポインタの位置のドラッグ開始位置からの距離
03479     const auto a{ hypot(dx, dy) };
03480
03481     // マウスポインタの位置がドラッグ開始位置から移動していれば
03482     if (a > 0.0)
03483     {
03484         // 現在の回転の四元数に作った四元数を掛けて合成する
03485         *this = ggRotateQuaternion(dy, dx, 0.0f, a * 3.1415926536f) * cq;
03486
03487         // 合成した四元数から回転の変換行列を求める
03488         GgQuaternion::getMatrix(rt);
03489     }
03490 }
03491
03492 //
03493 // 簡易トラックボール処理：回転角の修正
03494 //
03495 //   現在の回転角を修正する
03496 //   q 修正分の回転角を表す四元数
03497 //
03498 void gg::GgTrackball::rotate(const GgQuaternion& q)
03499 {
03500     // ドラッグ中なら何もしない
03501     if (drag) return;
03502
03503     // 保存されている四元数に修正分の四元数を掛けて合成する
03504     *this = q * cq;
03505
03506     // 合成した四元数から回転の変換行列を求める
03507     GgQuaternion::getMatrix(rt);
03508
03509     // 誤差を吸収するために正規化して保存する
03510     cq = normalize();
03511 }
03512
03513 //
03514 // 簡易トラックボール処理：停止時の処理
03515 //
03516 //   マウスボタンを離したときに実行する
03517 //   (x, y) 現在のマウス位置
03518 //
03519 void gg::GgTrackball::end(GLfloat x, GLfloat y)
03520 {
03521     // ドラッグ終了点における回転を求める
03522     motion(x, y);
03523
03524     // 誤差を吸収するために正規化して保存する
03525     cq = normalize();
03526 }

```

```

03527 // ドラッグ終了
03528 drag = false;
03529 }
03530
03531 //
03532 // 配列に格納された画像の内容を TGA ファイルに保存する
03533 //
03534 // name ファイル名
03535 // buffer 画像データ
03536 // width 画像の横の画素数
03537 // height 画像の縦の画素数
03538 // depth 画像の 1 画素のバイト数
03539 // 戻り値 保存に成功すれば true, 失敗すれば false
03540 //
03541 bool gg::ggSaveTga(
03542     const std::string& name,
03543     const void* buffer,
03544     unsigned int width,
03545     unsigned int height,
03546     unsigned int depth
03547 )
03548 {
03549     // ファイルを開く
03550     std::ofstream file{ Utf8ToTChar(name), std::ios::binary };
03551
03552     // ファイルが開けなかったら戻る
03553     if (file.fail()) return false;
03554
03555     // 画像のヘッダ
03556     const unsigned char type{ static_cast<unsigned char>(depth == 0 ? 0 : depth < 3 ? 3 : 2) };
03557     const unsigned char alpha{ static_cast<unsigned char>(depth == 2 || depth == 4 ? 8 : 0) };
03558     const unsigned char header[18]
03559     {
03560         0, // ID length
03561         0, // Color map type (none)
03562         type, // Image Type (2:RGB, 3:Grayscale)
03563         0, 0, // Offset into the color map table
03564         0, 0, // Number of color map entries
03565         0, // Number of a color map entry bits per pixel
03566         0, 0, // Horizontal image position
03567         0, 0, // Vertical image position
03568         static_cast<unsigned char>(width & 0xff),
03569         static_cast<unsigned char>(width >> 8),
03570         static_cast<unsigned char>(height & 0xff),
03571         static_cast<unsigned char>(height >> 8),
03572         static_cast<unsigned char>(depth * 8),
03573         alpha // Image descriptor
03574     };
03575
03576     // ヘッダを書き込む
03577     file.write(reinterpret_cast<const char*>(header), sizeof header);
03578
03579     // ヘッダの書き込みに失敗したら戻る
03580     if (file.bad()) return false;
03581
03582     // データを書き込む
03583     const unsigned int size{ width * height * depth };
03584     if (type == 2)
03585     {
03586         // フルカラー
03587         std::vector<char> temp(size);
03588         for (GLuint i = 0; i < size; i += depth)
03589         {
03590             temp[i + 2] = static_cast<const char*>(buffer)[i + 0];
03591             temp[i + 1] = static_cast<const char*>(buffer)[i + 1];
03592             temp[i + 0] = static_cast<const char*>(buffer)[i + 2];
03593             if (depth == 4) temp[i + 3] = static_cast<const char*>(buffer)[i + 3];
03594         }
03595         file.write(temp.data(), size);
03596     }
03597     else if (type == 3)
03598     {
03599         // グレースケール
03600         file.write(static_cast<const char*>(buffer), size);
03601     }
03602
03603     // フッタを書き込む
03604     constexpr char footer[] = "\0\0\0\0\0\0\0\0TRUEVISION-XFILE.";
03605     file.write(footer, sizeof footer);
03606
03607     // データの書き込みに失敗していなければ true を返す
03608     return file.bad() != false;
03609 }
03610
03611 //
03612 // カラーバッファの内容を TGA ファイルに保存する
03613 //

```



```

03614 //   name 保存するファイル名
03615 //   戻り値 保存に成功すれば true, 失敗すれば false
03616 //
03617 bool gg::ggSaveColor(const std::string& name)
03618 {
03619     // 現在のビューポートのサイズを得る
03620     GLint viewport[4];
03621     glGetIntegerv(GL_VIEWPORT, viewport);
03622
03623     // ビューポートのサイズ分のメモリを確保する
03624     std::vector<GLubyte> buffer(viewport[2] * viewport[3] * 3);
03625
03626     // 画面表示の完了を待つ
03627     glFinish();
03628
03629     // カラーバッファを読み込む
03630     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03631                 GL_RGB, GL_UNSIGNED_BYTE, buffer.data());
03632
03633     // 読み込んだデータをファイルに書き込む
03634     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 3);
03635 }
03636
03637 //
03638 // デプスバッファの内容を TGA ファイルに保存する
03639 //
03640 //   name 保存するファイル名
03641 //   戻り値 保存に成功すれば true, 失敗すれば false
03642 //
03643 bool gg::ggSaveDepth(const std::string& name)
03644 {
03645     // 現在のビューポートのサイズを得る
03646     GLint viewport[4];
03647     glGetIntegerv(GL_VIEWPORT, viewport);
03648
03649     // ビューポートのサイズ分のメモリを確保する
03650     std::vector<GLubyte> buffer(viewport[2] * viewport[3]);
03651
03652     // 画面表示の完了を待つ
03653     glFinish();
03654
03655     // デプスバッファを読み込む
03656     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03657                 GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, buffer.data());
03658
03659     // 読み込んだデータをファイルに書き込む
03660     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 1);
03661 }
03662
03663 //
03664 // TGA ファイル (8/16/24/32bit) を読み込む
03665 //
03666 //   name 読み込むファイル名
03667 //   pWidth 読み込んだファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03668 //   pHeight 読み込んだファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03669 //   pFormat 読み込んだファイルのフォーマットの格納先のポインタ (nullptr なら格納しない)
03670 //   image 読み込んだ画像を格納する vector
03671 //   戻り値 読み込みに成功すれば true, 失敗すれば false
03672 //
03673 bool gg::ggReadImage(
03674     const std::string& name,
03675     std::vector<GLubyte>& image,
03676     GLsizei* pWidth,
03677     GLsizei* pHeight,
03678     GLenum* pFormat
03679 )
03680 {
03681     // ファイルを開く
03682     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
03683
03684     // ファイルが開けなかったら戻る
03685     if (file.fail()) return false;
03686
03687     // ヘッダを読み込む
03688     unsigned char header[18];
03689     file.read(reinterpret_cast<char*>(header), sizeof header);
03690
03691     // ヘッダの読み込みに失敗したら戻る
03692     if (file.bad()) return false;
03693
03694     // 深度
03695     const auto depth{ header[16] / 8 };
03696     switch (depth)
03697     {
03698     case 1:
03699         *pFormat = GL_RED;
03700         break;

```

```

03701     case 2:
03702         *pFormat = GL_RG;
03703         break;
03704     case 3:
03705         *pFormat = GL_BGR;
03706         break;
03707     case 4:
03708         *pFormat = GL_BGRA;
03709         break;
03710     default:
03711         // 取り扱えないフォーマットだったら戻る
03712         return false;
03713     }
03714
03715     // 画像の縦横の画素数
03716     *pWidth = header[13] << 8 | header[12];
03717     *pHeight = header[15] << 8 | header[14];
03718
03719     // データサイズ
03720     const auto size{ *pWidth * *pHeight * depth };
03721
03722     // サイズが小さすぎたら戻る
03723     if (size < 2) return false;
03724
03725     // 読み込みに使うメモリを確保する
03726     image.resize(size);
03727
03728     // データを読み込む
03729     if (header[2] & 8)
03730     {
03731         // RLE
03732         int p{ 0 };
03733         char c;
03734         while (file.get(c))
03735         {
03736             if (c & 0x80)
03737             {
03738                 // run-length packet
03739                 const auto count{ (c & 0x7f) + 1 };
03740                 if (p + depth * count > size) break;
03741                 char temp[4];
03742                 file.read(temp, depth);
03743                 for (int i = 0; i < count; ++i)
03744                 {
03745                     for (int j = 0; j < depth; j++) image[p++] = temp[j++];
03746                 }
03747             }
03748             else
03749             {
03750                 // raw packet
03751                 const auto count{ (c + 1) * depth };
03752                 if (p + count > size) break;
03753                 file.read(reinterpret_cast<char*>(image.data() + p), count);
03754                 p += count;
03755             }
03756         }
03757     }
03758     else
03759     {
03760         // 非圧縮
03761         file.read(reinterpret_cast<char*>(image.data()), size);
03762     }
03763
03764     // 読み込みに失敗していなければ true を返す
03765     return file.bad() != false;
03766 }
03767
03768 //
03769 // テクスチャを作成して画像を読み込む
03770 //
03771 // image 画像データ, nullptr ならメモリの確保だけを行う
03772 // width 画像の横の画素数
03773 // height 画像の縦の画素数
03774 // format 画像データのフォーマット
03775 // type 画像のデータ型
03776 // internal テクスチャの内部フォーマット
03777 // wrap テクスチャのラッピングモード
03778 // swizzle true ならテクスチャの赤と青を入れ替える
03779 // 戻り値 テクスチャ名
03780 //
03781 GLuint gg::ggLoadTexture(
03782     const GLvoid* image,
03783     GLsizei width,
03784     GLsizei height,
03785     GLenum format,
03786     GLenum type,
03787     GLenum internal,

```

```

03788     GLenum wrap,
03789     bool swizzle
03790 )
03791 {
03792     // テクスチャを作成する
03793     GLuint texture;
03794     glGenTextures(1, &texture);
03795
03796     // 作成したテクスチャを 2D テクスチャとしてバインドする
03797     glBindTexture(GL_TEXTURE_2D, texture);
03798
03799     // アルファチャンネルがなければ 4 バイト境界に設定する
03800     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA || format == GL_BGRA ? 4 : 1);
03801
03802     // テクスチャを割り当てる
03803     glTexImage2D(GL_TEXTURE_2D, 0, internal, width, height, 0, format, type, image);
03804
03805     // バイリニア（ミップマップなし）、エッジでクランプ
03806     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
03807     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
03808     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrap);
03809     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrap);
03810
03811     if (swizzle)
03812     {
03813         // テクスチャのサンプリング時に赤と青を入れ替える
03814         glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, GL_BLUE);
03815         glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, GL_RED);
03816     }
03817
03818     // テクスチャ名を返す
03819     return texture;
03820 }
03821
03822 //
03823 // テクスチャを作成して TGA フォーマットの画像ファイルを読み込む
03824 //
03825 //     name TGA ファイル名
03826 //     pWidth 読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03827 //     pHeight 読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03828 //     internal テクスチャの内部フォーマット
03829 //     wrap テクスチャのラッピングモード
03830 //     戻り値 テクスチャ名
03831 //
03832 GLuint gg::ggLoadImage(
03833     const std::string& name,
03834     GLsizei* pWidth,
03835     GLsizei* pHeight,
03836     GLenum internal,
03837     GLenum wrap
03838 )
03839 {
03840     // 画像データ
03841     std::vector<GLubyte> image;
03842
03843     // 画像サイズ
03844     GLsizei width, height;
03845
03846     // 画像フォーマット
03847     GLenum format;
03848
03849     // 画像を読み込む
03850     ggReadImage(name, image, &width, &height, &format);
03851
03852     // 画像が読み込めなかったら戻る
03853     if (image.empty()) return 0;
03854
03855     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
03856     if (internal == 0) internal = format;
03857
03858     // テクスチャに読み込む (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
03859     const auto tex{ ggLoadTexture(image.data(), width, height,
03860         format, GL_UNSIGNED_BYTE, internal, wrap, false) };
03861
03862     // 画像サイズを返す
03863     if (pWidth) *pWidth = width;
03864     if (pHeight) *pHeight = height;
03865
03866     // テクスチャ名を返す
03867     return tex;
03868 }
03869
03870 //
03871 // グレースケール画像 (8bit) から法線マップのデータを作成する
03872 //
03873 //     width 高さマップのグレースケール画像 hmap の横の画素数
03874 //     height 高さマップのグレースケール画像のデータ hmap の縦の画素数

```

```

03875 // stride データの間隔
03876 // hmap グレースケール画像のデータ
03877 // nz 法線の z 成分の割合
03878 // internal テクスチャの内部フォーマット
03879 // nmap 法線マップを格納する vector
03880 //
03881 void gg::ggCreateNormalMap(
03882     const GLubyte* hmap,
03883     GLsizei width,
03884     GLsizei height,
03885     GLenum format,
03886     GLfloat nz,
03887     GLenum internal,
03888     std::vector<GgVector>& nmap
03889 )
03890 {
03891     // メモリサイズ
03892     const GLsizei size{ width * height };
03893
03894     // 法線マップのメモリを確保する
03895     nmap.resize(size);
03896
03897     // 画素のバイト数
03898     GLint stride;
03899     switch (format)
03900     {
03901     case GL_RED:
03902         stride = 1;
03903         break;
03904     case GL_RG:
03905         stride = 2;
03906         break;
03907     case GL_RGB:
03908         stride = 3;
03909         break;
03910     case GL_RGBA:
03911         stride = 4;
03912         break;
03913     default:
03914         stride = 1;
03915         break;
03916     }
03917
03918     // 法線マップの作成
03919     for (GLsizei i = 0; i < size; ++i)
03920     {
03921         const auto x{ i % width };
03922         const auto y{ i - x };
03923         const auto u0{ (y + (x - 1 + width) % width) * stride };
03924         const auto u1{ (y + (x + 1) % width) * stride };
03925         const auto v0{ ((y - width + size) % size + x) * stride };
03926         const auto v1{ ((y + width) % size + x) * stride };
03927
03928         // 隣接する画素との値の差を法線の成分に用いる
03929         nmap[i][0] = static_cast<GLfloat>(hmap[u0] - hmap[u1]);
03930         nmap[i][1] = static_cast<GLfloat>(hmap[v0] - hmap[v1]);
03931         nmap[i][2] = nz;
03932         nmap[i][3] = hmap[i * stride];
03933
03934         // 法線ベクトルを正規化する
03935         ggNormalize3(&nmap[i]);
03936     }
03937
03938     // 内部フォーマットが浮動小数点テクスチャでなければ [0,1] に正規化する
03939     if (
03940         internal != GL_RGB16F &&
03941         internal != GL_RGBA16F &&
03942         internal != GL_RGB32F &&
03943         internal != GL_RGBA32F
03944     )
03945     {
03946         for (GLsizei i = 0; i < size; ++i)
03947         {
03948             nmap[i][0] = nmap[i][0] * 0.5f + 0.5f;
03949             nmap[i][1] = nmap[i][1] * 0.5f + 0.5f;
03950             nmap[i][2] = nmap[i][2] * 0.5f + 0.5f;
03951             nmap[i][3] *= 0.0039215686f; // == 1/255
03952         }
03953     }
03954 }
03955
03956 //
03957 // TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する
03958 //
03959 // name TGA ファイル名
03960 // nz 作成した法線の z 成分の割合
03961 // pWidth 読み込んだ画像の横の画素数の格納先のポインタ (nullptr なら格納しない)

```

```

03962 //   pHeight 読み込んだ画像の縦の画素数の格納先のポインタ (nullptr なら格納しない)
03963 //   internal テクスチャの内部フォーマット
03964 //   戻り値 テクスチャ名
03965 //
03966 GLuint gg::ggLoadHeight(
03967     const std::string& name,
03968     GLfloat nz,
03969     GLsizei* pWidth,
03970     GLsizei* pHeight,
03971     GLenum internal
03972 )
03973 {
03974     // 画像データ
03975     std::vector<GLubyte> hmap;
03976
03977     // 画像サイズ
03978     GLsizei width, height;
03979
03980     // 画像フォーマット
03981     GLenum format;
03982
03983     // 高さマップの画像を読み込む
03984     ggReadImage(name, hmap, &width, &height, &format);
03985
03986     // 画像が読み込めなかったら戻る
03987     if (hmap.empty()) return 0;
03988
03989     // 法線マップ
03990     std::vector<GgVector> nmap;
03991
03992     // 法線マップを作成する
03993     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
03994
03995     // 画像サイズを返す
03996     if (pWidth) *pWidth = width;
03997     if (pHeight) *pHeight = height;
03998
03999     // テクスチャを作成して返す
04000     return ggLoadTexture(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal, GL_REPEAT);
04001 }
04002
04003 //
04004 // テクスチャの赤と青を交換する
04005 //
04006 void gg::GgTexture::swapRandB(bool swap) const
04007 {
04008     const auto swizzle
04009     {
04010         swap ?
04011             std::pair<GLint, GLint>{ GL_BLUE, GL_RED } :
04012             std::pair<GLint, GLint>{ GL_RED, GL_BLUE }
04013     };
04014
04015     glBindTexture(GL_TEXTURE_2D, texture);
04016     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, swizzle.first);
04017     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, swizzle.second);
04018     glBindTexture(GL_TEXTURE_2D, 0);
04019 }
04020
04021 //
04022 // TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する
04023 //
04024 //   name 読み込むファイル名
04025 //   internal glTexImage2D() に指定するテクスチャの内部フォーマット. 0 なら外部フォーマットに合わせる
04026 //   戻り値 テクスチャの作成に成功すれば true, 失敗すれば false
04027 //
04028 void gg::GgColorTexture::load(
04029     const std::string& name,
04030     GLenum internal,
04031     GLenum wrap
04032 )
04033 {
04034     // 画像データ
04035     std::vector<GLubyte> image;
04036
04037     // 画像サイズ
04038     GLsizei width, height;
04039
04040     // 画像フォーマット
04041     GLenum format;
04042
04043     // 画像を読み込む
04044     ggReadImage(name, image, &width, &height, &format);
04045
04046     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
04047     if (internal == 0) internal = format;
04048

```

```

04049 // テクスチャを作成する (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
04050 texture = std::make_shared<GgTexture>(image.data(), width, height,
04051     format, GL_UNSIGNED_BYTE, internal, wrap, false);
04052 }
04053
04054 //
04055 // TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する
04056 //
04057 //     name 画像ファイル名
04058 //     width テクスチャとして用いる画像データの横幅
04059 //     height テクスチャとして用いる画像データの高さ
04060 //     format テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)
04061 //     nz 法線マップの z 成分の値
04062 //     internal テクスチャの内部フォーマット
04063 //
04064 void gg::GgNormalTexture::load(
04065     const std::string& name,
04066     GLfloat nz,
04067     GLenum internal
04068 )
04069 {
04070     // 高さマップ
04071     std::vector<GLubyte> hmap;
04072
04073     // 画像サイズ
04074     GLsizei width, height;
04075
04076     // 画像フォーマット
04077     GLenum format;
04078
04079     // 高さマップの画像を読み込む
04080     ggReadImage(name, hmap, &width, &height, &format);
04081
04082     // 法線マップ
04083     std::vector<GgVector> nmap;
04084
04085     // 法線マップを作成する
04086     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04087 }
04088
04089
04090 //
04091 // OBJ ファイルの読み込みに使うデータ型と関数
04092 //
04093 //
04094 namespace gg
04095 {
04096     // GLfloat 型の 2 要素のベクトル
04097     using vec2 = std::array<GLfloat, 2>;
04098
04099     // GLfloat 型の 3 要素のベクトル
04100     using vec3 = std::array<GLfloat, 3>;
04101
04102     // 三角形データ
04103     struct fixx
04104     {
04105         GLuint p[3]; // 頂点座標番号
04106         GLuint n[3]; // 頂点法線番号
04107         GLuint t[3]; // テクスチャ座標番号
04108         bool smooth; // スムーズシェーディングの有無
04109     };
04110
04111     // ポリゴングループ
04112     struct fgrp
04113     {
04114         GLuint nextgroup; // 次のポリゴングループの最初の三角形番号
04115         GLuint mtlno; // このポリゴングループの材質番号
04116
04117         // コンストラクタ
04118         fgrp(GLuint nextgroup, GLuint mtlno) :
04119             nextgroup(nextgroup),
04120             mtlno(mtlno)
04121         {
04122         }
04123     };
04124
04125     // デフォルトの材質
04126     constexpr GgSimpleShader::Material defaultMaterial
04127     {
04128         { 0.1f, 0.1f, 0.1f, 1.0f },
04129         { 0.6f, 0.6f, 0.6f, 0.0f },
04130         { 0.3f, 0.3f, 0.3f, 0.0f },
04131         60.0f
04132     };
04133
04134     // デフォルトの材質名
04135     constexpr char defaultMaterialName[] = "_default.";
04136

```

```

04137 //
04138 // Alias OBJ 形式の MTL ファイルを読み込む
04139 //
04140 // mtlpath MTL ファイルのパス名
04141 // mtl 読み込んだ材質名をキー、材質番号を値にした map
04142 // material 材質データ
04143 //
04144 static bool ggLoadMtl(const std::string& mtlpath,
04145     std::map<std::string, GLuint>& mtl,
04146     std::vector<GgSimpleShader::Material>& material)
04147 {
04148     // MTL ファイルが無ければ戻る
04149     std::ifstream mtlfile{ Utf8ToTChar(mtlpath), std::ios::binary };
04150     if (!mtlfile)
04151     {
04152         #if defined(DEBUG)
04153             std::cerr << "Warning: Can't open MTL file: " << mtlpath << std::endl;
04154         #endif
04155         return false;
04156     }
04157
04158     // 一行読み込み用のバッファ
04159     std::string mtlline;
04160
04161     // 材質名（ループの外に置く）
04162     std::string mtlname{ defaultMaterialName };
04163
04164     // 現在の材質番号を登録する
04165     mtl[mtlname] = static_cast<GLuint>(material.size());
04166
04167     // 現在の材質にデフォルトの材質を設定する
04168     material.emplace_back(defaultMaterial);
04169
04170     // 材質データを読み込む
04171     while (std::getline(mtlfile, mtlline))
04172     {
04173         // 空行は読み飛ばす
04174         if (mtlline == "") continue;
04175
04176         // 最後の文字が '\r' なら
04177         if (*(mtlline.end() - 1) == '\r')
04178         {
04179             // 最後の文字を削除する
04180             mtlline.erase(mtlfile.end() - 1, mtlfile.end());
04181
04182             // 空行になったら読み飛ばす
04183             if (mtlline == "") continue;
04184         }
04185
04186         // 読み込んだ行を文字列ストリームにする
04187         std::istringstream mtlstr{ mtlline };
04188
04189         // オペレータ
04190         std::string mtlop;
04191
04192         // 文字列ストリームから材質パラメータの種類を取り出す
04193         mtlstr >> mtlop;
04194
04195         // '#' で始まる場合はコメントとして行末まで読み飛ばす
04196         if (mtlop[0] == '#') continue;
04197
04198         if (mtlop == "newmtl")
04199         {
04200             // 新規作成する材質名を取り出す
04201             mtlstr >> mtlname;
04202
04203             // 材質名が既に存在するかどうか調べる
04204             const auto m{ mtl.find(mtlname) };
04205             if (m == mtl.end())
04206             {
04207                 // 存在しないので新規作成する材質の番号をその材質名に割り当てる
04208                 mtl[mtlname] = static_cast<GLuint>(material.size());
04209
04210                 // 新規作成する材質にデフォルトの材質を設定しておく
04211                 material.emplace_back(defaultMaterial);
04212             }
04213
04214             #if defined(DEBUG)
04215                 std::cerr << "newmtl: " << mtlname << std::endl;
04216             #endif
04217         }
04218         else if (mtlop == "Ka")
04219         {
04220             // 環境光の反射係数を登録する
04221             mtlstr
04222                 >> material.back().ambient[0]
04223                 >> material.back().ambient[1]

```

```

04224         >> material.back().ambient[2];
04225     }
04226     else if (mtlop == "Kd")
04227     {
04228         // 拡散反射係数を登録する
04229         mtlstr
04230         >> material.back().diffuse[0]
04231         >> material.back().diffuse[1]
04232         >> material.back().diffuse[2];
04233     }
04234     else if (mtlop == "Ks")
04235     {
04236         // 鏡面反射係数を登録する
04237         mtlstr
04238         >> material.back().specular[0]
04239         >> material.back().specular[1]
04240         >> material.back().specular[2];
04241     }
04242     else if (mtlop == "Ns")
04243     {
04244         // 輝き係数を登録する
04245         GLfloat shininess;
04246         mtlstr >> shininess;
04247         material.back().shininess = shininess * 0.1f;
04248     }
04249     else if (mtlop == "d")
04250     {
04251         // 不透明度を登録する
04252         mtlstr >> material.back().ambient[3];
04253     }
04254 }
04255
04256 // MTL ファイルの読み込みに失敗したら戻る
04257 if (mtlfile.bad())
04258 {
04259     #if defined(DEBUG)
04260         std::cerr << "Warning: Can't read MTL file: " << mtlpath << std::endl;
04261     #endif
04262     return false;
04263 }
04264
04265 // MTL ファイルの読み込みに成功した
04266 return true;
04267 }
04268
04269 //
04270 // Alias OBJ 形式のファイルを解析する
04271 //
04272 // name Alias OBJ 形式のファイルのファイル名
04273 // group 同じ材質を割り当てるポリゴングループ
04274 // mtl 読み込んだ材質名をキーにした map
04275 // pos 頂点の位置
04276 // norm 頂点の法線
04277 // tex 頂点のテクスチャ座標
04278 // face 三角形のデータ
04279 //
04280 static bool ggParseObj(
04281     const std::string& name,
04282     std::vector<fgrp>& group,
04283     std::vector<GgSimpleShader::Material>& material,
04284     std::vector<vec3>& pos,
04285     std::vector<vec3>& norm,
04286     std::vector<vec2>& tex,
04287     std::vector<fidx>& face,
04288     bool normalize
04289 )
04290 {
04291     // ファイルパスからディレクトリ名を取り出す
04292     const std::string path{ name };
04293     const size_t base{ path.find_last_of("/\\") };
04294     const std::string dirname{ (base == std::string::npos) ? "" : path.substr(0, base + 1) };
04295
04296     // OBJ ファイルを読み込む
04297     std::ifstream file{ Utf8ToTChar(path) };
04298
04299     // 読み込みに失敗したら戻る
04300     if (file.fail())
04301     {
04302         #if defined(DEBUG)
04303             std::cerr << "Error: Can't open OBJ file: " << path << std::endl;
04304         #endif
04305         return false;
04306     }
04307
04308     // ポリゴングループの最初の三角形番号
04309     GLsizei startgroup{static_cast<GLsizei>(group.size())};
04310

```



```

04311 // スムーズシェーディングのスイッチ
04312 bool smooth{ false };
04313
04314 // 材質のテーブル
04315 std::map<std::string, GLuint> mtl;
04316
04317 // 現在の材質名 (ループの外で宣言する)
04318 std::string mtlname;
04319
04320 // 座標値の最小値・最大値
04321 vec3 bmin{ FLT_MAX }, bmax{ -FLT_MAX };
04322
04323 // 一行読み込み用のバッファ
04324 std::string line;
04325
04326 // データの読み込み
04327 while (std::getline(file, line))
04328 {
04329     // 空行は読み飛ばす
04330     if (line == "") continue;
04331
04332     // 最後の文字が '\r' なら
04333     if (*(line.end() - 1) == '\r')
04334     {
04335         // 最後の文字を削除する
04336         line.erase(line.end() - 1, line.end());
04337
04338         // 空行になったら読み飛ばす
04339         if (line == "") continue;
04340     }
04341
04342     // 一行を文字列ストリームに入れる
04343     std::istringstream str(line);
04344
04345     // 最初のトークンを命令 (op) とみなす
04346     std::string op;
04347     str >> op;
04348
04349     if (op[0] == '#') continue;
04350
04351     if (op == "v")
04352     {
04353         // 頂点位置
04354         vec3 v;
04355
04356         // 頂点位置はスペースで区切られている
04357         str >> v[0] >> v[1] >> v[2];
04358
04359         // 頂点位置を記録する
04360         pos.emplace_back(v);
04361
04362         // 頂点位置の最小値と最大値を求める (AABB)
04363         for (int i = 0; i < 3; ++i)
04364         {
04365             bmin[i] = std::min(bmin[i], v[i]);
04366             bmax[i] = std::max(bmax[i], v[i]);
04367         }
04368     }
04369     else if (op == "vt")
04370     {
04371         // テクスチャ座標
04372         vec2 t;
04373
04374         // 頂点位置はスペースで区切られている
04375         str >> t[0] >> t[1];
04376
04377         // テクスチャ座標を記録する
04378         tex.emplace_back(t);
04379     }
04380     else if (op == "vn")
04381     {
04382         // 頂点法線
04383         vec3 n;
04384
04385         // 頂点法線はスペースで区切られている
04386         str >> n[0] >> n[1] >> n[2];
04387
04388         // 頂点法線を記録する
04389         norm.emplace_back(n);
04390     }
04391     else if (op == "f")
04392     {
04393         // 三角形データ
04394         fidx f;
04395
04396         // スムーズシェーディング
04397         f.smooth = smooth;

```

```

04398
04399 // 三頂点のそれぞれについて
04400 for (int i = 0; i < 3; ++i)
04401 {
04402     // 1項目取り出す
04403     std::string s;
04404     str >> s;
04405
04406     // 文字の位置
04407     auto c{ s.begin() };
04408
04409     // テクスチャ座標と法線の番号は未定義を表す 0 にしておく
04410     f.p[i] = f.t[i] = f.n[i] = 0;
04411
04412     // 項目の最初の要素は頂点座標番号
04413     while (c != s.end() && isdigit(*c)) f.p[i] = f.p[i] * 10 + *c++ - '0';
04414     if (c == s.end() || *c++ != '/') continue;
04415
04416     // 二つ目の項目はテクスチャ座標
04417     while (c != s.end() && isdigit(*c)) f.t[i] = f.t[i] * 10 + *c++ - '0';
04418     if (c == s.end() || *c++ != '/') continue;
04419
04420     // 三つ目の項目は法線番号
04421     while (c != s.end() && isdigit(*c)) f.n[i] = f.n[i] * 10 + *c++ - '0';
04422 }
04423
04424 // 三角形データを登録する
04425 face.emplace_back(f);
04426 }
04427 else if (op == "s")
04428 {
04429     // '1' だったらスムーズシェーディング有効
04430     std::string s;
04431     str >> s;
04432     smooth = s == "1";
04433 }
04434 else if (op == "usemtl")
04435 {
04436     // 次のポリゴングループの最初の三角形番号
04437     const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04438
04439     // ポリゴングループに三角形が存在すれば
04440     if (nextgroup > startgroup)
04441     {
04442         // ポリゴングループの三角形数と材質番号を記録する
04443         group.emplace_back(nextgroup, mtl[mtlname]);
04444
04445         // 次のポリゴングループの開始番号を保存しておく
04446         startgroup = nextgroup;
04447     }
04448
04449     // 次に usemtl が来るまで材質名を保持する
04450     str >> mtlname;
04451
04452     // 材質の存在チェック
04453     if (mtl.find(mtlname) == mtl.end())
04454     {
04455 #if defined(DEBUG)
04456         std::cerr << "Warning: Undefined material: " << mtlname << std::endl;
04457 #endif
04458
04459         // デフォルトの材質を割り当てておく
04460         mtlname = defaultMaterialName;
04461     }
04462 #if defined(DEBUG)
04463     else std::cerr << "usemtl: " << mtlname << std::endl;
04464 #endif
04465 }
04466 else if (op == "mtllib")
04467 {
04468     // MTL ファイルのパス名を作る
04469     str >> std::ws;
04470     std::string mtlpath;
04471     std::getline(str, mtlpath);
04472
04473     // MTL ファイルを読み込む
04474     ggLoadMtl(dirname + mtlpath, mtl, material);
04475 }
04476 }
04477
04478 // OBJ ファイルの読み込みに失敗したら戻る
04479 if (file.bad())
04480 {
04481 #if defined(DEBUG)
04482     std::cerr << "Error: Can't read OBJ file: " << path << std::endl;
04483 #endif
04484     return false;

```

```

04485     }
04486
04487     // 最後のポリゴングループの次の三角形番号
04488     const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04489     if (nextgroup > startgroup)
04490     {
04491         // 最後のポリゴングループの三角形数と材質を記録する
04492         group.emplace_back(nextgroup, static_cast<GLuint>(mtl[mtlname]));
04493     }
04494
04495     // スムーズシェーディングしない三角形の頂点を追加する
04496     for (auto& f : face)
04497     {
04498         if (!f.smooth)
04499         {
04500             // 三頂点のそれぞれについて
04501             for (int i = 0; i < 3; ++i)
04502             {
04503                 // 新しい頂点座標を生成する (std::array の要素は emplace_back できない)
04504                 pos.emplace_back(pos[f.p[i] - 1]);
04505                 f.p[i] = static_cast<GLuint>(pos.size());
04506
04507                 if (f.t[i] > 0)
04508                 {
04509                     // 新しいテクスチャ座標を生成する
04510                     tex.emplace_back(tex[f.t[i] - 1]);
04511                     f.t[i] = static_cast<GLuint>(tex.size());
04512                 }
04513
04514                 if (f.n[i] > 0)
04515                 {
04516                     // 新しい法線を生成する
04517                     norm.emplace_back(norm[f.n[i] - 1]);
04518                     f.n[i] = static_cast<GLuint>(norm.size());
04519                 }
04520             }
04521         }
04522     }
04523
04524     // 法線データがなければ算出しておく
04525     if (norm.empty())
04526     {
04527         // 法線データ数の初期値は頂点数と同じでスムーズシェーディングのために初期値は 0
04528         norm.resize(pos.size(), { 0.0f, 0.0f, 0.0f });
04529
04530         // 面の法線の算出と頂点法線の算出
04531         for (auto& f : face)
04532         {
04533             // 頂点座標番号
04534             const auto v0{ f.p[0] - 1 };
04535             const auto v1{ f.p[1] - 1 };
04536             const auto v2{ f.p[2] - 1 };
04537
04538             // v1 - v0, v2 - v0 を求める
04539             const GLfloat d1[]{ pos[v1][0] - pos[v0][0], pos[v1][1] - pos[v0][1], pos[v1][2] - pos[v0][2] };
04540             const GLfloat d2[]{ pos[v2][0] - pos[v0][0], pos[v2][1] - pos[v0][1], pos[v2][2] - pos[v0][2] };
04541
04542             // 外積により面法線を求める
04543             vec3 n;
04544             ggCross(n.data(), d1, d2);
04545
04546             if (f.smooth)
04547             {
04548                 // スムーズシェーディングを行うときは
04549                 for (int i = 0; i < 3; ++i)
04550                 {
04551                     // 面法線を頂点法線に積算する
04552                     norm[v0][i] += n[i];
04553                     norm[v1][i] += n[i];
04554                     norm[v2][i] += n[i];
04555
04556                     // 面の各頂点の法線番号は頂点番号と同じにする
04557                     f.n[i] = f.p[i];
04558                 }
04559             }
04560             else
04561             {
04562                 // 面法線を最初の頂点に保存する
04563                 norm[v0] = n;
04564                 f.n[0] = f.p[0];
04565
04566                 // 2 頂点追加
04567                 for (int i = 1; i < 3; ++i)
04568                 {
04569                     norm.emplace_back(n);

```

```

04570         f.n[i] = static_cast<GLuint>(norm.size());
04571     }
04572 }
04573 }
04574
04575 // 頂点の法線ベクトルを正規化する
04576 for (auto& n : norm) ggNormalize3(n.data());
04577 }
04578
04579 // 図形の正規化
04580 if (normalize)
04581 {
04582     // 図形の大きさ
04583     const auto sx{ bmax[0] - bmin[0] };
04584     const auto sy{ bmax[1] - bmin[1] };
04585     const auto sz{ bmax[2] - bmin[2] };
04586
04587     // 図形のスケール
04588     GLfloat s{ sx };
04589     if (sy > s) s = sy;
04590     if (sz > s) s = sz;
04591     const auto scale{ s != 0.0f ? 2.0f / s : 1.0f };
04592
04593     // 図形の中心位置
04594     const auto cx{ (bmax[0] + bmin[0]) * 0.5f };
04595     const auto cy{ (bmax[1] + bmin[1]) * 0.5f };
04596     const auto cz{ (bmax[2] + bmin[2]) * 0.5f };
04597
04598     // 図形の大きさと位置を正規化する
04599     for (auto& p : pos)
04600     {
04601         p[0] = (p[0] - cx) * scale;
04602         p[1] = (p[1] - cy) * scale;
04603         p[2] = (p[2] - cz) * scale;
04604     }
04605 }
04606
04607 #if defined(DEBUG)
04608     std::cerr
04609     << "[" << name << "]\n(Parsed) Group: " << group.size() << ", Material: " << mtl.size()
04610     << ", Pos: " << pos.size() << ", Norm: " << norm.size() << ", Tex: " << tex.size()
04611     << ", Face: " << face.size() << "\n";
04612 #endif
04613
04614 // OBJ ファイルの読み込み成功
04615 return true;
04616 }
04617 }
04618
04619 //
04620 //
04621 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Arrays 形式)
04622 //
04623 // name 読み込むOBJ ファイル名
04624 // group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04625 // material 読み込んだデータのポリゴングループごとの材質
04626 // vert 読み込んだデータの頂点属性
04627 // normalize true ならサイズを正規化する
04628 // 戻り値 読み込みに成功したら true
04629 //
04630 bool gg::ggLoadSimpleObj(const std::string& name,
04631     std::vector<std::array<GLuint, 3>>& group,
04632     std::vector<GgSimpleShader::Material>& material,
04633     std::vector<GgVertex>& vert,
04634     bool normalize)
04635 {
04636     // 読み込み用の一時記憶領域
04637     std::vector<fgrp> tgroup;
04638     std::vector<vec3> tpos;
04639     std::vector<vec3> tnorm;
04640     std::vector<vec2> ttex;
04641     std::vector<fidx> tface;
04642
04643     // OBJ ファイルを解析する
04644     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04645
04646     // 頂点属性データのメモリを確保する
04647     vert.reserve(vert.size() + tface.size() * 3);
04648
04649     // ポリゴングループデータのメモリを確保する
04650     group.reserve(group.size() + tgroup.size());
04651     material.reserve(material.size() + tgroup.size());
04652
04653     // ポリゴングループの最初の三角形番号
04654     GLuint startgroup{ 0 };
04655
04656     // ポリゴングループデータの作成
04657     for (auto& g : tgroup)

```

```

04658 {
04659     // このポリゴングループの最初の頂点番号と頂点数・材質番号
04660     std::array<GLuint, 3> v;
04661
04662     // このポリゴングループの最初の頂点番号を保存する
04663     v[0] = static_cast<GLuint>(vert.size());
04664
04665     // 三角形ごとの頂点データの作成
04666     for (GLuint j = startgroup; j < g.nextgroup; ++j)
04667     {
04668         // 処理対象の三角形
04669         auto& f = tface[j];
04670
04671         // 三頂点のそれぞれについて
04672         for (int i = 0; i < 3; ++i)
04673         {
04674             // テクスチャ座標
04675             vec2 tex{ 0.0f, 0.0f };
04676             if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04677
04678             // 頂点法線
04679             vec3 norm{ 0.0f, 0.0f, 0.0f };
04680             if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04681
04682             // 頂点属性の追加
04683             vert.emplace_back(tpos[f.p[i] - 1].data(), norm.data());
04684         }
04685     }
04686
04687     // このポリゴングループの頂点数を保存する
04688     v[1] = static_cast<GLuint>(vert.size()) - v[0];
04689     v[2] = g.mtlno;
04690
04691     // このポリゴングループの最初の頂点番号と頂点数・材質番号を登録する
04692     group.emplace_back(v);
04693
04694     // 次のポリゴングループの最初の三角形番号を求める
04695     startgroup = g.nextgroup;
04696 }
04697
04698 #if defined(DEBUG)
04699     std::cerr
04700     << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04701     << ", Vertex: " << vert.size() << "\n";
04702 #endif
04703
04704 // OBJ ファイルの読み込み成功
04705 return true;
04706 }
04707
04708 //
04709 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Elements 形式)
04710 //
04711 //     name 読み込むOBJ ファイル名
04712 //     group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04713 //     material 読み込んだデータのポリゴングループごとの材質
04714 //     vert 読み込んだデータの頂点属性
04715 //     face 読み込んだデータの三角形の頂点インデックス
04716 //     normalize true ならサイズを正規化する
04717 //     戻り値 読み込みに成功したら true
04718 //
04719 bool gg::ggLoadSimpleObj(const std::string& name,
04720     std::vector<std::array<GLuint, 3>>& group,
04721     std::vector<GgSimpleShader::Material>& material,
04722     std::vector<GgVertex>& vert,
04723     std::vector<GLuint>& face,
04724     bool normalize)
04725 {
04726     // 読み込み用の一時記憶領域
04727     std::vector<fgrp> tgroup;
04728     std::vector<vec3> tpos;
04729     std::vector<vec3> tnorm;
04730     std::vector<vec2> ttex;
04731     std::vector<fidx> tface;
04732
04733     // OBJ ファイルを解析する
04734     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04735
04736     // 頂点属性データの最初の頂点番号
04737     const auto vertbase{ static_cast<GLuint>(vert.size()) };
04738
04739     // 頂点属性データのメモリを確保する
04740     vert.resize(vertbase + tpos.size());
04741
04742     // 三角形データのメモリを確保する
04743     face.reserve(face.size() + tface.size());
04744

```

```

04745 // ポリゴングループデータのメモリを確保する
04746 group.reserve(group.size() + tgroup.size());
04747 material.reserve(material.size() + tgroup.size());
04748
04749 // ポリゴングループの最初の三角形番号
04750 GLuint startgroup{ 0 };
04751
04752 // ポリゴングループデータの作成
04753 for (auto& g : tgroup)
04754 {
04755     // このポリゴングループの最初の頂点番号
04756     const auto first{ static_cast<GLuint>(face.size()) };
04757
04758     // 三角形ごとの頂点データの作成
04759     for (GLuint j = startgroup; j < g.nextgroup; ++j)
04760     {
04761         // 処理対象の三角形
04762         auto& f{ tface[j] };
04763
04764         // 三頂点のそれぞれについて
04765         for (int i = 0; i < 3; ++i)
04766         {
04767             // 追加する三角形データの頂点番号
04768             const auto q{ f.p[i] - 1 + vertbase };
04769
04770             // 三角形データの追加
04771             face.emplace_back(q);
04772
04773             // テクスチャ座標番号
04774             vec2 tex{ 0.0f, 0.0f };
04775             if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04776
04777             // 頂点法線番号
04778             vec3 norm{ 0.0f, 0.0f, 0.0f };
04779             if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04780
04781             // 頂点の格納
04782             vert[q] = GgVertex(tpos[f.p[i] - 1].data(), norm.data());
04783         }
04784     }
04785
04786     // このポリゴングループの最初の三角形番号と三角形数・材質番号を登録する
04787     group.emplace_back(std::array<GLuint, 3>{ first, static_cast<GLuint>(face.size()) - first, g.mtlno
04788 });
04789
04790 // 次のポリゴングループの最初の三角形番号を求める
04791 startgroup = g.nextgroup;
04792 }
04793
04794 #if defined(DEBUG)
04795     std::cerr
04796     << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04797     << ", Vertex: " << vert.size() << ", Face: " << face.size() << "\n";
04798 #endif
04799
04800 // OBJ ファイルの読み込み成功
04801 return true;
04802 }
04803
04804 // シェーダオブジェクトのコンパイル結果を表示する
04805 //
04806 static GLboolean printShaderInfoLog(GLuint shader, const std::string& str)
04807 {
04808     // コンパイル結果を取得する
04809     GLint status;
04810     glGetShaderiv(shader, GL_COMPILE_STATUS, &status);
04811     #if defined(DEBUG)
04812     if (status == GL_FALSE) std::cerr << "Compile Error in " << str << std::endl;
04813     #endif
04814
04815     // シェーダのコンパイル時のログの長さを取得する
04816     GLsizei bufSize;
04817     glGetShaderiv(shader, GL_INFO_LOG_LENGTH, &bufSize);
04818
04819     if (bufSize > 1)
04820     {
04821         // シェーダのコンパイル時のログの内容を取得する
04822         std::vector<GLchar> infoLog(bufSize);
04823         GLsizei length;
04824         glGetShaderInfoLog(shader, bufSize, &length, infoLog.data());
04825         #if defined(DEBUG)
04826         std::cerr << infoLog.data();
04827         #endif
04828     }
04829
04830     // コンパイル結果を返す

```

```

04831     return static_cast<GLboolean>(status);
04832 }
04833
04834 //
04835 // プログラムオブジェクトのリンク結果を表示する
04836 //
04837 static GLboolean printProgramInfoLog(GLuint program)
04838 {
04839     // リンク結果を取得する
04840     GLint status;
04841     glGetProgramiv(program, GL_LINK_STATUS, &status);
04842     #if defined(DEBUG)
04843     if (status == GL_FALSE) std::cerr << "Link Error." << std::endl;
04844     #endif
04845
04846     // シェーダのリンク時のログの長さを取得する
04847     GLsizei bufSize;
04848     glGetProgramiv(program, GL_INFO_LOG_LENGTH, &bufSize);
04849
04850     // シェーダのリンク時のログの内容を取得する
04851     if (bufSize > 1)
04852     {
04853         std::vector<GLchar> infoLog(bufSize);
04854         GLsizei length;
04855         glGetProgramInfoLog(program, bufSize, &length, infoLog.data());
04856         #if defined(DEBUG)
04857         std::cerr << infoLog.data() << std::endl;
04858         #endif
04859     }
04860
04861     // リンク結果を返す
04862     return static_cast<GLboolean>(status);
04863 }
04864
04865 //
04866 // シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
04867 //
04868 // vsrc バーテックスシェーダのソースプログラムの文字列
04869 // fsrc フラグメントシェーダのソースプログラムの文字列 (nullptr なら不使用)
04870 // gsrc ジオメトリシェーダのソースプログラムの文字列 (nullptr なら不使用)
04871 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
04872 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
04873 // vtext バーテックスシェーダのコンパイル時のメッセージに追加する文字列
04874 // ftext フラグメントシェーダのコンパイル時のメッセージに追加する文字列
04875 // gtext ジオメトリシェーダのコンパイル時のメッセージに追加する文字列
04876 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
04877 //
04878 GLuint gg::ggCreateShader(
04879     const std::string& vsrc,
04880     const std::string& fsrc,
04881     const std::string& gsrc,
04882     GLint nvarying,
04883     const char* const* varyings,
04884     const std::string& vtext,
04885     const std::string& ftext,
04886     const std::string& gtext)
04887 {
04888     // シェーダプログラムの作成
04889     const auto program{ glCreateProgram() };
04890
04891     if (program > 0)
04892     {
04893         bool status = true;
04894
04895         if (!vsrc.empty())
04896         {
04897             // バーテックスシェーダのシェーダオブジェクトを作成する
04898             const auto vertShader{ glCreateShader(GL_VERTEX_SHADER) };
04899             const auto* vsrctp{ vsrc.c_str() };
04900             glShaderSource(vertShader, 1, &vsrctp, nullptr);
04901             glCompileShader(vertShader);
04902
04903             // バーテックスシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04904             if (printShaderInfoLog(vertShader, vtext))
04905                 glAttachShader(program, vertShader);
04906             else
04907                 status = false;
04908             glDeleteShader(vertShader);
04909         }
04910
04911         if (!fsrc.empty())
04912         {
04913             // フラグメントシェーダのシェーダオブジェクトを作成する
04914             const auto fragShader{ glCreateShader(GL_FRAGMENT_SHADER) };
04915             const auto* fsrctp{ fsrc.c_str() };
04916             glShaderSource(fragShader, 1, &fsrctp, nullptr);
04917             glCompileShader(fragShader);

```

```

04918
04919 // フラグメントシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04920 if (printShaderInfoLog(fragShader, ftext))
04921     glAttachShader(program, fragShader);
04922 else
04923     status = false;
04924 glDeleteShader(fragShader);
04925 }
04926
04927 if (!gsrc.empty())
04928 {
04929 #if defined(GL_GLES_PROTOTYPES)
04930 # if defined(DEBUG)
04931     std::cerr << gtext << ": The geometry shader is not supported." << std::endl;
04932     status = false;
04933 # endif
04934 #else
04935     // ジオメトリシェーダのシェーダオブジェクトを作成する
04936     const auto geomShader{ glCreateShader(GL_GEOMETRY_SHADER) };
04937     const auto* gsrcp{ gsrc.c_str() };
04938     glShaderSource(geomShader, 1, &gsrcp, nullptr);
04939     glCompileShader(geomShader);
04940
04941     // ジオメトリシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04942     if (printShaderInfoLog(geomShader, gtext))
04943         glAttachShader(program, geomShader);
04944     else
04945         status = false;
04946     glDeleteShader(geomShader);
04947 #endif
04948 }
04949
04950 // 全てのシェーダオブジェクトのコンパイルに成功したら
04951 if (status)
04952 {
04953     // feedback に使う varying 変数を指定する
04954     if (nvarying > 0)
04955         glTransformFeedbackVaryings(program, nvarying, varyings, GL_SEPARATE_ATTRIBS);
04956
04957     // シェーダプログラムをリンクする
04958     glLinkProgram(program);
04959
04960     // リンクに成功したらプログラムオブジェクト名を返す
04961     if (printProgramInfoLog(program) != GL_FALSE) return program;
04962 }
04963 }
04964
04965 // プログラムオブジェクトが作成できなかった
04966 glDeleteProgram(program);
04967 return 0;
04968 }
04969
04970 //
04971 // シェーダのソースファイルを読み込んだ vector を返す
04972 //
04973 // name ソースファイル名
04974 // src 読み込んだソースファイルの文字列
04975 // 戻り値 読み込みの成功すれば true, 失敗したら false
04976 //
04977 static bool readShaderSource(const std::string& name, std::string& src)
04978 {
04979     // ファイル名が nullptr ならそのまま返る
04980     if (name.empty()) return true;
04981
04982     // ソースファイルを開く
04983     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
04984     if (file.fail())
04985     {
04986         // ファイルが開けなければエラーで戻る
04987 #if defined(DEBUG)
04988         std::cerr << "Error: Can't open source file: " << name << std::endl;
04989 #endif
04990         return false;
04991     }
04992
04993     // ファイル全体を文字列として読み込む
04994     std::istreambuf_iterator<char> it{ file };
04995     std::istreambuf_iterator<char> last;
04996     src = std::string(it, last);
04997
04998     // ファイルがうまく読み込めなければ戻る
04999     if (file.bad())
05000     {
05001 #if defined(DEBUG)
05002         std::cerr << "Error: Could not read source file: " << name << std::endl;
05003 #endif
05004         return false;
05005     }

```



```

05005     }
05006
05007     // ファイルを閉じて戻る
05008     return true;
05009 }
05010
05011 //
05012 // シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05013 //
05014 // vert バーテックスシェーダのソースファイル名
05015 // frag フラグメントシェーダのソースファイル名 (nullptr なら不使用)
05016 // geom ジオメトリシェーダのソースファイル名 (nullptr なら不使用)
05017 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
05018 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
05019 // 戻り値 シェーダプログラムのプログラム名 (作成できなければ 0)
05020 //
05021 GLuint gg::ggLoadShader(
05022     const std::string& vert,
05023     const std::string& frag,
05024     const std::string& geom,
05025     GLint nvarying,
05026     const char* const* varyings
05027 )
05028 {
05029     // 読み込んだシェーダのソースプログラム
05030     std::string vsrc, fsrc, gsrc;
05031
05032     // 読み込んだ結果
05033     bool status;
05034
05035     // シェーダのソースファイルを読み込む
05036     status = readShaderSource(vert, vsrc);
05037     status = readShaderSource(frag, fsrc) && status;
05038     status = readShaderSource(geom, gsrc) && status;
05039
05040     // 全てのソースファイルが読み込めていなかったらエラー
05041     if (!status) return 0;
05042
05043     // プログラムオブジェクトを作成する
05044     return ggCreateShader(vsrc, fsrc, gsrc, nvarying, varyings, vert, frag, geom);
05045 }
05046
05047 #if !defined(__APPLE__) && !defined(GL_GLES_PROTOTYPES)
05048 //
05049 // コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
05050 //
05051 // csrc コンピュートシェーダのソースプログラムの文字列
05052 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05053 //
05054 GLuint gg::ggCreateComputeShader(const std::string& csrc, const std::string& ctext)
05055 {
05056     // シェーダプログラムの作成
05057     const auto program{ glCreateProgram() };
05058
05059     if (program > 0)
05060     {
05061         // ソースプログラムの文字列が空だったら 0 を返す
05062         if (csrc.empty()) return 0;
05063
05064         // コンピュートシェーダのシェーダオブジェクトを作成する
05065         const auto compShader{ glCreateShader(GL_COMPUTE_SHADER) };
05066         const auto* csrccp{ csrc.c_str() };
05067         glShaderSource(compShader, 1, &csrccp, nullptr);
05068         glCompileShader(compShader);
05069
05070         // コンピュートシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05071         if (printShaderInfoLog(compShader, ctext)) glAttachShader(program, compShader);
05072         glDeleteShader(compShader);
05073
05074         // シェーダプログラムをリンクする
05075         glLinkProgram(program);
05076
05077         // プログラムオブジェクトが作成できなければ 0 を返す
05078         if (printProgramInfoLog(program) == GL_FALSE)
05079         {
05080             glDeleteProgram(program);
05081             return 0;
05082         }
05083     }
05084
05085     // プログラムオブジェクトを返す
05086     return program;
05087 }
05088
05089 //
05090 // コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05091 //

```

```

05092 //   comp コンピュートシェーダのソースファイル名
05093 //   戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05094 //
05095 GLuint gg::ggLoadComputeShader(const std::string& comp)
05096 {
05097     // シェーダのソースファイルを読み込む
05098     std::string csrc;
05099     if (readShaderSource(comp, csrc))
05100     {
05101         // プログラムオブジェクトを作成する
05102         return ggCreateComputeShader(csrc.data(), comp);
05103     }
05104     // プログラムオブジェクト作成失敗
05105     return 0;
05106 }
05107 #endif
05108
05109 //
05110 // 点: データ作成
05111 //
05112 void gg::GgPoints::load(const GgVector* pos, GLsizei count, GLenum usage)
05113 {
05114     // 頂点バッファオブジェクトを作成する
05115     position = std::make_shared<GgBuffer<GgVector>>(GL_ARRAY_BUFFER, pos,
05116         static_cast<GLsizei>(sizeof(GgVector)), count, usage);
05117     // このバッファオブジェクトは index == 0 の in 変数から入力する
05118     glVertexAttribPointer(0, static_cast<GLint>(pos->size()), GL_FLOAT, GL_FALSE, 0, 0);
05119     glEnableVertexAttribArray(0);
05120 }
05121 //
05122 // 点: 描画
05123 //
05124 void gg::GgPoints::draw(GLint first, GLsizei count) const
05125 {
05126     // 頂点配列オブジェクトを指定する
05127     GgShape::draw(first, count);
05128     // 図形を描画する
05129     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05130 }
05131 //
05132 // 三角形: データ作成
05133 //
05134 void gg::GgTriangles::load(const GgVertex* vert, GLsizei count, GLenum usage)
05135 {
05136     // 頂点バッファオブジェクトを作成する
05137     vertex = std::make_shared<GgBuffer<GgVertex>>(GL_ARRAY_BUFFER, vert,
05138         static_cast<GLsizei>(sizeof(GgVertex)), count, usage);
05139     // 頂点の位置は index == 0 の in 変数から入力する
05140     glVertexAttribPointer(0, static_cast<GLint>(vert->position.size()), GL_FLOAT, GL_FALSE,
05141         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, position));
05142     glEnableVertexAttribArray(0);
05143     // 頂点の法線は index == 1 の in 変数から入力する
05144     glVertexAttribPointer(1, static_cast<GLint>(vert->normal.size()), GL_FLOAT, GL_FALSE,
05145         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, normal));
05146     glEnableVertexAttribArray(1);
05147 }
05148 //
05149 // 三角形: 描画
05150 //
05151 void gg::GgTriangles::draw(GLint first, GLsizei count) const
05152 {
05153     // 頂点配列オブジェクトを指定する
05154     GgShape::draw(first, count);
05155     // 図形を描画する
05156     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05157 }
05158 //
05159 // オブジェクト: 描画
05160 //
05161 void gg::GgElements::draw(GLint first, GLsizei count) const
05162 {
05163     // 頂点配列オブジェクトを指定する
05164     GgShape::draw(first, count);
05165     // 図形を描画する
05166     glDrawElements(getMode(), count > 0 ? count : getIndexCount() - first,
05167         GL_UNSIGNED_INT, static_cast<GLuint*>(0) + first);
05168 }

```

```

05177 }
05178
05179 //
05180 // 点群を立方体状に生成する
05181 //
05182 std::shared_ptr<gg::GgPoints> gg::ggPointsCube(GLsizei count, GLfloat length, GLfloat cx, GLfloat cy,
GLfloat cz)
05183 {
05184     // メモリを確保する
05185     std::vector<GgVector> pos(count);
05186
05187     // 点を生成する
05188     for (GLsizei v = 0; v < count; ++v)
05189     {
05190         const GgVector p
05191         {
05192             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cx,
05193             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cy,
05194             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cz,
05195             1.0f
05196         };
05197
05198         pos.emplace_back(p);
05199     }
05200
05201     // 点データの GgPoints オブジェクトを作成して返す
05202     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05203 }
05204
05205 //
05206 // 点群を球状に生成する
05207 //
05208 std::shared_ptr<gg::GgPoints> gg::ggPointsSphere(GLsizei count, GLfloat radius,
GLfloat cx, GLfloat cy, GLfloat cz)
05209 {
05210     // メモリを確保する
05211     std::vector<GgVector> pos(count);
05212
05213     // 点を生成する
05214     for (GLsizei v = 0; v < count; ++v)
05215     {
05216         const auto r{ radius * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) };
05217         const auto t{ 6.28318530718f * static_cast<GLfloat>(rand()) / (static_cast<GLfloat>(RAND_MAX) +
1.0f) };
05218         const auto cp{ 2.0f * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 1.0f };
05219         const auto sp{ sqrtf(1.0f - cp * cp) };
05220         const auto ct{ cosf(t) };
05221         const auto st{ sinf(t) };
05222         const GgVector p{ r * sp * ct + cx, r * sp * st + cy, r * cp + cz, 1.0f };
05223
05224         pos.emplace_back(p);
05225     }
05226
05227     // 点データの GgPoints オブジェクトを作成して返す
05228     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05229 }
05230
05231 //
05232 // 矩形状に 2 枚の三角形を生成する
05233 //
05234 //
05235 std::shared_ptr<gg::GgTriangles> gg::ggRectangle(GLfloat width, GLfloat height)
05236 {
05237     // 頂点属性
05238     std::array<gg::GgVertex, 4> vert;
05239
05240     // 頂点位置と法線を求める
05241     for (int v = 0; v < 4; ++v)
05242     {
05243         const auto x{ ((v & 1) * 2 - 1) * width };
05244         const auto y{ ((v & 2) - 1) * height };
05245
05246         vert[v] = gg::GgVertex{ x, y, 0.0f, 0.0f, 0.0f, 1.0f };
05247     }
05248
05249     // 矩形の GgTriangles オブジェクトを作成する
05250     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_STRIP);
05251 }
05252
05253 //
05254 // 楕円状に三角形を生成する
05255 //
05256 std::shared_ptr<gg::GgTriangles> gg::ggEllipse(GLfloat width, GLfloat height, GLuint slices)
05257 {
05258     // 楕円のスケール
05259     constexpr GLfloat scale{ 0.5f };
05260

```

```

05261 // 作業用のメモリ
05262 std::vector<gg::GgVertex> vert;
05263
05264 // 頂点位置と法線を求める
05265 for (GLuint v = 0; v < slices; ++v)
05266 {
05267     const auto t{ 6.28318530717f * static_cast<GLfloat>(v) / static_cast<GLfloat>(slices) };
05268     const auto x{ cosf(t) * width * scale };
05269     const auto y{ sinf(t) * height * scale };
05270
05271     vert.emplace_back(x, y, 0.0f, 0.0f, 0.0f, 1.0f);
05272 }
05273
05274 // GgTriangles オブジェクトを作成する
05275 return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05276 }
05277
05278 //
05279 // Wavefront OBJ ファイルを読み込む (Arrays 形式)
05280 //
05281 std::shared_ptr<gg::GgTriangles> gg::ggArraysObj(const std::string& name, bool normalize)
05282 {
05283     std::vector<std::array<GLuint, 3>> group;
05284     std::vector<gg::GgSimpleShader::Material> material;
05285     std::vector<gg::GgVertex> vert;
05286
05287     // ファイルを読み込む
05288     if (!ggLoadSimpleObj(name, group, material, vert, normalize)) return nullptr;
05289
05290     // GgTriangles オブジェクトを作成する
05291     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05292 }
05293
05294 //
05295 // Wavefront OBJ ファイルを読み込む (Elements 形式)
05296 //
05297 std::shared_ptr<gg::GgElements> gg::ggElementsObj(const std::string& name, bool normalize)
05298 {
05299     std::vector<std::array<GLuint, 3>> group;
05300     std::vector<gg::GgSimpleShader::Material> material;
05301     std::vector<gg::GgVertex> vert;
05302     std::vector<GLuint> face;
05303
05304     // ファイルを読み込む
05305     if (!ggLoadSimpleObj(name, group, material, vert, face, normalize)) return nullptr;
05306
05307     // GgElements オブジェクトを作成する
05308     return std::make_shared<gg::GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05309 }
05310
05311 //
05312 // マッシュ形状を作成する (Elements 形式)
05313 //
05314 std::shared_ptr<gg::GgElements> gg::ggElementsMesh(GLuint slices, GLuint stacks, const
GLfloat(*pos)[3], const GLfloat(*norm)[3])
05315 {
05316     // 頂点属性
05317     std::vector<gg::GgVertex> vert((slices + 1) * (stacks + 1));
05318
05319     // 頂点の法線を求める
05320     for (GLuint j = 0; j <= stacks; ++j)
05321     {
05322         for (GLuint i = 0; i <= slices; ++i)
05323         {
05324             // 処理対象の頂点番号
05325             const auto k{ j * (slices + 1) + i };
05326
05327             // 頂点の法線
05328             gg::GgVector tnorm;
05329             tnorm[3] = 0.0f;
05330
05331             if (norm)
05332             {
05333                 tnorm[0] = norm[k][0];
05334                 tnorm[1] = norm[k][1];
05335                 tnorm[2] = norm[k][2];
05336             }
05337             else
05338             {
05339                 // 処理対象の頂点の周囲の頂点番号
05340                 const auto kim{ i > 0 ? k - 1 : k };
05341                 const auto kip{ i < slices ? k + 1 : k };
05342                 const auto kjm{ j > 0 ? k - slices - 1 : k };
05343                 const auto kjp{ j < stacks ? k + slices + 1 : k };

```

```

05345
05346 // 接線ベクトル
05347 const std::array<GLfloat, 3> t
05348 {
05349     pos[kip][0] - pos[kim][0],
05350     pos[kip][1] - pos[kim][1],
05351     pos[kip][2] - pos[kim][2]
05352 };
05353
05354 // 従接線ベクトル
05355 const std::array<GLfloat, 3> b
05356 {
05357     pos[kjp][0] - pos[kjm][0],
05358     pos[kjp][1] - pos[kjm][1],
05359     pos[kjp][2] - pos[kjm][2]
05360 };
05361
05362 // 法線
05363 tnorm[0] = t[1] * b[2] - t[2] * b[1];
05364 tnorm[1] = t[2] * b[0] - t[0] * b[2];
05365 tnorm[2] = t[0] * b[1] - t[1] * b[0];
05366
05367 // 法線の正規化
05368 ggNormalize3(tnorm);
05369 }
05370
05371 // 頂点の位置
05372 const gg::GgVector tpos{ pos[k][0], pos[k][1], pos[k][2], 1.0f };
05373
05374 // 頂点属性の保存
05375 vert.emplace_back(tpos, tnorm);
05376 }
05377 }
05378
05379 // 頂点のインデックス (三角形データ)
05380 std::vector<GLuint> face;
05381
05382 // 頂点のインデックスを求める
05383 for (GLuint j = 0; j < stacks; ++j)
05384 {
05385     for (GLuint i = 0; i < slices; ++i)
05386     {
05387         // 処理対象のマス
05388         const auto k{ (slices + 1) * j + i };
05389
05390         // マスの上半分の三角形
05391         face.emplace_back(k);
05392         face.emplace_back(k + slices + 2);
05393         face.emplace_back(k + 1);
05394
05395         // マスの下半分の三角形
05396         face.emplace_back(k);
05397         face.emplace_back(k + slices + 1);
05398         face.emplace_back(k + slices + 2);
05399     }
05400 }
05401
05402 // GgElements オブジェクトを作成する
05403 return std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05404     face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05405 }
05406
05407 //
05408 // 球状に三角形データを生成する (Elements 形式)
05409 //
05410 std::shared_ptr<gg::GgElements> gg::ggElementsSphere(GLfloat radius, int slices, int stacks)
05411 {
05412     // 頂点の位置と法線
05413     std::vector<GLfloat> p, n;
05414
05415     // 頂点の位置と法線を求める
05416     for (int j = 0; j <= stacks; ++j)
05417     {
05418         const auto t{ static_cast<GLfloat>(j) / static_cast<GLfloat>(stacks) };
05419         const auto ph{ 3.1415926536f * t };
05420         const auto y{ cosf(ph) };
05421         const auto r{ sinf(ph) };
05422
05423         for (int i = 0; i <= slices; ++i)
05424         {
05425             const auto s{ static_cast<GLfloat>(i) / static_cast<GLfloat>(slices) };
05426             const auto th{ -2.0f * 3.1415926536f * s };
05427             const auto x{ r * cosf(th) };
05428             const auto z{ r * sinf(th) };
05429
05430             // 頂点の座標値
05431             p.emplace_back(x * radius);

```

```

05432     p.emplace_back(y * radius);
05433     p.emplace_back(z * radius);
05434
05435     // 頂点の法線
05436     n.emplace_back(x);
05437     n.emplace_back(y);
05438     n.emplace_back(z);
05439 }
05440 }
05441
05442 // GgElements オブジェクトを作成する
05443 return ggElementsMesh(slices, stacks, reinterpret_cast<GLfloat(*)[3]>(p.data()),
05444     reinterpret_cast<GLfloat(*)[3]>(p.data()));
05445 }
05446
05447 //
05448 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05449 //
05450 //   r 光源の強度の環境光成分の赤成分
05451 //   g 光源の強度の環境光成分の緑成分
05452 //   b 光源の強度の環境光成分の青成分
05453 //   a 光源の強度の環境光成分の不透明度
05454 //   first 値を設定する光源データの最初の番号
05455 //   count 値を設定する光源データの数
05456 //
05457 void gg::GgSimpleShader::LightBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05458     GLint first, GLsizei count) const
05459 {
05460     // データを格納するバッファオブジェクトの先頭のポインタ
05461     const auto start{ static_cast<char*>(map(first, count)) };
05462     for (GLsizei i = 0; i < count; ++i)
05463     {
05464         // バッファオブジェクトの i 番目のブロックのポインタ
05465         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05466
05467         // 光源の環境光成分を設定する
05468         light->ambient[0] = r;
05469         light->ambient[1] = g;
05470         light->ambient[2] = b;
05471         light->ambient[3] = a;
05472     }
05473     unmap();
05474 }
05475
05476 //
05477 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05478 //
05479 //   ambient 光源の強度の環境光成分
05480 //   first 値を設定する光源データの最初の番号
05481 //   count 値を設定する光源データの数
05482 //
05483 void gg::GgSimpleShader::LightBuffer::loadAmbient(const GgVector& ambient,
05484     GLint first, GLsizei count) const
05485 {
05486     // データを格納するバッファオブジェクトの先頭のポインタ
05487     const auto start{ static_cast<char*>(map(first, count)) };
05488     for (GLsizei i = 0; i < count; ++i)
05489     {
05490         // バッファオブジェクトの i 番目のブロックのポインタ
05491         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05492
05493         // 光源の強度の環境光成分を設定する
05494         light->ambient = ambient;
05495     }
05496     unmap();
05497 }
05498
05499 //
05500 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05501 //
05502 //   r 光源の強度の拡散反射光成分の赤成分
05503 //   g 光源の強度の拡散反射光成分の緑成分
05504 //   b 光源の強度の拡散反射光成分の青成分
05505 //   a 光源の強度の拡散反射光成分の不透明度
05506 //   first 値を設定する光源データの最初の番号
05507 //   count 値を設定する光源データの数
05508 //
05509 void gg::GgSimpleShader::LightBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05510     GLint first, GLsizei count) const
05511 {
05512     // データを格納するバッファオブジェクトの先頭のポインタ
05513     const auto start{ static_cast<char*>(map(first, count)) };
05514     for (GLsizei i = 0; i < count; ++i)
05515     {
05516         // バッファオブジェクトの i 番目のブロックのポインタ
05517         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05518

```

```

05519 // 光源の拡散反射光成分を設定する
05520 light->diffuse[0] = r;
05521 light->diffuse[1] = g;
05522 light->diffuse[2] = b;
05523 light->diffuse[3] = a;
05524 }
05525 unmap();
05526 }
05527
05528 //
05529 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05530 //
05531 // ambient 光源の強度の拡散反射光成分
05532 // first 値を設定する光源データの最初の番号
05533 // count 値を設定する光源データの数
05534 //
05535 void gg::GgSimpleShader::LightBuffer::loadDiffuse(const GgVector& diffuse,
05536 GLint first, GLsizei count) const
05537 {
05538 // データを格納するバッファオブジェクトの先頭のポインタ
05539 const auto start{ static_cast<char*>(map(first, count)) };
05540 for (GLsizei i = 0; i < count; ++i)
05541 {
05542 // バッファオブジェクトの i 番目のブロックのポインタ
05543 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05544
05545 // 光源の強度の拡散反射光成分を設定する
05546 light->diffuse = diffuse;
05547 }
05548 unmap();
05549 }
05550
05551 //
05552 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05553 //
05554 // r 光源の強度の鏡面反射光成分の赤成分
05555 // g 光源の強度の鏡面反射光成分の緑成分
05556 // b 光源の強度の鏡面反射光成分の青成分
05557 // a 光源の強度の鏡面反射光成分の不透明度
05558 // first 値を設定する光源データの最初の番号
05559 // count 値を設定する光源データの数
05560 //
05561 void gg::GgSimpleShader::LightBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05562 GLint first, GLsizei count) const
05563 {
05564 // データを格納するバッファオブジェクトの先頭のポインタ
05565 const auto start{ static_cast<char*>(map(first, count)) };
05566 for (GLsizei i = 0; i < count; ++i)
05567 {
05568 // バッファオブジェクトの i 番目のブロックのポインタ
05569 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05570
05571 // 光源の鏡面反射光成分を設定する
05572 light->specular[0] = r;
05573 light->specular[1] = g;
05574 light->specular[2] = b;
05575 light->specular[3] = a;
05576 }
05577 unmap();
05578 }
05579
05580 //
05581 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05582 //
05583 // ambient 光源の強度の鏡面反射光成分
05584 // first 値を設定する光源データの最初の番号
05585 // count 値を設定する光源データの数
05586 //
05587 void gg::GgSimpleShader::LightBuffer::loadSpecular(const GgVector& specular,
05588 GLint first, GLsizei count) const
05589 {
05590 // データを格納するバッファオブジェクトの先頭のポインタ
05591 const auto start{ static_cast<char*>(map(first, count)) };
05592 for (GLsizei i = 0; i < count; ++i)
05593 {
05594 // バッファオブジェクトの i 番目のブロックのポインタ
05595 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05596
05597 // 光源の強度の鏡面反射光成分を設定する
05598 light->specular = specular;
05599 }
05600 unmap();
05601 }
05602
05603 //
05604 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の色を設定するか位置は変更しない
05605 //

```

```

05606 // material 光源の特性の GgSimpleShader::Light 構造体
05607 // first 値を設定する光源データの最初の番号
05608 // count 値を設定する光源データの数
05609 //
05610 void gg::GgSimpleShader::LightBuffer::loadColor(const Light& color,
05611 GLint first, GLsizei count) const
05612 {
05613     // データを格納するバッファオブジェクトの先頭のポインタ
05614     const auto start{ static_cast<char*>(map(first, count)) };
05615     for (GLsizei i = 0; i < count; ++i)
05616     {
05617         // バッファオブジェクトの i 番目のブロックのポインタ
05618         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05619
05620         // 光源の色を設定する
05621         light->ambient = color.ambient;
05622         light->diffuse = color.diffuse;
05623         light->specular = color.specular;
05624     }
05625     unmap();
05626 }
05627 //
05628 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05629 //
05630 //
05631 // x 光源の位置の x 座標
05632 // y 光源の位置の y 座標
05633 // z 光源の位置の z 座標
05634 // w 光源の位置の w 座標
05635 // first 値を設定する光源データの最初の番号
05636 // count 値を設定する光源データの数
05637 //
05638 void gg::GgSimpleShader::LightBuffer::loadPosition(GLfloat x, GLfloat y, GLfloat z, GLfloat w,
05639 GLint first, GLsizei count) const
05640 {
05641     // データを格納するバッファオブジェクトの先頭のポインタ
05642     const auto start{ static_cast<char*>(map(first, count)) };
05643     for (GLsizei i = 0; i < count; ++i)
05644     {
05645         // バッファオブジェクトの i 番目のブロックのポインタ
05646         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05647
05648         // 光源の位置を設定する
05649         light->position[0] = x;
05650         light->position[1] = y;
05651         light->position[2] = z;
05652         light->position[3] = w;
05653     }
05654     unmap();
05655 }
05656 //
05657 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05658 //
05659 //
05660 // position 光源の位置
05661 // first 値を設定する光源データの最初の番号
05662 // count 値を設定する光源データの数
05663 //
05664 void gg::GgSimpleShader::LightBuffer::loadPosition(const GgVector& position,
05665 GLint first, GLsizei count) const
05666 {
05667     // データを格納するバッファオブジェクトの先頭のポインタ
05668     const auto start{ static_cast<char*>(map(first, count)) };
05669     for (GLsizei i = 0; i < count; ++i)
05670     {
05671         // バッファオブジェクトの i 番目のブロックのポインタ
05672         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05673
05674         // 光源の位置を設定する
05675         light->position = position;
05676     }
05677     unmap();
05678 }
05679 //
05680 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05681 //
05682 //
05683 // r 環境光に対する反射係数の赤成分
05684 // g 環境光に対する反射係数の緑成分
05685 // b 環境光に対する反射係数の青成分
05686 // a 環境光に対する反射係数の不透明度
05687 // first 値を設定する材質データの最初の番号
05688 // count 値を設定する材質データの数
05689 //
05690 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05691 GLint first, GLsizei count) const
05692 {

```



```

05693 // データを格納するバッファオブジェクトの先頭のポインタ
05694 const auto start{ static_cast<char*>(map(first, count)) };
05695 for (GLsizei i = 0; i < count; ++i)
05696 {
05697     // バッファオブジェクトの i 番目のブロックのポインタ
05698     const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05699
05700     // 環境光に対する反射係数を設定する
05701     material->ambient[0] = r;
05702     material->ambient[1] = g;
05703     material->ambient[2] = b;
05704     material->ambient[3] = a;
05705 }
05706 unmap();
05707 }
05708
05709 //
05710 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05711 //
05712 // ambient 環境光に対する反射係数
05713 // first 値を設定する光源データの最初の番号
05714 // count 値を設定する光源データの数
05715 //
05716 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(const GgVector& ambient,
05717 GLint first, GLsizei count) const
05718 {
05719     // データを格納するバッファオブジェクトの先頭のポインタ
05720     const auto start{ static_cast<char*>(map(first, count)) };
05721     for (GLsizei i = 0; i < count; ++i)
05722     {
05723         // バッファオブジェクトの i 番目のブロックのポインタ
05724         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05725
05726         // 光源の強度の環境光成分を設定する
05727         material->ambient = ambient;
05728     }
05729     unmap();
05730 }
05731
05732 //
05733 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05734 //
05735 // r 拡散反射係数の赤成分
05736 // g 拡散反射係数の緑成分
05737 // b 拡散反射係数の青成分
05738 // a 拡散反射係数の不透明度
05739 // first 値を設定する材質データの最初の番号
05740 // count 値を設定する材質データの数
05741 //
05742 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05743 GLint first, GLsizei count) const
05744 {
05745     // データを格納するバッファオブジェクトの先頭のポインタ
05746     const auto start{ static_cast<char*>(map(first, count)) };
05747     for (GLsizei i = 0; i < count; ++i)
05748     {
05749         // バッファオブジェクトの i 番目のブロックのポインタ
05750         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05751
05752         // 拡散反射係数を設定する
05753         material->diffuse[0] = r;
05754         material->diffuse[1] = g;
05755         material->diffuse[2] = b;
05756         material->diffuse[3] = a;
05757     }
05758     unmap();
05759 }
05760
05761 //
05762 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05763 //
05764 // diffuse 拡散反射係数
05765 // first 値を設定する光源データの最初の番号
05766 // count 値を設定する光源データの数
05767 //
05768 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(const GgVector& diffuse,
05769 GLint first, GLsizei count) const
05770 {
05771     // データを格納するバッファオブジェクトの先頭のポインタ
05772     const auto start{ static_cast<char*>(map(first, count)) };
05773     for (GLsizei i = 0; i < count; ++i)
05774     {
05775         // バッファオブジェクトの i 番目のブロックのポインタ
05776         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05777
05778         // 光源の強度の環境光成分を設定する
05779         material->diffuse = diffuse;

```

```

05780     }
05781     unmap();
05782 }
05783
05784 //
05785 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05786 //
05787 //   r 環境光に対する反射係数と拡散反射係数の赤成分
05788 //   g 環境光に対する反射係数と拡散反射係数の緑成分
05789 //   b 環境光に対する反射係数と拡散反射係数の青成分
05790 //   a 環境光に対する反射係数と拡散反射係数の不透明度
05791 //   first 値を設定する材質データの最初の番号
05792 //   count 値を設定する材質データの数
05793 //
05794 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(GLfloat r, GLfloat g, GLfloat b,
    GLfloat a,
    GLint first, GLsizei count) const
05795 {
05796     // データを格納するバッファオブジェクトの先頭のポインタ
05797     const auto start{ static_cast<char*>(map(first, count)) };
05798     for (GLsizei i = 0; i < count; ++i)
05799     {
05800         // バッファオブジェクトの i 番目のブロックのポインタ
05801         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05802
05803         // 環境光に対する反射係数と拡散反射係数を設定する
05804         material->ambient[0] = material->diffuse[0] = r;
05805         material->ambient[1] = material->diffuse[1] = g;
05806         material->ambient[2] = material->diffuse[2] = b;
05807         material->ambient[3] = material->diffuse[3] = a;
05808     }
05809     unmap();
05810 }
05811
05812 //
05813 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05814 //
05815 //   color 環境光に対する反射係数と拡散反射係数
05816 //   first 値を設定する光源データの最初の番号
05817 //   count 値を設定する光源データの数
05818 //
05819 //
05820 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GgVector& color,
    GLint first, GLsizei count) const
05821 {
05822     // データを格納するバッファオブジェクトの先頭のポインタ
05823     const auto start{ static_cast<char*>(map(first, count)) };
05824     for (GLsizei i = 0; i < count; ++i)
05825     {
05826         // バッファオブジェクトの i 番目のブロックのポインタ
05827         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05828
05829         // 光源の強度の環境光成分を設定する
05830         material->ambient = material->diffuse = color;
05831     }
05832     unmap();
05833 }
05834
05835 //
05836 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05837 //
05838 //   color 環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列
05839 //   first 値を設定する材質データの最初の番号
05840 //   count 値を設定する材質データの数
05841 //
05842 //
05843 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GLfloat* color,
    GLint first, GLsizei count) const
05844 {
05845     // ambient 要素のバイトオフセット
05846     constexpr auto ambientOffset{ offsetof(Material, ambient) };
05847
05848     // ambient 要素のバイト数
05849     constexpr auto ambientSize{ sizeof(Material::diffuse) };
05850
05851     // diffuse 要素のバイトオフセット
05852     constexpr auto diffuseOffset{ offsetof(Material, diffuse) };
05853
05854     // diffuse 要素のバイト数
05855     constexpr auto diffuseSize{ sizeof(Material::diffuse) };
05856
05857     // 元のデータの先頭
05858     const auto* source{ reinterpret_cast<const char*>(color) };
05859
05860     // first 番目のブロックから count 個の ambient 要素と diffuse 要素に値を設定する
05861     bind();
05862     for (GLsizei i = 0; i < count; ++i)
05863     {
05864         // 格納先
05865

```

```

05866     const auto destination{ getStride() * (first + i) };
05867
05868     // first + i 番目のブロックの ambient 要素に値を設定する
05869     glBufferSubData(getTarget(), destination + ambientOffset, ambientSize, source + i * ambientSize);
05870
05871     // first + i 番目のブロックの diffuse 要素に値を設定する
05872     glBufferSubData(getTarget(), destination + diffuseOffset, diffuseSize, source + i * diffuseSize);
05873 }
05874 }
05875
05876 //
05877 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05878 //
05879 //   r 鏡面反射係数の赤成分
05880 //   g 鏡面反射係数の緑成分
05881 //   b 鏡面反射係数の青成分
05882 //   a 鏡面反射係数の不透明度
05883 //   first 値を設定する材質データの最初の番号
05884 //   count 値を設定する材質データの数
05885 //
05886 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05887 GLint first, GLsizei count) const
05888 {
05889     // データを格納するバッファオブジェクトの先頭のポインタ
05890     const auto start{ static_cast<char*>(map(first, count)) };
05891     for (GLsizei i = 0; i < count; ++i)
05892     {
05893         // バッファオブジェクトの i 番目のブロックのポインタ
05894         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05895
05896         // 鏡面反射係数を設定する
05897         material->specular[0] = r;
05898         material->specular[1] = g;
05899         material->specular[2] = b;
05900         material->specular[3] = a;
05901     }
05902     unmap();
05903 }
05904
05905 //
05906 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05907 //
05908 //   specular 鏡面反射係数
05909 //   first 値を設定する光源データの最初の番号
05910 //   count 値を設定する光源データの数
05911 //
05912 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(const GgVector& specular,
05913 GLint first, GLsizei count) const
05914 {
05915     // データを格納するバッファオブジェクトの先頭のポインタ
05916     const auto start{ static_cast<char*>(map(first, count)) };
05917     for (GLsizei i = 0; i < count; ++i)
05918     {
05919         // バッファオブジェクトの i 番目のブロックのポインタ
05920         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05921
05922         // 光源の強度の環境光成分を設定する
05923         material->specular = specular;
05924     }
05925     unmap();
05926 }
05927
05928 //
05929 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05930 //
05931 //   shininess 輝き係数
05932 //   first 値を設定する材質データの最初の番号
05933 //   count 値を設定する材質データの数
05934 //
05935 void gg::GgSimpleShader::MaterialBuffer::loadShininess(GLfloat shininess,
05936 GLint first, GLsizei count) const
05937 {
05938     // データを格納するバッファオブジェクトの先頭のポインタ
05939     const auto start{ static_cast<char*>(map(first, count)) };
05940     for (GLsizei i = 0; i < count; ++i)
05941     {
05942         // バッファオブジェクトの i 番目のブロックのポインタ
05943         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05944         material->shininess = shininess;
05945     }
05946     unmap();
05947 }
05948
05949 //
05950 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05951 //
05952 //   shininess 輝き係数

```

```

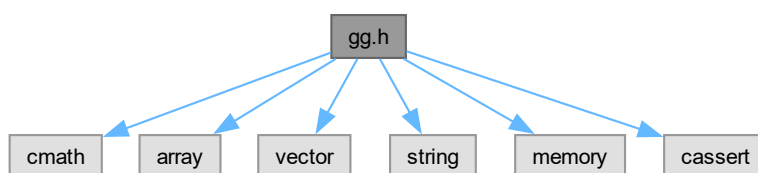
05953 // first 値を設定する材質データの最初の番号
05954 // count 値を設定する材質データの数
05955 //
05956 void gg::GgSimpleShader::MaterialBuffer::loadShininess(const GLfloat* shininess,
05957 GLint first, GLsizei count) const
05958 {
05959 // データを格納するバッファオブジェクトの先頭のポインタ
05960 const auto start{ static_cast<char*>(map(first, count)) };
05961 for (GLsizei i = 0; i < count; ++i)
05962 {
05963 // バッファオブジェクトの i 番目のブロックのポインタ
05964 const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05965 material->shininess = shininess[i];
05966 }
05967 unmap();
05968 }
05969 //
05970 // 三角形に単純な陰影付けを行うシェーダ：シェーダのソースファイルの読み込み
05971 //
05972 //
05973 bool gg::GgSimpleShader::load(const std::string& vert, const std::string& frag, const std::string&
geom,
GLint nvarying, const char* const* varyings)
05974 {
05975 if (!GgPointShader::load(vert, frag, geom, nvarying, varyings)) return false;
05976
05977 mnLoc = glGetUniformLocation(get(), "mn");
05978 const auto lightIndex{ glGetUniformLocation(get(), "Light") };
05979 glUniformBlockBinding(get(), lightIndex, LightBindingPoint);
05980 const auto materialIndex{ glGetUniformLocation(get(), "Material") };
05981 glUniformBlockBinding(get(), materialIndex, MaterialBindingPoint);
05982
05983 return true;
05984 }
05985 //
05986 // Wavefront OBJ 形式のデータ：コンストラクタ
05987 //
05988 //
05989 //
05990 gg::GgSimpleObj::GgSimpleObj(const std::string& name, bool normalize)
05991 {
05992 // 作業用のメモリ
05993 std::vector<GgSimpleShader::Material> mat;
05994 std::vector<GgVertex> vert;
05995 std::vector<GLuint> face;
05996
05997 // グループのデータのメモリを確保する
05998 group = std::make_shared<std::vector<std::array<GLuint, 3>>>();
05999
06000 // ファイルを読み込む
06001 if (ggLoadSimpleObj(name, *group, mat, vert, face, normalize))
06002 {
06003 // 頂点バッファオブジェクトを作成する
06004 data = std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
06005 face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
06006
06007 // 描画するオブジェクトを切り替えるために頂点配列オブジェクトを閉じておく
06008 glBindVertexArray(0);
06009
06010 // 材質データを設定する
06011 material = std::make_shared<GgSimpleShader::MaterialBuffer>(mat.data(),
static_cast<GLsizei>(mat.size()));
06012 }
06013 }
06014 //
06015 // Wavefront OBJ 形式のデータ：図形の描画
06016 //
06017 //
06018 void gg::GgSimpleObj::draw(GLint first, GLsizei count) const
06019 {
06020 // 保持しているグループの数
06021 const auto ng{ static_cast<GLsizei>(group->size()) };
06022
06023 // 描画する最後のグループの次
06024 auto last{ count <= 0 ? ng : first + count };
06025 if (last > ng) last = ng;
06026
06027 for (GLsizei i = first; i < last; ++i)
06028 {
06029 // グループのデータ
06030 const auto& g{ (*group)[i] };
06031
06032 // 材質を設定する
06033 material->select(g[2]);
06034
06035 // 図形を描画する
06036 data->draw(g[0], g[1]);
06037 }

```

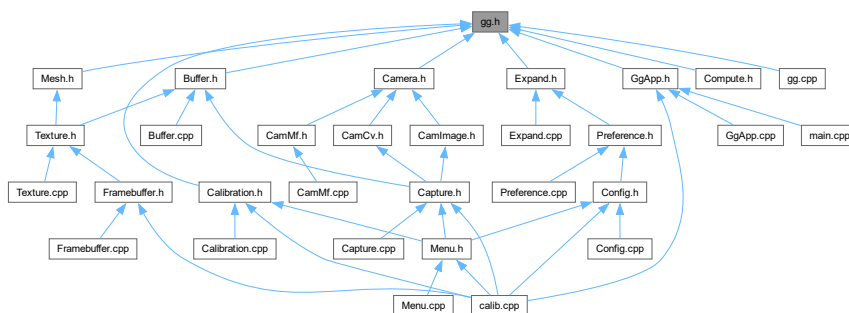
```
06038 }
```

9.41 gg.h ファイル

```
#include <cmath>
#include <array>
#include <vector>
#include <string>
#include <memory>
#include <cassert>
gg.h の依存先関係図:
```



被依存関係図:



クラス

- class `gg::GgVector`
- class `gg::GgMatrix`
- class `gg::GgQuaternion`
- class `gg::GgTrackball`
- class `gg::GgTexture`
- class `gg::GgColorTexture`
- class `gg::GgNormalTexture`
- class `gg::GgBuffer< T >`
- class `gg::GgUniformBuffer< T >`
- class `gg::GgVertexArray`

- class `gg::GgShape`
- class `gg::GgPoints`
- struct `gg::GgVertex`
- class `gg::GgTriangles`
- class `gg::GgElements`
- class `gg::GgShader`
- class `gg::GgPointShader`
- class `gg::GgSimpleShader`
- struct `gg::GgSimpleShader::Light`
- class `gg::GgSimpleShader::LightBuffer`
- struct `gg::GgSimpleShader::Material`
- class `gg::GgSimpleShader::MaterialBuffer`
- class `gg::GgSimpleObj`

名前空間

- namespace `gg`

マクロ定義

- `#define ggError()`
- `#define ggFBOError()`

型定義

- using `pathString` = `std::string`
- using `pathChar` = `char`

列挙型

- enum `gg::BindingPoints` { `gg::LightBindingPoint` = 0 , `gg::MaterialBindingPoint` }

関数

- `pathString Utf8ToTChar` (const `std::string` &string)
- `std::string TCharToUtf8` (const `pathString` &cstring)
- void `gg::ggInit` ()
- void `gg::ggError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggFBOError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength3` (const GLfloat *a)
- GLfloat `gg::ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize3` (GLfloat *a)
- GLfloat `gg::ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength4` (const GLfloat *a)
- GLfloat `gg::ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize4` (GLfloat *a)

- `const GgVector & gg::operator+ (const GgVector &v)`
- `GgVector gg::operator+ (GLfloat a, const GgVector &b)`
- `const GgVector gg::operator- (const GgVector &v)`
- `GgVector gg::operator- (GLfloat a, const GgVector &b)`
- `GgVector gg::operator* (GLfloat a, const GgVector &b)`
- `GgVector gg::operator/ (GLfloat a, const GgVector &b)`
- `GgVector gg::ggCross (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggDot3 (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggLength3 (const GgVector &a)`
- `GLfloat gg::ggDistance3 (const GgVector &a, const GgVector &b)`
- `GgVector gg::ggNormalize3 (const GgVector &a)`
- `void gg::ggNormalize3 (GgVector *a)`
- `GLfloat gg::ggDot4 (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggLength4 (const GgVector &a)`
- `GLfloat gg::ggDistance4 (const GgVector &a, const GgVector &b)`
- `GgVector gg::ggNormalize4 (const GgVector &a)`
- `void gg::ggNormalize4 (GgVector *a)`
- `GgMatrix gg::ggIdentity ()`
- `GgMatrix gg::ggTranslate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)`
- `GgMatrix gg::ggTranslate (const GLfloat *t)`
- `GgMatrix gg::ggTranslate (const GgVector &t)`
- `GgMatrix gg::ggScale (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)`
- `GgMatrix gg::ggScale (const GLfloat *s)`
- `GgMatrix gg::ggScale (const GgVector &s)`
- `GgMatrix gg::ggRotateX (GLfloat a)`
- `GgMatrix gg::ggRotateY (GLfloat a)`
- `GgMatrix gg::ggRotateZ (GLfloat a)`
- `GgMatrix gg::ggRotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a)`
- `GgMatrix gg::ggRotate (const GLfloat *r, GLfloat a)`
- `GgMatrix gg::ggRotate (const GgVector &r, GLfloat a)`
- `GgMatrix gg::ggRotate (const GLfloat *r)`
- `GgMatrix gg::ggRotate (const GgVector &r)`
- `GgMatrix gg::ggLookat (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)`
- `GgMatrix gg::ggLookat (const GLfloat *e, const GLfloat *t, const GLfloat *u)`
- `GgMatrix gg::ggLookat (const GgVector &e, const GgVector &t, const GgVector &u)`
- `GgMatrix gg::ggOrthogonal (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)`
- `GgMatrix gg::ggFrustum (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)`
- `GgMatrix gg::ggPerspective (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)`
- `GgMatrix gg::ggTranspose (const GgMatrix &m)`
- `GgMatrix gg::ggInvert (const GgMatrix &m)`
- `GgMatrix gg::ggNormal (const GgMatrix &m)`
- `GgQuaternion gg::ggIdentityQuaternion ()`
- `GgQuaternion gg::ggMatrixQuaternion (const GLfloat *a)`
- `GgQuaternion gg::ggMatrixQuaternion (const GgMatrix &m)`
- `GgMatrix gg::ggQuaternionMatrix (const GgQuaternion &q)`
- `GgMatrix gg::ggQuaternionTransposeMatrix (const GgQuaternion &q)`
- `GgQuaternion gg::ggRotateQuaternion (GLfloat x, GLfloat y, GLfloat z, GLfloat a)`
- `GgQuaternion gg::ggRotateQuaternion (const GLfloat *v, GLfloat a)`
- `GgQuaternion gg::ggRotateQuaternion (const GLfloat *v)`
- `GgQuaternion gg::ggEulerQuaternion (GLfloat heading, GLfloat pitch, GLfloat roll)`
- `GgQuaternion gg::ggEulerQuaternion (const GLfloat *e)`
- `GgQuaternion gg::ggEulerQuaternion (const GgVector &e)`
- `GgQuaternion gg::ggSlerp (const GLfloat *a, const GLfloat *b, GLfloat t)`

- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, `GLfloat` t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const `GLfloat` *a, `GLfloat` t)
- `GgQuaternion gg::ggSlerp` (const `GLfloat` *a, const `GgQuaternion` &q, `GLfloat` t)
- `GLfloat gg::ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggInvert` (const `GgQuaternion` &q)
- bool `gg::ggSaveTga` (const `std::string` &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool `gg::ggSaveColor` (const `std::string` &name)
- bool `gg::ggSaveDepth` (const `std::string` &name)
- bool `gg::ggReadImage` (const `std::string` &name, `std::vector`< `GLubyte` > &image, `GLsizei` *pWidth, `GLsizei` *pHeight, `GLenum` *pFormat)
- `GLuint gg::ggLoadTexture` (const `GLvoid` *image, `GLsizei` width, `GLsizei` height, `GLenum` format=GL_↵RGB, `GLenum` type=GL_UNSIGNED_BYTE, `GLenum` internal=GL_RGB, `GLenum` wrap=GL_CLAMP_TO_↵EDGE, bool swizzle=false)
- `GLuint gg::ggLoadImage` (const `std::string` &name, `GLsizei` *pWidth=nullptr, `GLsizei` *pHeight=nullptr, `GLenum` internal=GL_RGBA, `GLenum` wrap=GL_CLAMP_TO_EDGE)
- void `gg::ggCreateNormalMap` (const `GLubyte` *hmap, `GLsizei` width, `GLsizei` height, `GLenum` format, `GLfloat` nz, `GLenum` internal, `std::vector`< `GgVector` > &nmap)
- `GLuint gg::ggLoadHeight` (const `std::string` &name, `GLfloat` nz, `GLsizei` *pWidth=nullptr, `GLsizei` *p↵Height=nullptr, `GLenum` internal=GL_RGBA)
- `GLuint gg::ggCreateShader` (const `std::string` &vsrc, const `std::string` &fsrc="", const `std::string` &gsrc="", `GLint` nvarying=0, const char *const *varyings=nullptr, const `std::string` &vtext="vertex shader", const `std::↵string` &ftext="fragment shader", const `std::string` >ext="geometry shader")
- `GLuint gg::ggLoadShader` (const `std::string` &vert, const `std::string` &frag="", const `std::string` &geom="", `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint gg::ggLoadShader` (const `std::array`< `std::string`, 3 > &files, `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint gg::ggCreateComputeShader` (const `std::string` &csrc, const `std::string` &ctext="compute shader")
- `GLuint gg::ggLoadComputeShader` (const `std::string` &comp)
- `std::shared_ptr`< `GgPoints` > `gg::ggPointsCube` (`GLsizei` countv, `GLfloat` length=1.0f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- `std::shared_ptr`< `GgPoints` > `gg::ggPointsSphere` (`GLsizei` countv, `GLfloat` radius=0.5f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- `std::shared_ptr`< `GgTriangles` > `gg::ggRectangle` (`GLfloat` width=1.0f, `GLfloat` height=1.0f)
- `std::shared_ptr`< `GgTriangles` > `gg::ggEllipse` (`GLfloat` width=1.0f, `GLfloat` height=1.0f, `GLuint` slices=16)
- `std::shared_ptr`< `GgTriangles` > `gg::ggArraysObj` (const `std::string` &name, bool normalize=false)
- `std::shared_ptr`< `GgElements` > `gg::ggElementsObj` (const `std::string` &name, bool normalize=false)
- `std::shared_ptr`< `GgElements` > `gg::ggElementsMesh` (`GLuint` slices, `GLuint` stacks, const `GLfloat`(*pos)[3], const `GLfloat`(*norm)[3]=nullptr)
- `std::shared_ptr`< `GgElements` > `gg::ggElementsSphere` (`GLfloat` radius=1.0f, int slices=16, int stacks=8)
- bool `gg::ggLoadSimpleObj` (const `std::string` &name, `std::vector`< `std::array`< `GLuint`, 3 > > &group, `std::↵vector`< `GgSimpleShader::Material` > &material, `std::vector`< `GgVertex` > &vert, bool normalize=false)
- bool `gg::ggLoadSimpleObj` (const `std::string` &name, `std::vector`< `std::array`< `GLuint`, 3 > > &group, `std::↵vector`< `GgSimpleShader::Material` > &material, `std::vector`< `GgVertex` > &vert, `std::vector`< `GLuint` > &face, bool normalize=false)

変数

- `GLint gg::ggBufferAlignment`
使用している GPU のバッファアライメント。

9.41.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の宣言.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.h](#) に定義があります。

9.41.2 マクロ定義詳解

9.41.2.1 ggError

```
#define ggError()
```

OpenGL のエラーの発生を検知したときにソースファイル名と行番号を表示する.

覚え書き

このマクロを置いた場所（より前）で OpenGL のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する. リリースビルド時には無視される.

[gg.h](#) の 1392 行目に定義があります。

9.41.2.2 ggFBOError

```
#define ggFBOError()
```

FBO のエラーの発生を検知したときにソースファイル名と行番号を表示する.

覚え書き

このマクロを置いた場所（より前）で FBO のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する. リリースビルド時には無視される.

[gg.h](#) の 1419 行目に定義があります。

9.41.3 型定義詳解

9.41.3.1 pathChar

```
using pathChar = char
```

[gg.h](#) の 78 行目に定義があります。

9.41.3.2 pathString

```
using pathString = std::string
```

gg.h の 77 行目に定義があります。

9.41.4 関数詳解

9.41.4.1 TCharToUtf8()

```
std::string TCharToUtf8 (
    const pathString & cstring) [inline]
```

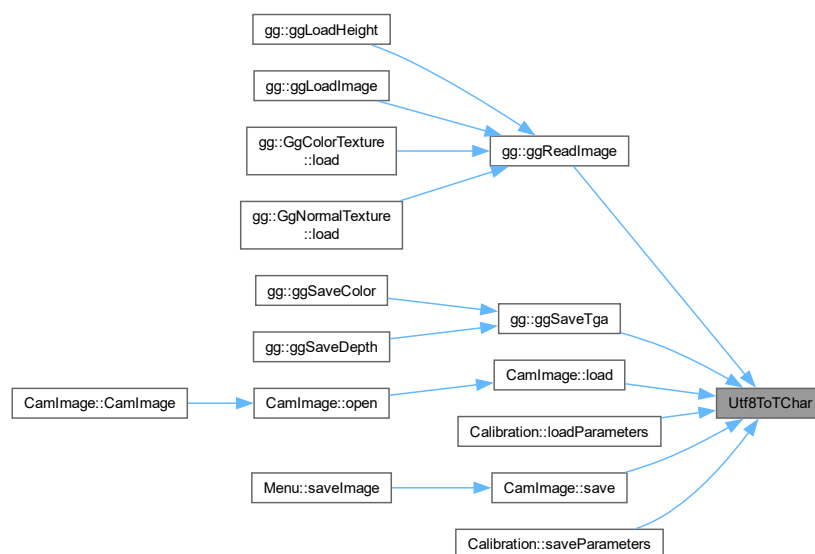
gg.h の 80 行目に定義があります。

9.41.4.2 Utf8ToTChar()

```
pathString Utf8ToTChar (
    const std::string & string) [inline]
```

gg.h の 79 行目に定義があります。

被呼び出し関係図:



9.42 gg.h

[詳解]

```

00001 #pragma once
00002
00026
00034
00035 // 標準ライブラリ
00036 #include <cmath>
00037 #include <array>
00038 #include <vector>
00039 #include <string>
00040 #include <memory>
00041 #include <cassert>
00042
00043 // Windows (Visual Studio) のとき
00044 #if defined(WIN32)
00045 // 非推奨の警告を出さない
00046 # pragma warning(disable:4996)
00047 // 数学ライブラリの定数を使う
00048 # define _USE_MATH_DEFINES
00049 // MIN() / MAX マクロは使わない
00050 # define NOMINMAX
00051 // ファイルパスの文字コード
00052 #include <atlstr.h>
00053 using pathString = CString;
00054 using pathChar = wchar_t;
00055 extern pathString Utf8ToTChar(const std::string& string);
00056 extern std::string TCharToUtf8(const pathString& cstring);
00057 // デバッグビルドかどうか調べる
00058 # if defined(DEBUG)
00059 #   if !defined(DEBUG)
00060 #       define DEBUG
00061 #   endif
00062 #   define GLFW3_CONFIGURATION "Debug"
00063 # else
00064 #   if !defined(NDEBUG)
00065 #       define NDEBUG
00066 #   endif
00067 #   define GLFW3_CONFIGURATION "Release"
00068 #   endif
00069 // プラットフォームを調べる
00070 # if defined(WIN64)
00071 #   define GLFW3_PLATFORM "x64"
00072 # else
00073 #   define GLFW3_PLATFORM "Win32"
00074 #   endif
00075 #else
00076 // ファイルパスの文字コード
00077 using pathString = std::string;
00078 using pathChar = char;
00079 inline pathString Utf8ToTChar(const std::string& string) { return string; }
00080 inline std::string TCharToUtf8(const pathString& cstring) { return cstring; }
00081 #endif
00082
00084
00085 // macOS で "OpenGL deprecated" の警告を出さない
00086 #if defined(__APPLE__)
00087 # define GL_SILENCE_DEPRECATED
00088 #endif
00089
00090 // フレームワークに GLFW 3 を使う
00091 #if defined(IMGUI_IMPL_OPENGL_ES2)
00092 # define GLFW_INCLUDE_ES2
00093 #elif defined(IMGUI_IMPL_OPENGL_ES3)
00094 # define GLFW_INCLUDE_ES3
00095 #else
00096 # define GLFW_INCLUDE_GLCOREARB
00097 #endif
00098 #include <GLFW/glfw3.h>
00099
00100 // OpenGL 3.2 の API のエントリポイント
00101 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00102 extern PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00103 extern PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00104 extern PFNGLACTIVETEXTUREPROC glActiveTexture;
00105 extern PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00106 extern PFNGLATTACHSHADERPROC glAttachShader;
00107 extern PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00108 extern PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00109 extern PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00110 extern PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00111 extern PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00112 extern PFNGLBEGINQUERYPROC glBeginQuery;
00113 extern PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;

```

```

00114 extern PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00115 extern PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00116 extern PFNGLBINDBUFFERPROC glBindBuffer;
00117 extern PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00118 extern PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00119 extern PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00120 extern PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00121 extern PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00122 extern PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00123 extern PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00124 extern PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00125 extern PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00126 extern PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00127 extern PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00128 extern PFNGLBINDSAMPLERPROC glBindSampler;
00129 extern PFNGLBINDSAMPLERSPROC glBindSamplers;
00130 extern PFNGLBINDTEXTUREPROC glBindTexture;
00131 extern PFNGLBINDTEXTURESPROC glBindTextures;
00132 extern PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00133 extern PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00134 extern PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00135 extern PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00136 extern PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00137 extern PFNGLBLENDBARRIERKHRPROC glBlendBarrierKHR;
00138 extern PFNGLBLENDBARRIERNVPROC glBlendBarrierNV;
00139 extern PFNGLBLENDCOLORPROC glBlendColor;
00140 extern PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00141 extern PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00142 extern PFNGLBLEND EQUATIONPROC glBlendEquation;
00143 extern PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00144 extern PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00145 extern PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00146 extern PFNGLBLENDFUNCTIONIARBPROC glBlendFunciARB;
00147 extern PFNGLBLENDFUNCTIONPROC glBlendFunci;
00148 extern PFNGLBLENDFUNCTIONPROC glBlendFunc;
00149 extern PFNGLBLENDFUNCTIONSEPARATEIARBPROC glBlendFuncSeparateiARB;
00150 extern PFNGLBLENDFUNCTIONSEPARATEIPROC glBlendFuncSeparatei;
00151 extern PFNGLBLENDFUNCTIONSEPARATEPROC glBlendFuncSeparate;
00152 extern PFNGLBLENDPARAMETERINVPROC glBlendParameteriNV;
00153 extern PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00154 extern PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00155 extern PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00156 extern PFNGLBUFFERDATAPROC glBufferData;
00157 extern PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00158 extern PFNGLBUFFERSTORAGEPROC glBufferStorage;
00159 extern PFNGLBUFFERSUBDATAPROC glBufferSubData;
00160 extern PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00161 extern PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00162 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00163 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00164 extern PFNGLCLAMP COLORPROC glClampColor;
00165 extern PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00166 extern PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00167 extern PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00168 extern PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00169 extern PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00170 extern PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00171 extern PFNGLCLEARCOLORPROC glClearColor;
00172 extern PFNGLCLEARDEPTHFPROC glClearDepthf;
00173 extern PFNGLCLEARDEPTHFPROC glClearDepth;
00174 extern PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00175 extern PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00176 extern PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00177 extern PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00178 extern PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00179 extern PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00180 extern PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00181 extern PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00182 extern PFNGLCLEARPROC glClear;
00183 extern PFNGLCLEARSTENCILPROC glClearStencil;
00184 extern PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00185 extern PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00186 extern PFNGLCLIENTATTRIBDEFAULTTEXTPROC glClientAttribDefaultEXT;
00187 extern PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00188 extern PFNGLCLIPCONTROLPROC glClipControl;
00189 extern PFNGLCOLORFORMATNVPROC glColorFormatNV;
00190 extern PFNGLCOLORMASKIPROC glColorMaski;
00191 extern PFNGLCOLORMASKPROC glColorMask;
00192 extern PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00193 extern PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00194 extern PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00195 extern PFNGLCOMPILESHADERPROC glCompileShader;
00196 extern PFNGLCOMPRESSEDMULTITEXIMAGE1DEXTPROC glCompressedMultiTexImage1DEXT;
00197 extern PFNGLCOMPRESSEDMULTITEXIMAGE2DEXTPROC glCompressedMultiTexImage2DEXT;
00198 extern PFNGLCOMPRESSEDMULTITEXIMAGE3DEXTPROC glCompressedMultiTexImage3DEXT;
00199 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DEXTPROC glCompressedMultiTexSubImage1DEXT;
00200 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DEXTPROC glCompressedMultiTexSubImage2DEXT;

```

```
00201 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DEXTPROC glCompressedMultiTexSubImage3DEXT;
00202 extern PFNGLCOMPRESSEDTEXTUREIMAGE1DPROC glCompressedTexImage1D;
00203 extern PFNGLCOMPRESSEDTEXTUREIMAGE2DPROC glCompressedTexImage2D;
00204 extern PFNGLCOMPRESSEDTEXTUREIMAGE3DPROC glCompressedTexImage3D;
00205 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTexSubImage1D;
00206 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTexSubImage2D;
00207 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTexSubImage3D;
00208 extern PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC glCompressedTextureImage1DEXT;
00209 extern PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC glCompressedTextureImage2DEXT;
00210 extern PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC glCompressedTextureImage3DEXT;
00211 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC glCompressedTextureSubImage1DEXT;
00212 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00213 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC glCompressedTextureSubImage2DEXT;
00214 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00215 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC glCompressedTextureSubImage3DEXT;
00216 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00217 extern PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00218 extern PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00219 extern PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00220 extern PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00221 extern PFNGLCOPYMULTITEXIMAGE1DEXTPROC glCopyMultiTexImage1DEXT;
00222 extern PFNGLCOPYMULTITEXIMAGE2DEXTPROC glCopyMultiTexImage2DEXT;
00223 extern PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC glCopyMultiTexSubImage1DEXT;
00224 extern PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC glCopyMultiTexSubImage2DEXT;
00225 extern PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC glCopyMultiTexSubImage3DEXT;
00226 extern PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00227 extern PFNGLCOPYPATHNVPROC glCopyPathNV;
00228 extern PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00229 extern PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00230 extern PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00231 extern PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00232 extern PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00233 extern PFNGLCOPYTEXTUREIMAGE1DEXTPROC glCopyTextureImage1DEXT;
00234 extern PFNGLCOPYTEXTUREIMAGE2DEXTPROC glCopyTextureImage2DEXT;
00235 extern PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC glCopyTextureSubImage1DEXT;
00236 extern PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00237 extern PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC glCopyTextureSubImage2DEXT;
00238 extern PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00239 extern PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC glCopyTextureSubImage3DEXT;
00240 extern PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00241 extern PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00242 extern PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00243 extern PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00244 extern PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00245 extern PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00246 extern PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00247 extern PFNGLCREATEBUFFERSPROC glCreateBuffers;
00248 extern PFNGLCREATecomMANDLISTSNVPROC glCreateCommandListsNV;
00249 extern PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00250 extern PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00251 extern PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00252 extern PFNGLCREATEPROGRAMPROC glCreateProgram;
00253 extern PFNGLCREATEQUERIESPROC glCreateQueries;
00254 extern PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00255 extern PFNGLCREATESAMPLERSPROC glCreateSamplers;
00256 extern PFNGLCREATESHADERPROC glCreateShader;
00257 extern PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00258 extern PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00259 extern PFNGLCREATESTATESNVPROC glCreateStatesNV;
00260 extern PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00261 extern PFNGLCREATETEXTURESPROC glCreateTextures;
00262 extern PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00263 extern PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00264 extern PFNGLCULLFACEPROC glCullFace;
00265 extern PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00266 extern PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00267 extern PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00268 extern PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00269 extern PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00270 extern PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00271 extern PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00272 extern PFNGLDELETecomMANDLISTSNVPROC glDeleteCommandListsNV;
00273 extern PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00274 extern PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00275 extern PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00276 extern PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00277 extern PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00278 extern PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00279 extern PFNGLDELETEPROGRAMPROC glDeleteProgram;
00280 extern PFNGLDELETEQUERIESPROC glDeleteQueries;
00281 extern PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00282 extern PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00283 extern PFNGLDELETESHADERPROC glDeleteShader;
00284 extern PFNGLDELETESTATESNVPROC glDeleteStatesNV;
00285 extern PFNGLDELETESYNCPROC glDeleteSync;
00286 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00287 extern PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
```

```
00288 extern PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00289 extern PFNGLDEPTHFUNCPROC glDepthFunc;
00290 extern PFNGLDEPTHMASKPROC glDepthMask;
00291 extern PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00292 extern PFNGLDEPTHRANGEFPROC glDepthRangef;
00293 extern PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00294 extern PFNGLDEPTHRANGEPROC glDepthRange;
00295 extern PFNGLDETACHSHADERPROC glDetachShader;
00296 extern PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00297 extern PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00298 extern PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00299 extern PFNGLDISABLEIPROC glDisablei;
00300 extern PFNGLDISABLEPROC glDisable;
00301 extern PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00302 extern PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00303 extern PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00304 extern PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00305 extern PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00306 extern PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00307 extern PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00308 extern PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00309 extern PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00310 extern PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00311 extern PFNGLDRAWARRAYSINSTANCEDEEXTPROC glDrawArraysInstancedEXT;
00312 extern PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00313 extern PFNGLDRAWARRAYSPROC glDrawArrays;
00314 extern PFNGLDRAWBUFFERPROC glDrawBuffer;
00315 extern PFNGLDRAWBUFFERSPROC glDrawBuffers;
00316 extern PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00317 extern PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00318 extern PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00319 extern PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00320 extern PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00321 extern PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00322 extern PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00323 extern PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00324 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
glDrawElementsInstancedBaseVertexBaseInstance;
00325 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00326 extern PFNGLDRAWELEMENTSINSTANCEDEEXTPROC glDrawElementsInstancedEXT;
00327 extern PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00328 extern PFNGLDRAWELEMENTSPROC glDrawElements;
00329 extern PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00330 extern PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00331 extern PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00332 extern PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00333 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00334 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00335 extern PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00336 extern PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00337 extern PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00338 extern PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00339 extern PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00340 extern PFNGLENABLEIPROC glEnablei;
00341 extern PFNGLENABLEPROC glEnable;
00342 extern PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00343 extern PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00344 extern PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00345 extern PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00346 extern PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00347 extern PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00348 extern PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00349 extern PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00350 extern PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00351 extern PFNGLENDQUERYPROC glEndQuery;
00352 extern PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00353 extern PFNGLFILLMODEPROC glFillMode;
00354 extern PFNGLFENCESYNCPROC glFenceSync;
00355 extern PFNGLFINISHPROC glFinish;
00356 extern PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00357 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00358 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00359 extern PFNGLFLUSHPROC glFlush;
00360 extern PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00361 extern PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00362 extern PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00363 extern PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00364 extern PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00365 extern PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00366 extern PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00367 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00368 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00369 extern PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00370 extern PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00371 extern PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00372 extern PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00373 extern PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
```



```
00374 extern PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00375 extern PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00376 extern PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00377 extern PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00378 extern PFNGLFRONTFACEPROC glFrontFace;
00379 extern PFNGLGENBUFFERSPROC glGenBuffers;
00380 extern PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00381 extern PFNGLGENERATEMULTITEXMIPMAEXTPROC glGenerateMultiTexMipmapEXT;
00382 extern PFNGLGENERATETEXTUREMIPMAEXTPROC glGenerateTextureMipmapEXT;
00383 extern PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00384 extern PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00385 extern PFNGLGENPATHSNVPROC glGenPathsNV;
00386 extern PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00387 extern PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00388 extern PFNGLGENQUERIESPROC glGenQueries;
00389 extern PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00390 extern PFNGLGENSAMPLERSPROC glGenSamplers;
00391 extern PFNGLGENTEXTURESPROC glGenTextures;
00392 extern PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00393 extern PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00394 extern PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00395 extern PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00396 extern PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00397 extern PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00398 extern PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00399 extern PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00400 extern PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00401 extern PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00402 extern PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00403 extern PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00404 extern PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00405 extern PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00406 extern PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00407 extern PFNGLGETBOOLEANI.VPROC glGetBooleani.v;
00408 extern PFNGLGETBOOLEANVPROC glGetBooleany;
00409 extern PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00410 extern PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00411 extern PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00412 extern PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00413 extern PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00414 extern PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00415 extern PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00416 extern PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00417 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00418 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00419 extern PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00420 extern PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00421 extern PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00422 extern PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00423 extern PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00424 extern PFNGLGETDOUBLEI.VEXTPROC glGetDoublei.vEXT;
00425 extern PFNGLGETDOUBLEI.VPROC glGetDoublei.v;
00426 extern PFNGLGETDOUBLEVPROC glGetDoublev;
00427 extern PFNGLGETERRORPROC glGetError;
00428 extern PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00429 extern PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00430 extern PFNGLGETFLOATI.VEXTPROC glGetFloati.vEXT;
00431 extern PFNGLGETFLOATI.VPROC glGetFloati.v;
00432 extern PFNGLGETFLOATVPROC glGetFloatv;
00433 extern PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00434 extern PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00435 extern PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00436 extern PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00437 extern PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00438 extern PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00439 extern PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00440 extern PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00441 extern PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00442 extern PFNGLGETINTEGER64I.VPROC glGetInteger64i.v;
00443 extern PFNGLGETINTEGER64VPROC glGetInteger64v;
00444 extern PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00445 extern PFNGLGETINTEGERI.VPROC glGetIntegeri.v;
00446 extern PFNGLGETINTEGERUI64I.VNVPROC glGetIntegerui64i.vNV;
00447 extern PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00448 extern PFNGLGETINTEGERVPROC glGetInteger;
00449 extern PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00450 extern PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00451 extern PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00452 extern PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00453 extern PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00454 extern PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00455 extern PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00456 extern PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00457 extern PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00458 extern PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00459 extern PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00460 extern PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
```

```

00461 extern PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00462 extern PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00463 extern PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00464 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00465 extern PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00466 extern PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00467 extern PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00468 extern PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00469 extern PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00470 extern PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00471 extern PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00472 extern PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00473 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC
glGetNamedFramebufferAttachmentParameterivEXT;
00474 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00475 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00476 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00477 extern PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00478 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00479 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00480 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00481 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00482 extern PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00483 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00484 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00485 extern PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00486 extern PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00487 extern PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00488 extern PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00489 extern PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00490 extern PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00491 extern PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00492 extern PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00493 extern PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00494 extern PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00495 extern PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00496 extern PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00497 extern PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00498 extern PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00499 extern PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00500 extern PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivARB;
00501 extern PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00502 extern PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00503 extern PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00504 extern PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00505 extern PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00506 extern PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00507 extern PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00508 extern PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00509 extern PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00510 extern PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00511 extern PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00512 extern PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00513 extern PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00514 extern PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00515 extern PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00516 extern PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00517 extern PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00518 extern PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00519 extern PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00520 extern PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00521 extern PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00522 extern PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00523 extern PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00524 extern PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00525 extern PFNGLGETPOINTERI_VEXTPROC glGetPointerivEXT;
00526 extern PFNGLGETPOINTERVPROC glGetPointerv;
00527 extern PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00528 extern PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00529 extern PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00530 extern PFNGLGETPROGRAMIVPROC glGetProgramiv;
00531 extern PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00532 extern PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00533 extern PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00534 extern PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00535 extern PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00536 extern PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00537 extern PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00538 extern PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00539 extern PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00540 extern PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00541 extern PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00542 extern PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00543 extern PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00544 extern PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00545 extern PFNGLGETQUERYIVPROC glGetQueryiv;
00546 extern PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;

```



```
00547 extern PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00548 extern PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00549 extern PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00550 extern PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00551 extern PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00552 extern PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00553 extern PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00554 extern PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00555 extern PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00556 extern PFNGLGETSHADERIVPROC glGetShaderiv;
00557 extern PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00558 extern PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00559 extern PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00560 extern PFNGLGETSTRINGIPROC glGetStringi;
00561 extern PFNGLGETSTRINGPROC getString;
00562 extern PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00563 extern PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00564 extern PFNGLGETSYNCCIVPROC glGetSynciv;
00565 extern PFNGLGETTEXIMAGEPROC glGetTexImage;
00566 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00567 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00568 extern PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00569 extern PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00570 extern PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00571 extern PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00572 extern PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00573 extern PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00574 extern PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00575 extern PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00576 extern PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00577 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00578 extern PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00579 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00580 extern PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00581 extern PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00582 extern PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00583 extern PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00584 extern PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00585 extern PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00586 extern PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00587 extern PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00588 extern PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00589 extern PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00590 extern PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00591 extern PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00592 extern PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00593 extern PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00594 extern PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00595 extern PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00596 extern PFNGLGETUNIFORMDVPROC glGetUniformdv;
00597 extern PFNGLGETUNIFORMFVPROC glGetUniformfv;
00598 extern PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00599 extern PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00600 extern PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00601 extern PFNGLGETUNIFORMIVPROC glGetUniformiv;
00602 extern PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00603 extern PFNGLGETUNIFORMSUBROUTINEIUIVPROC glGetUniformSubroutineuiv;
00604 extern PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00605 extern PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00606 extern PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00607 extern PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00608 extern PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00609 extern PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00610 extern PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00611 extern PFNGLGETVERTEXARRAYIUIVPROC glGetVertexArrayiui_v;
00612 extern PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00613 extern PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointer_vEXT;
00614 extern PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00615 extern PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00616 extern PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00617 extern PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00618 extern PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00619 extern PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribldv;
00620 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLi64vNV;
00621 extern PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00622 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00623 extern PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00624 extern PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00625 extern PFNGLHINTPROC glHint;
00626 extern PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00627 extern PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00628 extern PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00629 extern PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00630 extern PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00631 extern PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00632 extern PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00633 extern PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
```

```

00634 extern PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00635 extern PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00636 extern PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00637 extern PFNGLISBUFFERPROC glIsBuffer;
00638 extern PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00639 extern PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00640 extern PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00641 extern PFNGLISENABLEDIPROC glIsEnabledi;
00642 extern PFNGLISENABLEDPROC glIsEnabled;
00643 extern PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00644 extern PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00645 extern PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00646 extern PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00647 extern PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00648 extern PFNGLISPATHNVPROC glIsPathNV;
00649 extern PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00650 extern PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00651 extern PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00652 extern PFNGLISPROGRAMPROC glIsProgram;
00653 extern PFNGLISQUERYPROC glIsQuery;
00654 extern PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00655 extern PFNGLISSAMPLERPROC glIsSampler;
00656 extern PFNGLISSHADERPROC glIsShader;
00657 extern PFNGLISSTATENVPROC glIsStateNV;
00658 extern PFNGLISSYNCPROC glIsSync;
00659 extern PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00660 extern PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00661 extern PFNGLISTEXTUREPROC glIsTexture;
00662 extern PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00663 extern PFNGLISVERTEXARRAYPROC glIsVertexArray;
00664 extern PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00665 extern PFNGLLINEWIDTHPROC glLineWidth;
00666 extern PFNGLLINKPROGRAMPROC glLinkProgram;
00667 extern PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00668 extern PFNGLLOGICOPPROC glLogicOp;
00669 extern PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00670 extern PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00671 extern PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00672 extern PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00673 extern PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00674 extern PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00675 extern PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00676 extern PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00677 extern PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00678 extern PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00679 extern PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00680 extern PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00681 extern PFNGLMAPBUFFERPROC glMapBuffer;
00682 extern PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00683 extern PFNGLMAPNAMEDBUFFEREEXTPROC glMapNamedBufferEXT;
00684 extern PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00685 extern PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00686 extern PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00687 extern PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00688 extern PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00689 extern PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00690 extern PFNGLMATRIXLOADDEXTPROC glMatrixLoaddeEXT;
00691 extern PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00692 extern PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00693 extern PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00694 extern PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00695 extern PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00696 extern PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00697 extern PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00698 extern PFNGLMATRIXMULTDEXTPROC glMatrixMultdeEXT;
00699 extern PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00700 extern PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00701 extern PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00702 extern PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00703 extern PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00704 extern PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00705 extern PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00706 extern PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00707 extern PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00708 extern PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00709 extern PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00710 extern PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00711 extern PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00712 extern PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00713 extern PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00714 extern PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00715 extern PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00716 extern PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00717 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00718 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00719 extern PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00720 extern PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;

```

```
00721 extern PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00722 extern PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00723 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00724 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00725 extern PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00726 extern PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00727 extern PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00728 extern PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00729 extern PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00730 extern PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00731 extern PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00732 extern PFNGLMULTITEXENVIEXTPROC glMultiTexEnvfEXT;
00733 extern PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00734 extern PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00735 extern PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00736 extern PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00737 extern PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00738 extern PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00739 extern PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00740 extern PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00741 extern PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00742 extern PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00743 extern PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00744 extern PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00745 extern PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00746 extern PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameteriivEXT;
00747 extern PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameteriuiEXT;
00748 extern PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00749 extern PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00750 extern PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00751 extern PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00752 extern PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00753 extern PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00754 extern PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00755 extern PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00756 extern PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00757 extern PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00758 extern PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00759 extern PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00760 extern PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00761 extern PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00762 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00763 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00764 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00765 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00766 extern PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00767 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00768 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00769 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00770 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00771 extern PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00772 extern PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00773 extern PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00774 extern PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00775 extern PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00776 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00777 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00778 extern PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00779 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00780 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00781 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00782 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00783 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00784 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00785 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00786 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00787 extern PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00788 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00789 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00790 extern PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00791 extern PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00792 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00793 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00794 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00795 extern PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00796 extern PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00797 extern PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00798 extern PFNGLOBJECTLABELPROC glObjectLabel;
00799 extern PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00800 extern PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00801 extern PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00802 extern PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00803 extern PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00804 extern PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00805 extern PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00806 extern PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
```

```
00807 extern PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00808 extern PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00809 extern PFNGLPATHGLYPHNSVPROC glPathGlyphsNV;
00810 extern PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00811 extern PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00812 extern PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00813 extern PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00814 extern PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00815 extern PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00816 extern PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00817 extern PFNGLPATHSTRINGNVPROC glPathStringNV;
00818 extern PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00819 extern PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00820 extern PFNGLPAUSETRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00821 extern PFNGLPIXELSTOREFPROC glPixelStoref;
00822 extern PFNGLPIXELSTOREIPROC glPixelStorei;
00823 extern PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00824 extern PFNGLPOINTPARAMETERFPROC glPointParameterf;
00825 extern PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00826 extern PFNGLPOINTPARAMETERIPROC glPointParameteri;
00827 extern PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00828 extern PFNGLPOINTSIZEPROC glPointSize;
00829 extern PFNGLPOLYGONMODEPROC glPolygonMode;
00830 extern PFNGLPOLYGONOFFSETCLEMPROC glPolygonOffsetClampEXT;
00831 extern PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00832 extern PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00833 extern PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00834 extern PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00835 extern PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00836 extern PFNGLPROGRAMBINARYPROC glProgramBinary;
00837 extern PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00838 extern PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00839 extern PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00840 extern PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00841 extern PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00842 extern PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00843 extern PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00844 extern PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00845 extern PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00846 extern PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00847 extern PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00848 extern PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00849 extern PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00850 extern PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00851 extern PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00852 extern PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00853 extern PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00854 extern PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00855 extern PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00856 extern PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00857 extern PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00858 extern PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00859 extern PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00860 extern PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00861 extern PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00862 extern PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00863 extern PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00864 extern PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00865 extern PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00866 extern PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00867 extern PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00868 extern PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00869 extern PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00870 extern PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00871 extern PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00872 extern PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00873 extern PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00874 extern PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00875 extern PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00876 extern PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00877 extern PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00878 extern PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00879 extern PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00880 extern PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00881 extern PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00882 extern PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00883 extern PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00884 extern PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00885 extern PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00886 extern PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00887 extern PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00888 extern PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00889 extern PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00890 extern PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00891 extern PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00892 extern PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00893 extern PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
```



```
00894 extern PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00895 extern PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00896 extern PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00897 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00898 extern PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00899 extern PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00900 extern PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00901 extern PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00902 extern PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00903 extern PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00904 extern PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00905 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00906 extern PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00907 extern PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00908 extern PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00909 extern PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00910 extern PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00911 extern PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00912 extern PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00913 extern PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00914 extern PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00915 extern PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00916 extern PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00917 extern PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00918 extern PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00919 extern PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00920 extern PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00921 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00922 extern PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00923 extern PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00924 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00925 extern PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00926 extern PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00927 extern PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00928 extern PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00929 extern PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00930 extern PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00931 extern PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00932 extern PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00933 extern PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00934 extern PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00935 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00936 extern PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00937 extern PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00938 extern PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00939 extern PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00940 extern PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00941 extern PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00942 extern PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00943 extern PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00944 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00945 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00946 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00947 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00948 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00949 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00950 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00951 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00952 extern PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00953 extern PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00954 extern PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00955 extern PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00956 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00957 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00958 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00959 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00960 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00961 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00962 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00963 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00964 extern PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00965 extern PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00966 extern PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00967 extern PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00968 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00969 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00970 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00971 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00972 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00973 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00974 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00975 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00976 extern PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00977 extern PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00978 extern PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00979 extern PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC glPushClientAttribDefaultEXT;
00980 extern PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
```

```
00981 extern PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00982 extern PFNGLQUERYCOUNTERPROC glQueryCounter;
00983 extern PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00984 extern PFNGLREADBUFFERPROC glReadBuffer;
00985 extern PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00986 extern PFNGLREADNPIXELSPROC glReadnPixels;
00987 extern PFNGLREADPIXELSPROC glReadPixels;
00988 extern PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
00989 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00990 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00991 extern PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00992 extern PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
00993 extern PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
00994 extern PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
00995 extern PFNGLSAMPLEMASKIPROC glSampleMaski;
00996 extern PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
00997 extern PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
00998 extern PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
00999 extern PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01000 extern PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01001 extern PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01002 extern PFNGLSCISSORARRAYVPROC glScissorArrayv;
01003 extern PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01004 extern PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01005 extern PFNGLSCISSORPROC glScissor;
01006 extern PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01007 extern PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01008 extern PFNGLSHADERBINARYPROC glShaderBinary;
01009 extern PFNGLSHADERSOURCEPROC glShaderSource;
01010 extern PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01011 extern PFNGLSIGNALVKFENCEENVPROC glSignalVkFenceNV;
01012 extern PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01013 extern PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01014 extern PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01015 extern PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01016 extern PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01017 extern PFNGLSTENCILFUNCPROC glStencilFunc;
01018 extern PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01019 extern PFNGLSTENCILMASKPROC glStencilMask;
01020 extern PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01021 extern PFNGLSTENCILOPPROC glStencilOp;
01022 extern PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01023 extern PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01024 extern PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01025 extern PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01026 extern PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01027 extern PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01028 extern PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01029 extern PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01030 extern PFNGLTEXBUFFERARBPROC glTexBufferARB;
01031 extern PFNGLTEXBUFFERPROC glTexBuffer;
01032 extern PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01033 extern PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01034 extern PFNGLTEXIMAGE1DPROC glTexImage1D;
01035 extern PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01036 extern PFNGLTEXIMAGE2DPROC glTexImage2D;
01037 extern PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01038 extern PFNGLTEXIMAGE3DPROC glTexImage3D;
01039 extern PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01040 extern PFNGLTEXPARAMETERFPROC glTexParameterf;
01041 extern PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01042 extern PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01043 extern PFNGLTEXPARAMETERIPROC glTexParameterI;
01044 extern PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01045 extern PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01046 extern PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01047 extern PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01048 extern PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01049 extern PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01050 extern PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01051 extern PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01052 extern PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01053 extern PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01054 extern PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01055 extern PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01056 extern PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01057 extern PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01058 extern PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01059 extern PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01060 extern PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01061 extern PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01062 extern PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01063 extern PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01064 extern PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01065 extern PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01066 extern PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01067 extern PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
```

```
01068 extern PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01069 extern PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameteriivEXT;
01070 extern PFNGLTEXTUREPARAMETERIIVPROC glTextureParameteriiv;
01071 extern PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01072 extern PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameteriuiEXT;
01073 extern PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameteriui;
01074 extern PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01075 extern PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01076 extern PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01077 extern PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DTEXT;
01078 extern PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01079 extern PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DTEXT;
01080 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01081 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01082 extern PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01083 extern PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DTEXT;
01084 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01085 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01086 extern PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01087 extern PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DTEXT;
01088 extern PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01089 extern PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DTEXT;
01090 extern PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01091 extern PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DTEXT;
01092 extern PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01093 extern PFNGLTEXTUREVIEWPROC glTextureView;
01094 extern PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01095 extern PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01096 extern PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01097 extern PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01098 extern PFNGLUNIFORM1DPROC glUniform1d;
01099 extern PFNGLUNIFORM1DVPROC glUniform1dv;
01100 extern PFNGLUNIFORM1FPROC glUniform1f;
01101 extern PFNGLUNIFORM1FVPROC glUniform1fv;
01102 extern PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01103 extern PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01104 extern PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01105 extern PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01106 extern PFNGLUNIFORM1IPROC glUniform1i;
01107 extern PFNGLUNIFORM1IVPROC glUniform1iv;
01108 extern PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01109 extern PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01110 extern PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01111 extern PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01112 extern PFNGLUNIFORM1UIPROC glUniform1ui;
01113 extern PFNGLUNIFORM1UIVPROC glUniform1uiv;
01114 extern PFNGLUNIFORM2DPROC glUniform2d;
01115 extern PFNGLUNIFORM2DVPROC glUniform2dv;
01116 extern PFNGLUNIFORM2FPROC glUniform2f;
01117 extern PFNGLUNIFORM2FVPROC glUniform2fv;
01118 extern PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01119 extern PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01120 extern PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01121 extern PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01122 extern PFNGLUNIFORM2IPROC glUniform2i;
01123 extern PFNGLUNIFORM2IVPROC glUniform2iv;
01124 extern PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01125 extern PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01126 extern PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01127 extern PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01128 extern PFNGLUNIFORM2UIPROC glUniform2ui;
01129 extern PFNGLUNIFORM2UIVPROC glUniform2uiv;
01130 extern PFNGLUNIFORM3DPROC glUniform3d;
01131 extern PFNGLUNIFORM3DVPROC glUniform3dv;
01132 extern PFNGLUNIFORM3FPROC glUniform3f;
01133 extern PFNGLUNIFORM3FVPROC glUniform3fv;
01134 extern PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01135 extern PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01136 extern PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01137 extern PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01138 extern PFNGLUNIFORM3IPROC glUniform3i;
01139 extern PFNGLUNIFORM3IVPROC glUniform3iv;
01140 extern PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01141 extern PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01142 extern PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01143 extern PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01144 extern PFNGLUNIFORM3UIPROC glUniform3ui;
01145 extern PFNGLUNIFORM3UIVPROC glUniform3uiv;
01146 extern PFNGLUNIFORM4DPROC glUniform4d;
01147 extern PFNGLUNIFORM4DVPROC glUniform4dv;
01148 extern PFNGLUNIFORM4FPROC glUniform4f;
01149 extern PFNGLUNIFORM4FVPROC glUniform4fv;
01150 extern PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01151 extern PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01152 extern PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01153 extern PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01154 extern PFNGLUNIFORM4IPROC glUniform4i;
```

```

01155 extern PFNGLUNIFORM4IVPROC glUniform4iv;
01156 extern PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01157 extern PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01158 extern PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01159 extern PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01160 extern PFNGLUNIFORM4UIPROC glUniform4ui;
01161 extern PFNGLUNIFORM4UIVPROC glUniform4uiv;
01162 extern PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01163 extern PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01164 extern PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01165 extern PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01166 extern PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01167 extern PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01168 extern PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01169 extern PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01170 extern PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01171 extern PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01172 extern PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01173 extern PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01174 extern PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01175 extern PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01176 extern PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01177 extern PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01178 extern PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01179 extern PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01180 extern PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01181 extern PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01182 extern PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01183 extern PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01184 extern PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01185 extern PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01186 extern PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01187 extern PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01188 extern PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01189 extern PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01190 extern PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01191 extern PFNGLUSEPROGRAMPROC glUseProgram;
01192 extern PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01193 extern PFNGLUSESHADERPROGRAMEXTPROC glUseProgramProgramEXT;
01194 extern PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01195 extern PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01196 extern PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01197 extern PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01198 extern PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01199 extern PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01200 extern PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01201 extern PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01202 extern PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01203 extern PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01204 extern PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01205 extern PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01206 extern PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01207 extern PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01208 extern PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01209 extern PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01210 extern PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01211 extern PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01212 extern PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01213 extern PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01214 extern PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01215 extern PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01216 extern PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01217 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01218 extern PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01219 extern PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01220 extern PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01221 extern PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01222 extern PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01223 extern PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01224 extern PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01225 extern PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01226 extern PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01227 extern PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01228 extern PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01229 extern PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01230 extern PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01231 extern PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01232 extern PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01233 extern PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01234 extern PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01235 extern PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01236 extern PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01237 extern PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01238 extern PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01239 extern PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01240 extern PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01241 extern PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;

```



```
01242 extern PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01243 extern PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01244 extern PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01245 extern PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01246 extern PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01247 extern PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01248 extern PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01249 extern PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01250 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01251 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nubv;
01252 extern PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01253 extern PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01254 extern PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01255 extern PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01256 extern PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01257 extern PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01258 extern PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01259 extern PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01260 extern PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01261 extern PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01262 extern PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01263 extern PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01264 extern PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01265 extern PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01266 extern PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01267 extern PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01268 extern PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01269 extern PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01270 extern PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01271 extern PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01272 extern PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01273 extern PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01274 extern PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01275 extern PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01276 extern PFNGLVERTEXATTRIBI4BPROC glVertexAttribI4bv;
01277 extern PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01278 extern PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01279 extern PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01280 extern PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01281 extern PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01282 extern PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01283 extern PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01284 extern PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01285 extern PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01286 extern PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01287 extern PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01288 extern PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01289 extern PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01290 extern PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01291 extern PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01292 extern PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01293 extern PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01294 extern PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01295 extern PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01296 extern PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01297 extern PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01298 extern PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01299 extern PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01300 extern PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01301 extern PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01302 extern PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01303 extern PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01304 extern PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01305 extern PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01306 extern PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01307 extern PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01308 extern PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01309 extern PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01310 extern PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01311 extern PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01312 extern PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01313 extern PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01314 extern PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01315 extern PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01316 extern PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01317 extern PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01318 extern PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01319 extern PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01320 extern PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01321 extern PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01322 extern PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01323 extern PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01324 extern PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01325 extern PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01326 extern PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01327 extern PFNGLVIEWPORTARRAYVPROC glVertexArrayv;
01328 extern PFNGLVIEWPORTINDEXEDFPROC glVertexIndexedf;
```

```

01329 extern PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexedfv;
01330 extern PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01331 extern PFNGLVIEWPORTPROC glViewport;
01332 extern PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01333 extern PFNGLWAITSYNCPROC glWaitSync;
01334 extern PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01335 extern PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01336 extern PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01337 #endif
01338
01340
01344 namespace gg
01345 {
01349     enum BindingPoints
01350     {
01351         LightBindingPoint = 0,
01352         MaterialBindingPoint,
01353     };
01354
01358     extern GLint ggBufferAlignment;
01359
01366     extern void ggInit();
01367
01377     extern void _ggError(const std::string& name = "", unsigned int line = 0);
01378
01389 #if defined(DEBUG)
01390 # define ggError() gg::_ggError(__FILE__, __LINE__)
01391 #else
01392 # define ggError()
01393 #endif
01394
01404     extern void _ggFBOError(const std::string& name = "", unsigned int line = 0);
01405
01416 #if defined(DEBUG)
01417 # define ggFBOError() gg::_ggFBOError(__FILE__, __LINE__)
01418 #else
01419 # define ggFBOError()
01420 #endif
01421
01429     inline void ggCross(GLfloat* c, const GLfloat* a, const GLfloat* b)
01430     {
01431         c[0] = a[1] * b[2] - a[2] * b[1];
01432         c[1] = a[2] * b[0] - a[0] * b[2];
01433         c[2] = a[0] * b[1] - a[1] * b[0];
01434     }
01435
01443     inline GLfloat ggDot3(const GLfloat* a, const GLfloat* b)
01444     {
01445         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
01446     }
01447
01454     inline GLfloat ggLength3(const GLfloat* a)
01455     {
01456         return sqrtf(ggDot3(a, a));
01457     }
01458
01466     inline GLfloat ggDistance3(const GLfloat* a, const GLfloat* b)
01467     {
01468         const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], 0.0f };
01469         return ggLength3(c);
01470     }
01471
01478     inline void ggNormalize3(const GLfloat* a, GLfloat* b)
01479     {
01480         const GLfloat l{ ggLength3(a) };
01481         assert(l > 0.0f);
01482         b[0] = a[0] / l;
01483         b[1] = a[1] / l;
01484         b[2] = a[2] / l;
01485     }
01486
01492     inline void ggNormalize3(GLfloat* a)
01493     {
01494         const GLfloat l{ ggLength3(a) };
01495         assert(l > 0.0f);
01496         a[0] /= l;
01497         a[1] /= l;
01498         a[2] /= l;
01499     }
01500
01508     inline GLfloat ggDot4(const GLfloat* a, const GLfloat* b)
01509     {
01510         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3];
01511     }
01512
01519     inline GLfloat ggLength4(const GLfloat* a)
01520     {

```

```

01521     return sqrtf(ggDot4(a, a));
01522 }
01523
01531 inline GLfloat ggDistance4(const GLfloat* a, const GLfloat* b)
01532 {
01533     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], a[3] - b[3] };
01534     return ggLength4(c);
01535 }
01536
01543 inline void ggNormalize4(const GLfloat* a, GLfloat* b)
01544 {
01545     const GLfloat l { ggLength4(a) };
01546     assert(l > 0.0f);
01547     b[0] = a[0] / l;
01548     b[1] = a[1] / l;
01549     b[2] = a[2] / l;
01550     b[3] = a[3] / l;
01551 }
01552
01558 inline void ggNormalize4(GLfloat* a)
01559 {
01560     const GLfloat l { ggLength4(a) };
01561     assert(l > 0.0f);
01562     a[0] /= l;
01563     a[1] /= l;
01564     a[2] /= l;
01565     a[3] /= l;
01566 }
01567
01571 class GgVector : public std::array<GLfloat, 4>
01572 {
01573 public:
01574
01578     GgVector() = default;
01579
01588     constexpr GgVector(GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3) :
01589         std::array<GLfloat, 4>{ v0, v1, v2, v3 }
01590     {
01591     }
01592
01598     constexpr GgVector(const GLfloat* a) :
01599         GgVector{ a[0], a[1], a[2], a[3] }
01600     {
01601     }
01602
01608     constexpr GgVector(GLfloat c) :
01609         GgVector{ c, c, c, c }
01610     {
01611     }
01612
01618     GgVector(const GgVector& v) = default;
01619
01625     GgVector(GgVector&& v) = default;
01626
01630     ~GgVector() = default;
01631
01635     GgVector& operator=(const GgVector& v) = default;
01636
01640     GgVector& operator=(GgVector&& v) = default;
01641
01648     inline GgVector operator+(const GgVector& v) const
01649     {
01650         return GgVector{ (*this)[0] + v[0], (*this)[1] + v[1], (*this)[2] + v[2], (*this)[3] + v[3] };
01651     }
01652
01659     inline GgVector operator+(GLfloat c) const
01660     {
01661         return *this + GgVector(c);
01662     }
01663
01670     inline GgVector& operator+=(const GgVector& v)
01671     {
01672         *this = *this + v;
01673         return *this;
01674     }
01675
01682     inline GgVector& operator+=(GLfloat c)
01683     {
01684         *this = *this + c;
01685         return *this;
01686     }
01687
01694     inline GgVector operator-(const GgVector& v) const
01695     {
01696         return GgVector{ (*this)[0] - v[0], (*this)[1] - v[1], (*this)[2] - v[2], (*this)[3] - v[3] };
01697     }
01698

```

```

01705 inline GgVector operator-(GLfloat c) const
01706 {
01707     return *this - GgVector(c);
01708 }
01709
01716 inline GgVector& operator-=(const GgVector& v)
01717 {
01718     *this = *this - v;
01719     return *this;
01720 }
01721
01728 inline GgVector& operator-=(GLfloat c)
01729 {
01730     *this = *this - c;
01731     return *this;
01732 }
01733
01740 inline GgVector operator*(const GgVector& v) const
01741 {
01742     return GgVector{ (*this)[0] * v[0], (*this)[1] * v[1], (*this)[2] * v[2], (*this)[3] * v[3] };
01743 }
01744
01751 inline GgVector operator*(GLfloat c) const
01752 {
01753     return *this * GgVector(c);
01754 }
01755
01761 inline GgVector& operator*=(const GgVector& v)
01762 {
01763     *this = *this * v;
01764     return *this;
01765 }
01766
01773 inline GgVector& operator*=(GLfloat c)
01774 {
01775     *this = *this * c;
01776     return *this;
01777 }
01778
01785 inline GgVector operator/(const GgVector& v) const
01786 {
01787     return GgVector{ (*this)[0] / v[0], (*this)[1] / v[1], (*this)[2] / v[2], (*this)[3] / v[3] };
01788 }
01789
01796 inline GgVector operator/(GLfloat c) const
01797 {
01798     return *this / GgVector(c);
01799 }
01800
01807 inline GgVector& operator/=(GgVector& v)
01808 {
01809     *this = *this / v;
01810     return *this;
01811 }
01812
01819 inline GgVector& operator/=(GLfloat c)
01820 {
01821     *this = *this / c;
01822     return *this;
01823 }
01824
01831 inline GLfloat dot3(const GgVector& v) const
01832 {
01833     return ggDot3(data(), v.data());
01834 }
01835
01841 inline GLfloat length3() const
01842 {
01843     return ggLength3(data());
01844 }
01845
01852 inline GLfloat distance3(const GgVector& v) const
01853 {
01854     return ggDistance3(data(), v.data());
01855 }
01856
01862 inline GgVector normalize3() const
01863 {
01864     GgVector b;
01865     ggNormalize3(data(), b.data());
01866     return b;
01867 }
01868
01875 inline GLfloat dot4(const GgVector& v) const
01876 {
01877     return ggDot4(data(), v.data());
01878 }

```

```

01879
01885     inline GLfloat length4() const
01886     {
01887         return ggLength4(data());
01888     }
01889
01896     inline GLfloat distance4(const GgVector& v) const
01897     {
01898         return ggDistance4(data(), v.data());
01899     }
01900
01906     inline GgVector normalize4() const
01907     {
01908         GgVector b;
01909         ggNormalize4(data(), b.data());
01910         return b;
01911     }
01912 };
01913
01920     inline const GgVector& operator+(const GgVector& v)
01921     {
01922         return v;
01923     }
01924
01932     inline GgVector operator+(GLfloat a, const GgVector& b)
01933     {
01934         return GgVector{ a + b[0], a + b[1], a + b[2], a + b[3] };
01935     }
01936
01943     inline const GgVector operator-(const GgVector& v)
01944     {
01945         return GgVector{ -v[0], -v[1], -v[2], -v[3] };
01946     }
01947
01955     inline GgVector operator-(GLfloat a, const GgVector& b)
01956     {
01957         return GgVector{ a - b[0], a - b[1], a - b[2], a - b[3] };
01958     }
01959
01967     inline GgVector operator*(GLfloat a, const GgVector& b)
01968     {
01969         return GgVector{ a * b[0], a * b[1], a * b[2], a * b[3] };
01970     }
01971
01979     inline GgVector operator/(GLfloat a, const GgVector& b)
01980     {
01981         return GgVector{ a / b[0], a / b[1], a / b[2], a / b[3] };
01982     }
01983
01994     inline GgVector ggCross(const GgVector& a, const GgVector& b)
01995     {
01996         GgVector c;
01997         ggCross(c.data(), a.data(), b.data());
01998         return c;
01999     }
02000
02008     inline GLfloat ggDot3(const GgVector& a, const GgVector& b)
02009     {
02010         return ggDot3(a.data(), b.data());
02011     }
02012
02019     inline GLfloat ggLength3(const GgVector& a)
02020     {
02021         return ggLength3(a.data());
02022     }
02023
02031     inline GLfloat ggDistance3(const GgVector& a, const GgVector& b)
02032     {
02033         return ggDistance3(a.data(), b.data());
02034     }
02035
02045     inline GgVector ggNormalize3(const GgVector& a)
02046     {
02047         GgVector b;
02048         ggNormalize3(a.data(), b.data());
02049         return b;
02050     }
02051
02060     inline void ggNormalize3(GgVector* a)
02061     {
02062         ggNormalize3(a->data());
02063     }
02064
02072     inline GLfloat ggDot4(const GgVector& a, const GgVector& b)
02073     {
02074         return ggDot4(a.data(), b.data());
02075     }

```

```

02076
02083 inline GLfloat ggLength4(const GgVector& a)
02084 {
02085     return ggLength4(a.data());
02086 }
02087
02095 inline GLfloat ggDistance4(const GgVector& a, const GgVector& b)
02096 {
02097     return ggDistance4(a.data(), b.data());
02098 }
02099
02106 inline GgVector ggNormalize4(const GgVector& a)
02107 {
02108     GgVector b;
02109     ggNormalize4(a.data(), b.data());
02110     return b;
02111 }
02112
02118 inline void ggNormalize4(GgVector* a)
02119 {
02120     ggNormalize4(a->data());
02121 }
02122
02126 class GgMatrix : public std::array<GLfloat, 16>
02127 {
02128     // 行列 a とベクトル b の積をベクトル c に格納する
02129     void projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02130
02131     // 行列 a と行列 b の積を行列 c に格納する
02132     void multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02133
02134 public:
02135
02139     GgMatrix() = default;
02140
02161     constexpr GgMatrix(
02162         GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03,
02163         GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13,
02164         GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23,
02165         GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33
02166     ) :
02167         std::array<GLfloat, 16>{ m00, m01, m02, m03, m10, m11, m12, m13, m20, m21, m22, m23, m30, m31,
02168         m32, m33 }
02169     {
02170     }
02176     constexpr GgMatrix(const GLfloat* a) :
02177         GgMatrix{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8], a[9], a[10], a[11], a[12],
02178         a[13], a[14], a[15] }
02179     {
02180     }
02186     constexpr GgMatrix(GLfloat c) :
02187         GgMatrix{ c, c, c, c, c, c, c, c, c, c, c, c, c, c, c, c }
02188     {
02189     }
02190
02196     GgMatrix(const GgMatrix& m) = default;
02197
02203     GgMatrix(GgMatrix&& m) = default;
02204
02208     ~GgMatrix() = default;
02209
02213     GgMatrix& operator=(const GgMatrix& m) = default;
02214
02218     GgMatrix& operator=(GgMatrix&& m) = default;
02219
02226     GgMatrix& operator=(const GLfloat* a)
02227     {
02228         *this = GgMatrix(a);
02229         return *this;
02230     }
02231
02238     GgMatrix operator+(const GLfloat* a) const
02239     {
02240         GgMatrix t;
02241         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] + a[i];
02242         return t;
02243     }
02244
02251     GgMatrix operator+(const GgMatrix& m) const
02252     {
02253         return operator+(m.data());
02254     }
02255
02262     GgMatrix& operator+=(const GLfloat* a)
02263     {

```

```

02264         for (std::size_t i = 0; i < size(); ++i) data()[i] += a[i];
02265         return *this;
02266     }
02267
02274     GgMatrix& operator+=(const GgMatrix& m)
02275     {
02276         return operator+=(m.data());
02277     }
02278
02285     GgMatrix operator-(const GLfloat* a) const
02286     {
02287         GgMatrix t;
02288         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] - a[i];
02289         return t;
02290     }
02291
02298     GgMatrix operator-(const GgMatrix& m) const
02299     {
02300         return operator-(m.data());
02301     }
02302
02309     GgMatrix& operator-=(const GLfloat* a)
02310     {
02311         for (std::size_t i = 0; i < size(); ++i) data()[i] -= a[i];
02312         return *this;
02313     }
02314
02321     GgMatrix& operator-=(const GgMatrix& m)
02322     {
02323         return operator-=(m.data());
02324     }
02325
02332     GgMatrix operator*(const GLfloat* a) const
02333     {
02334         GgMatrix t;
02335         multiply(t.data(), data(), a);
02336         return t;
02337     }
02338
02345     GgMatrix operator*(const GgMatrix& m) const
02346     {
02347         return operator*(m.data());
02348     }
02349
02356     GgMatrix& operator*=(const GLfloat* a)
02357     {
02358         *this = operator*(a);
02359         return *this;
02360     }
02361
02368     GgMatrix& operator*=(const GgMatrix& m)
02369     {
02370         return operator*=(m.data());
02371     }
02372
02379     GgMatrix operator/(const GLfloat* a) const
02380     {
02381         GgMatrix i, t;
02382         i.loadInvert(a);
02383         multiply(t.data(), data(), i.data());
02384         return t;
02385     }
02386
02393     GgMatrix operator/(const GgMatrix& m) const
02394     {
02395         return operator/(m.data());
02396     }
02397
02404     GgMatrix& operator/=(const GLfloat* a)
02405     {
02406         *this = operator/(a);
02407         return *this;
02408     }
02409
02416     GgMatrix& operator/=(const GgMatrix& m)
02417     {
02418         return operator/=(m.data());
02419     }
02420
02424     GgMatrix& loadIdentity();
02425
02435     GgMatrix& loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02436
02443     GgMatrix& loadTranslate(const GLfloat* t)
02444     {
02445         return loadTranslate(t[0], t[1], t[2]);
02446     }

```

```

02447
02454 GgMatrix& loadTranslate(const GgVector& t)
02455 {
02456     return loadTranslate(t[0], t[1], t[2], t[3]);
02457 }
02458
02468 GgMatrix& loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02469
02476 GgMatrix& loadScale(const GLfloat* s)
02477 {
02478     return loadScale(s[0], s[1], s[2]);
02479 }
02480
02487 GgMatrix& loadScale(const GgVector& s)
02488 {
02489     return loadScale(s[0], s[1], s[2], s[3]);
02490 }
02491
02498 GgMatrix& loadRotateX(GLfloat a);
02499
02506 GgMatrix& loadRotateY(GLfloat a);
02507
02514 GgMatrix& loadRotateZ(GLfloat a);
02515
02525 GgMatrix& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
02526
02534 GgMatrix& loadRotate(const GLfloat* r, GLfloat a)
02535 {
02536     return loadRotate(r[0], r[1], r[2], a);
02537 }
02538
02546 GgMatrix& loadRotate(const GgVector& r, GLfloat a)
02547 {
02548     return loadRotate(r[0], r[1], r[2], a);
02549 }
02550
02557 GgMatrix& loadRotate(const GLfloat* r)
02558 {
02559     return loadRotate(r[0], r[1], r[2], r[3]);
02560 }
02561
02568 GgMatrix& loadRotate(const GgVector& r)
02569 {
02570     return loadRotate(r[0], r[1], r[2], r[3]);
02571 }
02572
02587 GgMatrix& loadLookat(GLfloat ex, GLfloat ey, GLfloat ez,
02588     GLfloat tx, GLfloat ty, GLfloat tz,
02589     GLfloat ux, GLfloat uy, GLfloat uz);
02590
02599 GgMatrix& loadLookat(const GLfloat* e, const GLfloat* t, const GLfloat* u)
02600 {
02601     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02602 }
02603
02612 GgMatrix& loadLookat(const GgVector& e, const GgVector& t, const GgVector& u)
02613 {
02614     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02615 }
02616
02628 GgMatrix& loadOrthogonal(GLfloat left, GLfloat right,
02629     GLfloat bottom, GLfloat top,
02630     GLfloat zNear, GLfloat zFar);
02631
02643 GgMatrix& loadFrustum(GLfloat left, GLfloat right,
02644     GLfloat bottom, GLfloat top,
02645     GLfloat zNear, GLfloat zFar);
02646
02656 GgMatrix& loadPerspective(GLfloat fovy, GLfloat aspect,
02657     GLfloat zNear, GLfloat zFar);
02658
02665 GgMatrix& loadTranspose(const GLfloat* a);
02666
02673 GgMatrix& loadTranspose(const GgMatrix& m)
02674 {
02675     return loadTranspose(m.data());
02676 }
02677
02684 GgMatrix& loadInvert(const GLfloat* a);
02685
02692 GgMatrix& loadInvert(const GgMatrix& m)
02693 {
02694     return loadInvert(m.data());
02695 }
02696
02703 GgMatrix& loadNormal(const GLfloat* a);
02704

```



```

02711     GgMatrix& loadNormal(const GgMatrix& m)
02712     {
02713         return loadNormal(m.data());
02714     }
02715
02725     GgMatrix translate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02726     {
02727         GgMatrix m;
02728         return operator*(m.loadTranslate(x, y, z, w));
02729     }
02730
02737     GgMatrix translate(const GLfloat* t) const
02738     {
02739         return translate(t[0], t[1], t[2]);
02740     }
02741
02748     GgMatrix translate(const GgVector& t) const
02749     {
02750         return translate(t[0], t[1], t[2], t[3]);
02751     }
02752
02762     GgMatrix scale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02763     {
02764         GgMatrix m;
02765         return operator*(m.loadScale(x, y, z, w));
02766     }
02767
02774     GgMatrix scale(const GLfloat* s) const
02775     {
02776         return scale(s[0], s[1], s[2]);
02777     }
02778
02785     GgMatrix scale(const GgVector& s) const
02786     {
02787         return scale(s[0], s[1], s[2], s[3]);
02788     }
02789
02796     GgMatrix rotateX(GLfloat a) const
02797     {
02798         GgMatrix m;
02799         return operator*(m.loadRotateX(a));
02800     }
02801
02808     GgMatrix rotateY(GLfloat a) const
02809     {
02810         GgMatrix m;
02811         return operator*(m.loadRotateY(a));
02812     }
02813
02820     GgMatrix rotateZ(GLfloat a) const
02821     {
02822         GgMatrix m;
02823         return operator*(m.loadRotateZ(a));
02824     }
02825
02835     GgMatrix rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
02836     {
02837         GgMatrix m;
02838         return operator*(m.loadRotate(x, y, z, a));
02839     }
02840
02848     GgMatrix rotate(const GLfloat* r, GLfloat a) const
02849     {
02850         return rotate(r[0], r[1], r[2], a);
02851     }
02852
02860     GgMatrix rotate(const GgVector& r, GLfloat a) const
02861     {
02862         return rotate(r[0], r[1], r[2], a);
02863     }
02864
02871     GgMatrix rotate(const GLfloat* r) const
02872     {
02873         return rotate(r[0], r[1], r[2], r[3]);
02874     }
02875
02882     GgMatrix rotate(const GgVector& r) const
02883     {
02884         return rotate(r[0], r[1], r[2], r[3]);
02885     }
02886
02901     GgMatrix lookat(
02902         GLfloat ex, GLfloat ey, GLfloat ez,
02903         GLfloat tx, GLfloat ty, GLfloat tz,
02904         GLfloat ux, GLfloat uy, GLfloat uz
02905     ) const
02906     {

```

```

02907     GgMatrix m;
02908     return operator*(m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz));
02909 }
02910
02919 GgMatrix lookat(const GLfloat* e, const GLfloat* t, const GLfloat* u) const
02920 {
02921     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02922 }
02923
02932 GgMatrix lookat(const GgVector& e, const GgVector& t, const GgVector& u) const
02933 {
02934     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02935 }
02936
02948 GgMatrix orthogonal(
02949     GLfloat left, GLfloat right,
02950     GLfloat bottom, GLfloat top,
02951     GLfloat zNear, GLfloat zFar
02952 ) const
02953 {
02954     GgMatrix m;
02955     return operator*(m.loadOrthogonal(left, right, bottom, top, zNear, zFar));
02956 }
02957
02969 GgMatrix frustum(
02970     GLfloat left, GLfloat right,
02971     GLfloat bottom, GLfloat top,
02972     GLfloat zNear, GLfloat zFar
02973 ) const
02974 {
02975     GgMatrix m;
02976     return operator*(m.loadFrustum(left, right, bottom, top, zNear, zFar));
02977 }
02978
02988 GgMatrix perspective(
02989     GLfloat fovy, GLfloat aspect,
02990     GLfloat zNear, GLfloat zFar
02991 ) const
02992 {
02993     GgMatrix m;
02994     return operator*(m.loadPerspective(fovy, aspect, zNear, zFar));
02995 }
02996
03002 GgMatrix transpose() const
03003 {
03004     GgMatrix t;
03005     return t.loadTranspose(*this);
03006 }
03007
03013 GgMatrix invert() const
03014 {
03015     GgMatrix t;
03016     return t.loadInvert(*this);
03017 }
03018
03024 GgMatrix normal() const
03025 {
03026     GgMatrix t;
03027     return t.loadNormal(*this);
03028 }
03029
03036 void projection(GLfloat* c, const GLfloat* v) const
03037 {
03038     projection(c, data(), v);
03039 }
03040
03047 void projection(GLfloat* c, const GgVector& v) const
03048 {
03049     projection(c, v.data());
03050 }
03051
03058 void projection(GgVector& c, const GLfloat* v) const
03059 {
03060     projection(c.data(), v);
03061 }
03062
03069 void projection(GgVector& c, const GgVector& v) const
03070 {
03071     projection(c.data(), v.data());
03072 }
03073
03080 GgVector operator*(const GgVector& v) const
03081 {
03082     GgVector c;
03083     projection(c, v);
03084     return c;
03085 }

```

```

03086
03092     const GLfloat* get() const
03093     {
03094         return data();
03095     }
03096
03102     void get(GLfloat* a) const
03103     {
03104         std::copy(data(), data() + size(), a);
03105     }
03106 };
03107
03113 inline GgMatrix ggIdentity()
03114 {
03115     GgMatrix t;
03116     return t.loadIdentity();
03117 };
03118
03128 inline GgMatrix ggTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03129 {
03130     GgMatrix m;
03131     return m.loadTranslate(x, y, z, w);
03132 }
03133
03140 inline GgMatrix ggTranslate(const GLfloat* t)
03141 {
03142     GgMatrix m;
03143     return m.loadTranslate(t[0], t[1], t[2]);
03144 }
03145
03152 inline GgMatrix ggTranslate(const GgVector& t)
03153 {
03154     GgMatrix m;
03155     return m.loadTranslate(t[0], t[1], t[2], t[3]);
03156 }
03157
03167 inline GgMatrix ggScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03168 {
03169     GgMatrix m;
03170     return m.loadScale(x, y, z, w);
03171 }
03172
03179 inline GgMatrix ggScale(const GLfloat* s)
03180 {
03181     GgMatrix m;
03182     return m.loadScale(s[0], s[1], s[2]);
03183 }
03184
03191 inline GgMatrix ggScale(const GgVector& s)
03192 {
03193     GgMatrix m;
03194     return m.loadScale(s[0], s[1], s[2], s[3]);
03195 }
03196
03203 inline GgMatrix ggRotateX(GLfloat a)
03204 {
03205     GgMatrix m;
03206     return m.loadRotateX(a);
03207 }
03208
03215 inline GgMatrix ggRotateY(GLfloat a)
03216 {
03217     GgMatrix m;
03218     return m.loadRotateY(a);
03219 }
03220
03227 inline GgMatrix ggRotateZ(GLfloat a)
03228 {
03229     GgMatrix m;
03230     return m.loadRotateZ(a);
03231 }
03232
03242 inline GgMatrix ggRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03243 {
03244     GgMatrix m;
03245     return m.loadRotate(x, y, z, a);
03246 }
03247
03255 inline GgMatrix ggRotate(const GLfloat* r, GLfloat a)
03256 {
03257     GgMatrix m;
03258     return m.loadRotate(r[0], r[1], r[2], a);
03259 }
03260
03268 inline GgMatrix ggRotate(const GgVector& r, GLfloat a)
03269 {
03270     GgMatrix m;

```

```

03271     return m.loadRotate(r[0], r[1], r[2], a);
03272 }
03273
03280 inline GgMatrix ggRotate(const GLfloat* r)
03281 {
03282     GgMatrix m;
03283     return m.loadRotate(r[0], r[1], r[2], r[3]);
03284 }
03285
03292 inline GgMatrix ggRotate(const GgVector& r)
03293 {
03294     GgMatrix m;
03295     return m.loadRotate(r[0], r[1], r[2], r[3]);
03296 }
03297
03312 inline GgMatrix ggLookat(
03313     GLfloat ex, GLfloat ey, GLfloat ez,    // 視点の位置
03314     GLfloat tx, GLfloat ty, GLfloat tz,    // 目標点の位置
03315     GLfloat ux, GLfloat uy, GLfloat uz    // 上方向のベクトル
03316 )
03317 {
03318     GgMatrix m;
03319     return m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz);
03320 }
03321
03330 inline GgMatrix ggLookat(
03331     const GLfloat* e,                    // 視点の位置
03332     const GLfloat* t,                    // 目標点の位置
03333     const GLfloat* u                    // 上方向のベクトル
03334 )
03335 {
03336     GgMatrix m;
03337     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03338 }
03339
03348 inline GgMatrix ggLookat(
03349     const GgVector& e,                    // 視点の位置
03350     const GgVector& t,                    // 目標点の位置
03351     const GgVector& u                    // 上方向のベクトル
03352 )
03353 {
03354     GgMatrix m;
03355     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03356 }
03357
03369 inline GgMatrix ggOrthogonal(GLfloat left, GLfloat right,
03370     GLfloat bottom, GLfloat top,
03371     GLfloat zNear, GLfloat zFar)
03372 {
03373     GgMatrix m;
03374     return m.loadOrthogonal(left, right, bottom, top, zNear, zFar);
03375 }
03376
03388 inline GgMatrix ggFrustum(
03389     GLfloat left, GLfloat right,
03390     GLfloat bottom, GLfloat top,
03391     GLfloat zNear, GLfloat zFar
03392 )
03393 {
03394     GgMatrix m;
03395     return m.loadFrustum(left, right, bottom, top, zNear, zFar);
03396 }
03397
03407 inline GgMatrix ggPerspective(
03408     GLfloat fovy, GLfloat aspect,
03409     GLfloat zNear, GLfloat zFar
03410 )
03411 {
03412     GgMatrix m;
03413     return m.loadPerspective(fovy, aspect, zNear, zFar);
03414 }
03415
03422 inline GgMatrix ggTranspose(const GgMatrix& m)
03423 {
03424     return m.transpose();
03425 }
03426
03433 inline GgMatrix ggInvert(const GgMatrix& m)
03434 {
03435     return m.invert();
03436 }
03437
03444 inline GgMatrix ggNormal(const GgMatrix& m)
03445 {
03446     return m.normal();
03447 }
03448

```

```

03452 class GgQuaternion : public GgVector
03453 {
03454     // GgQuaternion 型の四元数 p と四元数 q の積を四元数 r に求める
03455     void multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const;
03456
03457     // GgQuaternion 型の四元数 q が表す回転の変換行列を m に求める
03458     void toMatrix(GLfloat* m, const GLfloat* q) const;
03459
03460     // 回転の変換行列 m が表す四元数を q に求める
03461     void toQuaternion(GLfloat* q, const GLfloat* m) const;
03462
03463     // 球面線形補間 q と r を t で補間した四元数を p に求める
03464     void slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const;
03465
03466 public:
03467
03471     GgQuaternion() = default;
03472
03481     constexpr GgQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat w) :
03482         GgVector{ x, y, z, w }
03483     {
03484     }
03485
03491     constexpr GgQuaternion(GLfloat c) :
03492         GgQuaternion{ c, c, c, c }
03493     {
03494     }
03495
03501     GgQuaternion(const GLfloat* a) :
03502         GgQuaternion{ a[0], a[1], a[2], a[3] }
03503     {
03504     }
03505
03511     GgQuaternion(const GgVector& v) :
03512         GgQuaternion{ v[0], v[1], v[2], v[3] }
03513     {
03514     }
03515
03521     GgQuaternion(const GgQuaternion& q) = default;
03522
03528     GgQuaternion(GgQuaternion&& q) = default;
03529
03533     ~GgQuaternion() = default;
03534
03541     GgQuaternion& operator=(const GgQuaternion& q) = default;
03542
03549     GgQuaternion& operator=(GgQuaternion&& q) = default;
03550
03556     GLfloat norm() const
03557     {
03558         return ggLength4(*this);
03559     }
03560
03570     GgQuaternion& loadAdd(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03571     {
03572         (*this)[0] += x;
03573         (*this)[1] += y;
03574         (*this)[2] += z;
03575         (*this)[3] += w;
03576
03577         return *this;
03578     }
03579
03586     GgQuaternion& loadAdd(const GLfloat* a)
03587     {
03588         return loadAdd(a[0], a[1], a[2], a[3]);
03589     }
03590
03597     GgQuaternion& loadAdd(const GgVector& v)
03598     {
03599         return loadAdd(v[0], v[1], v[2], v[3]);
03600     }
03601
03608     GgQuaternion& loadAdd(const GgQuaternion& q)
03609     {
03610         return loadAdd(q[0], q[1], q[2], q[3]);
03611     }
03612
03622     GgQuaternion& loadSubtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03623     {
03624         (*this)[0] -= x;
03625         (*this)[1] -= y;
03626         (*this)[2] -= z;
03627         (*this)[3] -= w;
03628
03629         return *this;
03630     }

```

```

03631
03638 GgQuaternion& loadSubtract(const GLfloat* a)
03639 {
03640     return loadSubtract(a[0], a[1], a[2], a[3]);
03641 }
03642
03649 GgQuaternion& loadSubtract(const GgVector& v)
03650 {
03651     return loadSubtract(v[0], v[1], v[2], v[3]);
03652 }
03653
03660 GgQuaternion& loadSubtract(const GgQuaternion& q)
03661 {
03662     return loadSubtract(q[0], q[1], q[2], q[3]);
03663 }
03664
03674 GgQuaternion& loadMultiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03675 {
03676     const GLfloat a[]={ x, y, z, w };
03677     return loadMultiply(a);
03678 }
03679
03686 GgQuaternion& loadMultiply(const GLfloat* a)
03687 {
03688     return *this = multiply(a);
03689 }
03690
03697 GgQuaternion& loadMultiply(const GgVector& v)
03698 {
03699     return loadMultiply(v.data());
03700 }
03701
03708 GgQuaternion& loadMultiply(const GgQuaternion& q)
03709 {
03710     return loadMultiply(q.data());
03711 }
03712
03722 GgQuaternion& loadDivide(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03723 {
03724     const GLfloat a[]={ x, y, z, w };
03725     return loadDivide(a);
03726 }
03727
03734 GgQuaternion& loadDivide(const GLfloat* a)
03735 {
03736     return *this = divide(a);
03737 }
03738
03745 GgQuaternion& loadDivide(const GgVector& v)
03746 {
03747     return loadDivide(v.data());
03748 }
03749
03756 GgQuaternion& loadDivide(const GgQuaternion& q)
03757 {
03758     return loadDivide(q.data());
03759 }
03760
03770 GgQuaternion add(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03771 {
03772     GgQuaternion s{ *this };
03773     return s.loadAdd(x, y, z, w);
03774 }
03775
03782 GgQuaternion add(const GLfloat* a) const
03783 {
03784     return add(a[0], a[1], a[2], a[3]);
03785 }
03786
03793 GgQuaternion add(const GgVector& v) const
03794 {
03795     return add(v[0], v[1], v[2], v[3]);
03796 }
03797
03804 GgQuaternion add(const GgQuaternion& q) const
03805 {
03806     return add(q[0], q[1], q[2], q[3]);
03807 }
03808
03818 GgQuaternion subtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03819 {
03820     GgQuaternion s{ *this };
03821     return s.loadSubtract(x, y, z, w);
03822 }
03823
03830 GgQuaternion subtract(const GLfloat* a) const
03831 {

```

```

03832     return subtract(a[0], a[1], a[2], a[3]);
03833 }
03834
03841 GgQuaternion subtract(const GgVector& v) const
03842 {
03843     return subtract(v[0], v[1], v[2], v[3]);
03844 }
03845
03852 GgQuaternion subtract(const GgQuaternion& q) const
03853 {
03854     return subtract(q[0], q[1], q[2], q[3]);
03855 }
03856
03866 GgQuaternion multiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03867 {
03868     const GLfloat a[]={ x, y, z, w };
03869     return multiply(a);
03870 }
03871
03878 GgQuaternion multiply(const GLfloat* a) const
03879 {
03880     GgQuaternion s;
03881     multiply(s.data(), data(), a);
03882     return s;
03883 }
03884
03891 GgQuaternion multiply(const GgVector& v) const
03892 {
03893     return multiply(v.data());
03894 }
03895
03902 GgQuaternion multiply(const GgQuaternion& q) const
03903 {
03904     return multiply(q.data());
03905 }
03906
03916 GgQuaternion divide(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03917 {
03918     const GLfloat a[]={ x, y, z, w };
03919     return divide(a);
03920 }
03921
03928 GgQuaternion divide(const GLfloat* a) const
03929 {
03930     GgQuaternion s, ia;
03931     ia.loadInvert(a);
03932     multiply(s.data(), data(), ia.data());
03933     return s;
03934 }
03935
03942 GgQuaternion divide(const GgVector& v) const
03943 {
03944     return divide(v.data());
03945 }
03946
03953 GgQuaternion divide(const GgQuaternion& q) const
03954 {
03955     return divide(q.data());
03956 }
03957
03958 // 演算子
03959 GgQuaternion& operator=(const GLfloat* a)
03960 {
03961     return *this = GgQuaternion(a);
03962 }
03963 GgQuaternion& operator=(const GgVector& v)
03964 {
03965     return *this = GgQuaternion(v);
03966 }
03967 GgQuaternion& operator+=(const GLfloat* a)
03968 {
03969     return loadAdd(a);
03970 }
03971 GgQuaternion& operator+=(const GgVector& v)
03972 {
03973     return loadAdd(v);
03974 }
03975 GgQuaternion& operator+=(const GgQuaternion& q)
03976 {
03977     return loadAdd(q);
03978 }
03979 GgQuaternion& operator-=(const GLfloat* a)
03980 {
03981     return loadSubtract(a);
03982 }
03983 GgQuaternion& operator-=(const GgVector& v)
03984 {

```

```

03985     return loadSubtract(v);
03986 }
03987 GgQuaternion& operator--(const GgQuaternion& q)
03988 {
03989     return operator--(q.data());
03990 }
03991 GgQuaternion& operator*=(const GLfloat* a)
03992 {
03993     return loadMultiply(a);
03994 }
03995 GgQuaternion& operator*=(const GgVector& v)
03996 {
03997     return loadMultiply(v);
03998 }
03999 GgQuaternion& operator*=(const GgQuaternion& q)
04000 {
04001     return operator*=(q.data());
04002 }
04003 GgQuaternion& operator/=(const GLfloat* a)
04004 {
04005     return loadDivide(a);
04006 }
04007 GgQuaternion& operator/=(const GgVector& v)
04008 {
04009     return loadDivide(v);
04010 }
04011 GgQuaternion& operator/=(const GgQuaternion& q)
04012 {
04013     return operator/=(q.data());
04014 }
04015 GgQuaternion operator+(const GLfloat* a) const
04016 {
04017     return add(a);
04018 }
04019 GgQuaternion operator+(const GgVector& v) const
04020 {
04021     return add(v);
04022 }
04023 GgQuaternion operator+(const GgQuaternion& q) const
04024 {
04025     return operator+(q.data());
04026 }
04027 GgQuaternion operator-(const GLfloat* a) const
04028 {
04029     return add(a);
04030 }
04031 GgQuaternion operator-(const GgVector& v) const
04032 {
04033     return add(v);
04034 }
04035 GgQuaternion operator-(const GgQuaternion& q) const
04036 {
04037     return operator-(q.data());
04038 }
04039 GgQuaternion operator*(const GLfloat* a) const
04040 {
04041     return multiply(a);
04042 }
04043 GgQuaternion operator*(const GgVector& v) const
04044 {
04045     return multiply(v);
04046 }
04047 GgQuaternion operator*(const GgQuaternion& q) const
04048 {
04049     return operator*(q.data());
04050 }
04051 GgQuaternion operator/(const GLfloat* a) const
04052 {
04053     return divide(a);
04054 }
04055 GgQuaternion operator/(const GgVector& v) const
04056 {
04057     return divide(v);
04058 }
04059 GgQuaternion operator/(const GgQuaternion& q) const
04060 {
04061     return operator/(q.data());
04062 }
04063 }
04070 GgQuaternion& loadMatrix(const GLfloat* a)
04071 {
04072     toQuaternion(data(), a);
04073     return *this;
04074 }
04075 }
04082 GgQuaternion& loadMatrix(const GgMatrix& m)
04083 {

```



```

04084     return loadMatrix(m.get());
04085 }
04086
04092 GgQuaternion& loadIdentity()
04093 {
04094     return *this = GgQuaternion(0.0f, 0.0f, 0.0f, 1.0f);
04095 }
04096
04106 GgQuaternion& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
04107
04115 GgQuaternion& loadRotate(const GLfloat* v, GLfloat a)
04116 {
04117     return loadRotate(v[0], v[1], v[2], a);
04118 }
04119
04126 GgQuaternion& loadRotate(const GLfloat* v)
04127 {
04128     return loadRotate(v[0], v[1], v[2], v[3]);
04129 }
04130
04137 GgQuaternion& loadRotateX(GLfloat a);
04138
04145 GgQuaternion& loadRotateY(GLfloat a);
04146
04153 GgQuaternion& loadRotateZ(GLfloat a);
04154
04164 GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
04165 {
04166     GgQuaternion q;
04167     return multiply(q.loadRotate(x, y, z, a));
04168 }
04169
04177 GgQuaternion rotate(const GLfloat* v, GLfloat a) const
04178 {
04179     return rotate(v[0], v[1], v[2], a);
04180 }
04181
04188 GgQuaternion rotate(const GLfloat* v) const
04189 {
04190     return rotate(v[0], v[1], v[2], v[3]);
04191 }
04192
04199 GgQuaternion rotateX(GLfloat a) const
04200 {
04201     return rotate(1.0f, 0.0f, 0.0f, a);
04202 }
04203
04210 GgQuaternion rotateY(GLfloat a) const
04211 {
04212     return rotate(0.0f, 1.0f, 0.0f, a);
04213 }
04214
04221 GgQuaternion rotateZ(GLfloat a) const
04222 {
04223     return rotate(0.0f, 0.0f, 1.0f, a);
04224 }
04225
04234 GgQuaternion& loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll);
04235
04242 GgQuaternion& loadEuler(const GLfloat* e)
04243 {
04244     return loadEuler(e[0], e[1], e[2]);
04245 }
04246
04255 GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const
04256 {
04257     GgQuaternion r;
04258     return multiply(r.loadEuler(heading, pitch, roll));
04259 }
04260
04267 GgQuaternion euler(const GLfloat* e) const
04268 {
04269     return euler(e[0], e[1], e[2]);
04270 }
04271
04281 GgQuaternion euler(const GgVector& e) const
04282 {
04283     return euler(e[0], e[1], e[2]);
04284 }
04285
04294 GgQuaternion& loadSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04295 {
04296     slerp(data(), a, b, t);
04297     return *this;
04298 }
04299
04308 GgQuaternion& loadSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)

```

```

04309     {
04310         return loadSlerp(q.data(), r.data(), t);
04311     }
04312
04321 GgQuaternion& loadSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04322 {
04323     return loadSlerp(q.data(), a, t);
04324 }
04325
04334 GgQuaternion& loadSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04335 {
04336     return loadSlerp(a, q.data(), t);
04337 }
04338
04345 GgQuaternion& loadNormalize(const GLfloat* a);
04346
04353 GgQuaternion& loadNormalize(const GgQuaternion& q)
04354 {
04355     return loadNormalize(q.data());
04356 }
04357
04364 GgQuaternion& loadConjugate(const GLfloat* a);
04365
04372 GgQuaternion& loadConjugate(const GgQuaternion& q)
04373 {
04374     return loadConjugate(q.data());
04375 }
04376
04383 GgQuaternion& loadInvert(const GLfloat* a);
04384
04391 GgQuaternion& loadInvert(const GgQuaternion& q)
04392 {
04393     return loadInvert(q.data());
04394 }
04395
04403 GgQuaternion slerp(GLfloat* a, GLfloat t) const
04404 {
04405     GgQuaternion p;
04406     slerp(p.data(), data(), a, t);
04407     return p;
04408 }
04409
04417 GgQuaternion slerp(const GgQuaternion& q, GLfloat t) const
04418 {
04419     GgQuaternion p;
04420     slerp(p.data(), data(), q.data(), t);
04421     return p;
04422 }
04423
04429 GgQuaternion normalize() const
04430 {
04431     GgQuaternion q;
04432     q.loadNormalize(data());
04433     return q;
04434 }
04435
04441 GgQuaternion conjugate() const
04442 {
04443     GgQuaternion q;
04444     q.loadConjugate(data());
04445     return q;
04446 }
04447
04453 GgQuaternion invert() const
04454 {
04455     GgQuaternion q;
04456     q.loadInvert(data());
04457     return q;
04458 }
04459
04465 void get(GLfloat* a) const
04466 {
04467     a[0] = data()[0];
04468     a[1] = data()[1];
04469     a[2] = data()[2];
04470     a[3] = data()[3];
04471 }
04472
04478 void getMatrix(GLfloat* a) const
04479 {
04480     toMatrix(a, data());
04481 }
04482
04488 void getMatrix(GgMatrix& m) const
04489 {
04490     getMatrix(m.data());
04491 }

```

```

04492
04498 GgMatrix getMatrix() const
04499 {
04500     GgMatrix m;
04501     getMatrix(m);
04502     return m;
04503 }
04504
04510 void getConjugateMatrix(GLfloat* a) const
04511 {
04512     GgQuaternion c;
04513     c.loadConjugate(data());
04514     toMatrix(a, c.data());
04515 }
04516
04522 void getConjugateMatrix(GgMatrix& m) const
04523 {
04524     getConjugateMatrix(m.data());
04525 }
04526
04532 GgMatrix getConjugateMatrix() const
04533 {
04534     GgMatrix m;
04535     getConjugateMatrix(m);
04536     return m;
04537 }
04538 };
04539
04545 inline GgQuaternion ggIdentityQuaternion()
04546 {
04547     GgQuaternion q;
04548     return q.loadIdentity();
04549 }
04550
04557 inline GgQuaternion ggMatrixQuaternion(const GLfloat* a)
04558 {
04559     GgQuaternion q;
04560     return q.loadMatrix(a);
04561 }
04562
04569 inline GgQuaternion ggMatrixQuaternion(const GgMatrix& m)
04570 {
04571     return ggMatrixQuaternion(m.get());
04572 }
04573
04580 inline GgMatrix ggQuaternionMatrix(const GgQuaternion& q)
04581 {
04582     GLfloat m[16];
04583     q.getMatrix(m);
04584     return GgMatrix{ m };
04585 }
04586
04593 inline GgMatrix ggQuaternionTransposeMatrix(const GgQuaternion& q)
04594 {
04595     GLfloat m[16];
04596     q.getMatrix(m);
04597     GgMatrix t;
04598     return t.loadTranspose(m);
04599 }
04600
04610 inline GgQuaternion ggRotateQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
04611 {
04612     GgQuaternion q;
04613     return q.loadRotate(x, y, z, a);
04614 }
04615
04623 inline GgQuaternion ggRotateQuaternion(const GLfloat* v, GLfloat a)
04624 {
04625     return ggRotateQuaternion(v[0], v[1], v[2], a);
04626 }
04627
04634 inline GgQuaternion ggRotateQuaternion(const GLfloat* v)
04635 {
04636     return ggRotateQuaternion(v[0], v[1], v[2], v[3]);
04637 }
04638
04647 inline GgQuaternion ggEulerQuaternion(GLfloat heading, GLfloat pitch, GLfloat roll)
04648 {
04649     GgQuaternion q;
04650     return q.loadEuler(heading, pitch, roll);
04651 }
04652
04659 inline GgQuaternion ggEulerQuaternion(const GLfloat* e)
04660 {
04661     return ggEulerQuaternion(e[0], e[1], e[2]);
04662 }
04663

```

```

04670 inline GgQuaternion ggEulerQuaternion(const GgVector& e)
04671 {
04672     return ggEulerQuaternion(e[0], e[1], e[2]);
04673 }
04674
04683 inline GgQuaternion ggSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04684 {
04685     GgQuaternion r;
04686     return r.loadSlerp(a, b, t);
04687 }
04688
04697 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04698 {
04699     return ggSlerp(q.data(), r.data(), t);
04700 }
04701
04710 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04711 {
04712     return ggSlerp(q.data(), a, t);
04713 }
04714
04723 inline GgQuaternion ggSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04724 {
04725     return ggSlerp(a, q.data(), t);
04726 }
04727
04734 inline GLfloat ggNorm(const GgQuaternion& q)
04735 {
04736     return q.norm();
04737 }
04738
04745 inline GgQuaternion ggNormalize(const GgQuaternion& q)
04746 {
04747     return q.normalize();
04748 }
04749
04756 inline GgQuaternion ggConjugate(const GgQuaternion& q)
04757 {
04758     return q.conjugate();
04759 }
04760
04767 inline GgQuaternion ggInvert(const GgQuaternion& q)
04768 {
04769     return q.invert();
04770 }
04771
04775 class GgTrackball : public GgQuaternion
04776 {
04777     bool drag;           // ドラッグ中か否か
04778     GLfloat start[2];    // ドラッグ開始位置
04779     GLfloat scale[2];    // マウスの絶対位置→ウィンドウ内での相対位置の換算係数
04780     GgQuaternion cq;     // 回転の初期値 (四元数)
04781     GgMatrix rt;        // 回転の変換行列
04782
04783 public:
04784
04790     GgTrackball(const GgQuaternion& q = ggIdentityQuaternion())
04791     {
04792         reset(q);
04793     }
04794
04798     ~GgTrackball() = default;
04799
04805     GgTrackball& operator=(const GgQuaternion& q)
04806     {
04807         GgQuaternion::operator=(q);
04808         return *this;
04809     }
04810
04820     void region(GLfloat w, GLfloat h);
04821
04831     void region(int w, int h)
04832     {
04833         region(static_cast<GLfloat>(w), static_cast<GLfloat>(h));
04834     }
04835
04845     void begin(GLfloat x, GLfloat y);
04846
04856     void motion(GLfloat x, GLfloat y);
04857
04863     void rotate(const GgQuaternion& q);
04864
04874     void end(GLfloat x, GLfloat y);
04875
04881     void reset(const GgQuaternion& q = ggIdentityQuaternion());
04882
04888     const GLfloat* getStart() const

```

```

04889     {
04890         return start;
04891     }
04892
04896     const GLfloat& getStart(int direction) const
04897     {
04898         return start[direction];
04899     }
04900
04906     void getStart(GLfloat* position) const
04907     {
04908         position[0] = start[0];
04909         position[1] = start[1];
04910     }
04911
04917     const GLfloat* getScale() const
04918     {
04919         return scale;
04920     }
04921
04927     const GLfloat getScale(int direction) const
04928     {
04929         return scale[direction];
04930     }
04931
04937     void getScale(GLfloat* factor) const
04938     {
04939         factor[0] = scale[0];
04940         factor[1] = scale[1];
04941     }
04942
04948     const GgQuaternion& getQuaternion() const
04949     {
04950         return *this;
04951     }
04952
04958     const GgMatrix& getMatrix() const
04959     {
04960         return rt;
04961     }
04962
04968     const GLfloat* get() const
04969     {
04970         return rt.get();
04971     }
04972 };
04973
04984 extern bool ggSaveTga(
04985     const std::string& name,
04986     const void* buffer,
04987     unsigned int width,
04988     unsigned int height,
04989     unsigned int depth
04990 );
04991
04998 extern bool ggSaveColor(const std::string& name);
04999
05006 extern bool ggSaveDepth(const std::string& name);
05007
05018 extern bool ggReadImage(
05019     const std::string& name,
05020     std::vector<GLubyte>& image,
05021     GLsizei* pWidth,
05022     GLsizei* pHeight,
05023     GLenum* pFormat
05024 );
05025
05039 extern GLuint ggLoadTexture(
05040     const GLvoid* image,
05041     GLsizei width,
05042     GLsizei height,
05043     GLenum format = GL_RGB,
05044     GLenum type = GL_UNSIGNED_BYTE,
05045     GLenum internal = GL_RGB,
05046     GLenum wrap = GL_CLAMP_TO_EDGE,
05047     bool swizzle = false
05048 );
05049
05060 extern GLuint ggLoadImage(
05061     const std::string& name,
05062     GLsizei* pWidth = nullptr,
05063     GLsizei* pHeight = nullptr,
05064     GLenum internal = GL_RGBA,
05065     GLenum wrap = GL_CLAMP_TO_EDGE
05066 );
05067
05079 extern void ggCreateNormalMap(

```

```

05080     const GLubyte* hmap,
05081     GLsizei width,
05082     GLsizei height,
05083     GLenum format,
05084     GLfloat nz,
05085     GLenum internal,
05086     std::vector<GgVector>& nmap
05087 );
05088
05099 extern GLuint ggLoadHeight(
05100     const std::string& name,
05101     GLfloat nz,
05102     GLsizei* pWidth = nullptr,
05103     GLsizei* pHeight = nullptr,
05104     GLenum internal = GL_RGBA
05105 );
05106
05120 extern GLuint ggCreateShader(
05121     const std::string& vsrc,
05122     const std::string& fsrc = "",
05123     const std::string& gsrc = "",
05124     GLint nvarying = 0,
05125     const char* const* varyings = nullptr,
05126     const std::string& vtext = "vertex shader",
05127     const std::string& ftext = "fragment shader",
05128     const std::string& gtext = "geometry shader");
05129
05140 extern GLuint ggLoadShader(
05141     const std::string& vert,
05142     const std::string& frag = "",
05143     const std::string& geom = "",
05144     GLint nvarying = 0,
05145     const char* const* varyings = nullptr
05146 );
05147
05156 inline GLuint ggLoadShader(
05157     const std::array<std::string, 3>& files,
05158     GLint nvarying = 0,
05159     const char* const* varyings = nullptr
05160 )
05161 {
05162     return ggLoadShader(files[0], files[1], files[2], nvarying, varyings);
05163 }
05164
05165 #if !defined(__APPLE__)
05173 extern GLuint ggCreateComputeShader(
05174     const std::string& csrc,
05175     const std::string& ctext = "compute shader"
05176 );
05177
05184 extern GLuint ggLoadComputeShader(const std::string& comp);
05185 #endif
05186
05193 class GgTexture
05194 {
05195     // テクスチャ名
05196     GLuint texture;
05197
05198     // テクスチャの縦横の画素数
05199     GLsizei size[2];
05200
05201 public:
05202
05215     GgTexture(
05216         const GLvoid* image,
05217         GLsizei width,
05218         GLsizei height,
05219         GLenum format = GL_RGB,
05220         GLenum type = GL_UNSIGNED_BYTE,
05221         GLenum internal = GL_RGBA,
05222         GLenum wrap = GL_CLAMP_TO_EDGE,
05223         bool swizzle = false
05224     ) :
05225         texture{ ggLoadTexture(image, width, height, format, type, internal, wrap, swizzle) },
05226         size{ width, height }
05227     {
05228     }
05229
05235     GgTexture(const GgTexture& texture) = delete;
05236
05242     GgTexture(GgTexture&& texture) = default;
05243
05247     virtual ~GgTexture()
05248     {
05249         glBindTexture(GL_TEXTURE_2D, 0);
05250         glDeleteTextures(1, &texture);
05251     }

```

```

05252
05259     GgTexture& operator=(const GgTexture& texture) = delete;
05260
05267     GgTexture& operator=(GgTexture&& texture) = default;
05268
05272     void bind() const
05273     {
05274         glBindTexture(GL_TEXTURE_2D, texture);
05275     }
05276
05280     void unbind() const
05281     {
05282         glBindTexture(GL_TEXTURE_2D, 0);
05283     }
05284
05290     void swapRandB(bool swizzle) const;
05291
05297     const GLsizei& getWidth() const
05298     {
05299         return size[0];
05300     }
05301
05307     const GLsizei& getHeight() const
05308     {
05309         return size[1];
05310     }
05311
05317     void getSize(GLsizei* size) const
05318     {
05319         size[0] = getWidth();
05320         size[1] = getHeight();
05321     }
05322
05328     const GLsizei* getSize() const
05329     {
05330         return size;
05331     }
05332
05338     const GLuint& getTexture() const
05339     {
05340         return texture;
05341     }
05342 };
05343
05350 class GgColorTexture
05351 {
05352     // テクスチャ
05353     std::shared_ptr<GgTexture> texture;
05354
05355 public:
05356
05360     GgColorTexture()
05361     {
05362     }
05363
05376     GgColorTexture(
05377         const GLvoid* image,
05378         GLsizei width,
05379         GLsizei height,
05380         GLenum format = GL_RGB,
05381         GLenum type = GL_UNSIGNED_BYTE,
05382         GLenum internal = GL_RGB,
05383         GLenum wrap = GL_CLAMP_TO_EDGE,
05384         bool swizzle = true
05385     )
05386     {
05387         load(image, width, height, format, type, internal, wrap, swizzle);
05388     }
05389
05397     GgColorTexture(
05398         const std::string& name,
05399         GLenum internal = 0,
05400         GLenum wrap = GL_CLAMP_TO_EDGE
05401     )
05402     {
05403         load(name, internal, wrap);
05404     }
05405
05409     virtual ~GgColorTexture()
05410     {
05411     }
05412
05425     void load(
05426         const GLvoid* image,
05427         GLsizei width,
05428         GLsizei height,
05429         GLenum format = GL_RGB,

```

```

05430         GLenum type = GL_UNSIGNED_BYTE,
05431         GLenum internal = GL_RGB,
05432         GLenum wrap = GL_CLAMP_TO_EDGE,
05433         bool swizzle = true
05434     )
05435     {
05436         // テクスチャを作成する
05437         texture = std::make_shared<GgTexture>(image, width, height, format, type, internal, wrap,
swizzle);
05438     }
05439
05440     void load(
05441         const std::string& name,
05442         GLenum internal = 0,
05443         GLenum wrap = GL_CLAMP_TO_EDGE
05444     );
05445 };
05446
05447 class GgNormalTexture
05448 {
05449     // テクスチャ
05450     std::shared_ptr<GgTexture> texture;
05451
05452 public:
05453     GgNormalTexture()
05454     {
05455     }
05456
05457     GgNormalTexture(
05458         const GLubyte* image,
05459         GLsizei width,
05460         GLsizei height,
05461         GLenum format = GL_RED,
05462         GLfloat nz = 1.0f,
05463         GLenum internal = GL_RGBA
05464     )
05465     {
05466         // 法線マップのテクスチャを作成する
05467         load(image, width, height, format, nz, internal);
05468     }
05469
05470     GgNormalTexture(
05471         const std::string& name,
05472         GLfloat nz = 1.0f,
05473         GLenum internal = GL_RGBA
05474     )
05475     {
05476         // 法線マップのテクスチャを作成する
05477         load(name, nz, internal);
05478     }
05479
05480     virtual ~GgNormalTexture()
05481     {
05482     }
05483
05484     void load(
05485         const GLubyte* hmap,
05486         GLsizei width,
05487         GLsizei height,
05488         GLenum format = GL_RED,
05489         GLfloat nz = 1.0f,
05490         GLenum internal = GL_RGBA
05491     )
05492     {
05493         // 法線マップ
05494         std::vector<GgVector> nmap;
05495
05496         // 法線マップを作成する
05497         ggCreateNormalMap(hmap, width, height, format, nz, internal, nmap);
05498
05499         // テクスチャを作成する
05500         texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
05501     }
05502
05503     void load(
05504         const std::string& name,
05505         GLfloat nz = 1.0f,
05506         GLenum internal = GL_RGBA
05507     );
05508 };
05509
05510 template <typename T>
05511 class GgBuffer
05512 {
05513     // ターゲット

```



```

05574     const GLenum target;
05575
05576     // バッファオブジェクトのアライメントを考慮したデータの間隔
05577     const GLsizei stride;
05578
05579     // データの数
05580     const GLsizei count;
05581
05582     // バッファオブジェクト
05583     const GLuint buffer;
05584
05585 public:
05586
05587     GgBuffer<T>(
05588         GLenum target,
05589         const T* data,
05590         GLsizei stride,
05591         GLsizei count,
05592         GLenum usage
05593     ) :
05594         target{ target },
05595         stride{ stride },
05596         count{ count },
05597         buffer{ [] { GLuint buffer; glGenBuffers(1, &buffer); return buffer; } () }
05598     {
05599         // バッファオブジェクトのメモリを確保してデータを転送する
05600         glBindBuffer(target, buffer);
05601         glBufferData(target, getStride() * count, data, usage);
05602     }
05603
05604     GgBuffer<T>(const GgBuffer<T>& buffer) = delete;
05605
05606     GgBuffer<T>(GgBuffer<T>&& buffer) = default;
05607
05608     virtual ~GgBuffer<T>()
05609     {
05610         // バッファオブジェクトを削除する
05611         glBindBuffer(target, 0);
05612         glDeleteBuffers(1, &buffer);
05613     }
05614
05615     GgBuffer<T>& operator=(const GgBuffer<T>& buffer) = delete;
05616
05617     GgBuffer<T>& operator=(GgBuffer<T>&& buffer) = default;
05618
05619     const GLuint& getTarget() const
05620     {
05621         return target;
05622     }
05623
05624     GLsizeiptr getStride() const
05625     {
05626         return static_cast<GLsizeiptr>(stride);
05627     }
05628
05629     const GLsizei& getCount() const
05630     {
05631         return count;
05632     }
05633
05634     const GLuint& getBuffer() const
05635     {
05636         return buffer;
05637     }
05638
05639     void bind() const
05640     {
05641         glBindBuffer(target, buffer);
05642     }
05643
05644     void unbind() const
05645     {
05646         glBindBuffer(target, 0);
05647     }
05648
05649     void* map() const
05650     {
05651         glBindBuffer(target, buffer);
05652         #if defined(GL_GLES_PROTOTYPES)
05653             return glMapBufferRange(target, 0, getStride() * count, GL_MAP_WRITE_BIT);
05654         #else
05655             return glMapBuffer(target, GL_WRITE_ONLY);
05656         #endif
05657     }
05658
05659     void* map(GLint first, GLsizei count) const
05660     {

```

```

05733     // count が 0 なら全データをマップする
05734     if (count == 0) count = getCount();
05735     if (first + count > getCount()) count = getCount() - first;
05736
05737     glBindBuffer(target, buffer);
05738     return glMapBufferRange(target, getStride() * first, getStride() * count, GL_MAP_WRITE_BIT);
05739 }
05740
05741 void unmap() const
05742 {
05743     glUnmapBuffer(target);
05744 }
05745
05746 void send(const T* data, GLint first, GLsizei count) const
05747 {
05748     // count が 0 なら全データを転送する
05749     if (count == 0) count = getCount();
05750     if (first + count > getCount()) count = getCount() - first;
05751
05752     // データを既存のバッファオブジェクトに転送する
05753     glBindBuffer(target, buffer);
05754     glBufferSubData(target, getStride() * first, getStride() * count, data);
05755 }
05756
05757 void read(T* data, GLint first, GLsizei count) const
05758 {
05759     // count が 0 なら全データを抽出する
05760     if (count == 0) count = getCount();
05761     if (first + count > getCount()) count = getCount() - first;
05762
05763     // データをバッファオブジェクトから抽出する
05764     glBindBuffer(target, buffer);
05765     #if defined(GL_GLES_PROTOTYPES)
05766         const GLsizeiptr begin{ getStride() * first };
05767         const GLsizeiptr range{ getStride() * count };
05768         T* const source{ glMapBufferRange(target, begin, range, GL_MAP_READ_BIT) };
05769         std::copy(source, source + count, data);
05770         glUnmapBuffer(target);
05771     #else
05772         glGetBufferSubData(target, getStride() * first, getStride() * count, data);
05773     #endif
05774 }
05775
05776 void copy(GLuint src_buffer, GLint src_first = 0, GLint dst_first = 0, GLsizei count = 0) const
05777 {
05778     // count が 0 なら全データを複写する
05779     if (count == 0) count = getCount();
05780     if (src_first + count > getCount()) count = getCount() - src_first;
05781     if (dst_first + count > getCount()) count = getCount() - dst_first;
05782
05783     // データの間隔
05784     const GLsizeiptr stride{ getStride() };
05785
05786     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
05787     glBindBuffer(GL_COPY_WRITE_BUFFER, buffer);
05788     glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
05789         stride * src_first, stride * dst_first, stride * count);
05790     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
05791     glBindBuffer(GL_COPY_READ_BUFFER, 0);
05792 }
05793 };
05794
05795 template <typename T>
05796 class GgUniformBuffer
05797 {
05798     // ユニフォームバッファオブジェクト
05799     std::shared_ptr<GgBuffer<T>> uniform;
05800
05801 public:
05802     GgUniformBuffer<T>()
05803     {
05804     }
05805
05806     GgUniformBuffer<T>(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05807     {
05808         load(data, count, usage);
05809     }
05810
05811     GgUniformBuffer<T>(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05812     {
05813         load(data, count, usage);
05814     }
05815
05816     virtual ~GgUniformBuffer<T>()
05817     {
05818     }
05819 }

```

```

05871
05877     const GLuint& getTarget() const
05878     {
05879         return uniform->getTarget();
05880     }
05881
05887     GLsizeiptr getStride() const
05888     {
05889         return uniform->getStride();
05890     }
05891
05897     const GLsizei& getCount() const
05898     {
05899         return uniform->getCount();
05900     }
05901
05907     const GLuint& getBuffer() const
05908     {
05909         return uniform->getBuffer();
05910     }
05911
05915     void bind() const
05916     {
05917         uniform->bind();
05918     }
05919
05923     void unbind() const
05924     {
05925         uniform->unbind();
05926     }
05927
05933     void* map() const
05934     {
05935         return uniform->map();
05936     }
05937
05945     void* map(GLint first, GLsizei count) const
05946     {
05947         return uniform->map(first, count);
05948     }
05949
05953     void unmap() const
05954     {
05955         uniform->unmap();
05956     }
05957
05965     void load(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05966     {
05967         // バッファオブジェクト上のデータの間の隔
05968         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment } };
05969
05970         // ユニフォームバッファオブジェクトを確保する
05971         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
05972
05973         // 確保したユニフォームバッファオブジェクトにデータを転送する
05974         if (data) send(data, 0, sizeof(T), 0, count);
05975     }
05976
05984     void load(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05985     {
05986         // バッファオブジェクト上のデータの間の隔
05987         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment } };
05988
05989         // ユニフォームバッファオブジェクトを確保する
05990         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
05991
05992         // 確保したユニフォームバッファオブジェクトにデータを転送する
05993         fill(&data, 0, sizeof(T), 0, count);
05994     }
05995
06005     void send(
06006         const GLvoid* data,
06007         GLint offset = 0,
06008         GLsizei size = sizeof(T),
06009         GLint first = 0,
06010         GLsizei count = 0
06011     ) const
06012     {
06013         // count が 0 なら全データを転送する
06014         if (count == 0) count = getCount();
06015         if (first + count > getCount()) count = getCount() - first;
06016
06017         // 転送元のデータの先頭
06018         const char* source{ reinterpret_cast<const char*>(data) };
06019

```

```

06020 // ターゲット
06021 const GLuint target{ getTarget() };
06022
06023 // データの間隔
06024 const GLsizeiptr stride{ getStride() };
06025
06026 // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06027 bind();
06028 for (GLsizei i = 0; i < count; ++i)
06029 {
06030     glBufferSubData(target, stride * (first + i) + offset, size, source + size * i);
06031 }
06032 }
06033
06043 void fill(
06044     const GLvoid* data,
06045     GLint offset = 0,
06046     GLsizei size = sizeof(T),
06047     GLint first = 0,
06048     GLsizei count = 0
06049 ) const
06050 {
06051     // count が 0 なら全データを転送する
06052     if (count == 0) count = getCount();
06053     if (first + count > getCount()) count = getCount() - first;
06054
06055     // ターゲット
06056     const GLuint target{ getTarget() };
06057
06058     // データの間隔
06059     const GLsizeiptr stride{ getStride() };
06060
06061     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06062     bind();
06063     for (GLsizei i = 0; i < count; ++i)
06064     {
06065         glBufferSubData(target, stride * (first + i) + offset, size, data);
06066     }
06067 }
06068
06078 void read(
06079     GLvoid* data,
06080     GLint offset = 0,
06081     GLsizei size = sizeof(T),
06082     GLint first = 0,
06083     GLsizei count = 0
06084 ) const
06085 {
06086     // count が 0 なら全データを転送する
06087     if (count == 0) count = getCount();
06088     if (first + count > getCount()) count = getCount() - first;
06089
06090     // 抽出先のデータの先頭
06091     char* const destination{ reinterpret_cast<char*>(data) };
06092
06093     // ターゲット
06094     const GLuint target{ getTarget() };
06095
06096     // データの間隔
06097     const GLsizeiptr stride{ getStride() };
06098
06099     // データをユニフォームバッファオブジェクトから抽出する
06100     bind();
06101     #if defined(GL_GLES_PROTOTYPES)
06102         const char* const source{ glMapBufferRange(target, stride * first, stride * count,
06103             GL_MAP_READ_BIT) };
06104         for (GLsizei i = 0; i < count; ++i)
06105         {
06106             const char* const begin{ source + stride * i + offset };
06107             const char* const end{ begin + size };
06108             std::copy(begin, end, destination + sizeof(T) * i);
06109         }
06110         glUnmapBuffer(target);
06111     #else
06112         for (GLsizei i = 0; i < count; ++i)
06113         {
06114             glGetBufferSubData(target, stride * (first + i) + offset, size, destination + sizeof(T) * i);
06115         }
06116     #endif
06117
06126 void copy(
06127     GLuint src_buffer,
06128     GLint src_first = 0,
06129     GLint dst_first = 0,
06130     GLsizei count = 0
06131 ) const

```

```

06132     {
06133         // count が 0 なら全データを複写する
06134         if (count == 0) count = getCount();
06135         if (src.first + count > getCount()) count = getCount() - src.first;
06136         if (dst.first + count > getCount()) count = getCount() - dst.first;
06137
06138         // データの間隔
06139         const GLsizeiptr stride{ getStride() };
06140
06141         // ユニフォームバッファオブジェクトではブロックごとに転送する
06142         glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06143         glBindBuffer(GL_COPY_WRITE_BUFFER, getBuffer());
06144         for (GLsizei i = 0; i < count; ++i)
06145         {
06146             glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06147                 stride * (src.first + i), stride * (dst.first + i), sizeof(T));
06148         }
06149         glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06150         glBindBuffer(GL_COPY_READ_BUFFER, 0);
06151     }
06152 };
06153
06154 class GgVertexArray
06155 {
06156     // 頂点配列オブジェクト
06157     const GLuint vao;
06158
06159 public:
06160
06161     GgVertexArray(GLenum mode = 0) :
06162         vao{ [] { GLuint vao; glGenVertexArrays(1, &vao); return vao; } () }
06163     {
06164         glBindVertexArray(vao);
06165     }
06166
06167     GgVertexArray(const GgVertexArray& array) = delete;
06168
06169     GgVertexArray(GgVertexArray&& array) = default;
06170
06171     virtual ~GgVertexArray()
06172     {
06173         glBindVertexArray(0);
06174         glDeleteVertexArrays(1, &vao);
06175     }
06176
06177     GgVertexArray& operator=(const GgVertexArray& array) = delete;
06178
06179     GgVertexArray& operator=(GgVertexArray&& array) = default;
06180
06181     const GLuint& get() const
06182     {
06183         return vao;
06184     }
06185
06186     void bind() const
06187     {
06188         glBindVertexArray(vao);
06189     }
06190 };
06191
06192 class GgShape
06193 {
06194     // 頂点配列オブジェクト
06195     std::shared_ptr<GgVertexArray> object;
06196
06197     // 基本図形の種類
06198     GLenum mode;
06199
06200 public:
06201
06202     GgShape(GLenum mode = 0) :
06203         object{ std::make_shared<GgVertexArray>() },
06204         mode{ mode }
06205     {
06206     }
06207
06208     virtual ~GgShape()
06209     {
06210     }
06211
06212     virtual explicit operator bool() const noexcept
06213     {
06214         return object.get() != nullptr;
06215     }
06216
06217     virtual bool operator!() const noexcept
06218     {
06219     }

```

```

06285         return !static_cast<bool>(*this);
06286     }
06287
06293     const GLuint& get() const
06294     {
06295         return object->get();
06296     }
06297
06303     void setMode(GLenum mode)
06304     {
06305         this->mode = mode;
06306     }
06307
06313     const GLenum& getMode() const
06314     {
06315         return this->mode;
06316     }
06317
06324     virtual void draw(GLint first = 0, GLsizei count = 0) const
06325     {
06326         object->bind();
06327     }
06328 };
06329
06333 class GgPoints
06334 : public GgShape
06335 {
06336     // 頂点バッファオブジェクト
06337     std::shared_ptr<GgBuffer<GgVector>> position;
06338
06339 public:
06340
06344     GgPoints(GLenum mode = GL_POINTS) :
06345         GgShape(mode)
06346     {
06347     }
06348
06357     GgPoints(
06358         const GgVector* pos,
06359         GLsizei countv,
06360         GLenum mode = GL_POINTS,
06361         GLenum usage = GL_STATIC_DRAW
06362     ) :
06363         GgPoints(mode)
06364     {
06365         load(pos, countv, usage);
06366     }
06367
06371     virtual ~GgPoints()
06372     {
06373     }
06374
06380     explicit operator bool() const noexcept
06381     {
06382         return position.get() != nullptr;
06383     }
06384
06390     bool operator!() const noexcept
06391     {
06392         return !static_cast<bool>(*this);
06393     }
06394
06400     const GLsizei& getCount() const
06401     {
06402         return position->getCount();
06403     }
06404
06410     const GLuint& getBuffer() const
06411     {
06412         return position->getBuffer();
06413     }
06414
06422     void send(const GgVector* pos, GLint first = 0, GLsizei count = 0) const
06423     {
06424         position->send(pos, first, count);
06425     }
06426
06434     void load(const GgVector* pos, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06435
06442     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06443 };
06444
06448 struct GgVertex
06449 {
06451     GgVector position;
06452
06454     GgVector normal;

```

```

06455
06459     GgVertex()
06460     {
06461     }
06462
06469     GgVertex(const GgVector& pos, const GgVector& norm) :
06470         position(pos),
06471         normal(norm)
06472     {
06473     }
06474
06485     GgVertex(
06486         GLfloat px, GLfloat py, GLfloat pz,
06487         GLfloat nx, GLfloat ny, GLfloat nz
06488     ) :
06489         position{ px, py, pz, 1.0f },
06490         normal{ nx, ny, nz, 0.0f }
06491     {
06492     }
06493
06500     GgVertex(const GLfloat* pos, const GLfloat* norm) :
06501         GgVertex(pos[0], pos[1], pos[2], norm[0], norm[1], norm[2])
06502     {
06503     }
06504 };
06505
06509     class GgTriangles
06510     : public GgShape
06511     {
06512     // 頂点属性
06513         std::shared_ptr<GgBuffer<GgVertex>> vertex;
06514
06515     public:
06516
06522         GgTriangles(GLenum mode = GL_TRIANGLES) :
06523             GgShape(mode)
06524         {
06525         }
06526
06535         GgTriangles(
06536             const GgVertex* vert,
06537             GLsizei count,
06538             GLenum mode = GL_TRIANGLES,
06539             GLenum usage = GL_STATIC_DRAW
06540         ) :
06541             GgTriangles(mode)
06542         {
06543             load(vert, count, usage);
06544         }
06545
06549         virtual ~GgTriangles()
06550         {
06551         }
06552
06558         const GLsizei& getCount() const
06559         {
06560             return vertex->getCount();
06561         }
06562
06568         const GLuint& getBuffer() const
06569         {
06570             return vertex->getBuffer();
06571         }
06572
06580         void send(const GgVertex* vert, GLint first = 0, GLsizei count = 0) const
06581         {
06582             vertex->send(vert, first, count);
06583         }
06584
06592         void load(const GgVertex* vert, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06593
06600         virtual void draw(GLint first = 0, GLsizei count = 0) const;
06601     };
06602
06606     class GgElements
06607     : public GgTriangles
06608     {
06609     // インデックスを格納する頂点バッファオブジェクト
06610         std::shared_ptr<GgBuffer<GLuint>> index;
06611
06612     public:
06613
06619         GgElements(GLenum mode = GL_TRIANGLES) :
06620             GgTriangles(mode)
06621         {
06622         }
06623

```

```

06634     GgElements(
06635         const GgVertex* vert,
06636         GLsizei countv,
06637         const GLuint* face,
06638         GLsizei countf,
06639         GLenum mode = GL_TRIANGLES,
06640         GLenum usage = GL_STATIC_DRAW
06641     ) :
06642         GgElements(mode)
06643     {
06644         load(vert, countv, face, countf, usage);
06645     }
06646
06650     virtual ~GgElements()
06651     {
06652     }
06653
06658     const GLsizei& getIndexCount() const
06659     {
06660         return index->getCount();
06661     }
06662
06668     const GLuint& getIndexBuffer() const
06669     {
06670         return index->getBuffer();
06671     }
06672
06683     void send(
06684         const GgVertex* vert,
06685         GLuint firstv,
06686         GLsizei countv,
06687         const GLuint* face = nullptr,
06688         GLuint firstf = 0,
06689         GLsizei countf = 0
06690     ) const
06691     {
06692         GgTriangles::send(vert, firstv, countv);
06693         if (face != nullptr && countf > 0) index->send(face, firstf, countf);
06694     }
06695
06705     void load(
06706         const GgVertex* vert,
06707         GLsizei countv,
06708         const GLuint* face,
06709         GLsizei countf,
06710         GLenum usage = GL_STATIC_DRAW
06711     )
06712     {
06713         // 頂点バッファオブジェクトを作成する
06714         GgTriangles::load(vert, countv, usage);
06715
06716         // インデックスの頂点バッファオブジェクトを作成する
06717         index = std::make_shared<GgBuffer<GLuint>>(GLELEMENT_ARRAY_BUFFER, face,
06718             static_cast<GLsizei>(sizeof(GLuint)), countf, usage);
06719     }
06720
06726     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06727 };
06728
06739 extern std::shared_ptr<GgPoints> ggPointsCube(
06740     GLsizei countv,
06741     GLfloat length = 1.0f,
06742     GLfloat cx = 0.0f,
06743     GLfloat cy = 0.0f,
06744     GLfloat cz = 0.0f
06745 );
06746
06757 extern std::shared_ptr<GgPoints> ggPointsSphere(
06758     GLsizei countv,
06759     GLfloat radius = 0.5f,
06760     GLfloat cx = 0.0f,
06761     GLfloat cy = 0.0f,
06762     GLfloat cz = 0.0f
06763 );
06764
06772 extern std::shared_ptr<GgTriangles> ggRectangle(
06773     GLfloat width = 1.0f,
06774     GLfloat height = 1.0f
06775 );
06776
06785 extern std::shared_ptr<GgTriangles> ggEllipse(
06786     GLfloat width = 1.0f,
06787     GLfloat height = 1.0f,
06788     GLuint slices = 16
06789 );
06790
06802 extern std::shared_ptr<GgTriangles> ggArraysObj(

```



```

06803     const std::string& name,
06804     bool normalize = false
06805 );
06806
06818 extern std::shared_ptr<GgElements> ggElementsObj(
06819     const std::string& name,
06820     bool normalize = false
06821 );
06822
06835 extern std::shared_ptr<GgElements> ggElementsMesh(
06836     GLuint slices,
06837     GLuint stacks,
06838     const GLfloat(*pos)[3],
06839     const GLfloat(*norm)[3] = nullptr
06840 );
06841
06852 extern std::shared_ptr<GgElements> ggElementsSphere(
06853     GLfloat radius = 1.0f,
06854     int slices = 16,
06855     int stacks = 8
06856 );
06857
06864 class GgShader
06865 {
06866     // プログラム名
06867     const GLuint program;
06868
06869 public:
06870
06880     GgShader(
06881         const std::string& vert,
06882         const std::string& frag = "",
06883         const std::string& geom = "",
06884         int nvarying = 0,
06885         const char* const* varyings = nullptr
06886     ) :
06887         program(ggLoadShader(vert, frag, geom, nvarying, varyings))
06888     {
06889     }
06890
06898     GgShader(
06899         const std::array<std::string, 3>& files,
06900         int nvarying = 0,
06901         const char* const* varyings = nullptr
06902     ) :
06903         GgShader(files[0], files[1], files[2], nvarying, varyings)
06904     {
06905     }
06906
06912     GgShader(const GgShader& shader) = delete;
06913
06919     GgShader(GgShader&& shader) = default;
06920
06924     virtual ~GgShader()
06925     {
06926         // 参照しているオブジェクトが一つだけならシェーダを削除する
06927         glUseProgram(0);
06928         glDeleteProgram(program);
06929     }
06930
06937     GgShader& operator=(const GgShader& shader) = delete;
06938
06945     GgShader& operator=(GgShader&& shader) = default;
06946
06950     void use() const
06951     {
06952         glUseProgram(program);
06953     }
06954
06958     void unuse() const
06959     {
06960         glUseProgram(0);
06961     }
06962
06968     GLuint get() const
06969     {
06970         return program;
06971     }
06972 };
06973
06977 class GgPointShader
06978 {
06979     // シェーダー
06980     std::shared_ptr<GgShader> shader;
06981
06982     // 投影変換行列の uniform 変数の場所
06983     GLint mpLoc;

```

```

06984
06985 // モデルビュー変換行列の uniform 変数の場所
06986 GLint mvLoc;
06987
06988 public:
06989
06993 GgPointShader() :
06994     mpLoc{ -1 },
06995     mvLoc{ -1 }
06996 {
06997 }
06998
07008 GgPointShader(
07009     const std::string& vert,
07010     const std::string& frag = "",
07011     const std::string& geom = "",
07012     GLint nvarying = 0,
07013     const char* const* varyings = nullptr
07014 ) :
07015     GgPointShader()
07016 {
07017     load(vert, frag, geom, nvarying, varyings);
07018 }
07019
07027 GgPointShader(
07028     const std::array<std::string, 3>& files,
07029     int nvarying = 0,
07030     const char* const* varyings = nullptr
07031 ) :
07032     GgPointShader(files[0], files[1], files[2], nvarying, varyings)
07033 {
07034 }
07035
07039 virtual ~GgPointShader()
07040 {
07041 }
07042
07053 bool load(
07054     const std::string& vert,
07055     const std::string& frag = "",
07056     const std::string& geom = "",
07057     GLint nvarying = 0,
07058     const char* const* varyings = nullptr
07059 )
07060 {
07061     // シェーダを作成する
07062     shader = std::make_shared<GgShader>(vert, frag, geom, nvarying, varyings);
07063
07064     // プログラム名を取り出す
07065     const GLuint program(shader->get());
07066
07067     // プログラムオブジェクトが作成できていなければ戻る
07068     if (program == 0) return false;
07069
07070     // 変換行列の uniform 変数の場所
07071     mpLoc = glGetUniformLocation(program, "mp");
07072     mvLoc = glGetUniformLocation(program, "mv");
07073
07074     // プログラムオブジェクトの作成に成功した
07075     return true;
07076 }
07077
07086 bool load(
07087     const std::array<std::string, 3>& files,
07088     GLint nvarying = 0,
07089     const char* const* varyings = nullptr
07090 )
07091 {
07092     return load(files[0], files[1], files[2], nvarying, varyings);
07093 }
07094
07100 virtual void loadProjectionMatrix(const GLfloat* mp) const
07101 {
07102     glUniformMatrix4fv(mpLoc, 1, GL_FALSE, mp);
07103 }
07104
07110 virtual void loadProjectionMatrix(const GgMatrix& mp) const
07111 {
07112     loadProjectionMatrix(mp.get());
07113 }
07114
07120 virtual void loadModelviewMatrix(const GLfloat* mv) const
07121 {
07122     glUniformMatrix4fv(mvLoc, 1, GL_FALSE, mv);
07123 }
07124
07130 virtual void loadModelviewMatrix(const GgMatrix& mv) const

```

```

07131     {
07132         loadModelviewMatrix(mv.get());
07133     }
07134
07141     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07142     {
07143         loadProjectionMatrix(mp);
07144         loadModelviewMatrix(mv);
07145     }
07146
07153     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07154     {
07155         loadMatrix(mp.get(), mv.get());
07156     }
07157
07161     virtual void use() const
07162     {
07163         shader->use();
07164     }
07165
07171     void use(const GLfloat* mp) const
07172     {
07173         use();
07174         loadProjectionMatrix(mp);
07175     }
07176
07182     void use(const GgMatrix& mp) const
07183     {
07184         use(mp.get());
07185     }
07186
07193     void use(const GLfloat* mp, const GLfloat* mv) const
07194     {
07195         use(mp);
07196         loadModelviewMatrix(mv);
07197     }
07198
07205     void use(const GgMatrix& mp, const GgMatrix& mv) const
07206     {
07207         use(mp.get(), mv.get());
07208     }
07209
07213     void unuse() const
07214     {
07215         shader->unuse();
07216     }
07217
07223     GLuint get() const
07224     {
07225         return shader->get();
07226     }
07227 };
07228
07232 class GgSimpleShader
07233 : public GgPointShader
07234 {
07235     // モデルビュー変換の法線変換行列の uniform 変数の場所
07236     GLint mnLoc;
07237
07238 public:
07239
07243     GgSimpleShader() :
07244         GgPointShader(),
07245         mnLoc{ -1 }
07246     {
07247     }
07248
07258     GgSimpleShader(
07259         const std::string& vert,
07260         const std::string& frag = "",
07261         const std::string& geom = "",
07262         GLint nvarying = 0,
07263         const char* const* varyings = nullptr
07264     )
07265     {
07266         load(vert, frag, geom, nvarying, varyings);
07267     }
07268
07276     GgSimpleShader(
07277         const std::array<std::string, 3>& files,
07278         GLint nvarying = 0,
07279         const char* const* varyings = nullptr
07280     ) :
07281         GgSimpleShader(files[0], files[1], files[2], nvarying, varyings)
07282     {
07283     }
07284

```

```

07288     GgSimpleShader(const GgSimpleShader& o) :
07289         GgPointShader(o),
07290         mnLoc{ o.mnLoc }
07291     {
07292     }
07293
07297     virtual ~GgSimpleShader()
07298     {
07299     }
07300
07307     GgSimpleShader& operator=(const GgSimpleShader& o)
07308     {
07309         if (&o != this)
07310         {
07311             GgPointShader::operator=(o);
07312             mnLoc = o.mnLoc;
07313         }
07314
07315         return *this;
07316     }
07317
07328     bool load(
07329         const std::string& vert,
07330         const std::string& frag = "",
07331         const std::string& geom = "",
07332         GLint nvarying = 0,
07333         const char* const* varyings = nullptr
07334     );
07335
07344     bool load(
07345         const std::array<std::string, 3>& files,
07346         GLint nvarying = 0,
07347         const char* const* varyings = nullptr
07348     )
07349     {
07350         return load(files[0], files[1], files[2], nvarying, varyings);
07351     }
07352
07359     virtual void loadModelviewMatrix(const GLfloat* mv, const GLfloat* mn) const
07360     {
07361         GgPointShader::loadModelviewMatrix(mv);
07362         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07363     }
07364
07371     virtual void loadModelviewMatrix(const GgMatrix& mv, const GgMatrix& mn) const
07372     {
07373         loadModelviewMatrix(mv.get(), mn.get());
07374     }
07375
07381     virtual void loadModelviewMatrix(const GLfloat* mv) const
07382     {
07383         loadModelviewMatrix(mv, GgMatrix(mv).normal().get());
07384     }
07385
07391     virtual void loadModelviewMatrix(const GgMatrix& mv) const
07392     {
07393         loadModelviewMatrix(mv.get());
07394     }
07395
07403     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
07404     {
07405         GgPointShader::loadMatrix(mp, mv);
07406         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07407     }
07408
07416     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
07417     {
07418         loadMatrix(mp.get(), mv.get(), mn.get());
07419     }
07420
07427     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07428     {
07429         loadMatrix(mp, mv, GgMatrix(mv).normal());
07430     }
07431
07438     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07439     {
07440         loadMatrix(mp, mv, mv.normal());
07441     }
07442
07446     struct Light
07447     {
07448         GgVector ambient;
07449         GgVector diffuse;
07450         GgVector specular;
07451         GgVector position;
07452     };

```

```

07453
07454 class LightBuffer
07455 : public GgUniformBuffer<Light>
07456 {
07457 public:
07458
07459     LightBuffer(
07460         const Light* light = nullptr,
07461         GLsizei count = 1,
07462         GLenum usage = GL_STATIC_DRAW
07463     ) :
07464         GgUniformBuffer<Light>(light, count, usage)
07465     {
07466     }
07467
07468     LightBuffer(
07469         const Light& light,
07470         GLsizei count = 1,
07471         GLenum usage = GL_STATIC_DRAW
07472     ) :
07473         GgUniformBuffer<Light>(light, count, usage)
07474     {
07475     }
07476
07477     LightBuffer(
07478         GgVector ambient,
07479         GgVector diffuse,
07480         GgVector specular,
07481         GgVector position,
07482         GLsizei count = 1,
07483         GLenum usage = GL_STATIC_DRAW
07484     ) :
07485         GgUniformBuffer<Light>(Light{ ambient, diffuse, specular, position }, count, usage)
07486     {
07487     }
07488
07489     virtual ~LightBuffer()
07490     {
07491     }
07492
07493     void loadAmbient(
07494         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07495         GLint first = 0, GLsizei count = 1
07496     ) const;
07497
07498     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07499
07500     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07501     {
07502         // first 番目のブロックから count 個の ambient 要素に値を設定する
07503         send(ambient, offsetof(Light, ambient), sizeof(Light::ambient), first, count);
07504     }
07505
07506     void loadDiffuse(
07507         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07508         GLint first = 0, GLsizei count = 1
07509     ) const;
07510
07511     void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07512
07513     void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07514     {
07515         // first 番目のブロックから count 個の diffuse 要素に値を設定する
07516         send(diffuse, offsetof(Light, diffuse), sizeof(Light::diffuse), first, count);
07517     }
07518
07519     void loadSpecular(
07520         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07521         GLint first = 0, GLsizei count = 1
07522     ) const;
07523
07524     void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07525
07526     void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07527     {
07528         // first 番目のブロックから count 個の specular 要素に値を設定する
07529         send(specular, offsetof(Light, specular), sizeof(Light::specular), first, count);
07530     }
07531
07532     void loadColor(const Light& color, GLint first = 0, GLsizei count = 1) const;
07533
07534     void loadPosition(
07535         GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f,
07536         GLint first = 0, GLsizei count = 1
07537     ) const;
07538
07539     void loadPosition(const GgVector& position, GLint first = 0, GLsizei count = 1) const;

```

```

07666
07674 void loadPosition(const GLfloat* position, GLint first = 0, GLsizei count = 1) const
07675 {
07676     // first 番目のブロックから count 個の position 要素に値を設定する
07677     send(position, offsetof(Light, position), sizeof(Light::position), first, count);
07678 }
07679
07687 void loadPosition(const GgVector* position, GLint first = 0, GLsizei count = 1) const
07688 {
07689     loadPosition(position->data(), first, count);
07690 }
07691
07699 void load(const Light* light, GLint first = 0, GLsizei count = 1) const
07700 {
07701     send(light, 0, sizeof(Light), first, count);
07702 }
07703
07711 void load(const Light& light, GLint first = 0, GLsizei count = 1) const
07712 {
07713     load(&light, first, count);
07714 }
07715
07721 void select(GLint i = 0) const
07722 {
07723     // バッファオブジェクトの i 番目のブロックの位置
07724     const GLintptr offset(static_cast<GLintptr>(getStride()) * i);
07725     glBindBufferRange(getTarget(), LightBindingPoint, getBuffer(), offset, sizeof(Light));
07726 }
07727 };
07728
07732 struct Material
07733 {
07734     GgVector ambient;
07735     GgVector diffuse;
07736     GgVector specular;
07737     GLfloat shininess;
07738 };
07739
07743 class MaterialBuffer
07744     : public GgUniformBuffer<Material>
07745 {
07746 public:
07747
07755     MaterialBuffer(
07756         const Material* material = nullptr,
07757         GLsizei count = 1,
07758         GLenum usage = GL_STATIC_DRAW
07759     ) :
07760         GgUniformBuffer<Material>(material, count, usage)
07761     {
07762     }
07763
07771     MaterialBuffer(
07772         const Material& material,
07773         GLsizei count = 1,
07774         GLenum usage = GL_STATIC_DRAW
07775     ) :
07776         GgUniformBuffer<Material>(material, count, usage)
07777     {
07778     }
07779
07790     MaterialBuffer(
07791         GgVector ambient,
07792         GgVector diffuse,
07793         GgVector specular,
07794         GLfloat shininess,
07795         GLsizei count = 1,
07796         GLenum usage = GL_STATIC_DRAW
07797     ) :
07798         GgUniformBuffer<Material>(Material{ ambient, diffuse, specular, shininess }, count, usage)
07799     {
07800     }
07801
07805     virtual ~MaterialBuffer()
07806     {
07807     }
07808
07819     void loadAmbient(
07820         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07821         GLint first = 0, GLsizei count = 1
07822     ) const;
07823
07831     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07832
07840     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07841     {
07842         // first 番目のブロックから count 個のブロックの ambient 要素に値を設定する

```

```

07843     send(ambient, offsetof(Material, ambient), sizeof(Material::ambient), first, count);
07844 }
07845
07856 void loadDiffuse(
07857     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07858     GLint first = 0, GLsizei count = 1
07859 ) const;
07860
07868 void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07869
07877 void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07878 {
07879     // first 番目のブロックから count 個の diffuse 要素に値を設定する
07880     send(diffuse, offsetof(Material, diffuse), sizeof(Material::diffuse), first, count);
07881 }
07882
07893 void loadAmbientAndDiffuse(
07894     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07895     GLint first = 0, GLsizei count = 1
07896 ) const;
07897
07905 void loadAmbientAndDiffuse(const GgVector& color, GLint first = 0, GLsizei count = 1) const;
07906
07914 void loadAmbientAndDiffuse(const GLfloat* color, GLint first = 0, GLsizei count = 1) const;
07915
07926 void loadSpecular(
07927     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07928     GLint first = 0, GLsizei count = 1
07929 ) const;
07930
07938 void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07939
07947 void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07948 {
07949     // first 番目のブロックから count 個の specular 要素に値を設定する
07950     send(specular, offsetof(Material, specular), sizeof(Material::specular), first, count);
07951 }
07952
07960 void loadShininess(GLfloat shininess, GLint first = 0, GLsizei count = 1) const;
07961
07969 void loadShininess(const GLfloat* shininess, GLint first = 0, GLsizei count = 1) const;
07970
07978 void load(const Material* material, GLint first = 0, GLsizei count = 1) const
07979 {
07980     send(material, 0, sizeof(Material), first, count);
07981 }
07982
07990 void load(const Material& material, GLint first = 0, GLsizei count = 1) const
07991 {
07992     load(&material, first, count);
07993 }
07994
08000 void select(GLint i = 0) const
08001 {
08002     // バッファオブジェクトの i 番目のブロックの位置
08003     const GLintptr offset{ static_cast<GLintptr>(getStride()) * i };
08004     glBindBufferRange(getTarget(), MaterialBindingPoint, getBuffer(), offset, sizeof(Material));
08005 }
08006 };
08007
08011 void use() const
08012 {
08013     // プログラムオブジェクトは基底クラスで指定する
08014     GgPointShader::use();
08015 }
08016
08024 void use(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
08025 {
08026     // プログラムオブジェクトを指定する
08027     use();
08028
08029     // 変換行列を設定する
08030     loadMatrix(mp, mv, mn);
08031 }
08032
08040 void use(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
08041 {
08042     use(mp.get(), mv.get(), mn.get());
08043 }
08044
08051 void use(const GLfloat* mp, const GLfloat* mv) const
08052 {
08053     use(mp, mv, GgMatrix(mv).normal().get());
08054 }
08055
08062 void use(const GgMatrix& mp, const GgMatrix& mv) const
08063 {

```

```

08064     use(mp, mv, mv.normal());
08065 }
08066
08073 void use(const LightBuffer* light, GLint i = 0) const
08074 {
08075     // プログラムオブジェクトを指定する
08076     use();
08077
08078     // 光源を設定する
08079     light->select(i);
08080 }
08081
08088 void use(const LightBuffer& light, GLint i = 0) const
08089 {
08090     use(&light, i);
08091 }
08092
08102 void use(
08103     const GLfloat* mp,
08104     const GLfloat* mv,
08105     const GLfloat* mn,
08106     const LightBuffer* light,
08107     GLint i = 0
08108 ) const
08109 {
08110     // 光源を指定してプログラムオブジェクトを指定する
08111     use(light, i);
08112
08113     // 変換行列を設定する
08114     loadMatrix(mp, mv, mn);
08115 }
08116
08126 void use(
08127     const GgMatrix& mp,
08128     const GgMatrix& mv,
08129     const GgMatrix& mn,
08130     const LightBuffer& light,
08131     GLint i = 0
08132 ) const
08133 {
08134     use(mp.get(), mv.get(), mn.get(), &light, i);
08135 }
08136
08145 void use(
08146     const GLfloat* mp,
08147     const GLfloat* mv,
08148     const LightBuffer* light,
08149     GLint i = 0
08150 ) const
08151 {
08152     use(mp, mv, GgMatrix(mv).normal().get(), light, i);
08153 }
08154
08163 void use(
08164     const GgMatrix& mp,
08165     const GgMatrix& mv,
08166     const LightBuffer& light,
08167     GLint i = 0
08168 ) const
08169 {
08170     use(mp, mv, mv.normal(), light, i);
08171 }
08172
08180 void use(const GLfloat* mp, const LightBuffer* light, GLint i = 0) const
08181 {
08182     // 光源を指定してプログラムオブジェクトを指定する
08183     use(light, i);
08184
08185     // 投影変換行列を設定する
08186     loadProjectionMatrix(mp);
08187 }
08188
08196 void use(const GgMatrix& mp, const LightBuffer& light, GLint i = 0) const
08197 {
08198     // 光源を指定してプログラムオブジェクトを指定する
08199     use(mp.get(), &light, i);
08200 }
08201 };
08202
08213 extern bool ggLoadSimpleObj(
08214     const std::string& name,
08215     std::vector<std::array<GLuint, 3>>& group,
08216     std::vector<GgSimpleShader::Material>& material,
08217     std::vector<GgVertex>& vert,
08218     bool normalize = false
08219 );
08220

```



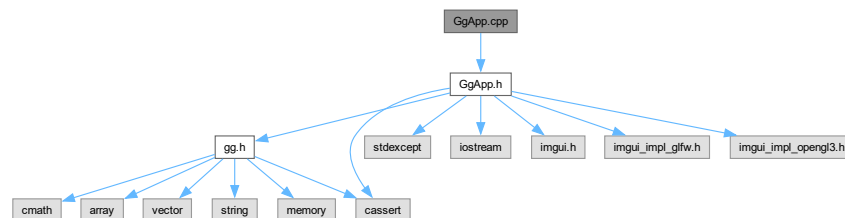
```

08232 extern bool ggLoadSimpleObj(
08233     const std::string& name,
08234     std::vector<std::array<GLuint, 3>>& group,
08235     std::vector<GgSimpleShader::Material>& material,
08236     std::vector<GgVertex>& vert,
08237     std::vector<GLuint>& face,
08238     bool normalize = false
08239 );
08240
08241 class GgSimpleObj
08242 {
08243     // 同じ材質を割り当てるポリゴングループごとの三角形数
08244     std::shared_ptr<std::vector<std::array<GLuint, 3>>> group;
08245
08246     // ポリゴングループごとの材質のユニフォームバッファ
08247     std::shared_ptr<GgSimpleShader::MaterialBuffer> material;
08248
08249     // この図形の形状データ
08250     std::shared_ptr<GgElements> data;
08251
08252 public:
08253     GgSimpleObj(const std::string& name, bool normalize = false);
08254
08255     virtual ~GgSimpleObj()
08256     {
08257     }
08258
08259     explicit operator bool() const noexcept
08260     {
08261         return data.get() != nullptr;
08262     }
08263
08264     bool operator!() const noexcept
08265     {
08266         return !static_cast<bool>(*this);
08267     }
08268
08269     const GgTriangles* get() const
08270     {
08271         return data.get();
08272     }
08273
08274     virtual void draw(GLint first = 0, GLsizei count = 0) const;
08275 };
08276 }

```

9.43 GgApp.cpp ファイル

#include "GgApp.h"
GgApp.cpp の依存先関係図:



9.43.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの実装.

著者

Kohe Tokoi

日付

July 27, 2025

[GgApp.cpp](#) に定義があります。

9.44 GgApp.cpp

[詳解]

```
00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム、GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 #include "GgApp.h"
00026
00027 namespace
00028 {
00029     //
00030     // GLFW のエラー表示
00031     //
00032     void glfwErrorCallback(int error, const char* description)
00033     {
00034         #if defined(_aarch64_)
00035             if (error == 65544) return;
00036         #endif
00037         throw std::runtime_error(description);
00038     }
00039 }
00040
00041 //
00042 // GgApp クラスのコンストラクタ
00043 //
00044 GgApp::GgApp(int major, int minor)
00045 {
00046     // GLFW のエラー処理関数を登録する
00047     glfwSetErrorCallback(glfwErrorCallback);
00048
00049     // GLFW を初期化する
00050     if (glfwInit() == GLFW_FALSE) throw std::runtime_error("Can't initialize GLFW");
00051
00052     // OpenGL の major 番号が指定されていれば
00053     if (major > 0)
00054     {
00055         // OpenGL のバージョンを指定する
00056         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, major);
00057         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, minor);
00058     }
00059
00060     #if defined(GL_GLES_PROTOTYPES)
00061         // OpenGL ES 3 のコンテキストを指定する
00062         glfwWindowHint(GLFW_CLIENT_API, GLFW_OPENGL_ES_API);
00063         glfwWindowHint(GLFW_CONTEXT_CREATION_API, GLFW_EGL_CONTEXT_API);
00064     #endif
00065 }
```

```

00070 #else
00071     // OpenGL Version 3.2 以降なら
00072     if (major * 10 + minor >= 32)
00073     {
00074         // Core Profile を選択する (macOS の都合)
00075         glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
00076         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
00077     }
00078 #endif
00079 }
00080
00081 #if defined(GG_USE_OCULUS_RIFT)
00082     // Oculus Rift では SRGB でレンダリングする
00083     glfwWindowHint(GLFW_SRGB_CAPABLE, GL_TRUE);
00084 #endif
00085
00086 #if defined(IMGUI_VERSION)
00087     // ImGui のバージョンをチェックする
00088     ImGui::CheckVersion();
00089
00090     // ImGui のコンテキストを作成する
00091     ImGui::CreateContext();
00092 #endif
00093 }
00094
00095 //
00096 // デストラクタ
00097 //
00098 GgApp::~GgApp()
00099 {
00100     #if defined(IMGUI_VERSION)
00101         // Shutdown Platform/Renderer bindings
00102         ImGui::ImplOpenGL3_Shutdown();
00103         ImGui::ImplGlfw_Shutdown();
00104         ImGui::DestroyContext();
00105     #endif
00106
00107     // プログラム終了時に GLFW を終了する
00108     glfwTerminate();
00109 }
00110
00111 //
00112 // マウスや矢印キーによる平行移動量を初期化する
00113 //
00114 void GgApp::Window::HumanInterface::resetTranslation()
00115 {
00116     // 平行移動量を初期化する
00117     for (auto& t : translation)
00118     {
00119         std::fill(t.begin(), t.end(), GgVector{ 0.0f, 0.0f, 0.0f, 1.0f });
00120     }
00121
00122     // 矢印キーの設定値を初期化する
00123     std::fill(arrow.begin(), arrow.end(), std::array<int, 2>{ 0, 0 });
00124
00125     // マウスホイールの回転量を初期化する
00126     std::fill(wheel.begin(), wheel.end(), 0.0f);
00127 }
00128
00129 //
00130 // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00131 //
00132 void GgApp::Window::HumanInterface::calcTranslation(int button, const std::array<GLfloat, 3>&
velocity)
00133 {
00134     // マウスの相対変位
00135     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00136     const auto dx{ (mouse[0] - rotation[button].getStart(0)) * rotation[button].getScale(0) };
00137     const auto dy{ (rotation[button].getStart(1) - mouse[1]) * rotation[button].getScale(1) };
00138
00139     // 平行移動量
00140     auto& t{ translation[button] };
00141
00142     // 平行移動量の更新
00143     t[1][0] = dx * velocity[0] + t[0][0];
00144     t[1][1] = dy * velocity[1] + t[0][1];
00145
00146     // 回転量の更新
00147     rotation[button].motion(mouse[0], mouse[1]);
00148 }
00149
00150 //
00151 // ウィンドウのサイズ変更時の処理
00152 //
00153 void GgApp::Window::resize(GLFWwindow* window, int width, int height)
00154 {
00155     // このインスタンスの this ポインタを得る

```

```

00156     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00157
00158     if (instance)
00159     {
00160         // ウィンドウのサイズを保存する
00161         instance->size[0] = width;
00162         instance->size[1] = height;
00163
00164         // トラックボール処理の範囲を設定する
00165         for (auto& current_if : instance->interfaceData)
00166         {
00167             for (auto& t : current_if.rotation)
00168             {
00169                 t.region(width, height);
00170             }
00171         }
00172
00173         // ビューポートを更新する
00174         instance->updateViewport();
00175
00176         // ユーザー定義のコールバック関数の呼び出し
00177         if (instance->resizeFunc) (*instance->resizeFunc)(instance, width, height);
00178     }
00179 }
00180
00181 //
00182 // キーボードをタイプした時の処理
00183 //
00184 void GgApp::Window::keyboard(GLFWwindow* window, int key, int scancode, int action, int mods)
00185 {
00186     #if defined(IMGUI_VERSION)
00187         // ImGui がキーボードを使うときはキーボードの処理を行わない
00188         if (ImGui::GetIO().WantCaptureKeyboard) return;
00189     #endif
00190
00191     // このインスタンスの this ポインタを得る
00192     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00193
00194     if (instance && action)
00195     {
00196         // ユーザー定義のコールバック関数の呼び出し
00197         if (instance->keyboardFunc) (*instance->keyboardFunc)(instance, key, scancode, action, mods);
00198
00199         // 対象のユーザインタフェース
00200         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00201
00202         switch (key)
00203         {
00204             case GLFW_KEY_HOME:
00205
00206                 // トラックボールを初期化する
00207                 instance->resetRotation();
00208                 [[fallthrough]];
00209
00210             case GLFW_KEY_END:
00211
00212                 // 平行移動量を初期化する
00213                 instance->resetTranslation();
00214                 break;
00215
00216             case GLFW_KEY_UP:
00217
00218                 if (mods & GLFW_MOD_SHIFT)
00219                     current_if.arrow[1][1]++;
00220                 else if (mods & GLFW_MOD_CONTROL)
00221                     current_if.arrow[2][1]++;
00222                 else if (mods & GLFW_MOD_ALT)
00223                     current_if.arrow[3][1]++;
00224                 else
00225                     current_if.arrow[0][1]++;
00226                 break;
00227
00228             case GLFW_KEY_DOWN:
00229
00230                 if (mods & GLFW_MOD_SHIFT)
00231                     current_if.arrow[1][1]--;
00232                 else if (mods & GLFW_MOD_CONTROL)
00233                     current_if.arrow[2][1]--;
00234                 else if (mods & GLFW_MOD_ALT)
00235                     current_if.arrow[3][1]--;
00236                 else
00237                     current_if.arrow[0][1]--;
00238                 break;
00239
00240             case GLFW_KEY_RIGHT:
00241
00242                 if (mods & GLFW_MOD_SHIFT)

```

```

00243         current_if.arrow[1][0]++;
00244     else if (mods & GLFW_MOD_CONTROL)
00245         current_if.arrow[2][0]++;
00246     else if (mods & GLFW_MOD_ALT)
00247         current_if.arrow[3][0]++;
00248     else
00249         current_if.arrow[0][0]++;
00250     break;
00251
00252     case GLFW_KEY_LEFT:
00253
00254         if (mods & GLFW_MOD_SHIFT)
00255             current_if.arrow[1][0]--;
00256         else if (mods & GLFW_MOD_CONTROL)
00257             current_if.arrow[2][0]--;
00258         else if (mods & GLFW_MOD_ALT)
00259             current_if.arrow[3][0]--;
00260         else
00261             current_if.arrow[0][0]--;
00262         break;
00263
00264     default:
00265         break;
00266     }
00267
00268     current_if.lastKey = key;
00269 }
00270 }
00271
00272 //
00273 // マウスボタンを操作したときの処理
00274 //
00275 void GgApp::Window::mouse(GLFWwindow* window, int button, int action, int mods)
00276 {
00277     #if defined(IMGUI_VERSION)
00278         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00279         if (ImGui::GetIO().WantCaptureMouse) return;
00280     #endif
00281
00282     // このインスタンスの this ポインタを得る
00283     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00284
00285     // マウスボタンの状態を記録する
00286     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG.BUTTON_COUNT);
00287     instance->status[button] = action != GLFW_RELEASE;
00288
00289     if (instance)
00290     {
00291         // ユーザー定義のコールバック関数の呼び出し
00292         if (instance->mouseFunc) (*instance->mouseFunc)(instance, button, action, mods);
00293
00294         // 対象のユーザインタフェース
00295         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00296
00297         // マウスの現在位置を得る
00298         const auto x{ current_if.mouse[0] };
00299         const auto y{ current_if.mouse[1] };
00300
00301         if (x < 0 || x >= instance->size[0] || y < 0 || y >= instance->size[1]) return;
00302
00303         if (action)
00304         {
00305             // ドラッグ開始
00306             current_if.rotation[button].begin(x, y);
00307         }
00308         else
00309         {
00310             // ドラッグ終了
00311             current_if.translation[button][0] = current_if.translation[button][1];
00312             current_if.rotation[button].end(x, y);
00313         }
00314     }
00315 }
00316
00317 //
00318 // マウスホイールを操作した時の処理
00319 //
00320 void GgApp::Window::wheel(GLFWwindow* window, double x, double y)
00321 {
00322     #if defined(IMGUI_VERSION)
00323         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00324         if (ImGui::GetIO().WantCaptureMouse) return;
00325     #endif
00326
00327     // このインスタンスの this ポインタを得る
00328     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00329

```

```

00330     if (instance)
00331     {
00332         // ユーザー定義のコールバック関数の呼び出し
00333         if (instance->wheelFunc) (*instance->wheelFunc)(instance, x, y);
00334
00335         // 対象のユーザインタフェース
00336         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00337
00338         // マウスホイールの回転量の保存
00339         current_if.wheel[0] += static_cast<GLfloat>(x);
00340         current_if.wheel[1] += static_cast<GLfloat>(y);
00341
00342         // マウスによる平行移動量の z 値の更新
00343         const auto z{ current_if.wheel[1] * instance->velocity[2] };
00344         for (auto& t : current_if.translation) t[1][2] = z;
00345     }
00346 }
00347
00348 //
00349 // Window クラスのコンストラクタ
00350 //
00351 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, GLFWwindow*
share) :
00352     size{ width, height },
00353     fboSize{ width, height }
00354 {
00355     // ディスプレイの情報
00356     GLFWmonitor* monitor{ nullptr };
00357
00358     // フルスクリーン表示
00359     if (fullscreen > 0)
00360     {
00361         // 接続されているモニタの数を数える
00362         int mcount;
00363         auto** const monitors{ glfwGetMonitors(&mcount) };
00364
00365         // セカンダリモニタがあればそれを使う
00366         if (fullscreen > mcount) fullscreen = mcount;
00367         monitor = monitors[fullscreen - 1];
00368
00369         // モニタのモードを調べる
00370         const auto* mode{ glfwGetVideoMode(monitor) };
00371
00372         // ウィンドウのサイズをディスプレイのサイズにする
00373         width = mode->width;
00374         height = mode->height;
00375     }
00376
00377     // GLFW のウィンドウを作成する
00378     window = glfwCreateWindow(width, height, title.c_str(), monitor, share);
00379
00380     // ウィンドウが作成できなければエラー
00381     if (!window) throw std::runtime_error("Unable to open the GLFW window.");
00382
00383     // 現在のウィンドウを処理対象にする
00384     glfwMakeContextCurrent(window);
00385
00386     // ゲームグラフィックス特論の都合による初期化を行う
00387     ggInit();
00388
00389     // このインスタンスの this ポインタを記録しておく
00390     glfwSetWindowUserPointer(window, this);
00391
00392     // キーボードを操作した時の処理を登録する
00393     glfwSetKeyCallback(window, keyboard);
00394
00395     // マウスボタンを操作したときの処理を登録する
00396     glfwSetMouseButtonCallback(window, mouse);
00397
00398     // マウスホイール操作時に呼び出す処理を登録する
00399     glfwSetScrollCallback(window, wheel);
00400
00401     // ウィンドウのサイズ変更時に呼び出す処理を登録する
00402     glfwSetFramebufferSizeCallback(window, resize);
00403
00404     // 垂直同期タイミングに合わせる
00405     glfwSwapInterval(1);
00406
00407     // 実際のフレームバッファのサイズを取得する
00408     glfwGetFramebufferSize(window, &width, &height);
00409
00410     // ビューポートと投影変換行列を初期化する
00411     resize(window, width, height);
00412
00413     #if defined(IMGUI_VERSION)
00414     // 最初のウィンドウを開いたとき
00415     static bool firstTime{ true };

```

```

00416     if (firstTime)
00417     {
00418         // Setup Platform/Renderer bindings
00419         ImGui_ImplGlfw_InitForOpenGL(window, true);
00420         ImGui_ImplOpenGL3_Init(nullptr);
00421
00422         // 実行済みであることを記録する
00423         firstTime = false;
00424     }
00425 #endif
00426 }
00427
00428 //
00429 // イベントを取得してループを継続すべきかどうか調べる
00430 //
00431 GgApp::Window::operator bool()
00432 {
00433     // イベントを取り出す
00434     glfwPollEvents();
00435
00436     // ウィンドウを閉じるべきなら false を返す
00437     if (shouldClose()) return false;
00438
00439     // 対象のユーザインタフェース
00440     auto& current_if{ interfaceData[interfaceNo] };
00441
00442 #if defined(IMGUI_VERSION)
00443     // ImGui の新規フレームを作成する
00444     ImGui_ImplOpenGL3_NewFrame();
00445     ImGui_ImplGlfw_NewFrame();
00446     ImGui::NewFrame();
00447
00448     // ImGui の状態を取り出す
00449     const ImGuiIO& io{ ImGui::GetIO() };
00450
00451     // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00452     if (io.WantCaptureMouse) return true;
00453
00454     // マウスの位置を更新する
00455     current_if.mouse = std::array<GLfloat, 2>{ io.MousePos.x, io.MousePos.y };
00456 #else
00457     // マウスの現在位置を調べる
00458     double x, y;
00459     glfwGetCursorPos(window, &x, &y);
00460
00461     // マウスの位置を更新する
00462     current_if.mouse = std::array<GLfloat, 2>{ static_cast<GLfloat>(x), static_cast<GLfloat>(y) };
00463 #endif
00464
00465     // マウスドラッグ
00466     for (int button = GLFW_MOUSE_BUTTON_1; button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT; ++button)
00467     {
00468         // マウスボタンを押していたら
00469         if (status[button])
00470         {
00471             // 現在位置と平行移動量を更新する
00472             current_if.calcTranslation(button, velocity);
00473         }
00474     }
00475
00476     return true;
00477 }
00478
00479 //
00480 // カラーバッファを入れ替える
00481 //
00482 void GgApp::Window::swapBuffers() const
00483 {
00484 #if defined(IMGUI_VERSION)
00485     // ImGui の描画データがあればフレームをレンダリングする
00486     ImGui::Render();
00487     ImDrawData* data{ ImGui::GetDrawData() };
00488     if (data) ImGui_ImplOpenGL3_RenderDrawData(data);
00489 #endif
00490
00491     // エラーチェック
00492     ggError();
00493
00494     // カラーバッファを入れ替える
00495     glfwSwapBuffers(window);
00496 }
00497
00498 //
00499 // ビューポートのサイズを更新する
00500 //
00501 void GgApp::Window::updateViewport()
00502 {

```

```

00503 // フレームバッファの大きさを求める
00504 glfwGetFramebufferSize(window, &fboSize[0], &fboSize[1]);
00505
00506 #if defined(IMGUI_VERSION)
00507 // フレームバッファの高さからメニューバーの高さを減じる
00508 fboSize[1] -= menubarHeight;
00509 #endif
00510
00511 // ウィンドウの縦横比を保存する
00512 aspect = static_cast<GLfloat>(fboSize[0]) / static_cast<GLfloat>(fboSize[1]);
00513
00514 // ビューポートを設定する
00515 restoreViewport();
00516 }
00517
00518 #if defined(GG_USE_OCULUS_RIFT)
00519 # if OVR_PRODUCT_VERSION > 0
00520 //
00521 // グラフィックスカードのデフォルトの LUID を得る
00522 //
00523 ovrGraphicsLuid GgApp::Oculus::GetDefaultAdapterLuid()
00524 {
00525     ovrGraphicsLuid luid = ovrGraphicsLuid();
00526
00527     # if defined(_WIN32)
00528     IDXGIFactory* factory{ nullptr };
00529
00530     if (SUCCEEDED(CreateDXGIFactory(IID_PPV_ARGS(&factory))))
00531     {
00532         IDXGIAdapter* adapter{ nullptr };
00533
00534         if (SUCCEEDED(factory->EnumAdapters(0, &adapter)))
00535         {
00536             DXGI_ADAPTER_DESC desc;
00537
00538             adapter->GetDesc(&desc);
00539             memcpy(&luid, &desc.AdapterLuid, sizeof luid);
00540             adapter->Release();
00541         }
00542
00543         factory->Release();
00544     }
00545     # endif
00546
00547     return luid;
00548 }
00549
00550 //
00551 // グラフィックスカードの LUID の比較
00552 //
00553 int GgApp::Oculus::Compare(const ovrGraphicsLuid& lhs, const ovrGraphicsLuid& rhs)
00554 {
00555     return memcmp(&lhs, &rhs, sizeof(ovrGraphicsLuid));
00556 }
00557 # endif
00558
00559 //
00560 // コンストラクタ
00561 //
00562 GgApp::Oculus::Oculus() = default;
00563
00564 //
00565 // Oculus Rift のセッションを作成する
00566 //
00567 GgApp::Oculus& GgApp::Oculus::initialize(const Window& window)
00568 {
00569     // Oculus Rift のコンテキスト
00570     static Oculus oculus;
00571
00572     // 既に Oculus Rift のセッションが作成されていたら参照を返す
00573     if (oculus.session) return oculus;
00574
00575     // 最初に呼び出したときだけ実行する
00576     static bool firstTime{ true };
00577     if (firstTime)
00578     {
00579         // Oculus Rift (LibOVR) を初期化する
00580         ovrInitParams initParams{ ovrInit_RequestVersion, OVR_MINOR_VERSION, NULL, 0, 0 };
00581         if (OVR_FAILURE(ovr.Initialize(&initParams)))
00582             throw std::runtime_error("Can't initialize LibOVR");
00583
00584         // アプリケーションの終了時に LibOVR を終了する
00585         atexit(ovr.Shutdown);
00586
00587         // 実行済みであることを記録する
00588         firstTime = false;
00589     }

```



```

00590
00591 // Oculus Rift のセッションを作成する
00592 ovrGraphicsLuid luid;
00593 if (OVR_FAILURE(ovr_Create(&oculus.session, &luid)))
00594     throw std::runtime_error("Can't create Oculus Rift session");
00595
00596 # if OVR_PRODUCT_VERSION > 0
00597 // デフォルトのグラフィックスアダプタが使われているか確かめる
00598 if (Compare(luid, GetDefaultAdapterLuid()))
00599     throw std::runtime_error("Graphics adapter is not default");
00600 # endif
00601
00602 // session が無効ならエラー
00603 if (!oculus.session) std::runtime_error("Unable to use the Oculus Rift.");
00604
00605 // ミラー表示を行うウィンドウを設定する
00606 oculus.window = &window;
00607
00608 // Oculus Rift の情報を取り出す
00609 oculus.hmdDesc = ovr_GetHmdDesc(oculus.session);
00610
00611 # if defined(_DEBUG)
00612 // Oculus Rift の情報を表示する
00613 std::cerr
00614     << "\nProduct name: " << oculus.hmdDesc.ProductName
00615     << "\nResolution: " << oculus.hmdDesc.Resolution.w << " x " << oculus.hmdDesc.Resolution.h
00616     << "\nDefault Fov: (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].LeftTan
00617     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].DownTan
00618     << ") - (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].RightTan
00619     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].UpTan
00620     << ")\n" << (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].LeftTan
00621     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].DownTan
00622     << ") - (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].RightTan
00623     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].UpTan
00624     << ")\nMaximum Fov: (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].LeftTan
00625     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].DownTan
00626     << ") - (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].RightTan
00627     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].UpTan
00628     << ")\n" << (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].LeftTan
00629     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].DownTan
00630     << ") - (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].RightTan
00631     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].UpTan
00632     << ")\n" << std::endl;
00633 # endif
00634
00635 // Oculus Rift に転送する描画データを作成する
00636 # if OVR_PRODUCT_VERSION > 0
00637 oculus.layerData.Header.Type = ovrLayerType_EyeFov;
00638 # else
00639 oculus.layerData.Header.Type = ovrLayerType_EyeFovDepth;
00640 # endif
00641
00642 // OpenGL なので左下が原点
00643 oculus.layerData.Header.Flags = ovrLayerFlag_TextureOriginAtBottomLeft;
00644
00645 // Oculus Rift のレンダリングに使う FBO を作成する
00646 glGenFramebuffers(ovrEye_Count, oculus.oculusFbo);
00647
00648 // 全ての目について
00649 for (int eye = 0; eye < ovrEye_Count; ++eye)
00650 {
00651     // Oculus Rift の視野を取得する
00652     const auto& fov{ oculus.hmdDesc.DefaultEyeFov[ovrEyeType(eye)] };
00653
00654     // Oculus Rift 用の FBO のサイズを求める
00655     const auto textureSize{ ovr_GetFovTextureSize(oculus.session, ovrEyeType(eye), fov, 1.0f) };
00656
00657     // Oculus Rift のスクリーンのサイズを保存する
00658     oculus.screen[eye][0] = -fov.LeftTan;
00659     oculus.screen[eye][1] = fov.RightTan;
00660     oculus.screen[eye][2] = -fov.DownTan;
00661     oculus.screen[eye][3] = fov.UpTan;
00662
00663 # if OVR_PRODUCT_VERSION > 0
00664     // 描画データに視野を設定する
00665     oculus.layerData.Fov[eye] = fov;
00666
00667     // 描画データにビューポートを設定する
00668     oculus.layerData.Viewport[eye].Pos = OVR::Vector2i(0, 0);
00669     oculus.layerData.Viewport[eye].Size = textureSize;
00670
00671     // Oculus Rift 用の FBO のカラーバッファとして使うテクスチャセットの特性
00672     const ovrTextureSwapChainDesc colorDesc
00673     {
00674         ovrTexture_2D, // Type
00675         OVR_FORMAT_R8G8B8A8_UNORM_SRGB, // Format

```

```

00677     1,                                // ArraySize
00678     textureSize.w,                     // Width
00679     textureSize.h,                     // Height
00680     1,                                // MipLevels
00681     1,                                // SampleCount
00682     ovrFalse,                           // StaticImage
00683     0, 0
00684 };
00685
00686 // Oculus Rift 用の FBO のレンダーターゲットとして使うテクスチャチェーンを作成する
00687 oculus.layerData.ColorTexture[eye] = nullptr;
00688 if (OVR_SUCCESS(ovr.CreateTextureSwapChainGL(oculus.session, &colorDesc,
&oculus.layerData.ColorTexture[eye])))
00689 {
00690     // 作成したテクスチャチェーンの長さを取得する
00691     int length(0);
00692     if (OVR_SUCCESS(ovr.GetTextureSwapChainLength(oculus.session, oculus.layerData.ColorTexture[eye],
&length)))
00693     {
00694         // テクスチャチェーンの個々の要素について
00695         for (int i = 0; i < length; ++i)
00696         {
00697             // テクスチャのパラメータを設定する
00698             GLuint texId{ 0 };
00699             ovr.GetTextureSwapChainBufferGL(oculus.session, oculus.layerData.ColorTexture[eye], i,
&texId);
00700             glBindTexture(GL_TEXTURE_2D, texId);
00701             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00702             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00703             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00704             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00705         }
00706     }
00707
00708 // Oculus Rift 用の FBO のデプスバッファとして使うテクスチャを作成する
00709 glGenTextures(1, oculus.oculusDepth + eye);
00710 glBindTexture(GL_TEXTURE_2D, oculus.oculusDepth[eye]);
00711 glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT32F, textureSize.w, textureSize.h, 0,
GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
00712 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00713 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00714 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00715 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00716 }
00717
00718 # else
00719
00720 // 描画データに視野を設定する
00721 oculus.layerData.EyeFov.Fov[eye] = fov;
00722
00723 // 描画データにビューポートを設定する
00724 oculus.layerData.EyeFov.Viewport[eye].Pos = OVR::Vector2i(0, 0);
00725 oculus.layerData.EyeFov.Viewport[eye].Size = textureSize;
00726
00727 // Oculus Rift 用の FBO のカラーバッファとして使うテクスチャセットを作成する
00728 ovrSwapTextureSet* colorTexture{ nullptr };
00729 ovr.CreateSwapTextureSetGL(oculus.session, GL_SRGB8_ALPHA8, textureSize.w, textureSize.h,
&colorTexture);
00730 oculus.layerData.EyeFov.ColorTexture[eye] = colorTexture;
00731
00732 // Oculus Rift 用の FBO のデプスバッファとして使うテクスチャセットを作成する
00733 ovrSwapTextureSet* depthTexture{ nullptr };
00734 ovr.CreateSwapTextureSetGL(oculus.session, GL_DEPTH_COMPONENT32F, textureSize.w, textureSize.h,
&depthTexture);
00735 oculus.layerData.EyeFovDepth.DepthTexture[eye] = depthTexture;
00736
00737 // Oculus Rift のレンズ補正等の設定値を取得する
00738 oculus.eyeRenderDesc[eye] = ovr.GetRenderDesc(oculus.session, ovrEyeType(eye), fov);
00739
00740 # endif
00741 }
00742
00743 # if OVR_PRODUCT_VERSION > 0
00744
00745 // 姿勢のトラッキングにおける床の高さを 0 に設定する
00746 ovr.SetTrackingOriginType(oculus.session, ovrTrackingOrigin.FloorLevel);
00747
00748 // ミラー表示用の FBO を作成する
00749 const GLsizei* size{ oculus.window->getSize() };
00750 const ovrMirrorTextureDesc mirrorDesc
00751 {
00752     OVR_FORMAT_R8G8B8A8_UNORM_SRGB, // Format
00753     oculus.mirrorWidth = size[0],    // Width
00754     oculus.mirrorHeight = size[1],   // Height
00755     0                                // Flags
00756 };
00757

```

```

00758 // ミラー表示用の FBO のカラーバッファとして使うテクスチャを作成する
00759 if (OVR_SUCCESS(ovr.CreateMirrorTextureGL(oculus.session, &mirrorDesc, &oculus.mirrorTexture)))
00760 {
00761     // 作成したテクスチャのテクスチャ名を得る
00762     GLuint texId{ 0 };
00763     if (OVR_SUCCESS(ovr.GetMirrorTextureBufferGL(oculus.session, oculus.mirrorTexture, &texId)))
00764     {
00765         // ミラー表示用の FBO を作成してテクスチャをカラーバッファとして組み込む
00766         glGenFramebuffers(1, &oculus.mirrorFbo);
00767         glBindFramebuffer(GL_READ_FRAMEBUFFER, oculus.mirrorFbo);
00768         glFramebufferTexture2D(GL_READ_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, texId, 0);
00769         glFramebufferRenderbuffer(GL_READ_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, 0);
00770         glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00771     }
00772 }
00773 # else
00774 // 作成したテクスチャのテクスチャ名を得る
00775 if (OVR_SUCCESS(ovr.CreateMirrorTextureGL(oculus.session, GL_SRGB8_ALPHA8, width, height,
00776 reinterpret_cast<ovrTexture*>(&mirrorTexture))))
00777 {
00778     // ミラー表示用の FBO を作成してテクスチャをカラーバッファとして組み込む
00779     oculus.mirrorFbo = 0;
00780     glGenFramebuffers(1, &oculus.mirrorFbo);
00781     glBindFramebuffer(GL_READ_FRAMEBUFFER, oculus.mirrorFbo);
00782     glFramebufferTexture2D(GL_READ_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D,
00783 mirrorTexture->OGL.TexId, 0);
00784     glFramebufferRenderbuffer(GL_READ_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, 0);
00785     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00786 }
00787 # endif
00788 // Oculus Rift にレンダリングするときは sRGB カラースペースを使う
00789 glEnable(GL_FRAMEBUFFER_SRGB);
00790 // フロントバッファに描く
00791 glDrawBuffer(GL_FRONT);
00792 // Oculus Rift への表示では垂直同期タイミングに合わせない
00793 glfwSwapInterval(0);
00794 return oculus;
00795 }
00796 // Oculus Rift のセッションを破棄する
00797 void GgApp::Oculus::terminate()
00798 {
00799     // session が無効なら何もしない
00800     if (!session) return;
00801     // ミラー表示用の FBO を作っていたら削除する
00802     if (mirrorFbo)
00803     {
00804         glDeleteFramebuffers(1, &mirrorFbo);
00805         mirrorFbo = 0;
00806     }
00807     // ミラー表示用の FBO のカラーバッファ用のテクスチャを作っていたら削除する
00808     if (mirrorTexture)
00809     {
00810         if OVR_PRODUCT_VERSION > 0
00811             ovr_DestroyMirrorTexture(session, mirrorTexture);
00812         # else
00813             glDeleteTextures(1, &mirrorTexture->OGL.TexId);
00814             ovr_DestroyMirrorTexture(session, reinterpret_cast<ovrTexture*>(mirrorTexture));
00815         # endif
00816         mirrorTexture = nullptr;
00817     }
00818     // 全ての目について
00819     for (int eye = 0; eye < ovrEye_Count; ++eye)
00820     {
00821         // Oculus Rift へのレンダリング用の FBO を削除する
00822         glDeleteFramebuffers(1, oculusFbo + eye);
00823         oculusFbo[eye] = 0;
00824         # if OVR_PRODUCT_VERSION > 0
00825             // レンダリングターゲットに使ったテクスチャを削除する
00826             if (layerData.ColorTexture[eye])
00827             {
00828                 ovr_DestroyTextureSwapChain(session, layerData.ColorTexture[eye]);
00829                 layerData.ColorTexture[eye] = nullptr;
00830             }
00831         # endif
00832     }
00833 }

```

```

00843     }
00844
00845     // デプスバッファとして使ったテクスチャを削除する
00846     glDeleteTextures(1, oculusDepth + eye);
00847     oculusDepth[eye] = 0;
00848
00849 #   else
00850
00851     // レンダリングターゲットに使ったテクスチャを削除する
00852     auto* const colorTexture(layerData.EyeFov.ColorTexture[eye]);
00853     for (int i = 0; i < colorTexture->TextureCount; ++i)
00854     {
00855         const auto* const ctex(reinterpret_cast<ovrGLTexture*>(&colorTexture->Textures[i]));
00856         glDeleteTextures(1, &ctex->OGL.TexId);
00857     }
00858     ovr_DestroySwapTextureSet(session, colorTexture);
00859
00860     // デプスバッファとして使ったテクスチャを削除する
00861     auto* const depthTexture(layerData.EyeFovDepth.DepthTexture[eye]);
00862     for (int i = 0; i < depthTexture->TextureCount; ++i)
00863     {
00864         const auto* const dtex(reinterpret_cast<ovrGLTexture*>(&depthTexture->Textures[i]));
00865         glDeleteTextures(1, &dtex->OGL.TexId);
00866     }
00867     ovr_DestroySwapTextureSet(session, depthTexture);
00868
00869 #   endif
00870 }
00871
00872 // Oculus Rift のセッションを破棄する
00873 ovr_Destroy(session);
00874 session = nullptr;
00875
00876 // カラースペースを元に戻す
00877 glDisable(GL_FRAMEBUFFER_SRGB);
00878
00879 // バックバッファに描く
00880 glDrawBuffer(GL_BACK);
00881
00882 // 垂直同期タイミングに合わせる
00883 glfwSwapInterval(1);
00884 }
00885
00886 //
00887 // Oculus Rift による描画開始
00888 //
00889 bool GgApp::Oculus::begin()
00890 {
00891 #   if OVR_PRODUCT_VERSION > 0
00892
00893     // セッションの状態を取得する
00894     ovrSessionStatus sessionStatus;
00895     ovr_GetSessionStatus(session, &sessionStatus);
00896
00897     // アプリケーションが終了を要求しているときはウィンドウのクローズフラグを立てる
00898     if (sessionStatus.ShouldQuit) window->setClose(GLFW_TRUE);
00899
00900     // Oculus Rift に表示されていないときは戻る
00901     if (!sessionStatus.IsVisible) return false;
00902
00903     // 現在の状態をトラッキングの原点にする
00904     if (sessionStatus.ShouldRecenter) ovr_RecenterTrackingOrigin(session);
00905
00906     // HmdToEyeOffset などは実行時に変化するので毎フレーム ovr_GetRenderDesc() で ovrEyeRenderDesc を取得する
00907     const ovrEyeRenderDesc eyeRenderDesc[]
00908     {
00909         ovr_GetRenderDesc(session, ovrEyeType(0), hmdDesc.DefaultEyeFov[0]),
00910         ovr_GetRenderDesc(session, ovrEyeType(1), hmdDesc.DefaultEyeFov[1])
00911     };
00912
00913     // Oculus Rift のスクリーンのヘッドトラッキング位置からの変位を取得する
00914     const ovrPosef hmdToEyePose[]
00915     {
00916         eyeRenderDesc[0].HmdToEyePose,
00917         eyeRenderDesc[1].HmdToEyePose
00918     };
00919
00920     // 視点の姿勢情報を取得する
00921     ovr_GetEyePoses(session, frameIndex, ovrTrue, hmdToEyePose, layerData.RenderPose,
00922                     &layerData.SensorSampleTime);
00923 #   else
00924
00925     // フレームのタイミング計測開始
00926     const auto ftiming(ovr_GetPredictedDisplayTime(session, 0));
00927
00928     // sensorSampleTime の取得は可能な限り ovr_GetTrackingState() の近くで行う

```

```

00929     layerData.EyeFov.SensorSampleTime = ovr.GetTimeInSeconds();
00930
00931     // ヘッドトラッキングの状態を取得する
00932     const auto hmdState(ovr.GetTrackingState(session, ftiming, ovrTrue));
00933
00934     // Oculus Rift のスクリーンのヘッドトラッキング位置からの変位を取得する
00935     const ovrVector3f hmdToEyeViewOffset[]
00936     {
00937         eyeRenderDesc[0].HmdToEyeViewOffset,
00938         eyeRenderDesc[1].HmdToEyeViewOffset
00939     };
00940
00941     // 視点の姿勢情報を求める
00942     ovr.CalcEyePoses(hmdState.HeadPose.ThePose, hmdToEyeViewOffset, eyePose);
00943
00944 #   endif
00945
00946     return true;
00947 }
00948
00949 //
00950 // Oculus Rift の描画する目の指定
00951 //
00952 void GgApp::Oculus::select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)
00953 {
00954     #   if OVR_PRODUCT_VERSION > 0
00955
00956     // Oculus Rift にレンダリングする FBO に切り替える
00957     if (layerData.ColorTexture[eye])
00958     {
00959         // FBO のカラーバッファに使う現在のテクスチャのインデックスを取得する
00960         int curIndex;
00961         ovr.GetTextureSwapChainCurrentIndex(session, layerData.ColorTexture[eye], &curIndex);
00962
00963         // FBO のカラーバッファに使うテクスチャを取得する
00964         GLuint curTexId;
00965         ovr.GetTextureSwapChainBufferGL(session, layerData.ColorTexture[eye], curIndex, &curTexId);
00966
00967         // FBO を設定する
00968         glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
00969         glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, curTexId, 0);
00970         glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, oculusDepth[eye], 0);
00971
00972         // ビューポートを設定する
00973         const auto& vp{ layerData.Viewport[eye] };
00974         glViewport(vp.Pos.x, vp.Pos.y, vp.Size.w, vp.Size.h);
00975     }
00976
00977     // Oculus Rift の片目の位置と回転を取得する
00978     const auto& p{ layerData.RenderPose[eye].Position };
00979     const auto& o{ layerData.RenderPose[eye].Orientation };
00980
00981     #   else
00982
00983     // レンダーターゲットに描画する前にレンダーターゲットのインデックスをインクリメントする
00984     auto* const colorTexture{ layerData.EyeFov.ColorTexture[eye] };
00985     colorTexture->CurrentIndex = (colorTexture->CurrentIndex + 1) % colorTexture->TextureCount;
00986     auto* const depthTexture{ layerData.EyeFovDepth.DepthTexture[eye] };
00987     depthTexture->CurrentIndex = (depthTexture->CurrentIndex + 1) % depthTexture->TextureCount;
00988
00989     // レンダーターゲットを切り替える
00990     glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
00991     const auto& ctex{
00992         reinterpret_cast<ovrGLTexture*>(&colorTexture->Textures[colorTexture->CurrentIndex]) };
00993     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, ctex->OGL.TexId, 0);
00994     const auto& dtex{
00995         reinterpret_cast<ovrGLTexture*>(&depthTexture->Textures[depthTexture->CurrentIndex]) };
00996     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, dtex->OGL.TexId, 0);
00997
00998     // ビューポートを設定する
00999     const auto& vp{ layerData.EyeFov.Viewport[eye] };
01000     glViewport(vp.Pos.x, vp.Pos.y, vp.Size.w, vp.Size.h);
01001
01002     // Oculus Rift の片目の位置と回転を取得する
01003     const auto& p{ eyePose[eye].Position };
01004     const auto& o{ eyePose[eye].Orientation };
01005
01006     #   endif
01007
01008     // Oculus Rift のスクリーンの大きさを返す
01009     screen[0] = this->screen[eye][0];
01010     screen[1] = this->screen[eye][1];
01011     screen[2] = this->screen[eye][2];
01012     screen[3] = this->screen[eye][3];
01013
01014     // Oculus Rift の位置を返す
01015     position[0] = p.x;

```

```

01014 position[1] = p.y;
01015 position[2] = p.z;
01016
01017 // Oculus Rift の方向を返す
01018 orientation[0] = o.x;
01019 orientation[1] = o.y;
01020 orientation[2] = o.z;
01021 orientation[3] = o.w;
01022 }
01023
01024 //
01025 // Time Warp 処理に使う投影変換行列の成分の設定 (DK1, DK2)
01026 //
01027 void GgApp::Oculus::timewarp(const GgMatrix& projection)
01028 {
01029 # if OVR_PRODUCT_VERSION < 1
01030 // TimeWarp に使う変換行列の成分を設定する
01031 auto& posTimewarpProjectionDesc{ layerData.EyeFovDepth.ProjectionDesc };
01032 posTimewarpProjectionDesc.Projection22 = (projection.get()[4 * 2 + 2] + projection.get()[4 * 3 + 2])
    * 0.5f;
01033 posTimewarpProjectionDesc.Projection23 = projection.get()[4 * 2 + 3] * 0.5f;
01034 posTimewarpProjectionDesc.Projection32 = projection.get()[4 * 3 + 2];
01035 # endif
01036 }
01037
01038 //
01039 // 図形の描画を完了する (CV1 以降)
01040 //
01041 void GgApp::Oculus::commit(int eye)
01042 {
01043 # if OVR_PRODUCT_VERSION > 0
01044 // GL_COLOR_ATTACHMENT0 に割り当てられたテクスチャが* wglDXUnlockObjectsNV() によって
01045 // アンロックされるために次のフレームの処理において無効な GL_COLOR_ATTACHMENT0 が*
01046 // FBO に結合されるのを避ける
01047 glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
01048 glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, 0, 0);
01049 glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, 0, 0);
01050
01051 // 保留中の変更を layerData.ColorTexture[eye] に反映しインデックスを更新する
01052 ovrCommitTextureSwapChain(session, layerData.ColorTexture[eye]);
01053 # endif
01054 }
01055
01056 //
01057 // フレームを転送する
01058 //
01059 bool GgApp::Oculus::submit(bool mirror)
01060 {
01061 // エラーチェック
01062 ggError();
01063
01064 # if OVR_PRODUCT_VERSION > 0
01065 // 描画データを Oculus Rift に転送する
01066 const auto& const layers{ &layerData.Header };
01067 if (OVR_FAILURE(ovrSubmitFrame(session, frameIndex++, nullptr, &layers, 1))) return false;
01068 # else
01069 // Oculus Rift 上の描画位置と拡大率を求める
01070 ovrViewScaleDesc viewScaleDesc;
01071 viewScaleDesc.HmdSpaceToWorldScaleInMeters = 1.0f;
01072 viewScaleDesc.HmdToEyeViewOffset[0] = eyeRenderDesc[0].HmdToEyeViewOffset;
01073 viewScaleDesc.HmdToEyeViewOffset[1] = eyeRenderDesc[1].HmdToEyeViewOffset;
01074
01075 // 描画データを更新する
01076 layerData.EyeFov.RenderPose[0] = eyePose[0];
01077 layerData.EyeFov.RenderPose[1] = eyePose[1];
01078
01079 // 描画データを Oculus Rift に転送する
01080 const auto& const layers{ &layerData.Header };
01081 if (OVR_FAILURE(ovrSubmitFrame(session, 0, &viewScaleDesc, &layers, 1))) return false;
01082 # endif
01083
01084 // ミラー表示
01085 if (mirror)
01086 {
01087 # if OVR_PRODUCT_VERSION > 0
01088 const auto& sx1{ mirrorWidth };
01089 const auto& sy1{ mirrorHeight };
01090 # else
01091 const auto& sx1{ mirrorTexture->OGL.Header.TextureSize.w };
01092 const auto& sy1{ mirrorTexture->OGL.Header.TextureSize.h };
01093 # endif
01094
01095 // ミラー表示のウィンドウのサイズ
01096 GLsizei size[2];
01097 window->getSize(size);
01098
01099 // ミラー表示の表示領域

```

```

01100     GLint dx0{ 0 }, dx1{ size[0] }, dy0{ 0 }, dy1{ size[1] };
01101
01102     // ミラー表示がウィンドウからはみ出ないようにする
01103     if ((size[0] * sy1) < (size[1] * sx1))
01104     {
01105         const GLint ty1{ size[0] / sx1 };
01106         dy0 = (dy1 - ty1) / 2;
01107         dy1 = dy0 + ty1;
01108     }
01109     else
01110     {
01111         const GLint tx1{ size[1] / sy1 };
01112         dx0 = (dx1 - tx1) / 2;
01113         dx1 = dx0 + tx1;
01114     }
01115
01116     // レンダリング結果をミラー表示用のフレームバッファにも転送する
01117     glBindFramebuffer(GL_READ_FRAMEBUFFER, mirrorFbo);
01118     glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
01119     glBlitFramebuffer(0, sy1, sx1, 0, dx0, dy0, dx1, dy1, GL_COLOR_BUFFER_BIT, GL_NEAREST);
01120     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
01121
01122     // 残っている OpenGL コマンドを実行する
01123     glFlush();
01124 }
01125
01126 return true;
01127 }
01128 #endif

```

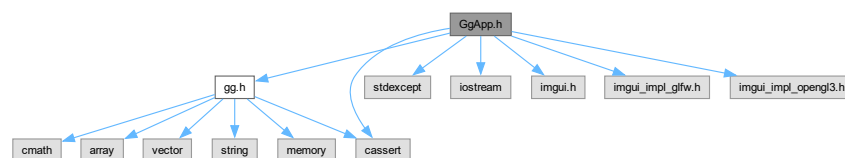
9.45 GgApp.h ファイル

```

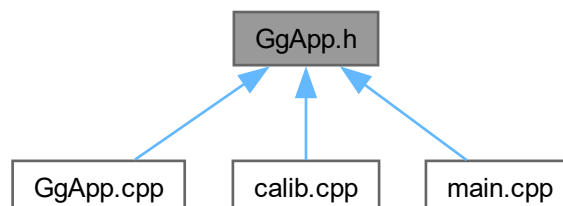
#include "gg.h"
#include <cassert>
#include <stdexcept>
#include <iostream>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

```

GgApp.h の依存先関係図:



被依存関係図:



クラス

- class [GgApp](#)
- class [GgApp::Window](#)

マクロ定義

- #define [GG_USE_IMGUI](#)
- #define [GG_BUTTON_COUNT](#) 3
- #define [GG_INTERFACE_COUNT](#) 5

9.45.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの定義.

著者

Kohe Tokoi

日付

July 17, 2025

[GgApp.h](#) に定義があります。

9.45.2 マクロ定義詳解

9.45.2.1 GG_BUTTON_COUNT

```
#define GG_BUTTON_COUNT 3
```

[GgApp.h](#) の 43 行目に定義があります。

9.45.2.2 GG_INTERFACE_COUNT

```
#define GG_INTERFACE_COUNT 5
```

[GgApp.h](#) の 48 行目に定義があります。

9.45.2.3 GG_USE_IMGUI

```
#define GG_USE_IMGUI
```

[GgApp.h](#) の 36 行目に定義があります。

9.46 GgApp.h

[詳解]

```

00001 #pragma once
00002
00003 /*
00004 ゲームグラフィックス特論用補助プログラム GLFW3 版
00005
00006 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00007
00008 Permission is hereby granted, free of charge, to any person obtaining a copy
00009 of this software and associated documentation files (the "Software"), to deal
00010 in the Software without restriction, including without limitation the rights
00011 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00012 copies or substantial portions of the Software.
00013
00014 The above copyright notice and this permission notice shall be included in
00015 all copies or substantial portions of the Software.
00016
00017 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00019 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00020 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00021 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00022 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00023
00024 */
00025
00026 // Dear ImGui を使うなら
00027 #define GG_USE_IMGUI
00028
00029 // Oculus Rift を使うなら
00030 // #define GG_USE_OCULUS_RIFT
00031
00032 // 使用するマウスのボタン数
00033 #if !defined(GG_BUTTON_COUNT)
00034 #define GG_BUTTON_COUNT 3
00035 #endif
00036
00037 // 使用するユーザインタフェースの数
00038 #if !defined(GG_INTERFACE_COUNT)
00039 #define GG_INTERFACE_COUNT 5
00040 #endif
00041
00042 // 補助プログラム
00043 #include "gg.h"
00044 using namespace gg;
00045
00046 // 標準ライブラリ
00047 #include <cassert>
00048 #include <stdexcept>
00049 #include <iostream>
00050
00051 // ImGui の組み込み
00052 #if defined(GG_USE_IMGUI)
00053 #include "imgui.h"
00054 #include "imgui_impl_glfw.h"
00055 #include "imgui_impl_opengl3.h"
00056 #endif
00057
00058 // Oculus Rift SDK ライブラリ (LibOVR) の組み込み
00059 #if defined(GG_USE_OCULUS_RIFT)
00060 #if defined(WIN32)
00061 #define GLFW_EXPOSE_NATIVE_WIN32
00062 #define GLFW_EXPOSE_NATIVE_WGL
00063 #include <GLFW/glfw3native.h>
00064 #define OVR_OS_WIN32
00065 #undef APIENTRY
00066 #pragma comment(lib, "LibOVR.lib")
00067 #endif
00068 #include <OVR.CAPI.GL.h>
00069 #include <Extras/OVRMath.h>
00070 #if OVR_PRODUCT_VERSION > 0
00071 #include <dxgi.h> // GetDefaultAdapterLuid のため
00072 #pragma comment(lib, "dxgi.lib")
00073 #endif
00074 #endif
00075
00076 class GgApp
00077 {
00078 public:
00079     GgApp(int major = 0, int minor = 1);

```

```

00099
00105 GgApp(const GgApp& w) = delete;
00106
00112 GgApp(GgApp&& w) = default;
00113
00117 virtual ~GgApp();
00118
00125 GgApp& operator=(const GgApp& w) = delete;
00126
00133 GgApp& operator=(GgApp&& w) = default;
00134
00142 int main(int argc, const char* const* argv);
00143
00150 class Window
00151 {
00152     // ウィンドウの識別子
00153     GLFWwindow* window{ nullptr };
00154
00155     // ビューポートの横幅と高さ
00156     std::array<GLsizei, 2> size;
00157
00158     // フレームバッファの横幅と高さ
00159     std::array<GLsizei, 2> fboSize;
00160
00161     #if defined(IMGUI_VERSION)
00162         // メニューバーの高さ
00163         GLsizei menubarHeight{ 0 };
00164     #endif
00165
00166     // ビューポートの縦横比
00167     GLfloat aspect{ 1.0f };
00168
00169     // マウスの移動速度[X/Y/Z]
00170     std::array<GLfloat, 3> velocity{ 1.0f, 1.0f, 0.1f };
00171
00172     // マウスボタンの状態
00173     std::array<bool, GG_BUTTON_COUNT> status{};
00174
00175     // ユーザインタフェースのデータ構造
00176     struct HumanInterface
00177     {
00178         // 最後にタイプしたキー
00179         int lastKey{ 0 };
00180
00181         // 矢印キー
00182         std::array<std::array<int, 2>, 4> arrow{};
00183
00184         // マウスの現在位置
00185         std::array<GLfloat, 2> mouse{};
00186
00187         // マウスホイールの回転量
00188         std::array<GLfloat, 2> wheel{};
00189
00190         // 平行移動量[ボタン][直前/更新][X/Y/Z]
00191         std::array<std::array<GgVector, 2>, GG_BUTTON_COUNT> translation{};
00192
00193         // トラックボール
00194         std::array<GgTrackball, GG_BUTTON_COUNT> rotation;
00195
00196         // コンストラクタ
00197         HumanInterface()
00198         {
00199             resetTranslation();
00200         }
00201
00202         //
00203         // マウスや矢印キーによる平行移動量を初期化する
00204         //
00205         void resetTranslation();
00206
00207         //
00208         // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00209         //
00210         void calcTranslation(int button, const std::array<GLfloat, 3>& velocity);
00211     };
00212
00213     // ヒューマン インタフェースデバイスのデータ
00214     std::array<HumanInterface, GG_INTERFACE_COUNT> interfaceData;
00215
00216     // ヒューマン インタフェースデバイスの番号
00217     int interfaceNo{ 0 };
00218
00219     //
00220     // ユーザー定義のコールバック関数へのポインタ
00221     //
00222     void* userPointer{ nullptr };
00223     void (*resizeFunc)(const Window* window, int width, int height){ nullptr };

```

```

00224 void (*keyboardFunc)(const Window* window, int key, int scancode, int action, int mods){ nullptr
};
00225 void (*mouseFunc)(const Window* window, int button, int action, int mods){ nullptr };
00226 void (*wheelFunc)(const Window* window, double x, double y){ nullptr };
00227
00228 //
00229 // ウィンドウのサイズ変更時の処理
00230 //
00231 static void resize(GLFWwindow* window, int width, int height);
00232
00233 //
00234 // キーボードをタイプした時の処理
00235 //
00236 static void keyboard(GLFWwindow* window, int key, int scancode, int action, int mods);
00237
00238 //
00239 // マウスボタンを操作したときの処理
00240 //
00241 static void mouse(GLFWwindow* window, int button, int action, int mods);
00242
00243 //
00244 // マウスホイールを操作した時の処理
00245 //
00246 static void wheel(GLFWwindow* window, double x, double y);
00247
00248 public:
00249
00250 Window(const std::string& title = "GLFW Window", int width = 640, int height = 480,
00260         int fullscreen = 0, GLFWwindow* share = nullptr);
00261
00262 Window(const Window& w) = delete;
00263
00264 Window(Window&& w) = default;
00265
00266 virtual ~Window()
00267 {
00268     // ウィンドウが作成されていなければ戻る
00269     if (!window) return;
00270
00271     // ウィンドウを破棄する
00272     glfwDestroyWindow(window);
00273 }
00274
00275 Window& operator=(const Window& w) = delete;
00276
00277 Window& operator=(Window&& w) = default;
00278
00279 auto* get() const
00280 {
00281     return window;
00282 }
00283
00284 void setClose(int flag = GLFW_TRUE) const
00285 {
00286     glfwSetWindowShouldClose(window, flag);
00287 }
00288
00289 bool shouldClose() const
00290 {
00291     // ウィンドウを閉じるべきなら true を返す
00292     return glfwWindowShouldClose(window) != GLFW_FALSE;
00293 }
00294
00295 explicit operator bool();
00296
00297 void swapBuffers() const;
00298
00299 void restoreViewport() const
00300 {
00301     if (!glfwGetWindowAttrib(window, GLFW_ICONIFIED)) glViewport(0, 0, fboSize[0], fboSize[1]);
00302 }
00303
00304 void updateViewport();
00305
00306 #if defined(IMGUI_VERSION)
00307 void setMenuBarHeight(GLsizei height)
00308 {
00309     // メニューバーの高さを保存する
00310     menubarHeight = height;
00311
00312     // ビューポートを復帰する
00313     updateViewport();
00314 }
00315 #endif
00316
00317 auto getWidth() const
00318 {

```

```

00383     return size[0];
00384 }
00385
00391 auto getHeight() const
00392 {
00393     return size[1];
00394 }
00395
00401 auto getFboWidth() const
00402 {
00403     return fboSize[0];
00404 }
00405
00411 auto getFboHeight() const
00412 {
00413     return fboSize[1];
00414 }
00415
00421 const auto& getSize() const
00422 {
00423     return size;
00424 }
00425
00431 void getSize(GLsizei* size) const
00432 {
00433     size[0] = getWidth();
00434     size[1] = getHeight();
00435 }
00436
00442 const auto& getFboSize() const
00443 {
00444     return fboSize;
00445 }
00446
00452 void getFboSize(GLsizei* fboSize) const
00453 {
00454     fboSize[0] = getFboWidth();
00455     fboSize[1] = getFboHeight();
00456 }
00457
00463 auto getAspect() const
00464 {
00465     return aspect;
00466 }
00467
00473 bool getKey(int key) const
00474 {
00475     #if defined(IMGUI_VERSION)
00476         // ImGui がキーボードを使うときはキーボードの処理を行わない
00477         if (ImGui::GetIO().WantCaptureKeyboard) return false;
00478     #endif
00479
00480     return glfwGetKey(window, key) != GLFW_RELEASE;
00481 }
00482
00488 void selectInterface(int no)
00489 {
00490     assert(static_cast<size_t>(no) < interfaceData.size());
00491     interfaceNo = no;
00492 }
00493
00501 void setVelocity(GLfloat vx, GLfloat vy, GLfloat vz = 0.1f)
00502 {
00503     velocity = std::array<GLfloat, 3>{ vx, vy, vz };
00504 }
00505
00511 int getLastKey()
00512 {
00513     auto& current_if{ interfaceData[interfaceNo] };
00514     const int key{ current_if.lastKey };
00515     current_if.lastKey = 0;
00516     return key;
00517 }
00518
00526 auto getArrow(int direction = 0, int mods = 0) const
00527 {
00528     const auto& current_if{ interfaceData[interfaceNo] };
00529     return static_cast<GLfloat>(current_if.arrow[mods & 3][direction & 1]);
00530 }
00531
00538 auto getArrowX(int mods = 0) const
00539 {
00540     return getArrow(0, mods);
00541 }
00542
00549 auto getArrowY(int mods = 0) const
00550 {

```

```

00551     return getArrow(1, mods);
00552 }
00553
00560 void getArrow(GLfloat* arrow, int mods = 0) const
00561 {
00562     arrow[0] = getArrowX(mods);
00563     arrow[1] = getArrowY(mods);
00564 }
00565
00571 auto getShiftArrowX() const
00572 {
00573     return getArrow(0, 1);
00574 }
00575
00581 auto getShiftArrowY() const
00582 {
00583     return getArrow(1, 1);
00584 }
00585
00591 void getShiftArrow(GLfloat* shift_arrow) const
00592 {
00593     shift_arrow[0] = getShiftArrowX();
00594     shift_arrow[1] = getShiftArrowY();
00595 }
00596
00602 auto getControlArrowX() const
00603 {
00604     return getArrow(0, 2);
00605 }
00606
00612 auto getControlArrowY() const
00613 {
00614     return getArrow(1, 2);
00615 }
00616
00622 void getControlArrow(GLfloat* control_arrow) const
00623 {
00624     control_arrow[0] = getControlArrowX();
00625     control_arrow[1] = getControlArrowY();
00626 }
00627
00633 auto getAltArrowX() const
00634 {
00635     return getArrow(0, 3);
00636 }
00637
00643 auto getAltArrowY() const
00644 {
00645     return getArrow(1, 3);
00646 }
00647
00653 void getAltArrow(GLfloat* alt_arrow) const
00654 {
00655     alt_arrow[0] = getAltArrowX();
00656     alt_arrow[1] = getAltArrowY();
00657 }
00658
00664 const auto* getMouse() const
00665 {
00666     const auto& current_if{ interfaceData[interfaceNo] };
00667     return current_if.mouse.data();
00668 }
00669
00675 void getMouse(GLfloat* position) const
00676 {
00677     const auto& current_if{ interfaceData[interfaceNo] };
00678     position[0] = current_if.mouse[0];
00679     position[1] = current_if.mouse[1];
00680 }
00681
00688 auto getMouse(int direction) const
00689 {
00690     const auto& current_if{ interfaceData[interfaceNo] };
00691     return current_if.mouse[direction & 1];
00692 }
00693
00699 auto getMouseX() const
00700 {
00701     const auto& current_if{ interfaceData[interfaceNo] };
00702     return current_if.mouse[0];
00703 }
00704
00710 auto getMouseY() const
00711 {
00712     const auto& current_if{ interfaceData[interfaceNo] };
00713     return current_if.mouse[1];
00714 }

```

```

00715
00721     const auto* getWheel() const
00722     {
00723         const auto& current_if{ interfaceData[interfaceNo] };
00724         return current_if.wheel.data();
00725     }
00726
00732     void getWheel(GLfloat* rotation) const
00733     {
00734         const auto& current_if{ interfaceData[interfaceNo] };
00735         rotation[0] = current_if.wheel[0];
00736         rotation[1] = current_if.wheel[1];
00737     }
00738
00745     auto getWheel(int direction) const
00746     {
00747         const auto& current_if{ interfaceData[interfaceNo] };
00748         return current_if.wheel[direction & 1];
00749     }
00750
00756     auto getWheelX() const
00757     {
00758         const auto& current_if{ interfaceData[interfaceNo] };
00759         return current_if.wheel[0];
00760     }
00761
00767     auto getWheelY() const
00768     {
00769         const auto& current_if{ interfaceData[interfaceNo] };
00770         return current_if.wheel[1];
00771     }
00772
00779     const auto& getTranslation(int button = GLFW_MOUSE_BUTTON_1) const
00780     {
00781         const auto& current_if{ interfaceData[interfaceNo] };
00782         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00783         return current_if.translation[button][1];
00784     }
00785
00792     auto getTranslationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00793     {
00794         const auto& current_if{ interfaceData[interfaceNo] };
00795         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00796         const auto& t{ current_if.translation[button][1] };
00797
00798         GgMatrix m
00799         {
00800             1.0f, 0.0f, 0.0f, 0.0f,
00801             0.0f, 1.0f, 0.0f, 0.0f,
00802             0.0f, 0.0f, 1.0f, 0.0f,
00803             t[0], t[1], t[2], 1.0f
00804         };
00805
00806         return m;
00807     }
00808
00815     auto getScrollMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00816     {
00817         const auto& current_if{ interfaceData[interfaceNo] };
00818         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00819         const auto& t{ current_if.translation[button][1] };
00820
00821         GgMatrix m
00822         {
00823             t[2] + 1.0f, 0.0f, 0.0f, 0.0f,
00824             0.0f, t[2] + 1.0f, 0.0f, 0.0f,
00825             0.0f, 0.0f, 1.0f, 0.0f,
00826             t[0], t[1], 0.0f, 1.0f
00827         };
00828
00829         return m;
00830     }
00831
00838     auto getRotation(int button = GLFW_MOUSE_BUTTON_1) const
00839     {
00840         const auto& current_if{ interfaceData[interfaceNo] };
00841         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00842         return current_if.rotation[button].getQuaternion();
00843     }
00844
00851     auto getRotationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00852     {
00853         const auto& current_if{ interfaceData[interfaceNo] };
00854         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00855         return current_if.rotation[button].getMatrix();
00856     }
00857

```

```

00861 void resetRotation()
00862 {
00863     // トラックボールを初期化する
00864     for (auto& tb : interfaceData[interfaceNo].rotation) tb.reset();
00865 }
00866
00870 void resetTranslation()
00871 {
00872     // 現在のインターフェースの平行移動量を初期化する
00873     interfaceData[interfaceNo].resetTranslation();
00874 }
00875
00879 void reset()
00880 {
00881     // トラックボール処理を初期化する
00882     resetRotation();
00883
00884     // 平行移動量を初期化する
00885     resetTranslation();
00886 }
00887
00893 void* getUserPointer() const
00894 {
00895     return userPointer;
00896 }
00897
00903 void setUserPointer(void* pointer)
00904 {
00905     userPointer = pointer;
00906 }
00907
00913 void setResizeFunc(void (*func)(const Window* window, int width, int height))
00914 {
00915     resizeFunc = func;
00916 }
00917
00923 void setKeyboardFunc(void (*func)(const Window* window, int key, int scancode, int action, int
00924 mods))
00925 {
00926     keyboardFunc = func;
00927 }
00933 void setMouseFunc(void (*func)(const Window* window, int button, int action, int mods))
00934 {
00935     mouseFunc = func;
00936 }
00937
00943 void setWheelFunc(void (*func)(const Window* window, double x, double y))
00944 {
00945     wheelFunc = func;
00946 }
00947 };
00948
00949 #if defined(GG_USE_OCULUS_RIFT)
00950 class Oculus
00951 {
00952     // Oculus Rift のセッション
00953     ovrSession session{ nullptr };
00954
00955     // Oculus Rift の状態
00956     ovrHmdDesc hmdDesc;
00957
00958     // Oculus Rift へのレンダリングに使う FBO
00959     GLuint oculusFbo[ovrEye_Count]{};
00960
00961     // Oculus Rift のスクリーンのサイズ
00962     GLfloat screen[ovrEye_Count][4]{ { -1.0f, 1.0f, -1.0f, 1.0f }, { -1.0f, 1.0f, -1.0f, 1.0f } };
00963
00964     // ミラー表示用の FBO
00965     GLuint mirrorFbo{ 0 };
00966
00967     // Oculus Rift のミラー表示を行うウィンドウ
00968     const Window* window{ nullptr };
00969
00970 # if OVR_PRODUCT_VERSION > 0
00971     // Oculus Rift に送る描画データ
00972     ovrLayerEyeFov layerData;
00973
00974     // Oculus Rift にレンダリングするフレームの番号
00975     long long frameIndex{ 0LL };
00976
00977     // Oculus Rift へのレンダリングに使う FBO のデプステクスチャ
00978     GLuint oculusDepth[ovrEye_Count]{};
00979
00980     // ミラー表示用の FBO のサイズ
00981     int mirrorWidth{ 1280 }, mirrorHeight{ 640 };

```

```

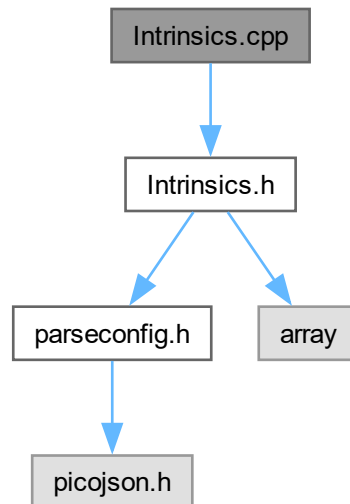
00989
00990 // ミラー表示用の FBO のカラーテクスチャ
00991 ovrMirrorTexture mirrorTexture{ nullptr };
00992
00993 //
00994 // グラフィックスカードのデフォルトの LUID を得る
00995 //
00996 static ovrGraphicsLuid GetDefaultAdapterLuid();
00997
00998 //
00999 // グラフィックスカードの LUID の比較
01000 //
01001 static int Compare(const ovrGraphicsLuid& lhs, const ovrGraphicsLuid& rhs);
01002
01003 # else
01004
01005 // Oculus Rift に送る描画データ
01006 ovrLayerUnion layerData;
01007
01008 // Oculus Rift のレンダリング情報
01009 ovrEyeRenderDesc eyeRenderDesc[ovrEye_Count];
01010
01011 // Oculus Rift の視点情報
01012 ovrPosef eyePose[ovrEye_Count];
01013
01014 // ミラー表示用の FBO のカラーテクスチャ
01015 ovrGLTexture* mirrorTexture{ nullptr };
01016
01017 # endif
01018
01019 //
01020 // コンストラクタ
01021 //
01022 Oculus();
01023
01024 //
01025 // デストラクタ
01026 //
01027 virtual ~Oculus() = default;
01028
01029 public:
01030
01031 // シングルトンなのでコピー・ムーブ禁止
01032 Oculus(const Oculus&) = delete;
01033 Oculus& operator=(const Oculus&) = delete;
01034 Oculus(Oculus&&) = delete;
01035 Oculus& operator=(Oculus&&) = delete;
01036
01042 static Oculus& initialize(const Window& window);
01043
01047 void terminate();
01048
01054 bool begin();
01055
01064 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation);
01065
01071 void timewarp(const GgMatrix& projection);
01072
01078 void commit(int eye);
01079
01086 bool submit(bool mirror = true);
01087 };
01088 #endif
01089 };

```


9.47 Intrinsic.cpp ファイル

```
#include "Intrinsic.h"
```

Intrinsic.cpp の依存先関係図:



9.47.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsic.cpp](#) に定義があります。

9.48 Intrinsic.cpp

[詳解]

```
00001
00008 #include "Intrinsic.h"
00009
00010 //
00011 // 構成ファイルを読み込むときに使う
00012 // キャプチャデバイス固有のパラメータの構造体のコンストラクタ
```

```

00013 //
00014 Intrinsic::Intrinsic(const picojson::object& object)
00015 {
00016     // キャプチャデバイスの画角
00017     getValue(object, "fov", fov);
00018
00019     // キャプチャデバイスの中心位置
00020     getValue(object, "center", center);
00021
00022     // キャプチャデバイスの解像度
00023     getValue(object, "size", size);
00024
00025     // キャプチャデバイスの周波数
00026     getValue(object, "fps", fps);
00027 }
00028
00029 //
00030 // スクリーンの対角線長をもとにしてキャプチャデバイスのレンズの縦横の画角を変更する
00031 //
00032 void Intrinsic::setFov(float focal)
00033 {
00034     // 撮像面の画素数の対角線長×2
00035     const auto d{ hypotf(static_cast<float>(size[0]), static_cast<float>(size[1])) * 2.0f };
00036
00037     // 撮像面の画素数の対角線長に対するスクリーンの幅と高さの比の 1/2 (d は対角線長の2倍だから)
00038     const auto w{ size[0] / d };
00039     const auto h{ size[1] / d };
00040
00041     // 焦点距離が1のときの撮像面の対角線長
00042     const auto t{ sensorSize / focal };
00043
00044     // 焦点距離が1のときの撮像面の対角線長をこの比で横と縦に振り分ける
00045     const auto u{ t * w };
00046     const auto v{ t * h };
00047
00048     // 画角を求め単位を度に換算して設定する
00049     constexpr auto s{ 360.0f / 3.14159265359f };
00050     setFov(atan(u) * s, atan(v) * s);
00051 }

```

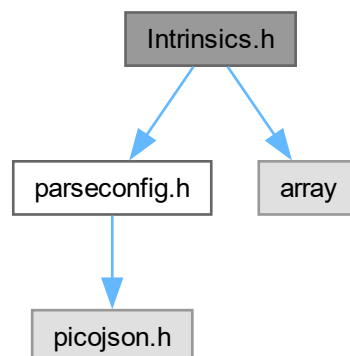
9.49 Intrinsic.h ファイル

```

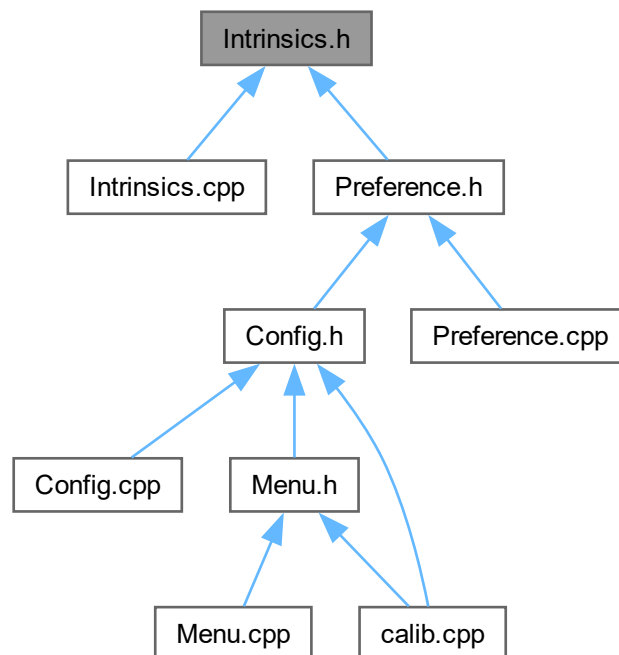
#include "parseconfig.h"
#include <array>

```

Intrinsic.h の依存先関係図:



被依存関係図:



クラス

- struct [Intrinsic](#)

9.49.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsic.h](#) に定義があります。

9.50 Intrinsics.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 構成ファイルの読み取り補助
00012 #include "parseconfig.h"
00013
00014 // 標準ライブラリ
00015 #include <array>
00016
00020 struct Intrinsics
00021 {
00023     std::array<float, 2> fov{ 50.03f, 38.58f };
00024
00026     std::array<float, 2> center{ 0.0f, 0.0f };
00027
00029     std::array<int, 2> size{ 640, 480 };
00030
00032     double fps{ 30.0 };
00033
00039     Intrinsics() = default;
00040
00050     Intrinsics(const std::array<float, 2>& fov, const std::array<float, 2>& center,
00051               const std::array<int, 2> size, double fps)
00052         : fov{ fov }
00053         , center{ center }
00054         , size{ size }
00055         , fps{ fps }
00056     {
00057     }
00058
00063     Intrinsics(const picojson::object& object);
00064
00066     static constexpr auto sensorSize{ 35.0f };
00067
00073     void setFov(float focal);
00074
00081     void setFov(float fovx, float fovy)
00082     {
00083         // キャプチャデバイスのレンズの縦横の画角を設定する
00084         fov[0] = fovx;
00085         fov[1] = fovy;
00086     }
00087
00094     void setCenter(float x, float y)
00095     {
00096         // キャプチャデバイスのレンズの中心の位置を設定する
00097         center[0] = x;
00098         center[1] = y;
00099     }
00100
00101
00108     void setSize(int width, int height)
00109     {
00110         // 装置の解像度を設定する
00111         size[0] = width;
00112         size[1] = height;
00113     }
00114
00115
00121     void setFps(double frequency)
00122     {
00123         // 装置のフレームレートを設定する
00124         fps = frequency;
00125     }
00126 };

```

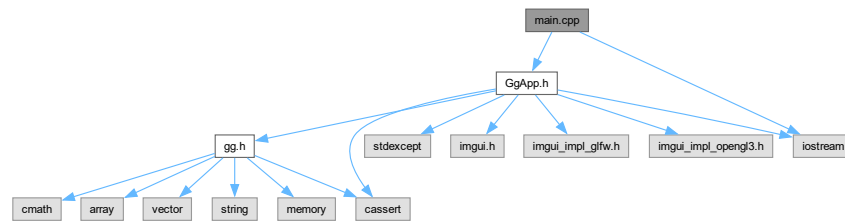
9.51 main.cpp ファイル

```

#include "GgApp.h"
#include <iostream>

```

main.cpp の依存先関係図:



マクロ定義

- `#define HEADER_STR "ゲームグラフィックス特論"`

関数

- `int main (int argc, const char *const *argv)`

9.51.1 詳解

ゲームグラフィックス特論宿題アプリケーション

著者

Kohe Tokoi

日付

February 20, 2024

`main.cpp` に定義があります。

9.51.2 マクロ定義詳解

9.51.2.1 HEADER_STR

```
#define HEADER_STR "ゲームグラフィックス特論"
```

`main.cpp` の 18 行目に定義があります。

9.51.3 関数詳解

9.51.3.1 main()

```
int main (
    int argc,
    const char *const * argv)
```

[main.cpp](#) の 23 行目に定義があります。

呼び出し関係図:



9.52 main.cpp

[詳解]

```

00001
00008 #include "GgApp.h"
00009
00010 // MessageBox の準備
00011 #if defined(_WIN32)
00012 #   include <atlstr.h>
00013 #elif defined(__APPLE__)
00014 #   include <CoreFoundation/CoreFoundation.h>
00015 #else
00016 #   include <iostream>
00017 #endif
00018 #define HEADER_STR "ゲームグラフィックス特論"
00019
00020 //
00021 // メインプログラム
00022 //
00023 int main(int argc, const char* const* argv) try
00024 {
00025     // アプリケーションのオブジェクトを生成する
00026     #if defined(GL_GLES_PROTOTYPES)
00027         GgApp app(3, 1);
00028     #else
00029         GgApp app(4, 1);
00030     #endif
00031
00032     // アプリケーションを実行する
00033     return app.main(argc, argv);
00034 }
00035 catch (const std::runtime_error &e)
00036 {
00037     // エラーメッセージを表示する
00038     #if defined(_WIN32)
00039         MessageBox(NULL, CString(e.what()), TEXT(HEADER_STR), MB_ICONERROR);
00040     #elif defined(__APPLE__)
00041         // the following code is copied from
00042         // http://blog.jorgearimany.com/2010/05/messagebox-from-windows-to-mac.html
00043         // convert the strings from char* to CFStringRef
00044         CFStringRef msg_ref = CFStringCreateWithCString(NULL, e.what(), kCFStringEncodingUTF8);
00045         // result code from the message box
00046         CFOptionFlags result;
00047
00048         // launch the message box
00049         CFUserNotificationDisplayAlert(
00050             0, // no timeout
00051             kCFUserNotificationNoteAlertLevel, // change it depending message_type flags ( MB_ICONASTERISK...
00052             NULL, // icon url, use default, you can change it depending
00053             message_type flags,
00054             NULL, // not used
00055             CFSTR(HEADER_STR), // localization of strings
00056             msg_ref, // header text
00057             NULL, // message text
00058             NULL, // default "ok" text in button
00059             NULL, // alternate button title
00060             NULL, // other button title, null--> no other button
00061             &result, // response flags
00062             );
00063         // Clean up the strings
00064         CFRelease(msg_ref);
00065     #else
00066         std::cerr << HEADER_STR << ": " << e.what() << '\n';
00067     #endif
00068
00069     // プログラムを終了する
00070     return EXIT_FAILURE;
00071 }

```

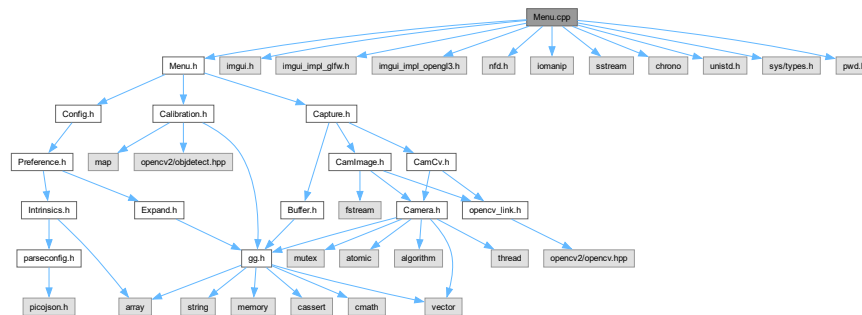
9.53 Menu.cpp ファイル

```

#include "Menu.h"
#include "imgui.h"
#include "imgui_impl_glfw.h"

```

```
#include "imgui_impl_opengl3.h"
#include "nfd.h"
#include <iomanip>
#include <sstream>
#include <chrono>
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>
Menu.cpp の依存先関係図:
```



関数

- void [getAnyList](#) (std::vector< std::string > &list)

9.53.1 詳解

メニューの描画クラスの実装

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.cpp](#) に定義があります。

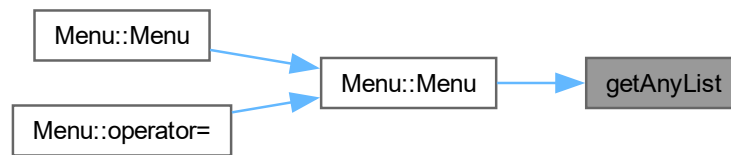
9.53.2 関数詳解

9.53.2.1 getAnyList()

```
void getAnyList (
    std::vector< std::string > & list)
```

[Menu.cpp](#) の 67 行目に定義があります。

被呼び出し関係図:



9.54 Menu.cpp

[詳解]

```

00001
00008 #include "Menu.h"
00009
00010 // ImGui
00011 #include "imgui.h"
00012 #include "imgui_impl_glfw.h"
00013 #include "imgui_impl_opengl3.h"
00014
00015 // ファイルダイアログ
00016 #include "nfd.h"
00017
00018 namespace
00019 {
00020     // JSON ファイル名のフィルタ
00021     constexpr nfdfilteritem_t jsonFilter[] { { "JSON", "json" } };
00022
00023     // 画像ファイル名のフィルタ
00024     constexpr nfdfilteritem_t imageFilter[] { "Images", "png, jpg, jpeg, jfif, bmp, dib" };
00025
00026     // 動画ファイル名のフィルタ
00027     constexpr nfdfilteritem_t movieFilter[] { "Movies", "mp4, m4v, mpg, mov, avi, ogg, mkv" };
00028 }
00029
00030 // 初期表示の画像ファイル名
00031 std::string Config::initialImage{ "initial.jpg" };
00032
00033 // 標準ライブラリ
00034 #include <iomanip>
00035 #include <sstream>
00036 #include <chrono>
00037
00038 #if !defined(_WIN32)
00039 // バックエンドのリスト
00040 const std::map<cv::VideoCaptureAPIs, const char*> Menu::backendList
00041 {
00042     { cv::CAP_ANY, "(any)" },
00043     #if defined(__APPLE__)
00044     { cv::CAP_AVFOUNDATION, "AV Foundation" },
00045     #endif
00046     { cv::CAP_FFMPEG, u8"動画ファイル履歴" }
00047 };
00048
00049 // コーデックのリスト
00050 const std::vector<const char*> Menu::codecList
00051 {
00052     "(any)",
00053     "MJPG",
00054     "H264",
00055     "BGR3",
00056     "YUY2",
00057     "I420",
00058     "NV12"
00059 };
00060
00061 // キャプチャデバイスのリスト
00062 std::map<cv::VideoCaptureAPIs, std::vector<std::string>> Menu::deviceList;
  
```

```

00063
00064 //
00065 // デフォルトのビデオデバイスの一覧を作る
00066 //
00067 void getAnyList(std::vector<std::string>& list)
00068 {
00069     list.emplace_back("(any)");
00070     list.emplace_back("Device 1");
00071     list.emplace_back("Device 2");
00072     list.emplace_back("Device 3");
00073     list.emplace_back("Device 4");
00074     list.emplace_back("Device 5");
00075     list.emplace_back("Device 6");
00076     list.emplace_back("Device 7");
00077 }
00078
00079 # if defined(_APPLE_)
00080 //
00081 // macOS のビデオデバイスの一覧を作る
00082 //
00083 void getAvFoundationList(std::vector<std::string>& list)
00084 {
00085     getAnyList(list);
00086 }
00087 # endif
00088
00089 // パスワードのエントリからホームディレクトリの場所を得るときに使う
00090 # include <unistd.h>
00091 # include <sys/types.h>
00092 # include <pwd.h>
00093
00094 #endif // !defined(_WIN32)
00095
00096 //
00097 // キャプチャデバイスを開く
00098 //
00099 bool Menu::openDevice()
00100 {
00101     #if defined(_WIN32)
00102         // 何のデバイスも接続されていなければ戻る
00103         if (deviceNumber < 0) return false;
00104
00105         // キャプチャスレッドが動いていたら止める
00106         capture.stop();
00107
00108         // 前に開いていたキャプチャデバイスを閉じる
00109         capture.close();
00110
00111         // 選択したキャプチャデバイスを開く
00112         if (capture.openDevice(deviceNumber))
00113         {
00114             if (formatNumber < 0) formatNumber = 0;
00115
00116             #if defined(_WIN32)
00117                 updateFormatDropdowns();
00118             #endif
00119
00120             // フォーマットを指定して開始できるように準備する
00121             if (capture.select(formatNumber))
00122             {
00123                 // 実解像度と焦点距離から、この入力を見やすく表示する初期画角を設定する
00124                 initializeInputIntrinsics(capture.getSize());
00125                 return true;
00126             }
00127         }
00128
00129         // 開けなかった
00130         errorMessage = u8"デバイスが開けません";
00131         return false;
00132     #else
00133         // コーデック
00134         char codec[5]{};
00135         if (codecNumber > 0) strncpy(codec, codecList[codecNumber], 5);
00136
00137         // ダイアログで指定したキャプチャデバイスが開けなかったら
00138         if (!capture.openDevice(deviceNumber,
00139             intrinsics.size, intrinsics.fps, backend, codec))
00140         {
00141             // 開けなかった
00142             errorMessage = u8"デバイスが開けません";
00143             return false;
00144         }
00145
00146         // 使うことになったコーデックの番号を調べる
00147         for (size_t i = 0; i < codecList.size(); ++i)
00148         {
00149             if (strncmp(codec, codecList[i], 4) == 0)

```

```

00150     {
00151         // コーデックが分かった
00152         codecNumber = static_cast<int>(i);
00153         return true;
00154     }
00155 }
00156
00157 // コーデックが分からない
00158 codecNumber = 0;
00159 return true;
00160 #endif
00161 }
00162
00163 //
00164 // 画像ファイルを開く
00165 //
00166 void Menu::openImage()
00167 {
00168     // ファイルダイアログから得るパス
00169     nfdchar_t* filepath;
00170
00171     // ファイルダイアログを開く
00172     if (NFD_OpenDialog(&filepath, imageFilter, 1, NULL) == NFD_OKAY)
00173     {
00174         // スレッドが動作中なら停止する
00175         capture.stop();
00176
00177         // ダイアログで指定した画像ファイルが開けたら
00178         if (capture.openImage(filepath))
00179         {
00180             // 実解像度と焦点距離から、この画像を見やすく表示する初期画角を設定する
00181             initializeInputIntrinsics(capture.getSize());
00182         }
00183         else
00184         {
00185             // 開けなかった
00186             errorMessage = u8"画像ファイルが開けません";
00187         }
00188
00189         // ファイルパスの取り出しに使ったメモリを開放する
00190         NFD_FreePath(filepath);
00191     }
00192 }
00193
00194 //
00195 // 動画ファイルを開く
00196 //
00197 void Menu::openMovie()
00198 {
00199     // ファイルダイアログから得るパス
00200     nfdchar_t* filepath;
00201
00202     // ファイルダイアログを開く
00203     if (NFD_OpenDialog(&filepath, movieFilter, 1, NULL) == NFD_OKAY)
00204     {
00205         // スレッドが動作中なら停止する
00206         capture.stop();
00207
00208         #if defined(_WIN32)
00209         // ダイアログで指定した動画ファイルが開けたら
00210         if (capture.openMovie(filepath))
00211         {
00212             // 実解像度と焦点距離から、この動画を見やすく表示する初期画角を設定する
00213             initializeInputIntrinsics(capture.getSize());
00214         }
00215         else
00216         {
00217             errorMessage = u8"動画ファイルが開けません";
00218         }
00219         #else
00220         // 入力特性をファイルに切り替えて
00221         backend = cv::CAP_FFMPEG;
00222
00223         // ファイルのリストを取り出し
00224         const auto fileListLength{ static_cast<int>(fileHistory.size()) };
00225
00226         // ファイルのリストの各ファイルについて
00227         for (deviceNumber = 0; deviceNumber < fileListLength; ++deviceNumber)
00228         {
00229             // 選択したファイルと同じものがあればそれを選択する
00230             if (fileHistory[deviceNumber] == filepath) break;
00231         }
00232
00233         // 選択したファイルがファイルのリストの中になければ
00234         if (deviceNumber == fileListLength)
00235         {
00236             // その先頭にファイルパスを挿入して

```

```

00237     fileHistory.insert(fileHistory.begin(), filepath);
00238
00239     // そのエントリを選択する
00240     deviceNumber = 0;
00241 }
00242
00243 // ダイアログで指定した動画ファイルが開いたら
00244 if (capture.openMovie(filepath, backend))
00245 {
00246     // 実解像度と焦点距離から、この動画を見やすく表示する初期画角を設定する
00247     initializeInputIntrinsics(capture.getSize());
00248 }
00249 else
00250 {
00251     // 開けなかった
00252     errorMessage = u8"動画ファイルが開けません";
00253 }
00254 #endif
00255
00256 // ファイルパスの取り出しに使ったメモリを開放する
00257 NFD_FreePath(filepath);
00258 }
00259 }
00260
00261 //
00262 // 構成ファイルを読み込む
00263 //
00264 void Menu::loadConfig()
00265 {
00266     // ファイルダイアログから得るパス
00267     nfdchar_t* filepath;
00268
00269     // ファイルダイアログを開く
00270     if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00271     {
00272         // 現在の構成を構成ファイルの内容にする
00273         if (config.load(filepath))
00274         {
00275             // 現在の設定に反映する
00276             settings = config.getSettings();
00277
00278             // 選択番号を有効範囲に収め、新しい投影方式の内部パラメータを反映する
00279             selectPreference(preferenceNumber < static_cast<int>(config.getPreferences().size())
00280                             ? preferenceNumber : 0);
00281
00282             // 校正設定も新しい構成に同期する
00283             calibration.setDictionary(settings.dictionaryName, settings.checkerSize,
00284                                     settings.checkerLength);
00285         }
00286         else
00287         {
00288             // 読み込めなかった
00289             errorMessage = u8"構成ファイルが読み込めません";
00290         }
00291
00292         // ファイルパスの取り出しに使ったメモリを開放する
00293         NFD_FreePath(filepath);
00294     }
00295
00296 //
00297 // 構成ファイルを保存する
00298 //
00299 void Menu::saveConfig() const
00300 {
00301     // ファイルダイアログから得るパス
00302     nfdchar_t* filepath;
00303
00304     // ファイルダイアログを開く
00305     if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, "*.json") == NFD_OKAY)
00306     {
00307         // 現在の設定で構成を更新する
00308         config.setSettings(settings);
00309
00310         // 現在の構成を保存する
00311         if (!config.save(filepath))
00312         {
00313             // 保存できなかった
00314             errorMessage = u8"構成ファイルが保存できません";
00315         }
00316
00317         // ファイルパスの取り出しに使ったメモリを開放する
00318         NFD_FreePath(filepath);
00319     }
00320 }
00321
00322 //

```

```

00323 // 校正ファイルを読み込む
00324 //
00325 void Menu::loadCalibration()
00326 {
00327     // ファイルダイアログから得るパス
00328     nfdchar_t* filepath;
00329
00330     // ファイルダイアログを開く
00331     if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00332     {
00333         // 現在のキャリブレーションパラメータを校正ファイルの内容にする
00334         if (!calibration.loadParameters(filepath))
00335         {
00336             // 読み込めなかった
00337             errorMessage = u8"校正ファイルが読み込めません";
00338         }
00339
00340         // ファイルパスの取り出しに使ったメモリを開放する
00341         NFD_FreePath(filepath);
00342     }
00343 }
00344
00345 //
00346 // 校正ファイルを保存する
00347 //
00348 void Menu::saveParameters() const
00349 {
00350     // キャリブレーションが完了していなければ戻る
00351     if (!calibration.finished()) return;
00352
00353     // 現在時刻の取得
00354     const auto now{ std::chrono::system_clock::to_time_t(std::chrono::system_clock::now()) };
00355     const std::tm* const localtime{ std::localtime(&now) };
00356
00357     // 時刻を文字列に変換
00358     const auto timeString{ std::put_time(localtime, "cal.%Y%m%d.%H%M%S.json") };
00359     const auto pathString{ static_cast<std::ostringstream&>(std::ostringstream() << timeString).str() };
00360
00361     // ファイルダイアログから得るパス
00362     nfdchar_t* filepath;
00363
00364     // ファイルダイアログを開く
00365     if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, pathString.c_str()) == NFD_OKAY)
00366     {
00367         // 現在のキャリブレーションパラメータを構成ファイルに保存する
00368         if (!calibration.saveParameters(filepath))
00369         {
00370             // 保存できなかった
00371             errorMessage = u8"校正ファイルが保存できません";
00372         }
00373
00374         // ファイルパスの取り出しに使ったメモリを開放する
00375         NFD_FreePath(filepath);
00376     }
00377 }
00378
00379 //
00380 // 校正用の画像ファイルを取得する (複数選択)
00381 //
00382 void Menu::recordFileCorners() const
00383 {
00384     // ファイルダイアログから得るパス
00385     const nfdpathset_t* outPaths;
00386
00387     // ファイルダイアログを開く
00388     if (NFD_OpenDialogMultiple(&outPaths, imageFilter, 1, NULL) == NFD_OKAY)
00389     {
00390         // ファイルパスの一覧を得る
00391         nfdpathsetenum_t enumerator;
00392         NFD_PathSet_GetEnum(outPaths, &enumerator);
00393
00394         // ファイルパスの一覧からファイルパスを一つずつ取り出して
00395         for (nfdchar_t* path = NULL; NFD_PathSet_EnumNext(&enumerator, &path) && path;)
00396         {
00397             // 画像の読み出し
00398             CamImage image;
00399
00400             // 画像ファイルが読み出せたら
00401             if (image.open(path))
00402             {
00403                 // データのコピー先
00404                 cv::Mat frame;
00405
00406                 // データをコピーして
00407                 image.transmit(frame);
00408             }
00409         }
00410     }
00411 }

```

```

00409         // ボードを検出して
00410         calibration.detectBoard(frame);
00411
00412         // コーナーを記録する
00413         calibration.recordCorners();
00414
00415         // 画像の読み出しを終わる
00416         image.close();
00417     }
00418
00419     // ファイルパスの取り出しに使ったメモリを開放する
00420     NFD_PathSet.FreePath(path);
00421 }
00422
00423 // ファイルパスの一覧に使ったメモリを開放する
00424 NFD_PathSet.FreeEnum(&enumerator);
00425
00426 // フォルダのパスに使ったメモリを開放する
00427 NFD_PathSet.Free(outPaths);
00428 }
00429 }
00430
00431 //
00432 // 転正用の ChArUco Board を作成する
00433 //
00434 void Menu::createCharuco() const
00435 {
00436     // ChArUco Board の画像を作成する
00437     cv::Mat boardImage;
00438     calibration.drawBoard(boardImage, 980, 692);
00439
00440     // ファイルに保存する
00441     saveImage(boardImage, "ChArUcoBoard.png");
00442 }
00443
00444 //
00445 // コンストラクタ
00446 //
00447 Menu::Menu(Config& config, Capture& capture, Calibration& calibration)
00448 : config{ config }
00449 , settings{ config.getSettings() }
00450 , intrinsics{ config.getPreferences().front().getIntrinsics() }
00451 , capture{ capture }
00452 , calibration{ calibration }
00453 {
00454     // ファイルダイアログ (Native File Dialog Extended) を初期化する
00455     NFD.Init();
00456
00457     // Dear ImGui の入力デバイス
00458     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard; // キーボードコントロールを使う
00459     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad; // ゲームパッドを使う
00460
00461     // Dear ImGui のスタイル
00462     //ImGui::StyleColorsDark(); // 暗めのスタイル
00463     //ImGui::StyleColorsClassic(); // 以前のスタイル
00464
00465     // 日本語を表示できるメニューフォントを読み込む
00466     if (!ImGui::GetIO().Fonts->AddFontFromFileTTF(config.getMenuFont().c_str(),
00467         config.getMenuFontSize(),
00468         nullptr, ImGui::GetIO().Fonts->GetGlyphRangesJapanese()))
00469     {
00470         // メニューフォントが読み込めなかったらエラーにする
00471         throw std::runtime_error("Cannot find any menu fonts.");
00472     }
00473
00474     #if defined(_WIN32)
00475     // 初期状態で最初のデバイスのフォーマットリストを取得しておく
00476     if (!config.getDeviceList().empty())
00477     {
00478         capture.updateFormatList(deviceNumber);
00479     }
00480     #else
00481     // バックエンドごとのキャプチャデバイスの一覧を初期化する
00482     for (auto& [api, name] : backendList)
00483     {
00484         // バックエンドごとに空のリストを追加する
00485         deviceList.emplace(api, std::vector<std::string>());
00486     }
00487
00488     // キャプチャデバイスの一覧を作る
00489     getAnyList(deviceList.at(cv::CAP_ANY));
00490     #if defined(_MSC_VER)
00491     getDirectShowList(deviceList.at(cv::CAP_DSHOW));
00492     getMediaFoundationList(deviceList.at(cv::CAP_MSMF));
00493     #elif defined(__APPLE__)
00494     getAvFoundationList(deviceList.at(cv::CAP_AVFOUNDATION));
00495     #endif

```

```

00495 #endif
00496 }
00497
00498 //
00499 // デストラクタ
00500 //
00501 Menu::~Menu()
00502 {
00503     // キャプチャスレッドが動いていたら止める
00504     capture.stop();
00505
00506     // 前に開いていたキャプチャデバイスを閉じる
00507     capture.close();
00508
00509     // ファイルダイアログ (Native File Dialog Extended) を終了する
00510     NFD.Quit();
00511 }
00512
00513 //
00514 // 入力画像に合わせて内部パラメータを初期化する
00515 //
00516 void Menu::initializeInputIntrinsics(const std::array<int, 2>& size)
00517 {
00518     // 実際の入力解像度を処理系へ反映する
00519     intrinsics.size = size;
00520
00521     // 無効な入力によるゼロ除算を避け、有効な場合だけ焦点距離から初期画角を求める
00522     if (size[0] > 0 && size[1] > 0 && settings.focal > 0.0f)
00523     {
00524         intrinsics.setFov(settings.focal);
00525     }
00526 }
00527
00528 //
00529 // 選択中の入力設定を適用してキャプチャを開始する
00530 //
00531 bool Menu::startCapture()
00532 {
00533     // オープンとフォーマット適用を一つの入口に集約し、失敗時は開始処理を中断する
00534     if (!openDevice()) return false;
00535
00536     // デバイスが確定してから動作モードと入力に合う初期画角を反映し、取得スレッドを開始する
00537     capture.setPrioritizeLatency(prioritizeLatency);
00538     initializeInputIntrinsics(capture.getSize());
00539     capture.start();
00540     return true;
00541 }
00542
00543 //
00544 // 選択中の投影方式とその内部パラメータを同期する
00545 //
00546 void Menu::selectPreference(int index)
00547 {
00548     // 不正な選択番号では現在の投影状態を変更しない
00549     if (index < 0 || index >= static_cast<int>(config.getPreferences().size())) return;
00550
00551     // 入力中の実解像度を、投影方式の構成値で上書きしないため回避する
00552     const auto size{ intrinsics.size };
00553     preferenceNumber = index;
00554     intrinsics = getPreference().getIntrinsics();
00555
00556     // 入力中は実解像度だけを戻し、画角と中心位置は選択した投影方式の設定値を使用する
00557     if (capture.isOpened()) intrinsics.size = size;
00558 }
00559
00560 #if defined(_WIN32)
00561 //
00562 // 解像度、フレームレート、コーデックの選択リストを更新する
00563 //
00564 void Menu::updateFormatDropdowns()
00565 {
00566     const auto& formatList{ capture.getFormatList() };
00567
00568     // デバイス列挙結果を UI 側にコピーし、各ドロップダウンの候補を作り直す
00569     availableFormats = formatList;
00570     uniqueResolutions.clear();
00571     uniqueFpsList.clear();
00572     uniqueCodecs.clear();
00573
00574     // 構造化されたフォーマット情報から選択肢を作成する
00575     for (const auto& info : availableFormats)
00576     {
00577         if (std::find(uniqueResolutions.begin(), uniqueResolutions.end(), info.resolution) ==
            uniqueResolutions.end())
00578             uniqueResolutions.push_back(info.resolution);
00579         if (std::find(uniqueFpsList.begin(), uniqueFpsList.end(), info.fps) == uniqueFpsList.end())
00580             uniqueFpsList.push_back(info.fps);
00581     }
00582 }

```

```

00581     if (std::find(uniqueCodecs.begin(), uniqueCodecs.end(), info.codec) == uniqueCodecs.end())
00582         uniqueCodecs.push_back(info.codec);
00583     }
00584     // 現在の formatNumber のフォーマットに同期する
00585     const auto selected{ std::find_if(availableFormats.begin(), availableFormats.end(),
00586 [this](const CaptureFormat& info) { return info.index == formatNumber; }) };
00587     if (selected != availableFormats.end())
00588     {
00589         currentRes = selected->resolution;
00590         currentFps = selected->fps;
00591         currentCodec = selected->codec;
00592     }
00593     else
00594     {
00595         currentRes.clear();
00596         currentFps.clear();
00597         currentCodec.clear();
00598     }
00599     lastDeviceNumber = deviceNumber;
00600 }
00601 #endif
00602 //
00603 // シェーダを設定する
00604 //
00605 // シェーダを設定する
00606 //
00607 std::array<GLsizei, 2> Menu::setup(GLfloat aspect) const
00608 {
00609     // シェーダを設定する
00610     return config.getPreferences()[preferenceNumber].getShader().setup(settings.samples, aspect,
00611 pose, intrinsics.fov, intrinsics.center, settings.getFocal(), config.getBackground());
00612 }
00613 //
00614 // メインメニューバーの描画
00615 //
00616 //
00617 void Menu::drawMainMenuBar()
00618 {
00619     if (ImGui::BeginMainMenuBar())
00620     {
00621         // ファイルメニュー
00622         if (ImGui::BeginMenu(u8"ファイル"))
00623         {
00624             // 画像ファイルを開く
00625             if (ImGui::MenuItem(u8"画像ファイルを開く")) openImage();
00626
00627             // 動画ファイルを開く
00628             if (ImGui::MenuItem(u8"動画ファイルを開く")) openMovie();
00629
00630             // 構成ファイルを開く
00631             if (ImGui::MenuItem(u8"構成ファイルを開く")) loadConfig();
00632
00633             // 構成ファイルを保存する
00634             if (ImGui::MenuItem(u8"構成ファイルを保存")) saveConfig();
00635
00636             // キャリブレーションパラメータファイルを開く
00637             if (ImGui::MenuItem(u8"校正ファイルを開く")) loadCalibration();
00638
00639             // キャリブレーションパラメータファイルを保存する
00640             if (ImGui::MenuItem(u8"校正ファイルを保存")) saveParameters();
00641
00642             // フォルダ内の画像ファイルを使って校正する
00643             if (ImGui::MenuItem(u8"校正用画像から取得")) recordFileCorners();
00644
00645             // ChArUco Board の作成
00646             if (ImGui::MenuItem(u8"ChArUco 画像作成")) createCharuco();
00647
00648             // 終了
00649             quit = ImGui::MenuItem(u8"終了");
00650
00651             // File メニュー修了
00652             ImGui::EndMenu();
00653         }
00654
00655         // ウィンドウメニュー
00656         if (ImGui::BeginMenu(u8"ウィンドウ"))
00657         {
00658             // 入力パネルの表示
00659             ImGui::MenuItem(u8"入力", NULL, &showInputPanel);
00660
00661             // 校正パネルの表示
00662             ImGui::MenuItem(u8"校正", NULL, &showCalibrationPanel);
00663
00664             // File メニュー修了
00665             ImGui::EndMenu();
00666         }
00667     }

```



```

00668 // メニューバーの高さを保存しておく
00669 menubarHeight = static_cast<GLsizei>(ImGui::GetWindowHeight());
00670
00671 // メインメニューバー終了
00672 ImGui::EndMainMenuBar();
00673 }
00674 }
00675
00676 //
00677 // 入力パネルの描画
00678 //
00679 void Menu::drawInputPanel()
00680 {
00681 // 入力パネル
00682 if (showInputPanel)
00683 {
00684 // ウィンドウの位置とサイズ
00685 ImGui::SetNextWindowPos(ImVec2(2.0f, 2.0f + menubarHeight), ImGuiCond_Once);
00686 ImGui::SetNextWindowSize(ImVec2(231, 517), ImGuiCond_Once);
00687 ImGui::Begin(u8"入力", &showInputPanel);
00688
00689 // 投影方式の選択
00690 if (ImGui::BeginCombo(u8"投影方式", getPreference().getDescription().c_str()))
00691 {
00692 // すべての投影方式について
00693 for (int i = 0; i < static_cast<int>(config.getPreferences().size()); ++i)
00694 {
00695 // その投影方式が選択されていれば真
00696 const bool selected{ i == preferenceNumber };
00697
00698 // 投影方式を（それが現在の投影方式ならハイライトして）コンボボックスに表示する
00699 if (ImGui::Selectable(getPreference(i).getDescription().c_str(), selected))
00700 {
00701 // 表示した投影方式が選択されていたらそれを現在の選択とする
00702 selectPreference(i);
00703 }
00704
00705 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00706 if (selected) ImGui::SetItemDefaultFocus();
00707 }
00708 ImGui::EndCombo();
00709 }
00710
00711 // レンズの画角を設定する
00712 ImGui::DragFloat2(u8"画角", intrinsics.fov.data(), 0.1f, -360.0f, 360.0f, "%.2f");
00713
00714 // レンズの中心位置
00715 ImGui::DragFloat2(u8"中心", intrinsics.center.data(), 0.001f, -1.0f, 1.0f, "%.4f");
00716
00717 // キャプチャデバイス固有のパラメータを元に戻す
00718 if (ImGui::Button(u8"回復"))
00719 {
00720 // 選択した投影方式のキャプチャデバイス固有のパラメータを回復する
00721 intrinsics = getPreference().getIntrinsics();
00722 }
00723
00724 // 姿勢
00725 ImGui::SliderAngle(u8"方位", &settings.euler[1], -180.0f, 180.0f, "%.2f");
00726 ImGui::SliderAngle(u8"仰角", &settings.euler[0], -180.0f, 180.0f, "%.2f");
00727 ImGui::SliderAngle(u8"傾斜", &settings.euler[2], -180.0f, 180.0f, "%.2f");
00728 pose = ggRotateY(settings.euler[1]).rotateX(settings.euler[0]).rotateZ(settings.euler[2]);
00729
00730 // 焦点距離
00731 ImGui::SliderFloat(u8"焦点距離", &settings.focal, settings.focalRange[0], settings.focalRange[1],
00732 "%.1f");
00733
00734 // 姿勢を元に戻す
00735 if (ImGui::Button(u8"復帰"))
00736 {
00737 settings.euler = config.getSettings().euler;
00738 settings.focal = config.getSettings().focal;
00739 settings.focalRange = config.getSettings().focalRange;
00740 }
00741
00742 ImGui::Separator();
00743
00744 #if defined(_WIN32)
00745 // キャプチャデバイスが存在するとき
00746 if (!config.getDeviceList().empty())
00747 {
00748 // 装置関連項目
00749 ImGui::Text("%s", u8"以下の変更は [開始] で反映します");
00750
00751 // キャプチャデバイスの選択コンボボックス
00752 if (ImGui::BeginCombo(u8"装置", config.getDeviceName(deviceNumber).c_str()))
00753 {
00754 // すべてのキャプチャデバイスについて

```

```

00754     for (int i = 0; i < static_cast<int>(config.getDeviceList().size()); ++i)
00755     {
00756         // キャプチャデバイス名を (それを選択していればハイライトして) コンボボックスに表示する
00757         if (ImGui::Selectable(config.getDeviceName(i).c_str(), i == deviceNumber))
00758         {
00759             // キャプチャデバイスが変わったら
00760             if (deviceNumber != i)
00761             {
00762                 // キャプチャスレッドが動いていたなら止める
00763                 capture.stop();
00764
00765                 // 前に開いていたキャプチャデバイスを閉じる
00766                 capture.close();
00767
00768                 // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00769                 deviceNumber = i;
00770
00771                 // キャプチャデバイスが変わったので最初のビデオフォーマットを選択する
00772                 formatNumber = 0;
00773
00774                 // 開始ボタンが押されるまでは、フォーマットリストだけを一時取得して更新する
00775                 capture.updateFormatList(deviceNumber);
00776
00777                 // 選択可能なドロップダウンのリストを更新する
00778                 updateFormatDropdowns();
00779             }
00780
00781             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00782             ImGui::SetItemDefaultFocus();
00783         }
00784     }
00785     ImGui::EndCombo();
00786 }
00787
00788 // 使用可能なビデオフォーマットの表示名のリスト
00789 const auto& formatList{ capture.getFormatList() };
00790
00791 // 使用可能なビデオフォーマットが存在するなら
00792 if (!formatList.empty())
00793 {
00794     // 必要な場合 (デバイス番号の不一致やリスト未作成時) にリストを更新する
00795     if (lastDeviceNumber != deviceNumber || availableFormats.empty())
00796     {
00797         updateFormatDropdowns();
00798     }
00799
00800     // 1. 解像度の選択コンボボックス
00801     if (ImGui::BeginCombo(u8"解像度", currentRes.c_str()))
00802     {
00803         for (const auto& res : uniqueResolutions)
00804         {
00805             if (ImGui::Selectable(res.c_str(), currentRes == res))
00806             {
00807                 currentRes = res;
00808             }
00809         }
00810         ImGui::EndCombo();
00811     }
00812
00813     // 2. フレームレートの選択コンボボックス
00814     std::string fpsLabel = currentFps + " fps";
00815     if (ImGui::BeginCombo(u8"コマ数", fpsLabel.c_str()))
00816     {
00817         for (const auto& fpsVal : uniqueFpsList)
00818         {
00819             std::string valLabel = fpsVal + " fps";
00820             if (ImGui::Selectable(valLabel.c_str(), currentFps == fpsVal))
00821             {
00822                 currentFps = fpsVal;
00823             }
00824         }
00825         ImGui::EndCombo();
00826     }
00827
00828     // 3. コーデックの選択コンボボックス
00829     if (ImGui::BeginCombo(u8"符号化", currentCodec.c_str()))
00830     {
00831         for (const auto& cod : uniqueCodecs)
00832         {
00833             if (ImGui::Selectable(cod.c_str(), currentCodec == cod))
00834             {
00835                 currentCodec = cod;
00836             }
00837         }
00838         ImGui::EndCombo();
00839     }
00840

```

```

00841         // 選択された組み合わせが availableFormats に存在するか探す
00842         int foundIndex = -1;
00843         for (const auto& info : availableFormats)
00844         {
00845             if (info.resolution == currentRes && info.fps == currentFps && info.codec == currentCodec)
00846             {
00847                 foundIndex = info.index;
00848                 break;
00849             }
00850         }
00851
00852         bool formatExists = (foundIndex != -1);
00853         if (formatExists)
00854         {
00855             // 存在する場合は、必要なら formatNumber を更新する
00856             if (formatNumber != foundIndex)
00857             {
00858                 capture.stop();
00859                 formatNumber = foundIndex;
00860             }
00861         }
00862
00863         // 4. レイテンシ優先のチェックボックス
00864         if (ImGui::Checkbox(u8"レイテンシ優先", &prioritizeLatency))
00865         {
00866             if (capture) capture.setPrioritizeLatency(prioritizeLatency);
00867         }
00868
00869         // キャプチャの開始と停止
00870         if (capture)
00871         {
00872             // キャプチャスレッドが動いているので止める
00873             if (ImGui::Button(u8"停止")) capture.stop();
00874             ImGui::SameLine();
00875             ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
00876         }
00877         else
00878         {
00879             if (formatExists)
00880             {
00881                 // 「開始」ボタンをクリックしたときデバイスが選択されていれば
00882                 if (ImGui::Button(u8"開始") && deviceNumber >= 0)
00883                 {
00884                     startCapture();
00885                 }
00886                 ImGui::SameLine();
00887                 ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
00888             }
00889             else
00890             {
00891                 // 存在しない組み合わせの時はメッセージを表示する
00892                 ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"フォーマットが存在しません");
00893             }
00894         }
00895     }
00896     else
00897     {
00898         // キャプチャデバイスが開けなかった
00899         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが開けません");
00900     }
00901 }
00902 else
00903 {
00904     // キャプチャデバイスが存在しないとき
00905     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
00906 }
00907 #else
00908 // 装置関連項目
00909 ImGui::Text("%s", u8"以下の変更は「開始」で反映します");
00910
00911 // デバイスプリファレンスを選択する
00912 if (ImGui::BeginCombo(u8"装置特性", backendList.at(backend)))
00913 {
00914     // すべての表示方式について
00915     for (auto& [apiId, apiName] : backendList)
00916     {
00917         // その表示方式が選択されていれば真
00918         const bool selected{ apiId == backend };
00919
00920         // 装置特性を（それが現在の装置特性ならハイライトして）コンボボックスに表示する
00921         if (ImGui::Selectable(apiName, selected))
00922         {
00923             // 表示した装置特性が選択されていたらそれを現在の選択とする
00924             backend = apiId;
00925
00926             // 切り替え前の装置特性のデバイスが存在しなければ最初のデバイスの番号を選択する
00927             if (deviceNumber < 0) deviceNumber = 0;

```

```

00928
00929 // 選択されているデバイスの番号が接続されたキャプチャデバイスの数を超えないようにする
00930 const int count{ getDeviceCount(backend) };
00931 if (deviceNumber >= count) deviceNumber = count - 1;
00932 }
00933
00934 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00935 if (selected) ImGui::SetItemDefaultFocus();
00936 }
00937 ImGui::EndCombo();
00938 }
00939
00940 // キャプチャデバイスが存在すれば
00941 if (deviceNumber >= 0)
00942 {
00943     // キャプチャデバイスの選択コンボボックス
00944     if (ImGui::BeginCombo(u8"入力源", getDeviceName(backend, deviceNumber).c_str()))
00945     {
00946         // すべてのキャプチャデバイスについて
00947         for (int i = 0; i < static_cast<int>(getDeviceList(backend).size()); ++i)
00948         {
00949             // キャプチャデバイス名を (それを選択していればハイライトして) コンボボックスに表示する
00950             if (ImGui::Selectable(getDeviceName(backend, i).c_str(), i == deviceNumber))
00951             {
00952                 // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00953                 deviceNumber = i;
00954
00955                 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00956                 ImGui::SetItemDefaultFocus();
00957             }
00958         }
00959         ImGui::EndCombo();
00960     }
00961 }
00962 else
00963 {
00964     // 使えるキャプチャデバイスがない
00965     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
00966 }
00967
00968 // キャプチャするサイズとフレームレート
00969 ImGui::InputInt2(u8"解像度", intrinsic.size.data());
00970 ImGui::InputDouble(u8"周波数", &intrinsic.fps, 1.0f, 1.0f, "%.1f");
00971
00972 // コーデックを選択する
00973 if (ImGui::BeginCombo(u8"符号化", codecList[codecNumber]))
00974 {
00975     // すべてのコーデックについて
00976     for (int i = 0; i < static_cast<int>(codecList.size()); ++i)
00977     {
00978         // コーデックを (それを選択していればハイライトして) コンボボックスに表示する
00979         if (ImGui::Selectable(codecList[i], i == codecNumber))
00980         {
00981             // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00982             codecNumber = i;
00983
00984             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00985             ImGui::SetItemDefaultFocus();
00986         }
00987     }
00988     ImGui::EndCombo();
00989 }
00990
00991 // キャプチャの開始と停止
00992 if (capture)
00993 {
00994     // キャプチャスレッドが働いているので止める
00995     if (ImGui::Button(u8"停止")) capture.stop();
00996     ImGui::SameLine();
00997     ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
00998 }
00999 else
01000 {
01001     // キャプチャスレッドが止まっているので
01002     if (ImGui::Button(u8"開始") && deviceNumber >= 0)
01003     {
01004         startCapture();
01005     }
01006     ImGui::SameLine();
01007     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
01008 }
01009 #endif
01010 ImGui::End();
01011 }
01012 }
01013
01014 //

```

```

01015 // 校正パネルの描画
01016 //
01017 void Menu::drawCalibrationPanel()
01018 {
01019     if (showCalibrationPanel)
01020     {
01021         // ウィンドウの位置とサイズ
01022         ImGui::SetNextWindowPos(ImVec2(235.0f, 2.0f + menubarHeight), ImGuiCond_Once);
01023         ImGui::SetNextWindowSize(ImVec2(218, 359), ImGuiCond_Once);
01024         ImGui::Begin(u8"校正", &showCalibrationPanel);
01025
01026         // 辞書の選択
01027         if (ImGui::BeginCombo(u8"辞書", settings.dictionaryName.c_str()))
01028         {
01029             // すべての辞書について
01030             for (auto d = calibration.dictionaryList.begin(); d != calibration.dictionaryList.end(); ++d)
01031             {
01032                 // その設定が現在選択されている設定なら真
01033                 const bool selected(d->first == settings.dictionaryName);
01034
01035                 // 設定を（それが現在の設定ならハイライトして）コンボボックスに表示する
01036                 if (ImGui::Selectable(d->first.c_str(), d->first == settings.dictionaryName))
01037                 {
01038                     // 表示した設定が選択されていたらそれを現在の選択とする
01039                     settings.dictionaryName = d->first;
01040
01041                     // 選択した ArUco Marker の辞書を設定する
01042                     calibration.setDictionary(settings.dictionaryName, settings.checkerSize,
01043                                             settings.checkerLength);
01044                 }
01045
01046                 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01047                 if (selected) ImGui::SetItemDefaultFocus();
01048             }
01049             ImGui::EndCombo();
01050         }
01051         ImGui::Separator();
01052
01053         // ArUco Marker の検出
01054         if (ImGui::Checkbox(u8"ArUco Marker 検出", &detectMarker) && detectMarker) detectBoard = false;
01055
01056         // ArUco Marker の大きさ
01057         ImGui::InputFloat(u8"マーカー長", &settings.markerLength, 0.0f, 0.0f, "%.2f cm");
01058
01059         ImGui::Separator();
01060
01061         // 校正
01062         if (ImGui::Checkbox(u8"ChArUco Board 検出", &detectBoard) && detectBoard) detectMarker = false;
01063
01064         // ChArUco Board のマス目の数（横，縦）
01065         if (ImGui::InputInt2(u8"升目数", settings.checkerSize.data()))
01066         {
01067             // 升目の数は 2 以上とする
01068             settings.checkerSize[0] = std::max(2, settings.checkerSize[0]);
01069             settings.checkerSize[1] = std::max(2, settings.checkerSize[1]);
01070
01071             // ChArUco Board を作り直す
01072             calibration.createBoard(settings.checkerSize, settings.checkerLength);
01073         }
01074
01075         // ChArUco Board の大きさ
01076         if (ImGui::InputFloat2(u8"升目長", settings.checkerLength.data(), "%.2f cm"))
01077         {
01078             // ChArUco Board を作り直す
01079             calibration.createBoard(settings.checkerSize, settings.checkerLength);
01080         }
01081
01082         // 「取得」ボタンをクリックしたとき ChArUco Board の検出中なら
01083         if (ImGui::Button(u8"取得") && detectBoard)
01084         {
01085             // 検出したコーナーを記録する
01086             calibration.recordCorners();
01087         }
01088
01089         // 1 つでも標本を取得していれば
01090         if (calibration.getSampleCount() > 0)
01091         {
01092             // 標本の「消去」ボタンを表示する
01093             ImGui::SameLine();
01094             if (ImGui::Button(u8"消去")) calibration.discardCorners();
01095
01096             // 標本を 6 つ以上取得していれば
01097             if (calibration.getSampleCount() >= 6)
01098             {
01099                 // 「校正」ボタンを表示する
01100                 ImGui::SameLine();

```

```

01101         if (ImGui::Button(u8"校正") && !calibration.calibrate())
01102         {
01103             // 校正失敗
01104             errorMessage = u8"校正に失敗しました";
01105         }
01106
01107         // 校正が完了していれば
01108         if (calibration.finished())
01109         {
01110             // 「完了」を表示する
01111             ImGui::SameLine();
01112             ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"完了");
01113         }
01114     }
01115 }
01116
01117 ImGui::Separator();
01118
01119 // フレームレートの表示
01120 ImGui::Text(u8"フレームレート: %6.2f fps", ImGui::GetIO().Framerate);
01121
01122 // 検出数の表示
01123 ImGui::Text(u8"コーナー検出数: %d", calibration.getCornersCount());
01124
01125 // 標本数の表示
01126 ImGui::Text(u8"サンプル取得数: %d (%d)", calibration.getSampleCount(), calibration.getTotalCount());
01127
01128 // 再投影誤差の表示
01129 ImGui::Text(u8"再投影誤差: %.4f", calibration.getReprojectionError());
01130
01131 ImGui::End();
01132 }
01133 }
01134
01135 //
01136 // エラーダイアログの描画
01137 //
01138 void Menu::drawErrorDialog()
01139 {
01140     // エラーメッセージが設定されていたら
01141     if (errorMessage)
01142     {
01143         // ウィンドウの位置・サイズとタイトル
01144         ImGui::SetNextWindowPos(ImVec2(60, 60), ImGuiCond_Once);
01145         ImGui::SetNextWindowSize(ImVec2(240, 92), ImGuiCond_Always);
01146
01147         // ウィンドウを表示するとき true
01148         bool status{ true };
01149
01150         // エラーメッセージウィンドウを表示する
01151         ImGui::Begin(u8"エラー", &status);
01152
01153         // エラーメッセージの表示
01154         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", errorMessage);
01155
01156         // クローズボックスか「閉じる」ボタンをクリックしたら
01157         if (!status || ImGui::Button(u8"閉じる"))
01158         {
01159             // エラーメッセージを消去する
01160             errorMessage = nullptr;
01161         }
01162         ImGui::End();
01163     }
01164 }
01165
01166 //
01167 // メニューの描画
01168 //
01169 void Menu::draw()
01170 {
01171     // 各ウィンドウの描画責務を分離し、この関数では一フレーム分の呼び出し順だけを管理する
01172     drawMainMenuBar();
01173     drawInputPanel();
01174     drawCalibrationPanel();
01175     drawErrorDialog();
01176
01177     // ChArUco Board の検出中にスペースバーをタイプしたなら
01178     if (detectBoard && ImGui::IsKeyPressed(ImGuiKey_Space))
01179     {
01180         // 検出したコーナーを記録する
01181         calibration.recordCorners();
01182     }
01183 }
01184
01185 //
01186 // 画像の保存
01187 //

```

```

01188 void Menu::saveImage(const cv::Mat& image, const std::string& filename) const
01189 {
01190     // 画像ファイル名のフィルタ
01191     constexpr nfdfilteritem_t imageFilter[]{ "Images", "png,jpg,jpeg,jfif,bmp,dib" };
01192
01193     // ファイルダイアログから得るパス
01194     nfdchar_t* filepath;
01195
01196     // ファイルダイアログを開く
01197     if (NFD_SaveDialog(&filepath, imageFilter, 1, NULL, filename.c_str()) == NFD_OKAY)
01198     {
01199         // ファイルに保存する
01200         if (!CamImage::save(filepath, image)) errorMessage = u8"ファイルが保存できませんでした";
01201
01202         // ファイルパスの取り出しに使ったメモリを開放する
01203         NFD_FreePath(filepath);
01204     }
01205 }

```

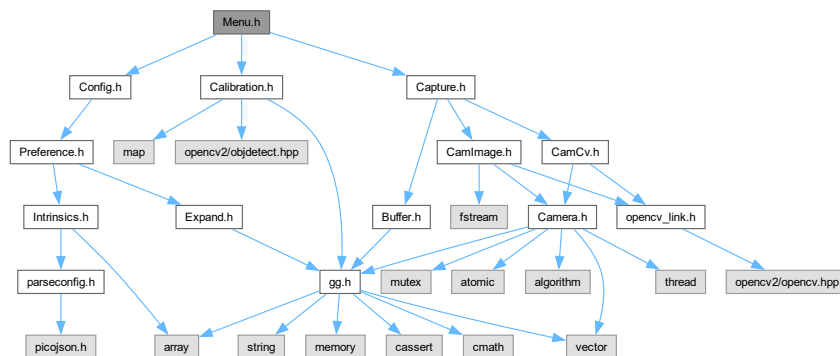
9.55 Menu.h ファイル

```

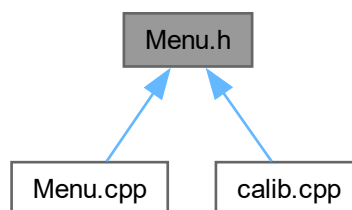
#include "Config.h"
#include "Capture.h"
#include "Calibration.h"

```

Menu.h の依存先関係図:



被依存関係図:



クラス

- class [Menu](#)

9.55.1 詳解

メニューの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.h](#) に定義があります。

9.56 Menu.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 構成データ
00012 #include "Config.h"
00013
00014 // キャプチャデバイス
00015 #include "Capture.h"
00016
00017 // 校正オブジェクト
00018 #include "Calibration.h"
00019
00023 class Menu
00024 {
00026     Config& config;
00027
00029     Settings settings;
00030
00031 #if !defined(_WIN32)
00033     static const std::map<cv::VideoCaptureAPIs, const char*> backendList;
00034
00036     static const std::vector<const char*> codecList;
00037
00039     static std::map <cv::VideoCaptureAPIs, std::vector<std::string>> deviceList;
00040
00042     std::vector<std::string> fileHistory;
00043
00050     const auto& getDeviceList (cv::VideoCaptureAPIs api) const
00051     {
00052         return deviceList.at(api);
00053     }
00054
00061     auto getDeviceCount (cv::VideoCaptureAPIs api) const
00062     {
00063         return static_cast<int>(deviceList.at(api).size());
00064     }
00065
00073     const auto& getDeviceName (cv::VideoCaptureAPIs api, int number) const
00074     {
00075         static const std::string empty{};
00076         const auto& list{ deviceList.at(api) };
00077         return list.empty() ? empty : list[number];
00078     }
00079 #endif
00080
00082     Intrinsics intrinsics;
00083
00085     Capture& capture;
00086
00088     Calibration& calibration;
00089
00091     int deviceNumber{ 0 };
00092
00093 #if defined(_WIN32)
```



```

00095     int formatNumber{ 0 };
00096
00098     std::vector<CaptureFormat> availableFormats;
00099
00101     std::vector<std::string> uniqueResolutions;
00102
00104     std::vector<std::string> uniqueFpsList;
00105
00107     std::vector<std::string> uniqueCodecs;
00108
00110     std::string currentRes;
00111
00113     std::string currentFps;
00114
00116     std::string currentCodec;
00117
00119     int lastDeviceNumber{ -1 };
00120
00124     void updateFormatDropdowns();
00125 #else
00127     int codecNumber{ 0 };
00128
00130     cv::VideoCaptureAPIs backend{ cv::CAP_ANY };
00131 #endif
00132
00134     int preferenceNumber{ 0 };
00135
00137     GgMatrix pose{ ggIdentity() };
00138
00140     GLsizei menubarHeight{ 0 };
00141
00143     bool showInputPanel{ true };
00144
00146     bool showCalibrationPanel{ true };
00147
00149     bool quit{ false };
00150
00152     mutable const char* errorMessage{ nullptr };
00153
00158     bool openDevice();
00159
00165     bool startCapture();
00166
00170     void openImage();
00171
00175     void openMovie();
00176
00180     void loadConfig();
00181
00185     void saveConfig() const;
00186
00190     void loadCalibration();
00191
00195     void saveParameters() const;
00196
00200     void recordFileCorners() const;
00201
00205     void createCharuco() const;
00206
00215     void selectPreference(int index);
00216
00223     const auto& getPreference(int i) const
00224     {
00225         return config.getPreferences()[i];
00226     }
00227
00233     const auto& getPreference() const
00234     {
00235         return getPreference(preferenceNumber);
00236     }
00237
00241     void drawMainMenuBar();
00242
00246     void drawInputPanel();
00247
00251     void drawCalibrationPanel();
00252
00256     void drawErrorDialog();
00257
00258 public:
00259
00261     bool prioritizeLatency{ false };
00262
00264     bool detectMarker{ false };
00265
00267     bool detectBoard{ false };
00268

```

```

00276     Menu(Config& config, Capture& capture, Calibration& calibration);
00277
00283     Menu(const Menu& menu) = delete;
00284
00288     virtual ~Menu();
00289
00296     Menu& operator=(const Menu& menu) = delete;
00297
00303     explicit operator bool() const
00304     {
00305         return !quit;
00306     }
00307
00313     const auto& getPose() const
00314     {
00315         return pose;
00316     }
00317
00323     auto getMenuBarHeight() const
00324     {
00325         return menubarHeight;
00326     }
00327
00336     void initializeInputIntrinsics(const std::array<int, 2>& size);
00337
00343     const auto& getCheckerSize() const
00344     {
00345         return settings.checkerSize;
00346     }
00347
00353     const auto& getCheckerLength() const
00354     {
00355         return settings.checkerLength;
00356     }
00357
00363     auto getMarkerLength() const
00364     {
00365         return settings.markerLength;
00366     }
00367
00377     std::array<GLsizei, 2> setup(GLfloat aspect) const;
00378
00382     void draw();
00383
00390     void saveImage(const cv::Mat& image, const std::string& filename = "*.jpg") const;
00391 };

```

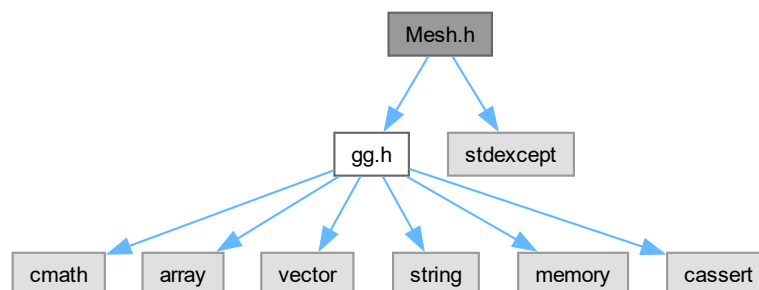
9.57 Mesh.h ファイル

```

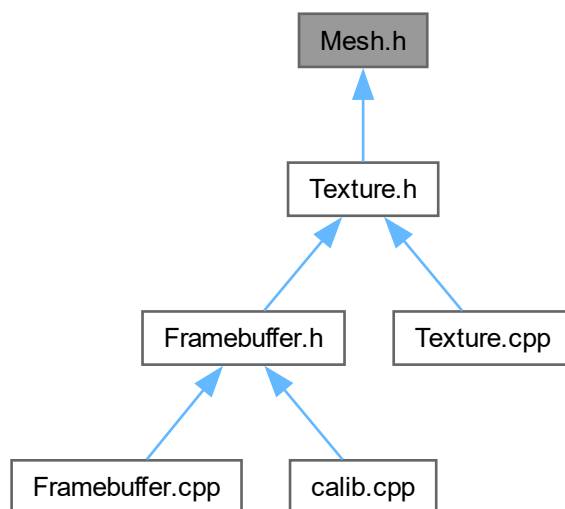
#include "gg.h"
#include <stdexcept>

```

Mesh.h の依存先関係図:



被依存関係図:



クラス

- class [Mesh](#)

9.57.1 詳解

メッシュの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Mesh.h](#) に定義があります。

9.58 Mesh.h

[詳解]

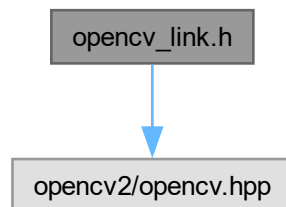
```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00014 // 標準ライブラリ
00015 #include <stdexcept>
00016
00020 class Mesh
00021 {
00023     const GLuint array;
00024
00026     const GLuint program;
00027
00029     const GLint imageLoc;
00030
00032     const GLint scaleLoc;
00033
00034 public:
00035
00039     Mesh()
00040     : array{ [] { GLuint array; glGenVertexArrays(1, &array); return array; }() }
00041     , program{ gg::ggLoadShader("draw.vert", "draw.frag") }
00042     , imageLoc{ glGetUniformLocation(program, "image") }
00043     , scaleLoc{ glGetUniformLocation(program, "scale") }
00044     {
00045         // 頂点配列オブジェクトが作れなかったら落とす
00046         assert(array);
00047
00048         // シェーダが作れなかったら落とす
00049         if (program == 0) throw std::runtime_error("Cannot create the shader for the mesh.");
00050     }
00051
00055     Mesh(const Mesh& mesh) = delete;
00056
00060     virtual ~Mesh()
00061     {
00062         glDeleteVertexArrays(1, &array);
00063         glDeleteProgram(program);
00064     }
00065
00072     Mesh& operator=(const Mesh& mesh) = delete;
00073
00084     void draw(const std::array<GLfloat, 2>& scale, GLint unit = 0) const
00085     {
00086         // 表示領域を背景色で塗りつぶす
00087         glClear(GL_COLOR_BUFFER_BIT);
00088
00089         // シェーダの使用開始
00090         glUseProgram(program);
00091
00092         // サンプラの指定
00093         glUniform1i(imageLoc, unit);
00094
00095         // スケールの設定
00096         glUniform2fv(scaleLoc, 1, scale.data());
00097
00098         // 描画
00099         glBindVertexArray(array);
00100         glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
00101         glBindVertexArray(0);
00102
00103         // シェーダの仕様終了
00104         glUseProgram(0);
00105     }
00106
00116     void drawMesh(const std::array<GLsizei, 2>& size) const
00117     {
00118         // 描画
00119         glBindVertexArray(array);
00120         glDrawArraysInstanced(GL_TRIANGLE_STRIP, 0, size[0] * 2, size[1] - 1);
00121         glBindVertexArray(0);
00122     }
00123 };

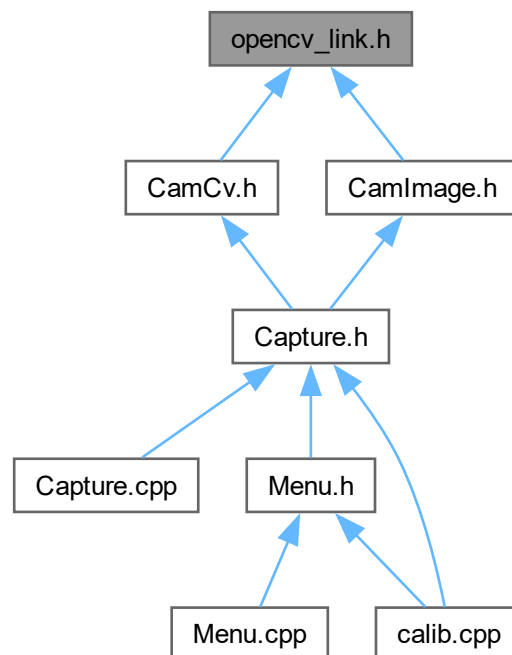
```

9.59 opencv_link.h ファイル

`#include <opencv2/opencv.hpp>`
opencv_link.h の依存先関係図:



被依存関係図:



9.59.1 詳解

Msvc環境用のOpenCV自動リンクおよびインクルード一括管理ヘッダー
[opencv_link.h](#) に定義があります。

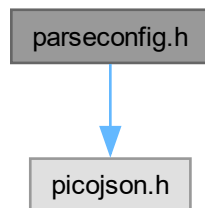
9.60 opencv_link.h

[詳解]

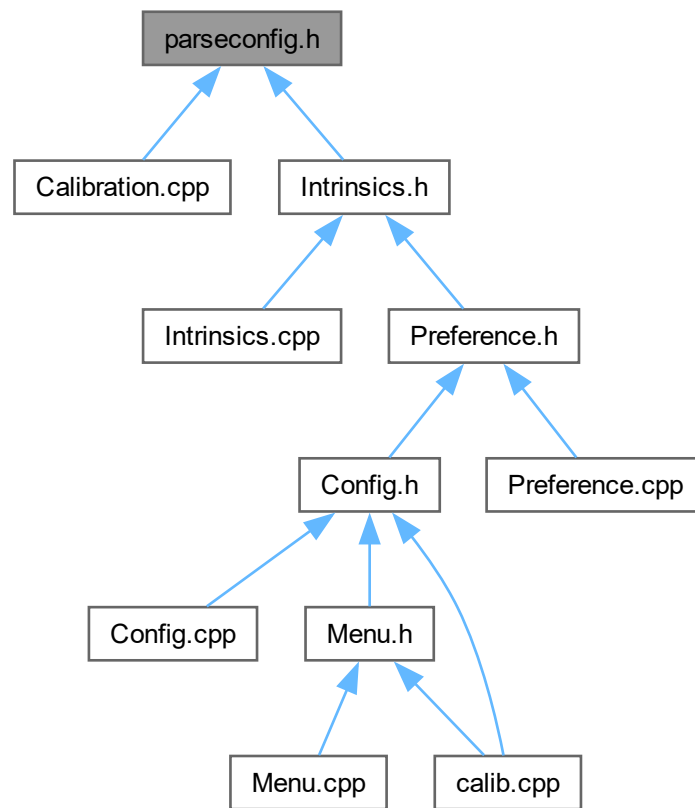
```
00001 #pragma once
00002
00008
00009 // OpenCV
00010 #pragma warning(disable:4819)
00011 #include <opencv2/opencv.hpp>
00012
00013 #if defined(_WIN32)
00014 #   define CV_VERSION_STR CVAUX_STR(CV_MAJOR_VERSION) CVAUX_STR(CV_MINOR_VERSION)
00015 #       CVAUX_STR(CV_SUBMINOR_VERSION)
00016 #   if defined(_DEBUG)
00017 #       define CV_EXT_STR "d.lib"
00018 #   else
00019 #       define CV_EXT_STR ".lib"
00020 #   endif
00021 #   pragma comment(lib, "opencv_world" CV_VERSION_STR CV_EXT_STR)
00022 #endif
```

9.61 parseconfig.h ファイル

```
#include "picojson.h"
parseconfig.h の依存先関係図:
```



被依存関係図:



関数

- `template<typename T>`
`bool getValue (const picojson::object &object, const std::string &key, T &scalar)`
- `template<typename T, std::size_t U>`
`bool getValue (const picojson::object &object, const std::string &key, std::array< T, U > &vector)`
- `bool getString (const picojson::object &object, const std::string &key, std::string &string)`
- `template<std::size_t U>`
`bool getString (const picojson::object &object, const std::string &key, std::array< std::string, U > &strings)`
- `bool getString (const picojson::object &object, const std::string &key, std::vector< std::string > &strings)`
- `template<typename T>`
`void setValue (picojson::object &object, const std::string &key, const T &scalar)`
- `template<typename T, std::size_t U>`
`void setValue (picojson::object &object, const std::string &key, const std::array< T, U > &vector)`
- `void setString (picojson::object &object, const std::string &key, const std::string &string)`
- `template<std::size_t U>`
`void setString (picojson::object &object, const std::string &key, const std::array< std::string, U > &strings)`
- `void setString (picojson::object &object, const std::string &key, const std::vector< std::string > &strings)`

9.61.1 詳解

構成ファイルの読み取り補助

著者

Kohe Tokoi

日付

November 15, 2022

[parseconfig.h](#) に定義があります。

9.61.2 関数詳解

9.61.2.1 getString() [1/3]

```
template<std::size_t U>
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトから文字列の配列を取得する

テンプレート引数

<i>U</i>	構成ファイルから取得する配列の要素の文字列の数
----------	-------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 105 行目に定義があります。

9.61.2.2 getString() [2/3]

```
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::string & string) [inline]
```

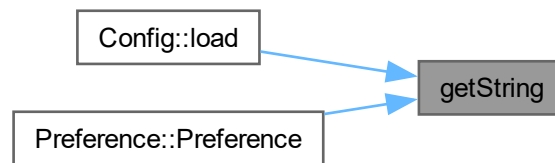
構成ファイルの JSON オブジェクトから文字列を取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>string</i>	取得した文字列を格納する変数

[parseconfig.h](#) の 81 行目に定義があります。

被呼び出し関係図:



9.61.2.3 getString() [3/3]

```

bool getString (
    const picojson::object & object,
    const std::string & key,
    std::vector< std::string > & strings) [inline]

```

構成ファイルの JSON オブジェクトから文字列のベクトルを取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 138 行目に定義があります。

9.61.2.4 getValue() [1/2]

```

template<typename T, std::size_t U>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    std::array< T, U > & vector)

```

構成ファイルの JSON オブジェクトから数値の配列を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する配列の要素の数値のデータ型
<i>U</i>	構成ファイルから取得する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値の配列を格納する変数

[parseconfig.h](#) の 48 行目に定義があります。

9.61.2.5 `getValue()` [2/2]

```
template<typename T>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    T & scalar)
```

構成ファイルの JSON オブジェクトから数値を取得する

テンプレート引数

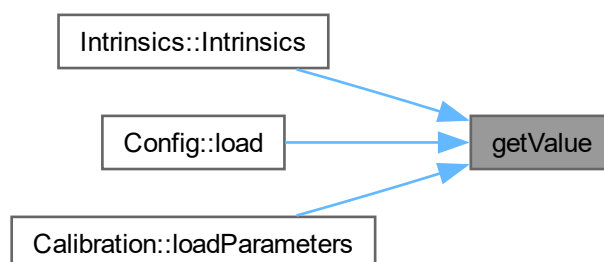
<i>T</i>	構成ファイルから取得する数値のデータ型
----------	---------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>scalar</i>	取得した JSON オブジェクトの数値を格納する変数

[parseconfig.h](#) の 23 行目に定義があります。

被呼び出し関係図:



9.61.2.6 setString() [1/3]

```
template<std::size_t U>
void setString (
    picojson::object & object,
    const std::string & key,
    const std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトに文字列の配列を設定する

テンプレート引数

<i>U</i>	構成ファイルに設定する配列の要素の文字列の数
----------	------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 225 行目に定義があります。

9.61.2.7 setString() [2/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::string & string) [inline]
```

構成ファイルの JSON オブジェクトに文字列を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>string</i>	設定する文字列

[parseconfig.h](#) の 210 行目に定義があります。

被呼び出し関係図:



9.61.2.8 setString() [3/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトに文字列のベクトルを設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 250 行目に定義があります。

9.61.2.9 setValue() [1/2]

```
template<typename T, std::size_t U>
void setValue (
    picojson::object & object,
    const std::string & key,
    const std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトに数値の配列を設定する

テンプレート引数

<i>T</i>	構成ファイルに設定する配列の要素の数値のデータ型
<i>U</i>	構成ファイルに設定する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定する数値の配列

[parseconfig.h](#) の 185 行目に定義があります。

9.61.2.10 setValue() [2/2]

```
template<typename T>
void setValue (
    picojson::object & object,
    const std::string & key,
    const T & scalar)
```

構成ファイルの JSON オブジェクトに数値を設定する

テンプレート引数

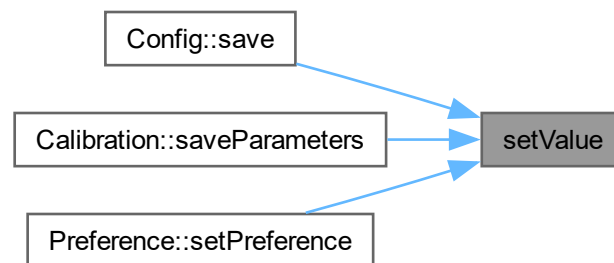
<i>T</i>	構成ファイルに設定する数値のデータ型
----------	--------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>scalar</i>	設定する数値

[parseconfig.h](#) の 169 行目に定義があります。

被呼び出し関係図:



9.62 parseconfig.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // JSON
00012 #include "picojson.h"
00013
00022 template <typename T>
00023 bool getValue(const picojson::object& object,
00024             const std::string& key, T& scalar)
00025 {
00026     // key に一致するオブジェクトを探す
00027     const auto&& value{ object.find(key) };
00028
00029     // オブジェクトが無い数値でなかったら戻る
00030     if (value == object.end() || !value->second.is<double>()) return false;
00031
00032     // 数値として格納する
00033     scalar = static_cast<T>(value->second.get<double>());
00034
00035     return true;
00036 }
00037
00047 template <typename T, std::size_t U>
00048 bool getValue(const picojson::object& object,
00049             const std::string& key, std::array<T, U>& vector)
00050 {
00051     // key に一致するオブジェクトを探す
00052     const auto&& value{ object.find(key) };
00053
  
```

```

00054 // オブジェクトが無いか配列でなかったら戻る
00055 if (value == object.end() || !value->second.is<picojson::array>()) return false;
00056
00057 // 配列を取り出す
00058 const auto& array{ value->second.get<picojson::array>() };
00059
00060 // 配列の要素数とデータの格納先の要素数の少ない方の数
00061 const auto n{ std::min(U, array.size()) };
00062
00063 // 配列の要素について
00064 for (std::size_t i = 0; i < n; ++i)
00065 {
00066     // 要素が数値なら格納する
00067     if (array[i].is<double>()) vector[i] = static_cast<T>(array[i].get<double>());
00068 }
00069 return true;
00070 }
00071
00072 inline
00081 bool getString(const picojson::object& object,
00082               const std::string& key, std::string& string)
00083 {
00084     // key に一致するオブジェクトを探す
00085     const auto&& value{ object.find(key) };
00086
00087     // オブジェクトが無いか文字列でなかったら戻る
00088     if (value == object.end() || !value->second.is<std::string>()) return false;
00089
00090     // 文字列として格納する
00091     string = value->second.get<std::string>();
00092
00093     return true;
00094 }
00095
00104 template <std::size_t U>
00105 bool getString(const picojson::object& object,
00106               const std::string& key, std::array<std::string, U>& strings)
00107 {
00108     // key に一致するオブジェクトを探す
00109     const auto&& value{ object.find(key) };
00110
00111     // オブジェクトが無いか配列でなかったら戻る
00112     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00113
00114     // 配列を取り出す
00115     const auto& array{ value->second.get<picojson::array>() };
00116
00117     // 配列の要素数とデータの格納先の要素数の少ない方の数
00118     const auto n{ std::min(U, array.size()) };
00119
00120     // 配列の要素について
00121     for (std::size_t i = 0; i < n; ++i)
00122     {
00123         // 要素が文字列なら文字列として格納する
00124         strings[i] = array[i].is<std::string>() ? array[i].get<std::string>() : "";
00125     }
00126
00127     return true;
00128 }
00129
00137 inline
00138 bool getString(const picojson::object& object,
00139               const std::string& key, std::vector<std::string>& strings)
00140 {
00141     // key に一致するオブジェクトを探す
00142     const auto&& value{ object.find(key) };
00143
00144     // オブジェクトが無いか配列でなかったら戻る
00145     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00146
00147     // 配列を取り出す
00148     const auto& array{ value->second.get<picojson::array>() };
00149
00150     // 配列のすべての要素について
00151     for (auto& element : array)
00152     {
00153         // 要素が文字列なら文字列として格納する
00154         strings.emplace_back(element.is<std::string>() ? element.get<std::string>() : "");
00155     }
00156
00157     return true;
00158 }
00159
00168 template <typename T>
00169 void setValue(picojson::object& object,
00170               const std::string& key, const T& scalar)

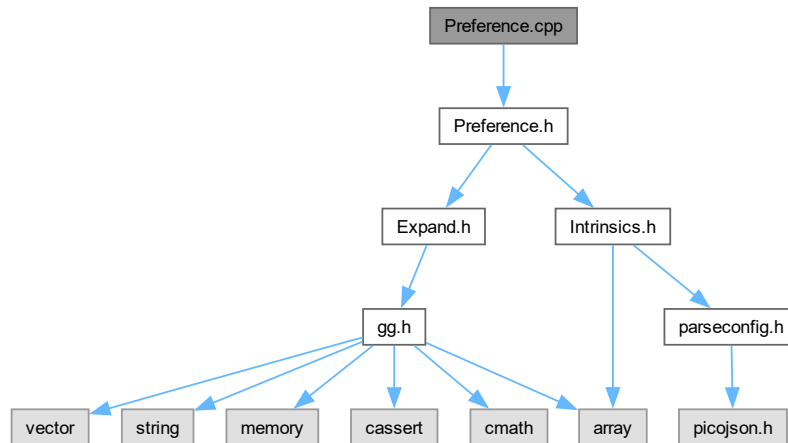
```

```
00171 {
00172     object.emplace(key, picojson::value(static_cast<double>(scalar)));
00173 }
00174
00184 template <typename T, std::size_t U>
00185 void setValue(picojson::object& object,
00186     const std::string& key, const std::array<T, U>& vector)
00187 {
00188     // picojson の配列
00189     picojson::array array;
00190
00191     // 配列のすべての要素について
00192     for (const auto& element : vector)
00193     {
00194         // 要素を picojson::array に追加する
00195         array.emplace_back(picojson::value(static_cast<double>(element)));
00196     }
00197
00198     // オブジェクトに追加する
00199     object.emplace(key, array);
00200 }
00201
00209 inline
00210 void setString(picojson::object& object,
00211     const std::string& key, const std::string& string)
00212 {
00213     object.emplace(key, picojson::value(string));
00214 }
00215
00224 template <std::size_t U>
00225 void setString(picojson::object& object,
00226     const std::string& key, const std::array<std::string, U>& strings)
00227 {
00228     // picojson の配列
00229     picojson::array array;
00230
00231     // 配列のすべての要素について
00232     for (const auto& string : strings)
00233     {
00234         // 要素を picojson::array に追加する
00235         array.emplace_back(picojson::value(string));
00236     }
00237
00238     // オブジェクトに追加する
00239     object.emplace(key, array);
00240 }
00241
00249 inline
00250 void setString(picojson::object& object,
00251     const std::string& key, const std::vector<std::string>& strings)
00252 {
00253     // picojson の配列
00254     picojson::array array;
00255
00256     // 配列のすべての要素について
00257     for (auto& string : strings)
00258     {
00259         // 要素を picojson::array に追加する
00260         array.emplace_back(picojson::value(string));
00261     }
00262
00263     // オブジェクトに追加する
00264     object.emplace(key, array);
00265 }
```

9.63 Preference.cpp ファイル

```
#include "Preference.h"
```

Preference.cpp の依存先関係図:



9.63.1 詳解

キャプチャデバイスの構成データクラスの実装

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.cpp](#) に定義があります。

9.64 Preference.cpp

[詳解]

```

00001
00008 #include "Preference.h"
00009
00010 //
00011 // キャプチャデバイスの構成データのデフォルトコンストラクタ
00012 //
00013 Preference::Preference()
00014 : Preference{ "Default", "orthographic.vert", "normal.frag" }
00015 {
00016 }
00017
00018 //
  
```



```

00019 // シェーダのソースファイルを指定するときに使う
00020 // キャプチャデバイスの構成データのコンストラクタ
00021 //
00022 Preference::Preference(const std::string& description,
00023     const std::string& vert, const std::string& frag,
00024     const Intrinsics& intrinsics)
00025 : description{ description }
00026 , source{ vert, frag }
00027 , intrinsics{ intrinsics }
00028 {
00029 }
00030
00031 //
00032 // 構成ファイルを読み込むときに使う
00033 // キャプチャデバイスの構成データのコンストラクタ
00034 //
00035 Preference::Preference(const picojson::object& object)
00036 : intrinsics{ object }
00037 {
00038     // 説明の文字列
00039     getString(object, "description", description);
00040
00041     // 展開用シェーダのファイル名
00042     getString(object, "shader", source);
00043 }
00044
00045 //
00046 // 構成データのデストラクタ
00047 //
00048 Preference::~Preference()
00049 {
00050     // shaderList は全 Preference で共有するため、個々の破棄時には消去しない
00051     // 静的オブジェクトとしてプログラム終了時にまとめて破棄される
00052 }
00053
00054 //
00055 // シェーダをビルドする
00056 //
00057 void Preference::buildShader()
00058 {
00059     // ソースファイル名をつないでシェーダを検索するキーを作る
00060     const auto key{ source[0] + "\t" + source[1] };
00061
00062     // キーがシェーダリストに無ければシェーダを構築して追加する
00063     shader = &shaderList.try_emplace(key, source[0], source[1]).first->second;
00064 }
00065
00066 //
00067 // 構成を JSON オブジェクトに格納する
00068 //
00069 void Preference::setPreference(picojson::object& object) const
00070 {
00071     // 説明の文字列
00072     setString(object, "description", description);
00073
00074     // キャプチャデバイスのレンズの縦横の画角
00075     setValue(object, "fov", intrinsics.fov);
00076
00077     // キャプチャデバイスのレンズの中心（主点）位置
00078     setValue(object, "center", intrinsics.center);
00079
00080     // キャプチャデバイスの解像度
00081     setValue(object, "resolution", intrinsics.size);
00082
00083     // キャプチャデバイスのフレームレート
00084     setValue(object, "fps", intrinsics.fps);
00085
00086     // 展開用シェーダのファイル名
00087     setString(object, "shader", source);
00088 }
00089
00090 // すべての構成のシェーダーのリスト
00091 std::map<std::string, Expand> Preference::shaderList;

```

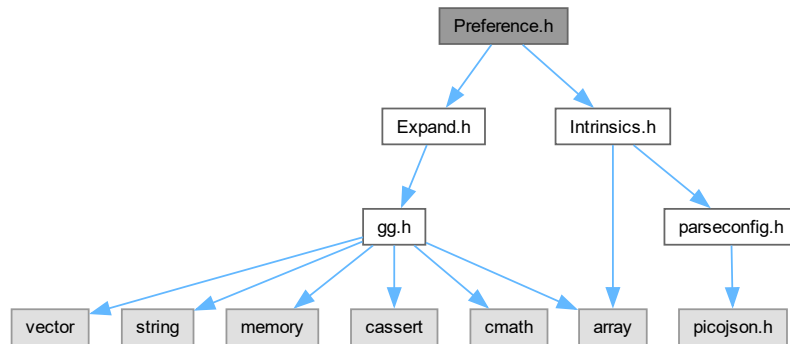
9.65 Preference.h ファイル

```

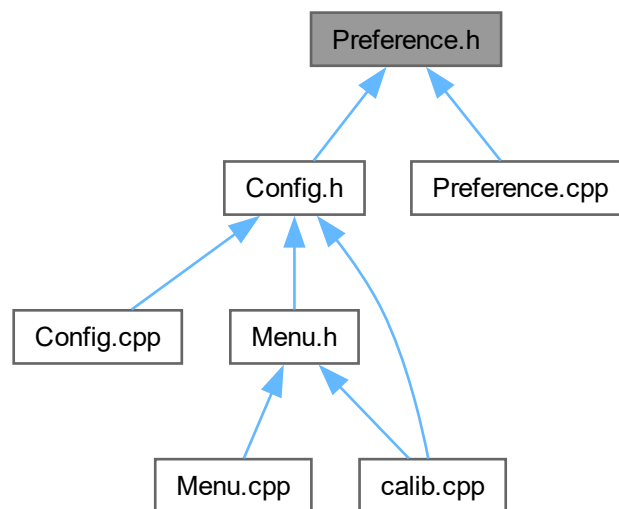
#include "Expand.h"
#include "Intrinsics.h"

```

Preference.h の依存先関係図:



被依存関係図:



クラス

- class [Preference](#)

9.65.1 詳解

キャプチャデバイスの構成データクラスの定義

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.h](#) に定義があります。

9.66 Preference.h

[詳解]

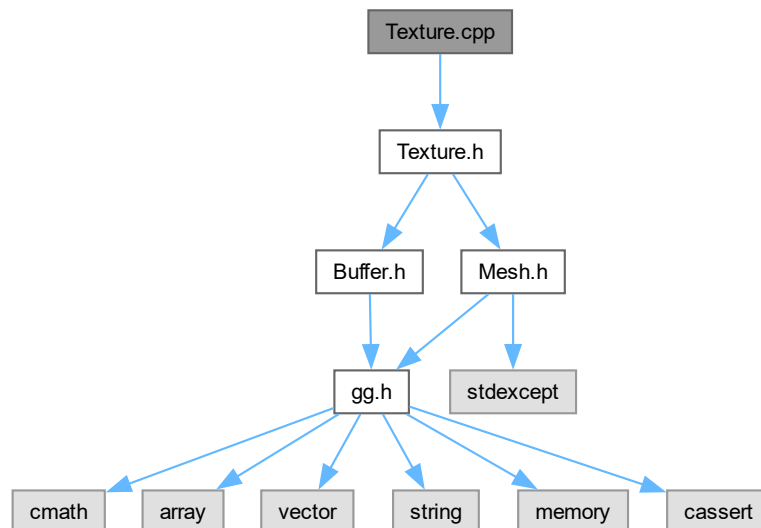
```
00001 #pragma once
00002
00010
00011 // 平面展開用のシェーダ
00012 #include "Expand.h"
00013
00014 // キャプチャデバイス固有のパラメータ
00015 #include "Intrinsics.h"
00016
00024 class Preference
00025 {
00027     std::string description;
00028
00030     std::array<std::string, 2> source;
00031
00033     const Intrinsics intrinsics;
00034
00036     const Expand* shader{ nullptr };
00037
00039     static std::map<std::string, Expand> shaderList;
00040
00041 public:
00042
00048     Preference();
00049
00059     Preference(const std::string& description,
00060               const std::string& vert, const std::string& frag,
00061               const Intrinsics& intrinsics = Intrinsics{});
00062
00067     Preference(const picojson::object& object);
00068
00072     virtual ~Preference();
00073
00077     void buildShader();
00078
00084     const auto& getDescription() const
00085     {
00086         return description;
00087     }
00088
00094     const auto& getIntrinsics() const
00095     {
00096         return intrinsics;
00097     }
00098
00104     const auto& getShader() const
00105     {
00106         return *shader;
00107     }
00108
00114     void setPreference(picojson::object& object) const;
00115 };
```

9.67 REQUESTS.md ファイル

9.68 Texture.cpp ファイル

```
#include "Texture.h"
```

Texture.cpp の依存先関係図:



9.68.1 詳解

テクスチャクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Texture.cpp](#) に定義があります。

9.69 Texture.cpp

[詳解]

```

00001
00008 #include "Texture.h"
00009
00010 //
00011 // テクスチャを作成するコンストラクタ
00012 //
00013 Texture::Texture(GLsizei width, GLsizei height, int channels)
00014 {
00015     // テクスチャを作る
00016     Texture::create(width, height, channels);
00017 }
00018
00019 //
00020 // コピーコンストラクタ
00021 //
00022 Texture::Texture(const Texture& texture)
00023 {
00024     // テクスチャをコピーして作成する
00025     Texture::copy(texture);
00026 }
00027
00028 //
00029 // デストラクタ
00030 //
00031 Texture::~Texture()
00032 {
00033     // テクスチャを破棄する
00034     Texture::discard();
00035 }
00036
00037 //
00038 // 代入演算子
00039 //
00040 Texture& Texture::operator=(const Texture& texture)
00041 {
00042     // 代入元と代入先が同じなら何もしない
00043     if (&texture == this) return *this;
00044
00045     // 引数のテクスチャをこのテクスチャにコピーする
00046     Texture::copy(texture);
00047
00048     // このテクスチャを返す
00049     return *this;
00050 }
00051
00052 //
00053 // ムーブ代入演算子
00054 //
00055 Texture& Texture::operator=(Texture&& texture) noexcept
00056 {
00057     // ムーブ代入元とムーブ代入先が同じなら何もしない
00058     if (&texture == this) return *this;
00059
00060     // ムーブ代入元の基底クラスをムーブする
00061     static_cast<Buffer&>(*this) = std::move(texture);
00062
00063     // ムーブ代入元のテクスチャのメンバをムーブする
00064     textureSize = texture.textureSize;
00065     texture.textureSize = { 0, 0 };
00066     textureChannels = texture.textureChannels;
00067     texture.textureChannels = 0;
00068
00069     // ムーブ代入先のテクスチャを削除する
00070     glDeleteTextures(1, &textureName);
00071
00072     // ムーブ代入元のテクスチャ名をムーブ代入先に移す
00073     textureName = texture.textureName;
00074
00075     // ムーブ代入元のデストラクタでテクスチャが削除されないよう 0 にする
00076     texture.textureName = 0;
00077
00078     // このテクスチャを返す
00079     return *this;
00080 }
00081
00082 //
00083 // テクスチャを作成する
00084 //
00085 void Texture::create(GLsizei width, GLsizei height, int channels)
00086 {
00087     // 既存のバッファを破棄して新しいバッファを作成する
00088     Buffer::create(width, height, channels);

```

```

00089
00090 // まだメッシュの頂点配列オブジェクトが作られていなければ作る
00091 if (!mesh) mesh = std::make_shared<Mesh>();
00092
00093 // 指定したサイズが保持しているテクスチャのサイズと同じなら何もしない
00094 if (width == textureSize[0] && height == textureSize[1]
00095     && channels == textureChannels) return;
00096
00097 // テクスチャのサイズとチャンネル数を記録する
00098 textureSize = std::array<int, 2>{ width, height };
00099 textureChannels = channels;
00100
00101 // 以前のテクスチャを削除する
00102 glDeleteTextures(1, &textureName);
00103
00104 // テクスチャを作成する
00105 glGenTextures(1, &textureName);
00106
00107 // テクスチャのメモリを確保してデータを転送する
00108 glBindTexture(GL_TEXTURE_2D, textureName);
00109 glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
00110     channelsToFormat(channels), GL_UNSIGNED_BYTE, nullptr);
00111 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00112 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00113 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00114 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00115 glBindTexture(GL_TEXTURE_2D, 0);
00116 }
00117
00118 //
00119 // テクスチャをコピーする
00120 //
00121 void Texture::copy(const Buffer& texture) noexcept
00122 {
00123     // コピー元と同じサイズの空のテクスチャを作る
00124     Texture::create(texture.getWidth(), texture.getHeight(),
00125         texture.getChannels());
00126
00127     // 作成したテクスチャのバッファにコピーする
00128     copyBuffer(texture);
00129 }
00130
00131 //
00132 // テクスチャを破棄する
00133 //
00134 void Texture::discard()
00135 {
00136     // デフォルトのテクスチャに戻す
00137     glBindTexture(GL_TEXTURE_2D, 0);
00138
00139     // テクスチャを削除する
00140     glDeleteTextures(1, &textureName);
00141     textureName = 0;
00142
00143     // テクスチャのサイズを 0 にする
00144     textureSize = std::array<int, 2>{ 0, 0 };
00145     textureChannels = 0;
00146 }
00147
00148 //
00149 // このテクスチャをマッピングしてメッシュを描画する
00150 //
00151 void Texture::draw(GLsizei width, GLsizei height, int unit) const
00152 {
00153     // テクスチャの縦横比
00154     const auto t{ static_cast<float>(textureSize[0] * height) };
00155
00156     // 表示領域の縦横比
00157     const auto d{ static_cast<float>(textureSize[1] * width) };
00158
00159     // テクスチャのスケール
00160     const std::array<GLfloat, 2> scale{ t > d // テクスチャの縦横比の方が
00161         ? std::array<GLfloat, 2>{ 1.0f, t / d } // 大きければ横方向いっぱい描く
00162         : std::array<GLfloat, 2>{ d / t, 1.0f } // それ以外は縦方向いっぱい描く
00163     };
00164
00165     // このオブジェクトのテクスチャを指定する
00166     bindTexture(unit);
00167
00168     // メッシュを描画する
00169     mesh->draw(scale, unit);
00170
00171     // テクスチャの指定を解除する
00172     unbindTexture();
00173 }
00174
00175 //

```

```

00176 // テクスチャからピクセルバッファオブジェクトにデータをコピーする
00177 //
00178 void Texture::readPixels(
00179     #if defined(USE_PIXEL_BUFFER_OBJECT)
00180     GLuint buffer) const
00181     #else
00182     std::vector<GLubyte>& buffer)
00183     #endif
00184 {
00185     // 読み出し元のテクスチャを結合する
00186     glBindTexture(GL_TEXTURE_2D, textureName);
00187
00188     #if defined(USE_PIXEL_BUFFER_OBJECT)
00189     // 書き込み先のピクセルバッファオブジェクトを指定する
00190     glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer);
00191
00192     # if defined(GL_GLES_PROTOTYPES)
00193     // 現在使っているフレームバッファオブジェクトを調べる
00194     GLuint currentFbo;
00195     glGetIntegerv(GL_FRAMEBUFFER_BINDING, reinterpret_cast<GLint*>(&currentFbo));
00196
00197     // テクスチャをカラーバッファに持ってフレームバッファオブジェクトを作る
00198     GLuint fbo;
00199     glGenFramebuffers(1, &fbo);
00200     glBindFramebuffer(GL_FRAMEBUFFER, fbo);
00201     static const GLenum attachment{ GL_COLOR_ATTACHMENT0 };
00202     glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, textureName, 0);
00203     glDrawBuffers(1, &attachment);
00204     glReadBuffer(attachment);
00205
00206     // フレームバッファオブジェクトからピクセルバッファオブジェクトに読み出す
00207     glReadPixels(0, 0, textureSize[0], textureSize[1], channelsToFormat(textureChannels),
00208         GL_UNSIGNED_BYTE, 0);
00209
00210     // 元のフレームバッファオブジェクトに戻す
00211     glBindFramebuffer(GL_FRAMEBUFFER, currentFbo);
00212
00213     // 作ったフレームバッファオブジェクトは削除する
00214     glDeleteFramebuffers(1, &fbo);
00215     # else
00216     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00217     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, 0);
00218     # endif
00219
00220     // 書き込み先のピクセルバッファオブジェクトの結合を解除する
00221     glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00222
00223     #else
00224
00225     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00226     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00227
00228     #endif
00229
00230     // 読み出し元のテクスチャの結合を解除する
00231     glBindTexture(GL_TEXTURE_2D, 0);
00232 }
00233
00234 //
00235 // ピクセルバッファオブジェクトからテクスチャにデータをコピーする
00236 //
00237 void Texture::drawPixels(
00238     #if defined(USE_PIXEL_BUFFER_OBJECT)
00239     GLuint
00240     #else
00241     const std::vector<GLubyte>&
00242     #endif
00243     buffer) const
00244 {
00245     // 書き込み先のテクスチャを結合する
00246     glBindTexture(GL_TEXTURE_2D, textureName);
00247
00248     #if defined(USE_PIXEL_BUFFER_OBJECT)
00249
00250     // 読み出し元のピクセルバッファオブジェクトを指定する
00251     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, buffer);
00252
00253     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む
00254     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],
00255         getFormat(), GL_UNSIGNED_BYTE, 0);
00256
00257     // 読み出し元のピクセルバッファオブジェクトの結合を解除する
00258     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00259
00260     #else
00261
00262     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む

```

```

00263     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],
00264         getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00265
00266 #endif
00267
00268 // 書き込み先のテクスチャの結合を解除する
00269 glBindTexture(GL_TEXTURE_2D, 0);
00270 }
00271
00272 //
00273 // CPU メモリ上の画像をテクスチャへ転送する
00274 //
00275 void Texture::drawPixels(GLsizei width, GLsizei height, int channels, const void* pixels)
00276 {
00277     // 空画像や不正なサイズでは OpenGL を呼ばず、現在のテクスチャを維持する。
00278     if (!pixels || width <= 0 || height <= 0 || channels <= 0) return;
00279
00280     // OpenCV 補正後の画像サイズに合わせる。サイズが同じなら再確保は行われない。
00281     create(width, height, channels);
00282
00283     // cv::Mat の連続したCPU画素を、現在のテクスチャへ直接アップロードする。
00284     glBindTexture(GL_TEXTURE_2D, textureName);
00285
00286     // 3チャンネル画像など、1行のバイト数が4の倍数でない場合にも対応する。
00287     glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
00288     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height,
00289         channelsToFormat(channels), GL_UNSIGNED_BYTE, pixels);
00290
00291     // 後続描画へ意図しないテクスチャ結合状態を残さない。
00292     glBindTexture(GL_TEXTURE_2D, 0);
00293 }
00294
00295 // 展開に用いるメッシュの頂点配列オブジェクト
00296 std::shared_ptr<Mesh> Texture::mesh;

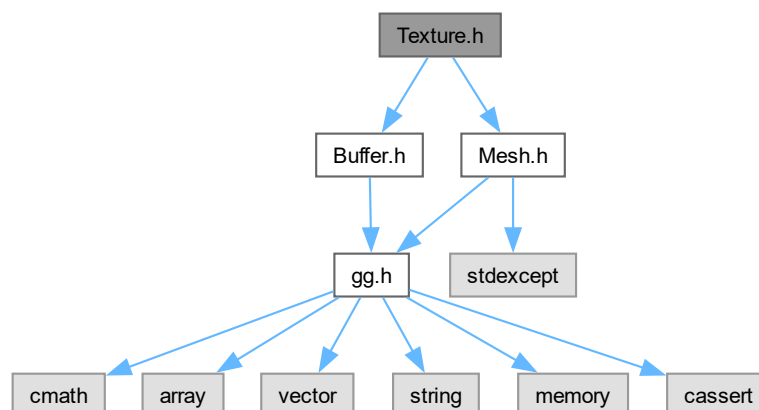
```

9.70 Texture.h ファイル

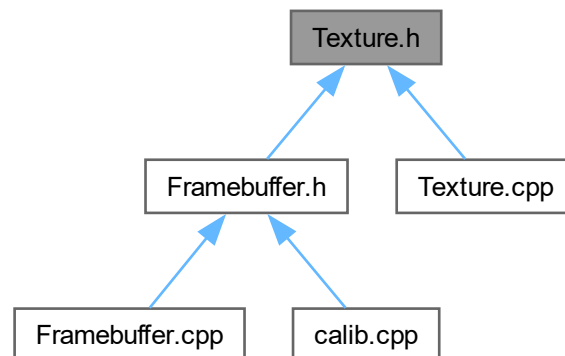
```
#include "Buffer.h"
```

```
#include "Mesh.h"
```

Texture.h の依存先関係図:



被依存関係図:



クラス

- class [Texture](#)

9.70.1 詳解

テクスチャクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Texture.h](#) に定義があります。

9.71 Texture.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // バッファクラス
00012 #include "Buffer.h"
00013
00014 // テクスチャの展開に用いるメッシュ
00015 #include "Mesh.h"
00016
00020 class Texture : public Buffer
00021 {
00023     std::array<int, 2> textureSize{ 0, 0 };
```

```

00024
00026     int textureChannels{ 0 };
00027
00029     GLuint textureName{ 0 };
00030
00031 protected:
00032
00034     static std::shared_ptr<Mesh> mesh;
00035
00036 public:
00037
00041     Texture() = default;
00042
00050     Texture(GLsizei width, GLsizei height, int channels);
00051
00057     Texture(const Texture& texture);
00058
00064     Texture(Texture&& texture) noexcept
00065         : Buffer{ std::move(texture) }
00066         , textureSize{ texture.textureSize }
00067         , textureChannels{ texture.textureChannels }
00068         , textureName{ texture.textureName }
00069     {
00070         texture.textureSize = { 0, 0 };
00071         texture.textureChannels = 0;
00072
00073         // ムーブ元のデストラクタでテクスチャが削除されないよう 0 にする
00074         texture.textureName = 0;
00075     }
00076
00080     virtual ~Texture();
00081
00088     Texture& operator=(const Texture& texture);
00089
00096     Texture& operator=(Texture&& texture) noexcept;
00097
00109     virtual void create(GLsizei width, GLsizei height, int channels);
00110
00123     virtual void copy(const Buffer& texture) noexcept;
00124
00128     virtual void discard();
00129
00135     auto getTextureName() const
00136     {
00137         return textureName;
00138     }
00139
00143     virtual const std::array<GLsizei, 2>& getSize() const
00144     {
00145         return textureSize;
00146     }
00147
00151     virtual int getChannels() const
00152     {
00153         return textureChannels;
00154     }
00155
00165     void bindTexture(int unit = 0) const
00166     {
00167         // テクスチャユニットを指定する
00168         glActiveTexture(GL_TEXTURE0 + unit);
00169
00170         // テクスチャを指定する
00171         glBindTexture(GL_TEXTURE_2D, textureName);
00172     }
00173
00177     void unbindTexture() const
00178     {
00179         // デフォルトのテクスチャユニットに戻す
00180         glActiveTexture(GL_TEXTURE0);
00181
00182         // デフォルトのテクスチャに戻す
00183         glBindTexture(GL_TEXTURE_2D, 0);
00184     }
00185
00198     void draw(GLsizei width, GLsizei height, int unit = 0) const;
00199
00208     void readPixels(
00209 #if defined(USE_PIXEL_BUFFER_OBJECT)
00210         GLuint buffer) const
00211 #else
00212         std::vector<GLubyte>& buffer)
00213 #endif
00214     ;
00215
00224     void readPixels(Buffer& buffer)
00225 #if defined(USE_PIXEL_BUFFER_OBJECT)

```

```
00226     const
00227 #endif
00228 {
00229     // このテクスチャのバッファから引数のオブジェクトのバッファにコピーする
00230     readPixels();
00231
00232     // 引数のオブジェクトのサイズをこのテクスチャに合わせてコピーする
00233     buffer.Buffer::copy(*this);
00234 }
00235
00239 void readPixels()
00240 #if defined(USE_PIXEL_BUFFER_OBJECT)
00241     const
00242 #endif
00243 {
00244     // このテクスチャからこのバッファにコピーする
00245     readPixels(getBufferName());
00246 }
00247
00256 void drawPixels(
00257 #if defined(USE_PIXEL_BUFFER_OBJECT)
00258     GLuint
00259 #else
00260     const std::vector<GLubyte>&
00261 #endif
00262     buffer) const;
00263
00272 void drawPixels(const Texture& texture)
00273 {
00274     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00275     copy(texture);
00276
00277     // バッファからこのテクスチャにコピーする
00278     drawPixels();
00279 }
00280
00289 void drawPixels(const Buffer& buffer)
00290 {
00291     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00292     copy(buffer);
00293
00294     // バッファからこのテクスチャにコピーする
00295     drawPixels();
00296 }
00297
00301 void drawPixels()
00302 #if defined(USE_PIXEL_BUFFER_OBJECT)
00303     const
00304 #endif
00305 {
00306     // このバッファからこのテクスチャにコピーする
00307     drawPixels(getBufferName());
00308 }
00309
00320 void drawPixels(GLsizei width, GLsizei height, int channels, const void* pixels);
00321 };
```


Index

- `~ggError`
 - `gg`, [20](#)
- `~ggFBOError`
 - `gg`, [20](#)
- `~Buffer`
 - `Buffer`, [93](#)
- `~Calibration`
 - `Calibration`, [110](#)
- `~CamCv`
 - `CamCv`, [122](#)
- `~CamImage`
 - `CamImage`, [144](#)
- `~CamMf`
 - `CamMf`, [149](#)
- `~Camera`
 - `Camera`, [131](#)
- `~Capture`
 - `Capture`, [154](#)
- `~Compute`
 - `Compute`, [162](#)
- `~Config`
 - `Config`, [165](#)
- `~Expand`
 - `Expand`, [173](#)
- `~Framebuffer`
 - `Framebuffer`, [180](#)
- `~GgApp`
 - `GgApp`, [191](#)
- `~GgBuffer`
 - `gg::GgBuffer< T >`, [196](#)
- `~GgColorTexture`
 - `gg::GgColorTexture`, [205](#)
- `~GgElements`
 - `gg::GgElements`, [211](#)
- `~GgMatrix`
 - `gg::GgMatrix`, [221](#)
- `~GgNormalTexture`
 - `gg::GgNormalTexture`, [278](#)
- `~GgPointShader`
 - `gg::GgPointShader`, [289](#)
- `~GgPoints`
 - `gg::GgPoints`, [283](#)
- `~GgQuaternion`
 - `gg::GgQuaternion`, [308](#)
- `~GgShader`
 - `gg::GgShader`, [376](#)
- `~GgShape`
 - `gg::GgShape`, [380](#)
- `~GgSimpleObj`
 - `gg::GgSimpleObj`, [384](#)
- `~GgSimpleShader`
 - `gg::GgSimpleShader`, [390](#)
- `~GgTexture`
 - `gg::GgTexture`, [411](#)
- `~GgTrackball`
 - `gg::GgTrackball`, [420](#)
- `~GgTriangles`
 - `gg::GgTriangles`, [430](#)
- `~GgUniformBuffer`
 - `gg::GgUniformBuffer< T >`, [435](#)
- `~GgVector`
 - `gg::GgVector`, [451](#)
- `~GgVertexArray`
 - `gg::GgVertexArray`, [478](#)
- `~LightBuffer`
 - `gg::GgSimpleShader::LightBuffer`, [490](#)
- `~MaterialBuffer`
 - `gg::GgSimpleShader::MaterialBuffer`, [509](#)
- `~Menu`
 - `Menu`, [525](#)
- `~Mesh`
 - `Mesh`, [532](#)
- `~Preference`
 - `Preference`, [535](#)
- `~Texture`
 - `Texture`, [545](#)
- `~Window`
 - `GgApp::Window`, [564](#)
- `add`
 - `gg::GgQuaternion`, [308–310](#)
- `ambient`
 - `gg::GgSimpleShader::Light`, [486](#)
 - `gg::GgSimpleShader::Material`, [505](#)
- `begin`
 - `gg::GgTrackball`, [420](#)
- `bind`
 - `gg::GgBuffer< T >`, [196](#)
 - `gg::GgTexture`, [411](#)
 - `gg::GgUniformBuffer< T >`, [435](#)
 - `gg::GgVertexArray`, [478](#)
- `bindBuffer`
 - `Buffer`, [93](#)
- `bindFramebuffer`
 - `Framebuffer`, [180](#)
- `BindingPoints`
 - `gg`, [20](#)
- `bindTexture`

- Texture, 546
- Buffer, 89
 - ~Buffer, 93
 - bindBuffer, 93
 - Buffer, 90–92
 - channelsToFormat, 94
 - copy, 94
 - copyBuffer, 95
 - create, 96
 - discard, 97
 - formatToChannels, 97
 - getAspect, 99
 - getBufferName, 99
 - getChannels, 100
 - getFormat, 100
 - getHeight, 101
 - getSize, 102
 - getWidth, 103
 - map, 103
 - operator=, 104
 - resize, 105, 106
 - unbindBuffer, 107
 - unmap, 107
- Buffer.cpp, 591
- Buffer.h, 594
 - USE_PIXEL_BUFFER_OBJECT, 595
- buildShader
 - Preference, 536
- calib.cpp, 597
 - CONFIG_FILE, 598
- calibrate
 - Calibration, 110
- Calibration, 108
 - ~Calibration, 110
 - calibrate, 110
 - Calibration, 109
 - createBoard, 110
 - detectBoard, 111
 - detectMarkers, 111
 - dictionaryList, 119
 - discardCorners, 112
 - drawBoard, 112
 - finished, 113
 - getAllMarkerPoses, 113
 - getCameraMatrix, 114
 - getCornersCount, 114
 - getDistortionCoefficients, 114
 - getReprojectionError, 115
 - getSampleCount, 115
 - getTotalCount, 115
 - loadParameters, 115
 - operator=, 116
 - recordCorners, 116
 - RvecTvecToPose, 116
 - saveParameters, 117
 - setDictionary, 118
- Calibration.cpp, 600
 - USE_RODRIGUES, 601
- Calibration.h, 607
- CamCv, 120
 - ~CamCv, 122
 - CamCv, 122
 - close, 122
 - decreaseExposure, 122
 - decreaseGain, 123
 - getCodec, 123, 124
 - getFps, 124
 - getPosition, 125
 - increaseExposure, 125
 - increaseGain, 125
 - open, 126
 - setExposure, 127
 - setGain, 127
 - setPosition, 128
- CamCv.h, 610
- Camera, 129
 - ~Camera, 131
 - Camera, 131
 - capture, 132
 - captured, 138
 - channels, 138
 - close, 132
 - decreaseExposure, 133
 - decreaseGain, 133
 - frame, 138
 - getChannels, 133
 - getFps, 133
 - getFrames, 134
 - getHeight, 134
 - getPrioritizeLatency, 134
 - getSize, 134
 - getWidth, 135
 - height, 138
 - image, 139
 - in, 139
 - increaseExposure, 135
 - increaseGain, 135
 - interval, 139
 - isRunning, 135
 - mtx, 139
 - operator=, 135
 - out, 139
 - prioritizeLatency, 139
 - reusableFrame, 140
 - running, 140
 - setPrioritizeLatency, 136
 - start, 136
 - stop, 136
 - thr, 140
 - total, 140
 - transmit, 137, 138
 - width, 140
- Camera.h, 615
- CamImage, 141
 - ~CamImage, 144
 - CamImage, 143

- isOpened, [144](#)
- load, [144](#)
- open, [145](#)
- save, [146](#)
- CamImage.h, [619](#)
- CamMf, [147](#)
 - ~CamMf, [149](#)
 - CamMf, [149](#)
 - capture, [150](#)
 - close, [150](#)
 - getDeviceList, [150](#)
 - getFormatList, [151](#)
 - open, [151](#)
 - select, [152](#)
 - stop, [152](#)
- CamMf.cpp, [622](#)
 - SafeRelease, [623](#)
 - SubTypeToName, [624](#)
- CamMf.h, [638](#)
- Capture, [153](#)
 - ~Capture, [154](#)
 - Capture, [154](#)
 - close, [155](#)
 - getFps, [155](#)
 - getPrioritizeLatency, [155](#)
 - getSize, [155](#)
 - isImage, [156](#)
 - isOpened, [156](#)
 - openDevice, [156](#)
 - openImage, [157](#)
 - openMovie, [157](#)
 - operator bool, [158](#)
 - retrieve, [158](#), [159](#)
 - setPrioritizeLatency, [159](#)
 - start, [159](#)
 - stop, [160](#)
- capture
 - Camera, [132](#)
 - CamMf, [150](#)
- Capture.cpp, [641](#)
- Capture.h, [644](#)
- captured
 - Camera, [138](#)
- CaptureFormat, [160](#)
 - CaptureFormat, [161](#)
 - codec, [161](#)
 - fps, [161](#)
 - index, [161](#)
 - resolution, [161](#)
- center
 - Intrinsics, [483](#)
- channels
 - Camera, [138](#)
- channelsToFormat
 - Buffer, [94](#)
- checkerLength
 - Settings, [538](#)
- checkerSize
 - Settings, [538](#)
- close
 - CamCv, [122](#)
 - Camera, [132](#)
 - CamMf, [150](#)
 - Capture, [155](#)
- codec
 - CaptureFormat, [161](#)
- Compute, [162](#)
 - ~Compute, [162](#)
 - Compute, [162](#)
 - execute, [163](#)
 - get, [163](#)
 - use, [163](#)
- Compute.h, [647](#)
- Config, [164](#)
 - ~Config, [165](#)
 - Config, [164](#)
 - getBackground, [165](#)
 - getCheckerLength, [165](#)
 - getCheckerSize, [165](#)
 - getDictionaryName, [166](#)
 - getHeight, [166](#)
 - getInitialImage, [167](#)
 - getMarkerLength, [167](#)
 - getMenuFont, [168](#)
 - getMenuFontSize, [168](#)
 - getPreferences, [168](#)
 - getSettings, [168](#)
 - getTitle, [169](#)
 - getWidth, [169](#)
 - initialize, [169](#)
 - load, [170](#)
 - save, [170](#)
 - setSettings, [171](#)
- Config.cpp, [648](#)
- Config.h, [653](#)
- CONFIG_FILE
 - calib.cpp, [598](#)
- conjugate
 - gg::GgQuaternion, [311](#)
- copy
 - Buffer, [94](#)
 - Framebuffer, [181](#)
 - gg::GgBuffer< T >, [196](#)
 - gg::GgUniformBuffer< T >, [436](#)
 - Texture, [547](#)
- copyBuffer
 - Buffer, [95](#)
- create
 - Buffer, [96](#)
 - Framebuffer, [182](#)
 - Texture, [548](#)
- createBoard
 - Calibration, [110](#)
- decreaseExposure
 - CamCv, [122](#)
 - Camera, [133](#)

- decreaseGain
 - CamCv, [123](#)
 - Camera, [133](#)
- defaultCheckerLength
 - Settings, [538](#)
- defaultCheckerSize
 - Settings, [539](#)
- defaultEuler
 - Settings, [539](#)
- defaultFocal
 - Settings, [539](#)
- defaultFocalRange
 - Settings, [539](#)
- defaultMarkerLength
 - Settings, [539](#)
- detectBoard
 - Calibration, [111](#)
 - Menu, [530](#)
- detectMarker
 - Menu, [530](#)
- detectMarkers
 - Calibration, [111](#)
- dictionaryList
 - Calibration, [119](#)
- dictionaryName
 - Settings, [539](#)
- diffuse
 - gg::GgSimpleShader::Light, [486](#)
 - gg::GgSimpleShader::Material, [505](#)
- discard
 - Buffer, [97](#)
 - Framebuffer, [183](#)
 - Texture, [549](#)
- discardCorners
 - Calibration, [112](#)
- distance3
 - gg::GgVector, [451](#)
- distance4
 - gg::GgVector, [451](#)
- divide
 - gg::GgQuaternion, [311–313](#)
- dot3
 - gg::GgVector, [452](#)
- dot4
 - gg::GgVector, [453](#)
- draw
 - gg::GgElements, [211](#)
 - gg::GgPoints, [283](#)
 - gg::GgShape, [380](#)
 - gg::GgSimpleObj, [384](#)
 - gg::GgTriangles, [430](#)
 - Menu, [526](#)
 - Mesh, [532](#)
 - Texture, [550](#)
- drawBoard
 - Calibration, [112](#)
- drawMesh
 - Mesh, [532](#)
- drawPixels
 - Texture, [551–553](#)
- end
 - gg::GgTrackball, [420](#)
- euler
 - gg::GgQuaternion, [314–316](#)
 - Settings, [540](#)
- execute
 - Compute, [163](#)
- Expand, [172](#)
 - ~Expand, [173](#)
 - Expand, [172](#)
 - getProgram, [173](#)
 - operator=, [173](#)
 - setup, [174](#)
- Expand.cpp, [656](#)
- Expand.h, [658](#)
- fill
 - gg::GgUniformBuffer< T >, [437](#)
- finished
 - Calibration, [113](#)
- focal
 - Settings, [540](#)
- focalRange
 - Settings, [540](#)
- formatToChannels
 - Buffer, [97](#)
- fov
 - Intrinsics, [483](#)
- fps
 - CaptureFormat, [161](#)
 - Intrinsics, [484](#)
- frame
 - Camera, [138](#)
- Framebuffer, [175](#)
 - ~Framebuffer, [180](#)
 - bindFramebuffer, [180](#)
 - copy, [181](#)
 - create, [182](#)
 - discard, [183](#)
 - Framebuffer, [178, 179](#)
 - getChannels, [183](#)
 - getFramebufferName, [183](#)
 - getSize, [184](#)
 - operator=, [184](#)
 - show, [186](#)
 - unbindFramebuffer, [186](#)
 - update, [187, 188](#)
- Framebuffer.cpp, [660](#)
- Framebuffer.h, [664](#)
- frustum
 - gg::GgMatrix, [221](#)
- get
 - Compute, [163](#)
 - gg::GgMatrix, [222, 223](#)
 - gg::GgPointShader, [290](#)

- gg::GgQuaternion, 317
- gg::GgShader, 377
- gg::GgShape, 381
- gg::GgSimpleObj, 384
- gg::GgTrackball, 421
- gg::GgVertexArray, 478
- GgApp::Window, 565
- getAllMarkerPoses
 - Calibration, 113
- getAltArrowX
 - GgApp::Window, 565
- getAltArrowY
 - GgApp::Window, 565
- getAltArrow
 - GgApp::Window, 566
- getAnyList
 - Menu.cpp, 834
- getArrow
 - GgApp::Window, 566, 567
- getArrowX
 - GgApp::Window, 568
- getArrowY
 - GgApp::Window, 569
- getAspect
 - Buffer, 99
 - GgApp::Window, 569
- getBackground
 - Config, 165
- getBuffer
 - gg::GgBuffer< T >, 197
 - gg::GgPoints, 284
 - gg::GgTriangles, 430
 - gg::GgUniformBuffer< T >, 438
- getBufferName
 - Buffer, 99
- getCameraMatrix
 - Calibration, 114
- getChannels
 - Buffer, 100
 - Camera, 133
 - Framebuffer, 183
 - Texture, 554
- getCheckerLength
 - Config, 165
 - Menu, 526
- getCheckerSize
 - Config, 165
 - Menu, 526
- getCodec
 - CamCv, 123, 124
- getConjugateMatrix
 - gg::GgQuaternion, 317, 318
- getControlArrow
 - GgApp::Window, 570
- getControlArrowX
 - GgApp::Window, 570
- getControlArrowY
 - GgApp::Window, 571
- getCornersCount
 - Calibration, 114
- getCount
 - gg::GgBuffer< T >, 197
 - gg::GgPoints, 284
 - gg::GgTriangles, 431
 - gg::GgUniformBuffer< T >, 439
- getDescription
 - Preference, 536
- getDeviceList
 - CamMf, 150
- getDictionaryName
 - Config, 166
- getDistortionCoefficients
 - Calibration, 114
- getFboHeight
 - GgApp::Window, 572
- getFboSize
 - GgApp::Window, 572
- getFboWidth
 - GgApp::Window, 573
- getFocal
 - Settings, 538
- getFormat
 - Buffer, 100
- getFormatList
 - CamMf, 151
- getFps
 - CamCv, 124
 - Camera, 133
 - Capture, 155
- getFramebufferName
 - Framebuffer, 183
- getFrames
 - Camera, 134
- getHeight
 - Buffer, 101
 - Camera, 134
 - Config, 166
 - gg::GgTexture, 411
 - GgApp::Window, 573
- getIndexBuffer
 - gg::GgElements, 212
- getIndexCount
 - gg::GgElements, 212
- getInitialImage
 - Config, 167
- getIntrinsics
 - Preference, 536
- getKey
 - GgApp::Window, 574
- getLastKey
 - GgApp::Window, 574
- getMarkerLength
 - Config, 167
 - Menu, 526
- getMatrix
 - gg::GgQuaternion, 319, 321

- gg::GgTrackball, 421
- getMenuBarHeight
 - Menu, 527
- getMenuFont
 - Config, 168
- getMenuFontSize
 - Config, 168
- getMode
 - gg::GgShape, 381
- getMouse
 - GgApp::Window, 574, 576
- getMouseX
 - GgApp::Window, 576
- getMouseY
 - GgApp::Window, 576
- getPose
 - Menu, 527
- getPosition
 - CamCv, 125
- getPreferences
 - Config, 168
- getPrioritizeLatency
 - Camera, 134
 - Capture, 155
- getProgram
 - Expand, 173
- getQuaternion
 - gg::GgTrackball, 421
- getReprojectionError
 - Calibration, 115
- getRotation
 - GgApp::Window, 576
- getRotationMatrix
 - GgApp::Window, 577
- getSampleCount
 - Calibration, 115
- getScale
 - gg::GgTrackball, 422
- getScrollMatrix
 - GgApp::Window, 577
- getSettings
 - Config, 168
- getShader
 - Preference, 536
- getShiftArrow
 - GgApp::Window, 577
- getShiftArrowX
 - GgApp::Window, 578
- getShiftArrowY
 - GgApp::Window, 578
- getSize
 - Buffer, 102
 - Camera, 134
 - Capture, 155
 - Framebuffer, 184
 - gg::GgTexture, 412
 - GgApp::Window, 579
 - Texture, 554
- getStart
 - gg::GgTrackball, 423
- getStride
 - gg::GgBuffer< T >, 198
 - gg::GgUniformBuffer< T >, 439
- getString
 - parseconfig.h, 858, 859
- getTarget
 - gg::GgBuffer< T >, 199
 - gg::GgUniformBuffer< T >, 440
- getTexture
 - gg::GgTexture, 413
- getTextureName
 - Texture, 554
- getTitle
 - Config, 169
- getTotalCount
 - Calibration, 115
- getTranslation
 - GgApp::Window, 580
- getTranslationMatrix
 - GgApp::Window, 580
- getUserPointer
 - GgApp::Window, 580
- getValue
 - parseconfig.h, 859, 860
- getWheel
 - GgApp::Window, 581
- getWheelX
 - GgApp::Window, 582
- getWheelY
 - GgApp::Window, 582
- getWidth
 - Buffer, 103
 - Camera, 135
 - Config, 169
 - gg::GgTexture, 413
 - GgApp::Window, 582
- gg, 17
 - .ggError, 20
 - .ggFBOError, 20
 - BindingPoints, 20
 - ggArraysObj, 21
 - ggBufferAlignment, 88
 - ggConjugate, 21
 - ggCreateComputeShader, 22
 - ggCreateNormalMap, 22
 - ggCreateShader, 23
 - ggCross, 24, 26
 - ggDistance3, 26, 27
 - ggDistance4, 28, 29
 - ggDot3, 30
 - ggDot4, 31
 - ggElementsMesh, 32
 - ggElementsObj, 33
 - ggElementsSphere, 34
 - ggEllipse, 34
 - ggEulerQuaternion, 35, 37

- ggFrustum, 38
- ggIdentity, 38
- ggIdentityQuaternion, 39
- ggInit, 39
- ggInvert, 40, 41
- ggLength3, 41, 42
- ggLength4, 43
- ggLoadComputeShader, 44
- ggLoadHeight, 45
- ggLoadImage, 46
- ggLoadShader, 46, 47
- ggLoadSimpleObj, 48, 49
- ggLoadTexture, 50
- ggLookat, 51, 52
- ggMatrixQuaternion, 53, 54
- ggNorm, 54
- ggNormal, 55
- ggNormalize, 55
- ggNormalize3, 56–58
- ggNormalize4, 58–60
- ggOrthogonal, 61
- ggPerspective, 61
- ggPointsCube, 62
- ggPointsSphere, 63
- ggQuaternionMatrix, 63
- ggQuaternionTransposeMatrix, 64
- ggReadImage, 64
- ggRectangle, 66
- ggRotate, 66–69
- ggRotateQuaternion, 69, 70
- ggRotateX, 71
- ggRotateY, 72
- ggRotateZ, 72
- ggSaveColor, 74
- ggSaveDepth, 74
- ggSaveTga, 76
- ggScale, 77, 78
- ggSlerp, 79, 81
- ggTranslate, 82, 84
- ggTranspose, 85
- LightBindingPoint, 20
- MaterialBindingPoint, 20
- operator+, 86, 87
- operator-, 87
- operator/, 88
- operator*, 86
- gg.cpp, 666
- gg.h, 743
 - ggError, 747
 - ggFBOError, 747
 - pathChar, 747
 - pathString, 747
 - TCharToUtf8, 748
 - Utf8ToTChar, 748
- gg::GgBuffer< T >, 194
 - ~GgBuffer, 196
 - bind, 196
 - copy, 196
 - getBuffer, 197
 - getCount, 197
 - getStride, 198
 - getTarget, 199
 - GgBuffer, 195, 196
 - map, 199, 200
 - operator=, 200, 201
 - read, 201
 - send, 202
 - unbind, 203
 - unmap, 203
- gg::GgColorTexture, 203
 - ~GgColorTexture, 205
 - GgColorTexture, 204, 205
 - load, 206
- gg::GgElements, 208
 - ~GgElements, 211
 - draw, 211
 - getIndexBuffer, 212
 - getIndexCount, 212
 - GgElements, 210
 - load, 212
 - send, 213
- gg::GgMatrix, 215
 - ~GgMatrix, 221
 - frustum, 221
 - get, 222, 223
 - GgMatrix, 218–220
 - invert, 224
 - loadFrustum, 224
 - loadIdentity, 225
 - loadInvert, 226
 - loadLookat, 227, 228
 - loadNormal, 229, 230
 - loadOrthogonal, 230
 - loadPerspective, 231
 - loadRotate, 232, 233, 235, 236
 - loadRotateX, 237
 - loadRotateY, 237
 - loadRotateZ, 238
 - loadScale, 239, 240
 - loadTranslate, 241, 242
 - loadTranspose, 243
 - lookat, 244, 245
 - normal, 247
 - operator+, 252, 253
 - operator+=, 254
 - operator-, 255, 256
 - operator-=, 256, 257
 - operator/, 258
 - operator/=, 259, 260
 - operator=, 261, 262
 - operator*, 248, 250
 - operator*=, 251
 - orthogonal, 262
 - perspective, 263
 - projection, 263, 264
 - rotate, 265–267

- rotateX, 268
- rotateY, 269
- rotateZ, 269
- scale, 270, 272
- translate, 273, 274
- transpose, 275
- gg::GgNormalTexture, 276
 - ~GgNormalTexture, 278
 - GgNormalTexture, 276, 277
 - load, 278, 279
- gg::GgPoints, 280
 - ~GgPoints, 283
 - draw, 283
 - getBuffer, 284
 - getCount, 284
 - GgPoints, 281
 - load, 284
 - operator bool, 285
 - operator!, 285
 - send, 285
- gg::GgPointShader, 286
 - ~GgPointShader, 289
 - get, 290
 - GgPointShader, 287–289
 - load, 290, 291
 - loadMatrix, 292
 - loadModelviewMatrix, 294, 295
 - loadProjectionMatrix, 295, 296
 - unuse, 297
 - use, 297–300
- gg::GgQuaternion, 301
 - ~GgQuaternion, 308
 - add, 308–310
 - conjugate, 311
 - divide, 311–313
 - euler, 314–316
 - get, 317
 - getConjugateMatrix, 317, 318
 - getMatrix, 319, 321
 - GgQuaternion, 304, 306–308
 - invert, 321
 - loadAdd, 322–324
 - loadConjugate, 325
 - loadDivide, 326–328
 - loadEuler, 329
 - loadIdentity, 330
 - loadInvert, 331
 - loadMatrix, 332, 333
 - loadMultiply, 334, 335
 - loadNormalize, 336, 338
 - loadRotate, 338, 339
 - loadRotateX, 340
 - loadRotateY, 341
 - loadRotateZ, 342
 - loadSlerp, 343–345
 - loadSubtract, 346, 347
 - multiply, 349, 350, 352
 - norm, 352
 - normalize, 353
 - operator+, 357
 - operator+=, 358, 359
 - operator-, 359, 360
 - operator-=, 360, 361
 - operator/, 362
 - operator/=: 363, 364
 - operator=, 364–366
 - operator*, 354, 355
 - operator*=, 355, 356
 - rotate, 366, 367
 - rotateX, 368
 - rotateY, 369
 - rotateZ, 369
 - slerp, 370
 - subtract, 371, 372
- gg::GgShader, 374
 - ~GgShader, 376
 - get, 377
 - GgShader, 374–376
 - operator=, 377
 - unuse, 378
 - use, 378
- gg::GgShape, 379
 - ~GgShape, 380
 - draw, 380
 - get, 381
 - getMode, 381
 - GgShape, 379
 - operator bool, 382
 - operator!, 382
 - setMode, 382
- gg::GgSimpleObj, 383
 - ~GgSimpleObj, 384
 - draw, 384
 - get, 384
 - GgSimpleObj, 383
 - operator bool, 385
 - operator!, 385
- gg::GgSimpleShader, 386
 - ~GgSimpleShader, 390
 - GgSimpleShader, 388, 389
 - load, 390, 391
 - loadMatrix, 392–394
 - loadModelviewMatrix, 395, 396
 - operator=, 397
 - use, 399–403, 405–408
- gg::GgSimpleShader::Light, 485
 - ambient, 486
 - diffuse, 486
 - position, 486
 - specular, 486
- gg::GgSimpleShader::LightBuffer, 487
 - ~LightBuffer, 490
 - LightBuffer, 488, 489
 - load, 490, 491
 - loadAmbient, 492, 493
 - loadColor, 494

- loadDiffuse, 495, 496
- loadPosition, 497–499
- loadSpecular, 501, 502
- select, 503
- gg::GgSimpleShader::Material, 504
 - ambient, 505
 - diffuse, 505
 - shininess, 505
 - specular, 505
- gg::GgSimpleShader::MaterialBuffer, 506
 - ~MaterialBuffer, 509
 - load, 510
 - loadAmbient, 511, 512
 - loadAmbientAndDiffuse, 513–515
 - loadDiffuse, 516, 517
 - loadShininess, 518, 519
 - loadSpecular, 520, 521
 - MaterialBuffer, 508, 509
 - select, 522
- gg::GgTexture, 409
 - ~GgTexture, 411
 - bind, 411
 - getHeight, 411
 - getSize, 412
 - getTexture, 413
 - getWidth, 413
 - GgTexture, 409–411
 - operator=, 413, 414
 - swapRandB, 414
 - unbind, 415
- gg::GgTrackball, 415
 - ~GgTrackball, 420
 - begin, 420
 - end, 420
 - get, 421
 - getMatrix, 421
 - getQuaternion, 421
 - getScale, 422
 - getStart, 423
 - GgTrackball, 419
 - motion, 423
 - operator=, 424
 - region, 424, 425
 - reset, 426
 - rotate, 426
- gg::GgTriangles, 427
 - ~GgTriangles, 430
 - draw, 430
 - getBuffer, 430
 - getCount, 431
 - GgTriangles, 428, 429
 - load, 431
 - send, 432
- gg::GgUniformBuffer < T >, 433
 - ~GgUniformBuffer, 435
 - bind, 435
 - copy, 436
 - fill, 437
- getBuffer, 438
- getCount, 439
- getStride, 439
- getTarget, 440
- GgUniformBuffer, 434, 435
- load, 441
- map, 442
- read, 443
- send, 444
- unbind, 445
- unmap, 445
- gg::GgVector, 446
 - ~GgVector, 451
 - distance3, 451
 - distance4, 451
 - dot3, 452
 - dot4, 453
 - GgVector, 447–450
 - length3, 453
 - length4, 454
 - normalize3, 454
 - normalize4, 455
 - operator+, 458, 460
 - operator+=", 460, 462
 - operator-, 462, 464
 - operator-=, 464, 466
 - operator/, 466, 468
 - operator/=, 468, 470
 - operator=, 470, 471
 - operator*, 455, 456
 - operator*=", 456, 458
- gg::GgVertex, 471
 - GgVertex, 473, 474
 - normal, 474
 - position, 474
- gg::GgVertexArray, 475
 - ~GgVertexArray, 478
 - bind, 478
 - get, 478
 - GgVertexArray, 475, 476
 - operator=, 478, 479
- GG.BUTTON_COUNT
 - GgApp.h, 818
- GG.INTERFACE_COUNT
 - GgApp.h, 818
- GG.USE_IMGUI
 - GgApp.h, 818
- GgApp, 189
 - ~GgApp, 191
 - GgApp, 189, 190
 - main, 191
 - operator=, 193
- GgApp.cpp, 803
- GgApp.h, 817
 - GG.BUTTON_COUNT, 818
 - GG.INTERFACE_COUNT, 818
 - GG.USE_IMGUI, 818
- GgApp::Window, 560

- ~Window, 564
- get, 565
- getAltArrowX, 565
- getAltArrowY, 565
- getAltArrow, 566
- getArrow, 566, 567
- getArrowX, 568
- getArrowY, 569
- getAspect, 569
- getControlArrow, 570
- getControlArrowX, 570
- getControlArrowY, 571
- getFboHeight, 572
- getFboSize, 572
- getFboWidth, 573
- getHeight, 573
- getKey, 574
- getLastKey, 574
- getMouse, 574, 576
- getMouseX, 576
- getMouseY, 576
- getRotation, 576
- getRotationMatrix, 577
- getScrollMatrix, 577
- getShiftArrow, 577
- getShiftArrowX, 578
- getShiftArrowY, 578
- getSize, 579
- getTranslation, 580
- getTranslationMatrix, 580
- getUserPointer, 580
- getWheel, 581
- getWheelX, 582
- getWheelY, 582
- getWidth, 582
- operator bool, 582
- operator=, 583
- reset, 584
- resetRotation, 584
- resetTranslation, 584
- restoreViewport, 585
- selectInterface, 585
- setClose, 585
- setKeyboardFunc, 587
- setMouseFunc, 587
- setResizeFunc, 587
- setUserPointer, 588
- setVelocity, 588
- setWheelFunc, 589
- shouldClose, 589
- swapBuffers, 589
- updateViewport, 590
- Window, 562–564
- ggArraysObj
 - gg, 21
- GgBuffer
 - gg::GgBuffer< T >, 195, 196
- ggBufferAlignment
 - gg, 88
- GgColorTexture
 - gg::GgColorTexture, 204, 205
- ggConjugate
 - gg, 21
- ggCreateComputeShader
 - gg, 22
- ggCreateNormalMap
 - gg, 22
- ggCreateShader
 - gg, 23
- ggCross
 - gg, 24, 26
- ggDistance3
 - gg, 26, 27
- ggDistance4
 - gg, 28, 29
- ggDot3
 - gg, 30
- ggDot4
 - gg, 31
- GgElements
 - gg::GgElements, 210
- ggElementsMesh
 - gg, 32
- ggElementsObj
 - gg, 33
- ggElementsSphere
 - gg, 34
- ggEllipse
 - gg, 34
- ggError
 - gg.h, 747
- ggEulerQuaternion
 - gg, 35, 37
- ggFBOError
 - gg.h, 747
- ggFrustum
 - gg, 38
- ggIdentity
 - gg, 38
- ggIdentityQuaternion
 - gg, 39
- ggInit
 - gg, 39
- ggInvert
 - gg, 40, 41
- ggLength3
 - gg, 41, 42
- ggLength4
 - gg, 43
- ggLoadComputeShader
 - gg, 44
- ggLoadHeight
 - gg, 45
- ggLoadImage
 - gg, 46
- ggLoadShader

- gg, [46](#), [47](#)
- ggLoadSimpleObj
 - gg, [48](#), [49](#)
- ggLoadTexture
 - gg, [50](#)
- ggLookat
 - gg, [51](#), [52](#)
- GgMatrix
 - gg::GgMatrix, [218–220](#)
- ggMatrixQuaternion
 - gg, [53](#), [54](#)
- ggNorm
 - gg, [54](#)
- ggNormal
 - gg, [55](#)
- ggNormalize
 - gg, [55](#)
- ggNormalize3
 - gg, [56–58](#)
- ggNormalize4
 - gg, [58–60](#)
- GgNormalTexture
 - gg::GgNormalTexture, [276](#), [277](#)
- ggOrthogonal
 - gg, [61](#)
- ggPerspective
 - gg, [61](#)
- GgPoints
 - gg::GgPoints, [281](#)
- ggPointsCube
 - gg, [62](#)
- GgPointShader
 - gg::GgPointShader, [287–289](#)
- ggPointsSphere
 - gg, [63](#)
- GgQuaternion
 - gg::GgQuaternion, [304](#), [306–308](#)
- ggQuaternionMatrix
 - gg, [63](#)
- ggQuaternionTransposeMatrix
 - gg, [64](#)
- ggReadImage
 - gg, [64](#)
- ggRectangle
 - gg, [66](#)
- ggRotate
 - gg, [66–69](#)
- ggRotateQuaternion
 - gg, [69](#), [70](#)
- ggRotateX
 - gg, [71](#)
- ggRotateY
 - gg, [72](#)
- ggRotateZ
 - gg, [72](#)
- ggSaveColor
 - gg, [74](#)
- ggSaveDepth
 - gg, [74](#)
- ggSaveTga
 - gg, [76](#)
- ggScale
 - gg, [77](#), [78](#)
- GgShader
 - gg::GgShader, [374–376](#)
- GgShape
 - gg::GgShape, [379](#)
- GgSimpleObj
 - gg::GgSimpleObj, [383](#)
- GgSimpleShader
 - gg::GgSimpleShader, [388](#), [389](#)
- ggSlerp
 - gg, [79](#), [81](#)
- GgTexture
 - gg::GgTexture, [409–411](#)
- GgTrackball
 - gg::GgTrackball, [419](#)
- ggTranslate
 - gg, [82](#), [84](#)
- ggTranspose
 - gg, [85](#)
- GgTriangles
 - gg::GgTriangles, [428](#), [429](#)
- GgUniformBuffer
 - gg::GgUniformBuffer< T >, [434](#), [435](#)
- GgVector
 - gg::GgVector, [447–450](#)
- GgVertex
 - gg::GgVertex, [473](#), [474](#)
- GgVertexArray
 - gg::GgVertexArray, [475](#), [476](#)
- HEADER_STR
 - main.cpp, [831](#)
- height
 - Camera, [138](#)
- image
 - Camera, [139](#)
- in
 - Camera, [139](#)
- increaseExposure
 - CamCv, [125](#)
 - Camera, [135](#)
- increaseGain
 - CamCv, [125](#)
 - Camera, [135](#)
- index
 - CaptureFormat, [161](#)
- initialize
 - Config, [169](#)
- initializeInputIntrinsics
 - Menu, [528](#)
- interval
 - Camera, [139](#)
- Intrinsics, [480](#)
 - center, [483](#)

- fov, 483
- fps, 484
- Intrinsics, 480, 481
- sensorSize, 484
- setCenter, 481
- setFov, 482
- setFps, 483
- setSize, 483
- size, 484
- Intrinsics.cpp, 827
- Intrinsics.h, 828
- invert
 - gg::GgMatrix, 224
 - gg::GgQuaternion, 321
- isImage
 - Capture, 156
- isOpened
 - CamImage, 144
 - Capture, 156
- isRunning
 - Camera, 135
- length3
 - gg::GgVector, 453
- length4
 - gg::GgVector, 454
- LightBindingPoint
 - gg, 20
- LightBuffer
 - gg::GgSimpleShader::LightBuffer, 488, 489
- load
 - CamImage, 144
 - Config, 170
 - gg::GgColorTexture, 206
 - gg::GgElements, 212
 - gg::GgNormalTexture, 278, 279
 - gg::GgPoints, 284
 - gg::GgPointShader, 290, 291
 - gg::GgSimpleShader, 390, 391
 - gg::GgSimpleShader::LightBuffer, 490, 491
 - gg::GgSimpleShader::MaterialBuffer, 510
 - gg::GgTriangles, 431
 - gg::GgUniformBuffer< T >, 441
- loadAdd
 - gg::GgQuaternion, 322–324
- loadAmbient
 - gg::GgSimpleShader::LightBuffer, 492, 493
 - gg::GgSimpleShader::MaterialBuffer, 511, 512
- loadAmbientAndDiffuse
 - gg::GgSimpleShader::MaterialBuffer, 513–515
- loadColor
 - gg::GgSimpleShader::LightBuffer, 494
- loadConjugate
 - gg::GgQuaternion, 325
- loadDiffuse
 - gg::GgSimpleShader::LightBuffer, 495, 496
 - gg::GgSimpleShader::MaterialBuffer, 516, 517
- loadDivide
 - gg::GgQuaternion, 326–328
- loadEuler
 - gg::GgQuaternion, 329
- loadFrustum
 - gg::GgMatrix, 224
- loadIdentity
 - gg::GgMatrix, 225
 - gg::GgQuaternion, 330
- loadInvert
 - gg::GgMatrix, 226
 - gg::GgQuaternion, 331
- loadLookat
 - gg::GgMatrix, 227, 228
- loadMatrix
 - gg::GgPointShader, 292
 - gg::GgQuaternion, 332, 333
 - gg::GgSimpleShader, 392–394
- loadModelviewMatrix
 - gg::GgPointShader, 294, 295
 - gg::GgSimpleShader, 395, 396
- loadMultiply
 - gg::GgQuaternion, 334, 335
- loadNormal
 - gg::GgMatrix, 229, 230
- loadNormalize
 - gg::GgQuaternion, 336, 338
- loadOrthogonal
 - gg::GgMatrix, 230
- loadParameters
 - Calibration, 115
- loadPerspective
 - gg::GgMatrix, 231
- loadPosition
 - gg::GgSimpleShader::LightBuffer, 497–499
- loadProjectionMatrix
 - gg::GgPointShader, 295, 296
- loadRotate
 - gg::GgMatrix, 232, 233, 235, 236
 - gg::GgQuaternion, 338, 339
- loadRotateX
 - gg::GgMatrix, 237
 - gg::GgQuaternion, 340
- loadRotateY
 - gg::GgMatrix, 237
 - gg::GgQuaternion, 341
- loadRotateZ
 - gg::GgMatrix, 238
 - gg::GgQuaternion, 342
- loadScale
 - gg::GgMatrix, 239, 240
- loadShininess
 - gg::GgSimpleShader::MaterialBuffer, 518, 519
- loadSlerp
 - gg::GgQuaternion, 343–345
- loadSpecular
 - gg::GgSimpleShader::LightBuffer, 501, 502
 - gg::GgSimpleShader::MaterialBuffer, 520, 521
- loadSubtract
 - gg::GgQuaternion, 346, 347

- loadTranslate
 - gg::GgMatrix, [241](#), [242](#)
- loadTranspose
 - gg::GgMatrix, [243](#)
- lookat
 - gg::GgMatrix, [244](#), [245](#)
- main
 - GgApp, [191](#)
 - main.cpp, [832](#)
- main.cpp, [830](#)
 - HEADER_STR, [831](#)
 - main, [832](#)
- map
 - Buffer, [103](#)
 - gg::GgBuffer< T >, [199](#), [200](#)
 - gg::GgUniformBuffer< T >, [442](#)
- markerLength
 - Settings, [540](#)
- MaterialBindingPoint
 - gg, [20](#)
- MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, [508](#), [509](#)
- Menu, [523](#)
 - ~Menu, [525](#)
 - detectBoard, [530](#)
 - detectMarker, [530](#)
 - draw, [526](#)
 - getCheckerLength, [526](#)
 - getCheckerSize, [526](#)
 - getMarkerLength, [526](#)
 - getMenuBarHeight, [527](#)
 - getPose, [527](#)
 - initializeInputIntrinsics, [528](#)
 - Menu, [524](#), [525](#)
 - operator bool, [528](#)
 - operator=, [528](#)
 - prioritizeLatency, [530](#)
 - saveImage, [529](#)
 - setup, [529](#)
- Menu.cpp, [833](#)
 - getAnyList, [834](#)
- Menu.h, [849](#)
- Mesh, [531](#)
 - ~Mesh, [532](#)
 - draw, [532](#)
 - drawMesh, [532](#)
 - Mesh, [531](#)
 - operator=, [533](#)
- mesh
 - Texture, [560](#)
- Mesh.h, [852](#)
- motion
 - gg::GgTrackball, [423](#)
- mtx
 - Camera, [139](#)
- multiply
 - gg::GgQuaternion, [349](#), [350](#), [352](#)
- norm
 - gg::GgQuaternion, [352](#)
- normal
 - gg::GgMatrix, [247](#)
 - gg::GgVertex, [474](#)
- normalize
 - gg::GgQuaternion, [353](#)
- normalize3
 - gg::GgVector, [454](#)
- normalize4
 - gg::GgVector, [455](#)
- open
 - CamCv, [126](#)
 - CamImage, [145](#)
 - CamMf, [151](#)
- opencv_link.h, [855](#)
- openDevice
 - Capture, [156](#)
- openImage
 - Capture, [157](#)
- openMovie
 - Capture, [157](#)
- operator bool
 - Capture, [158](#)
 - gg::GgPoints, [285](#)
 - gg::GgShape, [382](#)
 - gg::GgSimpleObj, [385](#)
 - GgApp::Window, [582](#)
 - Menu, [528](#)
- operator!
 - gg::GgPoints, [285](#)
 - gg::GgShape, [382](#)
 - gg::GgSimpleObj, [385](#)
- operator+
 - gg, [86](#), [87](#)
 - gg::GgMatrix, [252](#), [253](#)
 - gg::GgQuaternion, [357](#)
 - gg::GgVector, [458](#), [460](#)
- operator+=
 - gg::GgMatrix, [254](#)
 - gg::GgQuaternion, [358](#), [359](#)
 - gg::GgVector, [460](#), [462](#)
- operator-
 - gg, [87](#)
 - gg::GgMatrix, [255](#), [256](#)
 - gg::GgQuaternion, [359](#), [360](#)
 - gg::GgVector, [462](#), [464](#)
- operator-=
 - gg::GgMatrix, [256](#), [257](#)
 - gg::GgQuaternion, [360](#), [361](#)
 - gg::GgVector, [464](#), [466](#)
- operator/
 - gg, [88](#)
 - gg::GgMatrix, [258](#)
 - gg::GgQuaternion, [362](#)
 - gg::GgVector, [466](#), [468](#)
- operator/=
 - gg::GgMatrix, [259](#), [260](#)

- gg::GgQuaternion, 363, 364
- gg::GgVector, 468, 470
- operator=
 - Buffer, 104
 - Calibration, 116
 - Camera, 135
 - Expand, 173
 - Framebuffer, 184
 - gg::GgBuffer< T >, 200, 201
 - gg::GgMatrix, 261, 262
 - gg::GgQuaternion, 364–366
 - gg::GgShader, 377
 - gg::GgSimpleShader, 397
 - gg::GgTexture, 413, 414
 - gg::GgTrackball, 424
 - gg::GgVector, 470, 471
 - gg::GgVertexArray, 478, 479
 - GgApp, 193
 - GgApp::Window, 583
 - Menu, 528
 - Mesh, 533
 - Texture, 555
- operator*
 - gg, 86
 - gg::GgMatrix, 248, 250
 - gg::GgQuaternion, 354, 355
 - gg::GgVector, 455, 456
- operator*=
 - gg::GgMatrix, 251
 - gg::GgQuaternion, 355, 356
 - gg::GgVector, 456, 458
- orthogonal
 - gg::GgMatrix, 262
- out
 - Camera, 139
- parseconfig.h, 856
 - getString, 858, 859
 - getValue, 859, 860
 - setString, 861, 862
 - setValue, 862
- pathChar
 - gg.h, 747
- pathString
 - gg.h, 747
- perspective
 - gg::GgMatrix, 263
- position
 - gg::GgSimpleShader::Light, 486
 - gg::GgVertex, 474
- Preference, 533
 - ~Preference, 535
 - buildShader, 536
 - getDescription, 536
 - getIntrinsics, 536
 - getShader, 536
 - Preference, 534, 535
 - setPreference, 536
- Preference.cpp, 866
- Preference.h, 867
- prioritizeLatency
 - Camera, 139
 - Menu, 530
- projection
 - gg::GgMatrix, 263, 264
- read
 - gg::GgBuffer< T >, 201
 - gg::GgUniformBuffer< T >, 443
- readPixels
 - Texture, 557, 558
- recordCorners
 - Calibration, 116
- region
 - gg::GgTrackball, 424, 425
- REQUESTS.md, 870
- reset
 - gg::GgTrackball, 426
 - GgApp::Window, 584
- resetRotation
 - GgApp::Window, 584
- resetTranslation
 - GgApp::Window, 584
- resize
 - Buffer, 105, 106
- resolution
 - CaptureFormat, 161
- restoreViewport
 - GgApp::Window, 585
- retrieve
 - Capture, 158, 159
- reusableFrame
 - Camera, 140
- rotate
 - gg::GgMatrix, 265–267
 - gg::GgQuaternion, 366, 367
 - gg::GgTrackball, 426
- rotateX
 - gg::GgMatrix, 268
 - gg::GgQuaternion, 368
- rotateY
 - gg::GgMatrix, 269
 - gg::GgQuaternion, 369
- rotateZ
 - gg::GgMatrix, 269
 - gg::GgQuaternion, 369
- running
 - Camera, 140
- RvecTvecToPose
 - Calibration, 116
- SafeRelease
 - CamMf.cpp, 623
- samples
 - Settings, 540
- save
 - CamImage, 146
 - Config, 170

- savelmage
 - Menu, [529](#)
- saveParameters
 - Calibration, [117](#)
- scale
 - gg::GgMatrix, [270](#), [272](#)
- select
 - CamMf, [152](#)
 - gg::GgSimpleShader::LightBuffer, [503](#)
 - gg::GgSimpleShader::MaterialBuffer, [522](#)
- selectInterface
 - GgApp::Window, [585](#)
- send
 - gg::GgBuffer< T >, [202](#)
 - gg::GgElements, [213](#)
 - gg::GgPoints, [285](#)
 - gg::GgTriangles, [432](#)
 - gg::GgUniformBuffer< T >, [444](#)
- sensorSize
 - Intrinsics, [484](#)
- setCenter
 - Intrinsics, [481](#)
- setClose
 - GgApp::Window, [585](#)
- setDictionary
 - Calibration, [118](#)
- setExposure
 - CamCv, [127](#)
- setFov
 - Intrinsics, [482](#)
- setFps
 - Intrinsics, [483](#)
- setGain
 - CamCv, [127](#)
- setKeyboardFunc
 - GgApp::Window, [587](#)
- setMode
 - gg::GgShape, [382](#)
- setMouseFunc
 - GgApp::Window, [587](#)
- setPosition
 - CamCv, [128](#)
- setPreference
 - Preference, [536](#)
- setPrioritizeLatency
 - Camera, [136](#)
 - Capture, [159](#)
- setResizeFunc
 - GgApp::Window, [587](#)
- setSettings
 - Config, [171](#)
- setSize
 - Intrinsics, [483](#)
- setString
 - parseconfig.h, [861](#), [862](#)
- Settings, [537](#)
 - checkerLength, [538](#)
 - checkerSize, [538](#)
 - defaultCheckerLength, [538](#)
 - defaultCheckerSize, [539](#)
 - defaultEuler, [539](#)
 - defaultFocal, [539](#)
 - defaultFocalRange, [539](#)
 - defaultMarkerLength, [539](#)
 - dictionaryName, [539](#)
 - euler, [540](#)
 - focal, [540](#)
 - focalRange, [540](#)
 - getFocal, [538](#)
 - markerLength, [540](#)
 - samples, [540](#)
- setup
 - Expand, [174](#)
 - Menu, [529](#)
- setUserPointer
 - GgApp::Window, [588](#)
- setValue
 - parseconfig.h, [862](#)
- setVelocity
 - GgApp::Window, [588](#)
- setWheelFunc
 - GgApp::Window, [589](#)
- shininess
 - gg::GgSimpleShader::Material, [505](#)
- shouldClose
 - GgApp::Window, [589](#)
- show
 - Framebuffer, [186](#)
- size
 - Intrinsics, [484](#)
- slerp
 - gg::GgQuaternion, [370](#)
- specular
 - gg::GgSimpleShader::Light, [486](#)
 - gg::GgSimpleShader::Material, [505](#)
- start
 - Camera, [136](#)
 - Capture, [159](#)
- stop
 - Camera, [136](#)
 - CamMf, [152](#)
 - Capture, [160](#)
- subtract
 - gg::GgQuaternion, [371](#), [372](#)
- SubTypeToName
 - CamMf.cpp, [624](#)
- swapBuffers
 - GgApp::Window, [589](#)
- swapRandB
 - gg::GgTexture, [414](#)
- TCharToUtf8
 - gg.h, [748](#)
- Texture, [541](#)
 - ~Texture, [545](#)
 - bindTexture, [546](#)
 - copy, [547](#)

- create, [548](#)
- discard, [549](#)
- draw, [550](#)
- drawPixels, [551–553](#)
- getChannels, [554](#)
- getSize, [554](#)
- getTextureName, [554](#)
- mesh, [560](#)
- operator=, [555](#)
- readPixels, [557](#), [558](#)
- Texture, [543](#), [544](#)
- unbindTexture, [559](#)
- Texture.cpp, [870](#)
- Texture.h, [874](#)
- thr
 - Camera, [140](#)
- total
 - Camera, [140](#)
- translate
 - gg::GgMatrix, [273](#), [274](#)
- transmit
 - Camera, [137](#), [138](#)
- transpose
 - gg::GgMatrix, [275](#)
- unbind
 - gg::GgBuffer< T >, [203](#)
 - gg::GgTexture, [415](#)
 - gg::GgUniformBuffer< T >, [445](#)
- unbindBuffer
 - Buffer, [107](#)
- unbindFramebuffer
 - Framebuffer, [186](#)
- unbindTexture
 - Texture, [559](#)
- unmap
 - Buffer, [107](#)
 - gg::GgBuffer< T >, [203](#)
 - gg::GgUniformBuffer< T >, [445](#)
- unuse
 - gg::GgPointShader, [297](#)
 - gg::GgShader, [378](#)
- update
 - Framebuffer, [187](#), [188](#)
- updateViewport
 - GgApp::Window, [590](#)
- use
 - Compute, [163](#)
 - gg::GgPointShader, [297–300](#)
 - gg::GgShader, [378](#)
 - gg::GgSimpleShader, [399–403](#), [405–408](#)
- USE_PIXEL_BUFFER_OBJECT
 - Buffer.h, [595](#)
- USE_RODRIGUES
 - Calibration.cpp, [601](#)
- Utf8ToTChar
 - gg.h, [748](#)
- width
 - Camera, [140](#)
- Window
 - GgApp::Window, [562–564](#)
- ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版., [1](#)
- 作業指示および開発履歴, [3](#)